

Principia Softwarica: The Plan 9 C Compiler 5c
ARM (32 bits) edition
version 0.1

Yoann Padioleau
`yoann.padioleau@gmail.com`

with code from
Ken Thompson

April 28, 2026

The text and figures are Copyright © 2014–2026 Yoann Padioleau.
All rights reserved.

The source code is Copyright © 2021 Plan 9 Foundation.
MIT license.

Contents

1	Introduction	12
1.1	Motivations	12
1.2	The ARM Plan 9 C compiler: <code>5c</code>	13
1.3	Other compilers	13
1.4	Getting started	14
1.5	Requirements	14
1.6	About this document	15
1.7	Copyright	15
1.8	Acknowledgments	15
2	Overview	16
2.1	Compiler principles	16
2.1.1	Frontend	16
2.1.2	Scope resolution	17
2.1.3	Use/def checking	17
2.1.4	Type checking	17
2.1.5	Backend	17
2.1.6	Labelization and control-flow lowering	17
2.1.7	Linearization and three-address code	18
2.1.8	Instruction selection	18
2.1.9	Frameization and stack-frame layout	18
2.1.10	Optimizations	19
2.1.11	Intermediate representations and SSA	19
2.1.12	Register allocation	20
2.1.13	Peephole optimization	20
2.1.14	Undefined behavior	20
2.1.15	Cross-compilation	21
2.1.16	Bootstrapping	21
2.1.17	The most demanding reader of your code	21
2.2	<code>5c</code> command-line interface	21
2.3	Compiling <code>helloworld.c</code>	22
2.4	Input C language	23
2.5	Output object format	23
2.6	The ARM architecture	23
2.7	Plan 9 design choices	23
2.8	Code organization	24
2.9	Software architecture	24
2.10	Book structure	26

3	Core Data Structures	27
3.1	Abstract syntax tree: Node	27
3.2	Type system	30
3.3	Storage_class	33
3.4	Node and type attributes	33
3.5	Symbols and hash table	34
3.5.1	Sym	34
3.5.2	hash	35
3.5.3	slookup() and lookup()	35
3.5.4	Identifier symbols	36
3.5.5	Tag symbols	37
3.5.6	Other symbols	37
3.6	Declarations, namespaces, and scope: Decl	38
3.7	Token	42
3.8	Assembly Instruction output	42
4	main()	45
4.1	Arguments processing	46
4.2	compile()	47
4.3	gclean()	49
4.4	Initializations	49
4.4.1	tinit()	50
4.4.2	cinit()	50
4.4.3	ginit()	51
4.4.4	arginit()	51
5	Input	52
5.1	Files management: iostack	52
5.2	Buffer management: fi	54
5.3	GETC()	54
6	Lexing	56
6.1	Hand-written vs generated lexers	56
6.2	yylex()	56
6.3	Peek, seek, and look ahead	57
6.4	Comments	58
6.5	Keywords and identifiers	59
6.5.1	Unicode identifiers	61
6.5.2	Typedef trick	61
6.6	Operators	62
6.7	Numbers	64
6.7.1	Decimals	65
6.7.2	convvtox()	65
6.7.3	Hexadecimals and octals	67
6.7.4	Unsigned numbers	68
6.7.5	Long numbers	68
6.7.6	Floats	69
6.8	Characters	70
6.8.1	Escaping characters	71
6.8.2	Unicode characters	73

6.9	Strings	73
6.9.1	Unicode strings	74
7	Preprocessing	76
7.1	mactab and domacro() dispatch	76
7.2	#include	79
7.2.1	Include paths, -I	79
7.2.2	Tracing origin and Hist	80
7.2.3	#include	81
7.3	#define	84
7.3.1	-D	84
7.3.2	#define	85
7.3.3	Macro Expansion	90
7.4	#undef	94
7.5	#ifdef	94
7.6	#pragma	96
7.7	#line	96
8	Parsing	99
8.1	Generated parser vs hand-written recursive descent	99
8.2	Single-pass compilation	100
8.3	Overview	100
8.4	Declarations and definitions, part one	101
8.4.1	No-declarator declarations	102
8.4.2	Declarator-based declarations	102
8.4.3	Entity declarations, the declarator	103
8.4.4	Function definitions	105
8.5	Statements	105
8.5.1	Blocks	106
8.5.2	Conditionals	107
8.5.3	Loops	107
8.5.4	Control flow jumps	108
8.5.5	Labels and goto	108
8.5.6	Switch	109
8.6	Expressions	110
8.6.1	Numeric constants	111
8.6.2	String constants	112
8.6.3	Entity uses	114
8.6.4	Arithmetic expressions	114
8.6.5	Boolean expressions	115
8.6.6	Assignments	116
8.6.7	Pointers	116
8.6.8	Array accesses	116
8.6.9	Field accesses	117
8.6.10	Function calls	117
8.6.11	Cast	118
8.6.12	Ternary expressions	118
8.6.13	Prefix/postfix	118
8.6.14	sizeof()	119
8.7	Initializers and designators	119

8.8	Types and storage classes	120
8.8.1	Basic types	123
8.8.2	Storage classes	126
8.8.3	Qualifiers	127
8.8.4	Structures and unions	128
8.8.5	Enums	132
8.8.6	Pointer and array types	134
8.8.7	Function types, parameter types	135
8.8.8	Typedefs	137
8.9	Declarations and definitions, part two	137
8.9.1	Globals, external declarator	137
8.9.2	Function parameters	137
8.9.3	Locals, automatic declarator	139
8.9.4	Types, abstract declarator	141
9	Checking	142
9.1	Overview	142
9.2	Printing warnings: <code>5c -w</code> and <code>5c -W</code>	142
9.3	Symbol resolution	143
9.4	Storage resolution	143
9.4.1	<code>xdecl</code>	143
9.4.2	<code>doinit</code>	144
9.4.3	<code>pdecl</code>	144
9.4.4	<code>adecl</code>	144
9.5	Typechecking helpers	145
9.5.1	Overview	145
9.5.2	Type equality: <code>sametype()</code>	145
9.5.3	generic <code>void*</code> pointer conversions: <code>5c -V</code>	147
9.5.4	Type merge: <code>tmerge()</code>	147
9.5.5	Set of types constants: <code>typexxx[]</code>	148
9.5.6	Type compatibility: <code>tcompat()</code>	150
9.5.7	Compatibility policies: <code>txxx</code>	151
9.5.8	Typechecker: <code>tcom()</code>	152
9.5.9	Array to pointer conversions	154
9.6	Typechecking expressions	155
9.6.1	Numeric constants	155
9.6.2	String constants	155
9.6.3	Entity uses	155
9.6.4	Arithmetic expressions	156
9.6.5	Arithmetic conversions	160
9.6.6	Pointer arithmetic	162
9.6.7	Boolean expressions	163
9.6.8	Assignments	166
9.6.9	Pointers	170
9.6.10	Array accesses	170
9.6.11	Field accesses	170
9.6.12	Function calls	172
9.6.13	Cast	175
9.6.14	Ternary expressions	176
9.6.15	Prefix/postfix	177

9.6.16	<code>sizeof()</code>	178
9.7	Typechecking statements	178
9.7.1	<code>return</code>	179
9.7.2	<code>switch</code>	179
9.7.3	<code>if</code>	180
9.8	Typechecking declarations and constant evaluation	180
9.8.1	Enumerations	180
9.8.2	Arrays	181
9.8.3	Functions	181
9.8.4	Typedef expansions	182
9.9	Lint checking	183
9.9.1	Unused variables	183
9.9.2	Used before set	184
9.9.3	False positives silencing: <code>SET()/USED()</code>	184
9.9.4	Redeclared or unused labels	185
9.9.5	Unreachable code	185
9.9.6	Missing return	188
9.9.7	Unused expression	189
9.9.8	Constant if: <code>5c -c</code>	189
9.9.9	Out of range shifting	189
9.9.10	Useless comparisons	190
9.10	Const checking	190
10	Assembly Generation	192
10.1	No intermediate representation	192
10.2	Overview	192
10.3	<code>pc</code> , <code>p</code> , and <code>nextpc()</code>	193
10.4	Functions: <code>codgen()</code>	194
10.4.1	<code>gpseudo()</code>	195
10.4.2	<code>gbranch()</code>	195
10.4.3	<code>patch()</code>	196
10.4.4	Offset	196
10.5	Statements: <code>gen()</code>	197
10.5.1	Blocks, sequences	197
10.5.2	Conditionals	198
10.5.3	Labels and <code>goto</code>	199
10.5.4	Loops	201
10.5.5	Control flow jumps	206
10.6	Register allocation	207
10.6.1	<code>reg</code>	207
10.6.2	<code>OREGISTER</code>	208
10.6.3	<code>nodreg()</code>	208
10.6.4	<code>regfree()</code>	209
10.7	Typechecking, expliciting, annotating, and rewriting: <code>complex()</code>	209
10.8	Addressability and complexity: <code>xcom()</code>	210
10.8.1	Addressable: <code>INDEXED</code>	211
10.8.2	Addressability: <code>addable</code>	211
10.8.3	Complexity: <code>complex</code>	213
10.8.4	<code>FNX</code>	213
10.9	Memory moves	214

10.9.1	<code>gmove()</code>	214
10.9.2	Conversions	217
10.9.3	<code>gins()</code>	219
10.9.4	<code>naddr()</code>	219
10.9.5	<code>regalloc()</code>	221
10.10	Expressions: <code>cgen()</code>	223
10.10.1	Numeric constants	224
10.10.2	Entity uses	224
10.10.3	String constants	226
10.10.4	Arithmetic expressions	228
10.10.5	Boolean expressions	234
10.10.6	Assignments part 1	234
10.10.7	Pointers	235
10.10.8	Assignments part 2	238
10.10.9	Assignments and arithmetic	239
10.10.10	Array accesses	241
10.10.11	Field accesses	241
10.10.12	Cast	248
10.10.13	Ternary expressions	250
10.10.14	Prefix/postfix	250
10.10.15	<code>sizeof()</code>	252
10.11	Boolean expressions	252
10.12	Function calls	257
10.12.1	<code>gargs()</code>	258
10.12.2	Caller-save vs Callee-save	259
10.12.3	Indirect function calls	260
10.12.4	Complex calls	261
10.13	Initializers	262
10.14	Toplevel globals	270
10.14.1	<code>gextern()</code>	270
10.15	Complex values	271
10.15.1	Complex types	271
10.15.2	Complex expression	271
10.15.3	Complex argument	272
10.15.4	Complex return value	272
10.15.5	<code>vlong</code> constant	273
10.16	Other topics	273
10.16.1	Sizes	273
10.16.2	First argument	274
10.16.3	Alignment	275
10.17	Advanced assembly generation	278
10.17.1	Switch	278

11 Object File Generation 286

11.1	Instruction-stream object files	286
11.2	Object format	286
11.3	Instruction output: <code>outcode()</code>	287
11.4	Operand output: <code>zaddr()</code>	288
11.5	Symbol table, <code>zname()</code> , <code>h</code> and <code>ANAME</code>	290
11.6	File and line information: <code>outhist()</code>	290

12 AST-level Optimizations	292
12.1 AST simplifications	292
12.1.1 General rewrites: <code>ccom()</code>	292
12.1.2 Constant evaluation: <code>evconst()</code>	303
12.2 Comma hoisting: <code>comma()</code>	308
12.3 Bitshifting opportunities	310
12.4 And opportunities	312
12.5 Immediate operands	313
12.5.1 Subtraction	313
12.5.2 Addition, or, etc.	313
12.5.3 Multiplication	314
12.6 Associative-commutative arithmetic optimisations	327
12.6.1 <code>Term</code>	327
12.6.2 <code>acom()</code>	328
13 Assembly-level Optimisations	336
13.1 Nop detection	336
13.2 ARM special instructions	336
13.3 Register passing argument: <code>REGARG</code>	337
13.4 Register allocation optimisations	338
13.4.1 <code>regopt()</code>	339
13.4.2 <code>noretval()</code>	341
13.4.3 Pass 1	341
13.4.4 Pass 2	344
13.4.5 Pass 3	345
13.4.6 Pass 4	345
13.4.7 pass 5	346
13.4.8 pass 6	347
13.4.9 pass 7, peep hole optimizations	348
13.4.10 pass 8	348
13.4.11 pass 9	349
13.4.12 <code>Bits</code>	349
13.4.13 <code>Var</code>	351
13.4.14 <code>Reg</code>	352
13.4.15 <code>XXX</code>	352
13.5 Peephole optimizer: <code>peep()</code>	366
13.6 Dominators	391
14 Linking Support	393
14.1 <code>#pragma lib</code> and automagic linking	393
14.2 Safe linking with type signatures: <code>5c -T</code>	394
14.2.1 <code>sign()</code>	395
14.2.2 <code>#pragma incomplete</code>	398
15 Debugging Support	400
15.1 General debugging metadata	400
15.2 Acid debugger metadata: <code>5c -a</code>	400
15.2.1 Entities	401
15.2.2 Structure/union definitions	403
15.3 Pickle: <code>5c -Z</code>	407

16 Profiling Support	412
16.1 Text attributes	412
16.2 <code>#pragma profile</code>	412
17 Advanced Topics	414
17.1 Advanced C features	414
17.1.1 Function and function pointer automatic conversions	414
17.1.2 Array and pointer automatic conversions	414
17.1.3 Local <code>static</code>	415
17.1.4 Local <code>volatile</code>	416
17.1.5 Bitfields	417
17.1.6 Old style prototypes	422
17.1.7 <code>_Noreturn()</code>	426
17.2 C extensions	426
17.2.1 Typing extensions: <code>typeext()</code>	426
17.2.2 Unnamed structure elements	427
17.2.3 Struct constructors	429
17.2.4 Per-processor storage: <code>extern register</code>	432
17.2.5 Type reflection: <code>typestr()</code>	434
17.2.6 Signature reflection: <code>signof()</code>	441
17.2.7 Format's arguments checking	443
17.3 Floats	450
17.4 64 bits operations	456
17.4.1 <code>com64init()</code>	457
17.4.2 Rewriting: <code>com64()</code>	459
17.4.3 <code>bool64()</code>	466
17.5 Endianness	466
17.6 Using an external preprocessor: <code>/bin/cpp</code>	466
17.7 Processing multiple files	468
17.8 Other optimizations	469
17.8.1 Packing: <code>#pragma pack</code>	469
17.8.2 Minimizing <code>#line</code> history	470
18 Conclusion	471
18.1 Patterns and techniques	471
18.2 Connections to other books	471
18.3 Beyond the Plan 9 compiler	472
A Debugging	474
A.1 Dumpers	474
A.1.1 <code>cc/ Fmt</code> data structures	474
A.1.2 <code>5c/ Fmt</code> data structures	479
A.1.3 <code>xxxnames[]</code>	485
A.1.4 AST dumper: <code>prtree()</code> and <code>5c -x</code>	487
A.2 <code>5c -v</code> , verbose mode	489
A.3 Preprocessing debugging	489
A.3.1 Macro debugging: <code>5c -m</code>	489
A.3.2 File inclusion debugging: <code>5c -e</code>	489
A.3.3 Line information debugging: <code>5c -f:</code>	489
A.4 Parsing debugging	489

A.4.1	Printing names: <code>5c -L</code>	489
A.4.2	Printing declarations: <code>5c -d</code>	490
A.5	Typing debugging	490
A.5.1	Printing expression type trees: <code>5c -t</code>	490
A.6	Code generation debugging	491
A.6.1	Printing processed opcodes: <code>5c -G</code>	491
A.6.2	Printing code generation information: <code>5c -g</code>	491
A.6.3	Printing generated assembly: <code>5c -S</code>	492
A.7	Optimization debugging	494
A.7.1	Printing arithmetic trees: <code>5c -m</code>	494
A.7.2	Printing registerization: <code>5c -r</code>	494
A.8	Printing initializations: <code>5c -i</code>	494
B	Error Management	495
B.0.1	<code>errorexit()</code>	495
B.0.2	<code>yyerror()</code>	495
B.0.3	<code>diag()</code>	496
B.0.4	<code>fatal()</code>	496
B.0.5	Error location	497
C	Libc	499
C.1	Memory management	499
C.2	String management	501
C.3	Bit operations	501
C.4	Mathematic functions	502
C.5	Key/value data structure	502
C.6	Portability	503
D	Extra Code	504
D.1	<code>cc/</code>	504
D.1.1	<code>cc/cc.h</code>	504
D.1.2	<code>cc/globals.c</code>	511
D.1.3	<code>cc/omachcap.c</code>	513
D.1.4	<code>cc/compat.c</code>	513
D.1.5	<code>cc/utils.c</code>	514
D.1.6	<code>cc/mac.c</code>	514
D.1.7	<code>cc/sub.c</code>	515
D.1.8	<code>cc/bits.c</code>	521
D.1.9	<code>cc/lex.c</code>	521
D.1.10	<code>cc/acid.c</code>	523
D.1.11	<code>cc/pickle.c</code>	524
D.1.12	<code>cc/com64.c</code>	524
D.1.13	<code>cc/dcl.c</code>	525
D.1.14	<code>cc/funct.c</code>	527
D.1.15	<code>cc/scon.c</code>	528
D.1.16	<code>cc/dpchk.c</code>	528
D.1.17	<code>cc/com.c</code>	529
D.2	<code>cc2/</code>	530
D.2.1	<code>cc2/pswt.c</code>	530
D.2.2	<code>cc2/pgen.c</code>	530

D.3	5c/	531
D.3.1	5c/enam.c	531
D.3.2	5c/gc.h	531
D.3.3	5c/machcap.c	535
D.3.4	5c/globals2.c	535
D.3.5	5c/list.c	537
D.3.6	5c/txt.c	537
D.3.7	5c/peep.c	538
D.3.8	5c/sgen.c	540
D.3.9	5c/swt.c	540
D.3.10	5c/mul.c	540
D.3.11	5c/reg.c	541
D.3.12	5c/cgen.c	542
Glossary		543
Indexes		544
References		567

Chapter 1

Introduction

The goal of this book is to explain with full details the source code of a C compiler.

1.1 Motivations

Why a C compiler? Because I think you are a better programmer if you fully understand how things work under the hood, and understanding how a compiler translates high-level C code in low-level assembly, and ultimately machine code, is essential.

A compiler is a critical piece of the development toolchain: every C program in this book series—the SHELL book [Pad18], the BUILDER book [Pad16a], the KERNEL book [Pad14]—was compiled by a C compiler before it could run. Understanding how that translation works removes a layer of mystery and makes you a more effective programmer, whether you are debugging a miscompilation, writing performance-sensitive code, or simply trying to understand what the machine actually executes.

Compilers are also where many important ideas in computer science come together: formal grammars and parsing, type systems, graph algorithms for register allocation, and low-level code generation. These techniques are not confined to compilers—they appear in linters, code formatters, IDEs, query engines, and static analysis tools. Understanding a real compiler gives you a foundation for all of these.

The industry certainly takes compilers seriously: Google, Apple, and others invest heavily in LLVM and GCC, and compiler improvements directly translate into faster software for millions of users. Even in the age of AI coding assistants, understanding compilation matters—an LLM can generate C code, but only a compiler can tell you whether it is well-typed, and only a compiler can turn it into instructions the processor can execute.

Here are a few questions I hope this book will answer:

- How does a compiler parse a complex programming language?
- How does a compiler manage the scope of variables, structures, enumerations, or typedefs?
- How does a typechecker work?
- How does a compiler translate conditionals, loops, and other control-flow constructs in basic assembly jumps?
- How does a compiler translate a switch?
- How does a compiler convert arbitrary complex expressions into simpler assembly instructions using a small set of registers?
- How does a function pass arguments to another function? Are registers used? Is the stack used? How does a function communicate its return value to the caller?
- What are simple useful optimizations?

1.2 The ARM Plan 9 C compiler: 5c

I will explain in this book the code of the Plan 9 ARM C compiler 5c, written by Ken Thompson, which contains about 15 000 lines of C (plus a Yacc grammar for parsing). 5c is written in C, with its parser using Yacc [Joh79]. Like for most books in Principia Softwarica, I chose a Plan 9 program because those programs are simple, small, elegant, open source, and they form together a coherent set.

5c is unusual among C compilers: it has no intermediate representation, no control-flow graph, and no SSA form. Instead, it works by a series of tree transformations on the AST, generating assembly instructions directly. This makes it harder to optimize aggressively, but also makes it remarkably easy to follow—each phase is a straightforward tree walk. A nice bonus is that 5c includes lint-like checks (e.g., “variable set but not used”) that catch common mistakes at compile time.

5c is also historically significant: its design was carried forward into the Go compiler, which was originally written in C and derived directly from the Plan 9 compiler sources before being rewritten in Go in 2015. So studying 5c is also, in a sense, studying the ancestor of the Go toolchain.

1.3 Other compilers

Here are a few compilers that I considered for this book, but which I ultimately discarded:

- Dennis Ritchie’s original C compiler `cc` (1973) was the first C compiler, written for the PDP-11. It was a two-pass compiler with a recursive descent parser. The code is historically fascinating but tied to a now-obsolete architecture and hard to follow without PDP-11 knowledge.
- The Portable C Compiler `pcc` [?] (1979), written by Steve Johnson, made C portable across architectures and was used in UNIX V7 and BSD. At about 7 000 LOC, it is a reasonable size, but its code-generation tables are dense and hard to read. `pcc` was the standard UNIX compiler until GCC replaced it.
- `gcc` — about 15 000 000 LOC across all frontends and backends. Even the C frontend alone is enormous. GCC is an engineering marvel but far too large to read cover to cover.
- `clang/LLVM` — cleaner than GCC and with excellent error messages, but the LLVM infrastructure alone is several million LOC. The modular architecture is admirable, but the sheer size puts it out of reach for a book like this.
- `lcc` — about 20 000 LOC and arguably better structured than 5c, with a clean retargetable backend. But there is already a book on it [?], its copyright is restrictive, and it does not use yacc (or lex), so the frontend is harder to follow.
- `tcc` — Fabrice Bellard’s Tiny C Compiler, about 30 000 LOC. Compiles C almost instantly and is self-hosting, but it targets x86 (more complex than ARM) and its code is optimized for compilation speed rather than readability.
- `8cc` and `chibicc` — two small, self-hosting C compilers by Rui Ueyama (about 6 000 and 10 000 LOC respectively). `chibicc` in particular has excellent, readable code and would make a fine subject for a book. But neither uses yacc, and neither is a production compiler—they are educational projects that were never used to build a real operating system.
- `compcert` — a formally verified compiler written in Coq, with a proper IL and CFG, but much harder to read for anyone not fluent in Coq.
- `bcc` — Bruce Evans’s compiler (about 8 500 LOC), historically notable as the compiler Linus Torvalds used for early Linux development. It targets the 8086/80386 and is showing its age.

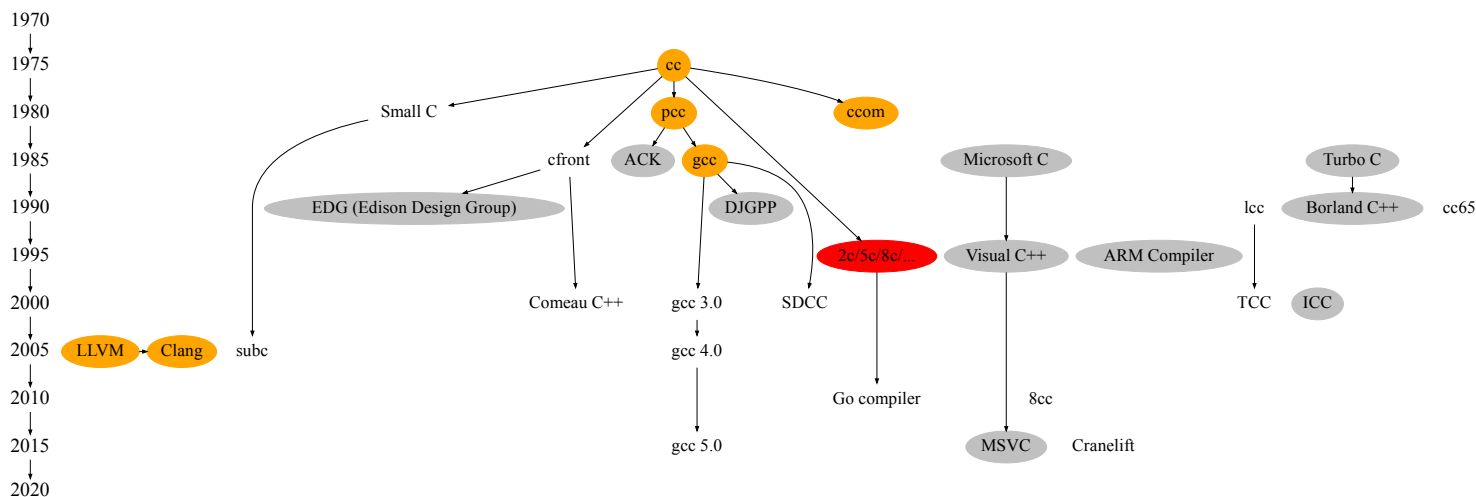


Figure 1.1: C compilers timeline

Figure 1.1 presents a timeline of major C compilers. Dennis Ritchie’s `cc` (1973) is the ancestor of all C compilers. The Portable C Compiler `pcc` (1979) made C available on many architectures and was the standard UNIX compiler for years. GCC (1987) and later Clang/LLVM (2007) dominate today. The Plan 9 compilers (in red) descend directly from Ritchie’s original `cc` and occupy a unique niche: production-quality compilers small enough to read in full.

I think `5c` represents the best compromise for this book: it is a real, production compiler—used to build an entire operating system—yet its 15 000 LOC are within reach of a single reader. It uses standard tools (Yacc for parsing), targets a clean RISC architecture (ARM), and its one-pass design, while limiting optimization, makes the code flow easy to follow from input to output.

1.4 Getting started

To play with `5c`, you will first need to install the Plan 9 fork used in Principia Softwarica (see <https://www.principia-softwarica.org/getting-started.html>).

Unlike `gcc`, which invokes the assembler and linker automatically, `5c` only compiles—you must call the linker `5l` separately to produce an executable:

```
$ 5c hello.c
$ 5l hello.5
$ 5.out
Hello World
```

You can also use `5c -S` to see the assembly output without generating an object file, which is useful for understanding what the compiler does.

The Plan 9 toolchain (`5a`, `5l`, `5c`) can also be compiled natively on Linux, macOS, or Windows through Goken9cc¹, so you can cross-compile ARM programs from your regular machine.

1.5 Requirements

This book assumes you know C well enough to read the compiler’s source code (which is itself written in C). Some familiarity with compiler concepts—ASTs, lexers, parsers, type systems—is helpful but not strictly required,

¹<https://github.com/aryx/goken9cc>

as the Principles section in the next chapter provides the essential background. Reading the ASSEMBLER book [Pad15a] or EMULATOR book [Pad15b] first is recommended, as 5c generates ARM assembly directly (there is no intermediate representation), so understanding the target helps. For a thorough treatment of compiler theory, see Aho et al. [ALSU06] (“the Dragon Book”), Cooper and Torczon [CT11], or Appel [App04].

If, while reading this book, you have specific questions on the command-line interface of 5c or its extensions to C, I suggest you to consult the manual pages in my Plan 9 repository: `docs/man/1/8c` (despite the name, this page documents all the Plan 9 compilers, including 5c) and `docs/man/1/cpp` for the pre-processor. The `compilers/docs/` directory also contains two papers by Ken Thompson and Rob Pike describing the design of the Plan 9 C compilers: `compilers/docs/compiler.pdf` (“Plan 9 C Compilers”) and `compilers/docs/comp.pdf` (“How to Use the Plan 9 C Compiler”). Finally, the “Portability and the C Compilers” paper (at `docs/articles/port.ps` in my Plan 9 repository) explains the portability strategy shared by the whole Plan 9 toolchain, which is useful background for the code generation chapters.

1.6 About this document

This document is a *literate program* [Knu92]. It derives from a set of files processed by a tool, `syncweb` [Pad09], generating either this book or the actual source code of the program. The code and its documentation are thus strongly connected.

1.7 Copyright

Most of this document is made of source code from Plan 9, so those parts are copyright by Plan 9 Foundation. The prose is mine and is licensed under the All Rights Reserved License.

1.8 Acknowledgments

I would like to acknowledge of course the author of 5c, Ken Thompson, who wrote in some sense most of this book.

Chapter 2

Overview

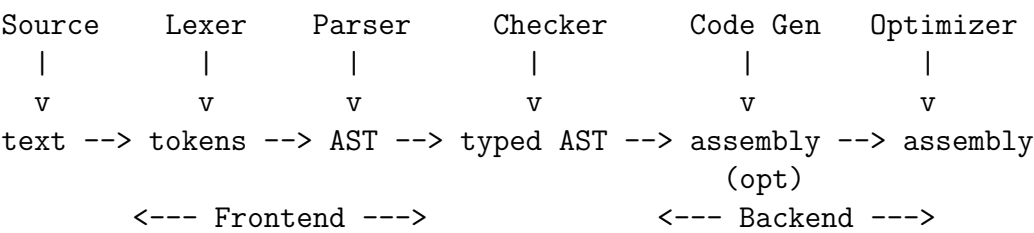
Before showing the source code of 5c in the following chapters, I first give an overview in this chapter of the general principles of a compiler, of its command-line interface, and of the input and output formats. I also define terms and more generally give the background necessary to understand the code I will show later.

2.1 Compiler principles

A compiler translates a high-level language into a low-level one. In the case of 5c, the input is C and the output is ARM assembly (in Plan 9’s AST-based object format). There are many books on compiler construction—the “Dragon Book” [ALSU06], Appel’s “Modern Compiler Implementation” [?], and others. I will be brief here and focus on the aspects most relevant to understanding 5c.

The compiler lineage starts with Fortran (John Backus, IBM, 1957)—the first *optimizing* compiler, which proved that a high-level language could produce code competitive with hand-written assembly. C (Dennis Ritchie, 1972) was the first language whose compiler was good enough to write an operating system kernel in. GNU’s gcc (Richard Stallman, 1987) became the dominant open source compiler for two decades. LLVM (Chris Lattner, 2003) offered a modular architecture that made it easy to add new frontends (Clang for C/C++, Rust, Swift) and new backends (ARM, x86, RISC-V, WebAssembly), and is now the foundation of most non-GNU toolchains. 5c sits at the minimal end of this spectrum: around 10 000 lines of C, readable end-to-end, with none of the framework machinery that makes gcc or LLVM powerful but enormous.

Compilers are traditionally divided into a *frontend* (parsing and analysis) and a *backend* (code generation and optimization). The following diagram shows the main phases of a typical compiler:



5c follows this structure, though it blurs some boundaries: parsing and code generation are interleaved (each function is compiled immediately after parsing), and there is no separate intermediate representation between the AST and the assembly instruction list.

2.1.1 Frontend

The frontend transforms source text into a structured representation. It consists of three phases: *lexing* (breaking text into tokens), *preprocessing* (handling #include, #define, and #ifdef), and *parsing* (building an abstract syntax tree from the token stream according to C’s grammar). If you have read the ASSEMBLER book [Pad15a]

or SHELL book [Pad18], you have already seen all three phases—the same techniques (hand-written lexer, macro expansion, grammar-driven parsing) appear in 5c, applied to the more complex syntax of C.

Modern compilers overwhelmingly use hand-written recursive-descent parsers rather than generated ones: GCC moved from yacc to hand-written in 2004 for better error recovery, Clang was always hand-written, and Rust, Go, and Swift followed suit. 5c is an exception—it still uses yacc, which keeps the grammar readable but limits error-message quality.

2.1.2 Scope resolution

C has nested blocks, and a variable in an inner block can shadow a variable of the same name in an outer block. C also has type aliases (`typedef`), structure tags, enumeration constants, and labels—all in partially overlapping namespaces. The goal of scope resolution is to associate each *use* of a name with the correct *definition*. This is done through the symbol table, which tracks the current scope and allows declarations to be pushed and popped as the compiler enters and exits blocks.

2.1.3 Use/def checking

Closely related to scope resolution, use/def checking catches three kinds of errors: *use of undefined* entities (referencing a variable that was never declared), *redefinition* (declaring the same name twice in the same scope), and *unused* definitions (declaring something that is never referenced). 5c performs these checks for labels, local variables, and parameters, and additionally includes dataflow checks for variables that are set but never used.

2.1.4 Type checking

Since C is a statically typed language, the compiler must verify that operations are applied to compatible types: that function arguments match parameters, that array indices are integers, that pointer dereferences apply to pointers, and so on. The type checker also annotates each AST node with its type, which the backend needs to generate correctly sized instructions (a `char` move vs. a `long` move, integer vs. floating-point arithmetic). The type checker also classifies expressions as *lvalues* (locations that can be assigned to) or *rvalues* (values that can only be read), which determines what operations are legal on each expression.

C’s type system is relatively weak: no generics, no sum types, `void*` as a universal escape hatch, and implicit integer conversions that silently lose data. Modern languages push the type checker much further: Rust’s *borrow checker* tracks ownership and lifetimes at compile time to prevent use-after-free without a garbage collector; OCaml’s *algebraic types* make unhandled cases a compile error; Haskell’s *type classes* express abstractions the C type system cannot even name. The trade-off is the familiar “worse is better” question: C’s weak types are simpler and more permissive, letting more bugs through but also letting more code compile without fighting the type checker.

2.1.5 Backend

The backend’s job is to translate the typed AST into assembly instructions. Assembly is a much simpler language than C: arithmetic on registers, memory loads and stores, labels and jumps, and a manually managed stack. The compiler must bridge the gap between C’s structured constructs and this flat instruction sequence. This bridge can be understood as three translations, which I call *labelization*, *linearization*, and *frameization*.

2.1.6 Labelization and control-flow lowering

C has structured control flow: `if/else`, `while`, `for`, `switch`, `break`, `continue`, `return`. Assembly has only labels and conditional/unconditional jumps. *Labelization* is the translation from one to the other: an `if(cond)`

A **else** B becomes a conditional branch past A to B, with a label at the join point. A **while** loop becomes a backward jump to the condition test.

C code	ARM assembly
-----	-----
if(x > 0)	CMP x, \$0
y = 1;	BLE L1
else	MOVW \$1, y
y = 2;	B L2
	L1:
	MOVW \$2, y
	L2:

2.1.7 Linearization and three-address code

C expressions can be arbitrarily nested: $x + y * z - f(a, b)$. Assembly instructions operate on at most two or three registers at a time. *Linearization* decomposes complex expressions into a sequence of simple instructions, allocating temporary registers for intermediate results. For example, $x + y * z$ becomes: load y into a register, multiply by z , add x . The same applies to data: a C struct with nested fields is decomposed into byte-level accesses at computed offsets.

C code	ARM assembly
-----	-----
z = x + y * 3;	MOVW y, R0
	MOVW \$3, R1
	MUL R1, R0 // R0 = y * 3
	MOVW x, R1
	ADD R1, R0 // R0 = x + y*3
	MOVW R0, z

2.1.8 Instruction selection

After linearization decomposes expressions into simple operations (load, add, multiply, store), the compiler must choose which *machine instructions* implement each operation. This is *instruction selection*, and it is less trivial than it sounds: ARM offers many ways to compute the same result. A multiply by 4 can be a shift left by 2. An add of a small constant can use an immediate operand instead of loading the constant into a register first. A load followed by an add can sometimes be combined into a single load-with-offset instruction. Choosing the best instruction (or sequence) for each operation is what separates good generated code from mediocre.

Classical compiler textbooks describe *tree pattern matching* (BURG, Aho and Johnson 1976): the compiler tiles each expression tree with instruction patterns and picks the minimum-cost tiling. GCC and LLVM use elaborate machine-description files that automatically generate instruction selectors from declarative rules. 5c uses a simpler approach: hand-written C code that walks the AST and emits instructions directly, with ad hoc rules for the special cases (shift-for-multiply, immediate folding, combined addressing modes). This is less systematic but far easier to read, and the ARM's relatively regular instruction set means the number of special cases stays manageable.

2.1.9 Frameization and stack-frame layout

C has function parameters, local variables, and return values. Assembly has only the stack pointer and registers. *Frameization* is the mapping between the two: the compiler assigns each local variable and parameter an offset

within the function’s *stack frame*, generates code to set up the frame on entry (adjusting the stack pointer), and tears it down on exit. The *calling convention* defines which registers hold arguments, where the return value goes, and which registers must be saved and restored across calls.

Calling conventions vary across architectures and operating systems. ARM’s AAPCS (ARM Architecture Procedure Call Standard) puts the first four arguments in R0–R3 and the rest on the stack; x86-64 System V (Linux, macOS) uses six registers; Windows x64 uses four different registers. Plan 9 defines its own convention (R0 for first argument and return value, described in LIBCORE book [Pad16b]’s assembly crash course). Getting the calling convention wrong produces bugs that are extraordinarily hard to diagnose: the callee reads its arguments from the wrong registers, and the program crashes far from the actual error.

C code	ARM assembly
-----	-----
int add(int a, int b) {	TEXT add+0(SB), \$8 // 8 bytes for locals
int sum;	MOVW a+0(FP), R0 // param a at FP+0
sum = a + b;	MOVW b+4(FP), R1 // param b at FP+4
return sum;	ADD R1, R0
}	MOVW R0, sum-4(SP) // local at SP-4
	MOVW R0, .ret+0(FP) // return value
	RET

2.1.10 Optimizations

After generating assembly, the compiler can improve it. 5c performs two levels of optimization: *AST-level* transformations (constant folding, strength reduction, algebraic simplification) and *assembly-level* optimizations (register allocation across basic blocks, and peephole optimization to eliminate redundant instructions). These are modest compared to what GCC or LLVM do, but they are effective and easy to understand.

To put 5c’s modest optimization in perspective: GCC runs roughly 200 optimization passes over its IR, LLVM runs about 100, and 5c runs about 10—a difference that reflects the “worse is better” trade-off between peak code quality and compiler simplicity.

2.1.11 Intermediate representations and SSA

Most modern compilers do not translate directly from AST to assembly. Instead, they lower the AST into an *intermediate representation* (IR)—a language simpler than C but richer than assembly—and perform most optimizations there. GCC has two IRs: GIMPLE (a simplified C-like tree) and RTL (register transfer language, close to assembly). LLVM’s IR is a single, well-defined language that every frontend (Clang, Rust, Swift) compiles *into* and every backend (ARM, x86, RISC-V) compiles *out of*—which is what makes LLVM so modular.

The most important property of modern IRs is *Static Single Assignment* (SSA), formalized by Cytron et al. in 1991: every variable is assigned exactly once, and when a variable needs to be updated, a new version is created. This sounds restrictive but it makes dataflow analysis trivial—you can follow any value back to its unique definition without searching through a list of possible assignments—which enables optimizations like constant propagation, dead-code elimination, and common-subexpression elimination to be expressed as simple graph walks. GCC adopted SSA in 2004 (GCC 4.0); LLVM’s IR has been SSA from the start.

5c has *neither* an IR nor SSA. It goes straight from the AST to an assembly instruction list, performing optimizations directly on both. This is a deliberate “worse is better” simplification: it keeps the compiler small (no IR data structures to maintain, no SSA ϕ -node insertion pass) at the cost of missing the cross-function and global optimizations that SSA-based compilers excel at. For the workloads Plan 9 was designed for—operating system code, command-line tools, compilers themselves—the simpler approach was fast enough.

2.1.12 Register allocation

One backend problem deserves its own mention because it is the most studied optimization in compiler history. The AST and linearization phase can use as many temporary values as they want, but the CPU has a finite number of registers (16 on ARM, even fewer effectively available after reserving SP, LR, PC, and SB). *Register allocation* is the mapping from an unlimited set of virtual temporaries to this finite set, with any overflow *spilled* to the stack.

The classical formulation (Gregory Chaitin, IBM, 1981) models the problem as *graph coloring*: build an *interference graph* whose nodes are temporaries and whose edges connect temporaries that are live at the same time; then color the graph with k colors (one per physical register). If a node cannot be colored, its temporary is spilled. This produces excellent code but is expensive (graph coloring is NP-complete in general, though Chaitin’s heuristic works well in practice). *Linear scan* (Poletto and Sarkar, 1999) trades some code quality for speed: it walks the instruction list once, assigning registers in order of live-range start, and is the algorithm of choice for JIT compilers (JVM HotSpot’s C1, V8’s Crankshaft) where compilation speed matters more than the last 5% of code quality. 5c uses a simpler per-basic-block allocation that is fast enough for the code sizes Plan 9 targets.

2.1.13 Peephole optimization

After instruction selection and register allocation, the generated code still contains local inefficiencies: a value loaded into a register and then immediately moved to another (the move is redundant); a store followed by a load of the same address (the load can be eliminated); a conditional branch over a single instruction (on ARM, the instruction can be conditionally executed instead). A *peephole optimizer* fixes these by scanning a small sliding window of consecutive instructions—the “peephole” through which it views the code, typically 2–5 instructions wide—and applying pattern-replacement rules to clean up what it sees. The name comes from the image of looking at a long instruction list through a tiny hole: you can only see a few instructions at a time, but that is enough to spot and fix local waste.

The idea dates to William McKeeman (1965), and the technique is one of the simplest in the compiler: each rule is a pattern match (“if you see instruction A followed by instruction B, replace with instruction C”), and the whole pass is a single loop over the instruction list. Despite its simplicity, it produces significant speedups by removing the overhead that the earlier, more structured passes left behind. 5c’s peephole optimizer is under 200 lines and is one of the most readable parts of the backend.

2.1.14 Undefined behavior

C grants the compiler permission to assume that certain operations *never happen*: signed integer overflow, dereferencing a null pointer, reading an uninitialized variable, shifting by more than the word width. These are called *undefined behavior* (UB), and the language standard says the compiler may do “anything” when they occur—including silently removing the code that would have detected them. Modern compilers exploit this aggressively: if signed overflow is UB, GCC and Clang can assume $x + 1 > x$ is always true and optimize away the overflow check, which sometimes causes programs that “worked” with older compilers to break silently under -O2.

5c is far less aggressive. It rarely exploits UB for optimization, which means it generates slower code but produces fewer surprises—a trade-off the Plan 9 developers considered worthwhile for systems code where predictability matters more than peak performance. The UB debate is one of the sharpest instances of the “worse is better” tension in modern C: GCC and Clang choose “better” code at the cost of surprising behavior; 5c and Plan 9 choose “worse” code that does what the programmer expected.

2.1.15 Cross-compilation

Most compilers treat cross-compilation—generating code for a different architecture than the one the compiler runs on—as a special configuration that requires passing flags like `--target=arm-linux-gnueabi`. Plan 9’s toolchain was designed for cross-compilation *from the start*. The numbered-tool convention (5c/5a/51 for ARM, 8c/8a/81 for x86, 6c/6a/61 for AMD64) makes every compiler a cross-compiler by default: you compile ARM code on an x86 machine by simply typing 5c instead of 8c, with no configuration, no sysroot, no target triple. This is possible because the Plan 9 compilers carry no host dependencies—they read source, emit object code, and link to Plan 9’s own `libc`, regardless of which machine they happen to be running on. The modern echo of this philosophy is Zig (Andrew Kelley, 2016), which ships its own C/C++ cross-compiler and has gained significant adoption precisely because `zig cc --target=aarch64-linux` “just works” where configuring a GCC or Clang cross-toolchain is a multi-step ordeal. Plan 9 got this right in the 1990s.

2.1.16 Bootstrapping

A compiler written in its own language faces a chicken-and-egg problem: you need the compiler to compile the compiler. The standard solution is *bootstrapping*: start with a version compiled by some other means (a previous version, a different compiler, or even a hand-written translator), then use that to compile the “real” compiler from source. 5c was bootstrapped from earlier Plan 9 compilers; Go’s compiler, which descends from the Plan 9 toolchain, was famously rewritten from C to Go in 2015 and bootstrapped via a chain: Go 1.4 (the last C version) compiles Go 1.5 (the first Go version), which compiles all subsequent versions.

Ken Thompson’s 1984 Turing Award lecture “*Reflections on Trusting Trust*” revealed a deeper problem with bootstrapping: a compiler can be modified to insert a backdoor into any program it compiles—including *future versions of itself*—so that the backdoor propagates invisibly through every bootstrap, even if you read every line of the compiler’s source code and find nothing suspicious. The attack is undetectable by source inspection alone. The modern response is *diverse double compilation* (David A. Wheeler, 2009): compile the compiler with two *independent* compilers and verify that the outputs match, which catches any compiler-specific backdoor because the two compilers would have to contain the same one. Thompson’s lecture remains one of the most cited in computer science and is a reminder that trust in software is always, ultimately, trust in the toolchain.

2.1.17 The most demanding reader of your code

A compiler reads your source code more carefully than any human ever will. It checks every type, traces every variable’s lifetime, verifies that every function call matches its declaration, detects unreachable code, warns about uninitialized variables, and then transforms the result into machine code you could not write by hand—all in a fraction of a second. This is easy to take for granted when the compiler is a black box behind `make`, but the rest of this book opens the box. What you will find is that most of the compiler’s complexity is devoted not to the translation itself (which is surprisingly mechanical) but to getting the *details* right: the edge cases in type promotion, the interaction between register allocation and spilling, the subtle correctness requirements of instruction selection. The compiler is, in effect, the programmer’s most demanding colleague—one who never gets tired, never skims, and never lets an inconsistency pass.

2.2 5c command-line interface

The main flags of 5c fall into a few categories:

Output:	<code>-o file</code>	output file
	<code>-S</code>	print assembly (text)
Warnings:	<code>-w</code>	enable warnings

	-V	enable void* conversion warnings
	-F	format specification check
	-X	abort on first error
Preprocessor:	-D name	define macro
	-I path	add include path
	-p	use external /bin/cpp
Debugger:	-a	generate acid declarations
	-T	safe linking (signature-based)
Internals:	-v	verbose mode
	-d	print declarations
	-t	print type trees
	-S	print generated assembly
	-r	print register allocation

A typical build rule uses `-FTVw` (format check, safe linking, void* warnings, all warnings), as seen in the standard `mkfile.proto`. The `-S` flag is especially useful for understanding the compiler: it dumps the assembly text for each function as it is compiled.

2.3 Compiling `helloworld.c`

Here is a minimal C program that exercises an arithmetic expression and a `print` call:

```
void main(void) {
    print("the answer is %d\n", 40 + 2);
}
```

Compiling it with `5c -S` produces assembly that shows the three translations from the Overview: frameization (the function prologue/epilogue), linearization (the constant 42 loaded into a register and pushed as an argument), and labelization (no branches here, just a straight-line sequence):

```
TEXT main+0(SB), $0
    MOVW $42, R0
    MOVW R0, 4(SP)
    MOVW $s+0(SB), R0
    MOVW R0, 0(SP)
    BL   print+0(SB)
    RET

GLOBL s+0(SB), $20
DATA   s+0(SB)/4, $"the "
DATA   s+4(SB)/4, $"answ"
DATA   s+8(SB)/4, $"er i"
DATA   s+12(SB)/4, $"s %d"
DATA   s+16(SB)/2, $"\\n\\z"
```

The constant expression `40 + 2` is folded at compile time into 42. The string literal `"the answer is %d\n"` becomes a static data item labeled `s`, declared with `GLOBL` (which reserves 20 bytes) and initialized in 4-byte chunks by a series of `DATA` directives.

See the ASSEMBLER book [[Pad15a](#)] for a full explanation of the `TEXT`, `GLOBL`, and `DATA` pseudo-instructions and the addressing modes used here (`SB`, `SP`, `FP`).

2.4 Input C language

5c compiles mostly ANSI C (C89), with a few Plan 9 extensions such as anonymous structs/unions and the `#pragma` directives for format-string checking and safe linking. C is a systems programming language—pointers, explicit memory management, and bit manipulation are central—and 5c must handle all of these, including the many dark corners of C’s syntax and semantics.

C’s main innovation over Fortran and Pascal was *pointer arithmetic*: pointers are first-class values that can be incremented, compared, and subtracted. Other languages used pointers internally, but C exposed them in the language with an algebra around them (`p + n`, `p - q`, `*p`). This gives C its power as a systems language, but it also makes the compiler’s job harder—there are no bounds checks and wild pointers are easy to create.

C also introduced a rich set of bit-manipulation operators (`&`, `|`, `^`, `<<`, `>>`, and their assignment forms), making it a natural Domain Specific Language (DSL) for the kind of hardware-level code that fills the LINKER book [Pad15c], EMULATOR book [Pad15b], and ASSEMBLER book [Pad15a]. Bitfields let structures map directly onto hardware register layouts. Finally, C’s overloaded arithmetic (the same `+` works for `int`, `long`, `float`, and `double`) simplifies the surface language at the cost of a complex type-checker that must insert the right coercions.

2.5 Output object format

Like 5a, 5c generates object files in Plan 9’s AST-based format (not machine code). The output is essentially a serialized list of assembly instructions—the same `Prog`^{531b} structures that the ASSEMBLER book [Pad15a] describes. The linker 5l then performs the actual machine code generation. You can use 5c `-S` to see the assembly in textual form.

2.6 The ARM architecture

You do not need deep knowledge of the ARM architecture to read this book. The Plan 9 assembler 5a abstracts away many hardware details with pseudo registers (FP, SP, SB) and virtual instructions (MOVW, RET). A few instructions are enough to see how C maps to assembly:

C code	ARM assembly (Asm5)
-----	-----
<code>x = 1;</code>	<code>MOVW \$1, R0</code> <code>MOVW R0, x(SB)</code>
<code>y = &x;</code>	<code>MOVW \$x(SB), R0</code> <code>MOVW R0, y(SB)</code>
<code>z = x + y;</code>	<code>MOVW x+0(FP), R0</code> <code>MOVW y+4(FP), R1</code> <code>ADD R1, R0</code> <code>MOVW R0, z-4(SP)</code>

The ASSEMBLER book [Pad15a] explains the full instruction set and addressing modes in detail.

2.7 Plan 9 design choices

A compiler must make many decisions about how to use the target machine’s registers. Some of these are *hardware constraints* imposed by the architecture (e.g., which register is the program counter), and some are *design choices* that the compiler or calling convention can decide freely (e.g., which registers to use for passing

arguments). The most important design choice is the *calling convention*: which registers are preserved across function calls and which are not.

In a *caller-saved* (“volatile”) convention, the caller saves any registers it still needs before the call and restores them after; the callee is free to clobber them. In a *callee-saved* (“non-volatile”) convention, the callee saves the registers it wants to use on entry and restores them before returning; the caller can assume they are untouched. In practice, architectures use a mix of both: some registers are caller-saved and some are callee-saved.

Caller-saved (R0-R3):	Callee-saved (R4-R11):
foo:	foo:
MOVW \$1, R0 // use R0	MOVW R4, -4(SP) // save R4
MOVW R0, -4(SP) // save R0	MOVW \$1, R4 // use R4
BL bar	BL bar
MOVW -4(SP), R0 // restore	// R4 still valid, no restore needed
...	MOVW -4(SP), R4 // restore R4
	RET

With caller-saved registers, the caller must save and restore around every call. With callee-saved registers, the callee saves once on entry and restores once on exit, but only if it actually uses the register. Functions that make no calls (leaf functions) benefit most from callee-saved registers: they can use them freely without saving.

On ARM, the split between the two is fixed by the ARM Procedure Call Standard (APCS): R0–R3 are caller-saved (used for arguments and scratch) and R4–R11 are callee-saved. This is a hardware constraint that 5c must respect. But within that framework, Plan 9 makes its own design choices: it reserves certain registers for fixed purposes (R12 as the static base *SB*, R13 as *SP*, R14 as the link register, R15 as *PC*), and it uses a leaf-function optimization where functions that make no calls can skip saving callee-saved registers entirely.

2.8 Code organization

The source is split into two layers: a portable front-end in *cc/* (shared by all architectures) and a back-end in *5c/* (ARM-specific code generation). The front-end contains the lexer (*lex.c*), the embedded preprocessor (*mac.c*), the yacc grammar (*cc.y*), declaration handling (*dcl.c*), type-checking (*com.c*), and helpers (*sub.c*, *globals.c*). The back-end in *5c/* has the ARM code generator (*cgen.c*, *sngen.c*), the peephole optimizer (*peep.c*), the register allocator (*reg.c*), and multiplication/division strength-reduction (*mul.c*, *div.c*). There is no *lex* tool: the lexer is hand-written, partly for speed and partly to handle Unicode identifiers directly. The yacc grammar, however, does use *yacc*.

Table 2.1 presents short descriptions of the source files of *5c*, together with the main entities (e.g., structures, functions, globals) the file defines, and the corresponding chapter in this document in which the code contained in the file is primarily discussed.

2.9 Software architecture

Unlike a textbook compiler with separate preprocessing, parsing, type-checking, optimization, and code-generation passes, *5c* is essentially a *one-pass* compiler: it preprocesses, parses, type-checks, and generates assembly for each function in a single streaming pass. The call chain is:

```
main -> compile -> yyparse -> yylex -> macxxx   (lex + cpp)
                                -> codgen -> gen(stmt) -> cgen(expr)
                                -> regopt -> peep
                                -> gclean -> outcode      (emit .5 file)
```

Function	Ch.	File	Entities	LOC
frontend types and globals	3	cc/cc.h	Node ^{504b} Type ^{504b} Sym ^{504b} Decl ^{504b}	1142
backend types and globals	3	5c/gc.h	Reg ^{531b} AddrX Multab ^{531b}	443
frontend globals	3	cc/globals.c	symb ^{35g} types ^{32b} ncast ^{249c}	215
backend globals	3	5c/globals2.c	regnode ^{209a} nregion ^{353h}	168
AST helpers	3	cc/sub.c	nodconst() ^{195c} new1() ^{29h} tcomo() ^{153b}	2353
entry point	4	cc/lex.c	main() ^{45d} compile() ^{47h} yylex() ^{57a}	1823
preprocessor	7	cc/mac.c	macexpand() ^{90c} macdef() ^{85b} macinc() ^{81f}	1058
parser (yacc grammar)	8	cc/cc.y	yyparse()X	1099
parser (generated)	8	cc/y.tab.c		1808
declarations	8	cc/dcl.c	dodecl() ^{103a} xdecl() ^{104e} adecl() ^{140c}	1974
type checker	9	cc/com.c	tcom() ^{152b} tcoma() ^{173a} typeconv()X	1707
64-bit type operations	9	cc/com64.c	com64() ^{459d}	710
string constants	9	cc/scon.c	evconst() ^{304a}	627
format/pragma checks	9	cc/dpchk.c	dpcheck() ^{447b} argfmt()X	554
code generator (expressions)	10	5c/cgen.c	cgen() ^{223b} boolgen() ²⁵³ gopcode() ^{229c}	1315
code generator (statements)	10	5c/sgen.c	codgen() ^{194c} gen() ^{197e}	242
code generator (portable)	10	5c/txt.c	ginit() ^{51c} gclean() ^{49c} gins() ^{219a}	1554
switch generation	10	5c/swt.c	swit()X outcode() ^{287c} outhist() ^{290b}	691
type to string	10	cc/funct.c	typestr()X typeconv()X	409
AST optimizations	12	cc/com.c	constant folding, strength reduction	
multiply optimization	12	5c/mul.c	mulcon() ^{314b}	639
register allocator	13	5c/reg.c	regopt() ^{339b} regalloc() ^{221f} paint()X	1226
peephole optimizer	13	5c/peep.c	peep() ^{366c} subprop() ³⁷¹ copyprop() ^{373a}	1510
linker support	14	5c/txt.c	zname() ^{290a} zaddr() ^{288b}	
debugger support	15	cc/acid.c	acidvar() ^{401c} acidtype() ^{403b}	339
debugger support (pickle)	15	cc/pickle.c	pickletype() ⁴¹⁰ picklesue() ^{408a}	222
profiling support	16	5c/txt.c	gclean() (APROF)	
bitset utilities	A	cc/bits.c	bset() bnum()	92
enum names	A	5c/enam.c	anames ^{492e}	99
pretty printer	A	5c/list.c	Pconv() ⁴⁸⁰ Aconv() ^{481b} Dconv() ^{481c}	328
error management	B	cc/lex.c	diag() ^{496b} warn() ¹⁴² yyerror() ^{495c}	
misc utilities	C	cc/utls.c	alloc() ^{500b}	111
compatibility	C	cc/compat.c	myaccess() ^{513b} mycreat() ^{513b}	76
Total				22 500

Table 2.1: Chapters and associated 5c source files.

Each function’s code is emitted immediately after it is parsed, so the compiler never holds the entire program in memory. Toplevel globals are collected and emitted at the end by `gclean`. There is no control-flow graph and no intermediate language: code generation walks the AST directly and produces ARM instructions into a linked list. Optimizations happen at two levels: AST-level rewrites (constant folding, strength reduction) and assembly-level passes (register allocation, peephole).

The central data structure is the symbol table, which does far more than name resolution: it tracks types, storage classes, stack offsets, and scope nesting for every identifier, struct tag, and label in the program.

2.10 Book structure

The book follows the compiler’s own pipeline. After this introduction and the core data structures (Chapter 3), we trace the flow of a C source file through `main` (Chapter 4), input buffering (Chapter 5), lexing (Chapter 6), preprocessing (Chapter 7), and parsing (Chapter 8). The type-checker is covered in Chapter 9, followed by ARM assembly generation (Chapter 10) and object file output (Chapter 11).

Two chapters cover optimizations: AST-level rewrites like constant folding and strength reduction (Chapter 12), and assembly-level passes including register allocation and peephole optimization (Chapter 13). Three shorter chapters cover the metadata that 5c emits for other tools: linking support for safe cross-file type-checking (Chapter 14), debugging metadata for the `acid` debugger (Chapter 15), and profiling hooks (Chapter 16). Chapter 17 covers advanced C features such as initializers, bitfields, floating-point, and variadic functions.

The appendix chapters contain the compiler’s self-debugging infrastructure (Chapter A), error management (Chapter B), and extra code that did not fit in the main narrative (Chapter D).

Chapter 3

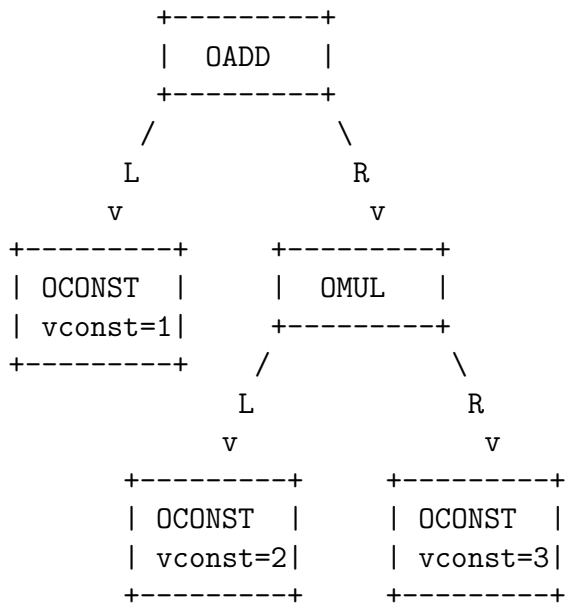
Core Data Structures

The compiler’s central data structures are the **Node** (AST), **Type**, **Sym** (symbol table entry), and **Prog** (assembly instruction). The AST is a simple binary tree: every node has a **kind** (**op**), a **left** and **right** child, and optional annotations for type, storage class, and code generation hints. The symbol table is a stack of scopes, supporting C’s nested block structure.

3.1 Abstract syntax tree: Node

The AST uses a single **Node** type for expressions, statements, and declarations alike — a uniform binary tree where **left** and **right** carry different meanings depending on the node kind (**op**). For instance, **OADD** uses them as arithmetic operands, while **OLIST** chains them as a linked list of statements. This generic structure is compact but means that many fields (type, storage class, register hints) are only meaningful for certain node kinds; the dumper **prtree** in the Debugging chapter is the best guide to which fields apply where.

For instance, the expression `12*3+` produces this tree (each box is one **Node**, with its **op** and its **left/right** pointers; leaves hold their value in **vconst**):



Every node has the same four mandatory fields (**op**, **left**, **right**, **lineno**) plus a grab-bag of optional ones. A statement list reuses the same shape: **OLIST**’s **left** is the first statement, **right** is an **OLIST** for the rest — a cons cell pretending to be a binary tree.

`<enum node_kind 27>≡` (504b)
`enum Node_kind`

```

{
    OXXX,

    // -----
    // Names
    // -----
    ONAME, // for uses and declarations

    // -----
    // Expressions
    // -----
    <expression nodes 110g>

    // -----
    // Statements
    // -----
    <statement nodes 107a>

    // -----
    // Declarations (parameters, initializers, bit fields)
    // -----
    <declaration nodes 119e>

    // -----
    // Misc
    // -----
    <misc nodes 29b>

    // -----
    // Post parsing nodes
    // -----
    <after parsing nodes 208f>

    OEND
};

<struct Node 28>≡ (504b)
struct Node
{
    // enum<node_kind>
    char op;

    // option<ref_own<Node>>
    Node* left;
    // option<ref_own<Node>>
    Node* right;

    // after preprocessing, global lineno
    long lineno;

    <Node value fields 29c>

    // -----
    // Post parsing annotations
    // -----
    <Node type and storage fields 31g>
    <Node code generation fields 33d>
    <Node origin tracking fields 118c>

    // -----

```

```

// Misc
// -----
⟨Node parsing helper fields 135b⟩
};

⟨constant Z 29a⟩≡ (504b)
#define Z ((Node*)nil)

⟨misc nodes 29b⟩≡ (27) ??▷
OLIST, // of stmts/labels/parameters/... and also for pairs/triples/...

⟨Node value fields 29c⟩≡ (28) 111b▷
Sym* sym; // for ONAME, ODOT/OELEM, OXXX of OLABEL/OGOTO

⟨function new 29d⟩≡ (518c)
Node*
new(int t, Node *l, Node *r)
{
    Node *n;

    n = alloc(sizeof(Node));
    n->op = t;
    n->left = l;
    n->right = r;

    ⟨new() set lineno 29e⟩

    return n;
}

Uses alloc() 500b.

⟨new() set lineno 29e⟩≡ (29d) 470c▷
if(l && t != OGOTO)
    n->lineno = l->lineno;
else if(r)
    n->lineno = r->lineno;
else
    n->lineno = lineno;

Uses OGOTO 108e and lineno 29f.

⟨global lineno 29f⟩≡ (511d)
long lineno;

⟨global nearln 29g⟩≡ (511d)
long nearln;

⟨function new1 29h⟩≡ (518c)
Node*
new1(int o, Node *l, Node *r)
{
    Node *n;

    n = new(o, l, r);
    n->lineno = nearln;
    return n;
}

Uses nearln 29g and new() 29d.

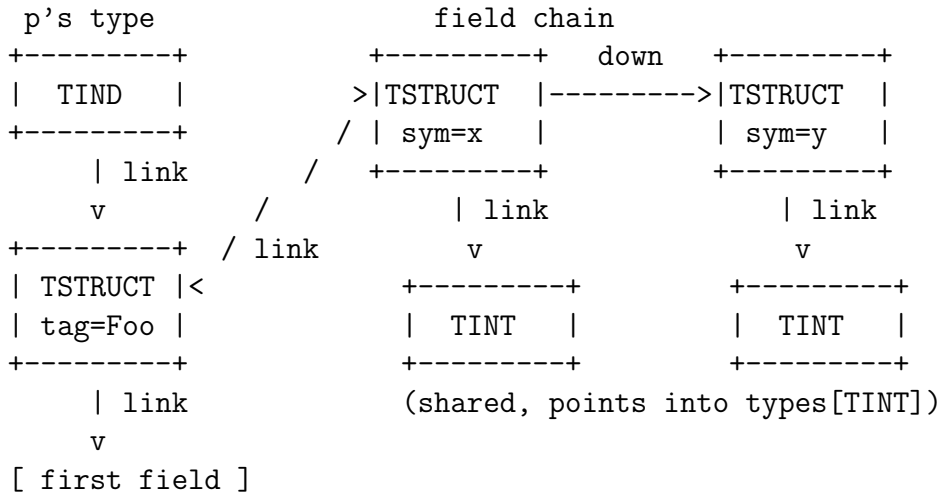
⟨global znode 29i⟩≡ (535b)
Node znode;

```

3.2 Type system

Types in 5c are not AST nodes: they form a separate graph of `Type` structures linked by `link` (for the “next level,” e.g. the pointed-to type of a pointer) and `down` (for lists of children, e.g. the fields of a struct or the parameters of a function). Composite types like `TIND` (pointer), `TARRAY`, `TFUNC`, and `TSTRUCT`/`TUNION` chain their components through these links. The basic scalar types (`TCHAR` through `TUVLONG`, `TFLOAT`, `TDOUBLE`) are pre-allocated at startup in the global `types` array and shared by reference.

Concretely, the declaration `struct Foo { int x; int y; }; struct Foo *p;` builds the following graph after parsing. I draw `link` as a vertical arrow and `down` as a horizontal one; shared leaves point at the global `types[]` slots:



Note that the struct tag `Foo` lives in `Sym.suetag`, and the fields’ names live in `Type.sym` — two different uses of the symbol table glued onto two different levels of this graph. For `TFUNC` the same layout means `link` is the return type and `down` is the parameter list.

`<enum type_kind 30a>≡` (504b)

```
enum Type_kind
{
```

```
    TXXX,
```

```
    <Type_kind integer cases 30b>
```

```
    <Type_kind float cases 31a>
```

```
    <Type_kind void case 31b>
```

```
    <Type_kind composite cases 31c>
```

```
    <Type_kind other cases 137f>
```

```
    NTYPE,
```

```
};
```

`<Type_kind integer cases 30b>≡` (30a)

```
TCHAR,
```

```
TUCHAR,
```

```
TSHORT,
```

```
TUSHORT,
```

```
TINT,
```

```
TUINT,
```

```
TLONG,
```

```
TULONG,
```

```

TVLONG,
TUVLONG,

⟨Type_kind float cases 31a⟩≡ (30a)
TFLOAT,
TDOUBLE,

⟨Type_kind void case 31b⟩≡ (30a)
TVOID,

⟨Type_kind composite cases 31c⟩≡ (30a)
TIND,
TARRAY,

TFUNC,

TSTRUCT,
TUNION,

TENUM,

⟨struct Type 31d⟩≡ (504b)
struct Type
{
    // enum<type_kind>
    char etype;

    // option<ref_own<Type>, e.g. for '*int' have TIND -link-> TINT
    Type* link;
    // option<list<ref_own<Type>>, next = Type.down, for TFUNC and TSTRUCT
    Type* down;

    ⟨Type qualifier fields 32d⟩

    // -----
    // Post parsing annotations
    // -----
    ⟨Type code generation fields 34b⟩

    ⟨Type other fields 31e⟩
};

⟨Type other fields 31e⟩≡ (31d) 31f▷
Sym* tag;

⟨Type other fields 31f⟩+≡ (31d) ◁31e 438b▷
Sym* sym; // for fields in structures

⟨Node type and storage fields 31g⟩≡ (28) 31h▷
Type* type;

⟨Node type and storage fields 31h⟩+≡ (28) ◁31g 33c▷
// enum<type_kind>, inline of Node.type->etype?
char etype;

⟨constant T 31i⟩≡ (504b)
#define T ((Type*)nil)

```



```

⟨function typ 32a⟩≡ (518c)
    Type*
    typ(int et, Type *d)
    {
        Type *t;

        t = alloc(sizeof(Type));
        t->etype = et;
        t->link = d;

        t->down = T;
        t->garb = GXXX;
        t->sym = S;

        t->width = ewidth[et];

        t->offset = 0;
        t->shift = 0;
        t->nbits = 0;

        return t;
    }

```

Uses GXXX 33a, S 35d, T 31i, and alloc() 500b.

```

⟨global types 32b⟩≡ (511d)
    Type* types[NTYPE];

```

Uses NTYPE 30a.

```

⟨cinit() types initialization 32c⟩≡ (50d)
    types[TXXX] = T;

```

```

    types[TCHAR]    = typ(TCHAR, T);
    types[TUCHAR]   = typ(TUCHAR, T);
    types[TSHORT]   = typ(TSHORT, T);
    types[TUSHORT]  = typ(TUSHORT, T);
    types[TINT]     = typ(TINT, T);
    types[TUINT]    = typ(TUINT, T);
    types[TLONG]    = typ(TLONG, T);
    types[TULONG]   = typ(TULONG, T);
    types[TVLONG]   = typ(TVLONG, T);
    types[TUVLONG]  = typ(TUVLONG, T);

```

```

    types[TFLOAT]   = typ(TFLOAT, T);
    types[TDOUBLE]  = typ(TDOUBLE, T);

```

```

    types[TVOID]    = typ(TVOID, T);

```

```

    types[TENUM]    = typ(TENUM, T);
    types[TFUNC]    = typ(TFUNC, types[TINT]);
    types[TIND]     = typ(TIND, types[TVOID]);

```

Uses T 31i, TCHAR 30b, TDOUBLE 31a, TENUM 31c, TFLOAT 31a, TFUNC 31c, TIND 31c, TINT 30b, TLONG 30b, TSHORT 30b, TUCHAR 30b, TUINT 30b, TULONG 30b, TUSHORT 30b, TUVLONG 30b, TVLONG 30b, TVOID 31b, TXXX 30a, typ() 32a, and types 32b.

```

⟨Type qualifier fields 32d⟩≡ (31d)
    // enum<qualifier>
    char garb;

```

```

⟨enum qualifier 33a⟩≡ (504b)
enum Qualifier
{
    GXXX = 0, // None

    GCONSTNT = 1<<0,
    GVOLATILE = 1<<1,

    NGTYPES = 1<<2,
    ⟨Qualifier other cases 399⟩
};

```

3.3 Storage_class

Storage class is an attribute of the *variable*, not of its type (unlike qualifiers such as `const`). It determines where a variable lives — on the stack (`CAUTO`, `CPARAM`), in a global segment (`CGLOBL`, `CSTATIC`), or resolved at link time (`CEXTERN`). The storage class is first assigned during declaration parsing and may be adjusted later: for example, `extern int x` followed by `int x = 1` in the same file promotes the symbol from `CEXTERN` to `CGLOBL`.

```

⟨enum storage_class 33b⟩≡ (504b)
enum Storage_class
{
    CXXX,

    CAUTO,
    CPARAM,

    CEXTERN,
    CGLOBL,
    CSTATIC,

    ⟨Storage_class cases 137i⟩

    NCTYPES,
};

⟨Node type and storage fields 33c⟩+≡ (28) <31h
// enum<storage_class>
char class;

```

3.4 Node and type attributes

Beyond the tree structure, each `Node` carries attributes that later phases use for code generation. The `addable` field marks whether an expression is an *lvalue* (can appear on the left of an assignment, and therefore has a memory address). The `complex` field estimates how many registers the expression needs, which the code generator uses to decide which subtree to evaluate first. Both fields are set by the type checker's `xcom` pass and consumed by the code generator.

```

⟨Node code generation fields 33d⟩≡ (28) 34a▷
// address-able, used as a bool to mark lvalues, if can assign you can take
// the address of. used by xcom() to assign ‘‘addressability’’.
// (also (ab)used as a bool to mark label uses (true = used in a goto))
char addable;

```

```

(Node code generation fields 34a) += (28) <33d 197d>
    // complexity in number of registers. for register allocation?
    // (also (ab)used as FMX special value)
    // (also (ab)used as a bool to mark label definitions (true = already defined))
    char complex;

(Type code generation fields 34b) = (31d) 277a>
    long width; // ewidth[Type.etype]

```

3.5 Symbols and hash table

The **Sym** structure is the compiler’s universal name-to-information map. A single **Sym** can simultaneously hold data for several C namespaces — identifiers (**type**, **class**, **offset**), struct/union tags (**suetag**), labels (**label**), and even preprocessor macros (**macro**) — because C allows the same spelling to name a variable and a struct tag at the same time. The “value” fields are not in a union; which ones are meaningful depends on the **Decl** entry that put this symbol in scope. All symbols live in a flat hash table (**hash**), and scope nesting is managed by the separate **Decl** stack.

3.5.1 Sym

The **Sym** structure groups all namespace value fields together (not in a union) because the same spelling can simultaneously denote a variable (**type**, **class**, **offset**) and a struct tag (**suetag**, **sueblock**). The lexer also stores keyword token codes in **lexical** and macro bodies in **macro**, overloading this single structure for every kind of named entity in C.

```

(struct Sym 34c) = (504b)
    struct Sym
    {
        // Symbolic names are used for:
        // - identifiers (locals/params/globals, functions, typedefs
        //   and also enum constants)
        // - tags (struct/union/enum)
        // - labels (for the goto)
        // and also during parsing for:
        // - macros (the #define)
        // - keywords lexeme (abuse)

        // -----
        // The "key"
        // -----
        char *name;

        // -----
        // The "value" for the different "namespaces"
        // -----
        (Sym identifier value fields 36b)
        (Sym enum value fields 134a)
        (Sym tag value fields 37d)
        (Sym label value fields 108h)

        (Sym macro value fields 37f)
        (Sym lexeme value fields 37g)

        // -----
        // Extra
        // -----
    }

```

```

    <Sym extra fields 35c>
};

```

3.5.2 hash

The hash table is a simple open-chaining table of 1024 buckets. Each bucket is a linked list of **Sym** nodes chained through **Sym.link**. The hash function multiplies-and-accumulates over the bytes of the name. This table holds *every* symbol ever seen—scope is not managed here but through the separate **Decl** stack.

```

<global hash 35a>≡ (511d)
    // hash<string, ref_own?<Sym>>, (next in bucket = Sym.link)
    Sym* hash[NHASH];

```

Uses **NHASH 35b**.

```

<constant NHASH 35b>≡ (504b)
    #define NHASH 1024

```

```

<Sym extra fields 35c>≡ (34c)
    // list<ref<Sym>> (next = Sym.link) bucket of hashtable 'hash'
    Sym* link;

```

```

<constant S 35d>≡ (504b)
    #define S ((Sym*)nil)

```

```

<cinit() hash initialization 35e>≡ (50d)
    for(i=0; i<NHASH; i++)
        hash[i] = S;

```

Uses **NHASH 35b**, **S 35d**, and **hash 35a**.

3.5.3 slookup() and lookup()

lookup is the core function: it hashes the global **symb** buffer, walks the bucket chain looking for a match, and if none is found, allocates a fresh **Sym** and inserts it at the head of the bucket. This means **lookup** never returns **nil**—it always returns a valid symbol, possibly newly created with default values (**LNAME** lexical class, no type, no storage). **slookup** is a convenience wrapper that copies a string into **symb** and calls **lookup**.

```

<function slookup 35f>≡ (521b)
    Sym*
    slookup(char *s)
    {
        strcpy(symb, s);
        return lookup();
    }

```

Uses **symb 35g**.

```

<global symb 35g>≡ (511d)
    char symb[NSYMB];

```

Uses **NSYMB 35h**.

```

<constant NSYMB 35h>≡ (504b)
    #define NSYMB 1500

```

<function lookup 36a>≡ (521b)

```

Sym*
lookup(void)
{
    Sym *s;
    ulong h;
    char *p;
    int c, n;

    // h = hash(symb)
    h = 0;
    for(p=symb; *p;) {
        h = h * 3;
        h += *p++;
    }
    n = (p - symb) + 1;
    if((long)h < 0)
        h = ~h;
    h %= NHASH;

    c = symb[0];
    for(s = hash[h]; s != S; s = s->link) {
        if(s->name[0] != c)
            continue;
        if(strcmp(s->name, symb) == 0)
            return s;
    }

    // else
    s = alloc(sizeof(Sym));
    s->name = alloc(n);
    memmove(s->name, symb, n);

    strcpy(s->name, symb);

    s->link = hash[h];
    hash[h] = s;

    syminit(s);

    return s;
}

```

Uses NHASH 35b, S 35d, alloc() 500b, hash 35a, symb 35g, and syminit() 37h.

3.5.4 Identifier symbols

When a **Sym** represents an identifier (variable, function, typedef, or enum constant), it carries several value fields: **type** and **class** give the type and storage class, **block** records which scope it belongs to (for detecting redeclarations), **offset** gives its position in the stack frame (for locals and parameters), and **varlineno** records the source location of the declaration (for error messages). When the parser encounters a use of an identifier, it copies these fields from the **Sym** into the **Node**, so the AST is self-contained for code generation even after the scope is reverted.

<Sym identifier value fields 36b>≡ (34c) 37c▷

- <Sym identifier value, type and storage fields 37a>*
- <Sym identifier value, scope fields 37b>*
- <Sym identifier value, checking fields 183a>*
- <Sym identifier value, code generation fields 197c>*

⟨Sym identifier value, type and storage fields 37a⟩≡ (36b)

```
// ref<Type>
Type* type;
// enum<Storage_class>
char class;
```

⟨Sym identifier value, scope fields 37b⟩≡ (36b)

```
ushort block;
```

⟨Sym identifier value fields 37c⟩+≡ (34c) <36b

```
long varlineno;
```

3.5.5 Tag symbols

Tags (struct, union, enum names) live in a separate C namespace, so **struct x** and **int x** can coexist. The **suetag** field points to the **Type** that represents the struct’s definition (its list of fields linked through **Type.link**). For forward declarations, **suetag** initially points to an incomplete type whose **link** is **T**; it gets filled in later when the full definition is parsed. The **sueblock** field tracks the scope depth, analogous to **block** for identifiers.

⟨Sym tag value fields 37d⟩≡ (34c) 37e▷

```
Type* suetag;
```

⟨Sym tag value fields 37e⟩+≡ (34c) <37d

```
ushort sueblock;
```

3.5.6 Other symbols

The remaining **Sym** fields serve two internal purposes. **macro** stores the replacement text for **#defined** macros (with a leading byte encoding the parameter count). **lexical** stores the token code for keywords: at startup, **cinit** inserts all C keywords into the hash table with their token codes, so **lookup** of “if” returns a **Sym** with **lexical == LIF**. Ordinary identifiers get **LNAME** by default from **syminit**.

⟨Sym macro value fields 37f⟩≡ (34c)

```
char* macro;
```

⟨Sym lexeme value fields 37g⟩≡ (34c)

```
// enum<lexeme>
ushort lexical;
```

⟨function syminit 37h⟩≡ (521b)

```
void
syminit(Sym *s)
{
    s->lexical = LNAME;
    s->block = 0;
    s->offset = 0;

    s->type = T;
    s->class = CXXX;
    s->suetag = T;

    s->sig = SIGNONE;
    ⟨syminit() remaining initialisations 183b⟩
}
```

Uses **CXXX** 33b, **LNAME**, **SIGNONE** 394a, and **T** 31i.

3.6 Declarations, namespaces, and scope: Decl

C has three separate namespaces — identifiers, struct/union/enum tags, and labels — and lexical scoping with nested blocks. The `Decl` structure implements both. The global `dclstack` is a stack of `Decl` entries separated by `DMARK` sentinels. When the parser enters a block (`markdcl`), it pushes a `DMARK` and bumps the `blockno` counter. Each new declaration pushes a `DAUTO`, `DSUE`, or `DLABEL` entry that *saves* the symbol's old value fields (type, offset, storage class) before overwriting them. When the block ends (`revert_dcl`), the stack is popped back to the `DMARK`, restoring every shadowed symbol to its previous meaning.

For a program like `int x; void foo(int y) { int x; ... }` the `dclstack` looks like this while the compiler is inside `foo`'s body (top of stack on the left, link pointers run right-to-left):

```
dclstack
|
v
+-----+ +-----+ +-----+ +-----+
| DAUTO |-->| DAUTO |-->| DMARK |-->| DMARK |-->D
| sym=x |   | sym=y |   | block=0|   | block=0|
| old:  |   | old:  |   | autoff|   | autoff|
| type  |   | (S,   |   | =0     |   | =0     |
| =int  |   | -, -) |   +-----+   +-----+
| cls   |   +-----+   ^               ^
| =Auto|   saved default |               |
| off=-4| for new y       |               global scope mark
+-----+               function body mark
^
saved previous (global) x: class=Glob, type=int
```

Each non-mark entry is the *old* value of its symbol, not the new one — the live meaning sits in the `Sym` itself and the stack is purely the undo log. `revert_dcl` walks the stack to the next `DMARK` and copies each saved triple back into its `Sym`, so the inner `x` evaporates and the outer global `x` springs back into view. The `blockno` in each `DMARK` is what lets duplicate-declaration diagnostics know whether a name was introduced in the current block or inherited.

```
<enum dxxx 38a>≡ (504b)
enum Namespace
{
```

```
    DMARK, // special mark to help separate the different scopes

    DAUTO, // locals/parameters/globals/typedefs/functions identifiers
    DSUE,  // struct/union/enum tags
    DLABEL, // labels, goto
};
```

```
<struct Decl 38b>≡ (504b)
struct Decl
{
```

```
    Sym* sym;
    // enum<Namespace>
    short val;

    <Decl sym copy fields 40b>

    // Extra fields
    <Decl extra fields 39b>
};
```

<global dclstack 39a>≡ (511d)
 // list<list<ref_own<Decl>>> (next = Decl.link, nested lists via DMARK)
 Decl* dclstack;

<Decl extra fields 39b>≡ (38b)
 // list<ref_own<Decl> of dclstack
 Decl* link;

<constant D 39c>≡ (504b)
 #define D ((Decl*)nil)

<cinit() dclstack initialization 39d>≡ (50d)
 dclstack = D;

Uses D 39c and dclstack 39a.

<function push 39e>≡ (525)
 Decl*
 push(void)
 {
 Decl *d;

 d = alloc(sizeof(Decl));
 // add_stack(d, dclstack)
 d->link = dclstack;
 dclstack = d;

 return d;
 }

Uses alloc() 500b and dclstack 39a.

<function push1 39f>≡ (525)
 Decl*
 push1(Sym *s)
 {
 Decl *d;

 d = push();
 d->sym = s;
 d->val = DAUTO;

 <push1() save symbol fields in decl 42a>

 return d;
 }

Uses DAUTO 38a and push() 39e.

<global blockno 39g>≡ (511d)
 int blockno;

<global autobn 39h>≡ (511d)
 int autobn;

<cinit() namespace globals initialization 39i>≡ (50d) 196e▷
 blockno = 0;
 autobn = blockno;

Uses autobn 39h and blockno 39g.


```

⟨function markdcl 40a⟩≡ (525)
  //@Scheck: used by cc.y
  void markdcl(void)
  {
    Decl *d;

    blockno++;

    d = push();
    d->val = DMARK;

    // saving current values
    d->block = autobn;
    d->offset = autoffset;

    autobn = blockno;
  }

```

Uses DMARK 38a, autobn 39h, autoffset 196d, blockno 39g, and push() 39e.

```

⟨Decl sym copy fields 40b⟩≡ (38b) 40c▷
  Type* type; // for Sym.type and Sym.suetag
  ushort block; // for Sym.block and Sym.sueblock and autobn

```

```

⟨Decl sym copy fields 40c⟩+≡ (38b) <40b 41e▷
  long offset; // for Sym.offset and autoffset

```

```

⟨function revertdcl 40d⟩≡ (525)
  //@Scheck: used by cc.y
  Node* revertdcl(void)
  {
    Decl *d;
    Sym *s;
    Node *n;
    ⟨revertdcl() other locals 416a⟩

    n = Z;
    for(;;) {
      // d = pop(dclstack)
      d = dclstack;
      ⟨revertdcl() sanity check d 40e⟩
      dclstack = d->link;

      s = d->sym;
      switch(d->val) {
        ⟨revertdcl() switch declaration type cases 41a⟩
      }
    }
    return n;
  }

```

Uses Z 29a and dclstack 39a.

```

⟨revertdcl() sanity check d 40e⟩≡ (40d)
  if(d == D) {
    diag(Z, "pop off dcl stack");
    break;
  }

```

Uses D 39c, Z 29a, and diag() 496b.

```

⟨revertdecl() switch declaration type cases 41a⟩≡ (40d) 41b▷
    case DMARK:
        // restore info previous block
        autobn = d->block;
        autooffset = d->offset;
        // we popped everything, exit loop and return
        return n;

```

Uses DMARK 38a, autobn 39h, and autooffset 196d.

```

⟨revertdecl() switch declaration type cases 41b⟩+≡ (40d) ◁41a 41c▷
    case DSUE:
        ⟨revertdecl() debug revert DSUE 490f⟩

        // restore info from previous tag
        s->suetag = d->type;
        s->sueblock = d->block;

        break;

```

Uses DSUE 38a.

```

⟨revertdecl() switch declaration type cases 41c⟩+≡ (40d) ◁41b 41d▷
    case DLABEL:
        ⟨revertdecl() debug revert DLABEL 490g⟩

        ⟨revertdecl() DLABEL case, warn if label not used 185c⟩
        s->label = Z;
        break;

```

Uses DLABEL 38a and Z 29a.

```

⟨revertdecl() switch declaration type cases 41d⟩+≡ (40d) ◁41c
    case DAUTO:
        ⟨revertdecl() debug revert DAUTO 490h⟩

        ⟨revertdecl() DAUTO case, warn if auto declared but not used 183e⟩
        ⟨revertdecl() if volatile symbol 416b⟩
        // restore info previous identifier
        ⟨revertdecl() DAUTO case, restore symbol fields from decl 41g⟩

        break;

```

Uses DAUTO 38a.

```

⟨Decl sym copy fields 41e⟩+≡ (38b) ◁40c 41f▷
    char class;
    long varlineno;

```

```

⟨Decl sym copy fields 41f⟩+≡ (38b) ◁41e
    bool aused;

```

```

⟨revertdecl() DAUTO case, restore symbol fields from decl 41g⟩≡ (41d)
    s->type = d->type;
    s->offset = d->offset;
    s->block = d->block;

    s->class = d->class;
    s->varlineno = d->varlineno;
    s->aused = d->aused;

```

```

⟨push1() save symbol fields in decl 42a⟩≡ (39f)
    d->type    = s->type;
    d->offset  = s->offset;
    d->block   = s->block;

    d->class   = s->class;
    d->aused   = s->aused;
    d->varlineno = s->varlineno;

```

3.7 Token

The yacc %union defines what values a token can carry. Numeric constants carry their value as `vval` (integer) or `dval` (floating-point). String literals carry a pointer and length in `sval`. Identifiers and keywords carry a `Sym` pointer, which the lexer looks up in the hash table. This is the interface between the lexer and the parser.

```

⟨union yacc 42b⟩≡ (100a)
%union {
    vlong    vval;
    double   dval;
    struct {
        char*  s;
        long   l;
    } sval;
    Sym*      sym;

    ⟨union yacc other fields 101a⟩
}

⟨token declarations 42c⟩≡ (100a) 42d▷
%token <sym>    LNAME LTYPE
%token <vval>   LCONST LUCONST LLCONST LULCONST LVLCONST LUVLCONST
%token <dval>   LFCONST LDCONST
%token <sval>   LSTRING LLSTRING

```

```

⟨token declarations 42d⟩+≡ (100a) ◁42c 184a▷
%token LVOID LCHAR LSHORT LINT LLONG LDOUBLE LFLOAT LSIGNED LUNSIGNED
%token LSTRUCT LUNION LENUM
%token LTYPEDEF
%token LCONSTNT LVOLATILE LRESTRICT LINLINE
%token LAUTO LSTATIC LEXTERN LREGISTER
%token LIF LELSE LWHILE LDO LFOR LBREAK LCONTINUE LRETURN LGOTO
%token LSWITCH LCASE LDEFAULT
%token LSIZEOF

```

3.8 Assembly Instruction output

The compiler’s output is a linked list of `Prog` structures, each representing one ARM instruction (or pseudo-instruction like `TEXT`, `DATA`, `GLOBL`). Each `Prog` has a “from” and “to” `Adr` operand, mirroring the two-address format of Plan 9 assembly. The `lineno` field ties each instruction back to the C source line that generated it, which is essential for the debugger. This is the same `Prog` structure used by the assembler and linker (see `ASSEMBLER` book [Pad15a] and `LINKER` book [Pad15c]).

Compiling `int f(void) { return 1+2; }` produces a chain of `Prog` nodes along `firstp/lastp` that looks like this (recall Plan 9 assembly puts the destination last, so `MOVW $3, R0` moves the constant 3 into R0):

firstp							lastp
v							v
+-----+		+-----+		+-----+		+-----+	
ATEXT	--->	MOVW	--->	MOVW	--->	RET	--->P
from:		from:		from:R0			
sym=f		\$3		to:			
to:		to:R0		8(FP)			
\$framesz							
+-----+		+-----+		+-----+		+-----+	

backpatched after gen() with stkoﬀ + maxargsafe

The list is built head-to-tail by `nextpc`, which appends one `Prog` and bumps the global `pc`. Branches are encoded as forward `Prog*` pointers in `to.offset` (see `patch`), which keeps the list editable all the way through `regopt` and `peep` — there are no numeric addresses to rewrite when an instruction is inserted or removed.

`<struct Prog(arm) 43a>≡` (531b)

```
struct Prog
{
    // enum<Opcode>, from 5.out.h
    char as;

    Adr from;
    Adr to;

    long lineno; // of the corresponding line in the C program!

    <Prog other fields(arm) 43b>

    // Extra
    <Prog extra fields(arm) 193f>
};
```

`<Prog other fields(arm) 43b>≡` (43a) 43c>

```
// option<enum<registr>>, None = R_NONE, see 5.out.h
// abused also to represent TEXT or DATA attributes (e.g. NOPROF)
char reg;
```

`<Prog other fields(arm) 43c>+≡` (43a) <43b

```
// bitset<?>
byte scond;
```

`<struct Adr(arm) 43d>≡` (531b)

```
struct Adr
{
    // enum<Operand_kind> (D_NONE by default)
    char type;

    long offset; // generic value, offset or simple constant
    char sval[NSNAME];
    double dval;
    Ieee ieee;

    <Adr other fields(arm) 44b>
    <Adr type fields(arm) 221e>
};
```

$\langle \text{constant A } 44a \rangle \equiv$ (531b)

```
#define A ((Adr*)nil)
```

$\langle \text{Adr other fields(arm) } 44b \rangle \equiv$ (43d) 44c $\triangleright$

```
// option<enum<registr>> None = R_NONE
char reg;
```

$\langle \text{Adr other fields(arm) } 44c \rangle + \equiv$ (43d) $\triangleleft$ 44b

```
Sym* sym;
// enum<Sym_kind>
char symkind;
```

$\langle \text{global zprog } 44d \rangle \equiv$ (535b)

```
Prog zprog;
```

$\langle \text{ginit() zprog initialisation } 44e \rangle \equiv$ (51c)

```
zprog.link = P;
zprog.as = AGOK;
zprog.reg = R_NONE;
zprog.from.type = D_NONE;
zprog.from.symkind = N_NONE;
zprog.from.reg = R_NONE;
zprog.to = zprog.from;
zprog.scond = COND_ALWAYS;
```

Chapter 4

main()

The compiler is a pipeline that processes one C source file at a time: `main` parses arguments, then `compile` opens the file and calls `yyparse`. Unlike traditional multi-pass compilers, Plan 9's 5c is essentially a one-pass design: code generation happens during parsing, one function at a time via `codgen`, and toplevel globals are emitted by `gclean` at the end.

```
<global thechar 45a>≡ (511d)
    int thechar;
```

```
<global thestring 45b>≡ (511d)
    char* thestring;
```

```
<global outfile 45c>≡ (511d)
    char* outfile = nil;
```

Uses outfile 45c.

```
<function main 45d>≡ (521b)
    //@Scheck: not dead, entry point :)
    void main(int argc, char *argv[])
    {
        int err;
        <main() locals 46a>

        memset(debug, 0, sizeof(debug));
        <main() xxxinit() 46b>
        <main() remaining initialisation 47b>

        ARGBEGIN {
            <main() command line processing 46d>
        } ARGEND

        if(argc < 1 && outfile == nil) {
            print("usage: %cc [-options] files\n", thechar);
            errexit();
        }

        <main() multiple files handling 468b>
        //else

        err = compile((argc == 0) ? "stdin" : argv[0], defs, ndef);

        if(err)
            errexit();
        exits(nil);
    }
```

```

⟨main() locals 46a⟩≡ (45d) 46e▷
    // growing_array<string>
    char **defs = nil;
    int ndef = 0;

```

```

⟨main() xxxinit() 46b⟩≡ (45d)
    tinit(); // type globals initialisation
    cinit(); // C lexing/parsing globals initialisation
    ginit(); // arch dependent globals initialisation, 5c/8c/...
    arginit(); // printf argument checking initialisation

```

```

⟨ginit() thexxx initialisation 46c⟩≡ (51c)
    thechar = '5';
    thestring = "arm";

```

Uses thechar 45a and thestring 45b.

4.1 Arguments processing

Command-line processing uses the ARGBEGIN/ARGEND macros from Plan 9's `libc.h`. The `-o` flag overrides the output file name, `-D` defines preprocessor macros (accumulated in a growable array for replay when compiling multiple files), and `-I` adds include search paths. All other single-character flags toggle entries in the `debug` array—a simple but effective scheme where `debug['S']` enables assembly output, `debug['w']` enables warnings, and so on. There is no `-c` flag as in `gcc`: 5c always produces an object file, never an executable.

```

⟨main() command line processing 46d⟩≡ (45d) 46f▷
    case 'o':
        outfile = ARGF();
        break;

```

```

⟨main() locals 46e⟩+≡ (45d) <46a 46g▷
    char *p;

```

```

⟨main() command line processing 46f⟩+≡ (45d) <46d 47a▷
    case 'D':
        p = ARGF();
        if(p) {
            ⟨main() when -D in command line processing, grow defs array 46h⟩
            defs[ndef++] = p;

            ddefine(p);
        }
        break;

```

```

⟨main() locals 46g⟩+≡ (45d) <46e 47c▷
    char **np;
    int maxdef = 0;

```

```

⟨main() when -D in command line processing, grow defs array 46h⟩≡ (46f)
    // realloc, growing array
    if(ndef >= maxdef){
        maxdef += 50;
        np = alloc(maxdef * sizeof *np);
        if(defs != nil)
            memmove(np, defs, (maxdef - 50) * sizeof *np);
        defs = np;
    }

```

```

⟨main() command line processing 47a⟩+≡ (45d) <46f 47d>
    case 'I':
        p = ARGF();
        if(p)
            setinclude(p);
        break;

⟨main() remaining initialisation 47b⟩≡ (45d) 412b>
    setinclude(".");

⟨main() locals 47c⟩+≡ (45d) <46g 468a>
    int c;

⟨main() command line processing 47d⟩+≡ (45d) <47a
    default:
        c = ARGC();
        if(c >= 0 && c < sizeof(debug))
            debug[c]++;
        break;

```

4.2 compile()

`compile` is the heart of the driver: it sets up include paths, opens the output .5 object file, initializes the I/O stack for the embedded preprocessor, and calls `yyparse` to parse the entire source file. After parsing completes, `gclean` emits `AGLOBL` pseudo-instructions for all toplevel globals and writes the final object file via `outcode`. The function returns `errors`, so `main` can exit with the right status.

```

⟨global outbuf 47e⟩≡ (511d)
    Biobuf outbuf;

⟨global diagbuf 47f⟩≡ (511d)
    Biobuf diagbuf;

⟨compile() locals 47g⟩≡ (47h) 48c>
    char *p;
    fdt ofd;
    static bool first = true;

⟨function compile 47h⟩≡ (521b)
    /// main -> <> (called multiple times if multifile parallel option)
    int
    compile(char *infile, char **defs, int ndef)
    {
        ⟨compile() locals 47g⟩

        ⟨compile() set p to basename(infile) and adjust include 48d⟩

        if(outfile == nil) {
            ⟨compile() set outfile, using p, to basename(infile).thechar 48e⟩
        }

        ⟨compile() setinclude("/thestring/include") or INCLUDE 49b⟩

        if (first)
            Binit(&diagbuf, STDOUT, OWRITE);
        first = false;

        ⟨compile() if writing acid to standard output 400a⟩
    }

```



```

else {
    ofd = mycreat(outfile, 0664);
    <compile() sanity check ofd 48a>
    Binit(&outbuf, ofd, OWRITE);
}

<compile() initialize IO 53a>

<compile() if use ANSI preprocessor 467c>
else {
    if(strcmp(infile, "stdin") == 0)
        newfile(infile, STDIN);
    else
        newfile(infile, -1);
}

// The big call!
yyparse();

if(!debug['a'] && !debug['Z'])
    // Another important call, calls outcode()
    gclean();
return nerrors;
}

```

Uses debug 474a, diagbuf 47f, mycreat() 513b, nerrors 495a, newfile() 53g, outfile 45c, and yyparse().

```

<compile() sanity check ofd 48a>≡ (47h)
if(ofd < 0) {
    diag(Z, "cannot open %s - %r", outfile);
    outfile = nil;
    errexit();
}

```

Uses Z 29a, diag() 496b, errexit() 495b, and outfile 45c.

```

<cinit() lexing globals initialization 48b>≡ (50d) 52a▷
nerrors = 0;

```

Uses nerrors 495a.

```

<compile() locals 48c>+≡ (47h) <47g 49a▷
char ofile[400];

```

```

<compile() set p to basename(infile) and adjust include 48d>≡ (47h)
strcpy(ofile, infile);
p = utfrune(ofile, pathchar());
if(p) {
    *p++ = '\0';
    if(!debug['.'])
        include[0] = strdup(ofile);
} else
    p = ofile;

```

Uses debug 474a, include 79b, and pathchar() 513b.

```

<compile() set outfile, using p, to basename(infile).thechar 48e>≡ (47h)
outfile = p;
if(outfile) {
    if(p = utfrune(outfile, '.'))
        if(p[1] == 'c' && p[2] == 0)
            p[0] = '\0';
    p = utfrune(outfile, 0);
}

```

```

    <compile() adjust p for outfile if acid option 400b>
    <compile() adjust p for outfile if pickle option 407a>
    else {
        p[0] = '.';
        p[1] = thechar;
        p[2] = '\0';
    }
} else
    outfile = "/dev/null";
Uses outfile 45c and thechar 45a.

<compile() locals 49a>+≡ (47h) <48c 467b>
    char incfile[20];

```

```

<compile() setinclude("/thestring/include") or INCLUDE 49b>≡ (47h)
    if(p = getenv("INCLUDE")) {
        setinclude(p);
    } else {
        if(systemtype(Plan9)) {
            sprintf(incfile, "%s/include", thestring);
            setinclude(strdup(incfile));
            setinclude("/sys/include");
        }
    }
}

```

Uses Plan9 503, setinclude() 79e, systemtype() 513b, and thestring 45b.

4.3 gclean()

While `codgen` emits code for each function during parsing, `gclean` runs after `yyparse` returns to emit the globals. It walks the compiler's symbol state, generates `AGLOBL` pseudo-instructions for every global variable, appends a final `AEND` marker, and then calls `outcode` to serialize the entire instruction list into the `.5` object file.

```

<function gclean(arm) 49c>≡ (537b)
    /// main -> compile -> yyparse; <>
    void
    gclean(void)
    {
        <gclean() locals 208d>

        <gclean() sanity check reg 208e>

        <gclean() adjust symstring width 226i>
        <gclean() adjust symrathole width 242f>

        <gclean() generate all AGLOBL pseudo opcodes 270b>
        <gclean() generate last opcode, AEND 194b>

        // generate the whole output file using outbuf global
        outcode();
    }

```

4.4 Initializations

The compiler has four initialization functions, called in order: `tinit` fills the type-related lookup tables (type compatibility matrices, type set bitmasks, debugging name arrays), `cinit` initializes the C language layer (hash

table, symbol table, declaration stack, basic types), `ginit` initializes the architecture-specific backend (zero-valued template `zprog`, special AST nodes, register state), and `arginit` sets up the `printf`-format argument checker.

4.4.1 tinit()

```
⟨function tinit 50a⟩≡ (518c)
  /// main -> <>
  void
  tinit(void)
  {
    int *ip;
    Init *p;

    ⟨tinit() initialise thash 50b⟩
    ⟨tinit() initialise xxxnames debugging arrays 486c⟩
    ⟨tinit() initialise typexxx type sets 149e⟩
    ⟨tinit() initialise tcompat arrays 152a⟩
    ⟨tinit() initialise 32 bits defaults type sets 180a⟩
  }
```

```
⟨tinit() initialise thash 50b⟩≡ (50a)
  for(p=thashinit; p->code >= 0; p++) {
    urk("thash", nelem(thash), p->code);
    thash[p->code] = p->value;
  }
```

Uses thash 396e, thashinit 397a, and urk() 50c.

```
⟨function urk 50c⟩≡ (518c)
  void
  urk(char *name, int max, int i)
  {
    if(i >= max) {
      fprintf(2, "bad tinit: %s %d>=%d\n", name, i, max);
      exits("init");
    }
  }
```

4.4.2 cinit()

```
⟨function cinit 50d⟩≡ (521b)
  /// main -> <>
  void
  cinit(void)
  {
    int i;
    ⟨cinit() locals 60a⟩

    ⟨cinit() lexing globals initialization 48b⟩
    ⟨cinit() memory globals initialization 499e⟩
    ⟨cinit() types initialization 32c⟩
    ⟨cinit() hash initialization 35e⟩
    ⟨cinit() symbol table initialization 60b⟩
    ⟨cinit() namespace globals initialization 39i⟩
    ⟨cinit() dclstack initialization 39d⟩
    ⟨cinit() symstring initialization 226e⟩
    ⟨cinit() nodproto initialization 175c⟩
    ⟨cinit() pathname initialisation from cwd 51b⟩
```

```

    <cinit() fmtinstall 474b>
}

<global pathname 51a>≡ (511d)
char* pathname;

<cinit() pathname initialisation from cwd 51b>≡ (50d)
pathname = allocn(pathname, 0, 100);
if(getwd(pathname, 99) == 0) {
    pathname = allocn(pathname, 100, 900);
    if(getwd(pathname, 999) == 0)
        strcpy(pathname, "/???");
}

```

Uses allocn() 500c and pathname 51a.

4.4.3 ginit()

```

<function ginit(arm) 51c>≡ (537b)
/// main -> <>
void
ginit(void)
{
    <ginit() locals 242d>

    <ginit() thexxx initialisation 46c>
    <ginit() pc initialisation 193b>
    <ginit() misc initialisations 194a>
    <ginit() zprog initialisation 44e>
    <ginit() special nodes initialisation 195b>
    <ginit() com64init initialisation 457a>
    <ginit() reg and resvreg initialisation 208a>
}

```

4.4.4 arginit()

Chapter 5

Input

Input management handles the stack of open files caused by `#include` nesting. The `Io` structures form a stack (`iostack`): when `macinc` processes an `#include`, it pushes a new `Io` with the opened file; when EOF is reached, the current `Io` is popped and lexing resumes in the parent file. The line history (`Hist`) tracks these transitions for debugging.

```
<cinit() lexing globals initialization 52a>+≡ (50d) <48b 53d>
    lineno = 1;
Uses lineno 29f.
```

5.1 Files management: `iostack`

Each open file (or macro expansion) gets an `Io` structure with its own 8 KB read buffer. The `Io` nodes form a stack through their `link` pointers: the current file is always at the top of `iostack`. When a file is exhausted, its `Io` is moved to the `iofree` list for reuse rather than freed—a simple allocator that avoids repeated `alloc` calls during deep `#include` nesting.

```
<struct Io 52b>≡ (504b)
    struct Io
    {
        // option<fdt> (None = -1)
        short f;
        <Io buffer fields 54b>
        // Extra
        <Io extra fields 52d>
    };

<global iostack 52c>≡ (511d)
    // list<ref_own<Io> (next = Io.link)
    Io* iostack;

<Io extra fields 52d>≡ (52b)
    // list<ref_own<Io>> (from = iostack or iofree)
    Io* link;

<constant I 52e>≡ (504b)
    #define I ((Io*)nil)

<global ionext 52f>≡ (511d)
    // option<ref<Io>>
    Io* ionext;
```

`<compile() initialize IO 53a>≡` (47h)

```
// for embedded cpp
newio();
```

Uses `newio()` 53b.

`<function newio 53b>≡` (521b)

```
void
newio(void)
{
    Io *i;

    <newio() allocate a new Io in i or find a free one 53e>
    i->c = 0;
    i->f = -1;
    ionext = i;
}
```

Uses `ionext` 52f.

`<global iofree 53c>≡` (511d)

```
Io* iofree;
```

`<cinit() lexing globals initialization 53d>+≡` (50d) <52a 57e>

```
iostack = I;
iofree = I;
```

Uses `I` 52e, `iofree` 53c, and `iostack` 52c.

`<newio() allocate a new Io in i or find a free one 53e>≡` (53b)

```
static int pushdepth = 0;

i = iofree;
if(i == I) {
    <newio() sanity check depth of macro expansion 53f>
    i = alloc(sizeof(*i));
} else
    // pop(iofree)
    iofree = i->link;
```

Uses `I` 52e, `alloc()` 500b, and `iofree` 53c.

`<newio() sanity check depth of macro expansion 53f>≡` (53e)

```
pushdepth++;
if(pushdepth > 1000) {
    yyerror("macro/io expansion too deep");
    errexit();
}
```

Uses `errexit()` 495b and `yyerror()` 495c.

`<function newfile 53g>≡` (521b)

```
/// main | macinc -> <>
void
newfile(char *s, int f)
{
    Io *i;

    <newfile() debug 489d>

    // add_list(ionext, iostack)
    i = ionext;
    i->link = iostack;
    iostack = i;
```

```

i->f = f;
if(f < 0)
    i->f = open(s, 0);
if(i->f < 0) {
    yyerror("%cc: %r: %s", thechar, s);
    errexit();
}
fi.c = 0;

⟨newfile() call linehist 81b⟩
}

```

Uses `errexit()` 495b, `fi` 54d, `ionext` 52f, `iostack` 52c, `thechar` 45a, and `yyerror()` 495c.

5.2 Buffer management: `fi`

The global `fi` structure is a “cursor” into the current `Io`’s buffer: `fi.p` points to the next character and `fi.c` counts the remaining characters. When `fi.c` reaches zero, `filbuf` refills the buffer from the underlying file descriptor. This two-level scheme (global cursor over per-file buffers) means the hot path—reading the next character—is a single decrement-and-dereference, inlined as the `GETC` macro.

```

⟨constant BUFSIZ 54a⟩≡ (504b)
#define BUFSIZ 8192

```

```

⟨Io buffer fields 54b⟩≡ (52b) 55c▷
char b[BUFSIZ];

```

```

⟨struct Fi 54c⟩≡ (504b)
struct Fi
{
    // ref<char> (target = Io.b)
    char* p;
    // remaining characters in Io.b to read
    int c;
};

```

```

⟨global fi 54d⟩≡ (511d)
struct Fi fi;

```

Uses `Fi` 54c.

5.3 `GETC()`

`GETC` is the compiler’s inner loop: every character passes through it. The macro fast-paths the common case (buffer not empty) as a pointer dereference, falling back to `filbuf` only when the buffer is exhausted. `filbuf` refills from the current file, and on EOF pops the `iostack` to resume reading the parent file—this is how `#include` returns seamlessly to the including file.

```

⟨function GETC 54e⟩≡ (521b)
#define GETC() ((--fi.c < 0)? filbuf(): (*fi.p++ & 0xff))

```

```

⟨constant EOF 54f⟩≡ (521b)
#define EOF (-1)

```

```

⟨function filbuf 55a⟩≡ (521b)
    int
    filbuf(void)
    {
        Io *i;

    loop:
        i = iostack;
        if(i == I)
            return EOF;
        if(i->f < 0)
            goto pop;
        fi.c = read(i->f, i->b, BUFSIZ) - 1;
        if(fi.c < 0) {
            close(i->f);
            ⟨filbuf() when close file, call linehist 81c⟩
            goto pop;
        }
        fi.p = i->b + 1;
        return i->b[0] & 0xff;

    ⟨filbuf() pop 55b⟩
}

```

Uses BUFSIZ 54a, EOF-81 54f, I 52e, fi 54d, and iostack 52c.

```

⟨filbuf() pop 55b⟩≡ (55a)
    pop:
        // pop(iostack)
        iostack = i->link;
        // push(i, iofree)
        i->link = iofree;
        iofree = i;

        // i = top(iostack), the fresh top of the stack input file
        i = iostack;
        if(i == I)
            return EOF;
        // restore file pointers
        fi.p = i->p;
        fi.c = i->c;
        if(--fi.c < 0)
            goto loop;
        return *fi.p++ & 0xff;

```

Uses EOF-81 54f, I 52e, fi 54d, iofree 53c, and iostack 52c.

```

⟨Io buffer fields 55c⟩+≡ (52b) <54b
    // like Fi, saved pointers in Io.b
    char* p;
    short c;

```


Chapter 6

Lexing

The lexer is hand-written rather than generated by `lex`, partly for speed and partly because C’s lexical structure interacts with the preprocessor and the typedef namespace in ways that are awkward to express in a regular-expression tool. The main loop in `yylex` reads characters via `GETC` (which goes through the preprocessor’s macro expansion), classifies them, and returns token codes for the yacc parser.

6.1 Hand-written vs generated lexers

There is a long-running debate in compiler construction about whether the lexer should be hand-written or generated from a specification. The generated option is older and more respectable: `lex` (Lesk, 1975) turns a list of regular expressions into an efficient DFA at build time; its modern descendants—`flex`, `re2c`, `ragel`, OCaml’s `ocamllex`—do the same with various refinements. The generated lexer is compact, provably correct with respect to its grammar, and easy to extend by editing one file. That is exactly how most compiler textbooks tell you to build a lexer.

And yet, almost every production C compiler hand-writes its lexer. `5c` hand-writes it; `gcc`’s `libcpp` hand-writes it; `clang`’s `Lexer.cpp` hand-writes it; `tcc` hand-writes it. The reasons are not primarily about performance, though performance helps: a hand-written lexer can make a lookahead decision based on information only the parser or the symbol table knows. The killer example in C is the typedef trick: after

```
typedef int foo;
```

the identifier `foo` must lex as a type name, not as an ordinary identifier, because `foo *x` is a declaration and not a multiplication. This requires the lexer to consult the symbol table, which a pure-regex DFA cannot do. Other reasons to hand-write include integrating the preprocessor’s macro expansion into `GETC` so the parser never sees the distinction, producing precise diagnostics with source-file and line-column pointers, handling Unicode identifiers with their own rules (`XID_Start`, `XID_Continue`) that regular expressions express awkwardly, and recovering from an invalid character by resuming at a sensible boundary.

Non-C compilers make the same choice for similar reasons. Go’s lexer is hand-written in `cmd/compile/internal/s`. Rust’s is hand-written in `rustc_lexer`. Java’s `javac` and Kotlin’s compilers are hand-written. The generated-lexer approach wins mostly for simple domain-specific languages and configuration files where performance and typedef-style context-sensitivity do not matter—things like `awk`’s action parser, `cpp` directives, or most protocol-message parsers. In `5c`, the hand-written approach pays for itself in the first 200 lines of `yylex` precisely because C’s declaration syntax is context-sensitive and the same character sequence can mean different things depending on what the symbol table says.

6.2 `yylex()`

`yylex` is a large function structured around a `switch` on the first character of each token. Whitespace is skipped

in a loop at 10, the `peekc` lookahead is consumed before calling `GETC`, and multi-character tokens (operators, numbers, strings) branch to labeled code sections (`tnum`, `talph`, `casedot`). The function returns either a single ASCII character (for one-character operators like `*` and `;`) or an `Lxxx` token code for keywords, identifiers, and multi-character operators.

```

⟨function yylex 57a⟩≡ (521b)
  // @Scheck: not dead, called by yyparse
  long yylex(void)
  {
    long c; // Rune?
    ⟨yylex() locals 58b⟩

    ⟨yylex() peekc handling, starting part 58a⟩
10:
    c = GETC();

11:
    ⟨yylex() if unicode character 61a⟩
    if (isspace(c)) {
      ⟨yylex() if c is newline 57b⟩
      // ignore spaces
      goto 10;
    }
    ⟨yylex() before switch, if isxxx 60e⟩
    switch(c) {
      ⟨yylex() switch c cases 58d⟩
    default:
      return c;
    }
    ⟨yylex() peekc handling, ending part 58c⟩
    return c;

    ⟨yylex() labels 60g⟩
  }

```

Uses `GETC`-84 54e.

```

⟨yylex() if c is newline 57b⟩≡ (57a)
  if (c == '\n')
    lineno++;

```

Uses `lineno` 29f.

6.3 Peek, seek, and look ahead

Many C operators require two characters to distinguish (e.g., `<` vs. `<<` vs. `<=`). The lexer handles this with `peekc`: after reading a second character and deciding it does not belong to the current token, it stores it in `peekc` so the next call to `yylex` picks it up without calling `GETC` again. The sentinel value `IGN` means “no character pending.”

```

⟨global peekc 57c⟩≡ (511d)
  int peekc;

```

```

⟨constant IGN 57d⟩≡ (521b)
  #define IGN (-2)

```

```

⟨cinit() lexing globals initialization 57e⟩+≡ (50d) <53d
  peekc = IGN;

```

Uses `IGN`-82 57d and `peekc` 57c.

```

⟨yylex() peekc handling, starting part 58a⟩≡ (57a)
    if(peekc != IGN) {
        c = peekc;
        peekc = IGN;
        goto l1;
    }

```

Uses IGN-82 57d and peekc 57c.

```

⟨yylex() locals 58b⟩≡ (57a) 60c▷
    long c1;

```

```

⟨yylex() peekc handling, ending part 58c⟩≡ (57a)
    peekc = c1;

```

Uses peekc 57c.

```

⟨yylex() switch c cases 58d⟩≡ (57a) 58e▷
    case EOF:
        peekc = EOF;
        return -1; // EOF

```

Uses EOF-81 54f and peekc 57c.

6.4 Comments

Comment handling is embedded in the / case of the lexer's switch. After reading /, the lexer peeks at the next character: * starts a block comment (scan until */), / starts a line comment (scan until newline), = returns the /= token (LDVE), and anything else produces the bare / division operator. The `getr` function is used instead of `GETC` so that Unicode characters inside comments are handled correctly.

```

⟨yylex() switch c cases 58e⟩+≡ (57a) ◁58d 60d▷
    case '/':
        c1 = GETC();
        if(c1 == '*') {
            // '/' '*' read; skip everything until next '*' '/'
            for(;;) {
                c = getr();
                while(c == '*') {
                    c = getr();
                    if(c == '/')
                        goto l0;
                }
                if(c == EOF) {
                    yyerror("eof in comment");
                    errexit();
                }
            }
        }
        if(c1 == '/') {
            // '/' '/' read; skip everything until next '\n'
            for(;;) {
                c = getr();
                if(c == '\n')
                    goto l0;
                if(c == EOF) {
                    yyerror("eof in comment");
                    errexit();
                }
            }
        }
    }

```

```

}
if(c1 == '=')
    return LDVE;
break;

```

Uses EOF-81 54f, GETC-84 54e, LDVE, errexit() 495b, getr() 72a, and yyerror() 495c.

6.5 Keywords and identifiers

Keywords and user identifiers share the same code path. At startup, `cinit` inserts all C keywords into the hash table with their token codes (e.g., `if` → `LIF`, `int` → `LINT` with type `TINT`). When `yylex` encounters an alphabetic token, it looks it up in the hash table: keywords return their pre-set `lexical` field, while ordinary identifiers return `LNAME`. This is also where the `typedef` trick lives: if `lookup` returns a symbol whose storage class is `CTYPEDEF`, the lexer returns `LTYPE` instead of `LNAME`, resolving the classic C parsing ambiguity.

```

(global itab 59)≡ (521b)
struct
{
    char *name;
    // enum<lexeme>
    ushort lexical;
    // option<enum<Type_kind>>, None = 0 (or TXXX)
    ushort type;
} itab[] =
{
    "auto", LAUTO, 0,
    "static", LSTATIC, 0,
    "extern", LEXTERN, 0,
    "register", LREGISTER, 0,

    "const", LCONSTNT, 0,
    "volatile", LVOLATILE, 0,

    "inline", LINLINE, 0,
    "restrict", LRESTRICT, 0,

    "void", LVOID, TVOID,
    "char", LCHAR, TCHAR,
    "int", LINT, TINT,
    "short", LSHORT, TSHORT,
    "long", LLONG, TLONG,
    "float", LFLOAT, TFLOAT,
    "double", LDOUBLE, TDOUBLE,

    "unsigned", LUNSIGNED, 0,
    "signed", LSIGNED, 0,

    "struct", LSTRUCT, 0,
    "union", LUNION, 0,
    "enum", LENUM, 0,

    "typedef", LTYPEDEF, 0,

    "if", LIF, 0,
    "else", LELSE, 0,
    "while", LWHILE, 0,
    "do", LDO, 0,
    "for", LFOR, 0,

```

```

"break", LBREAK, 0,
"continue", LCONTINUE, 0,
"switch", LSWITCH, 0,
"case", LCASE, 0,
"default", LDEFAULT, 0,
"return", LRETURN, 0,
"goto", LGOTO, 0,

"sizeof", LSIZEOF, 0,
⟨itab entries, kenv extensions 184b⟩
0
};

```

Uses LAUTO, LBREAK, LCASE, LCHAR, LCONSTNT, LCONTINUE, LDEFAULT, LDO, LDOUBLE, LELSE, LENUM, LEXTERN, LFLOAT, LFOR, LGOTO, LIF, LINLINE, LINT, LLONG, LREGISTER, LRESTRICT, LRETURN, LSHORT, LSIGNED, LSIZEOF, LSTATIC, LSTRUCT, LSWITCH, LTYPEDEF, LUNION, LUNSIGNED, LVOID, LVOLATILE, LWHILE, TCHAR 30b, TDOUBLE 31a, TFLOAT 31a, TINT 30b, TLONG 30b, TSHORT 30b, TVOID 31b, and __anon_struct_3 59.

```

⟨cinit() locals 60a⟩≡ (50d) 226d▷
Sym *s;

```

```

⟨cinit() symbol table initialization 60b⟩≡ (50d)
for(i=0; itab[i].name; i++) {
    s = slookup(itab[i].name);
    s->lexical = itab[i].lexical;
    if(itab[i].type != 0)
        s->type = types[itab[i].type];
}

```

Uses itab 59, slookup() 35f, and types 32b.

```

⟨yylex() locals 60c⟩+≡ (57a) <58b 60f>
char *cp;

```

```

⟨yylex() switch c cases 60d⟩+≡ (57a) <58e 62b>
case '_':
    cp = symb;
    goto talph;

```

Uses symb 35g.

```

⟨yylex() before switch, if isxxx 60e⟩≡ (57a) 64c▷
if(isalpha(c)) {
    cp = symb;
    ⟨yylex() if L followed by unicode strings or character 73⟩
    goto talph;
}

```

Uses symb 35g.

```

⟨yylex() locals 60f⟩+≡ (57a) <60c 65a>
Sym *s;

```

```

⟨yylex() labels 60g⟩≡ (57a) 64e▷
talph:
/*
 * cp is set to symb and some
 * prefix has been stored
 */
for(;;) {
    ⟨yylex() in talph case, if unicode character 61b⟩
    if(!isalnum(c) && c != '_')
        break;
}

```

```

    *cp++ = c;
    c = GETC();
}
*cp = '\0';
⟨yylex() in talph case, debug symbol 489f⟩
peekc = c;

// the important call!
s = lookup();

⟨yylex() in talph case, if macro symbol 90a⟩

yylval.sym = s;
⟨yylex() alpha case, return LTYPE if typedef symbol s 62a⟩
return s->lexical;

```

Uses GETC-84 54e, peekc 57c, and yylval.

6.5.1 Unicode identifiers

Plan 9 treats all multi-byte UTF-8 runes as alphabetic, so identifiers can contain Unicode characters. The Runeself constant (128) marks the boundary: any byte below it is ASCII and handled normally; any byte at or above it starts a UTF-8 sequence that fullrune reassembles into a complete rune.

```

⟨yylex() if unicode character 61a⟩≡ (57a)
    if(c >= Runeself) {
        /*
         * extension --
         * all multibyte runes are alpha
         */
        cp = symb;
        goto talph;
    }

```

Uses symb 35g.

```

⟨yylex() in talph case, if unicode character 61b⟩≡ (60g)
    if(c >= Runeself) {
        for(c1=0;;) {
            cp[c1++] = c;
            if(fullrune(cp, c1))
                break;
            c = GETC();
        }
        cp += c1;
        c = GETC();
        continue;
    }

```

Uses GETC-84 54e.

6.5.2 Typedef trick

C's grammar is not context-free: the statement `T * x;` is a pointer declaration if `T` is a typedef, but a multiplication expression if `T` is a variable. The standard solution is to have the lexer consult the symbol table: if `lookup` returns a symbol whose class is `CTYPEDEF`, the lexer returns `LTYPE` instead of `LNAME`, and the parser uses

different grammar rules for each. This lexer–parser–symbol-table cooperation is a classic source of complexity in C compilers.

```

<yylex() alpha case, return LTYPE if typedef symbol s 62a>≡
    if(s->class == CTYPEDEF || s->class == CTYPESTR)
        return LTYPE;

```

(60g)

Uses CTYPEDEF 137i, CTYPESTR 434f, and LTYPE.

6.6 Operators

Multi-character operators (`+=`, `>>`, `&&`, etc.) are recognized by a peek-ahead pattern: after reading the first character, the lexer reads one more via `GETC`. If the two-character sequence matches an operator, it returns the corresponding token immediately. Otherwise the second character is saved in `c1` and becomes `peekc` at the end of `yylex`, and the single-character token is returned. Single-character operators like `*` and `&` are returned as their ASCII value, which `yacc` handles directly.

```

<yylex() switch c cases 62b>+≡
    case '*':
        c1 = GETC();
        if(c1 == '=')
            return LMLE;
        break;

```

(57a) <60d 62c>

Uses `GETC`-84 54e and `LMLE`.

```

<yylex() switch c cases 62c>+≡
    case '%':
        c1 = GETC();
        if(c1 == '=')
            return LMDE;
        break;

```

(57a) <62b 62d>

Uses `GETC`-84 54e and `LMDE`.

```

<yylex() switch c cases 62d>+≡
    case '+':
        c1 = GETC();
        if(c1 == '+')
            return LPP;
        if(c1 == '=')
            return LPE;
        break;

```

(57a) <62c 62e>

Uses `GETC`-84 54e, `LPE`, and `LPP`.

```

<yylex() switch c cases 62e>+≡
    case '-':
        c1 = GETC();
        if(c1 == '-')
            return LMM;
        if(c1 == '=')
            return LME;
        if(c1 == '>')
            return LMG;
        break;

```

(57a) <62d 63a>

Uses `GETC`-84 54e, `LME`, `LMG`, and `LMM`.

$\langle \text{yylex}() \text{ switch } c \text{ cases } 63a \rangle + \equiv$ (57a) $\triangleleft 62e \ 63b \triangleright$

```
case '>':
    c1 = GETC();
    if(c1 == '>') {
        c = LRSR;
        c1 = GETC();
        if(c1 == '=')
            return LRSR;
        break;
    }
    if(c1 == '=')
        return LGE;
    break;
```

Uses GETC-84 54e, LGE, LRSR, and LRSR.

$\langle \text{yylex}() \text{ switch } c \text{ cases } 63b \rangle + \equiv$ (57a) $\triangleleft 63a \ 63c \triangleright$

```
case '<':
    c1 = GETC();
    if(c1 == '<') {
        c = LLSR;
        c1 = GETC();
        if(c1 == '=')
            return LLSR;
        break;
    }
    if(c1 == '=')
        return LLE;
    break;
```

Uses GETC-84 54e, LLE, LLSR, and LLSR.

$\langle \text{yylex}() \text{ switch } c \text{ cases } 63c \rangle + \equiv$ (57a) $\triangleleft 63b \ 63d \triangleright$

```
case '=':
    c1 = GETC();
    if(c1 == '=')
        return LEQ;
    break;
```

Uses GETC-84 54e.

$\langle \text{yylex}() \text{ switch } c \text{ cases } 63d \rangle + \equiv$ (57a) $\triangleleft 63c \ 63e \triangleright$

```
case '!':
    c1 = GETC();
    if(c1 == '=')
        return LNE;
    break;
```

Uses GETC-84 54e and LNE.

$\langle \text{yylex}() \text{ switch } c \text{ cases } 63e \rangle + \equiv$ (57a) $\triangleleft 63d \ 64a \triangleright$

```
case '&':
    c1 = GETC();
    if(c1 == '&')
        return LANDAND;
    if(c1 == '=')
        return LANDE;
    break;
```

Uses GETC-84 54e, LANDAND, and LANDE.

`<yylex() switch c cases 64a>+≡ (57a) <63e 64b>`

```
case '|':
    c1 = GETC();
    if(c1 == '|')
        return LOROR;
    if(c1 == '=')
        return LORE;
    break;
```

Uses GETC-84 54e, LORE, and LOROR.

`<yylex() switch c cases 64b>+≡ (57a) <64a 69c>`

```
case '^':
    c1 = GETC();
    if(c1 == '=')
        return LXORE;
    break;
```

Uses GETC-84 54e and LXORE.

6.7 Numbers

Number lexing must determine not just the numeric value but also the C type of the constant: `int`, `unsigned int`, `long`, `vlong`, `float`, or `double`. The suffix letters (U, L, LL, F) and the magnitude of the value both influence the choice. The `Numxxx` bitset tracks which suffixes have been seen, and `convvtox` truncates the value to the width of the chosen type, warning on overflow.

`<yylex() before switch, if isxxx 64c>+≡ (57a) <60e`

```
if(isdigit(c))
    goto tnum;
```

`<enum numxxx 64d>≡ (521b)`

```
enum Numxxx
{
    Numdec = 1<<0,
    <Numxxx cases 68a>
    Numflt = 1<<4,
};
```

`<yylex() labels 64e>+≡ (57a) <60g 65c>`

```
tnum:
    c1 = 0;
    cp = symb;
    if(c != '0') {
        // not an hexa or octal number
        c1 |= Numdec;
        for(;;) {
            *cp++ = c;
            c = GETC();
            if(isdigit(c))
                continue;
            goto dc;
        }
    }
```

`<yylex() in number case, 0xxx handling 67>`

`<yylex() in number case, decimal dc label handling 65b>`

Uses GETC-84 54e, Numdec-85 64d, and symb 35g.

6.7.1 Decimals

After collecting the digit characters of a decimal number into `symb`, `mpatov` converts the string to a `vlong` value. The code then determines the C type by checking suffix flags and magnitude: an `LL` suffix forces `TVLONG`, an `L` forces `TLONG`, a `U` forces the unsigned variant, and if no suffix is present the smallest type that fits is chosen. The result is truncated to the chosen width via `convvtox`, which warns if the value overflows.

```
<yylex() locals 65a>+≡ (57a) <60f 74b>
```

```
    vlong vv;  
    // enum<Type_kind>  
    long t;
```

```
<yylex() in number case, decimal dc label handling 65b>≡ (64e)  
dc:
```

```
    <yylex() in number case, in decimal case, float handling 69b>  
    //Fallthrough  
    <yylex() in number case, in decimal case, ncw suffix label handling 68b>  
    <yylex() in number case, in decimal case, long suffix label handling 68e>  
    *cp = '\0';  
    peekc = c;  
  
    if(mpatov(symb, &yylval.vval))  
        yyerror("overflow in constant");  
  
    vv = yylval.vval;  
    <yylex() in number case, in decimal case, if vlong number 68f>  
    <yylex() in number case, in decimal case, if long number 69a>  
    <yylex() in number case, in decimal case, if unsigned number 68c>  
    c = LCONST;  
    t = TINT;  
    goto nret;
```

Uses `LCONST`, `TINT` 30b, `mpatov()` 66, `peekc` 57c, `symb` 35g, `yyerror()` 495c, and `yylval`.

```
<yylex() labels 65c>+≡ (57a) <64e 69d>
```

```
nret:  
    yylval.vval = convvtox(vv, t);  
    if(yylval.vval != vv){  
        nearln = lineno;  
        warn(Z, "truncated constant: %T %s", types[t], symb);  
    }  
    return c;
```

Uses `Z` 29a, `convvtox()` 65d, `lineno` 29f, `nearln` 29g, `symb` 35g, `types` 32b, `warn()` 142, and `yylval`.

6.7.2 convvtox()

`convvtox` (“convert vlong to type X”) truncates a 64-bit value to the width of a given type, applying sign extension for signed types. This is used everywhere a constant must be narrowed to its declared type — in the lexer (to detect overflow in numeric literals), in the type checker (for casts), and in constant evaluation. `mpatov` handles the string-to-integer conversion for the different bases (decimal, octal, hex).

```
<function convvtox 65d>≡ (524b)
```

```
    vlong  
    convvtox(vlong c, int et)  
{  
    int n;  
  
    n = 8 * ewidth[et];  
    c &= MASK(n);
```

```

    if(!typeu[et])
        if(c & SIGN(n))
            c |= ~MASK(n);
    return c;
}

```

Uses MASK 501f, SIGN 501e, and typeu 231h.

```

⟨function mpatov 66⟩≡ (521b)
/// yylex()#ncu -> <>
/*
 * convert a string, s, to vlong in *v
 * return conversion overflow.
 * required syntax is [0[x]]d*
 */
bool
mpatov(char *s, vlong *v)
{
    vlong n, nn;
    int c;

    n = 0;
    c = *s;
    if(c == '0')
        goto oct;
    while(c = *s++) {
        if(c >= '0' && c <= '9')
            nn = n*10 + c-'0';
        else
            goto bad;
        if(n < 0 && nn >= 0)
            goto bad;
        n = nn;
    }
    goto out;

oct:
    s++;
    c = *s;
    if(c == 'x' || c == 'X')
        goto hex;
    while(c = *s++) {
        if(c >= '0' || c <= '7')
            nn = n*8 + c-'0';
        else
            goto bad;
        if(n < 0 && nn >= 0)
            goto bad;
        n = nn;
    }
    goto out;

hex:
    s++;
    while(c = *s++) {
        if(c >= '0' && c <= '9')
            c += 0-'0';
        else
            if(c >= 'a' && c <= 'f')
                c += 10-'a';
            else

```

```

        if(c >= 'A' && c <= 'F')
            c += 10-'A';
        else
            goto bad;
        nn = n*16 + c;
        if(n < 0 && nn >= 0)
            goto bad;
        n = nn;
    }
out:
    *v = n;
    return false;

bad:
    *v = ~0;
    return true;
}

```

6.7.3 Hexadecimals and octals

Numbers starting with 0 are either octal (0377) or hexadecimal (0xFF). The lexer peeks at the next character to decide: x/X means hex, otherwise octal. Both accumulate into **symb** and then jump to the suffix handling at **ncu** (unsigned/long suffixes) rather than **dc** (the decimal path, which also handles floats).

```

<yylex() in number case, 0xxx handling 67>≡ (64e)
// else, octal or hexadecimal number
*cp++ = c;
c = GETC();
if(c == 'x' || c == 'X')
    for(;;) {
        *cp++ = c;
        c = GETC();
        if(isdigit(c))
            continue;
        if(c >= 'a' && c <= 'f')
            continue;
        if(c >= 'A' && c <= 'F')
            continue;
        if(cp == symb+2)
            yyerror("malformed hex constant");
        goto ncu;
    }
if(c < '0' || c > '7')
    goto dc;
for(;;) {
    if(c >= '0' && c <= '7') {
        *cp++ = c;
        c = GETC();
        continue;
    }
    goto ncu;
}
//Fallthrough

```

Uses GETC-84 54e, symb 35g, and yyerror() 495c.

6.7.4 Unsigned numbers

The U suffix forces the constant to an unsigned type, even if the value would fit in a signed type. The Numuns flag is accumulated via `ncu`: and checked later: if set, or if the value does not fit in a signed `int`, the constant gets `TUINT` (or the appropriate wider unsigned type).

$\langle \text{Numxxx cases 68a} \rangle \equiv$ (64d) 68d▷
`Numuns = 1<<2,`

$\langle \text{yylex() in number case, in decimal case, ncu suffix label handling 68b} \rangle \equiv$ (65b)
`ncu:`
`if((c == 'U' || c == 'u') && !(c1 & Numuns)) {`
`c = GETC();`
`c1 |= Numuns;`
`goto ncu;`
`}`

Uses `GETC`-84 54e and `Numuns`-86 68a.

$\langle \text{yylex() in number case, in decimal case, if unsigned number 68c} \rangle \equiv$ (65b)
`if((c1 & Numuns) || convvtox(vv, TINT) < 0) {`
`c = LUCONST;`
`t = TUINT;`
`goto nret;`
`}`

Uses `LUCONST`, `Numuns`-86 68a, `TINT` 30b, `TUINT` 30b, and `convvtox()` 65d.

6.7.5 Long numbers

The L suffix promotes a constant to `TLONG`, and LL promotes it to `TVLONG` (64 bits). The second L sets `Numvlong`, and the first sets `Numlong`. The type determination cascade checks `vlong` first, then `long`, then unsigned, then plain `int` — picking the smallest type that both fits the value and satisfies the suffix constraints.

$\langle \text{Numxxx cases 68d} \rangle + \equiv$ (64d) ◁68a
`Numlong = 1<<1,`
`Numvlong = 1<<3,`

$\langle \text{yylex() in number case, in decimal case, long suffix label handling 68e} \rangle \equiv$ (65b)
`if((c == 'L' || c == 'l') && !(c1 & Numvlong)) {`
`c = GETC();`
`if(c1 & Numlong)`
`c1 |= Numvlong;`
`c1 |= Numlong;`
`goto ncu;`
`}`

Uses `GETC`-84 54e, `Numlong`-87 68d, and `Numvlong`-88 68d.

$\langle \text{yylex() in number case, in decimal case, if vlong number 68f} \rangle \equiv$ (65b)
`if(c1 & Numvlong) {`
`if((c1 & Numuns) || convvtox(vv, TVLONG) < 0) {`
`c = LUVLCONST;`
`t = TUVLONG;`
`goto nret;`
`}`
`c = LVLCONST;`
`t = TVLONG;`
`goto nret;`
`}`

Uses `LUVLCONST`, `LVLCONST`, `Numuns`-86 68a, `Numvlong`-88 68d, `TUVLONG` 30b, `TVLONG` 30b, and `convvtox()` 65d.

<yylex() in number case, in decimal case, if long number 69a>≡ (65b)

```

if(c1 & Numlong) {
    if((c1 & Numuns) || convvttox(vv, TLONG) < 0) {
        c = LULCONST;
        t = TULONG;
        goto nret;
    }
    c = LLCONST;
    t = TLONG;
    goto nret;
}

```

Uses LLCONST, LULCONST, Numlong-87 68d, Numuns-86 68a, TLONG 30b, TULONG 30b, and convvttox() 65d.

6.7.6 Floats

A decimal number becomes a float if a `.` or `e/E` follows the digit sequence. The lexer collects the entire float text (including exponent and sign) into `symb`, then calls `strtod` to convert it. An `F` suffix produces `LFCONST` (single precision), otherwise `LDCONST` (double). The `casedot` label is shared with the `.` operator case in the main switch — if a `.` is followed by a digit, it is a float, otherwise it is a field-access or struct operator.

<yylex() in number case, in decimal case, float handling 69b>≡ (65b)

```

if(c == '.')
    goto casedot;
if(c == 'e' || c == 'E')
    goto casee;

```

<yylex() switch c cases 69c>+≡ (57a) <64b 70>

```

case '.':
    c1 = GETC();
    if(isdigit(c1)) {
        cp = symb;
        *cp++ = c;
        c = c1;
        c1 = 0;
        goto casedot;
    }
    break;

```

Uses GETC-84 54e and symb 35g.

<yylex() labels 69d>+≡ (57a) <65c

```

casedot:
    for(;;) {
        *cp++ = c;
        c = GETC();
        if(!isdigit(c))
            break;
    }
    if(c != 'e' && c != 'E')
        goto caseout;
    // Fallthrough

casee:
    *cp++ = 'e';
    c = GETC();
    if(c == '+' || c == '-') {
        *cp++ = c;
        c = GETC();
    }

```

```

    if(!isdigit(c))
        yyerror("malformed fp constant exponent");
    while(isdigit(c)) {
        *cp++ = c;
        c = GETC();
    }
    // Fallthrough
caseout:
    if(c == 'L' || c == 'l') {
        c = GETC();
        c1 |= Numlong; // DEAD? used?
    } else
    if(c == 'F' || c == 'f') {
        c = GETC();
        c1 |= Numflt;
    }
    *cp = 0;
    peekc = c;
    yylval.dval = strtod(symb, nil);
    if(isInf(yylval.dval, 1) || isInf(yylval.dval, -1)) {
        yyerror("overflow in float constant");
        yylval.dval = 0;
    }
    if(c1 & Numflt)
        return LFCNST;
    // else
    return LDCNST;

```

Uses GETC-84 54e, LDCNST, LFCNST, Numflt-89 64d, Numlong-87 68d, peekc 57c, symb 35g, yyerror() 495c, and yylval.

6.8 Characters

Character constants are lexed by calling `escchar` to read one character (handling escape sequences like `\n`, `\t`, and hexadecimal/octal escapes). The result is converted to `TCHAR` via `convvtox` and returned as `LCONST`. A second call to `escchar` verifies that the closing quote follows immediately.

```

<yylex() switch c cases 70>+≡ (57a) <69c 74c>
case '\':
    /* ' ' */
    c = escchar('\'', false, false);
    if(c == EOF)
        c = '\';
    c1 = escchar('\'', false, false);
    if(c1 != EOF) {
        yyerror("missing '");
        peekc = c1;
    }

    vv = c;
    yylval.vval = convvtox(vv, TCHAR);
    if(yylval.vval != vv)
        yyerror("overflow in character constant: 0x%lx", c);
    else
    if(c & 0x80){
        nearln = lineno;
        warn(Z, "sign-extended character constant");
    }
    yylval.vval = convvtox(vv, TCHAR);
    return LCONST;

```

Uses EOF-81 54f, LCONST, TCHAR 30b, TUCCHAR 30b, Z 29a, convvtox() 65d, escchar() 71a, lineno 29f, nearln 29g, peekc 57c, warn() 142, yyerror() 495c, and yylval.

6.8.1 Escaping characters

`escchar` reads the next character, handling escape sequences. It takes three parameters: `e` is the closing delimiter (`\` or `"`), `longflg` extends octal escapes to 8 digits (for Unicode strings), and `escflg` ORs `ESC` into numeric escapes (so the string lexer can distinguish literal from escaped characters). The function returns EOF when it sees the closing delimiter, and the caller uses this to detect the end of the character or string literal.

<function escchar 71a>≡ (521b)

```
long
escchar(long e, int longflg, bool escflg)
{
    long c, l;
    int i;

loop:
    c = getr();
    if(c == '\n') {
        yyerror("newline in string");
        return EOF;
    }
    if(c != '\\') {
        if(c == e)
            c = EOF;
        return c;
    }
    // else c is '\\'
    c = getr();

    <escchar() if hexadecimal character 72b>
    <escchar() if octal character 71b>
    switch(c)
    {
        case '\n': goto loop; // escaped newline

        case 'n': return '\n';
        case 't': return '\t';
        case 'b': return '\b';
        case 'r': return '\r';

        case 'f': return '\f';
        case 'a': return '\a';
        case 'v': return '\v';
    }
    return c;
}
```

Uses EOF-81 54f, `getr()` 72a, and `yyerror()` 495c.

<escchar() if octal character 71b>≡ (71a)

```
if(c >= '0' && c <= '7') {
    /*
     * note this is not ansi,
     * supposed to only accept 3 oct
     */
    i = 2;
    if(longflg)
```



```

    i = 8;
    l = c - '0';
    for(; i>0; i--) {
        c = getc();
        if(c >= '0' && c <= '7') {
            l = l*8 + c-'0';
            continue;
        }
        ungetc(c);
    }
    if(escflg)
        l |= ESC;
    return l;
}

```

Uses ESC-83 74a, getc() 78d, and ungetc() 78b.

<function getr 72a>≡ (521b)

```

long
getr(void)
{
    int c, i;
    char str[UTFmax+1];
    Rune rune;

    c = getc();
    if(c < Runeself)
        return c;
    i = 0;
    str[i++] = c;

loop:
    c = getc();
    str[i++] = c;
    if(!fullrune(str, i))
        goto loop;
    c = chartorune(&rune, str);
    if(rune == Runeerror && c == 1) {
        nearln = lineno;
        diag(Z, "illegal rune in string");
        for(c=0; c<i; c++)
            print(" %.2x", *(uchar*)(str+c));
        print("\n");
    }
    return rune;
}

```

Uses Z 29a, diag() 496b, getc() 78d, lineno 29f, and nearln 29g.

<escchar() if hexadecimal character 72b>≡ (71a)

```

if(c == 'x') {
    /*
     * note this is not ansi,
     * supposed to only accept 2 hex
     */
    i = 2;
    if(longflg)
        i = 6;
    l = 0;
    for(; i>0; i--) {
        c = getc();
    }
}

```

```

        if(c >= '0' && c <= '9') {
            l = l*16 + c-'0';
            continue;
        }
        if(c >= 'a' && c <= 'f') {
            l = l*16 + c-'a' + 10;
            continue;
        }
        if(c >= 'A' && c <= 'F') {
            l = l*16 + c-'A' + 10;
            continue;
        }
        unget(c);
        break;
    }
    if(escflg)
        l |= ESC;
    return l;
}

```

Uses ESC-83 74a, `getc()` 78d, and `unget()` 78b.

6.8.2 Unicode characters

The `L` prefix introduces wide character (`L'x'`) and wide string (`L"..."`) literals. The lexer detects this by checking the character after `L`: a single quote goes to the wide character path (using `escchar` with `longflg` true, allowing 8-digit octal escapes for full Unicode), and a double quote goes to the wide string path at `caselq`. If neither follows, `L` is just the start of an identifier.

<yyllex() if L followed by unicode strings or character 73>≡ (60e)

```

    if(c != 'L')
        goto talph;

    *cp++ = c;
    c = GETC();
    if(c == '\\') {
        /* L'x' */
        c = escchar('\\', true, false);
        if(c == EOF)
            c = '\\';
        c1 = escchar('\\', true, false);
        if(c1 != EOF) {
            yyerror("missing '");
            peekc = c1;
        }
        yyval.vval = convvtox(c, TRUNE);
        return LUCONST;
    }
    if(c == '"') {
        goto caselq;
    }
}

```

Uses EOF-81 54f, `GETC`-84 54e, `LUCONST`, `TRUNE` 113c, `convvtox()` 65d, `escchar()` 71a, `peekc` 57c, `yyerror()` 495c, and `yyval`.

6.9 Strings

String literals are collected character by character using `escchar`, which marks escaped bytes with the `ESC` flag. Characters without `ESC` are UTF-8 encoded via `runetochar`, while escaped characters (`\\x41`) are stored as raw

bytes. The result is a dynamically grown buffer (via `alloca`) that holds the string's bytes plus a null terminator. The parser receives a pointer and length in `yylval.sval`.

<constant ESC 74a>≡ (521b)

```
#define ESC (Runemask+1) /* Rune flag: a literal byte */
```

<yylex() locals 74b>+≡ (57a) <65a

```
Rune rune;
```

<yylex() switch c cases 74c>+≡ (57a) <70 74d>

```
case '':
    strcpy(symb, "\"<string>\""); // for error reporting
    cp = alloc(0);
    c1 = 0;

    /* "... " */
    for(;;) {
        c = escchar('\'', false, true);
        if(c == EOF)
            break;
        if(c & ESC) {
            cp = allocn(cp, c1, 1);
            cp[c1++] = c;
        } else {
            rune = c;
            c = runelen(rune);
            cp = allocn(cp, c1, c);
            runetochar(cp+c1, &rune);
            c1 += c;
        }
    }
    yyval.sval.l = c1;
do {
    cp = allocn(cp, c1, 1);
    cp[c1++] = '\0';
} while(c1 & MAXALIGN);
yyval.sval.s = cp;
return LSTRING;
```

Uses EOF-81 54f, ESC-83 74a, LSTRING, MAXALIGN 500a, `alloc()` 500b, `allocn()` 500c, `escchar()` 71a, `symb` 35g, and `yyval`.

6.9.1 Unicode strings

Wide strings (`L"..."`) store each character as a `TRune` (typically 4 bytes) instead of UTF-8. `escchar` is called with `longflg` true so octal escapes can encode full Unicode code points. The result buffer is `TRune`-aligned and null-terminated with a `TRune`-sized zero. The grammar rule for `lstring` handles adjacent wide string concatenation, just like the ASCII string rule does for regular strings.

<yylex() switch c cases 74d>+≡ (57a) <74c 76a>

```
caselq:
    /* L"..." */
    strcpy(symb, "L<string>");
    cp = alloc(0);
    c1 = 0;
    for(;;) {
        c = escchar('\'', true, false);
        if(c == EOF)
            break;
        cp = allocn(cp, c1, sizeof(TRune));
        *(TRune*)(cp + c1) = c;
```

```

        c1 += sizeof(TRune);
    }
    yylval.sval.l = c1;
    do {
        cp = allocn(cp, c1, sizeof(TRune));
        *(TRune*)(cp + c1) = 0;
        c1 += sizeof(TRune);
    } while(c1 & MAXALIGN);
    yylval.sval.s = cp;
    return LLSTRING;

```

Uses EOF-81 54f, LLSTRING, MAXALIGN 500a, alloc() 500b, allocn() 500c, escchar() 71a, symb 35g, and yylval.

Chapter 7

Preprocessing

The C preprocessor is built directly into the compiler rather than being a separate pass. This avoids the overhead of forking a process and writing/reading an intermediate file, contributing to Plan 9's famously fast compilation times. The preprocessor handles `#include`, `#define` (with argument substitution), `#ifdef/#endif`, `#pragma`, and line tracking.

7.1 mactab and macro() dispatch

When `yylex` sees a `#` at the start of a line, it calls `macro`, which reads the directive name via `getsym` and dispatches through the `mactab` table. The `ifdef/ifndef/else` entries have `nil` function pointers and are handled specially by `macif` with a numeric argument (0, 1, or 2).

```
<yylex() switch c cases 76a>+≡ (57a) <74d
    case '#':
        macro();
        goto 10;
```

Uses `macro()` 76c.

```
<global mactab 76b>≡ (514b)
struct
{
    char *macname;
    void (*macf)(void);
} mactab[] =
{
    "ifdef",    nil,    /* macif(0) */
    "ifndef",   nil,    /* macif(1) */
    "else",     nil,    /* macif(2) */
    "endif",    macend,

    "include",  macinc,
    "line",     maclin,
    "define",   macdef,
    "undef",    macund,

    "pragma",   macprag,
    0

};
```

Uses `__anon_struct_2` 76b, `macdef()` 85b, `macend()` 78c, `macinc()` 81f, `maclin()` 96b, `macprag()` 96a, and `macund()` 94a.

```
<function macro 76c>≡ (514b)
void
macro(void)
```

```

{
    Sym *s;
    int i;

    s = getsym();
    <domacro() set s to endif symbol if no symbol 77a>

    for(i=0; mactab[i].macname; i++)
        if(strcmp(s->name, mactab[i].macname) == 0) {
            if(mactab[i].macf)
                // dispatcher!
                (*mactab[i].macf)();
            else
                macif(i);
            return;
        }
    // else
    yyerror("unknown #: %s", s->name);
    macend();
}

```

Uses getsym() 77b, macend() 78c, macif() 94b, mactab 76b, and yyerror() 495c.

```

<domacro() set s to endif symbol if no symbol 77a>≡ (76c)
    if(s == S)
        s = slookup("endif");

```

Uses S 35d and slookup() 35f.

```

<function getsym 77b>≡ (514b)
    Sym*
    getsym(void)
    {
        int c;
        char *cp;

        c = getnsc();
        if(!isalpha(c) && c != '_' && c < Runeself) {
            unget(c);
            return S;
        }
        // else isalpha() or '_'
        for(cp = symb;;) {
            if(cp <= symb+NSYMB-4)
                *cp++ = c;
            c = getc();
            if(isalnum(c) || c == '_' || c >= Runeself)
                continue;
            unget(c);
            break;
        }
        *cp = '\0';
        if(cp > symb+NSYMB-4)
            yyerror("symbol too large: %s", symb);
        return lookup();
    }

```

Uses NSYMB 35h, S 35d, getc() 78d, getnsc() 77c, symb 35g, unget() 78b, and yyerror() 495c.

```

<function getnsc 77c>≡ (521b)
    int
    getnsc(void)

```

```

{
    int c;

    <getnsc() peekc handling 78a>
    else
        c = GETC();

    for(;;) {
        if(c >= Runeself || !isspace(c))
            return c;
        if(c == '\n') {
            lineno++;
            return c;
        }
        // else, was a space, so continue
        c = GETC();
    }
    return -1; // unreachable
}

```

Uses GETC-84 54e and lineno 29f.

```

<getnsc() peekc handling 78a>≡ (77c)
    if(peekc != IGN) {
        c = peekc;
        peekc = IGN;
    }

```

Uses IGN-82 57d and peekc 57c.

```

<function unget 78b>≡ (521b)
    void
    unget(int c)
    {

        peekc = c;
        if(c == '\n')
            lineno--;
    }

```

Uses lineno 29f and peekc 57c.

```

<function macend 78c>≡ (514b)
    /// (domacro | macinc -> yyerror; <>) | (macund -> <>)
    void
    macend(void)
    {
        int c;

        for(;;) {
            c = getnsc();
            if(c < 0 || c == '\n')
                return;
        }
    }

```

Uses getnsc() 77c.

```

<function getc 78d>≡ (521b)
    int
    getc(void)
    {
        int c;

```

```

    <getc() peekc handling 79a>
    else
        c = GETC();

    if(c == '\n')
        lineno++;
    if(c == EOF) {
        yyerror("End of file");
        errexit();
    }
    return c;
}

```

Uses EOF-81 54f, GETC-84 54e, errexit() 495b, lineno 29f, and yyerror() 495c.

```

<getc() peekc handling 79a>≡ (78d)
    if(peekc != IGN) {
        c = peekc;
        peekc = IGN;
    }

```

Uses IGN-82 57d and peekc 57c.

7.2 #include

7.2.1 Include paths, -I

The include paths are stored in a growable array. Entry 0 is special: it is initially set to ".", then overwritten with the directory of the current source file (unless -.) is given). For #include "file", the compiler searches entry 0 first (the source directory), then the remaining paths. For #include <file>, it skips entry 0 and searches only the system directories.

```

<global include 79b>≡ (511d)
    // growing_array<string>
    char** include;

```

```

<global ninclude 79c>≡ (511d)
    int ninclude;

```

```

<global maxinclude 79d>≡ (511d)
    int maxinclude;

```

```

<function setinclude 79e>≡ (521b)
    /// main -> <>(".") | main -> <>
    void
    setinclude(char *p)
    {
        int i;
        char *e, **np;

        while(*p != '\0') {
            e = strchr(p, ' ');
            if(e != nil)
                *e = '\0';

            // already there?
            for(i=0; i < ninclude; i++)
                if(strcmp(p, include[i]) == 0)

```



```

        break;
    // else
    if(i >= ninclude){
        <setinclude() grow the array if necessary 80a>
        include[ninclude++] = p;
    }

    if(e == nil)
        break;
    p = e+1;
}
}

```

Uses include 79b and ninclude 79c.

```

<setinclude() grow the array if necessary 80a>≡ (79e)
// grow the array
if(i >= maxinclude){
    maxinclude += 20;
    np = alloc(maxinclude * sizeof *np);
    if(include != nil)
        memmove(np, include, (maxinclude - 20) * sizeof *np);
    include = np;
}

```

Uses alloc() 500b, include 79b, and maxinclude 79d.

7.2.2 Tracing origin and Hist

Because the preprocessor flattens `#includes` into a single stream, `lineno` is a global counter that keeps incrementing across file boundaries. To report errors in the right file, the compiler maintains a `Hist` linked list that records each “push” (`#include` opens a file) and “pop” (EOF returns to the parent). Given a global line number, the error printer walks this list to reconstruct the “file:line” pair. The `#line` directive also creates `Hist` entries with an explicit offset.

```

<struct Hist 80b>≡ (504b)
struct Hist
{
    // option<ref_own<string> (None = nil = a ‘‘pop’’)
    char* name;

    // global line of this Hist
    long line;
    // 0 for #include, +n for #line, -1 for #pragma lib (ugly)
    long offset;

    // Extra
    <Hist extra fields 80d>
};

```

```

<constant H 80c>≡ (504b)
#define H ((Hist*)nil)

```

```

<Hist extra fields 80d>≡ (80b)
Hist* link;

```

```

<global hist 80e>≡ (511d)
// list<ref_own<Hist>> (next = Hist.link)
Hist* hist;

```

```

⟨global ehist 81a⟩≡ (511d)
    Hist* ehist;

⟨newfile() call linehist 81b⟩≡ (53g)
    linehist(s, 0);
Uses linehist() 81d.

⟨filbuf() when close file, call linehist 81c⟩≡ (55a)
    linehist(0, 0);
Uses linehist() 81d.

⟨function linehist 81d⟩≡ (514b)
    void
    linehist(char *f, int offset)
    {
        Hist *h;

        ⟨linehist() possibly overwrite last line directive 470a⟩
        ⟨linehist() debug 489e⟩

        h = alloc(sizeof(Hist));
        h->name = f;
        h->line = lineno;
        h->offset = offset;

        // add_list(h, hist)
        h->link = H;
        if(ehist == H) {
            hist = h;
            ehist = h;
            return;
        }
        ehist->link = h;
        ehist = h;
    }
Uses H 80c, alloc() 500b, ehist 81a, hist 80e, and lineno 29f.

```

7.2.3 #include

macinc lexes the filename (either <file> or "file"), searches the include path array, opens the file, and pushes it onto the I/O stack via **newfile**. For quoted includes, it searches from **include[0]** (the source file's directory); for angle-bracket includes, it skips **include[0]** and starts from the system paths.

```

⟨constant STRINGSZ 81e⟩≡ (504b)
    #define STRINGSZ 200

⟨function macinc 81f⟩≡ (514b)
    void
    macinc(void)
    {
        char str[STRINGSZ];
        char *hp = str;
        int i;
        fdt f = -1;
        ⟨macinc() other locals 82a⟩

        // lexing
    }

```

```

<macinc() lexing the included filename 82b>

// action

<macinc() find and store the included filename full path in symb 83a>

c = strlen(symb) + 1;

// hp = malloc(c);
while(c & 3)
    c++;
while(nhunk < c)
    gethunk();
hp = hunk;
nhunk -= c;
hunk += c;

memcpy(hp, symb, c);

newio();
pushio();
newfile(hp, f);
return;

bad:
    unget(c);
    yyerror("syntax in #include");
    macend();
}

```

Uses STRINGSZ 81e, gethunk() 499f, hunk 499b, macend() 78c, newfile() 53g, newio() 53b, nhunk 499c, pushio() 83b, symb 35g, unget() 78b, and yyerror() 495c.

```

<macinc() other locals 82a>≡ (81f)
    int c0, c;

```

```

<macinc() lexing the included filename 82b>≡ (81f)
    c0 = getnsc();
    if(c0 != '') {
        c = c0;
        if(c0 != '<')
            goto bad;
        c0 = '>';
    }
    for(;;) {
        c = getc();
        if(c == c0)
            break;
        if(c == '\n')
            goto bad;
        *hp++ = c;
    }
    *hp = '\0';

    c = getcom();
    if(c != '\n')
        goto bad;

```

Uses getc() 78d, getcom() 83c, and getnsc() 77c.

<macinc() find and store the included filename full path in symb 83a>≡ (81f)

```
for(i=0; i<ninclude; i++) {
    if(i == 0 && c0 == '>') // do not look in '.' for system headers
        continue;

    strcpy(symb, include[i]);
    strcat(symb, "/");
    if(strcmp(symb, "./") == 0)
        symb[0] = '\\0';
    strcat(symb, str);

    f = open(symb, 0);
    if(f >= 0)
        break;

}
if(f < 0)
    strcpy(symb, str);
```

Uses include 79b, ninclude 79c, and symb 35g.

<function pushio 83b>≡ (521b)

```
void
pushio(void)
{
    Io *i;

    i = iostack;
    if(i == I) {
        yyerror("botch in pushio");
        errexit();
    }
    i->p = fi.p;
    i->c = fi.c;
}
```

Uses I 52e, errexit() 495b, fi 54d, iostack 52c, and yyerror() 495c.

<function getcom 83c>≡ (514b)

```
int
getcom(void)
{
    int c;

    for(;;) {
        c = getnsc();
        if(c != '/')
            break;
        // else c == '/'
        c = getc();
        <getcom() if // comment 84a>
        if(c != '*')
            break;
        // else c == '*'
        <getcom() when /* comment 84b>
        if(c == '\\n')
            break;
    }
    return c;
}
```

Uses getc() 78d and getnsc() 77c.

```

⟨getcom() if // comment 84a⟩≡ (83c)
    if(c == '/') {
        while(c != '\n')
            c = getc();
        break;
    }

```

Uses `getc()` 78d.

```

⟨getcom() when /* comment 84b⟩≡ (83c)
    c = getc();
    for(;;) {
        if(c == '*') {
            c = getc();
            if(c != '/')
                continue;
            c = getc();
            break;
        }
        if(c == '\n') {
            yyerror("comment across newline");
            break;
        }
        c = getc();
    }

```

Uses `getc()` 78d and `yyerror()` 495c.

7.3 #define

Macros are stored as strings in `Sym.macro`. The first byte encodes the parameter count (0 = object-like, $n + 1$ = function-like with n parameters), with the `VARMAC` bit for variadic macros. In the body, parameter references are encoded as `#a`, `#b`, etc., limiting macros to 25 parameters. During expansion, `macexpand` extracts the actual arguments, then performs textual substitution of these encoded markers. A subtle feature: inside string literals in macro bodies, parameter references are also substituted — this is why 5c does not need a separate stringify operator.

7.3.1 -D

Command-line `-D` definitions are handled by `doddefine`, which splits the argument at `=` (if present) to separate the name from the value, looks up the symbol, and stores the replacement text in `Sym.macro`. A bare `-Dfoo` is equivalent to `-Dfoo=1`. The first byte of the macro string is the parameter count (0 for object-like macros).

```

⟨function doddefine 84c⟩≡ (514b)
    void
    doddefine(char *cp)
    {
        Sym *s;
        char *p;
        long l;

        strcpy(symb, cp);
        p = strchr(symb, '=');
        if(p) {
            *p++ = '\0';
            s = lookup();
            l = strlen(p) + 2; /* +1 null, +1 nargs */

```

```

while(l & 3)
    l++;
while(nhunk < 1)
    gethunk();

*hunk = '\0'; // 0 nargs
strcpy(hunk+1, p);
s->macro = hunk;

hunk += 1;
nhunk -= 1;

} else {
    s = lookup();
    s->macro = "\0001"; /* \000 is nargs */
}
⟨dodefine() debug 489a⟩
}

```

Uses gethunk() 499f, hunk 499b, nhunk 499c, and symb 35g.

7.3.2 #define

macdef parses a macro definition in three steps: it lexes the macro name, lexes the parameter list (if any), then lexes the body. Parameters are limited to 25 (NARG) because they are encoded in the body string as #a, #b, ..., #y — one character per parameter, fitting within the alphabet. Variadic macros (...) are supported via the VARMAC flag and expand __VA_ARGS__ in the body.

⟨constant NARG 85a⟩≡ (514b)
 #define NARG 25

⟨function macdef 85b⟩≡ (514b)

```

void
macdef(void)
{
    Sym *s;
    char *args[NARG]; // parameters
    // option<int> (None = -1), size of parameters
    int n;
    bool dots;
    char *base; // body
    ⟨macdef() other locals 86c⟩

    // lexing
    ⟨macdef lexing the macro name, set s 86b⟩
    ⟨macdef lexing the parameters, set args, n, and dots 86d⟩
    ⟨macdef lexing the body, set base 87c⟩

    // action

    *base = n+1; // nargs (+1 to differentiate #define foo() from #define foo)
    if(dots)
        *base |= VARMAC;
    s->macro = base;
    ⟨macdef() debug 489b⟩
    return;

bad:
    if(s == S)

```

```

        yyerror("syntax in #define");
    else
        yyerror("syntax in #define: %s", s->name);
    macend();
}

```

Uses NARG-77 85a, S 35d, VARMAC-76 86a, macend() 78c, and yyerror() 495c.

```

⟨constant VARMAC 86a⟩≡ (514b)
    #define VARMAC 0x80

```

```

⟨macrodef lexing the macro name, set s 86b⟩≡ (85b)
    s = getsym();
    if(s == S)
        goto bad;
    if(s->macro)
        yyerror("macro redefined: %s", s->name);

```

Uses S 35d, getsym() 77b, and yyerror() 495c.

```

⟨macrodef() other locals 86c⟩≡ (85b) 87b▷
    int c;
    Sym *a;

```

```

⟨macrodef lexing the parameters, set args, n, and dots 86d⟩≡ (85b)
    c = getc();
    n = -1; // no parameter (which is different than 0 argument)
    dots = false;

    if(c == '(') {
        n++; // at least 0 parameters now
        c = getnsc();
        if(c != ')') {
            unget(c);
            for(;;) {
                a = getsymdots(&dots);
                if(a == S)
                    goto bad;
                if(n >= NARG) {
                    yyerror("too many arguments in #define: %s", s->name);
                    goto bad;
                }
                args[n++] = a->name;

                c = getnsc();
                if(c == ')')
                    break;
                if(c != ',' || dots)
                    goto bad;
            }
        }
        c = getc();
    }
    if(isspace(c))
        if(c != '\n')
            c = getnsc();

```

Uses NARG-77 85a, S 35d, getc() 78d, getnsc() 77c, getsymdots() 87a, unget() 78b, and yyerror() 495c.

```

⟨function getsymdots 87a⟩≡ (514b)
Sym*
getsymdots(bool *dots)
{
    int c;
    Sym *s;

    s = getsym();
    if(s != S)
        return s;

    c = getnsc();
    if(c != ' '){
        unget(c);
        return S;
    }
    if(getc() != ' ' || getc() != '.')
        yyerror("bad dots in macro");
    *dots = true;
    return slookup("__VA_ARGS__");
}

```

Uses S 35d, getc() 78d, getnsc() 77c, getsym() 77b, slookup() 35f, unget() 78b, and yyerror() 495c.

```

⟨macrodef() other locals 87b⟩+≡ (85b) <86c 88a>
//option<char> (None = 0, either ' or ")
int ischr; // quote character
int len;

```

```

⟨macrodef lexing the body, set base 87c⟩≡ (85b)
base = hunk;
len = 1;
ischr = 0;

for(;;) {
    ⟨macrodef() when lexing the body, if identifier 88b⟩
    ⟨macrodef() when lexing the body, if string or character or comment 88c⟩
    ⟨macrodef() when lexing the body, if antislash outside a string 89a⟩
    if(c == '\n')
        break;
    ⟨macrodef() when lexing the body, if sharp character, escape it 89b⟩

    // else, a regular character (space, punctuation, etc)
    base = allocn(base, len, 1);
    base[len++] = c;

    // GETC
    c = ((--fi.c < 0)? filbuf(): (*fi.p++ & 0xff));
    if(c == '\n')
        lineno++;
    if(c == -1) {
        yyerror("eof in a macro: %s", s->name);
        break;
    }
}
do {
    base = allocn(base, len, 1);
    base[len++] = '\0';
} while(len & 3);

```

Uses allocn() 500c, fi 54d, filbuf() 55a, hunk 499b, lineno 29f, and yyerror() 495c.

`<macrodef() other locals 88a>+≡ (85b) <87b`

```
char *np;
int i;
```

`<macrodef() when lexing the body, if identifier 88b>≡ (87c)`

```
// identifier
if(isalpha(c) || c == '_') {
    np = symb;
    *np++ = c;
    c = getc();
    while(isalnum(c) || c == '_') {
        *np++ = c;
        c = getc();
    }
    *np = '\0';

    for(i=0; i<n; i++)
        if(strcmp(symb, args[i]) == 0)
            break;
    if(i >= n) {
        i = strlen(symb);
        base = allocn(base, len, i);
        memcpy(base+len, symb, i);
        len += i;
        continue;
    }
    // else
    base = allocn(base, len, 2);
    base[len++] = '#';
    base[len++] = 'a' + i;
    continue;
}
```

Uses allocn() 500c, getc() 78d, and symb 35g.

`<macrodef() when lexing the body, if string or character or comment 88c>≡ (87c)`

```
if(ischr){
    // inside a string or character
    if(c == '\\'){
        base = allocn(base, len, 1);
        base[len++] = c;
        c = getc();
    }else if(c == ischr)
        ischr = 0;
}else{
    // string or character
    if(c == '"' || c == '\\'){
        base = allocn(base, len, 1);
        base[len++] = c;
        ischr = c;
        c = getc();
        continue;
    }
    <macrodef() if comment 88d>
}
```

Uses allocn() 500c and getc() 78d.

`<macrodef() if comment 88d>≡ (88c)`

```
// comment
if(c == '/') {
    c = getc();
```

```

if(c == '/') {
    // // comment
    c = getc();
    for(;;) {
        if(c == '\n')
            break;
        c = getc();
    }
    continue;
}
if(c == '*') {
    // /* comment
    c = getc();
    for(;;) {
        if(c == '*') {
            c = getc();
            if(c != '/')
                continue;
            c = getc();
            break;
        }
        if(c == '\n') {
            yyerror("comment and newline in define: %s", s->name);
            break;
        }
        c = getc();
    }
    continue;
}
base = allocn(base, len, 1);
base[len++] = '/';
continue;
}

```

Uses allocn() 500c, getc() 78d, and yyerror() 495c.

<macrodef() when lexing the body, if antislash outside a string 89a> ≡ (87c)

```

// antislash outside a string
if(c == '\\') {
    c = getc();
    if(c == '\n') {
        c = getc();
        continue;
    }
    // else
    base = allocn(base, len, 1);
    base[len++] = '\\';
    continue;
}

```

Uses allocn() 500c and getc() 78d.

<macrodef() when lexing the body, if sharp character, escape it 89b> ≡ (87c)

```

if(c == '#') // escape # by adding an extra #
    if(n > 0) {
        base = allocn(base, len, 1);
        base[len++] = c;
    }

```

Uses allocn() 500c.

7.3.3 Macro Expansion

Macro expansion reuses the I/O stack: `macexpand` writes the expanded text into a new Io buffer, which is then pushed onto `iostack`. The lexer's next `GETC` calls will read from this buffer, and when it is exhausted, `filbuf` pops it and resumes the previous input. This naturally handles nested macros: if the expanded text contains another macro name, `yylex` will call `macexpand` again, pushing yet another Io buffer. The `pushdepth` limit prevents runaway recursion.

```
<yylex() in talph case, if macro symbol 90a>≡ (60g)
    if(s->macro) {
        newio();
        cp = ionext->b;
        macexpand(s, cp);

        pushio();
        // push_list(ionext, iostack)
        ionext->link = iostack;
        iostack = ionext;

        fi.p = cp;
        fi.c = strlen(cp);

        <yylex() in talph case, when macro symbol, peekc handling 90b>
        goto 10;
    }
```

Uses `fi` 54d, `ionext` 52f, `iostack` 52c, `macexpand()` 90c, `newio()` 53b, and `pushio()` 83b.

```
<yylex() in talph case, when macro symbol, peekc handling 90b>≡ (90a)
    if(peekc != IGN) {
        cp[fi.c++] = peekc;
        cp[fi.c] = '\0';
        peekc = IGN;
    }
```

Uses `IGN`-82 57d, `fi` 54d, and `peekc` 57c.

`macexpand()`

`macexpand` performs textual macro expansion into a buffer `b`. For object-like macros (no parameters), it simply copies the body. For function-like macros, it first extracts the actual arguments by reading characters and tracking parenthesis depth, then walks the body and replaces each `#x` marker with the corresponding argument text. The result is written into an Io buffer and pushed onto the I/O stack, so the lexer reads the expansion next.

```
<function macexpand 90c>≡ (514b)
    void
    macexpand(Sym *s, char *b)
    {
        int  nargs;
        bool dots;
        int  c;
        <macexpand() other locals 91a>

        // if macro has no parameter
        if(*s->macro == '\0') {
            strcpy(b, s->macro+1);
            <macexpand() debug macro expansion 489c>
            return;
        }
```

```

// else, macro has parameters

nargs = (char)(*s->macro & ~VARMAC) - 1;
dots = *s->macro & VARMAC;

c = getnsc();
if(c != '(')
    goto bad;

⟨macexpand() parsing the arguments 91b⟩

⟨macexpand() substituting the arguments 93b⟩

⟨macexpand() debug macro expansion 489c⟩

return;

bad:
yyerror("syntax in macro expansion: %s", s->name);
*b = '\\0';
return;

toobig:
yyerror("too much text in macro expansion: %s", s->name);
*b = '\\0';
}

```

Uses VARMAC-76 86a, getnsc() 77c, and yyerror() 495c.

Extracting the arguments

Arguments are extracted by reading characters between the opening ‘(‘ and closing ‘)’, splitting at commas while respecting nested parenthesis depth (1). Each argument is stored as a null-terminated string in `buf`, with `arg[n]` pointing to the start. For variadic macros, all remaining arguments after the last named parameter are concatenated into `__VA_ARGS__`.

```

⟨macexpand() other locals 91a⟩≡ (90c)
char buf[2000];
int l; // depth
char *arg[NARG]; // the arguments
int n;
char *cp, *ecp;
char *ob = b;

```

Uses NARG-77 85a.

```

⟨macexpand() parsing the arguments 91b⟩≡ (90c)
n = 0;
c = getc();
if(c != ')') {
    ungetc(c);

    l = 0;
    cp = buf;
    ecp = cp + sizeof(buf)-4;
    arg[n++] = cp;

    for(;;) {
        if(cp >= ecp)
            goto toobig;
    }
}

```

```

c = getc();

⟨macexpand() when parsing the arguments, if string 92a⟩
⟨macexpand() when parsing the arguments, if character 92b⟩
⟨macexpand() when parsing the arguments, if comment 93a⟩
if(l == 0) {
    if(c == ',') {
        if(n == nargs && dots) {
            *cp++ = ',';
            continue;
        }
        *cp++ = '\\0';
        arg[n++] = cp;
        if(n > nargs)
            break;
        continue;
    }
    if(c == ')')
        break;
}
if(c == '\\n')
    c = ' ';
*cp++ = c;
if(c == '(')
    l++;
if(c == ')')
    l--;
}
*cp = '\\0';
}
if(n != nargs) {
    yyerror("argument mismatch expanding: %s", s->name);
    *b = '\\0';
    return;
}

```

Uses `getc()` 78d, `ungetc()` 78b, and `yyerror()` 495c.

```

⟨macexpand() when parsing the arguments, if string 92a⟩≡ (91b)
// string argument
if(c == '"')
    for(;;) {
        if(cp >= ecp)
            goto toobig;
        *cp++ = c;
        c = getc();
        if(c == '\\') {
            *cp++ = c;
            c = getc();
            continue;
        }
        if(c == '\\n')
            goto bad;
        if(c == '"')
            break;
    }

```

Uses `getc()` 78d.

```

⟨macexpand() when parsing the arguments, if character 92b⟩≡ (91b)
// quote argument
if(c == '\\')

```

```

for(;;) {
    if(cp >= ecp)
        goto toobig;
    *cp++ = c;
    c = getc();
    if(c == '\\') {
        *cp++ = c;
        c = getc();
        continue;
    }
    if(c == '\\n')
        goto bad;
    if(c == '\\')
        break;
}

```

Uses `getc()` 78d.

<macexpand() when parsing the arguments, if comment 93a>≡ (91b)

```

// comment
if(c == '/') {
    c = getc();
    switch(c) {
    case '*':
        for(;;) {
            c = getc();
            if(c == '*') {
                c = getc();
                if(c == '/')
                    break;
            }
        }
        *cp++ = ' '; // ???
        continue;
    case '/':
        while((c = getc()) != '\\n')
            ;
        break;
    default:
        unget(c);
        c = '/';
    }
}

```

Uses `getc()` 78d and `unget()` 78b.

Substituting the arguments

The substitution loop walks the macro body string character by character. Plain characters are copied directly. When `#` is encountered, the next character is either another `#` (an escaped literal hash) or a letter `a–y` indicating which argument to substitute. The argument text is copied inline via `strcpy`, building the complete expanded string in the output buffer `b`.

<macexpand() substituting the arguments 93b>≡ (90c)

```

cp = s->macro+1;
for(;;) {
    c = *cp++;
    if(c == '\\n') // can happen?
        c = ' ';
    if(c != '#') {

```

```

        *b++ = c;
        if(c == '\\0')
            break;
        continue;
    }
    // else, a #
    c = *cp++;
    if(c == '\\0')
        goto bad;
    if(c == '#') { // # are escaped as double ##
        *b++ = c;
        continue;
    }
    // else, a #a to #z
    c -= 'a';
    if(c < 0 || c >= n)
        continue;
    strcpy(b, arg[c]);
    b += strlen(arg[c]);
}
*b = '\\0';

```

7.4 #undef

`#undef` simply sets `Sym.macro` to `nil`, making the symbol invisible to macro expansion. `macend` consumes the rest of the line (comments, trailing whitespace).

```

⟨function macund 94a⟩≡ (514b)
void
macund(void)
{
    Sym *s;

    // lexing
    s = getsym();
    macend();
    if(s == S) {
        yyerror("syntax in #undef");
        return;
    }

    // action
    s->macro = nil;
}

```

Uses S 35d, `getsym()` 77b, `macend()` 78c, and `yyerror()` 495c.

7.5 #ifdef

Conditional compilation is handled by `macif`, which uses a numeric argument to distinguish `#ifdef` (0), `#ifndef` (1), and `#else` (2). When a condition is false, the skip loop reads characters without interpretation, tracking the nesting depth of `#ifdef`/`#endif` pairs so it can find the matching `#endif` or `#else`. Note that 5c does not support the full `#if` constant-expression form — only `#ifdef`/`#ifndef`.

```

⟨function macif 94b⟩≡ (514b)
void
macif(int f)

```

```

{
    Sym *s;
    <macif() other locals 95b>

    if(f == 2) // else
        goto skip;

    // lexing

    <macif() lexing the symbol 95a>

    if((s->macro != nil) ^ f)
        return;
    // else

    // action
    <macif() skipping text 95c>

bad:
    yyerror("syntax in #if(n)def");
    macend();
}

```

Uses macend() 78c and yyerror() 495c.

```

<macif() lexing the symbol 95a>≡ (94b)
    s = getsym();
    if(s == S)
        goto bad;
    if(getcom() != '\n')
        goto bad;

```

Uses S 35d, getcom() 83c, and getsym() 77b.

```

<macif() other locals 95b>≡ (94b)
    int c;
    int l; // depth
    bool bol; // beginning of line

```

```

<macif() skipping text 95c>≡ (94b)
    skip:
        bol = true;
        l = 0;

        for(;;) {
            c = getc();
            if(c != '#') {
                if(!isspace(c))
                    bol = false;
                if(c == '\n')
                    bol = true;
                continue;
            }
            // else a #xxx
            if(!bol)
                continue;

            s = getsym();
            if(s == S)
                continue;
            if(strcmp(s->name, "endif") == 0) {
                if(l) {

```



```

        l--;
        continue;
    }
    // else l == 0
    macend();
    return;
}
if(strcmp(s->name, "ifdef") == 0 || strcmp(s->name, "ifndef") == 0) {
    l++;
    continue;
}
if(l == 0 && f != 2 && strcmp(s->name, "else") == 0) {
    macend();
    return;
}
}
}

```

Uses S 35d, `getc()` 78d, `getsym()` 77b, and `macend()` 78c.

7.6 #pragma

`#pragma` is Plan 9's extension point. The compiler recognizes several pragmas: `lib` for automatic library linking, `pack` for structure packing, `profile` to disable profiling for a function, `varargck` for format-string checking, and `incomplete` to suppress warnings about incomplete struct definitions. Unknown pragmas are silently ignored.

```

⟨function macprag 96a⟩≡ (514b)
void
macprag(void)
{
    Sym *s;
    ⟨macprag() other locals 393a⟩

    // lexing

    s = getsym();

    ⟨macprag() if pragma xxx 393b⟩
    else {
        while(getnsc() != '\n')
            ;
        return;
    }
}

```

Uses `getnsc()` 77c and `getsym()` 77b.

7.7 #line

`#line` directives let tools like `yacc` map errors back to the original source file. `maclin` parses the line number and optional filename, then calls `linehist` with the offset so the error reporter can reconstruct the correct file and line.

```

⟨function maclin 96b⟩≡ (514b)
void
maclin(void)
{
    char *cp;
    int c;

```

```

long n;

// lexing

n = getnsn();
c = getc();
if(n < 0)
    goto bad;

for(;;) {
    if(c == ' ' || c == '\t') {
        c = getc();
        continue;
    }
    if(c == '"')
        break;
    if(c == '\n') {
        strcpy(symb, "<noname>");
        goto nn;
    }
    goto bad;
}
cp = symb;
for(;;) {
    c = getc();
    if(c == '"')
        break;
    *cp++ = c;
}
*cp = '\0';
c = getcom();
if(c != '\n')
    goto bad;

// action

nn:
    c = strlen(symb) + 1;
    while(c & 3)
        c++;

    while(nhunk < c)
        gethunk();
    cp = hunk;
    memcpy(hunk, symb, c);
    nhunk -= c;
    hunk += c;

    linehist(cp, n);
    return;

bad:
    unget(c);
    yyerror("syntax in #line");
    macend();
}

Uses getc() 78d, getcom() 83c, gethunk() 499f, getnsn() 97, hunk 499b, linehist() 81d, macend() 78c, nhunk 499c, symb 35g,
    unget() 78b, and yyerror() 495c.

⟨function getnsn 97⟩≡

```

(514b)

```

long
getnsn(void)
{
    long n;
    int c;

    c = getnsc();
    if(c < '0' || c > '9')
        return -1;
    n = 0;
    while(c >= '0' && c <= '9') {
        n = n*10 + c-'0';
        c = getc();
    }
    ungetc(c);
    return n;
}

```

Uses `getc()` 78d, `getnsc()` 77c, and `ungetc()` 78b.

Chapter 8

Parsing

The parser is a yacc grammar that builds an AST while simultaneously performing symbol resolution, typedef expansion, and partial type checking—there is no clean separation between parsing and semantic analysis. This one-pass approach is fast but makes the code harder to follow than a multi-pass compiler. C’s declaration syntax is notoriously complex (types are split across multiple grammar rules), and this chapter reflects that complexity.

8.1 Generated parser vs hand-written recursive descent

The Lexing chapter had a similar discussion for lexers; the same question arises for parsers, and the industry’s answer has shifted over the decades. 5c uses *yacc* (Stephen Johnson, Bell Labs, 1975), which reads a BNF-style grammar specification and generates an LALR(1) parser—a table-driven automaton that recognizes the grammar by shifting tokens onto a stack and reducing them when a grammar rule is complete. The advantage is that the grammar is an explicit, declarative artifact: you can read it, reason about it, and prove properties about it independently of the action code. This was the dominant approach from the 1970s through the 1990s, and it is how the Dragon Book and most textbooks still teach parsing.

And yet, like hand-written lexers, *hand-written recursive-descent parsers* have won the modern mainstream. GCC moved from a yacc grammar to hand-written recursive descent in 2004 (GCC 4.0); Clang was hand-written from the start (2007); Rust’s `rustc`, Go’s `cmd/compile`, Swift’s parser, and TypeScript’s parser are all hand-written. The reasons parallel the lexer case: better *error recovery* (a recursive-descent parser can skip to a synchronization point and resume, while a yacc parser’s error productions are notoriously hard to get right), better *diagnostics* (the parser knows where it is in the grammar and can say “expected a type name after `struct`” instead of “syntax error”), and easier *incremental parsing* for IDEs. The price is that the grammar is implicit in the code rather than declared in a file, so it is harder to verify that the parser matches the language specification. Interestingly, the generated-parser idea is making a comeback for a different audience: tree-sitter (the parser behind syntax highlighting in VS Code and Neovim) takes a grammar written in JavaScript, generates a large C parser using GLR (Generalized LR) technology, and adds robust error recovery and *incremental* re-parsing (only the changed region is re-parsed on each keystroke). It is, in a sense, yacc’s grandchild—fixing the two weaknesses (poor error recovery, no incrementalness) that drove compilers to hand-written parsers in the first place.

5c’s use of yacc is a historical choice that works well for C: the grammar is complex enough that having it in one file (`cc.y`) is a genuine aid to reading, and yacc’s LALR(1) table catches ambiguities that a hand-written parser might silently misparse. The trade-off is the error messages: when 5c hits a syntax error, yacc produces the generic “syntax error” and the recovery is minimal, whereas Clang’s hand-written parser can often point to the exact token and suggest a fix. For a compiler whose primary user is the Plan 9 build system (not a human typing interactively), the trade-off was acceptable.

8.2 Single-pass compilation

`5c` is a single-pass compiler in the `pcc/lcc` lineage. Each function is lexed, preprocessed, parsed, checked, code-generated, and written to the object file before the next function is read. The compiler never holds more than one function’s worth of AST in memory at a time, and the top-level `prog` rule in the grammar never returns an AST for the whole translation unit—it just loops on external declarations, handing each one off to the back end as soon as the closing brace is parsed.

This is how every C compiler worked before the late 1980s, for one very practical reason: machines did not have enough RAM to hold an entire translation unit’s AST plus all the symbol tables. `pcc` (Steve Johnson’s Portable C Compiler, 1978) on the PDP-11 had to fit in 64 KB of text and data combined; holding more than a function was simply not an option. Ritchie’s original C compiler worked the same way. `lcc` (Fraser and Hanson, 1995) kept the design on purpose as a teaching and research vehicle because the whole compiler then fits in a 300-page book. `gcc 1.x` was single-pass too; the move to holding the whole translation unit came later and was what enabled the GIMPLE / RTL / SSA pipelines we saw in the previous chapter-intro section. Modern compilers—`clang`, `rustc`, `Swift`, `ghc`—all hold the full AST plus multiple IRs and run many passes over them.

The single-pass approach has two visible consequences in `5c`. First, code generation is triggered during the grammar rule for a function definition, not after the parser returns—you will see the `codgen()` call right inside `cc.y`. Second, optimizations that need information from later in the file are simply impossible: `5c` cannot inline a function defined after the call site, cannot check whether a global is ever read before generating its initializer, and cannot decide register assignments based on whole-program use counts. Whole classes of modern optimization (link-time optimization, profile-guided optimization, whole-program devirtualization) have no place to live in a single-pass compiler and are left to the linker—and `5l` does in fact do a few of them, as covered in `LINKER` book [Pad15c]. The trade, as with the no-IR choice, is compile speed and small code size for peak generated-code performance, and the trade made sense for Plan 9.

8.3 Overview

The grammar is a standard yacc `.y` file with rules organized into five groups: declarators, statements, expressions, initializers, and types/storage classes. The top-level rule `prog` is just a list of `xdecl` (external declarations)—and since `5c` triggers code generation after each function body is parsed, `prog` never returns an AST for the whole file.

The priority and associativity declarations encode C’s 15-level operator precedence: `*` binds tighter than `+`, which binds tighter than `<<`, and so on. The yacc union carries either a numeric value (for constants), a `Sym` pointer (for identifiers), or a `Node` pointer (for AST subtrees).

```
<cc/cc.y 100a>≡
%{
#include "cc.h"
%}
<union yacc 42b>
<token declarations 42c>
<priority and associativity declarations 100b>
<type declarations 101b>
%%
<grammar 101c>
%%

<priority and associativity declarations 100b>≡ (100a)
%left  ';;'
%left  ' ','
%right '=' LPE LME LMLE LDVE LMDE LRSHE LLSHE LANDE LXORE LORE
%right '? ' ':'
```

```

%left  LOROR
%left  LANDAND
%left  '| '
%left  '^ '
%left  '& '
%left  LEQ LNE
%left  '<' '>' LLE LGE
%left  LLSH LRSH
%left  '+' '-'
%left  '*' '/' '%'

%right LMM LPP LMG '.' '[' '('

⟨union yacc other fields 101a⟩≡ (42b) 122b▷
Node* node;

⟨type declarations 101b⟩≡ (100a) 104a▷
%type <node> name tag
%type <node> stmtnt block slist ulstmtnt labels label forexpr
%type <node> expr cexpr lexpr xuexpr uexpr pexpr elist string lstring
%type <node> adec1 qual init ilist adlist qlist arglist
%type <node> xdecor xdecor2 edecor abdecor abdecor1 abdecor2 abdecor3
%type <node> zexpr zelist zcexpr zarglist

⟨grammar 101c⟩≡ (100a)
prog:
    /* empty */
    | prog xdecl

⟨declarator rules 101d⟩
⟨statements rules 105e⟩
⟨expressions rules 110a⟩
⟨initializers rules 119f⟩
⟨types, storage classes, qualifiers rules 122c⟩
⟨names rules 103e⟩

⟨extra grammar rules 102e⟩
⟨ebnf grammar rules 102d⟩

```

8.4 Declarations and definitions, part one

C declaration syntax splits a type across two places: the “base type” (e.g., `int`, `struct Foo`) comes first, then the “declarator” wraps it in pointers, arrays, and function signatures (e.g., `*f(int)`). The parser handles this in two phases: first the type keywords are collected into `lasttype/lastclass`, then `dodecl` is called for each declarator to build the final `Type` and install it in the symbol table. The grammar has four kinds of declarators: external (globals/functions), automatic (locals), parameter, and abstract (used in casts and `sizeof`).

```

⟨declarator rules 101d⟩≡ (101c)
⟨external declarator rules 101e⟩
⟨automatic declarator rules 140a⟩
⟨parameter declarator rules 135h⟩
⟨abstract declarator rules 141a⟩
⟨structure element declarator rules 131c⟩

⟨external declarator rules 101e⟩≡ (101d)
⟨xdecl rule 102a⟩
⟨xdlist rule 102g⟩
⟨xdecor rule 103c⟩
⟨xdecor2 rule 103d⟩

```

8.4.1 No-declarator declarations

The simplest form of `xdecl` has no declarator at all—just a type specifier and a semicolon. This covers forward struct declarations (`struct Foo;`), enum definitions, and the degenerate case `int;` (which compiles but is useless). The global `lastclass` and `lasttype` capture the type specifiers parsed by `ctlist`, and `dodecl` is called with `Z` (no name) to finalize any side effects like struct tag registration.

<xdecl rule 102a>≡ (101e) 102f▷

```
/*
 * external declarator
 */
xdecl:
    zctlist ';'          { dodecl(xdecl, lastclass, lasttype, Z); }
```

<global lastclass 102b>≡ (511d)

```
// enum<storage_class>
char lastclass;
```

<global lasttype 102c>≡ (511d)

```
Type* lasttype;
```

<ebnf grammar rules 102d>≡ (101c) 106e▷

```
zctlist:
/* empty */
{
    lastclass = CXXX;
    lasttype = types[TINT];
}
| ctlist
```

<extra grammar rules 102e>≡ (101c)

```
ctlist:
types
{
    lastclass = $1.c;
    lasttype = $1.t;
}
```

8.4.2 Declarator-based declarations

Most useful declarations have a declarator: `int x;`, `int *p;`, `int f(int);`, `typedef int size_t;`. The `xdlist` rule calls `dodecl` for each declarator in a comma-separated list. `dodecl` is the key function: it walks the nested `OIND` (pointer), `OFUNC`, and `OARRAY` nodes in the declarator tree, composing them with the base type from part one to build the final `Type`. For example, `int *f(int)` starts with base type `TINT`, then `dodecl` wraps it in `TFUNC` (for the parameter list) and `TIND` (for the pointer), yielding “pointer to function returning int.”

<xdecl rule 102f>+≡ (101e) <102a 105c>

```
| zctlist xdlist ';
```

<xdlist rule 102g>≡ (101e) 119d▷

```
xdlist:
    xdecor { dodecl(xdecl, lastclass, lasttype, $1); }
| xdlist ',' xdlist
```

<function dodecl 103a>≡ (525)

```
Node*
dodecl(void (*f)(int,Type*,Sym*), int class, Type *t, Node *n)
{
    <dodecl() locals 104c>

    nearln = lineno;
    <dodecl() other initializations 418c>

loop:
    if(n != Z)
        switch(n->op) {
            // can adjust t using information from node n (type information part2)
            <dodecl() switch node kind cases 104d>
            default:
                diag(n, "unknown declarator: %0", n->op);
                break;
        }

    lastdcltype = t;
    return n;
}
```

Uses Z 29a, diag() 496b, lastdcltype 103b, lineno 29f, and nearln 29g.

<global lastdcl 103b>≡ (511d)

```
Type* lastdcltype;
```

8.4.3 Entity declarations, the declarator

The declarator grammar mirrors C's inside-out type syntax. An **xdecor** is an optional pointer prefix (*) wrapping an **xdecor2**, which is either a bare identifier (**tag**) or a parenthesized subdeclarator. Array brackets ([]) and function parameter lists (()) are postfix on **xdecor2**. The parser builds a small tree of **OIND**, **OARRAY**, and **OFUNC** nodes around the **ONAME** leaf; **dodecl** then walks this tree to compose the final type.

<xdecor rule 103c>≡ (101e) 135a▷

```
xdecor:
    xdecor2
```

<xdecor2 rule 103d>≡ (101e) 135d▷

```
xdecor2:
    tag
|    '(' xdecor ')'          { $$ = $2; }
```

<names rules 103e>≡ (101c) 104b▷

```
tag:
    ltag
{
    $$ = new(ONAME, Z, Z);
    $$->sym = $1;
    <tag rule, set other fields to name node 103f>
}
```

<tag rule, set other fields to name node 103f>≡ (103e)

```
$$->type = $1->type;
$$->etype = ($1->type != T) ? $1->type->etype : TVOID;
$$->class = $1->class;

$$->xoffset = $1->offset;
```



```

⟨type declarations 104a⟩+≡ (100a) <101b 122a>
    %type <sym> ltag

⟨names rules 104b⟩+≡ (101c) <103e 114b>
    ltag:
        LNAME
    |    LTYPE

⟨dodecl() locals 104c⟩≡ (103a) 135e>
    Sym *s;

⟨dodecl() switch node kind cases 104d⟩≡ (103a) 135c>
    case ONAME:
        ⟨dodecl() in ONAME case, break if empty callback 118e⟩
        s = n->sym;

        // callback! should set s->type, s->class, etc.
        (*f)(class, t, s);

        ⟨dodecl() case ONAME, if local static variable 415d⟩
        ⟨dodecl() case ONAME, other initializations 417c⟩

        // propagate symbol information to node
        n->sym = s; ///? dead?
        n->type = s->type;
        n->etype = (s->type != T) ? s->type->etype : TVOID;
        n->class = s->class;
        n->xoffset = s->offset;

        ⟨dodecl() debug declaration 490a⟩
        ⟨dodecl() case ONAME, debugging support 401a⟩

        s->varlineno = lineno;
        break;

Uses ONAME 27, T 31i, TVOID 31b, and lineno 29f.

⟨function xdecl 104e⟩≡ (525)
    /// dodecl(<>, ..., node n != Z) -> <>
    void
    xdecl(int class, Type *t, Sym *s)
    {
        long o = 0; // offset

        // adjusting class (and possibly o for CEXREG)
        switch(class) {
            ⟨xdecl() switch class cases 143c⟩
        }
        ⟨xdecl() sanity check class after switch class 144a⟩
        ⟨xdecl() sanity check type after switch class 145b⟩
        ⟨xdecl() merge type declarations 148a⟩

        s->type = t;
        s->class = class;
        s->block = 0;
        s->offset = o;
    }

```

8.4.4 Function definitions

When the parser sees a declarator followed by a { instead of a ;, it is a function definition. The yacc action calls `dodecl` to extract the function name and type, pushes the parameters into the scope (via `markdcl` and `pdecl`), parses the body, and then immediately calls `codgen` to generate assembly for that function. After code generation, `revertdcl` restores the scope. This is why `5c` is a one-pass compiler: each function is fully compiled before the next one is even parsed.

<global firstdcl 105a>≡ (511d)

```
Decl* firstdcl;
```

<global thisfn 105b>≡ (511d)

```
Type* thisfn_type;
```

<xdecl rule 105c>+≡ (101e) <102f

```
| zclist xdecor
{
    lastdcltype = T; //dead??
    <xdecl rule, initializations before processing a function 274k>
    dodecl(xdecl, lastclass, lasttype, $2);
    <xdecl rule, sanity check lastdcltype is a function type 105d>
    thisfn_type = lastdcltype;
    markdcl();
    firstdcl = dclstack;
    argmark($2, 0);
}
pdecl
{
    argmark($2, 1);
}
block
{
    Node *n;
    n = revertdcl();
    <xdecl rule, adjust block body with possible hidden generated nodes 416c>
    <xdecl rule, debug function body 487a>
    if(!debug['a'] && !debug['Z'])
        codgen($6, $2); // !!!!!!!!!!!!!!!!!!!!!!!
}
```

<xdecl rule, sanity check lastdcltype is a function type 105d>≡ (105c)

```
if(lastdcltype == T || lastdcltype->etype != TFUNC) {
    diag($2, "not a function");
    lastdcltype = types[TFUNC];
}
```

8.5 Statements

Statements are straightforward: the `stmtnt` rule matches `if`, `while`, `for`, `switch`, `return`, `break`, `continue`, and `goto`, building the expected AST nodes. The statement list is built with `OLIST` nodes, then reversed by `invert` (since yacc's left-recursive rules produce the list backwards). Blocks call `markdcl`/`revertdcl` to manage scope, and labels (including `case`) are interleaved into the statement list.

<statements rules 105e>≡ (101c) 106b▷

```
block:
    '{' slist '}'
    {
        $$ = invert($2);
```

```

        if($$ == Z)
            $$ = new(OLIST, Z, Z);
    }

slist:
    /* empty */      { $$ = Z;      }
|   slist adec1     { $$ = new(OLIST, $1, $2); }
|   slist stmtnt    { $$ = new(OLIST, $1, $2); }

⟨function invert 106a⟩≡ (518c)
Node*
invert(Node *n)
{
    Node *i;

    if(n == Z || n->op != OLIST)
        return n;
    i = n;
    for(n = n->left; n != Z; n = n->left) {
        if(n->op != OLIST)
            break;
        i->left = n->right;
        n->right = i;
        i = n;
    }
    i->left = n;
    return i;
}

```

Uses OLIST 29b and Z 29a.

```

⟨statements rules 106b⟩+≡ (101c) <105e 106c>
stmtnt:
    ulstmtnt
|   labels ulstmtnt { $$ = new(OLIST, $1, $2); }
|   error ';'      { $$ = Z; }

⟨statements rules 106c⟩+≡ (101c) <106b 108a>
⟨ulstmtnt rule 106d⟩
⟨label rule 108f⟩

⟨ulstmtnt rule 106d⟩≡ (106c) 106f>
ulstmtnt:
    zcexpr ';'

⟨ebnf grammar rules 106e⟩+≡ (101c) <102d 110j>
labels:
    label
|   labels label { $$ = new(OLIST, $1, $2); }

```

8.5.1 Blocks

A block opens a new declaration scope via `markdcl()` and closes it with `revertdcl()`, which pops all symbols declared inside the block from the symbol table. This scope management happens at parse time, so by the time code generation runs, all names have already been resolved to their correct bindings.

```

⟨ulstmtnt rule 106f⟩+≡ (106c) <106d 107b>
|   {
        markdcl();
    }

```

```

block
{
    Node *n;
    n = revertdcl();
    <ulstmt rule, adjust block body with possible hidden generated nodes 416d>
    else
        $$ = $2;
}

```

8.5.2 Conditionals

An if statement becomes an OIF node whose left child is the condition expression and whose right child is an OLIST pair of (then-branch, else-branch), with Z for a missing `else`. The compiler warns on empty bodies, a useful diagnostic since a stray semicolon after `if(x);` is almost always a bug.

```

<statement nodes 107a>≡ (27) 107c▷
OIF,

<ulstmt rule 107b>+≡ (106c) <106f 107d>
| LIF '(' cexpr ')' stmtnt
| {
|     $$ = new(OIF, $3, new(OLIST, $5, Z));
|     if($5 == Z)
|         warn($3, "empty if body");
| }
| LIF '(' cexpr ')' stmtnt LELSE stmtnt
| {
|     $$ = new(OIF, $3, new(OLIST, $5, $7));
|     if($5 == Z)
|         warn($3, "empty if body");
|     if($7 == Z)
|         warn($3, "empty else body");
| }

```

8.5.3 Loops

Loops use three node kinds: OWHILE (condition on the left, body on the right), ODWHILE for do-while (same layout but with reversed evaluation order), and OFOR whose left child packs the three `for` clauses via nested OLIST nodes. The `for` rule calls `markdcl()`/`revertdcl()` because 5c supports the C++ extension `for(int i = 0; ...)`, which requires a fresh scope for the loop variable.

```

<statement nodes 107c>+≡ (27) <107a 108b>▷
OWHILE,
ODWHILE,
OFOR,

<ulstmt rule 107d>+≡ (106c) <107b 107e>▷
| LWHILE '(' cexpr ')' stmtnt { $$ = new(OWHILE, $3, $5); }
| LDO stmtnt LWHILE '(' cexpr ')' ';' { $$ = new(ODWHILE, $5, $2); }

<ulstmt rule 107e>+≡ (106c) <107d 108c>▷
| {
|     markdcl();
| }
| LFOR '(' forexpr ';' zcexpr ';' zcexpr ')' stmtnt
| {
|     Node *n;
|     n = revertdcl();
| }

```

```

    <ulstmt rule, adjust forexpr with possible hidden generated nodes 416e>
    $$ = new(OFOR, new(OLIST, $6, new(OLIST, $4, $8)), $10);
}

```

```

<statements rules 108a>+≡ (101c) <106c
forexpr:
  zcexpr
  | ctlist adlist { $$ = $2; }

```

8.5.4 Control flow jumps

ORETURN records `thisfntype->link` (the function's return type) directly in the node at parse time, so the type checker can later verify the returned expression matches. OBREAK and OCONTINUE are leaf nodes with no children—the compiler resolves which loop they refer to during code generation, not during parsing.

```

<statement nodes 108b>+≡ (27) <107c 108e>
ORETURN,

OBREAK,
OCONTINUE,

```

```

<ulstmt rule 108c>+≡ (106c) <107e 108d>
| LRETURN zcexpr ';'
{
  $$ = new(ORETURN, $2, Z);
  $$->type = thisfntype->link;
}

```

```

<ulstmt rule 108d>+≡ (106c) <108c 108g>
| LBREAK ';' { $$ = new(OBREAK, Z, Z); }
| LCONTINUE ';' { $$ = new(OCONTINUE, Z, Z); }

```

8.5.5 Labels and goto

Labels live in their own namespace (the `label` field of `Sym`) and have function scope—a `goto` can jump forward to a label not yet seen. The `dcllabel`^{109a} function achieves this by removing the label from the normal `dclstack` and inserting it at `firstdcl`, which is the function-level scope. This way, `revert dcl()` for inner blocks will not accidentally pop the label.

```

<statement nodes 108e>+≡ (27) <108b 109b>
OLABEL,
OGOTO,

```

```

<label rule 108f>≡ (106c) 109d>
| LNAME ':' { $$ = new(OLABEL, dcllabel($1, true), Z); }

```

```

<ulstmt rule 108g>+≡ (106c) <108d 109c>
| LGOTO ltag ';' { $$ = new(OGOTO, dcllabel($2, false), Z); }

```

```

<Sym label value fields 108h>≡ (34c)
Node* label;

```

```

⟨function dcllabel 109a⟩≡ (525)
  @@Scheck: used by cc.y
  Node* dcllabel(Sym *s, bool defcontext)
  {
    Decl *d;
    Decl d1;
    Node *n;

    n = s->label;
    ⟨dcllabel() if n not null, mark node as declared or used 185b⟩
    // else, new label

    d = push();
    d->sym = s;
    d->val = DLABEL;

    //pop_stack(dclstack) (we add the label to the function scope instead)
    dclstack = d->link;
    //add_list(d, firstdcl)
    d1 = *firstdcl;
    *firstdcl = *d;
    *d = d1;
    firstdcl->link = d;
    firstdcl = d;

    n = new(OXXX, Z, Z);
    n->sym = s;
    ⟨dcllabel() when new label, set def and use fields 185a⟩
    s->label = n;

    ⟨dcllabel() debug declaration 490b⟩
    return n;
  }

```

Uses DLABEL 38a, OXXX 27, Z 29a, dclstack 39a, firstdcl 105a, new() 29d, and push() 39e.

8.5.6 Switch

A **switch** becomes an **OSWITCH** node with the scrutinee expression on the left and the body statement on the right. **case** and **default** labels are **OCASE** nodes (with **left** being the constant expression, or **Z** for **default**). They appear in the grammar as label alternatives alongside **OLABEL**, so you can freely intermix **case 1: label_x: ...** in the source.

```

⟨statement nodes 109b⟩+≡ (27) <108e 184c>
  OSWITCH,
  OCASE, // for default too, in which case Node.left is null

```

```

⟨ulstmnt rule 109c⟩+≡ (106c) <108g 184d>
  | LSWITCH '(' cexpr ')' stmtnt
  {
    ⟨ulstmnt rule, SWITCH case, adjust cexpr node 179b⟩
    $$ = new(OSWITCH, $3, $5);
  }

```

```

⟨label rule 109d⟩+≡ (106c) <108f>
  label:
    LCASE expr ':' { $$ = new(OCASE, $2, Z); }
    | LDEFAULT ':' { $$ = new(OCASE, Z, Z); }

```

8.6 Expressions

The expression grammar uses a precedence hierarchy: **expr** (comma) → **cexpr** (assignment) → binary operators with yacc-declared precedences → **uexpr** (unary prefix) → **pexpr** (primary: names, constants, parenthesized expressions, postfix operators). Array access **a[i]** is immediately desugared into ***(a+i)**, and **a->f** into **(*a).f**, so the AST has fewer node kinds than the surface syntax suggests.

$\langle \text{expressions rules 110a} \rangle \equiv$ (101c) 110f $\triangleright$

$\langle \text{expr rule 110b} \rangle$
 $\langle \text{xuexpr rule 110c} \rangle$
 $\langle \text{uexpr rule 110d} \rangle$
 $\langle \text{pexpr rule 110e} \rangle$

$\langle \text{expr rule 110b} \rangle \equiv$ (110a) 114d $\triangleright$

expr:
xuexpr

$\langle \text{xuexpr rule 110c} \rangle \equiv$ (110a) 118b $\triangleright$

xuexpr:
uexpr

$\langle \text{uexpr rule 110d} \rangle \equiv$ (110a) 114f $\triangleright$

uexpr:
pexpr

$\langle \text{pexpr rule 110e} \rangle \equiv$ (110a) 111c $\triangleright$

pexpr:
 ' (' cexpr ') ' { \$\$ = \$2; }

$\langle \text{expressions rules 110f} \rangle + \equiv$ (101c) $\triangleleft$ 110a 110h $\triangleright$

cexpr:
expr
 | cexpr ' , ' cexpr { \$\$ = new(OCOMMA, \$1, \$3); }

$\langle \text{expression nodes 110g} \rangle \equiv$ (27) 111a $\triangleright$

OCOMMA,

$\langle \text{expressions rules 110h} \rangle + \equiv$ (101c) $\triangleleft$ 110f 110i $\triangleright$

zexpr:
 /* empty */ { \$\$ = Z; }
 | lexpr

$\langle \text{expressions rules 110i} \rangle + \equiv$ (101c) $\triangleleft$ 110h 112f $\triangleright$

lexpr:
expr
 {
 \$\$ = new(OCAST, \$1, Z);
 \$\$->type = types[TLONG];
 }

$\langle \text{ebnf grammar rules 110j} \rangle + \equiv$ (101c) $\triangleleft$ 106e 117e $\triangleright$

zcexpr:
 /* empty */ { \$\$ = Z; }
 | cexpr

8.6.1 Numeric constants

Each numeric token variant (LCONST, LLCONST, LFCNST, etc.) gets its own grammar action that creates an OCONST node with the appropriate type pre-set. The parser also saves the original text (**symb**) in **cstring** for debugging output and constant folding. Note that character constants like 'x' are already converted to integers by the lexer.

<expression nodes 111a>+≡ (27) <110g 112c>
OCONST,

Integers

Each integer constant variant (plain, unsigned, long, vlong) gets its own grammar action that creates an OCONST node with the right type and value pre-set. The **vconst** field uses **vlong** (64-bit) to hold any integer value; **cstring** saves the original text for debugging.

<Node value fields 111b>+≡ (28) <29c 112a>
vlong vconst; /* non fp const */ // for OCONST

<pepr rule 111c>+≡ (110a) <110e 111d>
| LCONST
{
 \$\$ = new(OCONST, Z, Z);
 \$\$->type = types[TINT];
 \$\$->vconst = \$1;
 \$\$->cstring = strdup(symb);
}
| LLCONST
{
 \$\$ = new(OCONST, Z, Z);
 \$\$->type = types[TLONG];
 \$\$->vconst = \$1;
 \$\$->cstring = strdup(symb);
}
| LVLCONST
{
 \$\$ = new(OCONST, Z, Z);
 \$\$->type = types[TVLONG];
 \$\$->vconst = \$1;
 \$\$->cstring = strdup(symb);
}

<pepr rule 111d>+≡ (110a) <111c 112b>
| LUCONST
{
 \$\$ = new(OCONST, Z, Z);
 \$\$->type = types[TUINT];
 \$\$->vconst = \$1;
 \$\$->cstring = strdup(symb);
}
| LULCONST
{
 \$\$ = new(OCONST, Z, Z);
 \$\$->type = types[TULONG];
 \$\$->vconst = \$1;
 \$\$->cstring = strdup(symb);
}
| LUVLCONST
{


```

    $$ = new(OCNST, Z, Z);
    $$->type = types[TUVLONG];
    $$->vconst = $1;
    $$->cstring = strdup(symb);
}

```

Floats

$\langle \text{Node value fields } 112a \rangle + \equiv$ (28) $\triangleleft 111b \ 112e \triangleright$
double fconst; /* fp constant */ // for OCONST

$\langle \text{pexpr rule } 112b \rangle + \equiv$ (110a) $\triangleleft 111d \ 112d \triangleright$
| LFCNST
{
 \$\$ = new(OCNST, Z, Z);
 \$\$->type = types[TFLOAT];
 \$\$->fconst = \$1;
 \$\$->cstring = strdup(symb);
}
| LDCNST
{
 \$\$ = new(OCNST, Z, Z);
 \$\$->type = types[TDOUBLE];
 \$\$->fconst = \$1;
 \$\$->cstring = strdup(symb);
}
}

8.6.2 String constants

ASCII strings

String literals produce OSTRING nodes with a TARRAY of TCHAR type. Adjacent strings are concatenated at parse time by appending the second to the first. Each string is assigned a unique static symbol (via symstring) so the code generator can emit it as a DATA item.

$\langle \text{expression nodes } 112c \rangle + \equiv$ (27) $\triangleleft 111a \ 113a \triangleright$
OSTRING,

$\langle \text{pexpr rule } 112d \rangle + \equiv$ (110a) $\triangleleft 112b \ 114a \triangleright$
| string
| lstring

$\langle \text{Node value fields } 112e \rangle + \equiv$ (28) $\triangleleft 112a \ 113b \triangleright$
char* cstring; /* character string */ // for OSTRING (and OCONST)

$\langle \text{expressions rules } 112f \rangle + \equiv$ (101c) $\triangleleft 110i \ 113d \triangleright$
string:
LSTRING
{
 \$\$ = new(OSTRING, Z, Z);
 \$\$->type = typ(TARRAY, types[TCHAR]);
 \$\$->etype = TARRAY;
 \$\$->type->width = \$1.l + 1;
 \$\$->cstring = \$1.s;
 $\langle \text{string rule, set sym and class for OSTRING } 226b \rangle$
}
| string LSTRING
{

```

char *s;
int n;

n = $1->type->width - 1;
s = alloc(n+$2.1+MAXALIGN);

memcpy(s, $1->cstring, n);
memcpy(s+n, $2.s, $2.1);
s[n+$2.1] = '\0';

$$ = $1;
$$->type->width += $2.1;
$$->cstring = s;
}

```

Unicode strings

<expression nodes 113a>+≡ (27) <112c 114c>
 OLSTRING,

<Node value fields 113b>+≡ (28) <112e 208g>
 TRune* rstring; /* rune string */ // for OLSTRING

<constant TRUNE 113c>≡ (120e)
 /* adapt size of Rune to target system's size */
 TRUNE = sizeof(TRune)==4? TUINT: TUSHORT,

Uses TUINT 30b and TUSHORT 30b.

<expressions rules 113d>+≡ (101c) <112f 117f>

```

lstring:
  LLSTRING
  {
    $$ = new(OLSTRING, Z, Z);
    $$->type = typ(TARRAY, types[TRUNE]);
    $$->etype = TARRAY;
    $$->type->width = $1.1 + sizeof(TRune);
    $$->rstring = (TRune*)$1.s;
    <string rule, set sym and class for OSTRING 226b>
  }
| lstring LLSTRING
  {
    char *s;
    int n;

    n = $1->type->width - sizeof(TRune);
    s = alloc(n+$2.1+MAXALIGN);

    memcpy(s, $1->rstring, n);
    memcpy(s+n, $2.s, $2.1);
    *(TRune*)(s+n+$2.1) = 0;

    $$ = $1;
    $$->type->width += $2.1;
    $$->rstring = (TRune*)s;
  }

```

8.6.3 Entity uses

When an identifier appears in an expression, the parser resolves it immediately: it copies `Sym.type`, `Sym.class`, and `Sym.offset` from the symbol table entry into the AST node. This is the bridge from name resolution to code generation—after this point, the node is self-contained and does not need the symbol table anymore. If the identifier has not been declared, 5c issues a warning but continues (with an implicit `int` type), matching traditional C behavior.

```

<expr rule 114a>+≡ (110a) <112d 116g>
|   name

<names rules 114b>+≡ (101c) <104b
name:
  LNAME
  {
    $$ = new(ONAME, Z, Z);
    <name rule, if local static variable 415e>
    $$->sym = $1;

    // propagate symbol information to node
    $$->type = $1->type;
    $$->etype = ($1->type != T) ? $1->type->etype : TVOID;
    $$->xoffset = $1->offset;
    $$->class = $1->class;
    <name rule, LNAME case, adjust more fields 183d>
  }

```

8.6.4 Arithmetic expressions

The expression rules use yacc's `%left` and `%right` precedence declarations to handle operator priority and associativity, avoiding the need for separate grammar nonterminals per precedence level. All binary operators share the same `expr op expr` pattern and yacc resolves conflicts using the declared precedences.

```

<expression nodes 114c>+≡ (27) <113a 114e>
OADD,
OSUB,

OMUL,
ODIV,
OMOD,

<expr rule 114d>+≡ (110a) <110b 115b>
|   expr '+' expr { $$ = new(OADD, $1, $3); }
|   expr '-' expr { $$ = new(OSUB, $1, $3); }
|   expr '*' expr { $$ = new(OMUL, $1, $3); }
|   expr '/' expr { $$ = new(ODIV, $1, $3); }
|   expr '%' expr { $$ = new(OMOD, $1, $3); }

<expression nodes 114e>+≡ (27) <114c 115a>
OPOS,
ONEG,

<uexpr rule 114f>+≡ (110a) <110d 115f>
|   '+' xuexpr { $$ = new(OPOS, $2, Z); }
|   '-' xuexpr { $$ = new(ONEG, $2, Z); }

```

$\langle \text{expression nodes } 115a \rangle + \equiv$ (27) $\triangleleft 114e \ 115c \triangleright$

```

OAND,
OOR,
OXOR,

OASHL,
OASHR,

```

$\langle \text{expr rule } 115b \rangle + \equiv$ (110a) $\triangleleft 114d \ 115d \triangleright$

```

|   expr '&' expr { $$ = new(OAND, $1, $3); }
|   expr '^' expr { $$ = new(OXOR, $1, $3); }
|   expr '|' expr { $$ = new(OOR, $1, $3); }

|   expr LRSH expr { $$ = new(OASHR, $1, $3); }
|   expr LLSH expr { $$ = new(OASHL, $1, $3); }

```

8.6.5 Boolean expressions

OANDAND and OOROR are short-circuit operators: unlike arithmetic operators that map directly to a single instruction, these require conditional branches during code generation, making them closer to control flow statements than pure expressions. ONOT (!) is the logical complement, while OCOM () is bitwise complement—grouped here but not truly boolean.

$\langle \text{expression nodes } 115c \rangle + \equiv$ (27) $\triangleleft 115a \ 115e \triangleright$

```

OANDAND,
OOROR,

```

$\langle \text{expr rule } 115d \rangle + \equiv$ (110a) $\triangleleft 115b \ 115h \triangleright$

```

|   expr LANDAND expr { $$ = new(OANDAND, $1, $3); }
|   expr LOROR expr   { $$ = new(OOROR, $1, $3); }

```

$\langle \text{expression nodes } 115e \rangle + \equiv$ (27) $\triangleleft 115c \ 115g \triangleright$

```

ONOT,
OCOM,

```

$\langle \text{uexpr rule } 115f \rangle + \equiv$ (110a) $\triangleleft 114f \ 116f \triangleright$

```

|   '!' xuexpr { $$ = new(ONOT, $2, Z); }
|   '~' xuexpr { $$ = new(OCOM, $2, Z); }

```

$\langle \text{expression nodes } 115g \rangle + \equiv$ (27) $\triangleleft 115e \ 116a \triangleright$

```

OEQ,
ONE,

OLT,
OGT,
OLE,
OGE,

```

$\langle \text{expr rule } 115h \rangle + \equiv$ (110a) $\triangleleft 115d \ 116b \triangleright$

```

|   expr LEQ expr { $$ = new(OEQ, $1, $3); }
|   expr LNE expr { $$ = new(ONE, $1, $3); }

|   expr '<' expr { $$ = new(OLT, $1, $3); }
|   expr '>' expr { $$ = new(OGT, $1, $3); }
|   expr LLE expr { $$ = new(OLE, $1, $3); }
|   expr LGE expr { $$ = new(OGE, $1, $3); }

```

8.6.6 Assignments

Plain assignment (=) and compound assignments (+=, -=, etc.) each have their own AST node kinds (OAS, OASADD, etc.). 5c does not desugar `x += y` into `x = x + y`, because the compound form evaluates the left-hand side only once—important when it has side effects (e.g., `*p++ += 1`).

<expression nodes 116a>+≡ (27) <115g 116c>
OAS,

<expr rule 116b>+≡ (110a) <115h 116d>
| expr '=' expr { \$\$ = new(OAS, \$1, \$3); }

<expression nodes 116c>+≡ (27) <116a 116e>
OASADD,
OASSUB,

OASMUL,
OASMOD,
OASDIV,

OASAND,
OASOR,
OASXOR,

OASASHL,
OASASHR,

<expr rule 116d>+≡ (110a) <116b 118g>
| expr LPE expr { \$\$ = new(OASADD, \$1, \$3); }
| expr LME expr { \$\$ = new(OASSUB, \$1, \$3); }
| expr LMLE expr { \$\$ = new(OASMUL, \$1, \$3); }
| expr LDVE expr { \$\$ = new(OASDIV, \$1, \$3); }
| expr LMDE expr { \$\$ = new(OASMOD, \$1, \$3); }
| expr LANDE expr { \$\$ = new(OASAND, \$1, \$3); }
| expr LXORE expr { \$\$ = new(OASXOR, \$1, \$3); }
| expr LORE expr { \$\$ = new(OASOR, \$1, \$3); }
| expr LLSHE expr { \$\$ = new(OASASHL, \$1, \$3); }
| expr LRSHE expr { \$\$ = new(OASASHR, \$1, \$3); }

8.6.7 Pointers

OIND (indirection, i.e., dereference `*p`) and OADDR (address-of `&x`) are the two pointer operators. Note that OIND is reused in the declaration grammar to represent pointer types (e.g., `int *x`), so the same node kind serves double duty—the context (expression vs. declarator) determines which meaning applies.

<expression nodes 116e>+≡ (27) <116c 117a>
OIND, // for uses (dereference) and also declarations
OADDR,

<uexpr rule 116f>+≡ (110a) <115f 118i>
| '*' xuexpr { \$\$ = new(OIND, \$2, Z); }
| '&' xuexpr { \$\$ = new(OADDR, \$2, Z); }

8.6.8 Array accesses

Array indexing is immediately desugared at parse time: `x[i]` becomes `*(x+i)`. This means the AST has no dedicated array-access node. The type checker later adjusts the addition for pointer arithmetic (scaling by element size), and if `x` is an array type, it gets an implicit OADDR to decay into a pointer.

<pexpr rule 116g>+≡ (110a) <114a 117b>
| pexpr '[' cexpr ']' { \$\$ = new(OIND, new(OADD, \$1, \$3), Z); }

8.6.9 Field accesses

Only ODOT exists in the AST—the arrow operator `->` is desugared at parse time into `(*x).f`, i.e., an ODOT wrapping an OIND. The field name is stored in `node->sym` rather than as a child node. Code generation will ultimately turn field access into a pointer-plus-offset computation, just like array access.

$\langle \text{expression nodes } 117a \rangle + \equiv$ (27) $\triangleleft 116e \ 117c \triangleright$
 ODOT,

$\langle \text{pexpr rule } 117b \rangle + \equiv$ (110a) $\triangleleft 116g \ 117d \triangleright$
 | pexpr '.' ltag
 {
 \$\$ = new(ODOT, \$1, Z);
 \$\$->sym = \$3;
 }
 | pexpr LMG ltag
 {
 \$\$ = new(ODOT, new(OIND, \$1, Z), Z);
 \$\$->sym = \$3;
 }

8.6.10 Function calls

OFUNC is overloaded: it represents both function calls (in expressions) and function types (in declarations). When the compiler sees a call to an undeclared function like `foo(x)`, it implicitly declares `foo` as returning `int` via `dodecl`—a legacy C behavior that the Plan 9 OCaml port sensibly forbids. The argument list is reversed with `invert()` to restore left-to-right order from the right-recursive grammar.

$\langle \text{expression nodes } 117c \rangle + \equiv$ (27) $\triangleleft 117a \ 118a \triangleright$
 OFUNC, // used for uses (calls) but also defs (and decls)

$\langle \text{pexpr rule } 117d \rangle + \equiv$ (110a) $\triangleleft 117b \ 118k \triangleright$
 | pexpr '(' zelist ')'
 {
 \$\$ = new(OFUNC, \$1, Z);
 $\langle \text{pexpr rule, implicit declaration of unknown function } 117g \rangle$
 \$\$->right = invert(\$3);
 }

$\langle \text{ebnf grammar rules } 117e \rangle + \equiv$ (101c) $\triangleleft 110j \triangleright$
 zelist:
 /* empty */ { \$\$ = Z; }
 | elist

$\langle \text{expressions rules } 117f \rangle + \equiv$ (101c) $\triangleleft 113d \triangleright$
 elist:
 expr
 | elist ',' elist { \$\$ = new(OLIST, \$1, \$3); }

$\langle \text{pexpr rule, implicit declaration of unknown function } 117g \rangle \equiv$ (117d)
 if((\$1->op == ONAME) && (\$1->type == T))
 dodecl(xdecl, CXXX, types[TINT], \$\$);

8.6.11 Cast

A cast expression like `(int*)p` is parsed as an extended unary expression. The trick is that the type in parentheses is an abstract declarator (no variable name), so the parser calls `dodecl(NODECL, ...)` to run the type-assembly machinery and recover the full type in `lastdcltype`, without actually declaring any symbol. The `xcast` flag marks this as an explicit cast, distinguishing it from the implicit casts the compiler inserts during type checking.

<expression nodes 118a>+≡ (27) <117c 118f>
OCAST,

<xuexpr rule 118b>+≡ (110a) <110c 430a>
| '(' tlist abdecor ')' xuexpr
{
 \$\$ = new(OCAST, \$5, Z);
 dodecl(NODECL, CXXX, \$2, \$3);
 \$\$->type = lastdcltype;
 \$\$->xcast = true;
}

<Node origin tracking fields 118c>≡ (28) 304b>
bool xcast;

<constant NODECL 118d>≡ (504b)
#define NODECL ((void*)(int, Type*, Sym*)) nil)

<dodecl() in ONAME case, break if empty callback 118e>≡ (104d)
if(f == NODECL)
 break;

Uses NODECL 118d.

8.6.12 Ternary expressions

<expression nodes 118f>+≡ (27) <118a 118h>
OCOND,

<expr rule 118g>+≡ (110a) <116d
| expr '?' cexpr ':' expr { \$\$ = new(OCOND, \$1, new(OLIST, \$3, \$5)); }

8.6.13 Prefix/postfix

<expression nodes 118h>+≡ (27) <118f 118j>
OPREDEC,
OPREINC,

<uexpr rule 118i>+≡ (110a) <116f 119c>
| LPP xuexpr { \$\$ = new(OPREINC, \$2, Z); }
| LMM xuexpr { \$\$ = new(OPREDEC, \$2, Z); }

<expression nodes 118j>+≡ (27) <118h 119a>
OPOSTINC,
OPOSTDEC,

<pexpr rule 118k>+≡ (110a) <117d 119b>
| pexpr LPP { \$\$ = new(OPOSTINC, \$1, Z); }
| pexpr LMM { \$\$ = new(OPOSTDEC, \$1, Z); }

8.6.14 sizeof()

<expression nodes 119a>+≡ (27) <118j 119i>
OSIZE,

<pepr rule 119b>+≡ (110a) <118k 442e>
| LSIZEOF '(' tlist abdecor ')' {
| {
| \$\$ = new(OSIZE, Z, Z);
| dodecl(NODECL, CXXX, \$3, \$4);
| \$\$->type = lastdcltype;
| }
| }

<uexpr rule 119c>+≡ (110a) <118i 442d>
| LSIZEOF uexpr { \$\$ = new(OSIZE, \$2, Z); }

8.7 Initializers and designators

Initializers handle both simple expressions (`int x = 1`) and nested brace-lists (`int a[] = \1, 2, 3\`). Designators (`.field` and `[index]`) allow targeting specific elements, following C99 syntax. The heavy lifting is done by `doinit`, which recursively walks the type structure and the initializer list, emitting `ADATA` pseudo-instructions to lay out the initial values in the object file.

<xclist rule 119d>+≡ (101e) <102g>
| xdecor
| {
| \$1 = dodecl(xdecl, lastclass, lasttype, \$1);
| }
| '=' init
| {
| doinit(\$1->sym, \$1->type, OL, \$4);
| }

<declaration nodes 119e>≡ (27) 119h>
OINIT,

<initializers rules 119f>≡ (101c) 119g>
init:
 expr
| '{' ilist '}' { \$\$ = new(OINIT, invert(\$2), Z); }

<initializers rules 119g>+≡ (101c) <119f 120a>
ilist:
 qlist
| init
| qlist init { \$\$ = new(OLIST, \$1, \$2); }

qlist:
 init ','
| qlist init ',' { \$\$ = new(OLIST, \$1, \$2); }
| qual
| qlist qual { \$\$ = new(OLIST, \$1, \$2); }

<declaration nodes 119h>+≡ (27) <119e 136f>
OELEM, // field designator

<expression nodes 119i>+≡ (27) <119a 262b>
OARRAY, // used for uses (designator) and declarations

<initializers rules 120a>+≡ (101c) *<119g*

```
qual:
    '[' lexpr ']' { $$ = new(OARRAY, $2, Z); }
|   '.' ltag
    {
        $$ = new(OELEM, Z, Z);
        $$->sym = $2;
    }
|   qual '='
```

<global initlist 120b>≡ (511d)

```
Node* initlist;
```

<function doinit 120c>≡ (525)

```
/// yyparse -> xdlist -> <>
Node*
doinit(Sym *s, Type *t, long o, Node *a)
{
    Node *n;

    <doinit() sanity check t 120d>
    <doinit() possibly adjust class 144b>
    n = initlist;

    if(a->op == OINIT)
        a = a->left;
    initlist = a;

    a = init1(s, t, o, false);
    <doinit() sanity check initlist after init1 262c>
    initlist = n;

    return a;
}
```

Uses OINIT 119e, init1() 262d, and initlist 120b.

<doinit() sanity check t 120d>≡ (120c)

```
if(t == T)
    return Z;
```

Uses T 31i and Z 29a.

8.8 Types and storage classes

C's type syntax allows keywords in any order (unsigned long int equals int long unsigned), so the parser cannot directly build a Type from a single grammar rule. Instead, it collects keywords as bits in a Bxxx bitset via the `types` and `tname` rules. The function `simplet` then resolves the combination into a concrete `Type_kind` (e.g., TULONG). Storage classes (`auto`, `static`, `extern`) and qualifiers (`const`, `volatile`) are tracked in the same bitset but extracted separately into `lastclass` and the type's `garb` field.

<enum type_kind_bis 120e>≡ (504b)

```
enum type_kind_bis {
    // -----
    // Type (see separate Type_kind)
    // -----

    // -----
    // Class storage (temporary, see CAUTO/CEXTERN/... for final storage)
```

```

// -----
<Type_kind_bis storage cases 126a>

// -----
// Qualifiers (aka garbage) (temporary, see GCONSTNT/GVOLATILE)
// -----
<Type_kind_bis qualifier cases 127a>

// -----
// Signs (temporary, see TUINT/TULONG/... for final types)
// -----
<Type_kind_bis sign cases 123g>

// -----
// Misc
// -----
<Type_kind_bis misc cases 424d>

NALLTYPES,

<constant TRUNE 113c>
};

<enum bxxx 121>≡ (504b)
enum Bxxx
{
    BCHAR   = 1L<<TCHAR,
    BUCHAR  = 1L<<TUCCHAR,
    BSHORT  = 1L<<TSHORT,
    BUSHORT = 1L<<TUSHORT,
    BINT    = 1L<<TINT,
    BUINT   = 1L<<TUINT,
    BLONG   = 1L<<TLONG,
    BULONG  = 1L<<TULONG,
    BVLONG  = 1L<<TVLONG,
    BUVLONG = 1L<<TUVLONG,
    BFLOAT  = 1L<<TFLOAT,
    BDOUBLE = 1L<<TDOUBLE,

    BIND    = 1L<<TIND,
    BFUNC   = 1L<<TFUNC,
    BARRAY  = 1L<<TARRAY,
    BVOID   = 1L<<TVOID,
    BSTRUCT = 1L<<TSTRUCT,
    BUNION  = 1L<<TUNION,
    BENUM   = 1L<<TENUM,

    BDOT    = 1L<<TDOT,

    BCONSTNT = 1L<<TCONSTNT,
    BVOLATILE = 1L<<TVOLATILE,

    BUNSIGNED = 1L<<TUNSIGNED,
    BSIGNED   = 1L<<TSIGNED,

    BAUTO    = 1L<<TAUTO,
    BEXTERN  = 1L<<TEXTERN,
    BSTATIC  = 1L<<TSTATIC,
    BTYPEDEF = 1L<<TTYPEDEF,
    BREGISTER = 1L<<TREGISTER,

```

```

    <Bxxx cases 434d>

    <Bxxx constants 124c>
};

Uses TARRAY 31c, TAUTO 126a, TCHAR 30b, TCONSTNT 127a, TDOT 137f, TDOUBLE 31a, TENUM 31c, TEXTERN 126a, TFLOAT 31a,
TFUNC 31c, TIND 31c, TINT 30b, TLONG 30b, TREGISTER 126a, TSHORT 30b, TSIGNED 123g, TSTATIC 126a, TSTRUCT 31c,
TTYPEDEF 126a, Tuchar 30b, TUINT 30b, TULONG 30b, TUNION 31c, TUNSIGNED 123g, TUSHORT 30b, TUVLONG 30b, TVLONG 30b,
TVOID 31b, and TVOLATILE 127a.

<type declarations 122a>+≡ (100a) <104a 122d>
    %type    <lval>    tname cname gname

<union yacc other fields 122b>+≡ (42b) <101a 122f>
    long    lval; //bitset<enum<Bxxx> >

<types, storage classes, qualifiers rules 122c>≡ (101c) 122e>
    <tname rule 123f>
    <cname rule 126b>
    <gname rule 127b>

<type declarations 122d>+≡ (100a) <122a 122g>
    %type    <lval>    gcname    gctname

<types, storage classes, qualifiers rules 122e>+≡ (101c) <122c 122h>
    gctname:
        tname
    |    gname
    |    cname

    gcname:
        gname
    |    cname

<union yacc other fields 122f>+≡ (42b) <122b 122i>
    Type*    type;

<type declarations 122g>+≡ (100a) <122d 122j>
    %type    <type> tlist

<types, storage classes, qualifiers rules 122h>+≡ (101c) <122e 123a>
    tlist:
        types
    {
        $$ = $1.t;
        if($1.c != CXXX)
            diag(Z, "illegal combination of class 4: %s", cnames[$1.c]);
    }

<union yacc other fields 122i>+≡ (42b) <122f 131a>
    struct
    {
        Type*    t;
        // option<enum<Storage_class> >, None = CXXX
        byte    c;
    } tycl;

<type declarations 122j>+≡ (100a) <122g 122k>
    %type    <tycl>    types

<type declarations 122k>+≡ (100a) <122j 128c>
    %type    <lval>    gctnlist gcnlist zgcnlist

```

⟨*types, storage classes, qualifiers rules 123a*⟩+≡ (101c) ◁122h 123b▷

```
gctnlist:
    gctname
    | gctnlist gctname { $$ = typebitor($1, $2); }
```

⟨*types, storage classes, qualifiers rules 123b*⟩+≡ (101c) ◁123a 123d▷

```
gcname:
    | gcname
    | gcname { $$ = typebitor($1, $2); }
```

⟨*function typebitor 123c*⟩≡ (518c)

```
/*@Scheck: used by cc.y
long typebitor(long a, long b)
{
    long c;

    c = a | b;
    if(a & b)
        if((a & b) == BLONG)
            c |= BVLONG; /* long long => vlong */
        else
            warn(Z, "once is enough: %Q", a & b);
    return c;
}
```

Uses BLONG 121, BVLONG 121, Z 29a, and warn() 142.

8.8.1 Basic types

C’s type specifiers can appear in any order and in combinations (unsigned long int, long unsigned, etc.), so the parser accumulates them as a bitmask of Bxxx flags. The `simplet`^{124e} function then maps this bitmask to a canonical `Type*` from the `types[]` hash table. This is why `simplet` has so many cases—each valid combination of keywords maps to a single type. Invalid combinations (e.g., `float short`) produce an error.

⟨*types, storage classes, qualifiers rules 123d*⟩+≡ (101c) ◁123b 125▷

⟨*types rule 123e*⟩

⟨*types rule 123e*⟩≡ (123d) 124b▷

```
types:
    tname
    {
        $.c = CXXX;
        $.t = simplet($1);
    }
```

⟨*tname rule 123f*⟩≡ (122c) 124a▷

```
tname: /* type words */
    LCHAR    { $$ = BCHAR; }
    | LSHORT { $$ = BSHORT; }
    | LINT    { $$ = BINT; }
    | LLONG   { $$ = BLONG; }

    | LFLOAT  { $$ = BFLOAT; }
    | LDOUBLE { $$ = BDOUBLE; }

    | LVOID    { $$ = BVOID; }
```

⟨*Type_kind_bis sign cases 123g*⟩≡ (120e)

```
TUNSIGNED,
TSIGNED,
```

< tname rule 124a >+≡ (122c) <123f

```
|  Lsigned { $$ = Bsigned; }
|  Lunsigned { $$ = Bunsigned; }
```

< types rule 124b >+≡ (123d) <123e 128a>

```
|  gcnlist
|  {
|      $$t = simplet($1);
|      $$c = simplec($1);
|      $$t = garbt($$.t, $1);
|  }
|  tname gcnlist
|  {
|      $$t = simplet(typebitor($1, $2));
|      $$c = simplec($2);
|      $$t = garbt($$.t, $2);
|  }
|  gcnlist tname
|  {
|      $$t = simplet($2);
|      $$c = simplec($1);
|      $$t = garbt($$.t, $1);
|  }
|  gcnlist tname gcnlist
|  {
|      $$t = simplet(typebitor($2, $3));
|      $$c = simplec($1|$3);
|      $$t = garbt($$.t, $1|$3);
|  }
```

< Bxxx constants 124c >≡ (121) 124d>

```
/* these can be overloaded with complex types */
BCLASS = BAUTO|BEXTERN|BSTATIC|BTYPDEF|BTYPSTR|BREGISTER,
```

Uses BAUTO 121, BEXTERN 121, BREGISTER 121, BSTATIC 121, BTYPDEF 121, and BTYPSTR 434d.

< Bxxx constants 124d >+≡ (121) <124c 151b>

```
BGARB = BCONSTNT|BVOLATILE,
```

Uses BCONSTNT 121 and BVOLATILE 121.

< function simplet 124e >≡ (518c)

```
Type*
simplet(long b)
{
    b &= ~BCLASS & ~BGARB;

    switch(b) {
    case BCHAR:
    case BCHAR|BSIGNED:
        return types[TCHAR];

    case BCHAR|BUNSIGNED:
        return types[TUCHAR];

    case BSHORT:
    case BSHORT|BINT:
    case BSHORT|BSIGNED:
    case BSHORT|BINT|BSIGNED:
        return types[TSHORT];
```

```

case BUNSIGNED|BSHORT:
case BUNSIGNED|BSHORT|BINT:
    return types[TUSHORT];

case 0: // default to 'int' when no type indicated at all
case BINT:
case BINT|BSIGNED:
case BSIGNED:
    return types[TINT];

case BUNSIGNED:
case BUNSIGNED|BINT:
    return types[TUINT];

case BLONG:
case BLONG|BINT:
case BLONG|BSIGNED:
case BLONG|BINT|BSIGNED:
    return types[TLONG];

case BUNSIGNED|BLONG:
case BUNSIGNED|BLONG|BINT:
    return types[TULONG];

case BVLONG|BLONG:
case BVLONG|BLONG|BINT:
case BVLONG|BLONG|BSIGNED:
case BVLONG|BLONG|BINT|BSIGNED:
    return types[TVLONG];

case BVLONG|BLONG|BUNSIGNED:
case BVLONG|BLONG|BINT|BUNSIGNED:
    return types[TUVLONG];

case BFLOAT:
    return types[TFLOAT];

case BDOUBLE:
case BDOUBLE|BLONG:
case BFLOAT|BLONG:
    return types[TDOUBLE];

case BVOID:
    return types[TVOID];
}

diag(Z, "illegal combination of types %Q", b);
return types[TINT];
}

```

Uses BCHAR 121, BCLASS 124c, BDOUBLE 121, BFLOAT 121, BGARB 124d, BINT 121, BLONG 121, BSHORT 121, BSIGNED 121, BUNSIGNED 121, BVLONG 121, BVOID 121, TCHAR 30b, TDOUBLE 31a, TFLOAT 31a, TINT 30b, TLONG 30b, TSHORT 30b, Tuchar 30b, TUINT 30b, TULONG 30b, TUSHORT 30b, TUVLONG 30b, TVLONG 30b, TVOID 31b, Z 29a, diag() 496b, and types 32b.

<types, storage classes, qualifiers rules 125>+≡ (101c) <123d 128b>

```

zgnlist:
/* empty */      { $$ = 0; }
| zgnlist gname { $$ = typebitor($1, $2); }

```

8.8.2 Storage classes

Storage classes (`static`, `extern`, `auto`, `register`) are parsed alongside type specifiers using the same bitmask technique. The `TAUTO`, `TEXTERN`, etc. values in `Type_kind_bis` are temporary tokens—they exist only to give each keyword a distinct bit position. The `simplec`^{126c} function then converts the bitmask into the real storage class enum (`CAUTO`, `CEXTERN`, `CSTATIC`). Note that `typedef` is also treated as a storage class, which is semantically odd but simplifies the grammar.

$\langle \text{Type_kind_bis storage cases } 126a \rangle \equiv$ (120e) 434e▷

```
TAUTO = NTYPE,
TEXTERN,
TSTATIC,

TTYPEDEF, // ugly, not really a storage class
TREGISTER,
```

Uses `NTYPE` 30a.

$\langle \text{cname rule } 126b \rangle \equiv$ (122c) 137g▷

```
cname: /* class words */
    LAUTO      { $$ = BAUTO; }
| LSTATIC     { $$ = BSTATIC; }
| LEXTERN     { $$ = BEXTERN; }
| LREGISTER   { $$ = BREGISTER; }
| LINLINE     { $$ = 0; }
```

$\langle \text{function simplec } 126c \rangle \equiv$ (518c)

```
/*@Scheck: used by cc.y
int simplec(long b)
{
    b &= BCLASS;
    switch(b) {
    case 0:
    case BAUTO:
        return CAUTO;
    case BEXTERN:
        return CEXTERN;
    case BSTATIC:
        return CSTATIC;

    <simplec() cases 126d>
    }
    diag(Z, "illegal combination of classes %Q", b);
    return CXXX;
}
```

Uses `BAUTO` 121, `BCLASS` 124c, `BEXTERN` 121, `BSTATIC` 121, `CAUTO` 33b, `CEXTERN` 33b, `CSTATIC` 33b, `CXXX` 33b, `Z` 29a, and `diag()` 496b.

$\langle \text{simplec() cases } 126d \rangle \equiv$ (126c) 137h▷

```
// skipping register
case BREGISTER:
    return CXXX;
case BAUTO|BREGISTER:
    return CAUTO;
```

Uses `BAUTO` 121, `BREGISTER` 121, `CAUTO` 33b, and `CXXX` 33b.

8.8.3 Qualifiers

Type qualifiers (`const`, `volatile`) follow the same bitmask pattern. The `garbt`^{127c} function extracts the qualifier bits and, if any are set, copies the type before attaching them—because basic types like `types[TINT]` are shared singletons, so modifying one in place would corrupt all `int` declarations.

```
<Type_kind_bis qualifier cases 127a>≡ (120e)
    TCONSTNT,
    TVOLATILE,
```

```
<gname rule 127b>≡ (122c)
    gname: /* garbage words */
           LCONSTNT { $$ = BCONSTNT; }
    |     LVOLATILE { $$ = BVOLATILE; }
    |     LRESTRICT { $$ = 0; }
```

```
<function garbt 127c>≡ (518c)
    //@Scheck: used by cc.y
    Type* garbt(Type *t, long b)
    {
        Type *t1;

        if(b & BGARB) {
            t1 = copytyp(t);
            t1->garb = simpleg(b);
            return t1;
        }
        return t;
    }
```

Uses `BGARB` 124d, `copytyp()` 127d, and `simpleg()` 127e.

```
<function copytyp 127d>≡ (518c)
    Type*
    copytyp(Type *t)
    {
        Type *nt;

        nt = typ(TXXX, T); // could just do alloc(sizeof(Type))
        *nt = *t;
        return nt;
    }
```

Uses `T` 31i, `TXXX` 30a, and `typ()` 32a.

```
<function simpleg 127e>≡ (518c)
    int
    simpleg(long b)
    {

        b &= BGARB;
        switch(b) {
        case BCONSTNT:      return GCONSTNT;
        case BVOLATILE:     return GVOLATILE;
        case BVOLATILE|BCONSTNT: return GCONSTNT|GVOLATILE;
        }
        return GXXX;
    }
```

Uses `BCONSTNT` 121, `BGARB` 124d, `BVOLATILE` 121, `GCONSTNT` 33a, `GVOLATILE` 33a, and `GXXX` 33a.

8.8.4 Structures and unions

Structure and union parsing must handle forward declarations (`struct Foo *p` before `Foo` is defined), recursive types, and scoped redefinitions. The key function is `dotag`: it looks up the tag name in the symbol table, and if the tag is new or in a different scope, allocates an incomplete `Type` whose `link` is `T (nil)`. The `link` is filled in later when the body is parsed. The `snap` function resolves these incomplete pointers after the fact, since different variables may hold copies that did not share the same `Type` pointer.

Complex types

The `complex` rule handles struct, union, and enum types — the “complex” types that can contain definitions. Each returns a `Type` directly (not a bitset like simple types), which is then wrapped in the `types` rule for integration with the declaration machinery.

```

<types rule 128a>+≡ (123d) <124b
|  complex
|  {
|      $$ .c = CXXX;
|      $$ .t = $1;
|  }
|  complex gctnlist
|  {
|      $$ .t = $1;
|      $$ .c = simplec($2);
|      $$ .t = garbt($$ .t, $2);
|      if($2 & ~BCLASS & ~BGARB)
|          diag(Z, "duplicate types given: %T and %Q", $1, $2);
|  }
|  gcnlist complex zgcnlist
|  {
|      $$ .t = $2;
|      $$ .c = simplec($1);
|      $$ .t = garbt($$ .t, $1|$3);
|  }

<types, storage classes, qualifiers rules 128b>+≡ (101c) <125 131b>
<complex rule 128d>

<type declarations 128c>+≡ (100a) <122k 130e>
%type <type> complex

```

Structure/union uses

When the parser sees `struct Foo` (without a body), it calls `dotag` to look up or create the tag. If `Foo` was already defined, `dotag` returns its type with all fields; if not, it returns an incomplete type (forward declaration) that will be completed later when the definition is parsed.

```

<complex rule 128d>≡ (128b) 129c>
complex:
    LSTRUCT ltag
    {
        dotag($2, TSTRUCT, 0);
        $$ = $2->suetag;
    }
|  LUNION ltag
|  {
        dotag($2, TUNION, 0);
        $$ = $2->suetag;
    }

```

```

⟨function dotag 129a⟩≡ (525)
  //@Scheck: used by cc.y
  Type* dotag(Sym *s, int et, int bn)
  {
    ⟨dotag() locals 130a⟩

    ⟨dotag() if bn not zero and bn not sueblock 130b⟩
    // never defined before, return a newly allocated (but incomplete) Type
    if(s->suetag == T) {
      s->suetag = typ(et, T); // link is null for now
      s->sueblock = autobn;
    }
    ⟨dotag() sanity check tag redeclaration 129b⟩
    if(s->suetag->tag == S)
      s->suetag->tag = s;

    return s->suetag;
  }

```

Uses S 35d, T 31i, autobn 39h, and typ() 32a.

```

⟨dotag() sanity check tag redeclaration 129b⟩≡ (129a)
  if(s->suetag->etype != et)
    diag(Z, "tag used for more than one type: %s", s->name);

```

Uses Z 29a and diag() 496b.

Structure/union definitions

When the parser sees `struct Foo \ ... \`, it first calls `dotag` to create (or look up) the tag, then parses the body (`sbody`) to get the list of field types, and fills in `suetag->link` with that list. `salign` then walks the fields to compute each field's offset and the struct's total width, respecting alignment constraints. For unions, all fields start at offset zero and the width is the maximum field width.

```

⟨complex rule 129c⟩+≡ (128b) <128d 130d>
  | LSTRUCT ltag
  {
    dotag($2, TSTRUCT, autobn);
  }
  sbody
  {
    $$ = $2->suetag;
    if($$->link != T)
      diag(Z, "redeclare tag: %s", $2->name);
    $$->link = $4;
    ⟨complex rule, when parse a structure definition, align the struct 277b⟩
  }
  | LUNION ltag
  {
    dotag($2, TUNION, autobn);
  }
  sbody
  {
    $$ = $2->suetag;
    if($$->link != T)
      diag(Z, "redeclare tag: %s", $2->name);
    $$->link = $4;
    ⟨complex rule, when parse a structure definition, align the struct 277b⟩
  }
}

```

`<dotag() locals 130a>≡` (129a)

```
Decl *d;
```

`<dotag() if bn not zero and bn not sueblock 130b>≡` (129a)

```
if(bn != 0 && bn != s->sueblock) {
    d = push();
    d->sym = s;
    d->val = DSUE;
    // save old values
    d->type = s->suetag;
    d->block = s->sueblock;
    // prepare for new value
    s->suetag = T;
}
```

Uses DSUE 38a, T 31i, and push() 39e.

`<global taggen 130c>≡` (511d)

```
/*@Scheck: used by cc.y
int taggen;
```

`<complex rule 130d>+≡` (128b) <129c 132d>

```
| LSTRUCT sbody
{
    taggen++;
    sprintf(symb, "%d_", taggen);
    $$ = dotag(lookup(), TSTRUCT, autobn);
    $$->link = $2;
    <complex rule, when parse a structure definition, align the struct 277b>
}
| LUNION sbody
{
    taggen++;
    sprintf(symb, "%d_", taggen);
    $$ = dotag(lookup(), TUNION, autobn);
    $$->link = $2;
    <complex rule, when parse a structure definition, align the struct 277b>
}
```

Fields

Field parsing uses the `tlist` and `edecor` rules to get each field's type and name, then chains them into a linked list via `Type.down`. Each field `Type` carries the field name in `Type.sym`, the field type in `Type.link`, and (after alignment) the byte offset in `Type.offset`. Anonymous struct members (a Plan 9 extension) get no name, and their fields are promoted to the enclosing struct's namespace.

`<type declarations 130e>+≡` (100a) <128c

```
%type <type> sbody
```

`<global strf 130f>≡` (511d)

```
// list<ref_own<Type>> (next = Type.down, last = strl)
Type* strf;
```

`<global strl 130g>≡` (511d)

```
// ref<Type> (head = strf)
Type* strl;
```

⟨union yacc other fields 131a⟩+≡ (42b) <122i

```
struct
{
    Type*    t1; // save strf
    Type*    t2; // save strl
    Type*    t3; // save lasttype
    uchar    c;  // save lastclass
} tyty;
```

⟨types, storage classes, qualifiers rules 131b⟩+≡ (101c) <128b 134b>

```
sbody:
'{'
{
    // save
    $<tyty>$.t1 = strf;
    $<tyty>$.t2 = strl;
    $<tyty>$.t3 = lasttype;
    $<tyty>$.c = lastclass;

    strf = T;
    strl = T;
    ⟨sbody rule, initializations before parsing the fields 417d⟩
    lastclass = CXXX;
    lasttype = T;
}
edec1
'}'
{
    $$ = strf;

    // restore
    strf = $<tyty>2.t1;
    strl = $<tyty>2.t2;
    lasttype = $<tyty>2.t3;
    lastclass = $<tyty>2.c;
}
```

⟨structure element declarator rules 131c⟩≡ (101d) 132a>

```
/*
 * structure element declarator
 */
edec1:
    tlist
    {
        lasttype = $1;
    }
    zedlist ','
| edec1 tlist
    {
        lasttype = $2;
    }
    zedlist ','
```

⟨zedlist rule 427c⟩

```
edlist:
    edecor          { dodecl(edec1, CXXX, lasttype, $1); }
| edlist ',' edlist
```

<structure element declarator rules 132a>+≡ (101d) <131c
<edecor rule 132b>

<edecor rule 132b>≡ (132a) 418e▷
 edecor:
 xdecor
 {
 <edecor rule, set fields after parsed a field declarator 418a>
 }

<function edecl 132c>≡ (525)
 /// dodecl(<>) -> <>
 //@Scheck: used by cc.y
 void edecl(int c, Type *t, Sym *s)
 {
 Type *t1;

 <edecl() if unnamed structure element 427d>
 else
 if(c != CXXX)
 diag(Z, "structure element cannot have class: %s", s->name);

 t1 = t;
 t = copytyp(t1);
 t->sym = s;
 t->down = T;

 <edecl() if lastfield 419a>

 // add_list(t, strf, strl)
 if(strf == T)
 strf = t;
 else
 strl->down = t;
 strl = t;
 }

Uses CXXX 33b, T 31i, Z 29a, copytyp() 127d, diag() 496b, strf 130f, and strl 130g.

8.8.5 Enums

Enum values are resolved at parse time: **doenum** evaluates each constant expression, assigns the value to the symbol's **vconst** field, and uses **xdecl** to give the symbol type **TENUM**. Later, **tcomo** replaces **ONAME** references to enum symbols with **ONCONST** nodes, so by the time code generation runs, enums have been completely eliminated.

Enumeration type uses and definitions

Enum types share the **dotag** mechanism with structs and unions (they are in the same “tag” namespace). The parser creates a **TENUM** type whose **link** points to the underlying integer type (typically **TINT**, but widened to **TLONG** if a constant exceeds **TINT** range). Forward-declaring an enum is allowed but the constants must be defined before use.

<complex rule 132d>+≡ (128b) <130d 133c▷
 | LENUM ltag
 {
 dotag(\$2, TENUM, 0);
 \$\$ = \$2->suetag;
 if(\$\$->link == T) // default type
 \$\$->link = types[TINT];
 }

```

    $$ = $$->link;
}

⟨struct En 133a⟩≡ (504b)
    struct En
    {
        Type* tenum; /* type of entire enum */
        Type* cenum; /* type of current enum run */
        ⟨En value fields 180e⟩
    };

⟨global en 133b⟩≡ (511d)
    struct En en;

Uses En 133a.

⟨complex rule 133c⟩+≡ (128b) ◁132d 137j▷
| LENUM ltag
{
    dotag($2, TENUM, autobn);
}
'{'
{
    en.tenum = T;
    en.cenum = T;
}
enum
'}',
{
    $$ = $2->suetag;
    if($$->link != T)
        diag(Z, "redeclare tag: %s", $2->name);
    ⟨complex rule, after processed an enum, sanity check tenum 133d⟩
    $$->link = en.tenum;
    $$ = en.tenum;
}
| LENUM
'{'
{
    en.tenum = T;
    en.cenum = T;
}
enum
'}',
{
    $$ = en.tenum;
}

⟨complex rule, after processed an enum, sanity check tenum 133d⟩≡ (133c)
    if(en.tenum == T) {
        diag(Z, "enum type ambiguous: %s", $2->name);
        en.tenum = types[TINT];
    }

```

Enumeration constant definitions

Each enum constant is installed in the symbol table by `doenum`: `Sym.type` is set to `TENUM` (so the typedef trick does not trigger), `Sym.tenum` records the underlying integer type, and `Sym.vconst` stores the constant's

value. If an explicit initializer is given (`FOO = 42`), `evconst` evaluates it at compile time; otherwise, the value auto-increments from the previous constant. The global `en.ceession` tracks the auto-increment state.

```
⟨Sym enum value fields 134a⟩≡ (34c) 180h▷
// ref<Type>
Type* tenum;
```

```
⟨types, storage classes, qualifiers rules 134b⟩+≡ (101c) ◁131b
enum:
    LNAME          { doenum($1, Z); }
|   LNAME '=' expr { doenum($1, $3); }

|   enum ',' enum
|   enum ','
```

```
⟨function doenum 134c⟩≡ (525)
/*@Scheck: used by cc.y
void doenum(Sym *s, Node *n)
{

    if(n) {
        ⟨doenum() typecheck and evaluate expression 180f⟩
        if(n->op != OCONST) {
            diag(n, "enum not a constant: %s", s->name);
            return;
        }

        en.cenum = n->type; // inferred type
        en.tenum = maxtype(en.cenum, en.tenum);
        ⟨doenum() save current value of enumeration constant 180g⟩
    }
    if(dclstack)
        push1(s); // will be reverted once out of scope

    // check for redeclaration and set s->type to TENUM
    xdecl(CXXX, types[TENUM], s);

    if(en.cenum == T) {
        en.cenum = types[TINT];
        en.tenum = types[TINT];
        en.lastenum = 0;
    }
    s->tenum = en.cenum; // Sym.tenum, not En.tenum here

    ⟨doenum() set value for symbol 181a⟩

    ⟨doenum() debug declaration 490d⟩
    ⟨doenum() debugging support 401b⟩
}
```

Uses CXXX 33b, OCONST 111a, T 31i, TENUM 31c, TINT 30b, dclstack 39a, diag() 496b, en 133b, maxtype() 181b, push1() 39f, types 32b, and xdecl() 104e.

8.8.6 Pointer and array types

C’s declaration syntax splits a type across two places: the base type (“type specifiers”) and the declarator (pointers, arrays, function signatures). The `xdecor` rules build a temporary AST of `OIND`, `OARRAY`, and `OFUNC`

nodes that `dodecl()` then walks in a loop, wrapping the base type with `TIND`, `TARRAY`, or `TFUNC` at each step and descending via `n = n->left` to reach the name.

```
<xdecor rule 135a>+≡ (101e) <103c>
|   '*' zgnlist xdecor
|   {
|       $$ = new(OIND, $3, Z);
|       $$->nodegarb = simpleg($2);
|   }
```

```
<Node parsing helper fields 135b>≡ (28)
// enum<qualifier>
char nodegarb;
```

```
<dodecl() switch node kind cases 135c>+≡ (103a) <104d 135f>
case OIND:
    t = typ(TIND, t);
    t->garb = n->nodegarb;
    n = n->left;
    goto loop;
```

Uses `OIND` 116e, `TIND` 31c, and `typ()` 32a.

```
<xdecor2 rule 135d>+≡ (101e) <103d 135g>
|   xdecor2 '[' zexpr ']' { $$ = new(OARRAY, $1, $3); }
```

```
<dodecl() locals 135e>+≡ (103a) <104c 181c>
Node *n1;
```

```
<dodecl() switch node kind cases 135f>+≡ (103a) <135c 135i>
case OARRAY:
    t = typ(TARRAY, t);
    n1 = n->right; // possible size
    n = n->left;
    <dodecl() switch node kind cases, case OARRAY, if array has a size 181d>
    goto loop;
```

Uses `OARRAY` 119i, `TARRAY` 31c, and `typ()` 32a.

8.8.7 Function types, parameter types

Function parameter lists are parsed into `OPROTO` and `OLIST` nodes by the argument grammar, then `fnproto`^{136a} converts them into a linked list of `Type` nodes (via `Type.down`) representing the parameter types. The `OFUNC` node in the declarator is temporary—`dodecl()` transforms it into a `TFUNC` type, just as it transforms `OIND` into `TIND`.

```
<xdecor2 rule 135g>+≡ (101e) <135d>
|   xdecor2 '(' zarglist ')' { $$ = new(OFUNC, $1, $3); }
```

```
<parameter declarator rules 135h>≡ (101d) 136c>
zarglist:
    /* empty */ { $$ = Z; }
|   arglist { $$ = invert($1); }
```

```
<dodecl() switch node kind cases 135i>+≡ (103a) <135f 418f>
case OFUNC:
    t = typ(TFUNC, t);
    t->down = fnproto(n);
    n = n->left;
    goto loop;
```

Uses `OFUNC` 117c, `TFUNC` 31c, `fnproto()` 136a, and `typ()` 32a.

⟨function fnproto 136a⟩≡ (525)

```
Type*
fnproto(Node *n)
{
  ⟨fnproto() old prototype style handling 424a⟩
  return fnproto1(n->right);
}
```

Uses fnproto1() 136b.

⟨function fnproto1 136b⟩≡ (525)

```
Type*
fnproto1(Node *n)
{
  Type *t;

  if(n == Z)
    return T;

  switch(n->op) {
    ⟨fnproto1() switch node kind cases 136e⟩
  }
  diag(n, "unknown op in fnproto");
  return T;
}
```

Uses T 31i, Z 29a, and diag() 496b.

⟨parameter declarator rules 136c⟩+≡ (101d) <135h 422b>
⟨arglist rule 136d⟩

⟨arglist rule 136d⟩≡ (136c) 136g>
 arglist:
 name

⟨fnproto1() switch node kind cases 136e⟩≡ (136b) 136h>
 case ONAME:
 diag(n, "incomplete argument prototype");
 return typ(TINT, T);

Uses ONAME 27, T 31i, TINT 30b, diag() 496b, and typ() 32a.

⟨declaration nodes 136f⟩+≡ (27) <119h 137c>
 OPROTO,

⟨arglist rule 136g⟩+≡ (136c) <136d 137a>
 | tlist xdecor
 {
 \$\$ = new(OPROTO, \$2, Z);
 \$\$->type = \$1;
 }
 | tlist abdecor
 {
 \$\$ = new(OPROTO, \$2, Z);
 \$\$->type = \$1;
 }

⟨fnproto1() switch node kind cases 136h⟩+≡ (136b) <136e 137b>
 case OPROTO:
 lastdcltype = T; // dead?
 dodecl(NODECL, CXXX, n->type, n->left);
 t = typ(TXXX, T);
⟨fnproto1() when OPROTO case, adjust parameter type 182a⟩
 return t;

Uses CXXX 33b, NODECL 118d, OPROTO 136f, T 31i, TXXX 30a, dodecl() 103a, lastdcltype 103b, and typ() 32a.

<arglist rule 137a>+≡ (136c) <136g 137d>
 | arglist ',' arglist { \$\$ = new(OLIST, \$1, \$3); }

<fnproto1() switch node kind cases 137b>+≡ (136b) <136h 137e>
 case OLIST:
 t = fnproto1(n->left);
 if(t != T)
 t->down = fnproto1(n->right);
 return t;

Uses OLIST 29b, T 31i, and fnproto1() 136b.

<declaration nodes 137c>+≡ (27) <136f 418d>
 ODOTDOT,

<arglist rule 137d>+≡ (136c) <137a>
 | '','.' '','.' '','.' { \$\$ = new(ODOTDOT, Z, Z); }

<fnproto1() switch node kind cases 137e>+≡ (136b) <137b>
 case ODOTDOT:
 return typ(TDOT, T);

Uses ODOTDOT 137c, T 31i, TDOT 137f, and typ() 32a.

<Type_kind other cases 137f>≡ (30a)
 TDOT, // ... in function types

8.8.8 Typedefs

typedef is treated as a storage class (CTYPEDEF) in the grammar, so **typedef int Foo;** goes through the same **dodecl** machinery as **static int x;**. The symbol **Foo** ends up in the symbol table with **class == CTYPEDEF** and a **type** pointing to **int**. The lexer then recognizes **Foo** as a **LTYPE** token in subsequent code, which is what makes **typedef** names work as type specifiers in the grammar.

<cname rule 137g>+≡ (122c) <126b 434c>
 | LTYPEDEF { \$\$ = BTYPEDEF; }

<simplec() cases 137h>+≡ (126c) <126d 432c>
 case BTYPEDEF:
 return CTYPEDEF;

Uses BTYPEDEF 121 and CTYPEDEF 137i.

<Storage_class cases 137i>≡ (33b) 415b>
 CTYPEDEF,

<complex rule 137j>+≡ (128b) <133c>
 | LTYPE { \$\$ = tcopy(\$1->type); }

8.9 Declarations and definitions, part two

8.9.1 Globals, external declarator

8.9.2 Function parameters

When a function definition (not just a prototype) is parsed, **argmark**^{138a} walks the parameter list and pushes each parameter name into the symbol table scope via **push1()**, with **offset == -1** as a sentinel. Later, **pdecl**^{139c}

(called through `dodecl`) checks this sentinel to confirm the symbol is indeed a parameter, then delegates to `adec1` because parameters are essentially local variables with `CPARAM` storage class.

```

⟨function argmark 138a⟩≡ (525)
  /// yyparse -> xdecl -> <>
  //@Scheck: not dead, used by cc.y
  void argmark(Node *n, int pass)
  {
    Type *t;

    ⟨argmark() initializations 197a⟩

    for(; n->left != Z; n = n->left) {
      if(n->op != OFUNC || n->left->op != ONAME)
        continue;
      // else, OFUNC with ONAME on n->left, so analyze the parameters
      walkparam(n->right, pass);
      ⟨argmark() if old proto style 423b⟩
      break;
    }

    ⟨argmark() finalizations 197b⟩
  }

```

Uses `OFUNC` 117c, `ONAME` 27, `Z` 29a, and `walkparam()` 138b.

```

⟨function walkparam 138b⟩≡ (525)
  void
  walkparam(Node *n, int pass)
  {
    Sym *s;
    Node *n1;

    if(n != Z && n->op == OPROTO && n->left == Z && n->type == types[TVOID])
      return;

loop:
    if(n == Z)
      return;
    switch(n->op) {
    case OLIST:
      walkparam(n->left, pass);
      n = n->right;
      goto loop;

    case OPROTO:
      for(n1 = n; n1 != Z; n1=n1->left)
        if(n1->op == ONAME) {
          if(pass == 0) {
            s = n1->sym;
            push1(s);
            s->offset = -1;
            break;
          }
          dodecl(pdecl, CPARAM, n->type, n->left);
          break;
        }

      if(n1)
        break;
      // else no name found
    }
  }

```

```

    <walkparam() when OPROTO case, if no name found 139b>
    break;

```

```

    <walkparam() switch op cases 139a>
    default:
        diag(n, "argument not a name/prototype: %0", n->op);
        break;

```

```

    }

```

Uses CPARAM 33b, OLIST 29b, ONAME 27, OPROTO 136f, TVOID 31b, Z 29a, diag() 496b, dodecl() 103a, pdecl() 139c, push1() 39f, types 32b, and walkparam() 138b.

```

<walkparam() switch op cases 139a>≡ (138b) 424c▷
    case ODOTDOT:
        break;

```

Uses ODOTDOT 137c.

```

<walkparam() when OPROTO case, if no name found 139b>≡ (138b)
    if(pass == 0) {
        /*
         * extension:
         * allow no name in argument declaration
         diag(Z, "no name in argument declaration");
         */
        break;
    }
    // else pass == 1
    <walkparam() when OPROTO case, if no name found and old style decl 423c>

```

```

<function pdecl 139c>≡ (525)
    /// dodecl(<>, ...) -> <>
    void
    pdecl(int class, Type *t, Sym *s)
    {
        <pdecl() sanity check s 139d>
        <pdecl() adjust type 182b>
        <pdecl() adjust class 144c>
        <pdecl() sanity check type 182d>
        adecl(class, t, s);
    }

```

Uses adecl() 140c.

```

<pdecl() sanity check s 139d>≡ (139c)
    if(s && s->offset != -1) {
        diag(Z, "not a parameter: %s", s->name);
        return;
    }

```

Uses Z 29a and diag() 496b.

8.9.3 Locals, automatic declarator

The `adecl` rules handle local variable declarations inside function bodies. Unlike global declarations (`xdecl`), local declarations return AST nodes for any initializers (e.g., `int x = 5;` yields an assignment node), because these must execute at runtime. The `adecl`^{140c} callback function pushes the variable into the current scope with

`push1()` and assigns its storage class—typically `CAUTO` for stack locals or `CLOCAL` for static locals (which get file-scope storage but block-scope visibility).

⟨automatic declarator rules 140a⟩≡ (101d) 140b▷

```
/*
 * automatic declarator
 */
adec1:
    ctlist ';'          { $$ = dodecl(adec1, lastclass, lasttype, Z); }
|   ctlist adlist ';' { $$ = $2; }
```

⟨automatic declarator rules 140b⟩+≡ (101d) ◁140a

```
adlist:
    xdecor
    {
        dodecl(adec1, lastclass, lasttype, $1);
        $$ = Z;
    }
|   xdecor
    {
        $1 = dodecl(adec1, lastclass, lasttype, $1);
    }
    '=' init
    {
        long w;

        w = $1->sym->type->width;
        $$ = doinit($1->sym, $1->type, OL, $4);
        ⟨adlist rule, after doinit 268a⟩
    }
|   adlist ',' adlist
    {
        $$ = $1;
        if($3 != Z) {
            $$ = $3;
            if($1 != Z)
                $$ = new(OLIST, $1, $3);
        }
    }
```

⟨function adec1 140c⟩≡ (525)

```
/// pdecl | dodecl(<>) -> <>
void
adec1(int c, Type *t, Sym *s)
{

    ⟨adec1() adjust storage to CLOCAL if static local variable 415c⟩
    ⟨adec1() adjust storage if function type 144d⟩
    ⟨adec1() adjust storage 144e⟩

    if(s) {
        ⟨adec1() adjust storage if symbol 145a⟩

        if(c != CPARAM)
            push1(s); // CPARAM pushed already in walkparam()

        s->block = autobn;
        s->type = t;
        s->class = c;
        s->offset = 0;
```

```

    <adec1() initialize symbol 183c>
}

switch(c) {
<adec1() switch class cases 225a>
}
}

```

Uses CPARAM 33b, autobn 39h, and push1() 39f.

8.9.4 Types, abstract declarator

<abstract declarator rules 141a>≡ (101d) 141b▷

```

/*
 * abstract declarator
 */
abdecor:
/* empty */ { $$ = Z; }
| abdecor1

```

<abstract declarator rules 141b>+≡ (101d) ◁141a 141c▷

```

abdecor1:
    '*' zgnlist
    {
        $$ = new(OIND, (Z), Z);
        $$->nodegarb = simpleg($2);
    }
| '*' zgnlist abdecor1
    {
        $$ = new(OIND, $3, Z);
        $$->nodegarb = simpleg($2);
    }
| abdecor2

```

<abstract declarator rules 141c>+≡ (101d) ◁141b

```

abdecor2:
    abdecor3
| abdecor2 '(' zarglist ')' { $$ = new(OFUNC, $1, $3); }
| abdecor2 '[' zexpr ']' { $$ = new(OARRAY, $1, $3); }

abdecor3:
    '(' ')' { $$ = new(OFUNC, Z, Z); }
| '[' zexpr ']' { $$ = new(OARRAY, Z, $2); }
| '(' abdecor1 ')' { $$ = $2; }

```

Chapter 9

Checking

Type checking, symbol resolution, and lint-like warnings are interleaved with parsing rather than forming a separate pass. This chapter collects the checking code scattered across the parser and code generator: type compatibility checks (including pointer/integer conversions), unused variable warnings, duplicate declarations, and format string validation (`#pragma argcheck`).

9.1 Overview

Checking in 5c falls into four categories: (1) *symbol resolution* — matching uses to definitions via the scope stack, catching undefined and unused identifiers; (2) *storage merging* — reconciling multiple declarations of the same global (e.g., `extern` then definition); (3) *type checking* — verifying that operand types are compatible, inserting implicit conversions, and annotating the AST with result types; (4) *lint-like warnings* — empty `if` bodies, division by zero, dead code, format string mismatches. These checks are scattered across the parser, `dodecl`, `revertdecl`, and the type checker `tcomo/tcom`.

9.2 Printing warnings: 5c -w and 5c -W

Warnings are only printed when `-w` (or `-W` for verbose mode) is given on the command line. This means 5c is silent by default—a design choice that keeps build output clean but requires the programmer to opt in to diagnostics. With `-w -v`, the warning also dumps the AST subtree that triggered it, which is invaluable for debugging the compiler itself.

```
<function warn 142>≡ (518c)
void
warn(Node *n, char *fmt, ...)
{
    char buf[STRINGSZ];
    va_list arg;

    if(debug['w'] || debug['W']) {
        va_start(arg, fmt);
        vseprint(buf, buf+sizeof(buf), fmt, arg);
        va_end(arg);
        <warn() if -W 143a>
        // else
        Bprint(&diagbuf, "warning: %L %s\n", (n==Z) ? nearln : n->lineno, buf);
        <warn() if -v 143b>
    }
}
```

Uses STRINGSZ 81e, Z 29a, debug 474a, diagbuf 47f, and nearln 29g.

```

<warn() if -W 143a>≡
    if(debug['W']) {
        diag(n, "%s", buf);
        return;
    }

```

(142)

Uses debug 474a and diag() 496b.

```

<warn() if -v 143b>≡
    if(n != Z)
        if(debug['v'])
            prtreen(n, "warning");

```

(142)

Uses Z 29a, debug 474a, and prtreen() 487b.

9.3 Symbol resolution

Scoping in 5c is handled entirely during parsing, using the Decl stack and the `markdecl()/revertdecl()` mechanism described earlier. Since C requires forward declarations for everything, all type information is already in the symbol table by the time the typechecker runs—cross-file consistency is left to the linker.

9.4 Storage resolution

C allows the same global name to be declared multiple times (e.g., `extern int x` in a header, then `int x = 1` in a source file). The compiler must merge these declarations, checking for consistency and adjusting the storage class. The function `xdecl` handles this: if a symbol was previously `CEXTERN` and is now being defined, it is promoted to `CGLOBL`. If the types conflict, an error is issued.

9.4.1 xdecl

```

<xdecl() switch class cases 143c>≡
    case CXXX:
        class = CGLOBL;
        if(s->class == CEXTERN)
            s->class = CGLOBL;
        break;

```

(104e) 143d▷

Uses CEXTERN 33b, CGLOBL 33b, and CXXX 33b.

```

<xdecl() switch class cases 143d>+≡
    case CEXTERN:
        if(s->class == CGLOBL)
            class = CGLOBL;
        break;

```

(104e) ◁143c 143e▷

Uses CEXTERN 33b and CGLOBL 33b.

```

<xdecl() switch class cases 143e>+≡
    case CAUTO:
        diag(Z, "overspecified class: %s %s %s", s->name, cnames[class], cnames[s->class]);
        class = CEXTERN;
        break;

```

(104e) ◁143d 432d▷

Uses CAUTO 33b, CEXTERN 33b, Z 29a, cnames 485a, and diag() 496b.


```

⟨xdecl() sanity check class after switch class 144a⟩≡ (104e)
    if(s->class == CSTATIC)
        if(class == CEXTERN || class == CGLOBL) {
            warn(Z, "overspecified class: %s %s %s", s->name, cnames[class], cnames[s->class]);
            class = CSTATIC;
        }

```

Uses CEXTERN 33b, CGLOBL 33b, CSTATIC 33b, Z 29a, cnames 485a, and warn() 142.

9.4.2 doinit

If a global was previously declared `extern` and is now being initialized, `doinit` silently promotes it to `CGLOBL`. This handles the common pattern of `extern int x;` in a header followed by `int x = 1;` in the source file.

```

⟨doinit() possibly adjust class 144b⟩≡ (120c)
    if(s->class == CEXTERN) {
        s->class = CGLOBL;
        ⟨doinit() debug declaration 490c⟩
    }
    ⟨doinit() debug initialization 494a⟩

```

Uses CEXTERN 33b and CGLOBL 33b.

9.4.3 pdecl

Function parameters must have storage class `CPARAM`. If no class was specified (`CXXX`), it defaults to `CPARAM`; any other class is an error.

```

⟨pdecl() adjust class 144c⟩≡ (139c)
    if(class == CXXX)
        class = CPARAM;
    if(class != CPARAM) {
        diag(Z, "parameter cannot have class: %s", s->name);
        class = CPARAM;
    }

```

Uses CPARAM 33b, CXXX 33b, Z 29a, and diag() 496b.

9.4.4 adecl

Local declarations default to `CAUTO` when no storage class is given. Functions cannot be `auto` or `extern register`, and a local redeclaration of a `static` as `extern` or `global` is corrected back to `static` with a warning.

```

⟨adecl() adjust storage if function type 144d⟩≡ (140c)
    if(t->etype == TFUNC) {
        if(c == CXXX)
            c = CEXTERN;
        if(c == CLOCAL)
            c = CSTATIC;
        if(c == CAUTO || c == CEXREG)
            diag(Z, "function cannot be %s %s", cnames[c], s->name);
    }

```

Uses CAUTO 33b, CEXREG 432a, CEXTERN 33b, CLOCAL 415b, CSTATIC 33b, CXXX 33b, TFUNC 31c, Z 29a, cnames 485a, and diag() 496b.

```

⟨adecl() adjust storage 144e⟩≡ (140c)
    if(c == CXXX)
        c = CAUTO;

```

Uses CAUTO 33b and CXXX 33b.

```
<adec1() adjust storage if symbol 145a>≡ (140c)
```

```
if(s->class == CSTATIC)
    if(c == CEXTERN || c == CGLOBL) {
        warn(Z, "just say static: %s", s->name);
        c = CSTATIC;
    }
if(s->class == CAUTO || s->class == CPARAM || s->class == CLOCAL)
if(s->block == autobn)
    diag(Z, "auto redeclaration of: %s", s->name);
```

Uses CAUTO 33b, CEXTERN 33b, CGLOBL 33b, CLOCAL 415b, CPARAM 33b, CSTATIC 33b, Z 29a, autobn 39h, diag() 496b, and warn() 142.

9.5 Typechecking helpers

9.5.1 Overview

Type checking in 5c uses two distinct notions: *equality* (**sametype**), which checks structural identity of two types, and *compatibility* (**tcompat**), which checks whether a value of one type can be used where another is expected (with implicit conversions). Compatibility is specified by policy tables (**tadd**, **tsub**, **trel**, **tcast**, etc.) that enumerate which type combinations are legal for each operator. The main entry point for expression type-checking is **tcom**, which is called from **complex** during code generation.

```
<xdecl() sanity check type after switch class 145b>≡ (104e)
if(s->type != T)
    if(s->class != class || !sametype(t, s->type) || t->etype == TENUM) {
        diag(Z, "external redeclaration of: %s", s->name);
        Bprint(&diagbuf, " %s %T %L\n", cnames[class], t, nearln);
        Bprint(&diagbuf, " %s %T %L\n", cnames[s->class], s->type, s->varlineno);
    }
```

Uses T 31i, TENUM 31c, Z 29a, cnames 485a, diag() 496b, diagbuf 47f, nearln 29g, and sametype() 145c.

9.5.2 Type equality: sametype()

sametype checks structural type equality: two types are the same if they have the same shape (same **etype**, same pointed-to type, same number and types of parameters, same fields in the same order). The recursion is depth-limited to 5 to handle mutually recursive structures that would cause an infinite loop via pointer cycles in the type graph.

```
<function sametype 145c>≡ (525)
/// xdecl -> <>
bool
sametype(Type *t1, Type *t2)
{
    if(t1 == t2)
        return true;
    return rsametype(t1, t2, 5, true);
}
```

Uses rsametype() 145d.

```
<function rsametype 145d>≡ (525)
bool
rsametype(Type *t1, Type *t2, int n, bool f)
{
    int et;
```

```

n--;
for(;;) {
    if(t1 == t2)
        return true;
    if(t1 == T || t2 == T)
        return false;
    <rsametype() if n less than zero 146a>

    et = t1->etype;
    if(et != t2->etype)
        return false;

    <rsametype() check function type equality 146b>
    <rsametype() check array type equality 146c>
    <rsametype() check structure type equality 147a>

    // recurse
    t1 = t1->link;
    t2 = t2->link;

    <rsametype() if f and not -V, allow void star 147c>
}
}

```

Uses T 31i.

```

<rsametype() if n less than zero 146a>≡ (145d)
    if(n <= 0)
        return true;

```

```

<rsametype() check function type equality 146b>≡ (145d)
    if(et == TFUNC) {
        if(!rsametype(t1->link, t2->link, n, false))
            return false;
        t1 = t1->down;
        t2 = t2->down;
        while(t1 != T && t2 != T) {

            <rsametype() continue if old style type 425a>

            while(t1 != T || t2 != T) {
                if(!rsametype(t1, t2, n, false))
                    return false;
                t1 = t1->down;
                t2 = t2->down;
            }
            break;
        }
        return true;
    }
}

```

Uses T 31i, TFUNC 31c, and rsametype() 145d.

```

<rsametype() check array type equality 146c>≡ (145d)
    if(et == TARRAY)
        if(t1->width != t2->width && t1->width != 0 && t2->width != 0)
            return false;

```

Uses TARRAY 31c.

```

⟨rsametype() check structure type equality 147a⟩≡ (145d)
    if(typesu[et]) {
        if(t1->link == T)
            snap(t1);
        if(t2->link == T)
            snap(t2);

        if(t1 != t2 && t1->link == T && t2->link == T){
            /* structs with missing or different tag names aren't considered equal */
            if(t1->tag == nil || t2->tag == nil ||
                strcmp(t1->tag->name, t2->tag->name) != 0)
                return false;
        }
        // equal if have same types (but do not care about field names)
        t1 = t1->link;
        t2 = t2->link;
        for(;;) {
            if(t1 == t2)
                return true;
            if(!rsametype(t1, t2, n, false))
                return false;
            t1 = t1->down;
            t2 = t2->down;
        }
    } // end su

```

Uses T 31i, rsametype() 145d, snap() 147b, and typesu 149a.

```

⟨function snap 147b⟩≡ (525)
    void
    snap(Type *t)
    {
        if(typesu[t->etype])
            if(t->link == T && t->tag && t->tag->suetag) {
                t->link = t->tag->suetag->link;
                t->width = t->tag->suetag->width;
            }
    }

```

Uses T 31i and typesu 149a.

9.5.3 generic void* pointer conversions: 5c -V

By default, 5c silently allows `void*` to match any pointer type, which is essential for functions like `pwrite` that take a `void*` buffer. The `-V` flag disables this leniency, requiring explicit casts. The `f` parameter restricts the match to the top-level pointer only—a `void**` will not silently match an `int**`.

```

⟨rsametype() if f and not -V, allow void star 147c⟩≡ (145d)
    if((f || !debug['V']) && et == TIND) {
        if(t1 != T && t1->etype == TVOID)
            return true;
        if(t2 != T && t2->etype == TVOID)
            return true;
    }

```

Uses T 31i, TIND 31c, TVOID 31b, and debug 474a.

9.5.4 Type merge: tmerge()

When a global is declared multiple times (e.g., a forward declaration in a header followed by a definition), `tmerge` walks both type chains in parallel and patches the older one. The key case is functions declared with old-style

(unprototyped) parameter lists that are later given a full prototype, and arrays declared with incomplete size (`extern int a[]`) that are later given a concrete width.

```
<xdecl() merge type declarations 148a>≡ (104e)
    tmerge(t, s);
```

Uses `tmerge()` 148b.

```
<function tmerge 148b>≡ (525)
    /// dodectl(xdecl,...) -> xdecl -> <>
    void
    tmerge(Type *t1, Sym *s)
    {
        Type *t2 = s->type;
        Type *ta, *tb;

        /*print("merge %T; %T\n", t1, t2);*/
        for(;;) {
            if(t1 == T || t2 == T || t1 == t2)
                break;
            if(t1->etype != t2->etype)
                break;
            switch(t1->etype) {
                <tmerge() switch etype t1 cases 425b>
                case TARRAY:
                    /* should we check array size change? */
                    if(t2->width > t1->width)
                        t1->width = t2->width;
                    break;

                case TUNION:
                case TSTRUCT:
                    return;
            }
            // recurse
            t1 = t1->link;
            t2 = t2->link;
        }
    }
```

Uses `T` 31i, `TARRAY` 31c, `TSTRUCT` 31c, and `TUNION` 31c.

9.5.5 Set of types constants: `typexxx[]`

The type checker needs to quickly test whether a type belongs to a category (“is this an integer?”, “is this a struct/union?”). Rather than writing switch statements everywhere, 5c uses boolean arrays indexed by `Type_kind`: `typei` is true for all integer types, `typeu` for unsigned types, `typesu` for struct/union, `typefd` for float/double, and so on. These arrays are filled at startup from initializer lists in `tinit`.

```
<global typei 148c>≡ (518c)
    // set<Type_kind>
    char typei[NTYPE];
```

Uses `NTYPE` 30a.

```
<global typeiinit 148d>≡ (518c)
    int typeiinit[] =
    {
        TCHAR, TCHAR, TSHORT, TUSHORT, TINT, TUINT, TLONG, TULONG, TVLONG, TUVLONG, -1,
    };
```

Uses `TCHAR` 30b, `TINT` 30b, `TLONG` 30b, `TSHORT` 30b, `TCHAR` 30b, `TUINT` 30b, `TULONG` 30b, `TUSHORT` 30b, `TUVLONG` 30b, and `TVLONG` 30b.

<global typesu 149a>≡ (518c)

```
char typesu[NTYPE];
```

Uses NTYPE 30a.

<global typesuinit 149b>≡ (518c)

```
int typesuinit[] =
{
    TSTRUCT, TUNION, -1,
};
```

Uses TSTRUCT 31c and TUNION 31c.

<global typefd 149c>≡ (518c)

```
char typefd[NTYPE];
```

Uses NTYPE 30a.

<global typefdinit 149d>≡ (518c)

```
int typefdinit[] =
{
    TFLOAT, TDOUBLE, -1,
};
```

Uses TDOUBLE 31a and TFLOAT 31a.

<tinit() initialise typexxx type sets 149e>≡ (50a) 518b▷

```
for(ip=typeiinit; *ip>=0; ip++) {
    urk("typei", nelem(typei), *ip);
    typei[*ip] = 1;
}
for(ip=typeuinit; *ip>=0; ip++) {
    urk("typeu", nelem(typeu), *ip);
    typeu[*ip] = 1;
}
for(ip=typesuvinit; *ip>=0; ip++) {
    urk("typesuv", nelem(typesuv), *ip);
    typesuv[*ip] = true;
}
for(ip=typeilpinit; *ip>=0; ip++) {
    urk("typeilp", nelem(typeilp), *ip);
    typeilp[*ip] = 1;
}
for(ip=typechlinit; *ip>=0; ip++) {
    urk("typechl", nelem(typechl), *ip);
    typechl[*ip] = 1;
    typechlv[*ip] = 1;
    //typechlvp[*ip] = 1;
}
for(ip=typechlpinit; *ip>=0; ip++) {
    urk("typechlp", nelem(typechlp), *ip);
    typechlp[*ip] = 1;
    //typechlv[*ip] = 1;
}
for(ip=typechlpfdinit; *ip>=0; ip++) {
    urk("typechlpfd", nelem(typechlpfd), *ip);
    typechlpfd[*ip] = 1;
}
for(ip=typevinit; *ip>=0; ip++) {
    urk("typev", nelem(typev), *ip);
    typev[*ip] = 1;
    typechlv[*ip] = 1;
    //typechlv[*ip] = 1;
```

```

}
for(ip=typefdinit; *ip>=0; ip++) {
    urk("typefd", nelem(typefd), *ip);
    typefd[*ip] = 1;
}
for(ip=typeafinit; *ip>=0; ip++) {
    urk("typeaf", nelem(typeaf), *ip);
    typeaf[*ip] = 1;
}
for(ip=typesuinit; *ip >= 0; ip++) {
    urk("typesu", nelem(typesu), *ip);
    typesu[*ip] = 1;
}

```

Uses typeaf 154b, typeafinit 154c, typechl 180b, typechlinit 180c, typechlp 215g, typechlpfd 357a, typechlpfdinit 357b, typechlpinit 215h, typechlv 517a, typefd 149c, typefdinit 149d, typei 148c, typeiinit 148d, typeilp 215e, typeilpinit 215f, typesu 149a, typesuinit 149b, typesuv 271c, typesuvinit 271d, typeu 231h, typeuinit 232a, typev 280a, typevinit 280b, and urk() 50c.

9.5.6 Type compatibility: tcompat()

tcompat checks whether two types are compatible for a given operation. Rather than hard-coding rules for each operator, it uses a table-driven approach: each operator has a “compatibility policy” (ttab) that encodes which type-category combinations are legal as a bitmask. stcompat encodes both types as bits and looks them up in the policy table. If the combination is not in the table, the types are incompatible and tcompat reports an error.

```

⟨function tcompat 150a⟩≡ (518c)
/// (complex -> tcom -> tcomo) | bcomplex -> <>
bool
tcompat(Node *n, Type *t1, Type *t2, long ttab[])
{
    if(stcompat(n, t1, t2, ttab)) {
        ⟨tcompat() if no T1 150b⟩
        else
            diag(n, "incompatible types: \"%T\" and \"%T\" for op \"%0\"",
                t1, t2, n->op);
        return true;
    }
    return false;
}

```

Uses diag() 496b and stcompat() 150c.

```

⟨tcompat() if no T1 150b⟩≡ (150a)
if(t1 == T)
    diag(n, "incompatible type: \"%T\" for op \"%0\"",
        t2, n->op);

```

Uses T 31i and diag() 496b.

```

⟨function stcompat 150c⟩≡ (518c)
/// tcompat -> <>
bool
stcompat(Node *n, Type *t1, Type *t2, long ttab[])
{
    int i;
    //Type_kind (as a bit)
    ulong b;

```

```

i = 0;
if(t2 != T)
    i = t2->etype;
b = 1L << i;
i = 0;
if(t1 != T)
    i = t1->etype;
// type t2 part of set of types compatible with type t1
if(b & ttab[i]) {
    <stcompat() when t1 matches t2, if ttab is tsign 167a>
    <stcompat() when t1 matches t2, if pointers and not a cast op 151a>
    return false; // true means error, so false means everything fine
}
return true;
}

```

Uses T 31i.

```

<stcompat() when t1 matches t2, if pointers and not a cast op 151a>≡ (150c)
if(n->op != OCAST)
    if(b == BIND && i == TIND)
        if(!sametype(t1, t2))
            return true;

```

Uses BIND 121, OCAST 118a, TIND 31c, and sametype() 145c.

9.5.7 Compatibility policies: txxx

Each C operator has its own compatibility policy — a bitset array indexed by `Type_kind` that specifies which type combinations are legal. For example, `tadd` allows integer+integer, integer+pointer, and pointer+integer but not pointer+pointer; `tsub` additionally allows pointer−pointer (which yields an integer offset). `tcompat` looks up both operand types in the policy table and calls `stcompat` for the detailed structural check.

```

<Bxxx constants 151b>+≡ (121) <124d
    BINTEGER = BCHAR|BUCHAR|BSHORT|BUSHORT|BINT|BUINT|BLONG|BULONG|BVLONG|BUVLONG,
    BNUMBER  = BINTEGER | BFLOAT|BDOUBLE,

```

Uses BCHAR 121, BDOUBLE 121, BFLOAT 121, BINT 121, BINTEGER 151b, BLONG 121, BSHORT 121, BUCHAR 121, BUINT 121, BULONG 121, BUSHORT 121, BUVLONG 121, and BVLONG 121.

```

<global tadd 151c>≡ (518c)
    long tadd[NTYPE];

```

Uses NTYPE 30a.

```

<global taddinit 151d>≡ (518c)
    Init taddinit[] =
    {
        TCHAR,  BNUMBER|BIND, 0,
        TUCAR,  BNUMBER|BIND, 0,
        TSHORT, BNUMBER|BIND, 0,
        TUSHORT, BNUMBER|BIND, 0,
        TINT,   BNUMBER|BIND, 0,
        TUINT,  BNUMBER|BIND, 0,
        TLONG,  BNUMBER|BIND, 0,
        TULONG, BNUMBER|BIND, 0,
        TVLONG, BNUMBER|BIND, 0,
        TUVLONG, BNUMBER|BIND, 0,

        TFLOAT, BNUMBER, 0,
        TDOUBLE, BNUMBER, 0,
        TIND,   BINTEGER, 0,
    }

```



```

    -1, 0, 0,
};

```

Uses BIND 121, BINTEGER 151b, BNUMBER 151b, TCHAR 30b, TDOUBLE 31a, TFLOAT 31a, TIND 31c, TINT 30b, TLONG 30b, TSHORT 30b, T UCHAR 30b, TUINT 30b, TULONG 30b, TUSHORT 30b, TVVLONG 30b, and TVLONG 30b.

```

<tinit() initialise tcompat arrays 152a>≡ (50a)
for(p=tasigninit; p->code >= 0; p++) {
    urk("tasign", nelem(tasign), p->code);
    tasign[p->code] = p->value;
}
for(p=tasaddinit; p->code >= 0; p++) {
    urk("tasadd", nelem(tasadd), p->code);
    tasadd[p->code] = p->value;
}
for(p=tcastinit; p->code >= 0; p++) {
    urk("tcast", nelem(tcast), p->code);
    tcast[p->code] = p->value;
}
for(p=taddinit; p->code >= 0; p++) {
    urk("tadd", nelem(tadd), p->code);
    tadd[p->code] = p->value;
}
for(p=tsubinit; p->code >= 0; p++) {
    urk("tsub", nelem(tsub), p->code);
    tsub[p->code] = p->value;
}
for(p=tmulinit; p->code >= 0; p++) {
    urk("tmul", nelem(tmul), p->code);
    tmul[p->code] = p->value;
}
for(p=tandinit; p->code >= 0; p++) {
    urk("tand", nelem(tand), p->code);
    tand[p->code] = p->value;
}
for(p=treinit; p->code >= 0; p++) {
    urk("tre", nelem(trel), p->code);
    trel[p->code] = p->value;
}
}

```

Uses tadd 151c, taddinit 151d, tand 159b, tandinit 159c, tasadd 168d, tasaddinit 168e, tasign 166c, tasigninit 166d, tcast 175e, tcastinit 175f, tmul 157b, tmulinit 157c, trel 164b, treinit 164c, tsub 156d, tsubinit 156e, and urk() 50c.

9.5.8 Typechecker: tcom()

tcom is the entry point but just calls tcomo with the ADDR OF context flag. The real work is in tcomo, which takes an extra f parameter encoding the syntactic context: ADDR OF means the node might need an implicit address-of (for arrays and functions), CAST OF signals a cast context, and ADDR OF signals an explicit &. This context drives the special adjustments for array-to-pointer decay and function pointers.

```

<function tcom 152b>≡ (529)
/// complex -> <> -> tcomo|tcoma|tcomx -> <>
/*
 * evaluate types
 * evaluate lvalues (addable == 1)
 */
bool
tcom(Node *n)
{

```

```
    return tcomo(n, ADDROF);
}
```

Uses ADDROF-92 153a and tcomo() 153b.

<enum _anon_ (cc/com.c) 153a>≡ (529)

```
enum
{
    ADDROF = 1<<0,
    CASTOF = 1<<1,
    ADDROP = 1<<2,
};
```

<function tcomo 153b>≡ (529)

```
bool
tcomo(Node *n, int f)
{
    Node *l, *r;
    Type *t;
    // bool | int
    int o;
    <tcomo() other locals 227d>

    <tcomo() sanity check n 153c>
    n->addable = false;
    l = n->left;
    r = n->right;

    switch(n->op) {
    <tcomo() switch node kind cases 155a>
    default:
        diag(n, "unknown op in type complex: %0", n->op);
        goto bad;

    }

    t = n->type;
    <tcomo() sanity check t after typing 153d>
    <tcomo() sanity check type width 154a>
    <tcomo() special adjustments for arrays and functions 154d>
    return false; // everything is fine
```

<tcomo() extra labels 154e>

```
bad:
    n->type = T;
    return true;
}
```

Uses T 31i and diag() 496b.

<tcomo() sanity check n 153c>≡ (153b)

```
if(n == Z) {
    diag(Z, "Z in tcom");
    errexit();
}
```

Uses Z 29a, diag() 496b, and errexit() 495b.

<tcomo() sanity check t after typing 153d>≡ (153b)

```
if(t == T)
    goto bad;
```

Uses T 31i.

```

⟨tcomo() sanity check type width 154a⟩≡ (153b)
    if(t->width < 0) {
        snap(t);
        if(t->width < 0) {
            if(typesu[t->etype] && t->tag)
                diag(n, "structure not fully declared %s", t->tag->name);
            else
                diag(n, "structure not fully declared");
            goto bad;
        }
    }
}

```

Uses `diag()` 496b, `snap()` 147b, and `typesu` 149a.

9.5.9 Array to pointer conversions

In C, arrays are a lie at the expression level: an array name almost always decays into a pointer to its first element. The typechecker implements this by detecting `TARRAY` nodes after type evaluation and wrapping them in an `OADDR` node whose type is `TIND` (pointer). This way, the rest of the compiler—code generation especially—only ever sees pointers.

```

⟨global typeaf 154b⟩≡ (518c)
    char typeaf[NTYPE];

```

Uses `NTYPE` 30a.

```

⟨global typeafinit 154c⟩≡ (518c)
    int typeafinit[] =
    {
        TFUNC, TARRAY, -1,
    };

```

Uses `TARRAY` 31c and `TFUNC` 31c.

```

⟨tcomo() special adjustments for arrays and functions 154d⟩≡ (153b)
    if(typeaf[t->etype]) {
        if(f & ADDROF)
            goto addaddr;
        if(f & ADDRROP)
            warn(n, "address of array/func ignored");
    }
}

```

Uses `ADDROF`-92 153a, `ADDRROP`-94 153a, `typeaf` 154b, and `warn()` 142.

```

⟨tcomo() extra labels 154e⟩≡ (153b)
    addaddr:
        if(tlvalue(n))
            goto bad;
        l = new1(OXXX, Z, Z);
        *l = *n;
        n->op = OADDR;
        if(l->type->etype == TARRAY)
            l->type = l->type->link;
        n->left = l;
        n->right = Z;
        n->addable = false;
        n->type = typ(TIND, l->type);
        n->type->width = types[TIND]->width;
        return false;

```

Uses `OADDR` 116e, `OXXX` 27, `TARRAY` 31c, `TIND` 31c, `Z` 29a, `new1()` 29h, `tlvalue()` 166b, `typ()` 32a, and `types` 32b.

9.6 Typechecking expressions

`tcomo` is the expression type-checker: a large switch on the node's `op` that recursively checks children, applies `tcompat` with the right policy for each operator, inserts implicit `OCAST` nodes when the types differ, and annotates `n->type` with the result type. For arithmetic, `arith` handles the “usual arithmetic conversions” (widening both operands to the same type). For pointer arithmetic, the compiler must scale the integer operand by the pointed-to type's width. For struct access (`ODOT`), `tcomd` walks the type's field list to find the named field.

```
<tcomo() switch node kind cases 155a>≡ (153b) 155b▷  
  case OCOMMA:  
    o = tcom(l);  
    if(o | tcom(r))  
      goto bad;  
    n->type = r->type;  
    break;
```

Uses `OCOMMA 110g` and `tcom() 152b`.

9.6.1 Numeric constants

Constants need no type checking—their type was already set by the parser. The `OCONST` case is a no-op.

```
<tcomo() switch node kind cases 155b>+≡ (153b) <155a 155c>  
  case OCONST:  
    break;
```

Uses `OCONST 111a`.

9.6.2 String constants

```
<tcomo() switch node kind cases 155c>+≡ (153b) <155b 155d>  
  case OSTRING:  
    <tcomo() in OSTRING case, if not an array or chars 227c>  
    <tcomo() transform OSTRING in ONAME 226a>  
    break;
```

Uses `OSTRING 112c`.

9.6.3 Entity uses

For identifiers, the type checker verifies the name was declared (its type is not `T`), marks it as an lvalue (`addable = true`), and handles enum constants specially: an `ONAME` referencing an enum value is rewritten into an `OCONST` with the enum's integer value and its underlying type.

```
<tcomo() switch node kind cases 155d>+≡ (153b) <155c 156b>  
  case ONAME:  
    <tcomo() in ONAME case, check if name declared 155e>  
    <tcomo() in ONAME case, if enum constant 156a>  
    // else  
    n->addable = true;  
    <tcomo() in ONAME case, if extern register 433c>  
    break;
```

Uses `ONAME 27`.

```
<tcomo() in ONAME case, check if name declared 155e>≡ (155d)  
  if(n->type == T) {  
    diag(n, "name not declared: %F", n);  
    goto bad;  
  }
```

Uses `T 31i` and `diag() 496b`.

```

⟨tcom() in ONAME case, if enum constant 156a⟩≡ (155d)
    if(n->type->etype == TENUM) {
        n->op = OCONST;
        if(!typefd[n->type->etype])
            n->vconst = n->sym->vconst;
        else
            n->fconst = n->sym->fconst;
        n->type = n->sym->tenum;
        break;
    }

```

Uses OCONST 111a, TENUM 31c, and typefd 149c.

9.6.4 Arithmetic expressions

The arithmetic cases all follow the same pattern: type-check both children via `tcom`, check compatibility via `tcompat` with the operator’s policy table (e.g., `tadd`, `tsub`, `tmul`), then call `arith` to perform the “usual arithmetic conversions”—widening both operands to a common type and setting `n->type` to the result. The policy tables differ: `tadd` allows pointer+integer (for pointer arithmetic), while `tmul` does not.

```

⟨tcom() switch node kind cases 156b⟩+≡ (153b) <155d 156c>
    case OADD:
        o = tcom(l);
        if(o | tcom(r))
            goto bad;
        ⟨tcom() break if isfunct 438a⟩
        if(tcompat(n, l->type, r->type, tadd))
            goto bad;
        arith(n, true);
        break;

```

Uses OADD 114c, `arith()` 160a, `tadd` 151c, `tcom()` 152b, and `tcompat()` 150a.

```

⟨tcom() switch node kind cases 156c⟩+≡ (153b) <156b 157a>
    case OSUB:
        o = tcom(l);
        if(o | tcom(r))
            goto bad;
        ⟨tcom() break if isfunct 438a⟩
        if(tcompat(n, l->type, r->type, tsub))
            goto bad;
        arith(n, true);
        break;

```

Uses OSUB 114c, `arith()` 160a, `tcom()` 152b, `tcompat()` 150a, and `tsub` 156d.

```

⟨global tsub 156d⟩≡ (518c)
    long tsub[NTYPE];

```

Uses NTYPE 30a.

```

⟨global tsubinit 156e⟩≡ (518c)
    Init tsubinit[] =
    {
        TCHAR,  BNUMBER, 0,
        TUCCHAR, BNUMBER, 0,
        TSHORT,  BNUMBER, 0,
        TUSHORT, BNUMBER, 0,
        TINT,    BNUMBER, 0,
        TUINT,   BNUMBER, 0,
        TLONG,   BNUMBER, 0,
        TULONG,  BNUMBER, 0,
    }

```

```

TVLONG, BNUMBER, 0,
TUVLONG, BNUMBER, 0,

TFLOAT, BNUMBER, 0,
TDOUBLE, BNUMBER, 0,
TIND, BINTEGER|BIND, 0,
-1, 0, 0,
};

```

Uses BIND 121, BINTEGER 151b, BNUMBER 151b, TCHAR 30b, TDOUBLE 31a, TFLOAT 31a, TIND 31c, TINT 30b, TLONG 30b, TSHORT 30b, TUCCHAR 30b, TUINT 30b, TULONG 30b, TUSHORT 30b, TUVLONG 30b, and TVLONG 30b.

```

⟨tcom() switch node kind cases 157a⟩+≡ (153b) <156c 157d>
case OMUL:
case ODIV:
case OLMUL:
case OLDIV:
    o = tcom(l);
    if(o | tcom(r))
        goto bad;
    ⟨tcom() break if isfunct 438a⟩
    if(tcompat(n, l->type, r->type, tmul))
        goto bad;
    arith(n, true);
    ⟨tcom() in OMUL/ODIV case, adjust opcode if unsigned type 232e⟩
    break;

```

Uses ODIV 114c, OLDIV 232d, OLMUL 232d, OMUL 114c, arith() 160a, tcom() 152b, tcompat() 150a, and tmul 157b.

```

⟨global tmul 157b⟩≡ (518c)
long tmul[NTYPE];

```

Uses NTYPE 30a.

```

⟨global tmulinit 157c⟩≡ (518c)
Init tmulinit[] =
{
    TCHAR, BNUMBER, 0,
    TUCCHAR, BNUMBER, 0,
    TSHORT, BNUMBER, 0,
    TUSHORT, BNUMBER, 0,
    TINT, BNUMBER, 0,
    TUINT, BNUMBER, 0,
    TLONG, BNUMBER, 0,
    TULONG, BNUMBER, 0,
    TVLONG, BNUMBER, 0,
    TUVLONG, BNUMBER, 0,

    TFLOAT, BNUMBER, 0,
    TDOUBLE, BNUMBER, 0,
    -1, 0, 0,
};

```

Uses BNUMBER 151b, TCHAR 30b, TDOUBLE 31a, TFLOAT 31a, TINT 30b, TLONG 30b, TSHORT 30b, TUCCHAR 30b, TUINT 30b, TULONG 30b, TUSHORT 30b, TUVLONG 30b, and TVLONG 30b.

```

⟨tcom() switch node kind cases 157d⟩+≡ (153b) <157a 158a>
case OMOD:
case OLMOD:
    o = tcom(l);
    if(o | tcom(r))
        goto bad;
    ⟨tcom() break if isfunct 438a⟩

```

```

    if(tcompat(n, l->type, r->type, tand))
        goto bad;
    arith(n, true);
    <tcom() in OMOD case, adjust opcode if unsigned type 232g>
    break;

```

Uses OLMOD 232f, OMOD 114c, arith() 160a, tand 159b, tcom() 152b, and tcompat() 150a.

```

<tcom() switch node kind cases 158a>+≡ (153b) <157d 158b>
case OPOS:
    if(tcom(l))
        goto bad;
    <tcom() break if isfunct 438a>

    r = l;
    l = new(OCONST, Z, Z);
    l->vconst = 0;
    l->type = types[TINT];
    n->op = OADD;
    n->right = r;
    n->left = l;

    if(tcom(l))
        goto bad;
    if(tcompat(n, l->type, r->type, tsub))
        goto bad;
    arith(n, true);
    break;

```

Uses OADD 114c, OCONST 111a, OPOS 114e, TINT 30b, Z 29a, arith() 160a, new() 29d, tcom() 152b, tcompat() 150a, tsub 156d, and types 32b.

```

<tcom() switch node kind cases 158b>+≡ (153b) <158a 159a>
case ONEG:
    if(tcom(l))
        goto bad;
    <tcom() break if isfunct 438a>

    if(!machcap(n)) {
        r = l;
        l = new(OCONST, Z, Z);
        l->vconst = 0;
        l->type = types[TINT];
        n->op = OSUB;
        n->right = r;
        n->left = l;

        if(tcom(l))
            goto bad;
        if(tcompat(n, l->type, r->type, tsub))
            goto bad;
    }
    arith(n, true);
    break;

```

Uses OCONST 111a, ONEG 114e, OSUB 114c, TINT 30b, Z 29a, arith() 160a, new() 29d, tcom() 152b, tcompat() 150a, tsub 156d, and types 32b.

```

<function machcap(arm) 158c>≡ (535a)
/* default, like old cc */
bool
machcap(Node*)

```

```

{
    return false;
}

```

$\langle \text{tcom}() \text{ switch node kind cases } 159a \rangle + \equiv$ (153b) $\triangleleft 158b \ 159d \triangleright$

```

case OAND:
case OOR:
case OXOR:
    o = tcom(l);
    if(o | tcom(r))
        goto bad;
     $\langle \text{tcom}() \text{ break if isfunct } 438a \rangle$ 
    if(tcompat(n, l->type, r->type, tand))
        goto bad;
    arith(n, true);
    break;

```

Uses OAND 115a, OOR 115a, OXOR 115a, arith() 160a, tand 159b, tcom() 152b, and tcompat() 150a.

$\langle \text{global tand } 159b \rangle \equiv$ (518c)
 long tand[NTYPE];

Uses NTYPE 30a.

$\langle \text{global tandinit } 159c \rangle \equiv$ (518c)

```

Init tandinit[] =
{
    TCHAR,  BINTEGER, 0,
    TUCCHAR, BINTEGER, 0,
    TSHORT, BINTEGER, 0,
    TUSHORT, BINTEGER, 0,

    TINT,  BNUMBER, 0,
    TUINT, BNUMBER, 0,

    TLONG, BINTEGER, 0,
    TULONG, BINTEGER, 0,
    TVLONG, BINTEGER, 0,
    TUVLONG, BINTEGER, 0,
    -1, 0, 0,
};

```

Uses BINTEGER 151b, BNUMBER 151b, TCHAR 30b, TINT 30b, TLONG 30b, TSHORT 30b, TUCCHAR 30b, TUINT 30b, TULONG 30b, TUSHORT 30b, TUVLONG 30b, and TVLONG 30b.

$\langle \text{tcom}() \text{ switch node kind cases } 159d \rangle + \equiv$ (153b) $\triangleleft 159a \ 164a \triangleright$

```

case OASHL:
case OASHR:
case OLSHR:
    o = tcom(l);
    if(o | tcom(r))
        goto bad;
     $\langle \text{tcom}() \text{ break if isfunct } 438a \rangle$ 
    if(tcompat(n, l->type, r->type, tand))
        goto bad;

    n->right = Z;
    arith(n, true);

    n->right = new1(OCast, r, Z);
    n->right->type = types[TINT];
     $\langle \text{tcom}() \text{ in OASHL/OASHR case, adjust opcode if unsigned type } 232c \rangle$ 

```



```
break;
```

Uses OASHL 115a, OASHR 115a, OCAST 118a, OLSHR 232b, TINT 30b, Z 29a, arith() 160a, new1() 29h, tand 159b, tcom() 152b, tcompat() 150a, and types 32b.

9.6.5 Arithmetic conversions

arith implements C's "usual arithmetic conversions": when a binary operator has operands of different types, both are widened to a common type. The hierarchy is roughly char < short < int < long < vlong, with unsigned types winning over signed types of the same width. For pointer arithmetic ($p + i$), arith scales the integer operand by the size of the pointed-to type (e.g., multiplying by 4 for `int*`).

```
<function arith 160a>≡ (518c)
  /// tcomo -> <>
  /*
   * "the usual arithmetic conversions are performed"
   */
  void
  arith(Node *n, bool f)
  {
    Type *t1, *t2;
    // enum<Type_kind>
    int i, j;
    // enum<Type_kind>
    int k;
    <arith() other locals 162b>

    <arith() set t1 and t2, left and right types 160b>
    <arith() set i and j, etype of left and right types 160c>

    k = tab[i][j];

    <arith() set node type 161b>

    <arith() if subtract two pointers 163c>
    // else
    <arith() if left type not sametype than node type, add OCAST 162c>
    <arith() if right type not sametype than node type, add OCAST 162d>
    return;
  bad:
    diag(n, "pointer addition not fully declared: %T", n->type->link);
  }
```

Uses diag() 496b and tab 161a.

```
<arith() set t1 and t2, left and right types 160b>≡ (160a)
  t1 = n->left->type;
  if(n->right == Z)
    t2 = t1;
  else
    t2 = n->right->type;
```

Uses Z 29a.

```
<arith() set i and j, etype of left and right types 160c>≡ (160a)
  i = TXXX;
  if(t1 != T)
    i = t1->etype;
  j = TXXX;
  if(t2 != T)
    j = t2->etype;
```

Uses T 31i and TXXX 30a.

```

⟨global tab 161a⟩≡ (518c)
char tab[NTYPE][NTYPE] =
{
    [TXXX] = { 0,
    },

    [TCHAR] = { 0, TCHAR, TCHAR, TSHORT, TSHORT, TINT, TUINT, TLONG,
    TULONG, TVLONG, TVLONG, TFLOAT, TDOUBLE, [TIND] = TIND,
    },
    [TCHAR] = { 0, TCHAR, TCHAR, TSHORT, TSHORT, TUINT, TUINT, TULONG,
    TULONG, TVLONG, TVLONG, TFLOAT, TDOUBLE, [TIND] = TIND,
    },
    [TSHORT] = { 0, TSHORT, TSHORT, TSHORT, TSHORT, TINT, TUINT, TLONG,
    TULONG, TVLONG, TVLONG, TFLOAT, TDOUBLE, [TIND] = TIND,
    },
    [TSHORT] = { 0, TSHORT, TSHORT, TSHORT, TSHORT, TUINT, TUINT, TULONG,
    TULONG, TVLONG, TVLONG, TFLOAT, TDOUBLE, [TIND] = TIND,
    },
    [TINT] = { 0, TINT, TUINT, TINT, TUINT, TINT, TUINT, TLONG,
    TULONG, TVLONG, TVLONG, TFLOAT, TDOUBLE, [TIND] = TIND,
    },
    [TUINT] = { 0, TUINT, TUINT, TUINT, TUINT, TUINT, TUINT, TULONG,
    TULONG, TVLONG, TVLONG, TFLOAT, TDOUBLE, [TIND] = TIND,
    },
    [TLONG] = { 0, TLONG, TULONG, TLONG, TULONG, TLONG, TULONG, TLONG,
    TULONG, TVLONG, TVLONG, TFLOAT, TDOUBLE, [TIND] = TIND,
    },
    [TULONG] = { 0, TULONG, TULONG, TULONG, TULONG, TULONG, TULONG, TULONG,
    TULONG, TVLONG, TVLONG, TFLOAT, TDOUBLE, [TIND] = TIND,
    },
    [TVLONG] = { 0, TVLONG, TVLONG, TVLONG, TVLONG, TVLONG, TVLONG, TVLONG,
    TVLONG, TVLONG, TVLONG, TFLOAT, TDOUBLE, [TIND] = TIND,
    },
    [TUVLONG] = { 0, TUVLONG, TUVLONG, TUVLONG, TUVLONG, TUVLONG, TUVLONG, TUVLONG,
    TUVLONG, TUVLONG, TUVLONG, TFLOAT, TDOUBLE, [TIND] = TIND,
    },

    [TFLOAT] = { 0, TFLOAT, TFLOAT, TFLOAT, TFLOAT, TFLOAT, TFLOAT, TFLOAT,
    TFLOAT, TFLOAT, TFLOAT, TFLOAT, TDOUBLE, [TIND] = TIND,
    },
    [TDOUBLE] = { 0, TDOUBLE, TDOUBLE, TDOUBLE, TDOUBLE, TDOUBLE, TDOUBLE, TDOUBLE,
    TDOUBLE, TDOUBLE, TDOUBLE, TFLOAT, TDOUBLE, [TIND] = TIND,
    },

    [TIND] = { 0, TIND, TIND, TIND, TIND, TIND, TIND, TIND,
    TIND, TIND, TIND, TIND, TIND, [TIND] = TIND,
    },
};

```

Uses NTYPE 30a, TCHAR 30b, TDOUBLE 31a, TFLOAT 31a, TIND 31c, TINT 30b, TLONG 30b, TSHORT 30b, TCHAR 30b, TUINT 30b, TULONG 30b, TSHORT 30b, TUVLONG 30b, and TVLONG 30b.

```

⟨arith() set node type 161b⟩≡ (160a)
⟨arith() if result type is a pointer 161c⟩
else {
    ⟨arith() if f, convert up to at least int 162a⟩
    n->type = types[k];
}

```

Uses types 32b.

```

⟨arith() if result type is a pointer 161c⟩≡ (161b)

```

```

if(k == TIND) {
    if(i == TIND)
        n->type = t1;
    else
        if(j == TIND)
            n->type = t2;
}

```

Uses TIND 31c.

```

⟨arith() if f, convert up to at least int 162a⟩≡ (161b)
/* convert up to at least int */
if(f)
    while(k < TINT)
        k += 2;

```

Uses TINT 30b.

```

⟨arith() other locals 162b⟩≡ (160a)
Node *n1;
long w;

```

```

⟨arith() if left type not sametype than node type, add OCAST 162c⟩≡ (160a)
if(!sametype(n->type, n->left->type)) {
    n->left = new1(OCAST, n->left, Z);
    n->left->type = n->type;
    ⟨arith() pointer arithmetic with left child 162e⟩
}

```

Uses OCAST 118a, Z 29a, new1() 29h, and sametype() 145c.

```

⟨arith() if right type not sametype than node type, add OCAST 162d⟩≡ (160a)
if(n->right != Z)
if(!sametype(n->type, n->right->type)) {
    n->right = new1(OCAST, n->right, Z);
    n->right->type = n->type;
    ⟨arith() pointer arithmetic with right child 163b⟩
}

```

Uses OCAST 118a, Z 29a, new1() 29h, and sametype() 145c.

9.6.6 Pointer arithmetic

When an addition or subtraction involves a pointer and an integer, the integer must be scaled by the size of the pointed-to type. For `int *p`, the expression `p + 3` means “advance by $3 \times \text{sizeof}(\text{int})$ bytes.” The type checker inserts an OMUL node to multiply the integer operand by the element width. For pointer subtraction (`p - q`), the result is divided by the element width to produce an element count.

```

⟨arith() pointer arithmetic with left child 162e⟩≡ (162c)
if(n->type->etype == TIND) {
    w = n->type->link->width;
    ⟨arith() if incomplete width 163a⟩
    if(w > 1) {
        n1 = new1(OCONST, Z, Z);
        n1->vconst = w;
        n1->type = n->type;
        n->left = new1(OMUL, n->left, n1);
        n->left->type = n->type;
    }
}

```

Uses OCONST 111a, OMUL 114c, TIND 31c, Z 29a, and new1() 29h.

```

⟨arith() if incomplete width 163a⟩≡ (163b 162e)
    if(w < 1) {
        snap(n->type->link);
        w = n->type->link->width;
        if(w < 1)
            goto bad;
    }

```

Uses snap() 147b.

```

⟨arith() pointer arithmetic with right child 163b⟩≡ (162d)
    if(n->type->etype == TIND) {
        w = n->type->link->width;
        ⟨arith() if incomplete width 163a⟩
        if(w != 1) {
            n1 = new1(OCNST, Z, Z);
            n1->vconst = w;
            n1->type = n->type;
            n->right = new1(OMUL, n->right, n1);
            n->right->type = n->type;
        }
    }

```

Uses OCONST 111a, OMUL 114c, TIND 31c, Z 29a, and new1() 29h.

```

⟨arith() if subtract two pointers 163c⟩≡ (160a)
    if(n->op == OSUB)
        if(i == TIND && j == TIND) {
            w = n->right->type->link->width;
            if(w < 1 || n->left->type->link == T || n->left->type->link->width < 1)
                goto bad;
            n->type = types[ewidth[TIND] <= ewidth[TLONG]? TLONG: TVLONG];
            if(1 && ewidth[TIND] > ewidth[TLONG]){
                n1 = new1(OXXX, Z, Z);
                *n1 = *n;
                n->op = OCAST;
                n->left = n1;
                n->right = Z;
                n->type = types[TLONG];
            }
            if(w > 1) {
                n1 = new1(OXXX, Z, Z);
                *n1 = *n;
                n->op = ODIV;
                n->left = n1;
                n1 = new1(OCNST, Z, Z);
                n1->vconst = w;
                n1->type = n->type;
                n->right = n1;
                ⟨arith() when subtract two pointers, convert DIV to SHR if can 310c⟩
            }
            return;
        }
    }

```

Uses OCAST 118a, OCONST 111a, ODIV 114c, OSUB 114c, OXXX 27, T 31i, TIND 31c, TLONG 30b, TVLONG 30b, Z 29a, new1() 29h, and types 32b.

9.6.7 Boolean expressions

Comparison (==, !=, <, etc.) and logical operators (&&, ||, !) accept any numeric type or pointer, and always produce TINT. The bitwise complement `x` is rewritten as `(-1) ^ x` if the target architecture does not have a

native complement instruction.

```
<tcom() switch node kind cases 164a>+≡ (153b) <159d 164d>
case OEQ:
case ONE:
    o = tcom(l);
    if(o | tcom(r))
        goto bad;
    <tcom() break if isfunct 438a>
    <tcom() when OEQ/ONE, typing extensions 426a>
    if(tcompat(n, l->type, r->type, trel))
        goto bad;
    arith(n, false);

    n->type = types[TINT];
    break;
```

Uses OEQ 115g, ONE 115g, TINT 30b, arith() 160a, tcom() 152b, tcompat() 150a, trel 164b, and types 32b.

```
<global trel 164b>≡ (518c)
long trel[NTYPE];
```

Uses NTYPE 30a.

```
<global trelinit 164c>≡ (518c)
Init trelinit[] =
{
    TCHAR, BNUMBER, 0,
    TUCCHAR, BNUMBER, 0,
    TSHORT, BNUMBER, 0,
    TUSHORT, BNUMBER, 0,
    TINT, BNUMBER, 0,
    TUINT, BNUMBER, 0,
    TLONG, BNUMBER, 0,
    TULONG, BNUMBER, 0,
    TVLONG, BNUMBER, 0,
    TUVLONG, BNUMBER, 0,

    TFLOAT, BNUMBER, 0,
    TDOUBLE, BNUMBER, 0,

    TIND, BIND, 0,
    -1, 0, 0,
};
```

Uses BIND 121, BNUMBER 151b, TCHAR 30b, TDOUBLE 31a, TFLOAT 31a, TIND 31c, TINT 30b, TLONG 30b, TSHORT 30b, TUCCHAR 30b, TUINT 30b, TULONG 30b, TUSHORT 30b, TUVLONG 30b, and TVLONG 30b.

```
<tcom() switch node kind cases 164d>+≡ (153b) <164a 165a>
case OLT:
case OGE:
case OGT:
case OLE:
    o = tcom(l);
    if(o | tcom(r))
        goto bad;
    <tcom() break if isfunct 438a>
    <tcom() when OLT/OGE/OGT/OLE, typing extensions 450c>
    if(tcompat(n, l->type, r->type, trel))
        goto bad;
    arith(n, false);
    <tcom() when OLT/OGE/OGT/OLE, adjust opcode if unsigned type 232h>
    n->type = types[TINT];
```

break;

Uses OGE 115g, OGT 115g, OLE 115g, OLT 115g, TINT 30b, arith() 160a, tcom() 152b, tcompat() 150a, trel 164b, and types 32b.

```
<tcom() switch node kind cases 165a>+≡ (153b) <164d 165c>
case ONOT:
    if(tcom(l))
        goto bad;
    <tcom() break if isfunct 438a>
    if(tcompat(n, T, l->type, tnot))
        goto bad;
    n->type = types[TINT];
    break;
```

Uses ONOT 115e, T 31i, TINT 30b, tcom() 152b, tcompat() 150a, tnot 165b, and types 32b.

```
<global tnot 165b>≡ (518c)
long tnot[1] =
{
    BNUMBER|BIND,
};
```

Uses BIND 121 and BNUMBER 151b.

```
<tcom() switch node kind cases 165c>+≡ (153b) <165a 165d>
case OCOM:
    if(tcom(l))
        goto bad;
    <tcom() break if isfunct 438a>

    if(!machcap(n)) {
        r = l;
        l = new(OCNST, Z, Z);
        l->vconst = -1;
        l->type = types[TINT];
        n->op = OXOR;
        n->right = r;
        n->left = l;

        if(tcom(l))
            goto bad;
        if(tcompat(n, l->type, r->type, tand))
            goto bad;
    }
    arith(n, false);
    break;
```

Uses OCOM 115e, OCONST 111a, OXOR 115a, TINT 30b, Z 29a, arith() 160a, new() 29d, tand 159b, tcom() 152b, tcompat() 150a, and types 32b.

```
<tcom() switch node kind cases 165d>+≡ (153b) <165c 166a>
case OANDAND:
case OOROR:
    o = tcom(l);
    if(o | tcom(r))
        goto bad;
    if(tcompat(n, T, l->type, tnot) |
        tcompat(n, T, r->type, tnot))
        goto bad;
    n->type = types[TINT];
    break;
```

Uses OANDAND 115c, OOROR 115c, T 31i, TINT 30b, tcom() 152b, tcompat() 150a, tnot 165b, and types 32b.

9.6.8 Assignments

Assignment type-checking first verifies that the left-hand side is an lvalue (via `tlvalue`), then checks type compatibility between the left and right sides. If the types differ but are compatible (e.g., assigning an `int` to a `long`), an implicit `OCAST` node is inserted. Compound assignments (`+=`, etc.) follow the same pattern but use their own policy table.

```

⟨tcom() switch node kind cases 166a⟩+≡ (153b) <165d 167b>
case OAS:
    o = tcom(l);
    if(o | tcom(r))
        goto bad;
    if(tlvalue(l))
        goto bad;
    ⟨tcom() break if isfunct 438a⟩
    ⟨tcom() when OAS, typing extension 426b⟩
    if(tcompat(n, l->type, r->type, tasign))
        goto bad;

    ⟨tcom() when OAS, const checking 190b⟩
    if(!sametype(l->type, r->type)) {
        r = new1(OCAST, r, Z);
        r->type = l->type;
        n->right = r;
    }
    n->type = l->type;
    break;

```

Uses `OAS` 116a, `OCAST` 118a, `Z` 29a, `new1()` 29h, `sametype()` 145c, `tasign` 166c, `tcom()` 152b, `tcompat()` 150a, and `tlvalue()` 166b.

```

⟨function tlvalue 166b⟩≡ (529)
bool
tlvalue(Node *n)
{
    if(!n->addable) {
        diag(n, "not an l-value");
        return true;
    }
    return false;
}

```

Uses `diag()` 496b.

```

⟨global tasign 166c⟩≡ (518c)
// map<Type_kind, set<Type_kind> >
long tasign[NTYPE];

```

Uses `NTYPE` 30a.

```

⟨global tasigninit 166d⟩≡ (518c)
Init tasigninit[] =
{
    TCHAR,  BNUMBER, 0,
    TCHAR,  BNUMBER, 0,
    TSHORT, BNUMBER, 0,
    TSHORT, BNUMBER, 0,
    TINT,   BNUMBER, 0,
    TINT,   BNUMBER, 0,
    TLONG,  BNUMBER, 0,
    TLONG,  BNUMBER, 0,
    TVLONG, BNUMBER, 0,

```

```

TUVLONG, BNUMBER, 0,

TFLOAT, BNUMBER, 0,
TDOUBLE, BNUMBER, 0,

TIND, BIND, 0,
TSTRUCT, BSTRUCT, 0,
TUNION, BUNION, 0,
-1, 0, 0,
};

```

Uses BIND 121, BNUMBER 151b, BSTRUCT 121, BUNION 121, TCHAR 30b, TDOUBLE 31a, TFLOAT 31a, TIND 31c, TINT 30b, TLONG 30b, TSHORT 30b, TSTRUCT 31c, TUCCHAR 30b, TUINT 30b, TULONG 30b, TUNION 31c, TUSHORT 30b, TUVLONG 30b, and TVLONG 30b.

```

<stcompat() when t1 matches t2, if ttab is tassign 167a>≡ (150c)
  if(ttab == tassign)
    if(b == BSTRUCT || b == BUNION)
      if(!sametype(t1, t2))
        return true;

```

Uses BSTRUCT 121, BUNION 121, sametype() 145c, and tassign 166c.

```

<tcom() switch node kind cases 167b>+≡ (153b) <166a 167c>
  case OASI: /* same as as, but no test for const */
    n->op = OAS; // converted in regular assign!
    o = tcom(l);
    if(o | tcom(r))
      goto bad;

  <tcom() when OAS, typing extension 426b>
  if(tlvalue(l) || tcompat(n, l->type, r->type, tassign))
    goto bad;
  if(!sametype(l->type, r->type)) {
    r = new1(OCAST, r, Z);
    r->type = l->type;
    n->right = r;
  }
  n->type = l->type;
  break;

```

Uses OAS 116a, OASI 262b, OCAST 118a, Z 29a, new1() 29h, sametype() 145c, tassign 166c, tcom() 152b, tcompat() 150a, and tlvalue() 166b.

```

<tcom() switch node kind cases 167c>+≡ (153b) <167b 168f>
  case OASADD:
  case OASSUB:
    o = tcom(l);
    if(o | tcom(r))
      goto bad;
    if(tlvalue(l))
      goto bad;

    <tcom() break if isfunct 438a>
    <tcom() when OASADD, typing extension 426c>
    if(tcompat(n, l->type, r->type, tasadd))
      goto bad;

    <tcom() when OASxxx, const checking 190c>
    t = l->type;
    arith(n, false);
    <tcom() when OASxxx, adjust node after arith 168a>
    break;

```

Uses OASADD 116c, OASSUB 116c, arith() 160a, tasadd 168d, tcom() 152b, tcompat() 150a, and tlvalue() 166b.


```

⟨tcom() when OASxxx, adjust node after arith 168a⟩≡ (169b 168f 167c)
⟨tcom() when OASxxx, remove casts added by arith 168b⟩
if(!sametype(t, n->type) && !mixedasop(t, n->type)) {
    r = new1(OCAST, n->right, Z);
    r->type = t;
    n->right = r;
    n->type = t;
}

```

Uses OCAST 118a, Z 29a, mixedasop() 168c, new1() 29h, and sametype() 145c.

```

⟨tcom() when OASxxx, remove casts added by arith 168b⟩≡ (168a)
while(n->left->op == OCAST)
    n->left = n->left->left;

```

Uses OCAST 118a.

```

⟨function mixedasop 168c⟩≡ (518c)
bool
mixedasop(Type *l, Type *r)
{
    return !typefd[l->etype] && typefd[r->etype];
}

```

Uses typefd 149c.

```

⟨global tasadd 168d⟩≡ (518c)
long tasadd[NTYPE];

```

Uses NTYPE 30a.

```

⟨global tasaddinit 168e⟩≡ (518c)
Init tasaddinit[] =
{
    TCHAR, BNUMBER, 0,
    TUCCHAR, BNUMBER, 0,
    TSHORT, BNUMBER, 0,
    TUSHORT, BNUMBER, 0,
    TINT, BNUMBER, 0,
    TUINT, BNUMBER, 0,
    TLONG, BNUMBER, 0,
    TULONG, BNUMBER, 0,
    TVLONG, BNUMBER, 0,
    TUVLONG, BNUMBER, 0,

    TFLOAT, BNUMBER, 0,
    TDOUBLE, BNUMBER, 0,

    TIND, BINTEGER, 0,
    -1, 0, 0,
};

```

Uses BINTEGER 151b, BNUMBER 151b, TCHAR 30b, TDOUBLE 31a, TFLOAT 31a, TIND 31c, TINT 30b, TLONG 30b, TSHORT 30b, TUCCHAR 30b, TUINT 30b, TULONG 30b, TUSHORT 30b, TUVLONG 30b, and TVLONG 30b.

```

⟨tcom() switch node kind cases 168f⟩+≡ (153b) <167c 169a>
case OASMUL:
case OASDIV:
case OASLMUL:
case OASLDIV:
    o = tcom(l);
    if(o | tcom(r))
        goto bad;
    if(tlvalue(l))

```

```

    goto bad;
    <tcom() break if isfunct 438a>
    <tcom() when OASMUL/OASDIV, typing extensions 450b>
    if(tcompat(n, l->type, r->type, tmul))
        goto bad;
    <tcom() when OASxxx, const checking 190c>
    t = l->type;
    arith(n, false);
    <tcom() when OASxxx, adjust node after arith 168a>
    <tcom() when OASMUL/OASDIV, adjust opcode if unsigned type 233d>
    break;

```

Uses OASDIV 116c, OASLDIV 233b, OASLMUL 233b, OASMUL 116c, arith() 160a, tcom() 152b, tcompat() 150a, tlvalue() 166b, and tmul 157b.

```

<tcom() switch node kind cases 169a>+≡ (153b) <168f 169b>
case OASLSHR:
case OASASHR:
case OASASHL:
    o = tcom(l);
    if(o | tcom(r))
        goto bad;
    if(tlvalue(l))
        goto bad;
    <tcom() break if isfunct 438a>
    if(tcompat(n, l->type, r->type, tand))
        goto bad;
    n->type = l->type;
    n->right = new1(OCAST, r, Z);
    n->right->type = types[TINT];
    <tcom() when OASLSHR, adjust opcode if unsigned type 233c>
    break;

```

Uses OASASHL 116c, OASASHR 116c, OASLSHR 233b, OCAST 118a, TINT 30b, Z 29a, new1() 29h, tand 159b, tcom() 152b, tcompat() 150a, tlvalue() 166b, and types 32b.

```

<tcom() switch node kind cases 169b>+≡ (153b) <169a 170a>
case OASMOD:
case OASLMOD:
case OASOR:
case OASAND:
case OASXOR:
    o = tcom(l);
    if(o | tcom(r))
        goto bad;
    if(tlvalue(l))
        goto bad;
    <tcom() break if isfunct 438a>
    if(tcompat(n, l->type, r->type, tand))
        goto bad;
    t = l->type;
    arith(n, false);
    <tcom() when OASxxx, adjust node after arith 168a>
    <tcom() when OASMOD, adjust opcode if unsigned type 233e>
    break;

```

Uses OASAND 116c, OASLMOD 233b, OASMOD 116c, OASOR 116c, OASXOR 116c, arith() 160a, tand 159b, tcom() 152b, tcompat() 150a, and tlvalue() 166b.

9.6.9 Pointers

For `OADDR (&x)`, the operand must be an lvalue, and cannot be a bitfield or register variable. Note the use of `ADDR_OP` context rather than `ADDR_OF`—this avoids the automatic array-to-pointer decay that `ADDR_OF` would trigger. For `OIND (*p)`, the operand must be a pointer (`TIND`), and the result is an lvalue (its `addable` is set to true, since you can write `*p = value`).

```
<tcom() switch node kind cases 170a>+≡ (153b) <169b 170b>
case OADDR:
    if(tcom(1, ADDR_OP))
        goto bad;
    if(tlvalue(1))
        goto bad;

    <tcom() when OADDR, check if bitfield 419f>
    <tcom() when OADDR, check if register 433e>

    n->type = typ(TIND, l->type);
    n->type->width = types[TIND]->width;
    break;
```

Uses `ADDR_OP`-94 153a, `OADDR` 116e, `TIND` 31c, `tcom()` 153b, `tlvalue()` 166b, `typ()` 32a, and `types` 32b.

```
<tcom() switch node kind cases 170b>+≡ (153b) <170a 170d>
case OIND:
    if(tcom(1))
        goto bad;
    if(tcompat(n, T, l->type, tindir))
        goto bad;
    n->type = l->type->link;
    n->addable = true;
    break;
```

Uses `OIND` 116e, `T` 31i, `tcom()` 152b, `tcompat()` 150a, and `tindir` 170c.

```
<global tindir 170c>≡ (518c)
long tindir[1] =
{
    BIND,
};
```

Uses `BIND` 121.

9.6.10 Array accesses

Array accesses (`a[i]`) are desugared into pointer arithmetic (`*(a+i)`) during parsing, so the type-checker never sees `OINDEX`—it handles the resulting `OADD` and `OIND` nodes instead. References to array-typed variables are converted to `OADDR` first (array-to-pointer decay).

9.6.11 Field accesses

For `a.field`, the type-checker first verifies that `a` is a struct or union (via `tcompat` with `tdot`), then `tcomd` searches the type's field list for the named field (using `dotsearch`, which also searches unnamed embedded structs). If found, the node is rewritten: the field's byte offset is encoded into the node, and `n->type` is set to the field's type.

```
<tcom() switch node kind cases 170d>+≡ (153b) <170b 172b>
case ODOT:
    if(tcom(1))
        goto bad;
```

```

    if(tcompat(n, T, l->type, tdot))
        goto bad;
    if(tcomd(n))
        goto bad;
    break;

```

Uses ODOT 117a, T 31i, tcom() 152b, tcomd() 171b, tcompat() 150a, and tdot 171a.

```

⟨global tdot 171a⟩≡ (518c)
    long tdot[1] =
    {
        BSTRUCT|BUNION,
    };

```

Uses BSTRUCT 121 and BUNION 121.

```

⟨function tcomd 171b⟩≡ (529)
    bool
    tcomd(Node *n)
    {
        Type *t;
        long o;

        o = 0;
        t = dotsearch(n->sym, n->left->type->link, n, &o);
        if(t == T) {
            diag(n, "not a member of struct/union: %F", n);
            return true;
        }
        // convert field and set n->type
        ⟨tcomd() convert field access in offset 241a⟩
        return false;
    }

```

Uses T 31i, diag() 496b, and dotsearch() 171c.

```

⟨function dotsearch 171c⟩≡ (518c)
    /// tcomd -> <>
    Type*
    dotsearch(Sym *s, Type *t, Node *n, long *off)
    {
        Type *t1, *xt, *rt;

        xt = T;

        /*
         * look it up by name
         */
        for(t1 = t; t1 != T; t1 = t1->down)
            if(t1->sym == s) {
                if(xt != T)
                    goto ambig;
                xt = t1;
            }

        ⟨dotsearch() look by type 172a⟩

        if(xt != T) {
            *off = xt->offset;
            return xt;
        }
    }

```

`<dotsearch() if unnamed substructures 427e>`

`return xt;`

```

ambig:
    diag(n, "ambiguous structure element: %s", s->name);
    return xt;
}

```

Uses T 31i and diag() 496b.

```

<dotsearch() look by type 172a>≡ (171c)
/*
 * look it up by type
 */
if(s->class == CTYPEDEF || s->class == CTYPESTR)
    for(t1 = t; t1 != T; t1 = t1->down)
        if(t1->sym == S && typesu[t1->etype])
            if(sametype(s->type, t1)) {
                if(xt != T)
                    goto ambig;
                xt = t1;
            }

```

Uses CTYPEDEF 137i, CTYPESTR 434f, S 35d, T 31i, sametype() 145c, and typesu 149a.

9.6.12 Function calls

For a call like `f(a, b)`, the left child is the function expression and the right child is the `OLIST` of arguments. The type checker verifies that the left side has function type, then calls `tcoma` to walk the parameter list and the argument list in parallel, checking type compatibility for each pair and inserting `OCAST` nodes as needed. If the function was declared without a prototype (old-style), a warning is issued that argument types are not checked. The result type is the function's return type (`Type.link`).

```

<tcoma() switch node kind cases 172b>+≡ (153b) <170d 175a>
case OFUNC:
    o = tcoma(l, 0);
    if(o)
        goto bad;
    <tcoma() when OFUNC case, add OIND if pointer function call 414a>
    if(tcompat(n, T, l->type, tfunct))
        goto bad;
    if(o | tcoma(l, r, l->type->down, true))
        goto bad;
    n->type = l->type->link;
    <tcoma() when OFUNC case, warn function args not checked if old proto 425c>
    <tcoma() when OFUNC case, format check 447a>
    break;

```

Uses OFUNC 117c, T 31i, tcoma() 173a, tcoma() 153b, tcompat() 150a, and tfunct 172c.

```

<global tfunct 172c>≡ (518c)
long tfunct[1] =
{
    BFUNC,
};

```

Uses BFUNC 121.

<function tcoma 173a>≡ (529)

```

bool
tcoma(Node *l, Node *n, Type *t, bool f)
{
    Node *n1;
    int o;

    <tcoma() start of tcoma, adjust t if TDOT 174a>
    <tcoma() if no arguments 174b>

    // recurse over arguments and parameters
    if(n->op == OLIST) {
        o = tcoma(l, n->left, t, false);
        if(t != T) {
            t = t->down;
            if(t == T)
                t = types[TVOID];
        }
        return o | tcoma(l, n->right, t, true);
    }

    // process each argument

    if(f && t != T)
        tcoma(l, Z, t->down, false);

    if(tcom(n) || tcompat(n, T, n->type, targ))
        return true;
    <tcoma() check if too many arguments 174c>
    if(t != T) {
        <tcoma() typing extension on argument 426e>
        if(stcompat(nodproto, t, n->type, tasign)) {
            diag(l, "argument prototype mismatch \"%T\" for \"%T\": %F",
                n->type, t, l);
            return true;
        }
        <tcoma() adjust type if argument with small type 174d>
    }
    else
        <tcoma() adjust type when empty type 174e>

    if(t != T && !sametype(t, n->type)) {
        n1 = new1(OXXX, Z, Z);
        *n1 = *n;
        n->op = OCAST;
        n->left = n1;
        n->right = Z;
        n->type = t;
        n->addable = false;
    }
    return false; // everything is fine
}

```

Uses OCAST 118a, OLIST 29b, OXXX 27, T 31i, TVOID 31b, Z 29a, diag() 496b, new1() 29h, nodproto 175b, sametype() 145c, stcompat() 150c, targ 173b, tasign 166c, tcom() 152b, tcoma() 173a, tcompat() 150a, and types 32b.

<global targ 173b>≡ (518c)

```

long targ[1] =
{
    BNUMBER|BIND|BSTRUCT|BUNION,
};

```

Uses BIND 121, BNUMBER 151b, BSTRUCT 121, and BUNION 121.

```
<tcoma() start of tcoma, adjust t if TDOT 174a>≡ (173a)
    if(t != T)
    if(t->etype == TOLD || t->etype == TDOT) /* .../old in prototype */
        t = T;
```

Uses T 31i, TDOT 137f, and TOLD 424d.

```
<tcoma() if no arguments 174b>≡ (173a)
    if(n == Z) {
        if(t != T && !sametype(t, types[TVOID])) {
            diag(n, "not enough function arguments: %F", 1);
            return true;
        }
        return false;
    }
```

Uses T 31i, TVOID 31b, Z 29a, diag() 496b, sametype() 145c, and types 32b.

```
<tcoma() check if too many arguments 174c>≡ (173a)
    if(sametype(t, types[TVOID])) {
        diag(n, "too many function arguments: %F", 1);
        return true;
    }
```

Uses TVOID 31b, diag() 496b, sametype() 145c, and types 32b.

```
<tcoma() adjust type if argument with small type 174d>≡ (173a)
    switch(t->etype) {
    case TCHAR:
    case TSHORT:
        t = types[TINT];
        break;

    case TUCCHAR:
    case TUSHORT:
        t = types[TUINT];
        break;
    }
```

Uses TCHAR 30b, TINT 30b, TSHORT 30b, TUCCHAR 30b, TUINT 30b, TUSHORT 30b, and types 32b.

```
<tcoma() adjust type when empty type 174e>≡ (173a)
    switch(n->type->etype)
    {
    case TCHAR:
    case TSHORT:
        t = types[TINT];
        break;

    case TUCCHAR:
    case TUSHORT:
        t = types[TUINT];
        break;

    case TFLOAT:
        t = types[TDOUBLE];
    }
```

Uses TCHAR 30b, TDOUBLE 31a, TFLOAT 31a, TINT 30b, TSHORT 30b, TUCCHAR 30b, TUINT 30b, TUSHORT 30b, and types 32b.

```

⟨tcom() switch node kind cases 175a⟩+≡ (153b) <172b 175d>
case ODOTDOT:
    /*
     * tcom has already been called on this subtree
     */
    *n = *n->left;
    if(n->type == T)
        goto bad;
    break;

```

Uses ODOTDOT 137c and T 31i.

```

⟨global nodproto 175b⟩≡ (511d)
Node* nodproto;

```

```

⟨cinit() nodproto initialization 175c⟩≡ (50d)
nodproto = new(OPROTO, Z, Z);

```

Uses OPROTO 136f, Z 29a, new() 29d, and nodproto 175b.

9.6.13 Cast

Explicit casts (OCAST) are checked via the `tcast` compatibility table, which is more permissive than assignment: you can cast between integers and pointers, or between structs of the same kind. The compiler also inserts implicit OCAST nodes during type checking (e.g., when assigning an `int` to a `char` variable), making all narrowing conversions explicit in the AST before code generation.

```

⟨tcom() switch node kind cases 175d⟩+≡ (153b) <175a 176a>
case OCAST:
    if(n->type == T)
        break;
    if(n->type->width == types[TLONG]->width) {
        if(tcom(1, ADDR_OF|CASTOF))
            goto bad;
    } else
        if(tcom(1))
            goto bad;
    ⟨tcom() break if isfunct 438a⟩
    if(tcompat(n, l->type, n->type, tcast))
        goto bad;
    break;

```

Uses ADDR_OF-92 153a, CASTOF-93 153a, OCAST 118a, T 31i, TLONG 30b, tcast 175e, tcom() 152b, tcom() 153b, tcompat() 150a, and types 32b.

```

⟨global tcast 175e⟩≡ (518c)
long tcast[NTYPE];

```

Uses NTYPE 30a.

```

⟨global tcastinit 175f⟩≡ (518c)
Init tcastinit[] =
{
    TCHAR,  BNUMBER|BIND|BVOID, 0,
    TCHAR,  BNUMBER|BIND|BVOID, 0,
    TSHORT, BNUMBER|BIND|BVOID, 0,
    TUSHORT, BNUMBER|BIND|BVOID, 0,
    TINT,   BNUMBER|BIND|BVOID, 0,
    TUINT,  BNUMBER|BIND|BVOID, 0,
    TLONG,  BNUMBER|BIND|BVOID, 0,
    TULONG, BNUMBER|BIND|BVOID, 0,
    TVLONG, BNUMBER|BIND|BVOID, 0,

```



```

TUVLONG, BNUMBER|BIND|BVOID, 0,

TFLOAT, BNUMBER|BVOID, 0,
TDOUBLE, BNUMBER|BVOID, 0,

TIND, BINTEGER|BIND|BVOID, 0,
TVOID, BVOID, 0,
TSTRUCT, BSTRUCT|BVOID, 0,
TUNION, BUNION|BVOID, 0,
-1, 0, 0,
};

Uses BIND 121, BINTEGER 151b, BNUMBER 151b, BSTRUCT 121, BUNION 121, BVOID 121, TCHAR 30b, TDOUBLE 31a, TFLOAT 31a,
TIND 31c, TINT 30b, TLONG 30b, TSHORT 30b, TSTRUCT 31c, TUCCHAR 30b, TUINT 30b, TULONG 30b, TUNION 31c, TUSHORT 30b,
TUVLONG 30b, TVLONG 30b, and TVOID 31b.

```

9.6.14 Ternary expressions

For `x ? a : b`, the two branches must have compatible types. If they are identical, the result type is used directly; otherwise the usual arithmetic conversions are applied via `tcompat` and `arith`. A special case handles the literal 0 as a null pointer: if one branch is a pointer and the other is the constant zero, the zero is silently promoted to the pointer type.

```

⟨tcom() switch node kind cases 176a⟩+≡ (153b) <175d 177>
case OCOND:
    o = tcom(l);
    o |= tcom(r->left);
    if(o | tcom(r->right))
        goto bad;
    ⟨tcom() in OCOND, nil handling 176b⟩
    if(sametype(r->right->type, r->left->type)) {
        r->type = r->right->type;
        n->type = r->type;
        break;
    }
    if(tcompat(r, r->left->type, r->right->type, trel))
        goto bad;
    arith(r, false);
    n->type = r->type;
    break;

```

Uses OCOND 118f, arith() 160a, sametype() 145c, tcom() 152b, tcompat() 150a, and trel 164b.

```

⟨tcom() in OCOND, nil handling 176b⟩≡ (176a)
if(r->right->type->etype == TIND && vconst(r->left) == 0) {
    r->left->type = r->right->type;
    r->left->vconst = 0;
}
if(r->left->type->etype == TIND && vconst(r->right) == 0) {
    r->right->type = r->left->type;
    r->right->vconst = 0;
}

```

Uses TIND 31c and vconst() 176c.

```

⟨function vconst 176c⟩≡ (518c)
int
vconst(Node *n)
{
    int i;

```

```

if(n == Z)
    goto no;
if(n->op != OCONST)
    goto no;
if(n->type == T)
    goto no;

switch(n->type->etype)
{
case TFLOAT:
case TDOUBLE:
    i = 100;
    if(n->fconst > i || n->fconst < -i)
        goto no;
    i = n->fconst;
    if(i != n->fconst)
        goto no;
    return i;

case TVLONG:
case TUVLONG:
    i = n->vconst;
    if(i != n->vconst)
        goto no;
    return i;

case TCHAR:
case TUCCHAR:
case TSHORT:
case TUSHORT:
case TINT:
case TUINT:
case TLONG:
case TULONG:
case TIND:
    i = n->vconst;
    if(i != n->vconst)
        goto no;
    return i;
}
no:
    return -159; /* first uninteresting constant */
}

```

Uses OCONST 111a, T 31i, TCHAR 30b, TDOUBLE 31a, TFLOAT 31a, TIND 31c, TINT 30b, TLONG 30b, TSHORT 30b, TUCCHAR 30b, TUINT 30b, TULONG 30b, TUSHORT 30b, TUVLONG 30b, TVLONG 30b, and Z 29a.

9.6.15 Prefix/postfix

```

⟨tcom() switch node kind cases 177⟩+≡ (153b) <176a 178a>
case OPREINC:
case OPREDEC:
case OPOSTINC:
case OPOSTDEC:
    if(tcom(1))
        goto bad;
    if(tlvalue(1))
        goto bad;
    ⟨tcom() break if isfunct 438a⟩
    if(tcompat(n, l->type, types[TINT], tadd))

```

```

        goto bad;
n->type = l->type;

if(n->type->etype == TIND)
if(n->type->link->width < 1) {
    snap(n->type->link);
    if(n->type->link->width < 1)
        diag(n, "inc/dec of a void pointer");
}
break;

```

Uses OPOSTDEC 118j, OPOSTINC 118j, OPREDEC 118h, OPREINC 118h, TIND 31c, TINT 30b, diag() 496b, snap() 147b, tadd 151c, tcom() 152b, tcompat() 150a, tlvalue() 166b, and types 32b.

9.6.16 sizeof()

```

<tcom() switch node kind cases 178a>+≡ (153b) <177 179a>
case OSIZE:
    if(l != Z) {
        if(l->op != OSTRING && l->op != OLSTRING)
            if(tcom(l, 0))
                goto bad;
        <tcom() when OSIZE, check if sizeof bitfield 419g>
        n->type = l->type;
    }
    <tcom() when OSIZE, sanity check n 178b>
    n->op = OCONST;
    n->left = Z;
    n->right = Z;
    n->vconst = convvtox(n->type->width, TINT);
    n->type = types[TINT];
    break;

```

Uses OCONST 111a, OLSTRING 113a, OSIZE 119a, OSTRING 112c, TINT 30b, Z 29a, convvtox() 65d, tcom() 153b, and types 32b.

```

<tcom() when OSIZE, sanity check n 178b>≡ (178a)
if(n->type == T)
    goto bad;
if(n->type->width <= 0) {
    diag(n, "sizeof undefined type");
    goto bad;
}
if(n->type->etype == TFUNC) {
    diag(n, "sizeof function");
    goto bad;
}

```

Uses T 31i, TFUNC 31c, and diag() 496b.

9.7 Typechecking statements

Statements like `return`, `switch`, and `if` also go through `tcom`—they are just more cases in the big switch. The typechecker validates that `return` values match the function’s declared return type, that `switch` expressions are integers, and that `if` conditions are valid boolean-like values.

9.7.1 return

The `ORETURN` node was stamped with the function's return type during parsing (from `thisfntype`). The type-checker verifies the returned expression is assignment-compatible with that type, and inserts an `OCAST` if the types differ but are compatible (e.g., returning an `int` from a `long` function).

```
<tcom() switch node kind cases 179a>+≡ (153b) <178a 227e>
case ORETURN:
    if(l == Z) {
        if(n->type->etype != TVOID)
            warn(n, "null return of a typed function");
        break;
    }
    if(tcom(l))
        goto bad;
    <tcom() when ORETURN, typing extensions 426d>
    if(tcompat(n, n->type, l->type, tasign))
        break;
    <tcom() when ORETURN, const checking 191>
    if(!sametype(n->type, l->type)) {
        l = new1(OCAST, l, Z);
        l->type = n->type;
        n->left = l;
    }
    break;
```

Uses `OCAST` 118a, `ORETURN` 108b, `TVOID` 31b, `Z` 29a, `new1()` 29h, `sametype()` 145c, `tasign` 166c, `tcom()` 152b, `tcompat()` 150a, and `warn()` 142.

9.7.2 switch

The `switch` expression must be an integer type (no pointers, no `vlong`). Rather than adding a dedicated check, the parser wraps the expression in `0 - (0 - x)`, which forces the usual arithmetic conversions and will produce a type error if `x` is a pointer. A separate check in `gen` also validates against the `typeswitch` set.

```
<ulstmt rule, SWITCH case, adjust cexpr node 179b>≡ (109c)
// generate (0:int - (0:int - x))
// which will force the usual arithmetic conversions
// (and will be simplified later by some transformations)

$$ = new(OCONST, Z, Z);
$$->vconst = 0;
$$->type = types[TINT];
$3 = new(OSUB, $$, $3);

$$ = new(OCONST, Z, Z);
$$->vconst = 0;
$$->type = types[TINT];
$3 = new(OSUB, $$, $3);

<gen() when OSWITCH, typecheck condition 179c>≡ (279b)
if(!typeswitch[l->type->etype]) {
    diag(n, "switch expression must be integer");
    break;
}
```

Uses `diag()` 496b and `typeswitch` 179d.

```
<global typeswitch 179d>≡ (511d)
// set<Type_kind>
char* typeswitch;
```

$\langle \text{tinit}() \text{ initialise 32 bits defaults type sets 180a} \rangle \equiv$ (50a) 271b▷

```
typeswitch = typechl;
```

Uses typechl 180b and typeswitch 179d.

$\langle \text{global typechl 180b} \rangle \equiv$ (518c)

```
char typechl[NTYPE];
```

Uses NTYPE 30a.

$\langle \text{global typechlinit 180c} \rangle \equiv$ (518c)

```
int typechlinit[] =
```

```
{
    TCHAR, TUCCHAR, TSHORT, TUSHORT, TINT, TUINT, TLONG, TULONG, -1,
};
```

Uses TCHAR 30b, TINT 30b, TLONG 30b, TSHORT 30b, TUCCHAR 30b, TUINT 30b, TULONG 30b, and TUSHORT 30b.

9.7.3 if

$\langle \text{bcomplex}() \text{ typecheck } n \text{ 180d} \rangle \equiv$ (199e)

```
if(n->type != T)
    if(tcompat(n, T, n->type, tnot))
        n->type = T;
```

Uses T 31i, tcompat() 150a, and tnot 165b.

9.8 Typechecking declarations and constant evaluation

Some type checking happens not in tcomo but during parsing itself, as part of dodecl and its helpers. Enumeration constants, array sizes, and function parameter types all require validation and constant evaluation at declaration time.

9.8.1 Enumerations

Enum constants are evaluated eagerly during parsing via complex/evconst. Each value is stored in En.lastenum (or En.floatenum for the exotic float-enum extension) and recorded in the symbol's vconst field. The type of the enum is widened via maxtype to accommodate the largest constant.

$\langle \text{En value fields 180e} \rangle \equiv$ (133a)

```
vlong lastenum; /* value of current enum */
double floatenum; /* value of current enum */ // for floats enums
```

$\langle \text{doenum}() \text{ typecheck and evaluate expression 180f} \rangle \equiv$ (134c)

```
complex(n); // will call evconst()
```

Uses complex() 210a.

$\langle \text{doenum}() \text{ save current value of enumeration constant 180g} \rangle \equiv$ (134c)

```
if(typefd[en.cenum->etype])
    en.floatenum = n->fconst;
else
    en.lastenum = n->vconst;
```

Uses en 133b and typefd 149c.

$\langle \text{Sym enum value fields 180h} \rangle + \equiv$ (34c) <134a

```
vlong vconst;
double fconst;
```

```

⟨doenum() set value for symbol 181a⟩≡ (134c)
    if(typefd[s->tenum->etype]) {
        s->fconst = en.floatenum;
        en.floatenum++;
    } else {
        s->vconst = convvtox(en.lastenum, s->tenum->etype);
        en.lastenum++;
    }

```

Uses convvtox() 65d, en 133b, and typefd 149c.

```

⟨function maxtype 181b⟩≡ (525)
/*
 * this routine is very suspect.
 * ANSI requires the enum type to
 * be represented as an 'int'
 * this means that 0x81234567
 * would be illegal. this routine
 * makes signed and unsigned go
 * to unsigned.
 */
Type*
maxtype(Type *t1, Type *t2)
{
    if(t1 == T)
        return t2;
    if(t2 == T)
        return t1;
    if(t1->etype > t2->etype)
        return t1;
    return t2;
}

```

Uses T 31i.

9.8.2 Arrays

```

⟨dodecl() locals 181c⟩+≡ (103a) <135e
    long v;

```

```

⟨dodecl() switch node kind cases, case OARRAY, if array has a size 181d⟩≡ (135f)
    t->width = 0; // array[], size could be set by doinit (or specified in n1)
    if(n1 != Z) {
        complex(n1); // will call evconst()
        v = (n1->op == OCONST) ? n1->vconst : -1;
        if(v <= 0) {
            diag(n, "array size must be a positive constant");
            v = 1;
        }
        t->width = v * t->link->width;
    }

```

Uses OCONST 111a, Z 29a, complex() 210a, and diag() 496b.

9.8.3 Functions

Function parameters undergo implicit type adjustments via `paramconv`: arrays become pointers, functions become function pointers, and in old-style (K&R) declarations, `char/short` are promoted to `int` and `float` to

double. The **f** parameter distinguishes prototype context (where narrow types are preserved) from old-style context (where they are widened).

```
<fnproto1() when OPROTO case, adjust parameter type 182a>≡ (136h)
    if(lastdcltype != T)
        *t = *paramconv(lastdcltype, true);
```

Uses **T 31i**, **lastdcltype 103b**, and **paramconv() 182c**.

```
<pdecl() adjust type 182b>≡ (139c)
    t = paramconv(t, class==CPARAM);
```

Uses **CPARAM 33b** and **paramconv() 182c**.

```
<function paramconv 182c>≡ (525)
    /// fnproto1 | pdecl -> <>
    Type*
    paramconv(Type *t, bool f)
    {

        switch(t->etype) {
            <paramconv() switch etype cases 414b>
            <paramconv() switch etype, adjust type when not f cases 182e>
            }
        return t;
    }
```

```
<pdecl() sanity check type 182d>≡ (139c)
    if(typesu[t->etype] && t->width <= 0)
        diag(Z, "incomplete structure: %s", t->tag->name);
```

Uses **Z 29a**, **diag() 496b**, and **typesu 149a**.

```
<paramconv() switch etype, adjust type when not f cases 182e>≡ (182c)
    case TFLOAT:
        if(!f)
            t = types[TDOUBLE];
        break;

    case TCHAR:
    case TSHORT:
        if(!f)
            t = types[TINT];
        break;

    case TUCCHAR:
    case TUSHORT:
        if(!f)
            t = types[TUINT];
        break;
```

Uses **TCHAR 30b**, **TDOUBLE 31a**, **TFLOAT 31a**, **TINT 30b**, **TSHORT 30b**, **TUCCHAR 30b**, **TUINT 30b**, **TUSHORT 30b**, and **types 32b**.

9.8.4 Typedef expansions

```
<function tcopy 182f>≡ (525)
    /*
    * make a copy of a typedef
    * the problem is to split out incomplete
    * arrays so that it is in the variable
    * rather than the typedef.
    */
```

```

Type*
tcopy(Type *t)
{
    Type *tl, *tx;
    int et;

    if(t == T)
        return t;
    et = t->etype;
    if(typesu[et])
        return t;

    tl = tcopy(t->link); // go deep

    if((et == TARRAY && t->width == 0) || tl != t->link) {
        tx = copytyp(t);
        tx->link = tl;
        return tx;
    }
    return t;
}

```

Uses T 31i, TARRAY 31c, copytyp() 127d, tcopy() 182f, and typesu 149a.

9.9 Lint checking

5c includes several lint-like checks that go beyond type safety: unused variables (`Sym.apsed`) and unused labels are detected at scope exit in `revertdecl`, “set but not used” warnings catch dead stores, constant `if` conditions are flagged, and unreachable code after `return/goto` is warned about. The `SET` and `USED` macros allow programmers to silence specific warnings. These checks are scattered across the parser, type-checker, and code generator.

9.9.1 Unused variables

The `Sym.apsed` flag tracks whether a local variable or parameter has been referenced. It is set to `true` during parsing whenever the variable’s name appears in an expression, and checked at scope exit in `revertdecl`: if `apsed` is still false for an `CAUTO` or `CPARAM` symbol, a warning is emitted. This is a simple dataflow approximation—any mention counts as a “use,” even in dead code.

⟨Sym identifier value, checking fields 183a⟩≡ (36b)

```
bool apsed;
```

⟨syminit() remaining initialisations 183b⟩≡ (37h)

```
s->apsed = false;
```

⟨adeccl() initialize symbol 183c⟩≡ (140c)

```
s->apsed = false;
```

⟨name rule, LNAME case, adjust more fields 183d⟩≡ (114b)

```
$1->apsed = true;
```

⟨revertdecl() DAUTO case, warn if auto declared but not used 183e⟩≡ (41d)

```

if(!s->apsed) {
    nearln = s->varlineno;
    if(s->class == CAUTO)
        warn(Z, "auto declared and not used: %s", s->name);
    if(s->class == CPARAM)
        warn(Z, "param declared and not used: %s", s->name);
}

```

Uses CAUTO 33b, CPARAM 33b, Z 29a, nearln 29g, and warn() 142.

9.9.2 Used before set

This warning is detected by the register allocator rather than the type-checker: during dataflow analysis, if a variable is live at a point where it has not yet been assigned, it was potentially used before set. This detection happens late because it requires dataflow information that is only available after code generation.

9.9.3 False positives silencing: SET()/USED()

The `USED(x)` and `SET(x)` macros are pseudo-statements that silence specific lint warnings: `USED(x)` marks `x` as used (preventing “declared but not used” warnings), and `SET(x)` marks it as set (preventing “used before set” warnings). These are parsed as `OUSED/OSET` statement nodes and handled at type-checking time by setting the appropriate flags on the symbol. They generate no code.

```
<token declarations 184a>+≡ (100a) <42d 434b>
%token LUSED LSET
```

```
<itab entries, kencc extensions 184b>≡ (59) 434a>
"SET", LSET, 0,
"USED", LUSED, 0,
```

Uses LSET and LUSED.

```
<statement nodes 184c>+≡ (27) <109b
OUSED,
OSET,
```

```
<ulstmnt rule 184d>+≡ (106c) <109c
| LUSED '(' zelist ')' ';' { $$ = new(OUSED, $3, Z); }
| LSET '(' zelist ')' ';' { $$ = new(OSET, $3, Z); }
```

```
<gen() switch node kind cases 184e>≡ (197e) 198a>
case OSET:
case OUSED:
    usedset(n->left, o);
    break;
```

Uses OSET 184c, OUSED 184c, and `usedset()` 184f.

```
<function usedset 184f>≡ (530b)
/// gen -> <>
void
usedset(Node *n, int o)
{
    if(n->op == OLIST) {
        usedset(n->left, o);
        usedset(n->right, o);
        return;
    }

    complex(n);

    switch(n->op) {
    case ONAME:
        if(o == OSET)
            gins(ANOP, Z, n);
        else
            gins(ANOP, n, Z);
        break;
    <usedset() switch node op cases 416f>
    }
}
```

Uses OLIST 29b, ONAME 27, OSET 184c, Z 29a, `complex()` 210a, and `usedset()` 184f.

9.9.4 Redeclared or unused labels

Labels are tracked by repurposing two Node fields: `complex` records whether the label has been defined (`label:`), and `addable` records whether it has been used (`goto label`). At scope exit, `revertdecl` warns if a label was defined but never used. Redefinition of the same label is caught immediately in `dcllabel`.

```
<dcllabel() when new label, set def and use fields 185a>≡ (109a)
  n->complex = defcontext;
  n->addable = !defcontext;
```

```
<dcllabel() if n not null, mark node as declared or used 185b>≡ (109a)
  if(n != Z) {
    if(defcontext) {
      if(n->complex)
        diag(Z, "label reused: %s", s->name);
      n->complex = true; // declared
    } else
      n->addable = true; // used
    return n;
  }
```

Uses Z 29a and diag() 496b.

```
<revertdecl() DLABEL case, warn if label not used 185c>≡ (41c)
  if(s->label && !s->label->addable)
    warn(s->label, "label declared and not used \"%s\"", s->name);
```

Uses warn() 142.

9.9.5 Unreachable code

The global `canreach` flag is set to false after processing a `return` or `goto` in a statement list. If subsequent statements are encountered while `canreach` is false, a warning is emitted (once, controlled by `warnreach`). Labels and `case` statements reset reachability since they can be jumped to from elsewhere.

```
<global canreach 185d>≡ (511d)
  bool canreach;
```

```
<global warnreach 185e>≡ (511d)
  bool warnreach;
```

```
<codgen() initialisation before call to gen 185f>≡ (194c)
  canreach = true;
  warnreach = true;
```

Uses `canreach` 185d and `warnreach` 185e.

```
<gen() if not canreach 185g>≡ (197e)
  if(!canreach) {
    switch(o) {
      case OLABEL:
      case OCASE:
      case OLIST:
      case OCOMMA:
      case OBREAK:
      case OFOR:
      case OWHILE:
      case ODWHILE:
        /* all handled specially - see switch body below */
        break;
    default:
      if(warnreach) {
```

```

        warn(n, "unreachable code %0", o);
        warnreach = false;
    }
}
}

```

Uses OBREAK 108b, OCASE 109b, OCOMMA 110g, ODWHILE 107c, OFOR 107c, OLABEL 108e, OLIST 29b, OWHILE 107c, canreach 185d, warn() 142, and warnreach 185e.

If

```

⟨bcomplex() check for deadheads 186a⟩≡ (199e)
    if(c != Z && n->op == OCONST && deadheads(c))
        return true;

```

Uses OCONST 111a, Z 29a, and deadheads() 187a.

```

⟨function deadhead 186b⟩≡ (518c)
    /// deadheads -> <>
    /*
     * return true if it is impossible to jump into the middle of n.
     */
    static bool
    deadhead(Node *n, bool caseok)
    {
    loop:
        if(n == Z)
            return true;

        switch(n->op) {

        case OCASE:
            if(!caseok)
                return false;
            goto rloop;
        case OLABEL:
            return false;

        case OLIST:
            if(!deadhead(n->left, caseok))
                return false;
        rloop:
            n = n->right;
            goto loop;

        case OWHILE:
        case ODWHILE:
        case OFOR:
            goto rloop;

        case OIF:
            return deadhead(n->right->left, caseok) &&
                deadhead(n->right->right, caseok);

        case OSWITCH:
            return deadhead(n->right, true);

```

```

case ORETURN:
case OGOTO:
case OCONTINUE:
case OBREAK:
    return true;

case OSET:
case OUSED:
    return true;
}
return true;
}

```

Uses OBREAK 108b, OCASE 109b, OCONTINUE 108b, ODWHILE 107c, OFOR 107c, OGOTO 108e, OIF 107a, OLABEL 108e, OLIST 29b, ORETURN 108b, OSET 184c, OSWITCH 109b, OUSED 184c, OWHILE 107c, Z 29a, and deadhead() 186b.

```

⟨function deadheads 187a⟩≡ (518c)
    /// gen | bcomplex -> <>
    bool
    deadheads(Node *c)
    {
        return deadhead(c->left, false) && deadhead(c->right, false);
    }

```

Uses deadhead() 186b.

```

⟨gen() locals 187b⟩≡ (197e) 198b▷
    bool f;

```

```

⟨gen() switch node kind cases, OIF case, if bcomplex error 187c⟩≡ (198c)
    if(err) {
        if(typefd[l->type->etype])
            f = !l->fconst;
        else
            f = !l->vconst;
        if(debug['c'])
            print("%L const if %s\n", nearln, f ? "false" : "true");

        if(f) {
            canreach = true;
            supgen(n->right->left);
            oldreach = canreach;
            canreach = true;
            gen(n->right->right);
            /*
             * treat constant ifs as regular ifs for
             * reachability warnings.
             */
            if(!canreach && oldreach && debug['w'] < 2)
                warnreach = false;
        } else {
            canreach = true;
            gen(n->right->left);
            oldreach = canreach;
            canreach = true;
            supgen(n->right->right);
            /*
             * treat constant ifs as regular ifs for
             * reachability warnings.
             */
            if(!oldreach && canreach && debug['w'] < 2)

```

```

        warnreach = false;
    canreach = oldreach;
}
}

```

Uses `canreach` 185d, `debug` 474a, `gen()` 197e, `nearln` 29g, `supgen()` 188b, `typefd` 149c, and `warnreach` 185e.

`supgen()`

```

⟨global suppress 188a⟩≡ (535b)
    bool suppress;

```

```

⟨function supgen 188b⟩≡ (530b)
    void
    supgen(Node *n)
    {
        bool owarn;
        long opc;
        Prog *olastp;

        if(n == Z)
            return;

        suppress++;
        owarn = warnreach;
        opc = pc;
        oldestp = lastp;

        warnreach = false;

        gen(n);

        lastp = oldestp;
        oldestp->link = nil;
        pc = opc;
        warnreach = owarn;
        suppress--;
    }

```

Uses `Z` 29a, `gen()` 197e, and `warnreach` 185e.

9.9.6 Missing return

After processing the entire function body, if `canreach` is still true and the return type is not `void`, the function falls off the end without a `return`. By default this is an error (ANSI C requires it); with `-B` it is downgraded to a warning.

```

⟨codgen() warn for possible missing return after call to gen 188c⟩≡ (194c)
    if(canreach && thisfntype->link->etype != TVOID){
        if(debug['B'])
            warn(Z, "no return at end of function: %s", n1->sym->name);
        else
            diag(Z, "no return at end of function: %s", n1->sym->name);
    }

```

Uses `TVOID` 31b, `Z` 29a, `canreach` 185d, `debug` 474a, `diag()` 496b, `thisfntype` 105b, and `warn()` 142.

9.9.7 Unused expression

If an expression like `a + b` appears as a statement (no assignment, no side effects), `nullwarn` warns that the result is discarded. The children are still evaluated for their side effects (function calls, increments) but the computed value is thrown away.

```
<cgenrel() nullwarn check if nn is null 189a>≡ (420a 252 248a 236h 235b 228b)
  if(nn == Z) {
    nullwarn(1, r);
    break;
  }
```

Uses `Z` 29a and `nullwarn()` 189b.

```
<function nullwarn 189b>≡ (530a)
void
nullwarn(Node *l, Node *r)
{
  warn(Z, "result of operation not used");
  if(l != Z)
    cgen(l, Z);
  if(r != Z)
    cgen(r, Z);
}
```

Uses `Z` 29a and `warn()` 142.

```
<cgenrel() warn if target node is nil 189c>≡ (224c)
  if(nn == Z) {
    switch(o) {
      case OINDEX:
        nullwarn(1, r);
        break;
      default:
        nullwarn(Z, Z);
        break;
    }
    return;
  }
```

Uses `OINDEX`, `Z` 29a, and `nullwarn()` 189b.

9.9.8 Constant if: 5c -c

The `-c` flag enables a warning when an `if` condition is a compile-time constant (like `if(1)` or `if(0)`). This is not on by default because many C programs use constant conditions for conditional compilation (`if(DEBUG)` or `if(sizeof(int)==4)`).

9.9.9 Out of range shifting

After `ccom` folds constants, the shift amount may be a known constant. If it exceeds the type width (e.g., shifting a 32-bit `int` by 35), the compiler warns about a “stupid shift.”

```
<ccom() when OASHR/OASHL/OLSHR, check for stupid shift 189d>≡ (296a)
  if(r->op == OCONST) {
    t = n->type->width * 8; /* bits per byte */
    if(r->vconst >= t || r->vconst <= -t)
      warn(n, "stupid shift: %lld", r->vconst);
  }
```

Uses `OCNST` 111a and `warn()` 142.

`<ccom() switch node kind cases 190a>≡ (292) 293a▷`

```

case OAS:
case OASADD:
case OASSUB:
case OASMUL:
case OASLMUL:
case OASDIV:
case OASLDIV:
case OASMOD:
case OASLMOD:
case OASASHR:
case OASASHL:
case OASLSHR:
case OASAND:
case OASOR:
case OASXOR:
    ccom(l);
    ccom(r);
    if(n->op == OASLSHR || n->op == OASASHR || n->op == OASASHL)
        if(r->op == OCONST) {
            t = n->type->width * 8; /* bits per byte */
            if(r->vconst >= t || r->vconst < 0)
                warn(n, "stupid shift: %lld", r->vconst);
        }
    break;

```

Uses OAS 116a, OASADD 116c, OASAND 116c, OASASHL 116c, OASASHR 116c, OASDIV 116c, OASLDIV 233b, OASLMOD 233b, OASLMUL 233b, OASLSHR 233b, OASMOD 116c, OASMUL 116c, OASOR 116c, OASSUB 116c, OASXOR 116c, OCONST 111a, ccom() 292, and warn() 142.

9.9.10 Useless comparisons

The `compar` function detects comparisons whose result is always true or always false due to type ranges—for example, comparing an `unsigned char` with a negative constant, or testing whether an `unsigned int` is less than zero. These are almost always bugs.

9.10 Const checking

The `constas` function warns when assigning to a `const`-qualified variable, or when assigning a `const` pointer to a non-`const` pointer (e.g., passing a `const char*` where a `char*` is expected). It walks the pointer chain level by level, checking the `GCONSTNT` bit in each type’s `garb` (“garbage”) field—Ken Thompson’s characteristically dismissive name for type qualifiers.

`<tcom() when OAS, const checking 190b>≡ (166a)`

```
constas(n, l->type, r->type);
```

Uses `constas()` 190d.

`<tcom() when OASxxx, const checking 190c>≡ (168f 167c)`

```
constas(n, l->type, r->type);
```

Uses `constas()` 190d.

`<function constas 190d>≡ (518c)`

```

void
constas(Node *n, Type *il, Type *ir)
{
    Type *l, *r;

    l = il;

```

```

r = ir;

if(l == T)
    return;
if(l->garb & GCONSTNT) {
    warn(n, "assignment to a constant type (%T)", il);
    return;
}
if(r == T)
    return;
for(;;) {
    if(l->etype != TIND || r->etype != TIND)
        break;
    l = l->link;
    r = r->link;
    if(l == T || r == T)
        break;
    if(r->garb & GCONSTNT)
        if(!(l->garb & GCONSTNT)) {
            warn(n, "assignment of a constant pointer type (%T)", ir);
            break;
        }
}
}

```

Uses GCONSTNT 33a, T 31i, TIND 31c, and warn() 142.

$\langle \text{tcomo}() \text{ when } ORETURN, \text{const checking } 191 \rangle \equiv$ (179a)
`constas(n, n->type, l->type);`

Uses constas() 190d.

Chapter 10

Assembly Generation

The compiler generates ARM assembly directly from the AST—there is no intermediate representation (no SSA, no RTL, no CFG). The code generator walks the tree recursively: `codgen` handles function prologues/epilogues and calls `gen` for statements, which calls `cgen` for expressions. Each expression node is compiled by choosing the right ARM instruction based on the node’s type and “addressability” (a classification of how easily a value can be accessed: register, memory, constant, etc.). The result is a linked list of `Prog` instructions written to the object file by `outcode`.

10.1 No intermediate representation

Every mainstream compiler in 2026 has at least one intermediate representation. `gcc` has GIMPLE, a tree-shaped IR added in 2003, plus RTL, a low-level expression-tree IR that goes back to the 1980s. LLVM has its SSA-based IR, which the whole toolchain was designed around from its first release in 2003. `rustc` has MIR (2016); Swift has SIL (2014). SSA itself was invented by Cytron, Ferrante, Rosen, Wegman, and Zadeck at IBM in 1988. None of these existed when `5c`’s ancestors were built. `pcc` (Johnson, 1978), Ritchie’s original C compiler, and `lcc` (Fraser and Hanson, 1995) all tree-walk the typed AST and emit target-specific instructions directly, exactly the way `5c` does. The absence of an IR is not a simplification choice `5c` made for this book; it is the older way compilers worked, preserved in the Plan 9 toolchain because Ken Thompson and Rob Pike valued compile speed and small code size over peak generated-code performance.

Working without an IR has a real cost. Classical optimizations that rely on a control-flow graph—dead code elimination across basic blocks, common subexpression elimination, loop-invariant code motion, induction-variable analysis, register allocation via graph coloring—are either impossible or painful to implement on a raw AST. `5c` gets away with the trade-off for three reasons. First, ARM assembly is regular and its instruction set maps cleanly onto C operators, so a naive one-to-one translation already produces reasonable code. Second, AST-level rewrites in `ccom()`²⁹² (Chapter 12) catch the easy wins: constant folding, algebraic simplification, strength reduction for multiplies by a constant. Third, a simple peephole pass and a linear-scan register allocator in `peep()`^{366c} and `regopt()`^{339b} (Chapter 13) catch most of what remains. The compiler fits in a few thousand lines of C and produces code that runs perhaps 20% slower than `gcc`’s—an acceptable trade when the speed of compilation matters more than the speed of compiled code, as it did for Plan 9.

10.2 Overview

The code generator works by recursive descent over the AST, directly producing ARM assembly instructions—there is no intermediate representation. The pipeline is `codgen` (per function) $\rightarrow$ `gen` (statements) $\rightarrow$ `cgen` (expressions), with a few helpers: `gpseudo` emits pseudo-instructions like `TEXT` and `DATA`, `gbranch` emits conditional/unconditional jumps, `gmove` emits typed moves between registers and memory, and `gins` emits general

arithmetic instructions. The assembly output is a linked list of `Prog` nodes that `outcode` later serializes. Use `5c -N -S` to see the unoptimized assembly for any C file.

10.3 `pc`, `p`, and `nextpc()`

Instructions are allocated one at a time by `nextpc`, which appends a new `Prog` to the linked list (via `p->link`) and assigns it a virtual program counter. The global `p` always points to the most recently emitted instruction, and `pc` is the running counter. Branch targets are expressed as `Prog` pointers, which are later resolved into offsets by `outcode`. This list-based representation makes it easy to insert, delete, and rearrange instructions during the optimization passes.

```
<global pc 193a>≡ (535b)
    long pc;
```

```
<ginit() pc initialisation 193b>≡ (51c)
    pc = 0;
```

```
<function nextpc 193c>≡ (537b)
    /// gins | gclean | ... -> <>
    void
    nextpc(void)
    {

        p = alloc(sizeof(Prog));
        *p = zprog;
        p->lineno = nearln; // for origin tracking in db (from assembly to C)
        pc++;

        // add_tail(p, firstp/lastp)
        if(firstp == P) {
            firstp = p;
            lastp = p;
            return;
        }
        lastp->link = p;
        lastp = p;
    }
```

Uses `alloc()` 500b and `nearln` 29g.

```
<global p 193d>≡ (535b)
    Prog* p;
```

```
<global firstp 193e>≡ (535b)
    // list<ref_own<Prog> (next = Prog.link, last = lastp)
    Prog* firstp;
```

```
<Prog extra fields(arm) 193f>≡ (43a)
    // list<ref<Prog>> (head = firstp)
    Prog* link;
```

```
<constant P 193g>≡ (531b)
    #define P ((Prog*)nil)
```

```
<global lastp 193h>≡ (535b)
    // ref<Prog> (head = firstp)
    Prog* lastp;
```

```

⟨ginit() misc initialisations 194a⟩≡ (51c) 201d▷
    firstp = P;
    lastp = P;

⟨gclean() generate last opcode, AEND 194b⟩≡ (49c)
    nextpc();
    p->as = AEND;

```

10.4 Functions: codgen()

`codgen` is called from the yacc action for each function definition. It emits a `TEXT` pseudo-instruction for the function prologue (recording the stack frame size), calls `gen` to compile the body, appends a `RET`, and then runs the optimization passes (`regopt` and `peep`). The stack frame size in the `TEXT` instruction is backpatched after `gen` returns, because local variables may be allocated during code generation.

```

⟨function codgen 194c⟩≡ (530b)
    // main -> compile -> yyparse -> xdecl -> <>
    //@Scheck: used by cc.y
    void codgen(Node *n, Node *nn)
    {
        Prog *sp;
        ⟨codgen() locals 194d⟩

        ⟨codgen() initialisation 258d⟩
        ⟨codgen() set n1 node to node in nn where have ONAME 194e⟩
        nearln = nn->lineno;

        gpseudo(ATEXT, n1->sym, nodconst(stkoff));
        sp = p;

        ⟨codgen() if complex return type 272d⟩
        ⟨codgen() if use REGARG 337b⟩

        ⟨codgen() initialisation before call to gen 185f⟩
        // generate the assembly for the statements in the body
        gen(n);

        ⟨codgen() warn for possible missing return after call to gen 188c⟩
        ⟨codgen() before RET 341b⟩
        gbranch(ORETURN);

        ⟨codgen() register optimisation 339a⟩

        sp->to.offset += maxargsafe;
    }

```

Uses `ORETURN` 108b, `gen()` 197e, `nearln` 29g, and `stkoff` 196c.

```

⟨codgen() locals 194d⟩≡ (194c) 337a▷
    Node *n1;

⟨codgen() set n1 node to node in nn where have ONAME 194e⟩≡ (194c)
    /*
     * isolate name
     */
    for(n1 = nn;; n1 = n1->left) {
        if(n1 == Z) {
            diag(nn, "cant find function name");
            return;
        }
    }

```

```

    }
    if(n1->op == ONAME)
        break;
}

```

Uses ONAME 27, Z 29a, and diag() 496b.

```

⟨global constnode 195a⟩≡ (535b)
Node constnode;

```

```

⟨ginit() special nodes initialisation 195b⟩≡ (51c) 209b▷
constnode.op = OCONST;
constnode.class = CXXX;
constnode.complex = 0;
constnode.addable = 20;
constnode.type = types[TLONG];

```

Uses CXXX 33b, OCONST 111a, TLONG 30b, and types 32b.

```

⟨function nodconst 195c⟩≡ (537b)
Node*
nodconst(long v)
{
    constnode.vconst = v;
    return &constnode;
}

```

10.4.1 gpseudo()

gpseudo emits pseudo-instructions like TEXT (function entry), DATA (initialized data), and GLOBL (global variable size). These are not real ARM instructions but directives that the linker interprets. The “from” operand is always a symbol name, and the “to” operand carries the payload (frame size for TEXT, initial value for DATA).

```

⟨function gpseudo(arm) 195d⟩≡ (537b)
/// codgen(function generation) | gextern -> <>
void
gpseudo(int a, Sym *s, Node *n)
{
    nextpc();

    p->as = a;
    p->from.type = D_OREG;
    p->from.sym = s;
    p->from.symkind = (s->class == CSTATIC) ? N_INTERN : N_EXTERN;
    ⟨gpseudo() if TEXT, set possible TEXT attributes 412c⟩
    naddr(n, &p->to);
    if(a == ADATA || a == AGLOBL)
        pc--;
}

```

Uses CSTATIC 33b.

10.4.2 gbranch()

gbranch emits a branch instruction to a given destination Prog. For unconditional branches it emits AB (always branch); for conditional branches, the condition code is set via scnd. ORETURN branches to the function’s

epilogue. The destination is stored as a `Prog` pointer in `to.offset`, which the linker later resolves to a PC-relative offset.

```
⟨function gbranch(arm) 196a⟩≡ (537b)
  /// codgen | ... -> <>
  void
  gbranch(int o)
  {
    int a;

    nextpc();

    switch(o) {
    case ORETURN: a = ARET; break;
    case OGOTO:   a = AB;   break;
    default:
      a = AGOK;
      diag(Z, "bad in gbranch %0", o);
      nextpc();
    }
    p->as = a;
  }
```

Uses `OGOTO` 108e, `ORETURN` 108b, `Z` 29a, and `diag()` 496b.

10.4.3 patch()

The compiler generates branch instructions before it knows where they will land — for instance, the false-branch of an `if` is emitted before the “else” body exists. `patch` fills in the target after the fact: it sets the branch’s destination to `pc`, the address of the next instruction that will be emitted. The typical idiom is `gbranch; sp = p; ...; patch(sp, pc)`.

```
⟨function patch 196b⟩≡ (537b)
  /// gmove | gen | ... -> <>
  void
  patch(Prog *op, long pc)
  {

    op->to.type = D_BRANCH;
    op->to.offset = pc;
  }
```

10.4.4 Offset

Local variables are assigned stack offsets during declaration parsing (in `adec1`). `autoffset` grows as locals are declared; `stkoff` records the maximum, which becomes the frame size in the `TEXT` instruction. Because `autoffset` is saved/restored by `markdcl/revertdcl`, locals in independent blocks can reuse the same stack slots—a simple space optimization.

```
⟨global stkoff 196c⟩≡ (511d)
  long stkoff;
```

```
⟨global autoffset 196d⟩≡ (511d)
  long autoffset;
```

```
⟨cinit() namespace globals initialization 196e⟩+≡ (50d) <39i
  autoffset = 0;
```

Uses `autoffset` 196d.

`<argmark() initializations 197a>≡ (138a)`

```
    autoffset = align(0, thisfntype->link, Aarg0);
    stkoff = 0;
```

Uses Aarg0 276b, autoffset 196d, stkoff 196c, and thisfntype 105b.

`<argmark() finalizations 197b>≡ (138a)`

```
    autoffset = 0;
    stkoff = 0;
```

Uses autoffset 196d and stkoff 196c.

`<Sym identifier value, code generation fields 197c>≡ (36b) 287b▷`

```
    long offset;
```

`<Node code generation fields 197d>+≡ (28) <34a 199g▷`

```
    long xoffset;
```

10.5 Statements: gen()

`gen` is a recursive switch on the statement's `op`: it handles sequences (`OLIST`), conditionals (`OIF`), loops (`OWHILE`, `ODWHILE`, `OFOR`), jumps (`ORETURN`, `OBREAK`, `OCONTINUE`), `switch`, labels, and `goto`. Control flow uses the “branch and patch” idiom: `gbranch` emits a branch with an unresolved target, and `patch` fills in the destination once the target address is known. For boolean expressions in conditions, `bcomplex` generates short-circuit evaluation code directly using conditional branches rather than materializing a boolean value.

`<function gen 197e>≡ (530b)`

```
    /// codgen -> <>
    void
    gen(Node *n)
    {
        // enum<Node_kind> of a statement
        int o;
        <gen() locals 187b>
```

```
    loop:
        if(n == Z)
            return;

        nearln = n->lineno;
        o = n->op;
        <gen() debug opcode 491e>
        <gen() if not canreach 185g>
        switch(o) {
            <gen() switch node kind cases 184e>
        }
    }
```

Uses Z 29a and nearln 29g.

10.5.1 Blocks, sequences

Statement lists are simply generated left-to-right: `gen` recurses on the left child then the right child. The `canreach` flag tracks whether control flow can reach each statement—if not, the code generator skips it and

optionally warns about dead code.

```
<gen() switch node kind cases 198a>+≡ (197e) <184e 198c>
  case OLIST:
  case OCOMMA:
    gen(n->left);

  rloop:
    n = n->right;
    goto loop;
```

Uses OCOMMA 110g, OLIST 29b, and gen() 197e.

10.5.2 Conditionals

An if/else is compiled as: evaluate the condition via `bcomplex` (which emits a conditional branch), generate the “then” body, emit an unconditional branch past the “else” body, patch the conditional branch to the “else” body, generate the “else” body, and patch the unconditional branch to the end. Without an `else`, the unconditional branch is omitted. The pattern in pseudo-assembly is:

```
<condition code>
ABxx  else_label      (conditional branch from bcomplex)
<then body>
AB    end_label       (skip else)
else_label:
  <else body>
end_label:

<gen() locals 198b>+≡ (197e) <187b 199a>
  Prog *sp;
  Node *l;
  bool err;

<gen() switch node kind cases 198c>+≡ (197e) <198a 200a>
  case OIF:
    l = n->left;
    err = bcomplex(l, n->right);
    <gen() switch node kind cases, OIF case, if bcomplex error 187c>
    else {
      sp = p; // AB instr for 'else' to patch later (created by bcomplex())
      <gen() when OIF, before gen then part, set canreach 199b>

      if(n->right->left != Z) // gen then part
        gen(n->right->left);

      <gen() when OIF, before gen else part, save and set canreach 199c>

      if(n->right->right != Z) { // gen else part
        gbranch(OGOTO);
        patch(sp, pc);
        sp = p;
        gen(n->right->right);
      }
      patch(sp, pc);

      <gen() when OIF, after gen everything, set canreach 199d>
    }
    break;
```

Uses OGOTO 108e, OIF 107a, Z 29a, bcomplex() 199e, and gen() 197e.

<gen() locals 199a>+≡ (197e) *<198b 201e>*
 bool oldreach;

<gen() when OIF, before gen then part, set canreach 199b>≡ (198c)
 canreach = true;

Uses canreach 185d.

<gen() when OIF, before gen else part, save and set canreach 199c>≡ (198c)
 oldreach = canreach;
 canreach = true;

Uses canreach 185d.

<gen() when OIF, after gen everything, set canreach 199d>≡ (198c)
 canreach = canreach || oldreach;
 if(!canreach)
 warnreach = !suppress;

Uses canreach 185d and warnreach 185e.

<function bcomplex 199e>≡ (530b)
 bool//TODO error_1
 bcomplex(Node *n, Node *c)
 {

 complex(n);
<bcomplex() typecheck n 180d>
<bcomplex() generate fake goto if type error 199f>
<bcomplex() check for deadheads 186a>

<bcomplex() possibly convert node to funcall if unsupported 64 bit op 466a>
 boolgen(n, true, Z);

 return false; // no error //TODO: OK_0
 }

Uses Z 29a and complex() 210a.

<bcomplex() generate fake goto if type error 199f>≡ (199e)
 if(n->type == T) {
 gbranch(OGOTO);
 return false;
 }

Uses OGOTO 108e and T 31i.

10.5.3 Labels and goto

Labels and gotos share a common Node (created by `dc1label` during parsing): the label definition records a `pc` value, and a `goto` branches to it. Forward gotos—where the label has not yet been seen—use the “branch and patch” idiom: emit a branch with an unresolved target, then when the label is defined, backpatch the branch’s destination.

<Node code generation fields 199g>+≡ (28) *<197d 199h>*
 long pc;

<Node code generation fields 199h>+≡ (28) *<199g ??>*
 // ref<Prog>, but use 'void*' to be archi independent
 void* label;

Labels

When a label is encountered, its `pc` is recorded so that future `gotos` can branch directly to it. If a `goto` was already emitted before the label was defined, the saved branch instruction is patched to point here. The seemingly redundant `gbranch(OGOTO)` at the end prevents the label's `pc` from coinciding with the address of a register-allocating instruction that follows, which would create a self-referencing branch target.

```
<gen() switch node kind cases 200a>+≡ (197e) <198c 200c>
case OLABEL:
  <gen() when LABEL, set canreach 200b>
  l = n->left;
  if(l) {
    l->pc = pc;
    if(l->label)
      patch(l->label, pc);
  }
  gbranch(OGOTO); /* prevent self reference in reg */
  patch(p, pc);

  goto rloop;
```

Uses OGOTO 108e and OLABEL 108e.

```
<gen() when LABEL, set canreach 200b>≡ (200a)
  canreach = true;
```

Uses canreach 185d.

Gotos

If the target label has already been seen, `goto` patches its branch directly to `n->pc`. Otherwise, the compiler chains forward references: it patches the *previous* unresolved `goto` to jump to *this* one, forming a linked list through `n->label`. When the label finally appears, patching the head of the chain fixes only the last `goto`; the linker's branch-chain optimizer collapses the rest.

```
<gen() switch node kind cases 200c>+≡ (197e) <200a 201f>
case OGOTO:
  <gen() when OGOTO/ORETURN/OBREAK/OCONTINUE, set canreach 200d>
  n = n->left;
  <gen() when OGOTO, sanity check n 201a>
  gbranch(OGOTO);
  if(n->pc) {
    patch(p, n->pc);
  } else {
    if(n->label)
      patch(n->label, pc-1);
    n->label = p;
  }
  return;
```

Uses OGOTO 108e.

```
<gen() when OGOTO/ORETURN/OBREAK/OCONTINUE, set canreach 200d>≡ (206b 204 200c)
  canreach = false;
  warnreach = !suppress;
```

Uses canreach 185d and warnreach 185e.

```

⟨gen() when OGOTO, sanity check n 201a⟩≡ (200c)
    if(n == Z) // possible?
        return;
    if(n->complex == 0) {
        diag(Z, "label undefined: %s", n->sym->name);
        return;
    }
    if(suppress)
        return;

```

Uses Z 29a and diag() 496b.

10.5.4 Loops

Loop code generation uses two global variables, `breakpc` and `continpc`, to record the branch targets for `break` and `continue`. Before compiling the loop body, the current values are saved and new ones are set; after the body, they are restored. This works because C forbids cross-function jumps, so the save/restore stack aligns with lexical nesting. The general pattern for `while(cond) body` is: emit condition, branch-if-false to after, emit body, branch back to condition, patch the false-branch.

```

⟨global breakpc 201b⟩≡ (535b)
    // option<pc>, None = -1
    long breakpc;

```

```

⟨global continpc 201c⟩≡ (535b)
    // option<pc>, None = -1
    long continpc;

```

```

⟨ginit() misc initialisations 201d⟩+≡ (51c) <194a 226h>
    breakpc = -1;
    continpc = -1;

```

while/dowhile

The generated code layout first emits placeholder branches for entry, continue, and break, then patches them as each target becomes known. For `while`, entry jumps straight to the condition test; for `do-while`, entry falls through to the body. The only difference between the two is which `patch(sp, pc)` comes before the test vs. before the body. The author's ASCII art below shows the resulting instruction layout.

```

⟨gen() locals 201e⟩+≡ (197e) <199a 204e>
    Prog *spc, *spb;

```

```

⟨gen() switch node kind cases 201f⟩+≡ (197e) <200c 202>
    case OWHILE:
    case ODWHILE:
        l = n->left;

        gbranch(OGOTO); /* entry */
        sp = p;

        ⟨gen() when OWHILE/ODWHILE, set continpc to pc before goto continue 204f⟩
        gbranch(OGOTO); // goto for continue
        spc = p;

        ⟨gen() when OWHILE/ODWHILE, set breakpc to pc before goto break 204g⟩
        gbranch(OGOTO); // goto for break
        spb = p;

```

```

patch(spc, pc);
if(n->op == OWHILE)
    patch(sp, pc);

bcomplex(l, Z); /* test */
patch(p, breakpc);
⟨gen() when OWHILE/ODWHILE, increment nbreak 205b⟩

if(n->op == ODWHILE)
    patch(sp, pc);

gen(n->right); /* body */

gbranch(OGOTO);
patch(p, continpc);

patch(spb, pc); // exit

⟨gen() when OWHILE/ODWHILE, before exit, restore continpc/breakpc 205e⟩
break;

```

Uses ODWHILE 107c, OGOTO 108e, OWHILE 107c, Z 29a, bcomplex() 199e, and gen() 197e.

for

The **for** loop is more complex than **while** because it has three parts: init, test, and increment. The layout is: emit init, branch to test, emit continue-target (which is the increment code, not the test), emit test, emit body, branch back to continue. This means **continue** in a **for** loop jumps to the increment, matching C semantics.

```

⟨gen() switch node kind cases 202⟩+≡ (197e) <201f 204a>
case OFOR:
    l = n->left;
    ⟨gen() when OFOR, check canreach before gen anything 203a⟩

    gen(l->right->left); /* init */

    gbranch(OGOTO); /* entry */
    sp = p;

    ⟨gen() when OFOR, check canreach after gen init 203b⟩

    ⟨gen() when OFOR, set continpc to pc before goto continue 204h⟩
    gbranch(OGOTO); // goto for continue
    spc = p;

    ⟨gen() when OFOR, set breakpc to pc before goto break 205a⟩
    gbranch(OGOTO); // goto for break
    spb = p;

    patch(spc, pc);

    gen(l->right->right); /* inc */

    patch(sp, pc); // entry

    if(l->left != Z) { /* test */
        bcomplex(l->left, Z);
        patch(p, breakpc);
        ⟨gen() when OFOR, increment nbreak 205c⟩
    }
}

```

```

⟨gen() when OFOR, set canreach before gen body 203c⟩

gen(n->right);  /* body */

if(canreach){
    gbranch(OGOTO);
    patch(p, continpc);
    ⟨gen() when OFOR, increment ncontin when canreach end of body 205d⟩
}
⟨gen() when OFOR, check can reach for inc 203d⟩

patch(spb, pc);

⟨gen() when OFOR, before exit, restore continpc/breakpc 205g⟩
break;

```

Uses OFOR 107c, OGOTO 108e, Z 29a, bcomplex() 199e, canreach 185d, and gen() 197e.

```

⟨gen() when OFOR, check canreach before gen anything 203a⟩≡ (202)
if(!canreach && warnreach && l->right->left) {
    warn(n, "unreachable code FOR");
    warnreach = false;
}

```

Uses canreach 185d, warn() 142, and warnreach 185e.

```

⟨gen() when OFOR, check canreach after gen init 203b⟩≡ (202)
/*
 * if there are no incoming labels in the
 * body and the top's not reachable, warn
 */
if(!canreach && warnreach && deadheads(n)) {
    warn(n, "unreachable code %0", o);
    warnreach = false;
}

```

Uses canreach 185d, deadheads() 187a, warn() 142, and warnreach 185e.

```

⟨gen() when OFOR, set canreach before gen body 203c⟩≡ (202)
canreach = true;

```

Uses canreach 185d.

```

⟨gen() when OFOR, check can reach for inc 203d⟩≡ (202)
if(!ncontin && l->right->right && warnreach) {
    warn(l->right->right, "unreachable FOR inc");
    warnreach = false;
}

```

Uses ncontin 205h, warn() 142, and warnreach 185e.

```

⟨gen() when OFOR, set canreach after gen 203e⟩≡ (205g)
canreach = nbreak!=0;
if(!canreach)
    warnreach = !suppress;

```

Uses canreach 185d and warnreach 185e.

Continue/break

continue and **break** simply emit an unconditional branch to **continpc** or **breakpc**, which were set when the enclosing loop began. The save/restore of these globals around each loop body (via the local **spc/spb** variables) means nested loops automatically get the right targets.

```
<gen() switch node kind cases 204a>+≡ (197e) <202 204c>
  case OCONTINUE:
    <gen() when OCONTINUE, sanity check continpc 204b>
    gbranch(OGOTO);
    patch(p, continpc);
    ncontin++;
    <gen() when OGOTO/ORETURN/OBREAK/OCONTINUE, set canreach 200d>
    break;
```

Uses OCONTINUE 108b, OGOTO 108e, and ncontin 205h.

```
<gen() when OCONTINUE, sanity check continpc 204b>≡ (204a)
  if(continpc < 0) {
    diag(n, "continue not in a loop");
    break;
  }
```

Uses diag() 496b.

```
<gen() switch node kind cases 204c>+≡ (197e) <204a 206b>
  case OBREAK:
    <gen() when OBREAK, sanity check breakpc 204d>
    <gen() when OBREAK, check canreach 205j>
    gbranch(OGOTO);
    patch(p, breakpc);
    nbreak++;
    <gen() when OGOTO/ORETURN/OBREAK/OCONTINUE, set canreach 200d>
    break;
```

Uses OBREAK 108b and OGOTO 108e.

```
<gen() when OBREAK, sanity check breakpc 204d>≡ (204c)
  if(breakpc < 0) {
    diag(n, "break not in a loop");
    break;
  }
```

Uses diag() 496b.

```
<gen() locals 204e>+≡ (197e) <201e 206a>
  long sbc, scc;
  int snbreak, sncontin;
```

```
<gen() when OWHILE/ODWHILE, set continpc to pc before goto continue 204f>≡ (201f)
  scc = continpc;
  continpc = pc;
```

```
<gen() when OWHILE/ODWHILE, set breakpc to pc before goto break 204g>≡ (201f)
  sbc = breakpc;
  snbreak = nbreak;
  breakpc = pc;
  nbreak = 0;
```

```
<gen() when OFOR, set continpc to pc before goto continue 204h>≡ (202)
  scc = continpc;
  continpc = pc;
```

<gen() when OFOR, set breakpc to pc before goto break 205a>≡ (202)

```
sbc = breakpc;
snbreak = nbreak;
sncontin = ncontin;
breakpc = pc;
nbreak = 0;
ncontin = 0;
```

Uses ncontin 205h.

<gen() when OWHILE/ODWHILE, increment nbreak 205b>≡ (201f)

```
if(l->op != OCONST || vconst(l) == 0)
    nbreak++;
```

Uses OCONST 111a and vconst() 176c.

<gen() when OFOR, increment nbreak 205c>≡ (202)

```
if(l->left->op != OCONST || vconst(l->left) == 0)
    nbreak++;
```

Uses OCONST 111a and vconst() 176c.

<gen() when OFOR, increment ncontin when canreach end of body 205d>≡ (202)

```
ncontin++;
```

Uses ncontin 205h.

<gen() when OWHILE/ODWHILE, before exit, restore continpc/breakpc 205e>≡ (201f)

```
// restore
continpc = scc;
breakpc = sbc;
<gen() when OWHILE/ODWHILE, set canreach 205f>
nbreak = snbreak;
```

<gen() when OWHILE/ODWHILE, set canreach 205f>≡ (205e)

```
canreach = nbreak!=0;
if(!canreach)
    warnreach = !suppress;
```

Uses canreach 185d and warnreach 185e.

<gen() when OFOR, before exit, restore continpc/breakpc 205g>≡ (202)

```
// restore
continpc = scc;
breakpc = sbc;
<gen() when OFOR, set canreach after gen 203e>
nbreak = snbreak;
ncontin = sncontin;
```

Uses ncontin 205h.

<global ncontin 205h>≡ (511d)

```
int ncontin;
```

<global nbreak 205i>≡ (535b)

```
long nbreak;
```

<gen() when OBREAK, check canreach 205j>≡ (204c)

```
/*
 * Don't complain about unreachable break statements.
 * There are breaks hidden in yacc's output and some people
 * write return; break; in their switch statements out of habit.
 * However, don't confuse the analysis by inserting an
 * unreachable reference to breakpc either.
 */
if(!canreach)
    break;
```

Uses canreach 185d.

10.5.5 Control flow jumps

`return`, `break`, and `continue` all compile to unconditional branches: `return` jumps to the function's epilogue (which restores registers and returns), `break` jumps to the loop or switch exit label, and `continue` jumps to the loop's re-test label. `goto` is resolved to the target label's `Prog`. All four set `canreach = false` for the code that follows.

```
<gen() locals 206a>+≡ (197e) <204e 279a>
Node nod;
```

```
<gen() switch node kind cases 206b>+≡ (197e) <204c 223a>
case ORETURN:
  <gen() when OGOTO/ORETURN/OBREAK/OCONTINUE, set canreach 200d>

  complex(n);
  <gen() when ORETURN, after complex, if no type 206c>

  l = uncomma(n->left);
  if(l == Z) {
    <gen() case ORETURN with no argument, before RET 341c>
    gbranch(ORETURN);
  }
  else
    <gen() case ORETURN, if complex type 272e>
  else {
    regret(&nod, n);
    cgen(l, &nod);
    regfree(&nod);
    <gen() case ORETURN with argument, before RET 341d>
    gbranch(ORETURN);
  }
  break;
```

Uses `ORETURN` 108b, `Z` 29a, `complex()` 210a, and `uncomma()` 206d.

```
<gen() when ORETURN, after complex, if no type 206c>≡ (206b)
  if(n->type == T)
    break;
```

Uses `T` 31i.

`uncomma()`

```
<function uncomma 206d>≡ (530b)
/// gen(case RETURN) -> <>
Node*
uncomma(Node *n)
{
  while(n != Z && n->op == OCOMMA) {
    cgen(n->left, Z);
    n = n->right;
  }
  return n;
}
```

Uses `OCOMMA` 110g and `Z` 29a.

```

regret()
⟨function regret(arm) 207a⟩≡ (537b)
void
regret(Node *n/*OUT*/, Node *nn/*IN*/)
{
    int r;

    r = REGRET;
    ⟨regret() if float expression, adjust return register 452c⟩
    nodreg(n, nn, r);
    reg[r]++;
}

```

10.6 Register allocation

During code generation, the compiler tracks which registers are in use via the `reg` reference-count array. `regalloc` finds a free register and marks it busy; `regfree` releases it. This is a simple local allocator — the global register promotion pass (`regopt`) runs later as a separate optimization.

10.6.1 `reg`

The `reg` array is a reference count for each ARM register. A value of 0 means the register is free; a positive value means it is in use by that many expressions. Registers `REGTMP`, `REGSB`, `REGSP`, and `REGLINK` are reserved and pre-incremented to 1 so they are never allocated for temporaries. `regalloc` finds the first free register, and `regfree` decrements the count.

After `ginit` the reference-count array pins the ABI-mandated registers so that `regalloc` will simply skip over them:

index	name	initial	role
-----	-----	-----	-----
R0	REGARG	= 0	first arg / return value (alloc.)
R1		= 0	general (allocatable)
R2..R8		= 0	general (REGMIN..REGMAX)
R9		= 0	general
R10	REGEXT	= 0	external base (allocatable)
R11	REGTMP	= 1	reserved: gmove/gins scratch
R12	REGSB	= 1	reserved: static base
R13	REGSP	= 1	reserved: stack pointer
R14	REGLINK	= 1	reserved: link register (LR)
R15	REGPC	= 1	reserved: program counter

The initial values are snapshot into `resvreg` so that `gclean` can, at the end of every function, assert that every count has returned to its baseline — a cheap end-to-end check that no `regalloc/regfree` pair was forgotten in `cgen`. Note that R0 plays two roles: it is the first-argument register (`REGARG`) *and* the return-value register (`REGRET`), which is why `codgen`'s `REGARG` prologue has to pre-allocate it when the function actually uses its first parameter.

```

⟨global reg(arm) 207b⟩≡ (535b)
// map<enum<registr>, int>
char reg[NREG+NFREG];

```


$\langle \text{ginit}() \text{ reg and resvreg initialisation } 208a \rangle \equiv$ (51c) 208c $\triangleright$

```
memset(reg, 0, sizeof(reg));
```

```
/* don't allocate */
reg[REGTMP] = 1;
reg[REGSB] = 1;
reg[REGSP] = 1;
reg[REGLINK] = 1;
reg[REGPC] = 1;
```

$\langle \text{global resvreg}(arm) 208b \rangle \equiv$ (537b)

```
static char resvreg[nelem(reg)];
```

$\langle \text{ginit}() \text{ reg and resvreg initialisation } 208c \rangle + \equiv$ (51c) $\triangleleft 208a \ 432b \triangleright$

```
memmove(resvreg, reg, sizeof(reg));
```

Uses resvreg-193 208b.

$\langle \text{gclean}() \text{ locals } 208d \rangle \equiv$ (49c) 270a $\triangleright$

```
int i;
```

$\langle \text{gclean}() \text{ sanity check reg } 208e \rangle \equiv$ (49c) 453a $\triangleright$

```
for(i=0; i<NREG; i++)
    if(reg[i] && !resvreg[i])
        diag(Z, "reg %d left allocated", i);
```

Uses Z 29a, diag() 496b, and resvreg-193 208b.

10.6.2 OREGISTER

OREGISTER is a synthetic node kind injected during code generation to represent a value that already lives in a physical register. The parser never produces these nodes; they are created by `regalloc` and `nodreg` so that `naddr` can translate them into `Adr` operands.

$\langle \text{after parsing nodes } 208f \rangle \equiv$ (27) 232b $\triangleright$

```
OREGISTER,
```

$\langle \text{Node value fields } 208g \rangle + \equiv$ (28) $\triangleleft 113b$

```
// option<enum<registr>>
int reg; // for OREGISTER
```

10.6.3 nodreg()

`nodreg` stamps a node as an `OREGISTER` for a specific physical register. It copies from a global `regnode` template (pre-initialized with `OREGISTER`, addable 11) and fills in the register number, type, and source location from `nn`. The `nn` parameter exists only to donate its type and line number — the resulting node refers to `reg`, not to `nn`'s value.

$\langle \text{function nodreg}(arm) 208h \rangle \equiv$ (537b)

```
/// regret | ... -> <>
void
nodreg(Node *n/*OUT*/, Node *nn/*IN*/, int reg)
{
    *n = regnode;
    n->reg = reg;
    n->type = nn->type;
    n->lineno = nn->lineno;
}
```

$\langle$ global regnode 209a $\rangle \equiv$ (535b)
Node regnode;

$\langle$ ginit() special nodes initialisation 209b $\rangle + \equiv$ (51c) $\triangleleft$ 195b 242e $\triangleright$
regnode.op = OREGISTER;
regnode.class = CEXREG;
regnode.reg = REGTMP;
regnode.complex = 0;
regnode.addable = 11;
regnode.type = types[TLONG];

Uses CEXREG 432a, OREGISTER 208f, TLONG 30b, and types 32b.

10.6.4 regfree()

regfree decrements the reference count in the global **reg** array for the register held by node **n**. Every **regalloc** must be balanced by a **regfree**; the **gclean** function at the end of each function body asserts that all non-reserved registers have count zero, catching leaks early.

$\langle$ function regfree(*arm*) 209c $\rangle \equiv$ (537b)
void
regfree(Node *n)
{
int i;

 $\langle$ regfree() sanity checks 209d $\rangle$

reg[n->reg]--;
return;

err:
diag(n, "error in regfree: %d", i);
}

Uses diag() 496b.

$\langle$ regfree() sanity checks 209d $\rangle \equiv$ (209c)
i = 0;
if(n->op != OREGISTER && n->op != OINDREG)
goto err;
i = n->reg;
if(i < 0 || i >= sizeof(reg))
goto err;
if(reg[i] <= 0)
goto err;

Uses OINDREG 237c and OREGISTER 208f.

10.7 Typechecking, expliciting, annotating, and rewriting: complex()

complex is the main entry point for preparing an expression for code generation. It runs four passes in sequence: **tcom** type-checks and annotates each node with its type (inserting implicit casts and resolving enum values); **comma** hoists comma operators so side-effects happen in order; **ccom** performs AST-level rewrites (constant

folding, identity elimination, $*\&x \rightarrow x$); and `xcom` classifies each node’s “addressability” and estimates its register cost. Only after `complex` has run can `cgen` generate code.

```

⟨function complex 210a⟩≡ (529)
  /// dodecl (array size) | doenum (evconst) | codgen | bcomplex -> <>
  void
  complex(Node *n)
  {

    if(n == Z)
      return;

    nearln = n->lineno;

    ⟨complex() debug tree pre complex 490i⟩
    // typechecking, removing some sugar, lvalue annotate, bitfield annot, etc
    if(tcom(n))
      return;

    // comma hoisting
    comma(n);
    ⟨complex() debug tree after tcom and comma 491a⟩
    // general rewrite
    ccom(n);
    ⟨complex() debug tree after ccom 491b⟩
    // arithmetic rewrite
    acom(n);
    ⟨complex() debug tree after acom 491c⟩

    // addressability and complexity (and shl/shr optimizations)
    xcom(n);
    ⟨complex() debug tree after xcom 491d⟩
  }

```

Uses `Z` 29a, `acom()` 328e, `ccom()` 292, `comma()` 308b, `nearln` 29g, and `tcom()` 152b.

10.8 Addressability and complexity: `xcom()`

`xcom` is the instruction selector: it walks each expression bottom-up, classifying how “addressable” each sub-expression is. The addressability values encode ARM addressing modes: 20 = immediate constant (`$value`), 10 = memory name (`name`), 11 = register, 12 = indirect register (`*(reg)+offset`). Values below 10 mean the expression needs to be computed into a register first. `xcom` also estimates the “complexity” (number of registers needed), which `cgen` uses to decide which subtree to evaluate first to minimize register pressure.

```

⟨function xcom(arm) 210b⟩≡ (540a)
  /*
  * calculate addressability as follows
  * CONST ==> 20 $value
  * NAME ==> 10 name
  * REGISTER ==> 11 register
  * INDREG ==> 12 *[(reg)+offset]
  *
  * &10 ==> 2 $name
  * ADD(2, 20) ==> 2 $name+offset
  * ADD(3, 20) ==> 3 $(reg)+offset
  * &12 ==> 3 $(reg)+offset
  *
  * *11 ==> 11 ??
  * *2 ==> 10 name

```

```

* *3 ==> 12 *(reg)+offset
* calculate complexity (number of registers) //$
*/
void
xcom(Node *n)
{
    Node *l, *r;
    int t;

    if(n == Z)
        return;

    l = n->left;
    r = n->right;
    n->addable = 0;
    n->complex = 0;

    switch(n->op) {
        <xcom() switch op cases to set addable 211b>
    default:
        if(l != Z)
            xcom(l);
        if(r != Z)
            xcom(r);
        break;
    }
    if(n->addable >= 10)
        return;
    // else
    <xcom() set complex if addable less than 10 213a>
}

```

Uses Z 29a.

10.8.1 Addressable: INDEXED

The INDEXED threshold (9) divides nodes into two worlds: nodes with `addable >= 10` can be used directly as instruction operands (constants, globals, registers), while nodes below that threshold need temporary registers and extra instructions. This single integer drives the key code-generation decision: emit a simple operand or spill through a register.

```

<constant INDEXED(arm) 211a>≡ (531b)
#define INDEXED 9

```

10.8.2 Addressability: addable

`xcom` assigns numeric “addressability” scores that encode how each node maps to ARM addressing modes: 20 = constant (immediate operand), 10 = named variable (can use SB/FP/SP relative), 11 = register, 12 = indirect register. Values 2 and 3 are intermediate: address-of-name and address-of-indirect, which become 10 or 12 when dereferenced. The key insight is that `OIND(OADD(OADDR(x), const))` collapses back to addable 10 — the ARM can encode `x+offset(SB)` in a single instruction.

```

<xcom() switch op cases to set addable 211b>≡ (210b) 212a▷
case OCONST:
    n->addable = 20;
    return;

```

Uses OCONST 111a.

`<xcom() switch op cases to set addable 212a>+≡ (210b) <211b 212b>`
`case ONAME:`
`n->addable = 10;`
`return;`
Uses ONAME 27.

`<xcom() switch op cases to set addable 212b>+≡ (210b) <212a 212c>`
`case OREGISTER:`
`n->addable = 11;`
`return;`
Uses OREGISTER 208f.

`<xcom() switch op cases to set addable 212c>+≡ (210b) <212b 212d>`
`case OINDREG:`
`n->addable = 12;`
`return;`
Uses OINDREG 237c.

`<xcom() switch op cases to set addable 212d>+≡ (210b) <212c 212e>`
`case OADDR:`
`xcom(1);`
`if(l->addable == 10)`
`n->addable = 2;`

`if(l->addable == 12)`
`n->addable = 3;`
`break;`
Uses OADDR 116e.

`<xcom() switch op cases to set addable 212e>+≡ (210b) <212d 212f>`
`case OIND:`
`xcom(1);`
`if(l->addable == 2)`
`n->addable = 10;`

`if(l->addable == 3)`
`n->addable = 12;`
`if(l->addable == 11)`
`n->addable = 12;`
`break;`
Uses OIND 116e.

`<xcom() switch op cases to set addable 212f>+≡ (210b) <212e 311a>`
`case OADD:`
`xcom(1);`
`xcom(r);`
`if(l->addable == 20) {`
`if(r->addable == 2)`
`n->addable = 2;`

`if(r->addable == 3)`
`n->addable = 3;`
`}`
`if(r->addable == 20) {`
`if(l->addable == 2)`
`n->addable = 2;`

`if(l->addable == 3)`

```

        n->addable = 3;
    }
    break;
Uses OADD 114c.

```

10.8.3 Complexity: complex

The complexity number estimates how many registers are needed to evaluate a subtree, following Sethi and Ullman's algorithm: if both children need the same number of registers, the parent needs one more (to hold the left result while computing the right); otherwise, it takes the maximum. Nodes that are already addressable (`addable >= 10`) have complexity 0 because they need no registers at all.

```

⟨xcom() set complex if addable less than 10 213a⟩≡ (210b)
    if(l != Z)
        n->complex = l->complex;
    if(r != Z) {
        if(r->complex == n->complex)
            n->complex = r->complex+1;
        else
            if(r->complex > n->complex)
                n->complex = r->complex;
    }

    if(n->complex == 0)
        n->complex++;

⟨xcom() if 64 bits operation, transform and return 459c⟩
// else
switch(n->op) {
⟨xcom() switch node opkind cases to set complexity 213b⟩
}

```

Uses Z 29a.

10.8.4 FNX

Function calls get a special complexity of 100 (FNX), far above any real register count. This forces `cgen` to spill all sub-expressions containing function calls into temporaries before evaluating them, since a call clobbers every caller-save register. The code generator checks `complex >= FNX` to detect this case and uses `regsallocc` to save the intermediate values on the stack.

```

⟨xcom() switch node opkind cases to set complexity 213b⟩≡ (213a) 313c▷
    case OFUNC:
        n->complex = FNX;
        break;

```

Uses OFUNC 117c.

```

⟨constant FNX(arm) 213c⟩≡ (531b)
#define FNX 100

```

```

⟨constant FNX 213d⟩≡ (524b)
/*
 * this is machine dependent, but it is totally
 * common on all of the 64-bit symulating machines.
 */

#define FNX 100 /* botch -- redefinition */

```

10.9 Memory moves

The lowest-level code generation primitives are `gmove` (emit a `MOV` between two addressable nodes, handling type conversions), `gins` (emit an arbitrary instruction from two node operands), and `naddr` (translate a `Node`'s addressability into an `Adr` operand for a `Prog`). These three functions are the bridge between the AST world and the instruction world: `cgen` decomposes expressions into register-sized pieces, and these functions produce the actual `Prog` instructions.

10.9.1 `gmove()`

`gmove` is a two-dimensional dispatch on source and destination types: it selects the right `MOV` variant (`AMOVW`, `AMOVb`, `AMOVBU`, `AMOVF`, `AMOVD`, etc.) and inserts sign-extension or truncation as needed. For example, moving a `TCHAR` to a `TINT` emits `AMOVb` (move byte with sign extension), while moving a `TUCHAR` emits `AMOVBU` (move byte unsigned). Float-to-integer and integer-to-float conversions route through `AMOVFW`/`AMOVWF` instructions.

```
<function gmove(arm) 214>≡ (537b)
  /// cgenrel | ... -> <>
  void
  gmove(Node *f, Node *t)
  {
    // enum<Type_kind> // Txxx
    int ft, tt;
    // enum<Opcode_kind> // Axxx
    int a;
    <gmove() locals 215b>

    ft = f->type->etype;
    tt = t->type->etype;

    /*
     * a load --
     * put it into a register then
     * worry what to do with it.
     */
    <gmove() if from is an indirect, registerize and return 215c>
    /*
     * a store --
     * put it into a register then
     * store it.
     */
    <gmove() if to is an indirect, registerize and return 216a>

    // at this point f and t should be simpler nodes with
    // registers or constants

    /*
     * type x type cross table
     */
    a = AGOK;
    switch(ft) {
    <gmove() switch from type cases 216d>
    }
    <gmove() sanity check a 215a>
    <gmove() if samaddr return 336a>
    // else

    gins(a, f, t);
  }
```

<gmove() sanity check a 215a>≡ (214)

```
if(a == AGOK)
    diag(Z, "bad opcode in gmove %T -> %T", f->type, t->type);
```

Uses Z 29a and diag() 496b.

A memory load

<gmove() locals 215b>≡ (214) 216c▷

```
Node nod;
```

<gmove() if from is an indirect, registerize and return 215c>≡ (214)

```
if(f->op == ONAME || f->op == OIND || f->op == OINDREG) {
    switch(ft) {
        <gmove() when indirect from, set MOVxx for a depending on type 217a>
        default:
            a = AMOVW;
            break;
    }
    <gmove() when indirect from, if chlp ft and ilp tt 215d>
    else
        regalloc(&nod, f, t);
    gins(a, f, &nod);
    gmove(&nod, t); // recurse!
    regfree(&nod);
    return;
}
```

Uses OIND 116e, OINDREG 237c, and ONAME 27.

<gmove() when indirect from, if chlp ft and ilp tt 215d>≡ (215c)

```
if(typechlp[ft] && typeilp[tt])
    regalloc(&nod, t, t);
```

Uses typechlp 215g and typeilp 215e.

<global typeilp 215e>≡ (518c)

```
char typeilp[NTYPE];
```

Uses NTYPE 30a.

<global typeilpinit 215f>≡ (518c)

```
int typeilpinit[] =
{
    TINT, TUINT, TLONG, TULONG, TIND, -1
};
```

Uses TIND 31c, TINT 30b, TLONG 30b, TUINT 30b, and TULONG 30b.

<global typechlp 215g>≡ (518c)

```
char typechlp[NTYPE];
```

Uses NTYPE 30a.

<global typechlpinit 215h>≡ (518c)

```
int typechlpinit[] =
{
    TCHAR, TUCCHAR, TSHORT, TUSHORT, TINT, TUINT, TLONG, TULONG, TIND, -1,
};
```

Uses TCHAR 30b, TIND 31c, TINT 30b, TLONG 30b, TSHORT 30b, TUCCHAR 30b, TUINT 30b, TULONG 30b, and TUSHORT 30b.

A memory store

```

⟨gmove() if to is an indirect, registerize and return 216a⟩≡ (214)
    if(t->op == ONAME || t->op == OIND || t->op == OINDREG) {
        switch(tt) {
            ⟨gmove() when indirect to, set MOVxx for a depending on type 217b⟩
            default:
                a = AMOVW;
                break;
        }
        ⟨gmove() when indirect to, if ft equal tt 216b⟩
        else
            regalloc(&nod, t, Z);
        gmove(f, &nod); // recurse!
        gins(a, &nod, t);
        regfree(&nod);
        return;
    }

```

Uses OIND 116e, OINDREG 237c, ONAME 27, and Z 29a.

```

⟨gmove() when indirect to, if ft equal tt 216b⟩≡ (216a)
    if(ft == tt)
        regalloc(&nod, t, f);

```

A set

```

⟨gmove() locals 216c⟩+≡ (214) ◁215b
    Node nod1;
    Prog *p1;

⟨gmove() switch from type cases 216d⟩≡ (214) 217c▷
    case TUINT:
    case TULONG:
        ⟨gmove() switch from type cases, TUINT/TULONG case, if float target 454a⟩
        // fall through
    case TINT:
    case TLONG:
    case TIND:
        switch(tt) {
            ⟨gmove() switch from type cases, TINT/TLONG case, if float target 454b⟩
            case TCHAR:
            case TUCCHAR:
            case TSHORT:
            case TUSHORT:
            case TINT:
            case TUINT:
            case TLONG:
            case TULONG:

            case TIND:
                a = AMOVW;
                break;
        }
        break;

```

Uses TCHAR 30b, TIND 31c, TINT 30b, TLONG 30b, TSHORT 30b, TUCCHAR 30b, TUINT 30b, TULONG 30b, and TUSHORT 30b.

10.9.2 Conversions

When `gmove` loads from or stores to memory, it must pick the right MOV variant (`MOVB`, `MOVBU`, `MOVH`, `MOVHU`, `MOVF`, `MOVD`) based on the source or destination type. The signed variants (`MOVB`, `MOVH`) sign-extend on load; the unsigned variants (`MOVBU`, `MOVHU`) zero-extend. This is where C's implicit integer promotions become real machine instructions.

`<gmove() when indirect from, set MOVxx for a depending on type 217a>≡ (215c)`

```
case TCHAR:
    a = AMOVB;
    break;
case TUCCHAR:
    a = AMOVBU;
    break;
case TSHORT:
    a = AMOVH;
    break;
case TUSHORT:
    a = AMOVHU;
    break;
case TFLOAT:
    a = AMOVF;
    break;
case TDOUBLE:
    a = AMOVD;
    break;
```

Uses `TCHAR 30b`, `TDOUBLE 31a`, `TFLOAT 31a`, `TSHORT 30b`, `TUCCHAR 30b`, and `TUSHORT 30b`.

`<gmove() when indirect to, set MOVxx for a depending on type 217b>≡ (216a)`

```
case TCHAR:
    a = AMOVB;
    break;
case TUCCHAR:
    a = AMOVBU;
    break;
case TSHORT:
    a = AMOVH;
    break;
case TUSHORT:
    a = AMOVHU;
    break;
case TFLOAT:
    a = AMOVF;
    break;
case TVLONG: // addon?
case TDOUBLE:
    a = AMOVD;
    break;
```

Uses `TCHAR 30b`, `TDOUBLE 31a`, `TFLOAT 31a`, `TSHORT 30b`, `TUCCHAR 30b`, `TUSHORT 30b`, and `TVLONG 30b`.

`<gmove() switch from type cases 217c>+≡ (214) <216d 218a>`

```
case TSHORT:
    switch(tt) {
        <gmove() switch from type cases, TSHORT case, if float target 455a>
        case TUINT:
        case TINT:
        case TULONG:
        case TLONG:
        case TIND:
            a = AMOVH;
```

```

        break;
    case TCHAR:
    case TUCCHAR:
    case TSHORT:
    case TUSHORT:
        a = AMOVW;
        break;
    }
    break;

```

Uses TCHAR 30b, TIND 31c, TINT 30b, TLONG 30b, TSHORT 30b, TUCCHAR 30b, TUINT 30b, TULONG 30b, and TUSHORT 30b.

<gmove() switch from type cases 218a>+≡ (214) <217c 218b>

```

    case TUSHORT:
        switch(tt) {
            <gmove() switch from type cases, TUSHORT case, if float target 455b>
        case TINT:
        case TUINT:
        case TLONG:
        case TULONG:
        case TIND:
            a = AMOVHU;
            break;
        case TCHAR:
        case TUCCHAR:
        case TSHORT:
        case TUSHORT:
            a = AMOVW;
            break;
        }
        break;

```

Uses TCHAR 30b, TIND 31c, TINT 30b, TLONG 30b, TSHORT 30b, TUCCHAR 30b, TUINT 30b, TULONG 30b, and TUSHORT 30b.

<gmove() switch from type cases 218b>+≡ (214) <218a 218c>

```

    case TCHAR:
        switch(tt) {
            <gmove() switch from type cases, TCHAR case, if float target 455c>
        case TINT:
        case TUINT:
        case TLONG:
        case TULONG:
        case TIND:
        case TSHORT:
        case TUSHORT:
            a = AMOVW;
            break;
        case TUCCHAR:
        case TUCCHAR:
            a = AMOVW;
            break;
        }
        break;

```

Uses TCHAR 30b, TIND 31c, TINT 30b, TLONG 30b, TSHORT 30b, TUCCHAR 30b, TUINT 30b, TULONG 30b, and TUSHORT 30b.

<gmove() switch from type cases 218c>+≡ (214) <218b 453c>

```

    case TUCCHAR:
        switch(tt) {
            <gmove() switch from type cases, TUCCHAR case, if float target 455d>
        case TINT:
        case TUINT:

```

```

case TLONG:
case TULONG:
case TIND:
case TSHORT:
case TUSHORT:
    a = AMOVBU;
    break;
case TCHAR:
case TUCCHAR:
    a = AMOVW;
    break;
}
break;

```

Uses TCHAR 30b, TIND 31c, TINT 30b, TLONG 30b, TSHORT 30b, TUCCHAR 30b, TUINT 30b, TULONG 30b, and TUSHORT 30b.

10.9.3 gins()

gins (“generate instruction”) is the most general instruction emitter: it takes an opcode and two node operands, calls **naddr** on each to fill the “from” and “to” **Adr** fields, and appends the resulting **Prog** to the instruction list. It handles the conversion from the AST world (nodes with types and addressability) to the assembly world (operands with register numbers and addressing modes).

```

⟨function gins(arm) 219a⟩≡ (537b)
  /// gmove | ... -> <>
  void
  gins(int a, Node *f, Node *t)
  {

    nextpc();
    p->as = a;

    if(f != Z)
        naddr(f, &p->from);
    if(t != Z)
        naddr(t, &p->to);

    ⟨gins() debug 491h⟩
  }

```

Uses Z 29a.

10.9.4 naddr()

naddr translates a **Node**’s addressability annotation into a concrete **Adr** operand. An **ONAME** with class **CGLOBL** becomes **name(SB)**; a **CPARAM** becomes **name+offset(FP)**; a **CAUTO** becomes **name-offset(SP)**; an **OREGISTER** becomes a bare register; an **ONCONST** becomes an immediate **\$value**. This is where the abstract AST meets the concrete ARM addressing modes from the **ASSEMBLER** book [Pad15a].

```

⟨function naddr(arm) 219b⟩≡ (537b)
  void
  naddr(Node *n/*IN*/, Adr *a/*OUT*/)
  {

    ⟨naddr() locals 221b⟩

    a->type = D_NONE;
    if(n == Z)
        return;

```

```

switch(n->op) {
⟨naddr() switch node kind cases 220a⟩
default:
bad:
    diag(n, "bad in naddr: %0", n->op);
    break;

}
}

```

Uses Z 29a and diag() 496b.

```

⟨naddr() switch node kind cases 220a⟩≡ (219b) 220b▷
case OCONST:
    a->sym = S;
    a->reg = R_NONE;
    ⟨naddr() if float type 452a⟩
    else {
        a->type = D_CONST;
        a->offset = n->vconst;
    }
    break;

```

Uses OCONST 111a and S 35d.

```

⟨naddr() switch node kind cases 220b⟩+≡ (219b) ◁220a 220c▷
case OREGISTER:
    a->type = D_REG;
    a->sym = S;
    a->reg = n->reg;
    ⟨naddr() if float register, adjust a 452b⟩
    break;

```

Uses OREGISTER 208f and S 35d.

```

⟨naddr() switch node kind cases 220c⟩+≡ (219b) ◁220b 220d▷
case ONAME:
    a->type = D_OREG;
    a->offset = n->xoffset;
    a->sym = n->sym;
    ⟨naddr() when ONAME, save etype 221d⟩

    a->symkind = N_INTERN;
    switch(n->class) {
    ⟨naddr() when ONAME, switch class cases 225b⟩
    default:
        goto bad;
    }
    break;

```

Uses ONAME 27.

```

⟨naddr() switch node kind cases 220d⟩+≡ (219b) ◁220c 221a▷
case OIND:
    naddr(n->left, a);
    if(a->type == D_REG || a->type == D_CONST) {
        a->type = D_OREG;
        break;
    }
    goto bad;

```

Uses OIND 116e.

`<naddr() switch node kind cases 221a>+≡ (219b) <220d 221c>`

```
case OADDR:
    naddr(n->left, a);
    if(a->type == D_OREG) {
        a->type = D_CONST;
        break;
    }
    // else
    goto bad;
```

Uses OADDR 116e.

`<naddr() locals 221b>≡ (219b)`
`long v;`

`<naddr() switch node kind cases 221c>+≡ (219b) <221a 237d>`

```
case OADD:
    if(n->left->op == OCONST) {
        naddr(n->left, a);
        v = a->offset;
        naddr(n->right, a);
    } else {
        naddr(n->right, a);
        v = a->offset;
        naddr(n->left, a);
    }
    a->offset += v;
    break;
```

Uses OADD 114c and OCONST 111a.

`<naddr() when ONAME, save etype 221d>≡ (220c)`
`a->etype = n->etype;`

`<Adr type fields(arm) 221e>≡ (43d)`
`// enum<Type_kind>`
`char etype;`

10.9.5 regalloc()

`regalloc` finds a free general-purpose register for a temporary value. If the optional destination `o` is already a register, it reuses that register to avoid an extra `MOV` (ARM's three-operand instructions can write the result into a source register). The search starts from a rotating `lasti` index to spread pressure across registers, skipping reserved ones (`resvreg`). The `reg` array is a simple reference-counting allocator.

`<function regalloc(arm) 221f>≡ (537b)`

```
/// gmove | -> <>
void
regalloc(Node *n/*OUT*/, Node *tn/*IN*/, Node *o/*option<IN>*/)
{
    int i, j;
    // opti to remember last one allocated
    static int lasti;

    switch(tn->type->etype) {
    case TCHAR:
    case TUCHAR:
    case TSHORT:
    case TUSHORT:
    case TINT:
```

```

case TUINT:
case TLONG:
case TULONG:

case TIND:
    <regalloc() when integer type tn and if OREGISTER o 222d>

    j = lasti + REGRET+1;
    for(i=REGRET+1; i<NREG; i++) {
        if(j >= NREG)
            j = REGRET+1;
        if(reg[j] == 0 && resvreg[j] == 0) {
            i = j;
            goto out;
        }
        j++;
    }
    <regalloc() if reach here, out of fixed registers 222a>

    <regalloc() switch tn type, float or vlong case 456>
}
<regalloc() if reach here, unknown type 222b>
<regalloc() error management 222c>

```

```

out:
    reg[i]++;
    nodreg(n, tn, i);

    lasti++;
    if(lasti >= 5)
        lasti = 0;

}

```

Uses TCHAR 30b, TIND 31c, TINT 30b, TLONG 30b, TSHORT 30b, TUCCHAR 30b, TUINT 30b, TULONG 30b, TUSHORT 30b, and resvreg-193 208b.

```

<regalloc() if reach here, out of fixed registers 222a>≡ (221f)
    diag(tn, "out of fixed registers");
    goto err;

```

Uses diag() 496b.

```

<regalloc() if reach here, unknown type 222b>≡ (221f)
    diag(tn, "unknown type in regalloc: %T", tn->type);
    // fallthrough

```

Uses diag() 496b.

```

<regalloc() error management 222c>≡ (221f)
err:
    nodreg(n, tn, 0);
    return;

```

```

<regalloc() when integer type tn and if OREGISTER o 222d>≡ (221f)
    if(o != Z && o->op == OREGISTER) {
        i = o->reg;
        if(i >= 0 && i < NREG)
            goto out;
    }

```

Uses OREGISTER 208f and Z 29a.

10.10 Expressions: cgen()

cgen compiles an expression into a destination node **nn** (typically a register allocated by **regalloc**). It works by cases on the expression's **op**, using the addressability and complexity annotations from **xcom** to decide the evaluation order. The key invariant is: evaluate the more complex subtree first (into a register), then combine it with the simpler subtree using a single instruction. For binary operations, **gopcode** generates the appropriate ARM arithmetic instruction. Assignments use **gmove** to transfer a value to the destination's address.

```
<gen() switch node kind cases 223a>+≡ (197e) <206b 279b>
default:
    complex(n);
    cgen(n, Z);
    break;
```

Uses **Z 29a** and **complex() 210a**.

```
<function cgen(arm) 223b>≡ (542)
/// codgen -> gen -> <>
void
cgen(Node *n, Node *nn)
{
    cgenrel(n, nn, false);
}
```

Uses **cgenrel() 223c**.

```
<function cgenrel(arm) 223c>≡ (542)
/// codgen -> gen -> cgen -> <>
void
cgenrel(Node *n, Node *nn, bool inrel)
{
    // enum<node_kind>
    int o;
    Node *l, *r;
    // enum<Type_kind>
    int t;
    long curs; // to save cursafe
    <cgenrel() locals 224b>

    <cgenrel() debug 491f>
    if(n == Z || n->type == T)
        return;

    <cgenrel() if complex type node 271e>
    else {

        l = n->left;
        r = n->right;
        o = n->op;

        <cgenrel() if node addressable 224c>
        // else

        curs = cursafe; // save
        <cgenrel() if all complex fields more than FNX 261a>

        switch(o) {
        <cgenrel() switch node kind cases 224a>
        default:
            diag(n, "unknown op in cgen: %0", o);
            break;
```



```

    }
    cursafe = curs; // restore
}
return;
}

```

Uses T 31i, Z 29a, and diag() 496b.

```

⟨cgenrel() switch node kind cases 224a⟩≡ (223c) 228b▷
case OCOMMA:
    cgen(l, Z);
    cgen(r, nn);
    break;

```

Uses OCOMMA 110g and Z 29a.

```

⟨cgenrel() locals 224b⟩≡ (223c)
Prog *p1;
Node nod, nod1, nod2, nod3, nod4;
long v;

```

10.10.1 Numeric constants

Constants and other fully addressable expressions (addable $\geq$ INDEXED) are handled by a single `gmove`: the value is loaded directly into the target register. This covers constants, global names, and register variables.

```

⟨cgenrel() if node addressable 224c⟩≡ (223c)
if(n->addable >= INDEXED) {
    ⟨cgenrel() warn if target node is nil 189c⟩
    gmove(n, nn);
    return;
}

```

10.10.2 Entity uses

Parameters

Parameters live at *positive* offsets from the frame pointer FP (which is a fiction of the assembler that the linker turns into an SP-relative address after the prologue), and locals live at *negative* offsets, so both grow outward from the same anchor. For `int f(int a, int b) { int x; int y; ... }` the resulting frame looks like this (high addresses at the top, as memory naturally runs):

high address			
+-----+			
	b	8(FP)	CPARAM, autoffset grows up
+-----+			
	a	4(FP)	CPARAM (or REGARG in R0)
+-----+			
	return addr	0(FP)	saved LR, placed by TEXT
+-----+			
		<----	conceptual FP (== entry SP)
	x	-4(SP)	CAUTO, autoffset grows down
+-----+			
			stkoff tracks max
	y	-8(SP)	CAUTO
+-----+			
	arg build area		for nested calls, sized by
	(maxargsafe)		maxargsafe, added by codgen

```
+-----+ <---- runtime SP after prologue
low address
```

The TEXT instruction's size operand initially holds just `stkoff` (locals); `codgen` adds `maxargsafe` at the very end once every nested call in the body has been seen. `adec1` assigns each parameter a positive offset and each local a negative one via the two-phase `align` calls (`Aarg1/Aarg2` for params, `Aaut3` for autos). The first parameter is tracked specially for the REGARG optimization.

```
<adec1() switch class cases 225a>≡ (140c) 225c▷
case CPARAM:
  <adec1() if first parameter 274i>
  autoffset = align(autoffset, t, Aarg1);
  if(s)
    s->offset = autoffset;
  autoffset = align(autoffset, t, Aarg2);
  break;
```

Uses `Aarg1 276b`, `Aarg2 276b`, `CPARAM 33b`, and `autoffset 196d`.

```
<naddr() when ONAME, switch class cases 225b>≡ (220c) 225d▷
case CPARAM:
  a->symkind = N_PARAM;
  break;
```

Uses `CPARAM 33b`.

Locals

```
<adec1() switch class cases 225c>+≡ (140c) <225a
case CAUTO:
  autoffset = align(autoffset, t, Aaut3);
  stkoff = maxround(stkoff, autoffset);
  s->offset = -autoffset;
  break;
```

Uses `Aaut3 276b`, `CAUTO 33b`, `autoffset 196d`, and `stkoff 196c`.

```
<naddr() when ONAME, switch class cases 225d>+≡ (220c) <225b 225e▷
case CAUTO:
  a->symkind = N_LOCAL;
  break;
```

Uses `CAUTO 33b`.

Globals

```
<naddr() when ONAME, switch class cases 225e>+≡ (220c) <225d 225f▷
case CEXTERN: case CGLOBL:
  a->symkind = N_EXTERN;
  break;
```

Uses `CEXTERN 33b` and `CGLOBL 33b`.

```
<naddr() when ONAME, switch class cases 225f>+≡ (220c) <225e
case CSTATIC:
  a->symkind = N_INTERN;
  break;
```

Uses `CSTATIC 33b`.

Enumeration constants

10.10.3 String constants

ASCII strings

String constants are compiled in two steps. First, the type checker transforms `OSTRING` into `ONAME` referencing a unique static symbol (e.g., `.string<>`). Then, the code generator emits `ADATA` pseudo-instructions to define that symbol's content in the data segment. Duplicate strings are not shared—each literal gets its own static.

```
<tcocomo() transform OSTRING in ONAME 226a>≡ (155c)
    n->op = ONAME;
    n->xoffset = outstring(n->cstring, n->type->width);
    n->addable = true;
```

Uses `ONAME 27`.

```
<string rule, set sym and class for OSTRING 226b>≡ (113d 112f)
    $$->sym = symstring;
    $$->class = CSTATIC;
```

```
<global symstring 226c>≡ (511d)
    Sym* symstring;
```

```
<cinit() locals 226d>+≡ (50d) <60a
    Type *t;
```

```
<cinit() symstring initialization 226e>≡ (50d)
    symstring = slookup(".string");
    symstring->class = CSTATIC;
    t = typ(TARRAY, types[TCHAR]);
    t->width = 0;
    symstring->type = t;
```

Uses `CSTATIC 33b`, `TARRAY 31c`, `TCHAR 30b`, `slookup() 35f`, `symstring 226c`, `typ() 32a`, and `types 32b`.

```
<global nstring 226f>≡ (535b)
    long nstring;
```

```
<global mnstring 226g>≡ (535b)
    int mnstring;
```

```
<ginit() misc initialisations 226h>+≡ (51c) <201d 243a>
    nstring = 0;
    mnstring = 0;
```

```
<gclean() adjust symstring width 226i>≡ (49c)
    while(mnstring)
        outstring("", 1L);
    symstring->type->width = nstring;
```

Uses `symstring 226c`.

```
<global string 226j>≡ (535b)
    char string[NSNAME];
```

```

⟨function outstring(arm) 227a⟩≡ (540b)
    long
    outstring(char *s, long n)
    {
        long r;

        ⟨outstring() return if suppress 227b⟩
        r = nstring;
        while(n) {
            string[mnstring] = *s++;
            mnstring++;
            nstring++;
            if(mnstring >= NSNAME) {
                gpseudo(ADATA, symstring, nodconst(OL));
                p->from.offset += nstring - NSNAME;
                p->reg = NSNAME;
                p->to.type = D_SCONST;
                memmove(p->to.sval, string, NSNAME);
                mnstring = 0;
            }
            n--;
        }
        return r;
    }

```

Uses symstring 226c.

```

⟨outstring() return if suppress 227b⟩≡ (227a)
    if(suppress)
        return nstring;

```

```

⟨tcomo() in OSTRING case, if not an array or chars 227c⟩≡ (155c)
    if(n->type->link != types[TCHAR]) {
        o = outstring(0, 0);
        while(o & 3) {
            outstring("", 1);
            o = outstring(0, 0);
        }
    }

```

Uses TCHAR 30b and types 32b.

Unicode strings

```

⟨tcomo() other locals 227d⟩≡ (153b)
    static TRune zer;

```

```

⟨tcomo() switch node kind cases 227e⟩+≡ (153b) <179a 430b>
    case OLSTRING:
        if(n->type->link != types[TRUNE]) {
            o = outstring(0, 0);
            while(o & 3) {
                outlstring(&zer, sizeof(TRune));
                o = outlstring(0, 0);
            }
        }
        n->op = ONAME;
        n->xoffset = outlstring(n->rstring, n->type->width);
        n->addable = 1;
        break;

```

Uses OLSTRING 113a, ONAME 27, TRUNE 113c, outlstring() 228a, and types 32b.

```

⟨function outlstring 228a⟩≡ (530a)
long
outlstring(TRune *s, long n)
{
    char buf[sizeof(TRune)];
    uint c;
    int i;
    long r;

    if(suppress)
        return nstring;
    while(nstring & (sizeof(TRune)-1))
        outstring("", 1);
    r = nstring;
    while(n > 0) {
        c = *s++;
        if(align(0, types[TCHAR], Aarg1)) {
            for(i = 0; i < sizeof(TRune); i++)
                buf[i] = c>>(8*(sizeof(TRune) - i - 1));
        } else {
            for(i = 0; i < sizeof(TRune); i++)
                buf[i] = c>>(8*i);
        }
        outstring(buf, sizeof(TRune));
        n -= sizeof(TRune);
    }
    return r;
}

```

Uses Aarg1 276b, TCHAR 30b, and types 32b.

10.10.4 Arithmetic expressions

Arithmetic code generation follows a uniform pattern: allocate registers for both operands, generate their values, emit gopcode (which maps AST operators like OADD to ARM opcodes like ADD), then move the result to the destination nn. The subtlety lies in the evaluation order: the more complex child is evaluated first so that fewer registers are live simultaneously (Sethi-Ullman ordering).

```

⟨cgenrel() switch node kind cases 228b⟩+≡ (223c) <224a 229a>
    case OSUB:
        ⟨cgenrel() when OSUB, if l is OCONST 313a⟩
        // else, Fallthrough
    case OADD:
    case OAND:
    case OOR:
    case OXOR:
    case OASHL:
    case OASHR:
    case OLSHR:
        ⟨cgenrel() when OADD/OAND/etc, if r is OCONST 313b⟩
        // Fallthrough
    case OMUL:
    case OLMUL:
    case OLDIV:
    case OLMOD:
    muldiv:
        ⟨cgenrel() nullwarn check if nn is null 189a⟩
        ⟨cgenrel() when OMUL/OLMUL, mulcon opportunity 314a⟩
        // else, the general case!

```

```
<cgenrel() when OADD/OAND/..., if l more complex than r 229b>
```

```
else {
    regalloc(&nod, r, nn);
    cgen(r, &nod);
    regalloc(&nod1, l, Z);
    cgen(l, &nod1);
    gopcode(o, &nod, &nod1, &nod);
}
gopcode(OAS, &nod, Z, nn);
regfree(&nod);
regfree(&nod1);
break;
```

Uses OADD 114c, OAND 115a, OAS 116a, OASHL 115a, OASHR 115a, OLDIV 232d, OLMOD 232f, OLMUL 232d, OLSHR 232b, OMUL 114c, OOR 115a, OSUB 114c, OXOR 115a, and Z 29a.

```
<cgenrel() switch node kind cases 229a>+≡ (223c) <228b 234d>
```

```
case ODIV:
case OMOD:
    <cgenrel() when ODIV/OMOD, convert to SHR if possible 312a>
    // else
    goto muldiv;
```

Uses ODIV 114c and OMOD 114c.

Complexity and register allocation

When the left child is more complex (needs more registers), it is evaluated first into `nod` — which is allocated targeting `nn` so the final `gopcode(OAS, \&nod, Z, nn)` can often become a no-op. The result is stored in `nod` regardless of which branch runs, but the operand order to `gopcode` flips so that the ARM instruction writes into whichever register holds the first-evaluated child, saving a `MOV`.

```
<cgenrel() when OADD/OAND/..., if l more complex than r 229b>≡ (228b)
```

```
if(l->complex >= r->complex) {
    regalloc(&nod, l, nn);
    cgen(l, &nod);
    regalloc(&nod1, r, Z);
    cgen(r, &nod1);
    gopcode(o, &nod1, Z, &nod);
}
```

Uses Z 29a.

gopcode()

`gopcode` translates a node-level operation (OADD, OSUB, etc.) into the corresponding ARM opcode (AADD, ASUB, etc.) and emits it via `gins`. ARM's three-operand format (ADD R1, R2, R3) means that `gopcode` takes three node arguments: two sources and a destination. The function also handles operator-specific special cases like using RSB (reverse subtract) when the operand order allows it.

```
<function gopcode(arm) 229c>≡ (537b)
```

```
/// cgenrel -> <>
void
gopcode(int o, Node *f1, Node *f2, Node *t)
{
    // enum<opcode_kind> AXXX
    int a;
    // enum<type_kind> TXXX
    int et;
    Adr ta;
```

```

⟨gopcode() locals 252e⟩

et = (f1 != Z && f1->type != T)? f1->type->etype : TLONG;
⟨gopcode() initialisation 252f⟩

a = AGOK;
switch(o) {
⟨gopcode() switch opcode cases 230b⟩
}
⟨gopcode() sanity check a 230a⟩

nextpc();
p->as = a;

if(f1 != Z)
    naddr(f1, &p->from);
if(f2 != Z) {
    naddr(f2, &ta);
    p->reg = ta.reg;
}
if(t != Z)
    naddr(t, &p->to);

⟨gopcode() debug 492a⟩
}

```

Uses T 31i, TLONG 30b, and Z 29a.

```

⟨gopcode() sanity check a 230a⟩≡ (229c)
    if(a == AGOK)
        diag(Z, "bad in gopcode %0", o);

```

Uses Z 29a and diag() 496b.

```

⟨gopcode() switch opcode cases 230b⟩≡ (229c) 230c▷
    case OADD:
    case OASADD:
        a = AADD;
        ⟨gopcode() when OADD, adjust a if float type 451a⟩
        break;

```

Uses OADD 114c and OASADD 116c.

```

⟨gopcode() switch opcode cases 230c⟩+≡ (229c) <230b 230d▷
    case OSUB:
    case OASSUB:
        a = ASUB;
        ⟨gopcode() when OSUB, ARSB opportunity 336c⟩
        ⟨gopcode() when OSUB, adjust a if float type 451b⟩
        break;

```

Uses OASSUB 116c and OSUB 114c.

```

⟨gopcode() switch opcode cases 230d⟩+≡ (229c) <230c 231a▷
    case OOR:
    case OASOR:
        a = AORR;
        break;

```

Uses OASOR 116c and OOR 115a.

`<gopcode() switch opcode cases 231a>+≡` (229c) `<230d 231b>`

```
case OAND:
case OASAND:
    a = AAND;
    break;
```

Uses OAND 115a and OASAND 116c.

`<gopcode() switch opcode cases 231b>+≡` (229c) `<231a 231c>`

```
case OXOR:
case OASXOR:
    a = AEOR;
    break;
```

Uses OASXOR 116c and OXOR 115a.

`<gopcode() switch opcode cases 231c>+≡` (229c) `<231b 231d>`

```
case OASHL:
case OASASHL:
    a = ASLL;
    break;
```

Uses OASASHL 116c and OASHL 115a.

`<gopcode() switch opcode cases 231d>+≡` (229c) `<231c 231e>`

```
case OASHR:
case OASASHR:
    a = ASRA;
    break;
```

Uses OASASHR 116c and OASHR 115a.

`<gopcode() switch opcode cases 231e>+≡` (229c) `<231d 231f>`

```
case OMUL:
case OASMUL:
    a = AMUL;
    <gopcode() when OMUL, adjust a if float type 451c>
    break;
```

Uses OASMUL 116c and OMUL 114c.

`<gopcode() switch opcode cases 231f>+≡` (229c) `<231e 231g>`

```
case ODIV:
case OASDIV:
    a = ADIV;
    <gopcode() when ODIV, adjust a if float type 451d>
    break;
```

Uses OASDIV 116c and ODIV 114c.

`<gopcode() switch opcode cases 231g>+≡` (229c) `<231f 233f>`

```
case OMOD:
case OASMOD:
    a = AMOD;
    break;
```

Uses OASMOD 116c and OMOD 114c.

Unsigned operations

ARM uses the same ADD/SUB for signed and unsigned integers (two's complement makes them identical), but multiplication and shifts differ: UMULL vs. SMULL, LSR (logical) vs. ASR (arithmetic). The `typeu` array marks which type kinds are unsigned, so `gopcode` can select the correct variant. Pointers (TIND) count as unsigned since addresses are never negative.

`<global typeu 231h>≡` (518c)

```
char typeu[NTYPE];
```

Uses NTYPE 30a.

<global typeuinit 232a>≡ (518c)
 int typeuinit[] =
 {
 TCHAR, TUSHORT, TUINT, TULONG, TUVLONG, TIND, -1,
 };
 Uses TIND 31c, TCHAR 30b, TUINT 30b, TULONG 30b, TUSHORT 30b, and TUVLONG 30b.

<after parsing nodes 232b>+≡ (27) <208f 232d>
 OLSHR,

<tcomo() in OASHL/OASHR case, adjust opcode if unsigned type 232c>≡ (159d)
 if(typeu[n->type->etype])
 if(n->op == OASHR)
 n->op = OLSHR;

Uses OASHR 115a, OLSHR 232b, and typeu 231h.

<after parsing nodes 232d>+≡ (27) <232b 232f>
 OLMUL,
 OLDIV,

<tcomo() in OMUL/ODIV case, adjust opcode if unsigned type 232e>≡ (157a)
 if(typeu[n->type->etype]) {
 if(n->op == OMUL)
 n->op = OLMUL;
 if(n->op == ODIV)
 n->op = OLDIV;
 }

Uses ODIV 114c, OLDIV 232d, OLMUL 232d, OMUL 114c, and typeu 231h.

<after parsing nodes 232f>+≡ (27) <232d 233b>
 OLMOD,

<tcomo() in OMOD case, adjust opcode if unsigned type 232g>≡ (157d)
 if(typeu[n->type->etype])
 n->op = OLMOD;

Uses OLMOD 232f and typeu 231h.

<tcomo() when OLT/OGE/OGT/OLE, adjust opcode if unsigned type 232h>≡ (164d)
 if(typeu[n->type->etype])
 n->op = logrel[reindex(n->op)];

Uses logrel 232i, reindex() 233a, and typeu 231h.

<global logrel 232i>≡ (518c)
 char logrel[12] =
 {
 OEQ, ONE, OLS, OLS, OLO, OLO, OHS, OHS, OHI, OHI,
 };
 Uses OEQ 115g, OHI 300d, OHS 300d, OLO 300d, OLS 300d, and ONE 115g.

<function relindex 233a>≡ (518c)

```
int
relindex(int o)
{
    switch(o) {
    case OEQ: return 0;
    case ONE: return 1;
    case OLE: return 2;
    case OLS: return 3;
    case OLT: return 4;
    case OLO: return 5;
    case OGE: return 6;
    case OHS: return 7;
    case OGT: return 8;
    case OHI: return 9;
    default:
        diag(Z, "bad in relindex: %0", o);
    }
    return -1; // unreachable
}
```

Uses OEQ 115g, OGE 115g, OGT 115g, OHI 300d, OHS 300d, OLE 115g, OLO 300d, OLS 300d, OLT 115g, ONE 115g, Z 29a, and diag() 496b.

<after parsing nodes 233b>+≡ (27) <232f 237c>

```
OASLSHR,
OASLMUL,
OASLDIV,
OASLMOD,
```

<tcomo() when OASLSHR, adjust opcode if unsigned type 233c>≡ (169a)

```
if(typeu[n->type->etype]) {
    if(n->op == OASASHR)
        n->op = OASLSHR;
}
```

Uses OASASHR 116c, OASLSHR 233b, and typeu 231h.

<tcomo() when OASMUL/OASDIV, adjust opcode if unsigned type 233d>≡ (168f)

```
if(typeu[n->type->etype]) {
    if(n->op == OASDIV)
        n->op = OASLDIV;
    if(n->op == OASMUL)
        n->op = OASLMUL;
}
```

Uses OASDIV 116c, OASLDIV 233b, OASLMUL 233b, OASMUL 116c, and typeu 231h.

<tcomo() when OASMOD, adjust opcode if unsigned type 233e>≡ (169b)

```
if(typeu[n->type->etype]) {
    if(n->op == OASMOD)
        n->op = OASLMOD;
}
```

Uses OASLMOD 233b, OASMOD 116c, and typeu 231h.

<gopcode() switch opcode cases 233f>+≡ (229c) <231g 234a>

```
case OLSHR:
case OASLSHR:
    a = ASRL;
    break;
```

Uses OASLSHR 233b and OLSHR 232b.

```

<gopcode() switch opcode cases 234a>+≡ (229c) <233f 234b>
    case OLMUL:
    case OASLMUL:
        a = AMULU;
        break;

```

Uses OASLMUL 233b and OLMUL 232d.

```

<gopcode() switch opcode cases 234b>+≡ (229c) <234a 234c>
    case OLMOD:
    case OASLMOD:
        a = AMODU;
        break;

```

Uses OASLMOD 233b and OLMOD 232f.

```

<gopcode() switch opcode cases 234c>+≡ (229c) <234b 240a>
    case OLDIV:
    case OASLDIV:
        a = ADIVU;
        break;

```

Uses OASLDIV 233b and OLDIV 232d.

10.10.5 Boolean expressions

Boolean expressions (&&, ||, comparisons) are not compiled by **cgen** at all—they are handled by **boolgen**, which generates conditional branches directly. This avoids materializing a 0 or 1 into a register; instead, the truth value controls the branch direction. See Section 10.11 for details.

10.10.6 Assignments part 1

Assignment code generation has two main paths: if the left-hand side is directly addressable (a simple variable, global, or indexed access), **gmove** can emit a single **MOV** from the right-hand side. Otherwise, the left-hand side must be evaluated into a register to get its address first, then the right-hand side is moved to that indirect location.

```

<cgenrel() switch node kind cases 234d>+≡ (223c) <229a 235b>
    case OAS:
        <cgenrel() when OAS, if bitfield assignment 420b>
        if(l->addable >= INDEXED && l->complex < FNX) {
            <cgenrel() when OAS, if right node not addressable or nn 234e>
            else
                // both left and right are addressable and nn == Z
                gmove(r, l);
            break;
        }
        // else
        <cgenrel() when OAS, when left node not addressable or too complex 238a>

```

Uses OAS 116a.

```

<cgenrel() when OAS, if right node not addressable or nn 234e>≡ (234d)
    if(nn != Z || r->addable < INDEXED) {
        <cgenrel() when OAS, if right node contain a call 235a>
        else
            regalloc(&nod, r, nn);
        cgen(r, &nod);
        gmove(&nod, l);
        if(nn != Z)
            gmove(&nod, nn);
    }

```

```
    regfree(&nod);
}
```

Uses Z 29a.

```
<cgenrel() when OAS, if right node contain a call 235a>≡ (234e)
    if(r->complex >= FNX && nn == Z)
        regret(&nod, r);
```

Uses Z 29a.

10.10.7 Pointers

Pointer operations use `lcgen` (“lvalue cgen”), which computes the *address* of an expression rather than its value. For `OADDR(\&x)`, `lcgen` returns the address of `x`; for `OIND(*p)`, `cgen` generates the load through the pointer. `lcgen` is also used on the left side of assignments when the destination is not directly addressable.

Getting the address, `lcgen()`

`lcgen` (“lvalue code generator”) computes the *address* of a node rather than its value. This is needed for `&x`, for the left side of assignments to non-addressable locations (e.g., `*p = v`), and for passing struct arguments by reference. It recurses through `OIND` nodes and falls back to `naddr` for simple names, which can express their address directly via the SB/FP-relative addressing modes.

```
<cgenrel() switch node kind cases 235b>+≡ (223c) <234d 236h>
    case OADDR:
        <cgenrel() nullwarn check if nn is null 189a>
        lcgen(l, nn);
        break;
```

Uses OADDR 116e.

```
<function lcgen(arm) 235c>≡ (542)
    void
    lcgen(Node *n, Node *nn)
    {
        Node nod;
        <lcgen() other locals 236e>

        <lcgen() debug 492b>
        <lcgen() sanity check n 236g>
        <lcgen() if nn nil 236b>
        switch(n->op) {
            <lcgen() switch node cases 236c>
            default:
                <lcgen() sanity check n in default case 236a>
                nod = *n;
                nod.op = OADDR;
                nod.left = n;
                nod.right = Z;
                nod.type = types[TIND];
                gopcode(OAS, &nod, Z, nn);
                break;

        }
    }
```

Uses OADDR 116e, OAS 116a, TIND 31c, Z 29a, and types 32b.

$\langle \text{lcgen}() \text{ sanity check } n \text{ in default case } 236a \rangle \equiv$ (235c)

```

    if(n->addable < INDEXED) {
        diag(n, "unknown op in lcgen: %0", n->op);
        break;
    }

```

Uses diag() 496b.

$\langle \text{lcgen}() \text{ if } nn \text{ nil } 236b \rangle \equiv$ (235c)

```

    if(nn == Z) {
        nn = &nod;
        regalloc(&nod, n, Z);
    }

```

Uses Z 29a.

$\langle \text{lcgen}() \text{ switch node cases } 236c \rangle \equiv$ (235c) 236d▷

```

    case OIND:
        cgen(n->left, nn);
        break;

```

Uses OIND 116e.

$\langle \text{lcgen}() \text{ switch node cases } 236d \rangle + \equiv$ (235c) ◁236c 236f▷

```

    case OCOMMA:
        cgen(n->left, n->left);
        lcgen(n->right, nn);
        break;

```

Uses OCOMMA 110g.

$\langle \text{lcgen}() \text{ other locals } 236e \rangle \equiv$ (235c)

```

    Prog *p1;

```

$\langle \text{lcgen}() \text{ switch node cases } 236f \rangle + \equiv$ (235c) ◁236d

```

    case OCOND:
        bcgen(n->left, 1);
        p1 = p;
        lcgen(n->right->left, nn);
        gbranch(OGOTO);
        patch(p1, pc);
        p1 = p;
        lcgen(n->right->right, nn);
        patch(p1, pc);
        break;

```

Uses OCOND 118f and OGOTO 108e.

$\langle \text{lcgen}() \text{ sanity check } n \text{ } 236g \rangle \equiv$ (235c)

```

    if(n == Z || n->type == T)
        return;

```

Uses T 31i and Z 29a.

Dereferencing the address

$\langle \text{cgenrel}() \text{ switch node kind cases } 236h \rangle + \equiv$ (223c) ◁235b 239c▷

```

    case OIND:
        ◁cgenrel() nullwarn check if nn is null 189a⟩
        regialloc(&nod, n, nn);
        ◁cgenrel() when OIND, if indirect of OADD constants 237e⟩
        else
            cgen(1, &nod);
        regind(&nod, n);

```

```

    gopcode(OAS, &nod, Z, nn);
    regfree(&nod);
    break;

```

Uses OAS 116a, OIND 116e, and Z 29a.

```

⟨function regialloc 237a⟩≡ (537b)
void
regialloc(Node *n, Node *tn, Node *o)
{
    Node nod;

    nod = *tn;
    nod.type = types[TIND];
    regalloc(n, &nod, o);
}

```

Uses TIND 31c and types 32b.

```

⟨function regind 237b⟩≡ (537b)
void
regind(Node *n, Node *nn)
{
    if(n->op != OREGISTER) {
        diag(n, "regind not OREGISTER");
        return;
    }
    n->op = OINDREG;
    n->type = nn->type;
}

```

Uses OINDREG 237c, OREGISTER 208f, and diag() 496b.

```

⟨after parsing nodes 237c⟩+≡ (27) ◁233b 300d▷
OINDREG,

```

```

⟨naddr() switch node kind cases 237d⟩+≡ (219b) ◁221c
case OINDREG:
    a->type = D_OREG;
    a->sym = S;
    a->offset = n->xoffset;
    a->reg = n->reg;
    break;

```

Uses OINDREG 237c and S 35d.

```

⟨cgenrel() when OIND, if indirect of OADD constants 237e⟩≡ (236h)
r = l;
while(r->op == OADD)
    r = r->right;
if(sconst(r) && (v = r->vconst + nod.xoffset) > -4096 && v < 4096) {
    v = r->vconst;
    r->vconst = 0;
    cgen(1, &nod);
    nod.xoffset += v;
    r->vconst = v;
}

```

Uses OADD 114c.

10.10.8 Assignments part 2

When the left side of an assignment is not directly addressable (e.g., `*p` where `p` is itself computed), the compiler must first load the destination address into a register via `reglcgen`, then store the right-hand value through that register. The Sethi-Ullman ordering applies here too: whichever side is more complex is evaluated first.

```

⟨cgenrel() when OAS, when left node not addressable or too complex 238a⟩≡      (234d)
  ⟨cgenrel() when OAS, when l not addressable, if l more complex than r 238b⟩
  else {
    regalloc(&nod, r, nn);
    cgen(r, &nod);
    reglcgen(&nod1, l, Z);
  }
  gmove(&nod, &nod1);
  regfree(&nod);
  regfree(&nod1);
  break;

```

Uses Z 29a.

```

⟨cgenrel() when OAS, when l not addressable, if l more complex than r 238b⟩≡      (238a)
  if(l->complex >= r->complex) {
    reglcgen(&nod1, l, Z);
    if(r->addable >= INDEXED) {
      gmove(r, &nod1);
      if(nn != Z)
        gmove(r, nn);
      regfree(&nod1);
      break;
    }
    // else
    regalloc(&nod, r, nn);
    cgen(r, &nod);
  }

```

Uses Z 29a.

`reglcgen()`

```

⟨function reglcgen(arm) 238c⟩≡      (542)
void
reglcgen(Node *t, Node *n, Node *nn)
{
  Node *r;
  long v;

  regialloc(t, n, nn);
  ⟨reglcgen() if OIND of OADD of constants 238d⟩
  ⟨reglcgen() if OINDREG with small offsets 239b⟩
  // else
  lcgen(n, t);
  regind(t, n);
}

```

```

⟨reglcgen() if OIND of OADD of constants 238d⟩≡      (238c)
  if(n->op == OIND) {
    r = n->left;
    while(r->op == OADD)
      r = r->right;
    if(sconst(r) && (v = r->vconst+t->xoffset) > -4096 && v < 4096) {
      v = r->vconst;
    }
  }

```

```

        r->vconst = 0;
        lcgen(n, t);
        t->xoffset += v;
        r->vconst = v;
        regind(t, n);
        return;
    }
}

```

Uses OADD 114c and OIND 116e.

```

⟨function sconst(arm) 239a⟩≡ (537b)
bool
sconst(Node *n)
{
    vlong vv;

    if(n->op == OCONST) {
        if(!typefd[n->type->etype]) {
            vv = n->vconst;
            if(vv >= (vlong)(-32766) && vv < (vlong)32766)
                return true;
            /*
             * should be specialised for constant values which will
             * fit in different instructions; for now, let 5l
             * sort it out
             */
            return true;
        }
    }
    return false;
}

```

Uses OCONST 111a and typefd 149c.

```

⟨reglcgen() if OINDREG with small offsets 239b⟩≡ (238c)
else if(n->op == OINDREG) {
    if((v = n->xoffset) > -4096 && v < 4096) {
        n->op = OREGISTER;
        cgen(n, t);
        t->xoffset += v;
        n->op = OINDREG;
        regind(t, n);
        return;
    }
}

```

Uses OINDREG 237c and OREGISTER 208f.

10.10.9 Assignments and arithmetic

Compound assignment operators like += and <=> combine a load, an arithmetic operation, and a store. The generated code loads the left-hand side into a temporary, applies `gopcode` with the right-hand operand, then stores back. If `nn` is non-null (the expression's value is used, as in `y = (x += 1)`), the result is also copied to `nn`.

```

⟨cgenrel() switch node kind cases 239c⟩+≡ (223c) <236h 242a>
case OASADD:
case OASSUB:
case OASAND:
case OASXOR:

```



```

case OASOR:
case OASASHL:
case OASASHR:
case OASLSHR:
    <cgenrel() when OASADD/OASSUB/etc, if bitfield assignment 420d>
    <cgenrel() when OASADD/OASSUB/etc, if r is OCONST 240c>
    // Fallthrough
case OASMUL:
case OASDIV:
case OASMOD:
case OASLMUL:
case OASLDIV:
case OASLMOD:
    <cgenrel() when OASMUL/OASDIV/etc, if bitfield assignment 420c>
    <cgenrel() when OASADD/OASUB/etc, if l more complex than r 240b>
    else {
        regalloc(&nod1, r, Z);
        cgen(r, &nod1);
        if(l->addable < INDEXED)
            reglcgen(&nod2, l, Z);
        else
            nod2 = *l;
    }

    regalloc(&nod, n, nn);
    gmove(&nod2, &nod);
    gopcode(o, &nod1, Z, &nod);
    gmove(&nod, &nod2);
    if(nn != Z)
        gopcode(OAS, &nod, Z, nn);
    regfree(&nod);
    regfree(&nod1);
    if(l->addable < INDEXED)
        regfree(&nod2);
    break;

```

Uses OAS 116a, OASADD 116c, OASAND 116c, OASASHL 116c, OASASHR 116c, OASDIV 116c, OASLDIV 233b, OASLMOD 233b, OASLMUL 233b, OASLSHR 233b, OASMOD 116c, OASMUL 116c, OASOR 116c, OASSUB 116c, OASXOR 116c, and Z 29a.

```

<gopcode() switch opcode cases 240a>+≡ (229c) <234c 256c>
case OAS:
    gmove(f1, t);
    return;

```

Uses OAS 116a.

```

<cgenrel() when OASADD/OASUB/etc, if l more complex than r 240b>≡ (239c)
if(l->complex >= r->complex) {
    if(l->addable < INDEXED)
        reglcgen(&nod2, l, Z);
    else
        nod2 = *l;
    regalloc(&nod1, r, Z);
    cgen(r, &nod1);
}

```

Uses Z 29a.

```

<cgenrel() when OASADD/OASSUB/etc, if r is OCONST 240c>≡ (239c)
if(r->op == OCONST)
    if(!typefd[r->type->etype])
        if(!typefd[n->type->etype]) {

```

```

    if(l->addable < INDEXED)
        reglcodegen(&nod2, l, Z);
    else
        nod2 = *l;
    regalloc(&nod, r, nn);
    gopcode(OAS, &nod2, Z, &nod);
    gopcode(o, r, Z, &nod);
    gopcode(OAS, &nod, Z, &nod2);

    regfree(&nod);
    if(l->addable < INDEXED)
        regfree(&nod2);
    break;
}

```

Uses OAS 116a, OCONST 111a, Z 29a, and typefd 149c.

10.10.10 Array accesses

10.10.11 Field accesses

By the time code generation runs, `a.field` has been converted by `tcomd` into an address computation: the field's byte offset is added to the struct's base address, and the node's type is set to the field's type. The `ODOT` node becomes an `OIND` (indirect load) with the computed offset, so `cgen` handles it like any other pointer dereference.

<tcomd() convert field access in offset 241a>≡ (171b)

```

    makedot(n, t, o);

```

Uses `makedot()` 241b.

<function makedot 241b>≡ (518c)

```

/// tcomd -> <>
void
makedot(Node *n, Type *t, long o)
{
    Node *n1, *n2;

    <makedot() if bitfields 419i>
    n->addable = n->left->addable;
    if(!n->addable) {
        n1 = new1(OCNST, Z, Z);
        n1->vconst = o;
        n1->type = types[TLONG];
        n->right = n1;
        n->type = t;
        return;
    }
    // else
    n->left->type = t;
    if(o == 0) {
        *n = *n->left;
        return;
    }
    n->type = t;
    n1 = new1(OCNST, Z, Z);
    n1->vconst = o;
    t = typ(TIND, t);
    t->width = types[TIND]->width;
    n1->type = t;

```

```

n2 = new1(OADDR, n->left, Z);
n2->type = t;

n1 = new1(OADD, n1, n2);
n1->type = t;

n->op = OIND;
n->left = n1;
n->right = Z;
}

```

Uses OADD 114c, OADDR 116e, OCONST 111a, OIND 116e, TIND 31c, TLONG 30b, Z 29a, new1() 29h, typ() 32a, and types 32b.

```

⟨cgenrel() switch node kind cases 242a⟩+≡ (223c) <239c 248a>
case ODOT:
    sugen(l, nodrat, l->type->width);
    if(nn != Z) {
        warn(n, "non-interruptable temporary");
        nod = *nodrat;
        if(!r || r->op != OCONST) {
            diag(n, "DOT and no offset");
            break;
        }
        nod.xoffset += (long)r->vconst;
        nod.type = n->type;
        cgen(&nod, nn);
    }
    break;

```

Uses OCONST 111a, ODOT 117a, Z 29a, diag() 496b, and warn() 142.

```

⟨global nodrat 242b⟩≡ (535b)
Node* nodrat;

```

```

⟨global symrathole 242c⟩≡ (535b)
Sym* symrathole;

```

```

⟨ginit() locals 242d⟩≡ (51c)
Type *t;

```

```

⟨ginit() special nodes initialisation 242e⟩+≡ (51c) <209b 260b>
t = typ(TARRAY, types[TCHAR]);
symrathole = slookup(".rathole");
symrathole->class = CGLOBL;
symrathole->type = t;

nodrat = new(ONAME, Z, Z);
nodrat->sym = symrathole;
nodrat->type = types[TIND];
nodrat->etype = TVOID; // not TIND?
nodrat->class = CGLOBL;
complex(nodrat);
nodrat->type = t;

```

Uses CGLOBL 33b, ONAME 27, TARRAY 31c, TCHAR 30b, TIND 31c, TVOID 31b, Z 29a, complex() 210a, new() 29d, slookup() 35f, typ() 32a, and types 32b.

```

⟨gclean() adjust symrathole width 242f⟩≡ (49c)
symrathole->type->width = nrathole;

```

```

⟨global nrathole 242g⟩≡ (535b)
long nrathole;

```

```

⟨ginit() misc initialisations 243a⟩+≡ (51c) <226h 281e>
    nrathole = 0;

⟨function sugen(arm) 243b⟩≡ (542)
void
sugen(Node *n, Node *nn, long w)
{
    Prog *p1;
    Node nod0, nod1, nod2, nod3, nod4, *l, *r;
    Type *t;
    long pc1;
    int i, m, c;

    if(n == Z || n->type == T)
        return;

    ⟨sugen() debug 492c⟩
    if(nn == nodrat)
        if(w > nrathole)
            nrathole = w;

    switch(n->op) {
    case OIND:
        if(nn == Z) {
            nullwarn(n->left, Z);
            break;
        }
        // Fallthrough

    default:
        goto copy;

    case OCONST:
        if(n->type && typev[n->type->etype]) {
            if(nn == Z) {
                nullwarn(n->left, Z);
                break;
            }

            t = nn->type;
            nn->type = types[TLONG];
            reglcgen(&nod1, nn, Z);
            nn->type = t;

            if(aligned(0, types[TCHAR], Aarg1)) /* isbigendian */
                gopcode(OAS, nod32const(n->vconst>>32), Z, &nod1);
            else
                gopcode(OAS, nod32const(n->vconst), Z, &nod1);
            nod1.xoffset += SZ_LONG;
            if(aligned(0, types[TCHAR], Aarg1)) /* isbigendian */
                gopcode(OAS, nod32const(n->vconst), Z, &nod1);
            else
                gopcode(OAS, nod32const(n->vconst>>32), Z, &nod1);

            regfree(&nod1);
            break;
        }
        goto copy;

    case ODOT:

```

```

l = n->left;
sugen(l, nodrat, l->type->width);
if(nn != Z) {
    warn(n, "non-interruptable temporary");
    nod1 = *nodrat;
    r = n->right;
    if(!r || r->op != OCONST) {
        diag(n, "DOT and no offset");
        break;
    }
    nod1.xoffset += (long)r->vconst;
    nod1.type = n->type;
    sugen(&nod1, nn, w);
}
break;
⟨sugen() switch op cases 431⟩

case OAS:
    if(nn == Z) {
        if(n->addable < INDEXED)
            sugen(n->right, n->left, w);
        break;
    }
    sugen(n->right, nodrat, w);
    warn(n, "non-interruptable temporary");
    sugen(nodrat, n->left, w);
    sugen(nodrat, nn, w);
    break;

case OFUNC:
    if(nn == Z) {
        sugen(n, nodrat, w);
        break;
    }
    if(nn->op != OIND) {
        nn = new1(OADDR, nn, Z);
        nn->type = types[TIND];
        nn->addable = 0;
    } else
        nn = nn->left;
    n = new(OFUNC, n->left, new(OLIST, nn, n->right));
    n->type = types[TVOID];
    n->left->type = types[TVOID];
    cgen(n, Z);
    break;

//TODO: lpsplit and move with other OCOND stuff later
case OCOND:
    bcgen(n->left, 1);
    p1 = p;
    sugen(n->right->left, nn, w);
    gbranch(OGOTO);
    patch(p1, pc);
    p1 = p;
    sugen(n->right->right, nn, w);
    patch(p1, pc);
    break;

case OCOMMA:
    cgen(n->left, Z);

```

```

        sugen(n->right, nn, w);
        break;
    }
    return;
copy:
    if(nn == Z)
        return;
    if(n->complex >= FNX && nn->complex >= FNX) {
        t = nn->type;
        nn->type = types[TLONG];
        regialloc(&nod1, nn, Z);
        lcgen(nn, &nod1);
        regsalloc(&nod2, nn);
        nn->type = t;

        gopcode(OAS, &nod1, Z, &nod2);
        regfree(&nod1);

        nod2.type = typ(TIND, t);

        nod1 = nod2;
        nod1.op = OIND;
        nod1.left = &nod2;
        nod1.right = Z;
        nod1.complex = 1;
        nod1.type = t;

        sugen(n, &nod1, w);
        return;
    }

    w /= SZ_LONG;
    if(w <= 2) {
        if(n->complex > nn->complex) {
            reglpcgen(&nod1, n, 1);
            reglpcgen(&nod2, nn, 1);
        } else {
            reglpcgen(&nod2, nn, 1);
            reglpcgen(&nod1, n, 1);
        }
        regalloc(&nod3, &regnode, Z);
        regalloc(&nod4, &regnode, Z);
        if(nod3.reg > nod4.reg){
            /* code below assumes nod3 loaded first */
            Node t = nod3; nod3 = nod4; nod4 = t;
        }
        nod0 = *nodconst((1<<nod3.reg)|(1<<nod4.reg));
        if(w == 2 && nod1.xoffset == 0)
            gmovm(&nod1, &nod0, 0);
        else {
            gmove(&nod1, &nod3);
            if(w == 2) {
                nod1.xoffset += SZ_LONG;
                gmove(&nod1, &nod4);
            }
        }
        if(w == 2 && nod2.xoffset == 0)
            gmovm(&nod0, &nod2, 0);
        else {

```

```

        gmove(&nod3, &nod2);
        if(w == 2) {
            nod2.xoffset += SZ_LONG;
            gmove(&nod4, &nod2);
        }
    }
    regfree(&nod1);
    regfree(&nod2);
    regfree(&nod3);
    regfree(&nod4);
    return;
}

if(n->complex > nn->complex) {
    reglpcgen(&nod1, n, 0);
    reglpcgen(&nod2, nn, 0);
} else {
    reglpcgen(&nod2, nn, 0);
    reglpcgen(&nod1, n, 0);
}

m = 0;
for(c = 0; c < w && c < 4; c++) {
    i = tmpreg();
    if (i == 0)
        break;
    reg[i]++;
    m |= 1<<i;
}
nod4 = *(nodconst(m));
if(w < 3*c) {
    for (; w>c; w-=c) {
        gmovm(&nod1, &nod4, 1);
        gmovm(&nod4, &nod2, 1);
    }
    goto out;
}

regalloc(&nod3, &regnode, Z);
gopcode(OAS, nodconst(w/c), Z, &nod3);
w %= c;

pc1 = pc;
gmovm(&nod1, &nod4, 1);
gmovm(&nod4, &nod2, 1);

gopcode(OSUB, nodconst(1), Z, &nod3);
gopcode(OEQ, nodconst(0), &nod3, Z);
p->as = ABGT;
patch(p, pc1);
regfree(&nod3);

out:
if (w) {
    i = 0;
    while (c>w) {
        while ((m&(1<<i)) == 0)
            i++;
        m &= ~(1<<i);
        reg[i] = 0;
    }
}

```

```

        c--;
        i++;
    }
    nod4.vconst = m;
    gmovm(&nod1, &nod4, 0);
    gmovm(&nod4, &nod2, 0);
}
i = 0;
do {
    while ((m&(1<<i)) == 0)
        i++;
    reg[i] = 0;
    c--;
    i++;
} while (c>0);
regfree(&nod1);
regfree(&nod2);
}

```

Uses Aarg1 276b, OADDR 116e, OAS 116a, OCOMMA 110g, OCOND 118f, OCONST 111a, ODOT 117a, OEQ 115g, OFUNC 117c, OGOTO 108e, OIND 116e, OLIST 29b, OSUB 114c, T 31i, TCHAR 30b, TIND 31c, TLONG 30b, TVOID 31b, Z 29a, diag() 496b, gmovm() 247a, new() 29d, new1() 29h, nod32const() 273c, nullwarn() 189b, reglpcgen() 247c, tmpreg() 247b, typ() 32a, types 32b, typev 280a, and warn() 142.

```

⟨function gmovm(arm) 247a⟩≡ (537b)
void
gmovm(Node *f, Node *t, int w)
{
    gins(AMOVm, f, t);
    p->scond |= C_UBIT;
    if(w)
        p->scond |= C_WBIT;
}

```

```

⟨function tmpreg(arm) 247b⟩≡ (537b)
int
tmpreg(void)
{
    int i;

    for(i=REGRET+1; i<NREG; i++)
        if(reg[i] == 0)
            return i;
    diag(Z, "out of fixed registers");
    return 0;
}

```

Uses Z 29a and diag() 496b.

```

⟨function reglpcgen(arm) 247c⟩≡ (542)
void
reglpcgen(Node *n, Node *nn, int f)
{
    Type *t;

    t = nn->type;
    nn->type = types[TLONG];
    if(f)
        reglcgen(n, nn, Z);
    else {
        regialloc(n, nn, Z);
        lcgen(nn, n);
    }
}

```



```

        regind(n, nn);
    }
    nn->type = t;
}

```

Uses TLONG 30b, Z 29a, and types 32b.

10.10.12 Cast

A cast is a two-step conversion: the source value is loaded into a register typed as the source type, then moved to a register typed as the destination type. The `gmove` call between the two registers handles the actual conversion (sign extension, truncation, int $\leftrightarrow$ float). If the types are the same width and signedness, the cast is optimized away.

```

⟨cgenrel() switch node kind cases 248a⟩+≡ (223c) <242a 250a>
    case OCAST:
        ⟨cgenrel() nullwarn check if nn is null 189a⟩
        /*
         * convert from types l->n->nn
         */
        ⟨cgenrel() if same types 249a⟩
        // else
        regalloc(&nod, l, nn);
        cgen(l, &nod);
        regalloc(&nod1, n, &nod);
        if(inrel)
            gmover(&nod, &nod1);
        else
            gopcode(OAS, &nod, Z, &nod1);
            gopcode(OAS, &nod1, Z, nn);
            regfree(&nod1);
            regfree(&nod);
            break;

```

Uses OAS 116a, OCAST 118a, Z 29a, and `gmover()` 248b.

```

⟨function gmover(arm) 248b⟩≡ (537b)
    void
    gmover(Node *f, Node *t)
    {
        int ft, tt, a;

        ft = f->type->etype;
        tt = t->type->etype;
        a = AGOK;
        if(typechlp[ft] && typechlp[tt] && ewidth[ft] >= ewidth[tt]){
            switch(tt){
            case TSHORT:
                a = AMOVH;
                break;
            case TUSHORT:
                a = AMOVHU;
                break;
            case TCHAR:
                a = AMOVb;
                break;
            case TUCCHAR:
                a = AMOVBU;
                break;
            }
        }
    }

```

```

}
if(a == AGOK)
    gmove(f, t);
else
    gins(a, f, t);
}

```

Uses TCHAR 30b, TSHORT 30b, TUCCHAR 30b, TUSHORT 30b, and typechlp 215g.

```

⟨cgenrel() if same types 249a⟩≡ (248a)
    if(nocast(l->type, n->type)) {
        if(nocast(n->type, nn->type)) {
            cgen(l, nn);
            break;
        }
    }
}

```

Uses nocast() 249b.

```

⟨function nocast 249b⟩≡ (518c)
    /// cgen -> cgenrel -> <>
    /*
     * a cast that generates no code
     * (same size move)
     */
    bool
    nocast(Type *t1, Type *t2)
    {
        int i, b;

        if(t1->nbits)
            return false;
        i = 0;
        if(t2 != T)
            i = t2->etype;
        b = 1<<i;
        i = 0;
        if(t1 != T)
            i = t1->etype;
        if(b & ncast[i])
            return true;
        return false;
    }
}

```

Uses T 31i.

```

⟨global ncast 249c⟩≡ (537b)
    long ncast[NTYPE] =
    {
        [TXXX] = 0,
        [TCHAR] = BCHAR|BUCHAR,
        [TUCCHAR] = BCHAR|BUCHAR,
        [TSHORT] = BSHORT|BUSHORT,
        [TUSHORT] = BSHORT|BUSHORT,
        [TINT] = BINT|BUINT|BLONG|BULONG|BIND,
        [TUINT] = BINT|BUINT|BLONG|BULONG|BIND,
        [TLONG] = BINT|BUINT|BLONG|BULONG|BIND,
        [TULONG] = BINT|BUINT|BLONG|BULONG|BIND,
        [TVLONG] = BVLONG|BUVLONG,
        [TUVLONG] = BVLONG|BUVLONG,
        [TFLOAT] = BFLOAT,
        [TDOUBLE] = BDOUBLE,
    }

```

```

[TIND] = BLONG|BULONG|BIND,
[TFUNC] = 0,
[TARRAY] = 0,
[TVOID] = 0,
[TSTRUCT] = BSTRUCT,
[TUNION] = BUNION,
[TENUM] = 0,
};

```

Uses BCHAR 121, BDOUBLE 121, BFLOAT 121, BIND 121, BINT 121, BLONG 121, BSHORT 121, BSTRUCT 121, BCHAR 121, BUINT 121, BULONG 121, BUNION 121, BUSHORT 121, BUVLONG 121, BVLONG 121, and NTYPE 30a.

10.10.13 Ternary expressions

The ternary operator `a ? b : c` is compiled like an `if/else` that produces a value: evaluate the condition with `bcgen`, generate code for the “then” branch into the destination register, branch past the “else” branch, generate code for the “else” branch into the same destination, and patch the branch targets.

```

⟨cgenrel() switch node kind cases 250a⟩+≡ (223c) <248a 250b>
  case OCOND:
    bcgen(1, true);
    p1 = p;
    cgen(r->left, nn);
    gbranch(OGOTO);
    patch(p1, pc);
    p1 = p;
    cgen(r->right, nn);
    patch(p1, pc);
    break;

```

Uses OCOND 118f and OGOTO 108e.

10.10.14 Prefix/postfix

Post-increment (`x++`) returns the old value and then increments: the value is saved in a register (`nod`), the incremented value is computed in another register (`nod1`), and the incremented value is stored back. Pre-increment (`++x`) is simpler: just increment and return the new value. For pointers, the increment is scaled by the pointed-to type’s width (e.g., `int*` increments by 4).

```

⟨cgenrel() switch node kind cases 250b⟩+≡ (223c) <250a 251>
  case OPOSTINC:
  case OPOSTDEC:
    v = 1;
    if(l->type->etype == TIND)
      v = l->type->link->width;
    if(o == OPOSTDEC)
      v = -v;
    ⟨cgenrel() when OPOSTINC/OPOSTDEC, if bitfield 420f⟩
    if(nn == Z)
      goto pre;

    if(l->addable < INDEXED)
      reglcgen(&nod2, l, Z);
    else
      nod2 = *l;

    regalloc(&nod, l, nn);
    gopcode(OAS, &nod2, Z, &nod);
    regalloc(&nod1, l, Z);

```

```

if(typefd[l->type->etype]) {
    regalloc(&nod3, 1, Z);
    if(v < 0) {
        gopcode(OAS, nodfconst(-v), Z, &nod3);
        gopcode(OSUB, &nod3, &nod, &nod1);
    } else {
        gopcode(OAS, nodfconst(v), Z, &nod3);
        gopcode(OADD, &nod3, &nod, &nod1);
    }
    regfree(&nod3);
} else
    gopcode(OADD, nodconst(v), &nod, &nod1);
gopcode(OAS, &nod1, Z, &nod2);

regfree(&nod);
regfree(&nod1);
if(l->addable < INDEXED)
    regfree(&nod2);
break;

```

Uses OADD 114c, OAS 116a, OPOSTDEC 118j, OPOSTINC 118j, OSUB 114c, TIND 31c, Z 29a, and typefd 149c.

```

⟨cgenrel() switch node kind cases 251⟩+≡ (223c) ◁250b 252a▷
case OPREINC:
case OPREDEC:
    v = 1;
    if(l->type->etype == TIND)
        v = l->type->link->width;
    if(o == OPREDEC)
        v = -v;
⟨cgenrel() when OPREINC/OPREDEC, if bitfield 420g⟩

```

```

pre:
if(l->addable < INDEXED)
    reglcgen(&nod2, 1, Z);
else
    nod2 = *l;

regalloc(&nod, 1, nn);
gopcode(OAS, &nod2, Z, &nod);
if(typefd[l->type->etype]) {
    regalloc(&nod3, 1, Z);
    if(v < 0) {
        gopcode(OAS, nodfconst(-v), Z, &nod3);
        gopcode(OSUB, &nod3, Z, &nod);
    } else {
        gopcode(OAS, nodfconst(v), Z, &nod3);
        gopcode(OADD, &nod3, Z, &nod);
    }
    regfree(&nod3);
} else
    gopcode(OADD, nodconst(v), Z, &nod);
gopcode(OAS, &nod, Z, &nod2);
if(nn && l->op == ONAME) /* in x=++i, emit USED(i) */
    gins(ANOP, 1, Z);

regfree(&nod);
if(l->addable < INDEXED)
    regfree(&nod2);
break;

```

Uses OADD 114c, OAS 116a, ONAME 27, OPREDEC 118h, OPREINC 118h, OSUB 114c, TIND 31c, Z 29a, and typefd 149c.

10.10.15 sizeof()

10.11 Boolean expressions

Boolean expressions (`&&`, `||`, comparisons) are compiled differently from arithmetic: instead of producing a value in a register, they produce a *branch*. `boolgen` generates code that evaluates the expression and ends with a conditional branch (ABxx) whose target is later filled by `patch`. This enables short-circuit evaluation: `a && b` generates code for `a`, branches to “false” if zero, then evaluates `b`. The `bcomplex` wrapper in the statement generator uses this to wire boolean expressions directly into `if/while` control flow without materializing a temporary boolean value.

```
<cgenrel() switch node kind cases 252a>+≡ (223c) <251 252b>
case OANDAND:
case OOROR:
    boolgen(n, true, nn);
    if(nn == Z)
        patch(p, pc);
    break;
```

Uses OANDAND 115c, OOROR 115c, and Z 29a.

```
<cgenrel() switch node kind cases 252b>+≡ (223c) <252a 252c>
case ONOT:
    <cgenrel() nullwarn check if nn is null 189a>
    boolgen(n, true, nn);
    break;
```

Uses ONOT 115e.

```
<cgenrel() switch node kind cases 252c>+≡ (223c) <252b 257>
case OEQ:
case ONE:
case OLE:
case OLT:
case OGE:
case OGT:
case OLO:
case OLS:
case OHI:
case OHS:
    <cgenrel() nullwarn check if nn is null 189a>
    boolgen(n, true, nn);
    break;
```

Uses OEQ 115g, OGE 115g, OGT 115g, OHI 300d, OHS 300d, OLE 115g, OLO 300d, OLS 300d, OLT 115g, and ONE 115g.

```
<constant BTRUE(arm) 252d>≡ (531b)
#define BTRUE 0x1000
```

```
<gopcode() locals 252e>≡ (229c)
bool btrue;
```

```
<gopcode() initialisation 252f>≡ (229c)
btrue = o & BTRUE;
o &= ~BTRUE;
```

Uses BTRUE 252d.

```

⟨function boolgen(arm) 253⟩≡ (542)
  /// cgenrel | bcomplex -> <>
  void
  boolgen(Node *n, int btrue, Node *nn)
  {
    int o;
    Prog *p1, *p2;
    Node *l, *r, nod, nod1;
    long curs; // to save cursafe

    ⟨boolgen() debug 491g⟩
    curs = cursafe; // save

    l = n->left;
    r = n->right;
    switch(n->op) {

    case OCONST:
      o = vconst(n);
      if(!btrue)
        o = !o;
      gbranch(OGOTO);
      if(o) {
        p1 = p;
        gbranch(OGOTO);
        patch(p1, pc);
      }
      goto com;

    case OCOMMA:
      cgen(l, Z);
      boolgen(r, btrue, nn);
      break;

    case ONOT:
      boolgen(l, !btrue, nn);
      break;

    case OCOND:
      bcgen(l, 1);
      p1 = p;
      bcgen(r->left, btrue);
      p2 = p;
      gbranch(OGOTO);
      patch(p1, pc);
      p1 = p;
      bcgen(r->right, !btrue);
      patch(p2, pc);
      p2 = p;
      gbranch(OGOTO);
      patch(p1, pc);
      patch(p2, pc);
      goto com;

    case OANDAND:
      if(!btrue)
        goto caseor;

    caseand:
      bcgen(l, btrue);

```

```

    p1 = p;
    bcgen(r, !btrue);
    p2 = p;
    patch(p1, pc);
    gbranch(OGOTO);
    patch(p2, pc);
    goto com;

case OOROR:
    if(!btrue)
        goto caseand;

caseor:
    bcgen(l, !btrue);
    p1 = p;
    bcgen(r, !btrue);
    p2 = p;
    gbranch(OGOTO);
    patch(p1, pc);
    patch(p2, pc);
    goto com;

case OEQ:
case ONE:
case OLE:
case OLT:
case OGE:
case OGT:
case OHI:
case OHS:
case OLO:
case OLS:
    o = n->op;
    if(btrue)
        o = comrel[relindex(o)];
    if(l->complex >= FNX && r->complex >= FNX) {
        regret(&nod, r);
        cgenrel(r, &nod, btrue);
        regsalloc(&nod1, r);
        gopcode(OAS, &nod, Z, &nod1);
        regfree(&nod);
        nod = *n;
        nod.right = &nod1;
        boolgen(&nod, btrue, nn);
        break;
    }
    if(sconst(l)) {
        regalloc(&nod, r, nn);
        cgenrel(r, &nod, btrue);
        o = invrel[relindex(o)];
        gopcode(btrue ? o | BTRUE : o, l, &nod, Z);
        regfree(&nod);
        goto com;
    }
    if(sconst(r)) {
        regalloc(&nod, l, nn);
        cgenrel(l, &nod, btrue);
        gopcode(btrue ? o | BTRUE : o, r, &nod, Z);
        regfree(&nod);
        goto com;
    }

```

```

}

if(l->complex >= r->complex) {
    regalloc(&nod1, l, nn);
    cgenrel(l, &nod1, btrue);
    regalloc(&nod, r, Z);
    cgenrel(r, &nod, btrue);
} else {
    regalloc(&nod, r, nn);
    cgenrel(r, &nod, btrue);
    regalloc(&nod1, l, Z);
    cgenrel(l, &nod1, btrue);
}
gopcode(btrue ? o | BTRUE : o, &nod, &nod1, Z);
regfree(&nod);
regfree(&nod1);

com:
    if(nn != Z) {
        p1 = p;
        gopcode(OAS, nodconst(1), Z, nn);
        gbranch(OGOTO);
        p2 = p;
        patch(p1, pc);
        gopcode(OAS, nodconst(0), Z, nn);
        patch(p2, pc);
    }
    break;

default:
    regalloc(&nod, n, nn);
    cgen(n, &nod);
    o = ONE;
    if(btrue)
        o = comrel[relindex(o)];
    if(typefd[n->type->etype]) {
        gopcode(btrue ? o | BTRUE : o, nodfconst(0), &nod, Z);
    } else
        gopcode(o, nodconst(0), &nod, Z);
    regfree(&nod);
    goto com;

}
cursafe = curs; // restore
}

```

Uses BTRUE 252d, OANDAND 115c, OAS 116a, OCOMMA 110g, OCOND 118f, OCONST 111a, OEQ 115g, OGE 115g, OGOTO 108e, OGT 115g, OHI 300d, OHS 300d, OLE 115g, OLO 300d, OLS 300d, OLT 115g, ONE 115g, ONOT 115e, OOROR 115c, Z 29a, cgenrel() 223c, comrel 256a, invrel 256b, relindex() 233a, typefd 149c, and vconst() 176c.

```

⟨function bcgen 255⟩≡ (542)
void
bcgen(Node *n, bool btrue)
{
    if(n->type == T)
        gbranch(OGOTO);
    else
        boolgen(n, btrue, Z);
}

```

Uses OGOTO 108e, T 31i, and Z 29a.

`<global comrel 256a>≡` (518c)

```
/* OEQ, ONE, OLE, OLS, OLT, OLO, OGE, OHS, OGT, OHI */
char comrel[12] =
{
    ONE, OEQ, OGT, OHI, OGE, OHS, OLT, OLO, OLE, OLS,
};
```

Uses OEQ 115g, OGE 115g, OGT 115g, OHI 300d, OHS 300d, OLE 115g, OLO 300d, OLS 300d, OLT 115g, and ONE 115g.

`<global invrel 256b>≡` (518c)

```
char invrel[12] =
{
    OEQ, ONE, OGE, OHS, OGT, OHI, OLE, OLS, OLT, OLO,
};
```

Uses OEQ 115g, OGE 115g, OGT 115g, OHI 300d, OHS 300d, OLE 115g, OLO 300d, OLS 300d, OLT 115g, and ONE 115g.

`<gopcode() switch opcode cases 256c>+≡` (229c) <240a 258a>

```
case OCASE:
case OEQ:
case ONE:
case OLT:
case OLE:
case OGE:
case OGT:
case OLO:
case OLS:
case OHS:
case OHI:
    a = ACMP;
    if(et == TFLOAT)
        a = ACMPF;
    else
        if(et == TDOUBLE || et == TVLONG)
            a = ACMPD;
    nextpc();
    p->as = a;
    naddr(f1, &p->from);
    if(a == ACMP && f1->op == OCONST && p->from.offset < 0 &&
        p->from.offset != 0x80000000) {
        p->as = ACMN;
        p->from.offset = -p->from.offset;
    }
    raddr(f2, p);
    switch(o) {
case OEQ:
    a = ABEQ;
    break;
case ONE:
    a = ABNE;
    break;
case OLT:
    a = ABLT;
    /* ensure NaN comparison is always false */
    if(typefd[et] && !btrue)
        a = ABMI;
    break;
case OLE:
    a = ABLE;
    if(typefd[et] && !btrue)
        a = ABLS;
    break;
```

```

case OGE:
    a = ABGE;
    if(typefd[et] && btrue)
        a = ABPL;
    break;
case OGT:
    a = ABGT;
    if(typefd[et] && btrue)
        a = ABHI;
    break;
case OLO:
    a = ABLO;
    break;
case OLS:
    a = ABLS;
    break;
case OHS:
    a = ABHS;
    break;
case OHI:
    a = ABHI;
    break;

case OCASE:
    nextpc();
    p->as = ACASE;
    p->scond = 0x9;
    naddr(f2, &p->from);
    a = ABHI;
    break;
}
f1 = Z;
f2 = Z;
break;

```

Uses OCASE 109b, OCONST 111a, OEQ 115g, OGE 115g, OGT 115g, OHI 300d, OHS 300d, OLE 115g, OLO 300d, OLS 300d, OLT 115g, ONE 115g, TDOUBLE 31a, TFLOAT 31a, TVLONG 30b, Z 29a, raddr() 453b, and typefd 149c.

10.12 Function calls

A function call generates three phases: push arguments onto the stack via **gargs** (which walks the argument list right-to-left using **regaalloc** to assign stack slots), emit BL to the callee, then copy the return value from REGRET (R0) to the destination. If the callee is not directly addressable (e.g., a function pointer), its address is loaded into a register first.

```

⟨cgenrel() switch node kind cases 257⟩+≡ (223c) <252c 420a>
case OFUNC:
    ⟨cgenrel() when OFUNC, if complex indirect function call 260e⟩
    // else

    ⟨cgenrel() OFUNC case, if use REGARG, part one 337e⟩
    gargs(r, &nod, &nod1);
    ⟨cgenrel() OFUNC case, if l not addressable 260d⟩
    else
        gopcode(OFUNC, Z, Z, 1);
    ⟨cgenrel() OFUNC case, if use REGARG, part two 337f⟩
    if(nn != Z) {
        // store result of function call (in REGRET) to dest nn
        regret(&nod, n);
    }

```

```

        gopcode(OAS, &nod, Z, nn);
        regfree(&nod);
    }
    break;

```

Uses OAS 116a, OFUNC 117c, and Z 29a.

```

⟨gopcode() switch opcode cases 258a⟩+≡ (229c) ◁256c
    case OFUNC:
        a = ABL;
        break;

```

Uses OFUNC 117c.

10.12.1 gargs()

gargs pushes function arguments onto the stack in two passes. The first pass (**garg1**) handles simple arguments: each is evaluated and stored at the appropriate stack offset. The first word-sized argument goes to **REGARG** (R0) instead of the stack. The second pass evaluates complex arguments (those containing function calls, marked with **FNX**) that were deferred in pass one because they might clobber registers. **cursafe** and **maxargsafe** track the maximum stack depth needed for nested calls.

```

⟨global curarg 258b⟩≡ (535b)
    long curarg;

```

```

⟨global maxargsafe 258c⟩≡ (535b)
    long maxargsafe;

```

```

⟨codgen() initialisation 258d⟩≡ (194c) 258f▷
    curarg = 0;
    maxargsafe = 0;

```

```

⟨global cursafe 258e⟩≡ (535b)
    long cursafe;

```

```

⟨codgen() initialisation 258f⟩+≡ (194c) ◁258d
    cursafe = 0;

```

```

⟨function gargs 258g⟩≡ (537b)
    /// cgenrel -> <>
    void
    gargs(Node *n, Node *tn1, Node *tn2)
    {
        Node fnxargs[20], *fnxp;
        long regs; // to save cursafe

        regs = cursafe; // save

        fnxp = fnxargs;
        // step1
        garg1(n, tn1, tn2, 0, &fnxp); /* compile fns to temps */

        curarg = 0;
        fnxp = fnxargs;
        // step2
        garg1(n, tn1, tn2, 1, &fnxp); /* compile normal args and temps */

        cursafe = regs; // restore
    }

```

```

⟨function garg1(arm) 259a⟩≡ (537b)
  /// cgenrel -> gargs -> <>
  void
  garg1(Node *n, Node *tn1, Node *tn2, int f, Node **fnxp)
  {
    Node nod;

    if(n == Z)
      return;
    if(n->op == OLIST) {
      garg1(n->left, tn1, tn2, f, fnxp);
      garg1(n->right, tn1, tn2, f, fnxp);
      return;
    }
    // else, an argument

    if(f == 0) {
      ⟨garg() when f == 0, if n is complex part1 261b⟩
    }
    else

    ⟨garg1() if complex argument type 272a⟩
    else
    ⟨garg1() if use REGARG and curarg is zero and simple type 338a⟩
    else {
      regalloc(tn1, n, Z);
      ⟨garg() when f != 0, if n is complex part2 261c⟩
      else
        cgen(n, tn1);
      regaalloc(tn2, n);
      gopcode(OAS, tn1, Z, tn2);
      regfree(tn1);
    }
  }

```

Uses OAS 116a, OLIST 29b, and Z 29a.

```

⟨function regaalloc(arm) 259b⟩≡ (537b)
  void
  regaalloc(Node *n, Node *nn)
  {
    curarg = align(curarg, nn->type, Aarg1);
    *n = *nn;
    n->op = OINDREG;
    n->reg = REGSP;
    n->xoffset = curarg + SZ_LONG;
    n->complex = 0;
    n->addable = 20;
    curarg = align(curarg, nn->type, Aarg2);
    maxargsafe = maxround(maxargsafe, cursafe+curarg);
  }

```

Uses Aarg1 276b, Aarg2 276b, and OINDREG 237c.

10.12.2 Caller-save vs Callee-save

Plan 9's C compilers use a caller-save convention: the caller must save any live temporaries before a function call, because the callee is free to clobber all registers. `regalloc` allocates stack space in the caller's frame for

these spills, using the synthetic `.safe` variable to track the high-water mark. This is simpler than callee-save (no need for the callee to save/restore), at the cost of slightly more stack traffic.

`<global nodsafe 260a>≡ (535b)`
`Node* nodsafe;`

`<ginit() special nodes initialisation 260b>+≡ (51c) <242e 272c>`
`nodsafe = new(ONAME, Z, Z);`
`nodsafe->sym = slookup(".safe");`
`nodsafe->type = types[TINT];`
`nodsafe->etype = types[TINT]->etype; // TINT`
`nodsafe->class = CAUTO;`
`complex(nodsafe);`

Uses CAUTO 33b, ONAME 27, TINT 30b, Z 29a, complex() 210a, new() 29d, slookup() 35f, and types 32b.

`<function regsalloc 260c>≡ (537b)`
`/// garg1 -> <>`
`void`
`regsalloc(Node *n, Node *nn)`
`{`
`cursafe = align(cursafe, nn->type, Aaut3);`
`maxargsafe = maxround(maxargsafe, cursafe+curarg);`
`*n = *nodsafe;`
`n->xoffset = -(stkoff + cursafe);`
`n->type = nn->type;`
`n->etype = nn->type->etype;`
`n->lineno = nn->lineno;`
`}`

Uses Aaut3 276b and stkoff 196c.

10.12.3 Indirect function calls

When calling through a function pointer (e.g., `fp(args)`), the pointer must first be loaded into a register, then BL branches to the register's value. If the function pointer expression itself contains a function call (FNX complexity), the pointer is evaluated first and saved before pushing arguments.

`<cgenrel() OFUNC case, if l not addressable 260d>≡ (257)`
`if(l->addable < INDEXED) {`
`reglcgen(&nod, l, Z);`
`gopcode(OFUNC, Z, Z, &nod);`
`regfree(&nod);`
`}`

Uses OFUNC 117c and Z 29a.

`<cgenrel() when OFUNC, if complex indirect function call 260e>≡ (257)`
`if(l->complex >= FNX) {`
`if(l->op != OIND)`
`diag(n, "bad function call");`

`regret(&nod, l->left);`
`cgen(l->left, &nod);`
`regsalloc(&nod1, l->left);`
`gopcode(OAS, &nod, Z, &nod1);`
`regfree(&nod);`

`nod = *n;`
`nod.left = &nod2;`
`nod2 = *l;`

```

    nod2.left = &nod1;
    nod2.complex = 1;
    cgen(&nod, nn);

    return;
}

```

Uses OAS 116a, OIND 116e, Z 29a, and diag() 496b.

10.12.4 Complex calls

<cgenrel() if all complex fields more than FNX 261a>≡ (223c)

```

if(n->complex >= FNX)
    if(l->complex >= FNX)
        if(r != Z && r->complex >= FNX)
            switch(o) {
                case OFUNC:
                case OCOMMA:
                case OANDAND:
                case OOROR:
                case OCOND:
                case ODOT:
                    break;

                default:
                    regret(&nod, r);
                    cgen(r, &nod);

                    regsalloc(&nod1, r);
                    gopcode(OAS, &nod, Z, &nod1);

                    regfree(&nod);
                    nod = *n;
                    nod.right = &nod1;
                    cgen(&nod, nn);
                    return;
            }
}

```

Uses OANDAND 115c, OAS 116a, OCOMMA 110g, OCOND 118f, ODOT 117a, OFUNC 117c, OOROR 115c, and Z 29a.

<garg() when f == 0, if n is complex part1 261b>≡ (259a)

```

if(n->complex >= FNX) {
    regsalloc(*fnxp, n);
    nod = znode;
    nod.op = OAS;
    nod.left = *fnxp;
    nod.right = n;
    nod.type = n->type;
    cgen(&nod, Z);
    (*fnxp)++;
}

```

Uses OAS 116a and Z 29a.

<garg() when f != 0, if n is complex part2 261c>≡ (259a)

```

if(n->complex >= FNX) {
    cgen(*fnxp, tn1);
    (*fnxp)++;
}

```

10.13 Initializers

Initializers for global and static variables are emitted at compile time as **ADATA** pseudo-instructions (data constants in the object file), while local initializers become runtime assignments. `init1` walks the initializer list recursively, matching each value against the target type: scalars emit a single datum, arrays and structs recurse into their elements, and `doinit` handles the outer bookkeeping including designated initializers.

```
<global nodcast 262a>≡ (511d)
Node* nodcast;
```

```
<expression nodes 262b>+≡ (27) <119i 429>
OASI, // appears during parsing
```

```
<doinit() sanity check initlist after init1 262c>≡ (120c)
if(initlist != Z)
    diag(initlist, "more initializers than structure: %s",
        s->name);
```

Uses Z 29a, `diag()` 496b, and `initlist` 120b.

```
<function init1 262d>≡ (525)
/// yyparse -> xdlist -> doinit -> <>
Node*
init1(Sym *s, Type *t, long o, bool exflag)
{
    Node *a, *l, *r, nod;
    Type *t1;
    long e, w, so, mw;

    a = peekinit();
    if(a == Z)
        return Z;

    <init1() debug initialization 494b>

    if(exflag && a->op == OINIT)
        return doinit(s, t, o, nextinit());

    // else
    switch(t->etype) {
    case TCHAR:
    case TUCCHAR:
    case TINT:
    case TUINT:
    case TSHORT:
    case TUSHORT:
    case TLONG:
    case TULONG:
    case TVLONG:
    case TUVLONG:
    case TFLOAT:
    case TDOUBLE:

    case TIND:
    single:
        if(a->op == OARRAY || a->op == OELEM)
            return Z;

        a = nextinit();
        if(a == Z)
            return Z;
```

```

<init1() sanity check t has no bitfields 419h>
if(s->class == CAUTO) {
    l = new(ONAME, Z, Z);
    l->sym = s;
    l->type = t;
    l->etype = s->type ? s->type->etype : TVOID;
    l->xoffset = s->offset + o;
    l->class = s->class;

    l = new(OASI, l, a);
    return l;
}

complex(a);
if(a->type == T)
    return Z;

if(a->op == OCONST) {
    if(vconst(a) && t->etype == TIND && a->type && a->type->etype != TIND){
        diag(a, "initialize pointer to an integer: %s", s->name);
        return Z;
    }
    if(!sametype(a->type, t)) {
        /* hoop jumping to save malloc */
        if(nodcast == Z)
            nodcast = new(OCAST, Z, Z);
        nod = *nodcast;
        nod.left = a;
        nod.type = t;
        nod.lineno = a->lineno;
        complex(&nod);
        if(nod.type)
            *a = nod;
    }
    if(a->op != OCONST) {
        diag(a, "initializer is not a constant: %s",
            s->name);
        return Z;
    }
    if(vconst(a) == 0)
        return Z;
    goto gext;
}

if(t->etype == TIND) {
    while(a->op == OCAST) {
        warn(a, "CAST in initialization ignored");
        a = a->left;
    }
    if(!sametype(t, a->type)) {
        diag(a, "initialization of incompatible pointers: %s\n%T and %T",
            s->name, t, a->type);
    }
    if(a->op == OADDR)
        a = a->left;
    goto gext;
}

while(a->op == OCAST)
    a = a->left;

```



```

    if(a->op == OADDR) {
        warn(a, "initialize pointer to an integer: %s", s->name);
        a = a->left;
        goto gext;
    }
    diag(a, "initializer is not a constant: %s", s->name);
    return Z;

gext:
    gextern(s, a, o, t->width);

    return Z;

case TARRAY:
    w = t->link->width;
    if(a->op == OSTRING || a->op == OLSTRING)
    if(typei[t->link->etype]) {
        /*
         * get rid of null if sizes match exactly
         */
        a = nextinit();
        mw = t->width/w;
        so = a->type->width/a->type->link->width;
        if(mw && so > mw) {
            if(so != mw+1)
                diag(a, "string initialization larger than array");
            a->type->width -= a->type->link->width;
        }

        /*
         * arrange strings to be expanded
         * inside OINIT braces.
         */
        a = new(OUSED, a, Z);
        return doinit(s, t, o, a);
    }

    mw = -w;
    l = Z;
    for(e=0;;) {
        /*
         * peek ahead for element initializer
         */
        a = peekinit();
        if(a == Z)
            break;
        if(a->op == OELEM && t->link->etype != TSTRUCT)
            break;
        if(a->op == OARRAY) {
            if(e && exflag)
                break;
            a = nextinit();
            r = a->left;
            complex(r);
            if(r->op != OCONST) {
                diag(r, "initializer subscript must be constant");
                return Z;
            }
            e = r->vconst;
            if(t->width != 0)

```

```

        if(e < 0 || e*w >= t->width) {
            diag(a, "initialization index out of range: %ld", e);
            continue;
        }
    }

    so = e*w;
    if(so > mw)
        mw = so;
    if(t->width != 0)
        if(mw >= t->width)
            break;
    r = init1(s, t->link, o+so, true);
    l = newlist(l, r);
    e++;
}
if(t->width == 0)
    t->width = mw+w;
return l;

case TUNION:
case TSTRUCT:
    /*
     * peek ahead to find type of rhs.
     * if its a structure, then treat
     * this element as a variable
     * rather than an aggregate.
     */
    if(isstruct(a, t))
        goto single;

    if(t->width <= 0) {
        diag(Z, "incomplete structure: %s", s->name);
        return Z;
    }
    l = Z;

again:
    for(t1 = t->link; t1 != T; t1 = t1->down) {
        if(a->op == OARRAY && t1->etype != TARRAY)
            break;
        if(a->op == OELEM) {
            if(t1->sym != a->sym)
                continue;
            nextinit();
        }
        r = init1(s, t1, o+t1->offset, true);
        l = newlist(l, r);
        a = peekinit();
        if(a == Z)
            break;
        if(a->op == OELEM)
            goto again;
    }
    if(a && a->op == OELEM)
        diag(a, "structure element not found %F", a);
    return l;

default:
    diag(Z, "unknown type in initialization: %T to: %s", t, s->name);

```

```

    return Z;
}
}

```

Uses CAUTO 33b, OADDR 116e, OARRAY 119i, OASI 262b, OCAST 118a, OCONST 111a, OELEM 119h, OINIT 119e, OLSTRING 113a, ONAME 27, OSTRING 112c, OUSED 184c, T 31i, TARRAY 31c, TCHAR 30b, TDOUBLE 31a, TFLOAT 31a, TIND 31c, TINT 30b, TLONG 30b, TSHORT 30b, TSTRUCT 31c, Tuchar 30b, TUINT 30b, TULONG 30b, TUNION 31c, TUSHORT 30b, TUVLONG 30b, TVLONG 30b, TVOID 31b, Z 29a, complex() 210a, diag() 496b, dinit() 120c, init1() 262d, isstruct() 266b, new() 29d, newlist() 266a, nextinit() 267b, nodcast 262a, peekinit() 267a, sametype() 145c, typei 148c, vconst() 176c, and warn() 142.

<function newlist 266a>≡ (525)

```

Node*
newlist(Node *l, Node *r)
{
    if(r == Z)
        return l;
    if(l == Z)
        return r;
    return new(OLIST, l, r);
}

```

Uses OLIST 29b, Z 29a, and new() 29d.

<function isstruct 266b>≡ (525)

```

bool
isstruct(Node *a, Type *t)
{
    Node *n;

    switch(a->op) {
    case ODOTDOT:
        n = a->left;
        if(n && n->type && sametype(n->type, t))
            return true;
    case OSTRING: case OLSTRING:
    case OCONST:
    case OINIT:
    case OELEM:
        return false;
    }

    n = new(ODOTDOT, Z, Z);
    *n = *a;

    /*
     * ODOTDOT is a flag for tcom
     * a second tcom will not be performed
     */
    a->op = ODOTDOT;
    a->left = n;
    a->right = Z;

    if(tcom(n))
        return false;

    if(sametype(n->type, t))
        return true;
    return false;
}

```

Uses OCONST 111a, ODOTDOT 137c, OELEM 119h, OINIT 119e, OLSTRING 113a, OSTRING 112c, Z 29a, new() 29d, sametype() 145c, and tcom() 152b.

<function peekinit 267a>≡ (525)

```
/*
 * get next major operator,
 * dont advance initlist.
 */
Node*
peekinit(void)
{
    Node *a;

    a = initlist;

loop:
    if(a == Z)
        return a;
    if(a->op == OLIST) {
        a = a->left;
        goto loop;
    }
    return a;
}
```

Uses OLIST 29b, Z 29a, and initlist 120b.

<function nextinit 267b>≡ (525)

```
/*
 * consume and return next element on
 * initlist. expand strings.
 */
Node*
nextinit(void)
{
    Node *a, *b, *n;

    a = initlist;
    n = Z;

    if(a == Z)
        return a;
    if(a->op == OLIST) {
        n = a->right;
        a = a->left;
    }
    if(a->op == OUSED) {
        a = a->left;
        b = new(OCONST, Z, Z);
        b->type = a->type->link;
        if(a->op == OSTRING) {
            b->vconst = convvtox(*a->cstring, TCHAR);
            a->cstring++;
        }
        if(a->op == OLSTRING) {
            b->vconst = convvtox(*a->rstring, TRUNE);
            a->rstring++;
        }
        a->type->width -= b->type->width;
        if(a->type->width <= 0)
            initlist = n;
        return b;
    }
    initlist = n;
}
```

```

    return a;
}

Uses OCONST 111a, OLIST 29b, OLSTRING 113a, OSTRING 112c, OUSED 184c, TCHAR 30b, TRUNE 113c, Z 29a, convvttox() 65d,
initlist 120b, and new() 29d.

⟨adlist rule, after doinit 268a⟩≡ (140b)
    $$ = contig($1->sym, $$, w);

⟨function contig 268b⟩≡ (525)
    //@Scheck: used by cc.y
    Node* contig(Sym *s, Node *n, long v)
    {
        Node *p, *r, *q, *m;
        long w;
        Type *zt;

        ⟨contig() debug initialization 494c⟩

        if(n == Z)
            goto no;
        w = s->type->width;

        /*
         * nightmare: an automatic array whose size
         * increases when it is initialized
         */
        if(v != w) {
            if(v != 0)
                diag(n, "automatic adjustable array: %s", s->name);
            v = s->offset;
            autoffset = align(autoffset, s->type, Aaut3);
            s->offset = -autoffset;
            stkoff = maxround(stkoff, autoffset);
            symadjust(s, n, v - s->offset);
        }
        if(w <= ewidth[TIND])
            goto no;
        if(n->op == OAS)
            diag(Z, "oops in contig");
        /*ZZZ this appears incorrect
        need to check if the list completely covers the data.
        if not, bail
        */
        if(n->op == OLIST)
            goto no;
        if(n->op == OASI)
            if(n->left->type)
                if(n->left->type->width == w)
                    goto no;
        while(w & (ewidth[TIND]-1))
            w++;
        /*
         * insert the following code, where long becomes vlong if pointers are fat
         */
        *(long**)&X = (long*)((char*)X + sizeof(X));
        do {
            *(long**)&X -= 1;
            **(long**)&X = 0;
        } while(*(long**)&X);
    }
    */

```

```

for(q=n; q->op != ONAME; q=q->left)
    ;

zt = ewidth[TIND] > ewidth[TLONG]? types[TVLONG]: types[TLONG];

p = new(ONAME, Z, Z);
*p = *q;
p->type = typ(TIND, zt);
p->xoffset = s->offset;

r = new(ONAME, Z, Z);
*r = *p;
r = new(OPOSTDEC, r, Z);

q = new(ONAME, Z, Z);
*q = *p;
q = new(OIND, q, Z);

m = new(OCONST, Z, Z);
m->vconst = 0;
m->type = zt;

q = new(OAS, q, m);

r = new(OLIST, r, q);

q = new(ONAME, Z, Z);
*q = *p;
r = new(ODWHILE, q, r);

q = new(ONAME, Z, Z);
*q = *p;
q->type = q->type->link;
q->xoffset += w;
q = new(OADDR, q, 0);

q = new(OASI, p, q);
r = new(OLIST, q, r);

n = new(OLIST, r, n);

no:
    return n;
}

```

Uses Aaut3 276b, OADDR 116e, OAS 116a, OASI 262b, OCONST 111a, ODWHILE 107c, OIND 116e, OLIST 29b, ONAME 27, OPOSTDEC 118j, TIND 31c, TLONG 30b, TVLONG 30b, Z 29a, autoffset 196d, diag() 496b, new() 29d, stkoff 196c, symadjust() 269, typ() 32a, and types 32b.

```

⟨function symadjust 269⟩≡ (525)
void
symadjust(Sym *s, Node *n, long del)
{

    switch(n->op) {
    case ONAME:
        if(n->sym == s)
            n->xoffset -= del;
        return;

```

```

case OCONST:
case OSTRING: case OLSTRING:
case OREGISTER: case OINDREG:
    return;

default:
    if(n->left)
        symadjust(s, n->left, del);
    if(n->right)
        symadjust(s, n->right, del);
    return;

}
}

```

Uses OCONST 111a, OINDREG 237c, OLSTRING 113a, ONAME 27, OREGISTER 208f, OSTRING 112c, and symadjust() 269.

10.14 Toplevel globals

At the end of compilation, `gclean` walks the symbol table and emits an `AGLOBL` pseudo-instruction for every global or static variable, declaring its name and size to the linker. The corresponding `ADATA` instructions (emitted earlier during initializer processing) fill in the actual values. This two-part scheme lets the compiler emit data incrementally as it encounters initializers, then declare all symbols in one pass at the end.

```

⟨gclean() locals 270a⟩ += (49c) <208d
    Sym *s;

```

```

⟨gclean() generate all AGLOBL pseudo opcodes 270b⟩ ≡ (49c)
    for(i=0; i<NHASH; i++)
        for(s = hash[i]; s != S; s = s->link) {
            ⟨gclean() when generate all AGLOBL, filter and sanity check symbol 270c⟩
            // else
            gpseudo(AGLOBL, s, nodconst(s->type->width));
        }

```

Uses NHASH 35b, S 35d, and hash 35a.

```

⟨gclean() when generate all AGLOBL, filter and sanity check symbol 270c⟩ ≡ (270b)
    if(s->type == T)
        continue;
    if(s->type->width == 0)
        continue;
    if(s->class != CGLOBL && s->class != CSTATIC)
        continue;
    if(s->type == types[TENUM])
        continue;

```

Uses CGLOBL 33b, CSTATIC 33b, T 31i, TENUM 31c, and types 32b.

10.14.1 gextern()

```

⟨function gextern(arm) 270d⟩ ≡ (540b)
    void
    gextern(Sym *s, Node *a, long o, long w)
    {
        ⟨gextern() if OCONST and a vlong type 273b⟩
        else {
            gpseudo(ADATA, s, a);
            p->from.offset += o;
        }
    }

```

```

        p->reg = w; // attributes
        if(p->to.type == D_OREG)
            p->to.type = D_CONST; // D_ADDR?
    }
}

```

10.15 Complex values

Values that do not fit in a single register—structs, unions, and 64-bit `vlongs`—require special handling throughout code generation. The `sugen` function (“structure generator”) handles these by copying them word-by-word via `MOVM` (block move) instructions. For function calls, complex arguments are pushed onto the stack, and complex return values are returned through a hidden pointer argument (`.ret`).

10.15.1 Complex types

“Complex types” in this context means types too large for a register: structs, unions, and 64-bit `vlong`. The `typecmplx` set (aliased to `typesuv`) identifies them, triggering the `sugen` code path instead of the normal register-based `cgen`.

```

⟨global typecmplx 271a⟩≡ (511d)
    // set<type_kind>
    char* typecmplx;

```

```

⟨tinit() initialise 32 bits defaults type sets 271b⟩+≡ (50a) <180a 337d>
    typecmplx = typesuv;

```

Uses `typecmplx 271a` and `typesuv 271c`.

```

⟨global typesuv 271c⟩≡ (518c)
    char typesuv[NTYPE];

```

Uses `NTYPE 30a`.

```

⟨global typesuvinit 271d⟩≡ (518c)
    int typesuvinit[] =
    {
        TVLONG, TUVLONG, TSTRUCT, TUNION, -1,
    };

```

Uses `TSTRUCT 31c`, `TUNION 31c`, `TUVLONG 30b`, and `TVLONG 30b`.

10.15.2 Complex expression

When `cgen` encounters an expression whose type is a struct, union, or `vlong`, it delegates to `sugen` which copies the value word-by-word rather than through a single register.

```

⟨cgenrel() if complex type node 271e⟩≡ (223c)
    if(typesuv[n->type->etype]) {
        sugen(n, nn, n->type->width);
    }

```

Uses `typesuv 271c`.

10.15.3 Complex argument

Complex arguments (struct/union/vlong) are pushed onto the stack via **sugen** rather than passed in a register. The **regalloc** call reserves the argument's stack slot.

```
<garg1() if complex argument type 272a>≡ (259a)
    if(typesuv[n->type->etype]) {
        regalloc(tn2, n);
        if(n->complex >= FNX) {
            sugen(*fnxp, tn2, n->type->width);
            (*fnxp)++;
        } else
            sugen(n, tn2, n->type->width);
    }
```

Uses typesuv 271c.

10.15.4 Complex return value

When a function returns a complex type (struct/union/vlong), the caller passes a hidden pointer argument in **REGARG** pointing to scratch space for the return value. The callee writes through this pointer (the **.ret** synthetic variable) instead of using **REGRET**.

```
<global nodret 272b>≡ (535b)
    Node* nodret;

<ginit() special nodes initialisation 272c>+≡ (51c) <260b 452e>
    nodret = new(ONAME, Z, Z);
    nodret->sym = slookup(".ret");
    nodret->type = types[TIND];
    nodret->etype = TIND;
    nodret->class = CPARAM;
    nodret = new(OIND, nodret, Z);
    complex(nodret);
```

Uses CPARAM 33b, OIND 116e, ONAME 27, TIND 31c, Z 29a, complex() 210a, new() 29d, slookup() 35f, and types 32b.

```
<codgen() if complex return type 272d>≡ (194c)
    if(typecplx[thisfntype->link->etype]) {
        n1 = nodret->left;
        if(n1->type == T || n1->type->link != thisfntype->link) {
            n1->type = typ(TIND, thisfntype->link);
            n1->etype = n1->type->etype; // TIND, dead instruction
            nodret = new(OIND, n1, Z); // useful? to force retyping?
            complex(nodret);
        }
    }
```

Uses OIND 116e, T 31i, TIND 31c, Z 29a, complex() 210a, new() 29d, thisfntype 105b, typ() 32a, and typecplx 271a.

```
<gen() case ORETURN, if complex type 272e>≡ (206b)
    if(typecplx[n->type->etype]) {
        nod = znode;
        nod.op = OAS;
        nod.left = nodret;
        nod.right = 1;
        nod.type = n->type;
        nod.complex = 1->complex;
        cgen(&nod, Z);
        noretval(1 | 2);
        gbranch(ORETURN);
        break;
    }
```

Uses OAS 116a, ORETURN 108b, Z 29a, and typecplx 271a.

```

⟨codegen() if use REGARG, if complex return type 273a⟩≡ (337b)
    if(typecplx[thisfntype->link->etype]) {
        nod1 = *nodret->left;
        nodreg(&nod, &nod1, REGARG);
        gmove(&nod, &nod1);
    }

```

Uses thisfntype 105b and typecplx 271a.

10.15.5 vlong constant

A 64-bit constant must be split into two 32-bit halves for the **ADATA** pseudo-instruction, with the order depending on endianness. On little-endian ARM, the low 32 bits come first.

```

⟨gextern() if OCONST and a vlong type 273b⟩≡ (270d)
    if(a->op == OCONST && typev[a->type->etype]) {
        if(align(0, types[TCHAR], Aarg1)) /* isbigendian */
            gpseudo(ADATA, s, nod32const(a->vconst>>32));
        else
            gpseudo(ADATA, s, nod32const(a->vconst));
        p->from.offset += o;
        p->reg = 4;
        if(align(0, types[TCHAR], Aarg1)) /* isbigendian */
            gpseudo(ADATA, s, nod32const(a->vconst));
        else
            gpseudo(ADATA, s, nod32const(a->vconst>>32));
        p->from.offset += o + 4;
        p->reg = 4;
    }

```

Uses Aarg1 276b, OCONST 111a, TCHAR 30b, nod32const() 273c, types 32b, and typev 280a.

```

⟨function nod32const(arm) 273c⟩≡ (537b)
    Node*
    nod32const(vlong v)
    {
        constnode.vconst = v & MASK(32);
        return &constnode;
    }

```

Uses MASK 501f.

10.16 Other topics

10.16.1 Sizes

The **ewidth** array maps each type kind to its size in bytes. This is the machine-dependent table that makes **sizeof** work: the front end looks up **ewidth[t->etype]** and also uses it to scale pointer arithmetic (e.g., **p + 1** becomes **p + ewidth[*p]**). Composite types (struct, union, array) have **-1** here because their sizes depend on layout and are computed by **sualign** instead.

```

⟨global ewidth 273d⟩≡ (537b)
    schar ewidth[NTYPE] =
    {
        [TXXX] = -1,
        [TCHAR] = SZ_CHAR,

        [TCHAR] = SZ_CHAR,
        [TSHORT] = SZ_SHORT,
    }

```

```

[TUSHORT] = SZ_SHORT,
[TINT] = SZ_INT,
[TUINT] = SZ_INT,
[TLONG] = SZ_LONG,
[TULONG] = SZ_LONG,
[TVLONG] = SZ_VLONG,
[TUVLONG] = SZ_VLONG,
[TFLOAT] = SZ_FLOAT,
[TDOUBLE] = SZ_DOUBLE,

```

```

[TIND] = SZ_IND,
[TFUNC] = 0,
[TARRAY] = -1,
[TVOID] = 0,
[TSTRUCT] = -1,
[TUNION] = -1,
[TENUM] = SZ_INT,

```

```
};
```

Uses NTYPE 30a.

```

⟨constant SZ_CHAR(arm) 274a⟩≡ (531b)
#define SZ_CHAR 1

```

```

⟨constant SZ_SHORT(arm) 274b⟩≡ (531b)
#define SZ_SHORT 2

```

```

⟨constant SZ_INT(arm) 274c⟩≡ (531b)
#define SZ_INT 4

```

```

⟨constant SZ_LONG(arm) 274d⟩≡ (531b)
#define SZ_LONG 4

```

```

⟨constant SZ_IND(arm) 274e⟩≡ (531b)
#define SZ_IND 4

```

```

⟨constant SZ_FLOAT(arm) 274f⟩≡ (531b)
#define SZ_FLOAT 4

```

```

⟨constant SZ_VLONG(arm) 274g⟩≡ (531b)
#define SZ_VLONG 8

```

```

⟨constant SZ_DOUBLE(arm) 274h⟩≡ (531b)
#define SZ_DOUBLE 8

```

10.16.2 First argument

```

⟨adec1() if first parameter 274i⟩≡ (225a)
if(autooffset == 0) {
    firstarg = s;
    firstargtype = t;
}

```

Uses autooffset 196d, firstarg 274j, and firstargtype 275a.

```

⟨global firstarg 274j⟩≡ (511d)
Sym* firstarg;

```

```

⟨xdecl rule, initializations before processing a function 274k⟩≡ (105c)
firstarg = S; // can mv after call to dodecl? No I think.

```

```

⟨global firstargtype 275a⟩≡ (511d)
    Type* firstargtype;

```

10.16.3 Alignment

`align` is a multi-purpose function dispatched by an `op` code: it handles struct element alignment (`Ael1/Ael2`), end-of-struct padding (`Asu2`), function parameter layout (`Aarg1/Aarg2`), and auto variable allocation (`Aaut3`). On ARM, the natural alignment is `SZ_LONG` (4 bytes); smaller types align to their own width. The `packflg` pragma can override this for packed structs.

```

⟨function align(arm) 275b⟩≡ (540b)
    /// ??? -> <>
    long
    align(long i, Type *t, int op)
    {
        long o;
        Type *v;
        int w;

        o = i;
        w = 1;
        switch(op) {
        default:
            diag(Z, "unknown align opcode %d", op);
            break;

        case Asu2: /* padding at end of a struct */
            w = SZ_LONG;
            ⟨align() if packflg override w 469e⟩
            break;

        case Ael1: /* initial align of struct element */
            for(v=t; v->etype==TARRAY; v=v->link)
                ;
            w = ewidth[v->etype];
            if(w <= 0 || w >= SZ_LONG)
                w = SZ_LONG;
            ⟨align() if packflg override w 469e⟩
            break;

        case Ael2: /* width of a struct element */
            o += t->width;
            break;

        case Aarg0: /* initial passbyptr argument in arg list */
            if(typesuv[t->etype]) {
                o = align(o, types[TIND], Aarg1);
                o = align(o, types[TIND], Aarg2);
            }
            break;

        case Aarg1: /* initial align of parameter */
            w = ewidth[t->etype];
            if(w <= 0 || w >= SZ_LONG) {
                w = SZ_LONG;
                break;
            }
            w = 1; /* little endian no adjustment */
            break;
    }

```

```

case Aarg2: /* width of a parameter */
    o += t->width;
    w = SZ_LONG;
    break;

case Aaut3: /* total align of automatic */
    o = align(o, t, Ael2);
    o = align(o, t, Ael1);
    w = SZ_LONG; /* because of a pun in cc/dcl.c:contig() */
    break;
}
o = round(o, w);
if(debug['A'])
    print("align %s %ld %T = %ld\n", bnames[op], i, t, o);
return o;
}

```

Uses Aarg0 276b, Aarg1 276b, Aarg2 276b, Aaut3 276b, Ael1 276b, Ael2 276b, Asu2 276b, TARRAY 31c, TIND 31c, Z 29a, bnames 276c, debug 474a, diag() 496b, round() 502a, types 32b, and typesuv 271c.

```

⟨function maxround 276a⟩≡ (540b)
long
maxround(long max, long v)
{
    v = round(v, SZ_LONG);
    if(v > max)
        return v;
    return max;
}

```

Uses round() 502a.

```

⟨enum align 276b⟩≡ (504b)
enum align
{
    Axxx,

    Ael1,
    Ael2,
    Asu2,
    Aarg0,
    Aarg1,
    Aarg2,
    Aaut3,

    NALIGN,
};

```

```

⟨global bnames 276c⟩≡ (518c)
char* bnames[NALIGN];

```

Uses NALIGN 276b.

```

⟨global bnamesinit 276d⟩≡ (518c)
Init bnamesinit[] =
{
    Axxx, 0, "Axxx",

    Ael1, 0, "el1",
    Ael2, 0, "el2",
    Asu2, 0, "su2",

```

```

    Aarg0, 0, "arg0",
    Aarg1, 0, "arg1",
    Aarg2, 0, "arg2",
    Aaut3, 0, "aut3",
    -1, 0, 0,
};

```

Uses Aarg0 276b, Aarg1 276b, Aarg2 276b, Aaut3 276b, Ael1 276b, Ael2 276b, Asu2 276b, and Axxx 276b.

<Type code generation fields 277a>+≡ (31d) <34b 277c>
 long offset;

<complex rule, when parse a structure definition, align the struct 277b>≡ (130 129)
 salign(\$\$);

<Type code generation fields 277c>+≡ (31d) <277a
 schar shift;
 char nbits;

<function salign 277d>≡ (525)

```

//@Scheck: not dead, used by cc.y
void salign(Type *t)
{
    Type *l;
    long o, w;

    o = 0;
    switch(t->etype) {

case TSTRUCT:
    t->offset = 0;
    w = 0;
    for(l = t->link; l != T; l = l->down) {
        if(l->nbits) {
            if(l->shift <= 0) {
                l->shift = -l->shift;
                w = round(w, tfield->width);
                o = w;
                w += tfield->width;
            }
            l->offset = o;
        } else {
            if(l->width < 0 ||
               l->width == 0 && l->down != T)
                if(l->sym)
                    diag(Z, "incomplete structure element: %s",
                        l->sym->name);
                else
                    diag(Z, "incomplete structure element");
            w = align(w, l, Ael1);
            l->offset = w;
            w = align(w, l, Ael2);
        }
    }
    w = align(w, t, Asu2);
    t->width = w;
    acidtype(t);
    pickletype(t);
    return;

case TUNION:

```

```

t->offset = 0;
w = 0;
for(l = t->link; l != T; l = l->down) {
    if(l->width <= 0)
        if(l->sym)
            diag(Z, "incomplete union element: %s",
                l->sym->name);
        else
            diag(Z, "incomplete union element");
    l->offset = 0;
    l->shift = 0;
    o = align(align(0, l, Ael1), l, Ael2);
    if(o > w)
        w = o;
}
w = align(w, t, Asu2);
t->width = w;
acidtype(t);
pickletype(t);
return;

default:
    diag(Z, "unknown type in sualign: %T", t);
    break;
}
}

```

Uses Ael1 [276b](#), Ael2 [276b](#), Asu2 [276b](#), T [31i](#), TSTRUCT [31c](#), TUNION [31c](#), Z [29a](#), acidtype() [403b](#), diag() [496b](#), pickletype() [410](#), round() [502a](#), and tfield [419b](#).

10.17 Advanced assembly generation

This section covers code generation for constructs that go beyond simple expressions and control flow: switch statements, 64-bit arithmetic on a 32-bit machine, floating point, and bitfield access. Each requires specialized instruction sequences that do not fit the uniform `cgen/gopcode` pattern.

10.17.1 Switch

Switch compilation first generates the body (collecting case labels into a `Case` list), then emits the dispatch code at the branch point saved by the initial `gbranch`. The dispatch can be a binary search (`swit1`) for sparse cases or a jump table (`swit2`) for dense ranges, chosen by heuristic.

The choice of dispatch strategy is what makes `switch` faster than a chain of `ifs`. The heuristic in `swit2` is concrete and worth understanding:

- Fewer than 5 cases: emit a sequence of compares (`CMP $val,reg; B.EQ label` for each case). Linear search wins because the constant overhead of a jump table dwarfs 4 compares.
- 3 cases or more and the value range (`max - min`) is less than $2 \times nc$ (i.e., the cases are dense): emit a direct jump table. The runtime `CASE` pseudo-instruction subtracts `min`, range-checks against `max-min`, then indexes into a table of `BCASE` branches—one per integer in the range, with holes pointing at `default`.
- Otherwise: recursive binary search. Pick the median case, compare against it, branch to the matching label on equal, and recursively dispatch on the lower or upper half. This is $O(\log n)$ compares.

A worked example. For the C source

```

switch(x) {
case 1: return "a";
case 2: return "b";
case 3: return "c";
case 4: return "d";
case 100: return "x";
default: return "?";
}

```

Sorted: {1,2,3,4,100}, range = 99, $nc = 5$. $99 \not< 10$, so the dense-table heuristic fails. With $nc = 5$ the linear branch fails too, so `swit2` falls through to binary search and recurses on {1,2} and {4,100} around the median 3. Now drop the outlier and try `switch(x) { case 1..4: ... default: ... }`. The range is 3, $nc = 4$, so $3 < 8$ and we get a direct table:

```

SUB    $1, R(x)          ; rebase
CASE   $3, R(x)          ; range check (default if x>3)
BCASE  case_1
BCASE  case_2
BCASE  case_3
BCASE  case_4

```

The linker (LINKER book [Pad15c]) collapses this idiom into two instructions on most architectures: a bound-check and a load-branch through a PC-relative table. Constant time, regardless of nc .

```

⟨gen() locals 279a⟩+≡ (197e) ◁206a
    Case *cn;

```

```

⟨gen() switch node kind cases 279b⟩+≡ (197e) ◁223a 280c▷
    case OSWITCH:
        l = n->left;
        complex(l);
        if(l->type == T)
            break;
        ⟨gen() when OSWITCH, typecheck condition 179c⟩

        gbranch(OGOTO); /* entry */
        sp = p;

        cn = cases;
        cases = C;
        casf();

        sbc = breakpc;
        breakpc = pc;
        snbreak = nbreak;
        nbreak = 0;

        gbranch(OGOTO);
        spb = p;

        gen(n->right); /* body */
        if(canreach){
            gbranch(OGOTO);
            patch(p, breakpc);

            nbreak++;
        }

```



```

patch(sp, pc);
regalloc(&nod, l, Z);
/* always signed */
if(typev[l->type->etype])
    nod.type = types[TVLONG];
else
    nod.type = types[TLONG];
cgen(l, &nod);
doswit(&nod);
regfree(&nod);
patch(sp, pc);

cases = cn;
breakpc = sbc;
canreach = nbreak!=0;
if(canreach == false)
    warnreach = !suppress;
nbreak = snbreak;
break;

```

Uses OGOTO 108e, OSWITCH 109b, T 31i, TLONG 30b, TVLONG 30b, Z 29a, canreach 185d, casf() 283a, complex() 210a, doswit() 281h, gen() 197e, types 32b, typev 280a, and warnreach 185e.

<global typev 280a>≡ (518c)
char typev[NTYPE];

Uses NTYPE 30a.

<global typevinit 280b>≡ (518c)
int typevinit[] =
{
 TVLONG, TUVLONG, -1,
};

Uses TUVLONG 30b and TVLONG 30b.

<gen() switch node kind cases 280c>+≡ (197e) <279b
case OCASE:
 canreach = true;
 l = n->left;
 if(cases == C)
 diag(n, "case/default outside a switch");
 if(l == Z) {
 casf();
 cases->val = 0;
 cases->def = 1;
 cases->label = pc;
 cases->isv = 0;
 goto rloop;
 }
 complex(l);
 if(l->type == T)
 goto rloop;
 if(l->op != OCONST || !typeswitch[l->type->etype]) {
 diag(n, "case expression must be integer constant");
 goto rloop;
 }
 casf();
 cases->val = l->vconst;
 cases->def = 0;
 cases->label = pc;

```

    cases->isv = typev[l->type->etype];
    goto rloop;
Uses OCASE 109b, OCONST 111a, T 31i, Z 29a, canreach 185d, casf() 283a, complex() 210a, diag() 496b, typeswitch 179d,
    and typev 280a.

⟨struct Case 281a⟩≡ (531b)
    struct Case
    {
        vlong val;
        long label;
        char def;
        char isv;

        Case* link;
    };

⟨constant C 281b⟩≡ (531b)
    #define C ((Case*)nil)

⟨struct C1 281c⟩≡ (531b)
    struct C1
    {
        vlong val;
        long label;
    };

⟨global cases 281d⟩≡ (535b) 281f▷
    Case* cases;

⟨ginit() misc initialisations 281e⟩+≡ (51c) ◁243a 419d▷
    cases = C;

⟨global cases 281f⟩+≡ (535b) ◁281d
    Case* cases;

⟨function swcmp 281g⟩≡ (530a)
    int
    swcmp(const void *a1, const void *a2)
    {
        C1 *p1, *p2;

        p1 = (C1*)a1;
        p2 = (C1*)a2;
        if(p1->val < p2->val)
            return -1;
        return p1->val > p2->val;
    }

⟨function doswit 281h⟩≡ (530a)
    void
    doswit(Node *n)
    {
        Case *c;
        C1 *q, *iq, *iqh, *iql;
        long def, nc, i, j, isv, nh;
        Prog *hsb;
        Node *vr[2];
        int dup;

        def = 0;

```

```

nc = 0;
isv = 0;
for(c = cases; c->link != C; c = c->link) {
    if(c->def) {
        if(def)
            diag(n, "more than one default in switch");
        def = c->label;
        continue;
    }
    isv |= c->isv;
    nc++;
}
if(typev[n->type->etype])
    isv = 1;
else if(isv){
    warn(n, "32-bit switch expression with 64-bit case constant");
    isv = 0;
}

iq = alloc(nc*sizeof(C1));
q = iq;
for(c = cases; c->link != C; c = c->link) {
    if(c->def)
        continue;
    if(c->isv && !isv)
        continue; /* can never match */
    q->label = c->label;
    if(isv)
        q->val = c->val;
    else
        q->val = (long)c->val; /* cast ensures correct value for 32-bit switch on 64-bit architecture */
    q++;
}
qsort(iq, nc, sizeof(C1), swcmp);

if(debug['K'])
    for(i=0; i<nc; i++)
        print("case %2ld: = %.8llx\n", i, (vlong)iq[i].val);

dup = 0;
for(i=0; i<nc-1; i++)
    if(iq[i].val == iq[i+1].val) {
        diag(n, "duplicate cases in switch %lld", (vlong)iq[i].val);
        dup = 1;
    }
if(dup)
    return;
if(def == 0) {
    def = breakpc;
    nbreak++;
}
if(!isv || ewidth[TIND] > ewidth[TLONG] || n->op == OREGISTER) {
    swit1(iq, nc, def, n);
    return;
}

/*
 * 64-bit case on 32-bit machine:
 * switch on high-order words, and
 * in each of those, switch on low-order words

```

```

    */
    if(n->op != OREGPAIR)
        fatal(n, "internal: expected register pair");
    if(thechar == '8'){ /* TO DO: need an enquiry function */
        vr[0] = n->left; /* low */
        vr[1] = n->right; /* high */
    }else{
        vr[0] = n->right;
        vr[1] = n->left;
    }
    vr[0]->type = types[TLONG];
    vr[1]->type = types[TLONG];

    gbranch(OGOTO);
    hsb = p;

    iqh = alloc(nc*sizeof(C1));
    iql = alloc(nc*sizeof(C1));
    nh = 0;
    for(i=0; i<nc;){
        iqh[nh].val = iq[i].val >> 32;
        q = iql;
        /* iq is sorted, so equal top halves are adjacent */
        for(j = i; j < nc; j++){
            if((iq[j].val>>32) != iqh[nh].val)
                break;
            q->val = (long)iq[j].val;
            q->label = iq[j].label;
            q++;
        }
        qsort(iql, q-iql, sizeof(C1), swcmp);
        //if(0){for(int k=0; k<(q-iql); k++)print("nh=%ld k=%d h=%#llx l=%#llx lab=%ld\n", nh, k, (vl
        iqh[nh].label = pc;
        nh++;
        swit1(iql, q-iql, def, vr[0]);
        i = j;
    }
    patch(hsb, pc);
    //if(0){for(int k=0; k<nh; k++)print("k*=%d h=%#llx lab=%ld\n", k, (vlong)iqh[k].val, iqh[k].label);}
    swit1(iqh, nh, def, vr[1]);
}

```

Uses OGOTO 108e, OREGISTER 208f, OREGPAIR, TIND 31c, TLONG 30b, alloc() 500b, debug 474a, diag() 496b, fatal() 496c, swcmp() 281g, thechar 45a, types 32b, typev 280a, and warn() 142.

```

⟨function casf 283a⟩≡ (530a)
void
casf(void)
{
    Case *c;

    c = alloc(sizeof(*c));
    c->link = cases;
    cases = c;
}

```

Uses alloc() 500b.

```

⟨function swit1(arm) 283b⟩≡ (540b)
void
swit1(C1 *q, int nc, long def, Node *n)
{

```

```

Node tn;

regalloc(&tn, &regnode, Z);
swit2(q, nc, def, n, &tn);
regfree(&tn);
}

```

Uses Z 29a and swit2() 284.

```

⟨function swit2(arm) 284⟩≡ (540b)
void
swit2(C1 *q, int nc, long def, Node *n, Node *tn)
{
    C1 *r;
    int i;
    long v;
    Prog *sp;

    if(nc >= 3) {
        i = (q+nc-1)->val - (q+0)->val;
        if(i > 0 && i < nc*2)
            goto direct;
    }
    if(nc < 5) {
        for(i=0; i<nc; i++) {
            if(debug['K'])
                print("case = %.8llx\n", q->val);
            gopcode(OEQ, nodconst(q->val), n, Z);
            patch(p, q->label);
            q++;
        }
        gbranch(OGOTO);
        patch(p, def);

        return;
    }

    i = nc / 2;
    r = q+i;
    if(debug['K'])
        print("case > %.8llx\n", r->val);
    gopcode(OGT, nodconst(r->val), n, Z);
    sp = p;
    gopcode(OEQ, nodconst(r->val), n, Z); /* just gen the B.EQ */
    patch(p, r->label);
    swit2(q, i, def, n, tn);

    if(debug['K'])
        print("case < %.8llx\n", r->val);
    patch(sp, pc);
    swit2(r+1, nc-i-1, def, n, tn);
    return;

direct:
    v = q->val;
    if(v != 0)
        gopcode(OSUB, nodconst(v), Z, n);
    gopcode(OCASE, nodconst((q+nc-1)->val - v), n, Z);
    patch(p, def);
    for(i=0; i<nc; i++) {
        if(debug['K'])

```

```

        print("case = %.8lux\n", q->val);
while(q->val != v) {
    nextpc();
    p->as = ABCASE;
    patch(p, def);
    v++;
}
nextpc();
p->as = ABCASE;
patch(p, q->label);
q++;
v++;
}

gbranch(OGOTO); /* so that regopt() won't be confused */
patch(p, def);
}

```

Uses OCASE 109b, OEQ 115g, OGOTO 108e, OGT 115g, OSUB 114c, Z 29a, debug 474a, and swit2() 284.

Chapter 11

Object File Generation

`outcode` serializes the linked list of `Prog` instructions into the same object file format used by the assembler (see ASSEMBLER book [Pad15a]). Symbol references are encoded using a small hash table (`Htab`) that maps symbols to compact indices, avoiding redundant symbol name output. The file history (`outhist`) records which source files contributed to each range of instructions, enabling the debugger to map object code back to C source lines.

11.1 Instruction-stream object files

Most modern object-file formats—ELF, Mach-O, COFF, PE—are section-oriented. The file is a header, a section table, and for each section a block of already-assembled machine code plus a relocation table that tells the linker how to patch addresses when the section is placed at a final load address. The assembler has already emitted final instruction encodings; the linker’s job is to concatenate sections, resolve symbols, and rewrite the relocations. Once assembly is done the instructions are opaque bytes from the linker’s point of view.

Plan 9’s `.5`, `.8`, `.6`, and `.v` object files take a different route. They are a serialized stream of `Prog` records—the same `Prog` instruction structure the compiler and assembler keep in memory—written out with a compact per-instruction symbol-index table (`Htab`) so that symbol names appear only once. Because the instructions are still in structured form, the linker can do work that section-oriented formats make difficult: a second pass of peephole optimization after symbol resolution, ARM-specific tricks like immediate rotation when the final constant becomes known, leaf-function optimization that needs to see all call sites, and in-place editing of individual instructions. This is why LINKER book [Pad15c] is unusually meaty for a Plan 9 tool—the linker is a second-stage optimizer that still sees and rewrites machine instructions, not just a loader that patches relocations.

The design is Ken Thompson’s, reused across the whole Plan 9 toolchain. LLVM bitcode (2003) is the closest modern analogue in spirit, except that LLVM stores high-level SSA IR rather than target-specific machine instructions, and bitcode’s purpose is to defer code generation to link time, whereas Plan 9’s purpose is to defer final optimization. ELF and Mach-O still dominate in the mainstream world because section-oriented linkers are easier to parallelize, easier to standardize across competing toolchains, and do not tie the linker to any one compiler’s internal representation. Plan 9 pays those costs because it owns the whole toolchain and Ken valued pass-composition over standardization.

11.2 Object format

The object file is a sequence of variable-length records: symbol name definitions (emitted by `zname`) and instruction records (emitted by `zwrite`). The format is compact and designed for fast one-pass reading by the linker—the same format described in the ASSEMBLER book [Pad15a].

11.3 Instruction output: outcode()

`outcode` walks the `Prog` list from `firstp` to the end. For each instruction, it checks whether the “from” and “to” symbols have already been emitted; if not, it calls `zname` to define them. A small hash table (`h`) of size `NSYM` maps recently emitted symbols to their indices, so the common case (reusing a symbol from the previous instruction) requires no re-emission. The `jackpot` label handles the rare collision case where both operands’ symbols compete for the same index.

```
⟨struct Htab 287a⟩≡ (540b)
struct Htab {
    Sym *sym;
    short type;
};
```

```
⟨Sym identifier value, code generation fields 287b⟩+≡ (36b) <197c 394b>
// index in h when the Sym is really a symbol, 0 when not a symbol
char symidx;
```

```
⟨function outcode(arm) 287c⟩≡ (540b)
/// main -> compile -> yyparse; gclean -> <>
void
outcode(void)
{
    struct Htab h[NSYM];
    Prog *p;
    Sym *s;
    int sf, st, t, sym;

    ⟨outcode() debug 492d⟩
    outhist(&outbuf);
    for(sym=0; sym<NSYM; sym++) {
        h[sym].sym = S;
        h[sym].type = 0;
    }
    sym = 1;
    for(p = firstp; p != P; p = p->link) {
jackpot:
        sf = 0;
        s = p->from.sym;
        while(s != S) {
            sf = s->symidx;
            if(sf < 0 || sf >= NSYM)
                sf = 0;
            t = p->from.symkind;
            if(h[sf].type == t)
            if(h[sf].sym == s)
                break;
            s->symidx = sym;
            zname(&outbuf, s, t);
            h[sym].sym = s;
            h[sym].type = t;
            sf = sym;
            sym++;
            if(sym >= NSYM)
                sym = 1;
            break;
        }
        st = 0;
        s = p->to.sym;
        while(s != S) {
```



```

        st = s->symidx;
        if(st < 0 || st >= NSYM)
            st = 0;
        t = p->to.symkind;
        if(h[st].type == t)
            if(h[st].sym == s)
                break;
        s->symidx = sym;
        zname(&outbuf, s, t);
        h[sym].sym = s;
        h[sym].type = t;
        st = sym;
        sym++;
        if(sym >= NSYM)
            sym = 1;
        if(st == sf)
            goto jackpot;
        break;
    }
    zwrite(&outbuf, p, sf, st);
}
firstp = P;
lastp = P;
}

```

Uses Htab 287a, S 35d, and zwrite() 288a.

```

⟨function zwrite(arm) 288a⟩≡ (540b)
void
zwrite(Biobuf *b, Prog *p, int sf, int st)
{
    char bf[100], *bp;

    bf[0] = p->as;
    bf[1] = p->scond;
    bf[2] = p->reg;
    bf[3] = p->lineno;
    bf[4] = p->lineno>>8;
    bf[5] = p->lineno>>16;
    bf[6] = p->lineno>>24;
    bp = zaddr(bf+7, &p->from, sf);
    bp = zaddr(bp, &p->to, st);
    Bwrite(b, bf, bp-bf);
}

```

11.4 Operand output: zaddr()

zaddr serializes an **Adr** operand into the binary object file format. It writes the addressing mode, register, symbol index, and offset/value as a variable-length byte sequence. Float constants are encoded in IEEE format via the **Ieee** conversion. The format is compact: each operand takes 2–10 bytes depending on the addressing mode.

```

⟨function zaddr(arm) 288b⟩≡ (540b)
char*
zaddr(char *bp, Adr *a, int s)
{
    long l;
    Ieee e;
}

```

```

bp[0] = a->type;
bp[1] = a->reg;
bp[2] = s;
bp[3] = a->symkind;
bp += 4;

//if(a->type == D_CONST && a->symkind != N_NONE) {
//    diag(Z, "missing D_CONST -> D_ADDR migration");
//}
//if(a->type == D_ADDR && a->symkind == N_NONE) {
//    diag(Z, "wrong D_CONST -> D_ADDR migration");
//}

switch(a->type) {
case D_NONE:
case D_REG:
case D_FREG:
case D_PSR:
    break;

case D_OREG:
case D_CONST:
case D_ADDR:
case D_BRANCH:
case D_SHIFT:
    l = a->offset;
    bp[0] = l;
    bp[1] = l>>8;
    bp[2] = l>>16;
    bp[3] = l>>24;
    bp += 4;
    break;

case D_SCONST:
    memmove(bp, a->sval, NSNAME);
    bp += NSNAME;
    break;

case D_FCONST:
    ieeeedtod(&e, a->dval);
    l = e.l;
    bp[0] = l;
    bp[1] = l>>8;
    bp[2] = l>>16;
    bp[3] = l>>24;
    bp += 4;
    l = e.h;
    bp[0] = l;
    bp[1] = l>>8;
    bp[2] = l>>16;
    bp[3] = l>>24;
    bp += 4;
    break;

default:
    diag(Z, "unknown type %d in zaddr", a->type);
}

return bp;
}

```

Uses Z 29a, diag() 496b, and ieeeedtod() 452g.

11.5 Symbol table, zname(), h and ANAME

zname emits an **ANAME** record to the object file, defining a symbol's name, type (extern/intern/auto/param), and index. The index is used by subsequent instructions to refer to the symbol compactly. If safe linking (5c-T) is enabled, **zname** also emits an **ASIGNAME** record with the type signature hash, allowing the linker to catch declaration mismatches.

```
<function zname(arm) 290a>≡ (540b)
void
zname(Biobuf *b, Sym *s, int t)
{
    char *n, bf[7];
    ulong sig;

    n = s->name;
    <zname() if generate signature for symbol s 395a>
    else{
        bf[0] = ANAME;
        bf[1] = t; /* type */
        bf[2] = s->symidx; /* sym */
        Bwrite(b, bf, 3);
    }
    Bwrite(b, n, strlen(n)+1);
}
```

11.6 File and line information: outhist()

outhist writes the file inclusion history into the object file as **AHISTORY** pseudo-instructions. Each entry records either a “push” (entering a new file via **#include**) with the filename, or a “pop” (returning to the parent file) with a nil name. The debugger uses this history to map global line numbers back to (filename, local-line) pairs. Directory path components are split into separate records for compact encoding.

```
<function outhist(arm) 290b>≡ (540b)
void
outhist(Biobuf *b)
{
    Hist *h;
    char *p, *q, *op, c;
    Prog pg;
    int n;

    pg = zprog;
    pg.as = AHISTORY;
    c = pathchar();
    for(h = hist; h != H; h = h->link) {
        p = h->name;
        op = 0;
        /* on windows skip drive specifier in pathname */
        //if(systemtype(Windows) && p && p[1] == ':'){
        // p += 2;
        // c = *p;
        //}
        if(p && p[0] != c && h->offset == 0 && pathname){
            /* on windows skip drive specifier in pathname */

```

```

    //if(systemtype(Windows) && pathname[1] == ':') {
    // op = p;
    // p = pathname+2;
    // c = *p;
    //} else
    if(pathname[0] == c){
        op = p;
        p = pathname;
    }
}
while(p) {
    q = utfrune(p, c);
    if(q) {
        n = q-p;
        if(n == 0){
            n = 1; /* leading "/" */
            *p = '/'; /* don't emit "\" on windows */
        }
        q++;
    } else {
        n = strlen(p);
        q = 0;
    }
    if(n) {
        Bputc(b, ANAME);
        Bputc(b, N_FILE);
        Bputc(b, 1);
        Bputc(b, '<');
        Bwrite(b, p, n);
        Bputc(b, '\\0');
    }
    p = q;
    if(p == 0 && op) {
        p = op;
        op = 0;
    }
}
pg.lineno = h->line;
pg.to.type = zprog.to.type;
pg.to.offset = h->offset;
if(h->offset)
    pg.to.type = D_CONST;

zwrite(b, &pg, 0, 0);
}
}

```

Uses H 80c, hist 80e, pathchar() 513b, pathname 51a, and zwrite() 288a.

Chapter 12

AST-level Optimizations

AST-level optimizations are architecture-independent transformations on the typed syntax tree, performed before code generation. The main workhorse is `ccom` (“compile common”), which recursively simplifies the tree: canceling inverse operations (`IND(ADDR x)` becomes `x`), eliminating identity operations (add/subtract zero, multiply by one), and evaluating constant expressions (`evconst`). These simplifications reduce the work for the code generator and often eliminate entire subtrees.

12.1 AST simplifications

12.1.1 General rewrites: `ccom()`

`ccom` performs a bottom-up pass over the AST, applying algebraic simplifications: `*(&x)` becomes `x`, `x + 0` or `x * 1` become `x`, `x * 0` becomes 0 (if no side effects), `x / 0` is a compile-time error, and when both operands are constants, `evconst` folds the expression. The function also canonicalizes commutative operations so that the more complex operand is on the left (to help `cgen`’s register allocation heuristic).

```
<function ccom 292>≡ (529)
  /// complex -> <>
  /*
    * general rewrite
    * (IND(ADDR x)) ==> x
    * (ADDR(IND x)) ==> x
    * remove some zero operands
    * remove no op casts
    * evaluate constants // important one!
    */
void
ccom(Node *n)
{
    Node *l, *r;
    int t;

loop:
    if(n == Z)
        return;

    l = n->left;
    r = n->right;
    switch(n->op) {
<ccom() switch node kind cases 190a>
    default:
        // recurse
        if(l != Z)
```

```

        ccom(l);
    if(r != Z)
        ccom(r);
    // Fallthrough
common:
    if(((l == Z) || (l->op == OCONST)) && ((r == Z) || (r->op == OCONST)))
        evconst(n);
}
}

```

Uses OCONST 111a, Z 29a, ccom() 292, and evconst() 304a.

```

⟨ccom() switch node kind cases 293a⟩+≡ (292) <190a 293b>
    case OCONST:
    case ONAME:
        break;

```

Uses OCONST 111a and ONAME 27.

```

⟨ccom() switch node kind cases 293b⟩+≡ (292) <293a 293c>
    case OREGISTER:
    case OINDREG:
        break;

```

Uses OINDREG 237c and OREGISTER 208f.

No op casts

A cast between types of the same size and signedness is a no-op: ccom removes it by replacing the OCAST node with its child. If the child is a constant, evconst folds the cast into the constant (e.g., (long)42 becomes a TLONG constant).

```

⟨ccom() switch node kind cases 293c⟩+≡ (292) <293b 293d>
    case OCAST:
        ccom(l);
        if(l->op == OCONST) {
            evconst(n);
            if(n->op == OCONST)
                break;
        }
        if(nocast(l->type, n->type) &&
            (!typefd[l->type->etype] ||
             typeu[l->type->etype] && typeu[n->type->etype]) ) {
            l->type = n->type;
            *n = *l;
        }
        break;

```

Uses OCAST 118a, OCONST 111a, ccom() 292, evconst() 304a, nocast() 249b, typefd 149c, and typeu 231h.

Pointers and addresses

Taking the address of a dereference (&(*p)) cancels out to just p, and dereferencing an address (*(&x)) cancels to just x. These inverse-operation eliminations are among the most effective simplifications, since the type checker introduces many implicit OADDR nodes (for array-to-pointer decay) that can often be immediately canceled.

```

⟨ccom() switch node kind cases 293d⟩+≡ (292) <293c 294a>
    case OADDR:
        ccom(l);
        l->etype = TVOID; // ???
        if(l->op == OIND) {

```

```

    l->left->type = n->type;
    *n = *l->left;
    break;
}
goto common;

```

Uses OADDR 116e, OIND 116e, TVOID 31b, and ccom() 292.

```

⟨ccom() switch node kind cases 294a⟩+≡ (292) <293d 294b>
case OIND:
    ccom(l);
    if(l->op == OADDR) {
        l->left->type = n->type;
        *n = *l->left;
        break;
    }
    goto common;

```

Uses OADDR 116e, OIND 116e, and ccom() 292.

Zero operands

Zero is the identity element for addition, OR, and XOR: $x + 0$, $x \mid 0$, and $x \wedge 0$ all simplify to x . Zero is also an annihilator for multiplication: $x * 0$ becomes 0. The `commute`: label additionally normalizes commutative operations to push constants toward the right side, which enables better code generation (ARM immediate operands). If both children of a commutative operator are themselves the same operator with a constant child, the constants are reassociated to enable further folding.

```

⟨ccom() switch node kind cases 294b⟩+≡ (292) <294a 294c>
case OADD:
case OOR:
case OXOR:
    ccom(l);
    if(vconst(l) == 0) {
        *n = *r;
        goto loop;
    }
    ccom(r);
    if(vconst(r) == 0) {
        *n = *l;
        break;
    }
    goto commute;

```

Uses OADD 114c, OOR 115a, OXOR 115a, ccom() 292, and vconst() 176c.

```

⟨ccom() switch node kind cases 294c⟩+≡ (292) <294b 295a>
commute:
    /* look for commutative constant */
    if(r->op == OCONST) {
        if(l->op == n->op) {
            if(l->left->op == OCONST) {
                n->right = l->right;
                l->right = r;
                goto loop;
            }
            if(l->right->op == OCONST) {
                n->right = l->left;
                l->left = r;
                goto loop;
            }
        }
    }

```

```

    }
}
if(l->op == OCONST) {
    if(r->op == n->op) {
        if(r->left->op == OCONST) {
            n->left = r->right;
            r->right = l;
            goto loop;
        }
        if(r->right->op == OCONST) {
            n->left = r->left;
            r->left = l;
            goto loop;
        }
    }
}
goto common;

```

Uses OCONST 111a.

```

⟨ccom() switch node kind cases 295a⟩+≡ (292) <294c 296a>
case OAND:
    ccom(l);
    ccom(r);
    if(vconst(l)== 0 && !side(r)) {
        *n = *l;
        break;
    }
    if(vconst(r) == 0 && !side(l)) {
        *n = *r;
        break;
    }
    // Fallthrough, goto commute

```

Uses OAND 115a, ccom() 292, side() 295b, and vconst() 176c.

```

⟨function side 295b⟩≡ (518c)
/// ccom -> <>
bool
side(Node *n)
{
loop:
    if(n != Z)
        switch(n->op) {
        case OCAST:
        case ONOT:
        case OADDR:
        case OIND:
            n = n->left;
            goto loop;

        case OCOND:
            if(side(n->left))
                break;
            n = n->right;

        case OEQ:
        case ONE:
        case OLT:
        case OGE:
        case OGT:

```



```

case OLE:
case OADD:
case OSUB:
case OMUL:
case OLMUL:
case ODIV:
case OLDIV:
case OLSHR:
case OASHL:
case OASHR:
case OAND:
case OOR:
case OXOR:
case OMOD:
case OLMOD:
case OANDAND:
case OOROR:
case OCOMMA:
case ODOT:
    if(side(n->left))
        break;
    n = n->right;
    goto loop;

case OSIGN:
case OSIZE:
case OCONST:
case OSTRING:
case OLSTRING:
case ONAME:
    return false;
}
// OFUNC/OAS/OASADD/...
return true;
}

```

Uses OADD 114c, OADDR 116e, OAND 115a, OANDAND 115c, OASHL 115a, OASHR 115a, OCAST 118a, OCOMMA 110g, OCOND 118f, OCONST 111a, ODIV 114c, ODOT 117a, OEQ 115g, OGE 115g, OGT 115g, OIND 116e, OLDIV 232d, OLE 115g, OLMOD 232f, OLMUL 232d, OLSHR 232b, OLSTRING 113a, OLT 115g, OMOD 114c, OMUL 114c, ONAME 27, ONE 115g, ONOT 115e, OOR 115a, OOROR 115c, OSIGN 442a, OSIZE 119a, OSTRING 112c, OSUB 114c, OXOR 115a, Z 29a, and side() 295b.

```

⟨ccom() switch node kind cases 296a⟩+≡ (292) <295a 296b>
case OASHR:
case OASHL:
case OLSHR:
    ccom(l);
    if(vconst(l) == 0 && !side(r)) {
        *n = *l;
        break;
    }
    ccom(r);
    if(vconst(r) == 0) {
        *n = *l;
        break;
    }
    ⟨ccom() when OASHR/OASHL/OLSHR, check for stupid shift 189d⟩
    goto common;

```

Uses OASHL 115a, OASHR 115a, OLSHR 232b, ccom() 292, side() 295b, and vconst() 176c.

```

⟨ccom() switch node kind cases 296b⟩+≡ (292) <296a 297a>
case OMUL:

```

```

case OLMUL:
    ccom(l);
    t = vconst(l);
    if(t == 0 && !side(r)) {
        *n = *l;
        break;
    }
    if(t == 1) {
        *n = *r;
        goto loop;
    }
    ccom(r);
    t = vconst(r);
    if(t == 0 && !side(l)) {
        *n = *r;
        break;
    }
    if(t == 1) {
        *n = *l;
        break;
    }
    goto common;

```

Uses OLMUL 232d, OMUL 114c, ccom() 292, side() 295b, and vconst() 176c.

```

⟨ccom() switch node kind cases 297a⟩+≡ (292) <296b 297b>
case ODIV:
case OLDIV:
    ccom(l);
    if(vconst(l) == 0 && !side(r)) {
        *n = *l;
        break;
    }
    ccom(r);
    t = vconst(r);
    if(t == 0) {
        diag(n, "divide check");
        *n = *r;
        break;
    }
    if(t == 1) {
        *n = *l;
        break;
    }
    goto common;

```

Uses ODIV 114c, OLDIV 232d, ccom() 292, diag() 496b, side() 295b, and vconst() 176c.

Sub to Add

Subtraction of a constant is rewritten as addition of the negated constant: $x - 3$ becomes $x + (-3)$. This canonicalization helps later optimizations that look for addition patterns (commutative reassociation, immediate operand fitting).

```

⟨ccom() switch node kind cases 297b⟩+≡ (292) <297a 298a>
case OSUB:
    ccom(r);
    if(r->op == OCONST) {
        if(typefd[r->type->etype]) {
            n->op = OADD;
            r->fconst = -r->fconst;

```

```

        goto loop;
    } else {
        n->op = OADD;
        r->vconst = -r->vconst;
        goto loop;
    }
}
ccom(1);
goto common;

```

Uses OADD 114c, OCONST 111a, OSUB 114c, ccom() 292, and typefd 149c.

Comparisons

The comparison simplifier `compar` handles several patterns: comparing a variable to itself (`x == x` is always true), comparing against zero (which ARM can do without loading the constant), and normalizing the constant to the right side. When both operands are constants, `evconst` folds the entire comparison. The unsigned comparison operators (OLS, OHS, OLO, OHI) are treated alongside their signed counterparts.

```

⟨ccom() switch node kind cases 298a⟩+≡ (292) <297b 303a>
    case OEQ:
    case ONE:

    case OLE:
    case OGE:
    case OLT:
    case OGT:

    case OLS:
    case OHS:
    case OLO:
    case OHI:
        ccom(1);
        ccom(r);
        if(compar(n, 0) || compar(n, 1))
            break;
        relcon(1, r);
        relcon(r, 1);
        goto common;

```

Uses OEQ 115g, OGE 115g, OGT 115g, OHI 300d, OHS 300d, OLE 115g, OLO 300d, OLS 300d, OLT 115g, ONE 115g, ccom() 292, compar() 300f, and relcon() 298b.

```

⟨function relcon 298b⟩≡ (518c)
/*
 * try to cast a constant down
 * rather than cast a variable up
 * example:
 * if(c == 'a')
 */
void
relcon(Node *l, Node *r)
{
    vlong v;

    if(l->op != OCONST)
        return;
    if(r->op != OCAST)
        return;
    if(!nilcast(r->left->type, r->type))

```

```

        return;
switch(r->type->etype) {
default:
    return;
case TCHAR:
case TUCCHAR:
case TSHORT:
case TUSHORT:
    v = convvtox(l->vconst, r->type->etype);
    if(v != l->vconst)
        return;
    break;
}
l->type = r->left->type;
*r = *r->left;
}

```

Uses OCAST 118a, OCONST 111a, TCHAR 30b, TSHORT 30b, TUCCHAR 30b, TUSHORT 30b, convvtox() 65d, and nilcast() 299a.

(function nilcast 299a)≡ (518c)

```

/*
 * a cast that has a noop semantic
 * (small to large, convert)
 */
bool
nilcast(Type *t1, Type *t2)
{
    int et1, et2;

    if(t1 == T)
        return false;
    if(t1->nbits)
        return false;
    if(t2 == T)
        return false;
    et1 = t1->etype;
    et2 = t2->etype;
    if(et1 == et2)
        return true;
    if(typefd[et1] && typefd[et2]) {
        if(ewidth[et1] < ewidth[et2])
            return true;
        return false;
    }
    if(typechlp[et1] && typechlp[et2]) {
        if(ewidth[et1] < ewidth[et2])
            return true;
        return false;
    }
    return false;
}

```

Uses T 31i, typechlp 215g, and typefd 149c.

(struct Big 299b)≡ (529)

```

/* 128-bit numbers */
struct Big
{
    vlong a;
    uulong b;
};

```

<function big 300a>≡ (529)

```
Big
big(vlong a, uulong b)
{
    Big x;

    x.a = a;
    x.b = b;
    return x;
}
```

<function cmp 300b>≡ (529)

```
static int
cmp(Big x, Big y)
{
    if(x.a != y.a){
        if(x.a < y.a)
            return -1;
        return 1;
    }
    if(x.b != y.b){
        if(x.b < y.b)
            return -1;
        return 1;
    }
    return 0;
}
```

<function add 300c>≡ (529)

```
static Big
add(Big x, int y)
{
    uulong ob;

    ob = x.b;
    x.b += y;
    if(y > 0 && x.b < ob)
        x.a++;
    if(y < 0 && x.b > ob)
        x.a--;
    return x;
}
```

<after parsing nodes 300d>+≡ (27) <237c ??>

```
OHI,
OHS,
OLO,
OLS,
```

<global cmps 300e>≡ (529)

```
/* OEQ, ONE, OLE, OLS, OLT, OLO, OGE, OHS, OGT, OHI */
static char *cmps[12] =
{
    "==", "!=", "<=", "<=", "<", "<", ">=", ">=", ">", ">",
};
```

<function compar 300f>≡ (529)

```
int
compar(Node *n, int reverse)
{
```

```

Big lo, hi, x;
int op;
char xbuf[40], cmpbuf[50];
Node *l, *r;
Type *lt, *rt;

/*
 * The point of this function is to diagnose comparisons
 * that can never be true or that look misleading because
 * of the 'usual arithmetic conversions'. As an example
 * of the latter, if x is a ulong, then if(x <= -1) really means
 * if(x <= 0xFFFFFFFF), while if(x <= -1LL) really means
 * what it says (but 8c compiles it wrong anyway).
 */

if(reverse){
    r = n->left;
    l = n->right;
    op = comrel[relindex(n->op)];
}else{
    l = n->left;
    r = n->right;
    op = n->op;
}

/*
 * Skip over left casts to find out the original expression range.
 */
while(l->op == OCAST)
    l = l->left;
if(l->op == OCONST)
    return 0;

lt = l->type;
if(l->op == ONAME && l->sym->type){
    lt = l->sym->type;
    if(lt->etype == TARRAY)
        lt = lt->link;
}
if(lt == T)
    return 0;
if(lt->etype == TXXX || lt->etype > TUVLONG)
    return 0;

/*
 * Skip over the right casts to find the on-screen value.
 */
if(r->op != OCONST)
    return 0;
while(r->oldop == OCAST && !r->xcast)
    r = r->left;
rt = r->type;
if(rt == T)
    return 0;

x.b = r->vconst;
x.a = 0;
if((rt->etype&1) && r->vconst < 0) /* signed negative */
    x.a = ~0ULL;

```

```

if((lt->etype&1)==0){
    /* unsigned */
    lo = big(0, 0);
    if(lt->width == 8)
        hi = big(0, ~0ULL);
    else
        hi = big(0, (1LL<<(lt->type->width*8))-1);
}else{
    lo = big(~0ULL, -(1LL<<(lt->type->width*8-1)));
    hi = big(0, (1LL<<(lt->type->width*8-1))-1);
}

switch(op){
case OLT:
case OLO:
case OGE:
case OHS:
    if(cmp(x, lo) <= 0)
        goto useless;
    if(cmp(x, add(hi, 1)) >= 0)
        goto useless;
    break;
case OLE:
case OLS:
case OGT:
case OHI:
    if(cmp(x, add(lo, -1)) <= 0)
        goto useless;
    if(cmp(x, hi) >= 0)
        goto useless;
    break;
case OEQ:
case ONE:
    /*
     * Don't warn about comparisons if the expression
     * is as wide as the value: the compiler-supplied casts
     * will make both outcomes possible.
     */
    if(lt->width >= rt->width && debug['w'] < 2)
        return 0;
    if(cmp(x, lo) < 0 || cmp(x, hi) > 0)
        goto useless;
    break;
}
return 0;

useless:
if((x.a==0 && x.b<=9) || (x.a==~0LL && x.b >= -9ULL))
    snprintf(xbuf, sizeof xbuf, "%lld", x.b);
else if(x.a == 0)
    snprintf(xbuf, sizeof xbuf, "%#llx", x.b);
else
    snprintf(xbuf, sizeof xbuf, "%#llx", x.b);
if(reverse)
    snprintf(cmpbuf, sizeof cmpbuf, "%s %s %T",
        xbuf, cmps[relindex(n->op)], lt);
else
    snprintf(cmpbuf, sizeof cmpbuf, "%T %s %s",
        lt, cmps[relindex(n->op)], xbuf);
if(debug['y'])

```

```

    prttree(n, "strange");
    warn(n, "useless or misleading comparison: %s", cmpbuf);
    return 0;
}

```

Uses OCAST 118a, OCONST 111a, OEQ 115g, OGE 115g, OGT 115g, OHI 300d, OHS 300d, OLE 115g, OLO 300d, OLS 300d, OLT 115g, ONAME 27, ONE 115g, T 31i, TARRAY 31c, TUVLONG 30b, TXXX 30a, add() 300c, big() 300a, cmp() 300b, cmps-95 300e, comrel 256a, debug 474a, prttree() 487b, relindex() 233a, and warn() 142.

Boolean expressions

If one operand of `&&` or `||` is a constant, the result can be determined statically: `0 && x` is always 0, `1 || x` is always 1, and the other operand is checked for side effects before being eliminated. This is also where `!0` folds to 1 and `!1` to 0.

```

⟨ccom() switch node kind cases 303a⟩+≡ (292) <298a 303b>
    case OANDAND:
        ccom(l);
        if(vconst(l) == 0) {
            *n = *l;
            break;
        }
        ccom(r);
        goto common;

```

Uses OANDAND 115c, ccom() 292, and vconst() 176c.

```

⟨ccom() switch node kind cases 303b⟩+≡ (292) <303a 303c>
    case OOROR:
        ccom(l);
        if(l->op == OCONST && l->vconst != 0) {
            *n = *l;
            n->vconst = 1;
            break;
        }
        ccom(r);
        goto common;

```

Uses OCONST 111a, OOROR 115c, and ccom() 292.

```

⟨ccom() switch node kind cases 303c⟩+≡ (292) <303b
    case OCOND:
        ccom(l);
        ccom(r);
        if(l->op == OCONST)
            if(vconst(l) == 0)
                *n = *r->right;
            else
                *n = *r->left;
        break;

```

Uses OCOND 118f, OCONST 111a, ccom() 292, and vconst() 176c.

12.1.2 Constant evaluation: evconst()

`evconst` folds constant subexpressions at compile time: `3 + 4` becomes 7, `sizeof(int)` becomes 4, etc. It is called by `ccom` when both children of a binary node are constants. Beyond optimization, constant folding is

required for array sizes and enum values, which must be compile-time constants in C.

```

⟨function evconst 304a⟩≡ (528a)
  /// gen | ... -> complex -> ccom -> <>
  void
  evconst(Node *n)
  {
    Node *l, *r;
    int et;
    bool isf;
    vlong v = 0;
    double d = 0;

    if(n == Z || n->type == T)
      return;

    et = n->type->etype;
    isf = typefd[et];

    l = n->left;
    r = n->right;

    switch(n->op) {
      ⟨evconst() switch node kind cases 304c⟩
    default:
      return;
    }
    // reach this point if the node is indeed a candidate to "constification"
    if(isf)
      n->fconst = d;
    else
      n->vconst = convvtox(v, n->type->etype);
    n->oldop = n->op;
    n->op = OCONST;
  }

```

Uses OCONST 111a, T 31i, Z 29a, convvtox() 65d, and typefd 149c.

```

⟨Node origin tracking fields 304b⟩+≡ (28) <118c
  // enum<node_kind>
  char oldop;

```

```

⟨evconst() switch node kind cases 304c⟩≡ (304a) 304d>
  case OCONST:
    break;

```

Uses OCONST 111a.

```

⟨evconst() switch node kind cases 304d⟩+≡ (304a) <304c 305a>
  case OCAST:
    if(et == TVOID)
      return;
    et = l->type->etype;
    if(isf) {
      if(typefd[et])
        d = l->fconst;
      else
        d = l->vconst;
    } else {
      if(typefd[et])
        v = l->fconst;
      else

```

```

        v = convvtox(l->vconst, n->type->etype);
    }
    break;

```

Uses OCAST 118a, TVOID 31b, convvtox() 65d, and typefd 149c.

(evconst() switch node kind cases 305a)+≡ (304a) <304d 305b>

```

case OADD:
    if(isf)
        d = l->fconst + r->fconst;
    else
        v = l->vconst + r->vconst;
    break;

case OSUB:
    if(isf)
        d = l->fconst - r->fconst;
    else
        v = l->vconst - r->vconst;
    break;

case OMUL:
    if(isf)
        d = l->fconst * r->fconst;
    else
        v = l->vconst * r->vconst;
    break;

case ODIV:
    if(vconst(r) == 0) {
        warn(n, "divide by zero");
        return;
    }
    if(isf)
        d = l->fconst / r->fconst;
    else
        v = l->vconst / r->vconst;
    break;

case OMOD:
    if(vconst(r) == 0) {
        warn(n, "modulo by zero");
        return;
    }
    v = l->vconst % r->vconst;
    break;

```

Uses OADD 114c, ODIV 114c, OMOD 114c, OMUL 114c, OSUB 114c, vconst() 176c, and warn() 142.

(evconst() switch node kind cases 305b)+≡ (304a) <305a 306a>

```

case OLMUL:
    v = (uulong)l->vconst * (uulong)r->vconst;
    break;

case OLDIV:
    if(vconst(r) == 0) {
        warn(n, "divide by zero");
        return;
    }
    v = (uulong)l->vconst / (uulong)r->vconst;
    break;

```

```

case OLMOD:
    if(vconst(r) == 0) {
        warn(n, "modulo by zero");
        return;
    }
    v = (uulong)l->vconst % (uulong)r->vconst;
    break;

```

Uses OLDIV 232d, OLMOD 232f, OLMUL 232d, vconst() 176c, and warn() 142.

(evconst() switch node kind cases 306a)+≡ (304a) <305b 306b>

```

case ONEG:
    if(isf)
        d = -l->fconst;
    else
        v = -l->vconst;
    break;

case OCOM:
    v = ~l->vconst;
    break;

```

Uses OCOM 115e and ONEG 114e.

(evconst() switch node kind cases 306b)+≡ (304a) <306a 306c>

```

case OAND:
    v = l->vconst & r->vconst;
    break;

case OOR:
    v = l->vconst | r->vconst;
    break;

case OXOR:
    v = l->vconst ^ r->vconst;
    break;

```

Uses OAND 115a, OOR 115a, and OXOR 115a.

(evconst() switch node kind cases 306c)+≡ (304a) <306b 306d>

```

case OLSHR:
    v = (uulong)l->vconst >> r->vconst;
    break;

case OASHR:
    v = l->vconst >> r->vconst;
    break;

case OASHL:
    v = l->vconst << r->vconst;
    break;

```

Uses OASHL 115a, OASHR 115a, and OLSHR 232b.

(evconst() switch node kind cases 306d)+≡ (304a) <306c 307a>

```

case OLT:
    if(typefd[l->type->etype])
        v = l->fconst < r->fconst;
    else
        v = l->vconst < r->vconst;
    break;

case OGT:

```

```

    if(typefd[l->type->etype])
        v = l->fconst > r->fconst;
    else
        v = l->vconst > r->vconst;
    break;

```

```

case OLE:
    if(typefd[l->type->etype])
        v = l->fconst <= r->fconst;
    else
        v = l->vconst <= r->vconst;
    break;

```

```

case OGE:
    if(typefd[l->type->etype])
        v = l->fconst >= r->fconst;
    else
        v = l->vconst >= r->vconst;
    break;

```

Uses OGE 115g, OGT 115g, OLE 115g, OLT 115g, and typefd 149c.

⟨evconst() switch node kind cases 307a⟩+≡ (304a) ◁306d 307b▷

```

case OLO:
    v = (uulong)l->vconst < (uulong)r->vconst;
    break;

```

```

case OHI:
    v = (uulong)l->vconst > (uulong)r->vconst;
    break;

```

```

case OLS:
    v = (uulong)l->vconst <= (uulong)r->vconst;
    break;

```

```

case OHS:
    v = (uulong)l->vconst >= (uulong)r->vconst;
    break;

```

Uses OHI 300d, OHS 300d, OLO 300d, and OLS 300d.

⟨evconst() switch node kind cases 307b⟩+≡ (304a) ◁307a 307c▷

```

case OEQ:
    if(typefd[l->type->etype])
        v = l->fconst == r->fconst;
    else
        v = l->vconst == r->vconst;
    break;

```

```

case ONE:
    if(typefd[l->type->etype])
        v = l->fconst != r->fconst;
    else
        v = l->vconst != r->vconst;
    break;

```

Uses OEQ 115g, ONE 115g, and typefd 149c.

⟨evconst() switch node kind cases 307c⟩+≡ (304a) ◁307b

```

case OANDAND:
    if(typefd[l->type->etype])
        v = l->fconst && r->fconst;

```

```

    else
        v = l->vconst && r->vconst;
    break;

case OOROR:
    if(typefd[l->type->etype])
        v = l->fconst || r->fconst;
    else
        v = l->vconst || r->vconst;
    break;
case ONOT:
    if(typefd[l->type->etype])
        v = !l->fconst;
    else
        v = !l->vconst;
    break;

```

Uses OANDAND 115c, ONOT 115e, OOROR 115c, and typefd 149c.

12.2 Comma hoisting: comma()

`comma` restructures expressions that contain comma operators buried inside other operators. It “hoists” the side effects to the top level, so the code generator sees a clean sequence of side effects followed by the value expression. For example, `(a++, b) + c` becomes `a++; b + c`. This simplifies the code generator, which would otherwise need to handle comma operators in arbitrary positions.

```

⟨struct Com 308a⟩≡ (529)
struct Com
{
    int n;
    Node *t[500];
};

```

```

⟨function comma 308b⟩≡ (529)
/// complex -> <>
static void
comma(Node *n)
{
    Com com;
    Node *nn;

    com.n = 0;
    nn = commas(&com, n);

    if(com.n > 0){

if(debug['y'])print("n=%d\n", com.n);
if(debug['y']) prtree(nn, "res");

        if(nn != n)
            *n = *nn;
        while(com.n > 0){

if(debug['y']) prtree(com.t[com.n-1], "tree");

            nn = new1(OXXX, Z, Z);
            *nn = *n;
            n->op = OCOMMA;
            n->type = nn->type;

```

```

        n->left = com.t[--com.n];
        n->right = nn;
        n->lineno = n->left->lineno;
    }
    if(debug['y']) prttree(n, "final");

    }else if(n != nn)
        fatal(n, "odd tree");
}

```

Uses OCOMMA 110g, OXXX 27, Z 29a, commas() 309, debug 474a, fatal() 496c, new1() 29h, and prttree() 487b.

<function commas 309>≡ (529)

```

static Node*
commas(Com *com, Node *n)
{
    Node *t;

    if(n == Z)
        return n;
    switch(n->op){
    case OREGISTER:
    case OINDREG:
    case OCONST:
    case ONAME:
    case OSTRING:
        /* leaf */
        return n;

    case OCOMMA:
        t = commas(com, n->left);
        if(com->n >= nelem(com->t))
            fatal(n, "comma list overflow");
        com->t[com->n++] = t;
        return commas(com, n->right);

    case OFUNC:
        n->left = commas(com, n->left);
        n->right = comargs(com, n->right);
        return n;

    case OCOND:
        n->left = commas(com, n->left);
        comma(n->right->left);
        comma(n->right->right);
        return n;

    case OANDAND:
    case OOROR:
        n->left = commas(com, n->left);
        comma(n->right);
        return n;

    case ORETURN:
        comma(n->left);
        return n;
    }
    n->left = commas(com, n->left);
    if(n->right != Z)
        n->right = commas(com, n->right);
    return n;
}

```

```
}
```

Uses OANDAND 115c, OCOMMA 110g, OCOND 118f, OCONST 111a, OFUNC 117c, OINDREG 237c, ONAME 27, OOROR 115c, OREGISTER 208f, ORETURN 108b, OSTRING 112c, Z 29a, comargs() 310a, comma() 308b, commas() 309, and fatal() 496c.

```
<function comargs 310a>≡ (529)
/*
 * hoist comma operators out of expressions
 * (a,b) OP c => (a, b OP c)
 * OP(a,b) => (a, OP b)
 * a OP (b,c) => (b, a OP c)
 */

static Node*
comargs(Com *com, Node *n)
{
    if(n != Z && n->op == OLIST){
        n->left = comargs(com, n->left);
        n->right = comargs(com, n->right);
    }
    return commas(com, n);
}
```

Uses OLIST 29b, Z 29a, comargs() 310a, and commas() 309.

12.3 Bitshifting opportunities

When the compiler sees a multiplication or division by a power of two, it replaces the operation with a left or right shift: $x * 8$ becomes $x \ll 3$, and $x / 4$ becomes $x \gg 2$. vlog checks whether a constant is a power of two by calling log2. This applies both in xcom (for general expressions) and in arith (for pointer-difference scaling).

```
<function vlog 310b>≡ (518c)
/// arith -> <>
int
vlog(Node *n)
{
    if(n->op != OCONST)
        goto bad;
    if(typefd[n->type->etype])
        goto bad;

    return log2(n->vconst);

bad:
    return -1;
}
```

Uses OCONST 111a, log2() 502b, and typefd 149c.

```
<arith() when subtract two pointers, convert DIV to SHR if can 310c>≡ (163c)
w = vlog(n1);
if(w >= 0) {
    n->op = OASHR;
    n1->vconst = w;
}
```

Uses OASHR 115a and vlog() 310b.

`<xcom() switch op cases to set addable 311a>+≡` (210b) `<212f 311b>`

```

case OMUL:
case OLMUL:
    xcom(l);
    xcom(r);
    t = vlog(r);
    if(t >= 0) {
        n->op = OASHL;
        r->vconst = t;
        r->type = types[TINT];
    }
    t = vlog(l);
    if(t >= 0) {
        n->op = OASHL;
        n->left = r;
        n->right = l;
        r = l;
        l = n->left;
        r->vconst = t;
        r->type = types[TINT];
    }
    break;

```

Uses OASHL 115a, OLMUL 232d, OMUL 114c, TINT 30b, types 32b, and vlog() 310b.

`<xcom() switch op cases to set addable 311b>+≡` (210b) `<311a 311c>`

```

case OASLMUL:
case OASMUL:
    xcom(l);
    xcom(r);
    t = vlog(r);
    if(t >= 0) {
        n->op = OASASHL;
        r->vconst = t;
        r->type = types[TINT];
    }
    break;

```

Uses OASASHL 116c, OASLMUL 233b, OASMUL 116c, TINT 30b, types 32b, and vlog() 310b.

`<xcom() switch op cases to set addable 311c>+≡` (210b) `<311b 311d>`

```

case OLDIV:
    xcom(l);
    xcom(r);
    t = vlog(r);
    if(t >= 0) {
        n->op = OLSHR;
        r->vconst = t;
        r->type = types[TINT];
    }
    break;

```

Uses OLDIV 232d, OLSHR 232b, TINT 30b, types 32b, and vlog() 310b.

`<xcom() switch op cases to set addable 311d>+≡` (210b) `<311c 312b>`

```

case OASLDIV:
    xcom(l);
    xcom(r);
    t = vlog(r);
    if(t >= 0) {
        n->op = OASLSHR;
        r->vconst = t;
    }

```



```

    r->type = types[TINT];
}
break;

```

Uses OASLDIV 233b, OASLSHR 233b, TINT 30b, types 32b, and vlog() 310b.

```

⟨cgenrel() when ODIV/OMOD, convert to SHR if possible 312a⟩≡ (229a)
if(nn != Z)
if((t = vlog(r)) >= 0) {
/* signed div/mod by constant power of 2 */
cgen(1, nn);
gopcode(OGE, nodconst(0), nn, Z);
p1 = p;
if(o == ODIV) {
gopcode(OADD, nodconst((1<<t)-1), Z, nn);
patch(p1, pc);
gopcode(OASHR, nodconst(t), Z, nn);
} else {
gopcode(OSUB, nn, nodconst(0), nn);
gopcode(OAND, nodconst((1<<t)-1), Z, nn);
gopcode(OSUB, nn, nodconst(0), nn);
gbranch(OGOTO);
patch(p1, pc);
p1 = p;
gopcode(OAND, nodconst((1<<t)-1), Z, nn);
patch(p1, pc);
}
break;
}

```

Uses OADD 114c, OAND 115a, OASHR 115a, ODIV 114c, OGE 115g, OGOTO 108e, OSUB 114c, Z 29a, and vlog() 310b.

12.4 And opportunities

Unsigned modulo by a power of two is replaced by a bitwise AND: $x \% 8$ becomes $x \& 7$. This is valid only for unsigned types (OLMOD is the unsigned modulo operation).

```

⟨xcom() switch op cases to set addable 312b⟩+≡ (210b) <311d 312c>
case OLMOD:
xcom(1);
xcom(r);
t = vlog(r);
if(t >= 0) {
n->op = OAND;
r->vconst--;
}
break;

```

Uses OAND 115a, OLMOD 232f, and vlog() 310b.

```

⟨xcom() switch op cases to set addable 312c⟩+≡ (210b) <312b>
case OASLMOD:
xcom(1);
xcom(r);
t = vlog(r);
if(t >= 0) {
n->op = OASAND;
r->vconst--;
}
break;

```

Uses OASAND 116c, OASLMOD 233b, and vlog() 310b.

12.5 Immediate operands

ARM instructions can encode small constants directly in the instruction (as an 8-bit value with an even rotation). When one operand is such a constant, the code generator avoids loading it into a register first—it just embeds the value in the instruction’s immediate field. This section handles subtraction (constant on the left), and addition/bitwise ops (constant on the right, after `xcom`’s normalization).

12.5.1 Subtraction

```
<cgenrel() when OSUB, if l is OCONST 313a>≡ (228b)
    if(nn != Z)
        if(l->op == OCONST)
            if(!typefd[n->type->etype]) {
                cgen(r, nn);
                gopcode(o, Z, l, nn);
                break;
            }
}
```

Uses OCONST 111a, Z 29a, and typefd 149c.

12.5.2 Addition, or, etc.

When one operand of an addition or bitwise operation is a small constant, the code generator can use ARM’s immediate operand mode instead of loading the constant into a register first. This subsection handles this optimization by checking if the right operand is a constant that fits in the ARM immediate field (an 8-bit value rotated by an even number of bits).

```
<cgenrel() when OADD/OAND/etc, if r is OCONST 313b>≡ (228b)
/*
 * immediate operands
 */
if(nn != Z)
    if(r->op == OCONST)
        if(!typefd[n->type->etype]) {
            cgen(l, nn);
            if((r->vconst == 0) && (o != OAND))
                break;
            if(nn != Z)
                gopcode(o, r, Z, nn);
            break;
        }
}
```

Uses OAND 115a, OCONST 111a, Z 29a, and typefd 149c.

```
<xcom() switch node opkind cases to set complexity 313c>+≡ (213a) <213b
    case OADD:
    case OAND:
    case OOR:
    case OXOR:
    case OEQ:
    case ONE:
        /*
         * immediate operators, make const on right
         */
        if(l->op == OCONST) {
            n->left = r;
            n->right = l;
        }
        break;
```

Uses OADD 114c, OAND 115a, OCONST 111a, OEQ 115g, ONE 115g, OOR 115a, and OXOR 115a.

12.5.3 Multiplication

When one operand of a multiply is a known constant, the compiler can replace the multiplication with a sequence of shifts and adds (strength reduction). The function `mulcon` tries to find such a decomposition using a precomputed table of shift/add “programs.” For example, multiply by 5 becomes `ADD R0<<2, R0, R1` (shift left 2 and add). Similarly, `divcon` replaces constant division by multiplication by a magic reciprocal.

Why bother? On classic ARM, `MUL` takes 4–16 cycles depending on the multiplier (Booth’s algorithm runs faster on small operands), while a shift-and-add fuses into a single barrel-shifted `ADD` (one cycle). Two or three chained shift-adds still beat one `MUL`, and they leave no result-use stall. Cortex-M0 and other small cores have no hardware multiplier at all—there, `MUL` is a libcall, and replacing it pays for itself many times over. The compiler caps the search at `maxmulops = 3` operations because beyond three the win disappears.

The “code” string in `Multab` is a tiny stack-machine program for the multiply. Each step is a 2-character instruction:

- `a..z 0..7`—a shift: letter $c \in \{a, \dots, z\}$ encodes a shift count of $c - a$; the digit selects the source/dest register pair.
- `+ 0..7` or `- 0..7`—an add or subtract of two of the three slots (`r0, r1, result`), where the digit packs three 1-bit selectors `r,n,l` into 0–7.

For multiplication by 5, the search produces the code `c1+1\0` (one shift then one add). `mulcon` walks this code, allocates two registers `nod1` and `nod2`, and emits two instructions:

```
c1  ->  ASHL  $2, R(nod1), R(nod2)    ; nod2 = nod1 << 2  = 4*x
+1  ->  ADD   R(nod1), R(nod2), R(nod1) ; nod1 = nod1 + nod2 = 5*x
\0  ->  MOV   R(nod1), R(result)       ; emit final move
```

For multiplication by 100, the search finds `c1+1d2\0`: shift left by 2 ($\times 4$), add ($\times 5$), shift left by 4 ($\times 80$), Wait—that does not quite reach 100. The actual code is found by `mulcon0`’s recursive “shift suffix” branch: $100 = 25 \times 4$, and 25 has its own program `c1+1c2`, so the result is `c1+1c2c0`. Notice the elegance: any multiplier whose binary representation has few set bits decomposes cheaply, and the cache of recently computed programs (the `multab` ring) means a hot loop with the same constant only pays the search cost once.

```
<cgenrel() when OMUL/OLMUL, mulcon opportunity 314a>≡ (228b)
  if(o == OMUL || o == OLMUL) {
    if(mulcon(n, nn))
      break;
  }
```

Uses `OLMUL 232d`, `OMUL 114c`, and `mulcon() 314b`.

```
<function mulcon(arm) 314b>≡ (540b)
  /// cgen -> cgenrel -> <>
  bool
  mulcon(Node *n, Node *nn)
  {
    Node *l, *r, nod1, nod2;
    Multab *m;
    long v, vs;
    int o;
    char code[sizeof(m->code)+2], *p;

    if(typefd[n->type->etype])
      return false;

    l = n->left;
    r = n->right;
```

```

    if(l->op == OCONST) {
        l = r;
        r = n->left;
    }
    if(r->op != OCONST)
        return false;

    v = convvtox(r->vconst, n->type->etype);
    if(v != r->vconst) {
        if(debug['M'])
            print("%L multiply conv: %lld\n", n->lineno, r->vconst);
        return false;
    }
    m = mulcon0(v);
    if(!m) {
        if(debug['M'])
            print("%L multiply table: %lld\n", n->lineno, r->vconst);
        return false;
    }
    if(debug['M'] && debug['v'])
        print("%L multiply: %ld\n", n->lineno, v);

    memmove(code, m->code, sizeof(m->code));
    code[sizeof(m->code)] = 0;

    p = code;
    if(p[1] == 'i')
        p += 2;
    regalloc(&nod1, n, nn);
    cgen(1, &nod1);
    vs = v;
    regalloc(&nod2, n, Z);

loop:
    switch(*p) {
    case 0:
        regfree(&nod2);
        if(vs < 0) {
            gopcode(OAS, &nod1, Z, &nod1);
            gopcode(OSUB, &nod1, nodconst(0), nn);
        } else
            gopcode(OAS, &nod1, Z, nn);
        regfree(&nod1);
        return true;
    case '+':
        o = OADD;
        goto addsub;
    case '-':
        o = OSUB;
    addsub: /* number is r,n,l */
        v = p[1] - '0';
        r = &nod1;
        if(v&4)
            r = &nod2;
        n = &nod1;
        if(v&2)
            n = &nod2;
        l = &nod1;
        if(v&1)

```

```

        l = &nod2;
        gopcode(o, l, n, r);
        break;
default: /* op is shiftcount, number is r,l */
    v = p[1] - '0';
    r = &nod1;
    if(v&2)
        r = &nod2;
    l = &nod1;
    if(v&1)
        l = &nod2;
    v = *p - 'a';
    if(v < 0 || v >= 32) {
        diag(n, "mulcon unknown op: %c%c", p[0], p[1]);
        break;
    }
    gopcode(OASHL, nodconst(v), l, r);
    break;
}
p += 2;
goto loop;
}

```

Uses OADD 114c, OAS 116a, OASHL 115a, OCONST 111a, OSUB 114c, Z 29a, convvtov() 65d, debug 474a, diag() 496b, mulcon0() 317b, and typefd 149c.

```

⟨struct Multab(arm) 316a⟩≡ (531b)
    struct Multab
    {
        long val;
        char code[20];
    };

```

```

⟨global multab(arm) 316b⟩≡ (535b)
    Multab multab[20];

```

```

⟨global maxmulops(arm) 316c⟩≡ (540c)
    static int maxmulops = 3; /* max # of ops to replace mul with */

```

Uses maxmulops-205 316c.

```

⟨global multabp(arm) 316d⟩≡ (540c)
    static int multabp;

```

```

⟨global mulval(arm) 316e⟩≡ (540c)
    static long mulval;

```

```

⟨global mulcp(arm) 316f⟩≡ (540c)
    static char* mulcp;

```

```

⟨global valmax(arm) 316g⟩≡ (540c)
    static long valmax;

```

```

⟨global shmax(arm) 316h⟩≡ (540c)
    static int shmax;

```

```

⟨enum _anon_ (5c/mul.c) (arm) 316i⟩≡ (540c)
    enum
    {
        SR1 = 1<<0, /* r1 has been shifted */
        SR0 = 1<<1, /* r0 has been shifted */
        UR1 = 1<<2, /* r1 has not been used */
        UR0 = 1<<3, /* r0 has not been used */
    };

```

*<struct Hintab(*arm*) 317a>*≡ (531b)

```
struct Hintab
{
    ushort val;
    char hint[10];
};
```

*<function mulcon0(*arm*) 317b>*≡ (540c)

```
Multab*
mulcon0(long v)
{
    int a1, a2, g;
    Multab *m, *m1;
    char hint[10];

    if(v < 0)
        v = -v;

    /*
     * look in cache
     */
    m = multab;
    for(g=0; g<nelem(multab); g++) {
        if(m->val == v) {
            if(m->code[0] == 0)
                return 0;
            return m;
        }
        m++;
    }

    /*
     * select a spot in cache to overwrite
     */
    multabp++;
    if(multabp < 0 || multabp >= nelem(multab))
        multabp = 0;
    m = multab+multabp;
    m->val = v;
    mulval = v;

    /*
     * look in exeption hint table
     */
    a1 = 0;
    a2 = hintabsize;
    for(;;) {
        if(a1 >= a2)
            goto no;
        g = (a2 + a1)/2;
        if(v < hintab[g].val) {
            a2 = g;
            continue;
        }
        if(v > hintab[g].val) {
            a1 = g+1;
            continue;
        }
        break;
    }
}
```

```

    if(docode(hintab[g].hint, m->code, 1, 0))
        return m;
    print("multiply table failure %ld\n", v);
    m->code[0] = 0;
    return 0;

no:
    /*
     * try to search
     */
    hint[0] = 0;
    for(g=1; g<=maxmulops; g++) {
        if(g >= maxmulops && v >= 65535)
            break;
        mulcp = hint+g;
        *mulcp = 0;
        if(gen1(g)) {
            if(docode(hint, m->code, 1, 0))
                return m;
            print("multiply table failure %ld\n", v);
            break;
        }
    }

    /*
     * try a recur followed by a shift
     */
    g = 0;
    while(!(v & 1)) {
        g++;
        v >>= 1;
    }
    if(g) {
        m1 = mulcon0(v);
        if(m1) {
            strcpy(m->code, m1->code);
            sprintf(strchr(m->code, 0), "%c0", g+'a');
            return m;
        }
    }
    m->code[0] = 0;
    return 0;
}

```

Uses docode() 318, gen1() 320a, hintab 323, maxmulops-205 316c, mulcon0() 317b, mulcp-208 316f, multab 316b, multabp-206 316d, and mulval-207 316e.

⟨function docode(*arm*) 318⟩≡ (540c)

```

static int
docode(char *hp, char *cp, int r0, int r1)
{
    int c, i;

    c = *hp++;
    *cp = c;
    cp += 2;
    switch(c) {
    default:
        c -= 'a';
        if(c < 1 || c >= 30)

```

```

        break;
    for(i=0; i<4; i++) {
        switch(i) {
            case 0:
                if(docode(hp, cp, r0<<c, r1))
                    goto out;
                break;
            case 1:
                if(docode(hp, cp, r1<<c, r1))
                    goto out;
                break;
            case 2:
                if(docode(hp, cp, r0, r0<<c))
                    goto out;
                break;
            case 3:
                if(docode(hp, cp, r0, r1<<c))
                    goto out;
                break;
        }
    }
    break;

case '+':
    for(i=0; i<8; i++) {
        cp[-1] = i+'0';
        switch(i) {
            case 1:
                if(docode(hp, cp, r0+r1, r1))
                    goto out;
                break;
            case 5:
                if(docode(hp, cp, r0, r0+r1))
                    goto out;
                break;
        }
    }
    break;

case '-':
    for(i=0; i<8; i++) {
        cp[-1] = i+'0';
        switch(i) {
            case 1:
                if(docode(hp, cp, r0-r1, r1))
                    goto out;
                break;
            case 2:
                if(docode(hp, cp, r1-r0, r1))
                    goto out;
                break;
            case 5:
                if(docode(hp, cp, r0, r0-r1))
                    goto out;
                break;
            case 6:
                if(docode(hp, cp, r0, r1-r0))
                    goto out;
                break;
        }
    }

```



```

    }
    break;

case 0:
    if(r0 == mulval)
        return 1;
}
return 0;

out:
    cp[-1] = i+'0';
    return 1;
}

```

Uses `docode()` 318 and `mulval-207` 316e.

```

⟨function gen1(arm) 320a⟩≡ (540c)
static int
gen1(int len)
{
    int i;

    for(shmax=1; shmax<30; shmax++) {
        valmax = 1<<shmax;
        if(valmax >= mulval)
            break;
    }
    if(mulval == 1)
        return 1;

    len--;
    for(i=1; i<=shmax; i++)
        if(gen2(len, 1<<i)) {
            *--mulcp = 'a'+i;
            return 1;
        }
    return 0;
}

```

Uses `gen2()` 320b, `mulcp-208` 316f, `mulval-207` 316e, `shmax-210` 316h, and `valmax-209` 316g.

```

⟨function gen2(arm) 320b⟩≡ (540c)
static int
gen2(int len, long r1)
{
    int i;

    if(len <= 0) {
        if(r1 == mulval)
            return 1;
        return 0;
    }

    len--;
    if(len == 0)
        goto calcr0;

    if(gen3(len, r1, r1+1, UR1)) {
        i = '+';
        goto out;
    }
    if(gen3(len, r1-1, r1, UR0)) {

```

```

        i = '-';
        goto out;
    }
    if(gen3(len, 1, r1+1, UR1)) {
        i = '+';
        goto out;
    }
    if(gen3(len, 1, r1-1, UR1)) {
        i = '-';
        goto out;
    }

    return 0;

calcr0:
    if(mulval == r1+1) {
        i = '+';
        goto out;
    }
    if(mulval == r1-1) {
        i = '-';
        goto out;
    }
    return 0;

out:
    *--mulcp = i;
    return 1;
}

```

Uses UR0-214 [316i](#), UR1-213 [316i](#), gen3() [321](#), mulcp-208 [316f](#), and mulval-207 [316e](#).

```

⟨function gen3(arm) 321⟩≡ (540c)
static int
gen3(int len, long r0, long r1, int flag)
{
    int i, f1, f2;
    long x;

    if(r0 <= 0 ||
       r0 >= r1 ||
       r1 > valmax)
        return 0;

    len--;
    if(len == 0)
        goto calcr0;

    if(!(flag & UR1)) {
        f1 = UR1|SR1;
        for(i=1; i<=shmax; i++) {
            x = r0<<i;
            if(x > valmax)
                break;
            if(gen3(len, r0, x, f1)) {
                i += 'a';
                goto out;
            }
        }
    }
}

```

```

if(!(flag & UR0)) {
    f1 = UR1|SR1;
    for(i=1; i<=shmax; i++) {
        x = r1<<i;
        if(x > valmax)
            break;
        if(gen3(len, r1, x, f1)) {
            i += 'a';
            goto out;
        }
    }
}

if(!(flag & SR1)) {
    f1 = UR1|SR1|(flag&UR0);
    for(i=1; i<=shmax; i++) {
        x = r1<<i;
        if(x > valmax)
            break;
        if(gen3(len, r0, x, f1)) {
            i += 'a';
            goto out;
        }
    }
}

if(!(flag & SR0)) {
    f1 = UR0|SR0|(flag&(SR1|UR1));

    f2 = UR1|SR1;
    if(flag & UR1)
        f2 |= UR0;
    if(flag & SR1)
        f2 |= SR0;

    for(i=1; i<=shmax; i++) {
        x = r0<<i;
        if(x > valmax)
            break;
        if(x > r1) {
            if(gen3(len, r1, x, f2)) {
                i += 'a';
                goto out;
            }
        } else
            if(gen3(len, x, r1, f1)) {
                i += 'a';
                goto out;
            }
    }
}

x = r1+r0;
if(gen3(len, r0, x, UR1)) {
    i = '+';
    goto out;
}

if(gen3(len, r1, x, UR1)) {
    i = '+';

```

```

        goto out;
    }

    x = r1-r0;
    if(gen3(len, x, r1, UR0)) {
        i = '-';
        goto out;
    }

    if(x > r0) {
        if(gen3(len, r0, x, UR1)) {
            i = '-';
            goto out;
        }
    } else
        if(gen3(len, x, r0, UR0)) {
            i = '-';
            goto out;
        }

    return 0;

calcr0:
    f1 = flag & (UR0|UR1);
    if(f1 == UR1) {
        for(i=1; i<=shmax; i++) {
            x = r1<<i;
            if(x >= mulval) {
                if(x == mulval) {
                    i += 'a';
                    goto out;
                }
                break;
            }
        }
    }

    if(mulval == r1+r0) {
        i = '+';
        goto out;
    }
    if(mulval == r1-r0) {
        i = '-';
        goto out;
    }

    return 0;

out:
    *--mulcp = i;
    return 1;
}

Uses SR0-212 316i, SR1-211 316i, UR0-214 316i, UR1-213 316i, gen3() 321, mulcp-208 316f, mulval-207 316e, shmax-210 316h,
and valmax-209 316g.

⟨global hintab(arm) 323⟩≡ (540c)
/*
 * hint table has numbers that
 * the search algorithm fails on.
 * <1000:

```

```

* all numbers
* <5000:
* ÷ by 5
* <10000:
* ÷ by 50
* <65536:
* ÷ by 250
*/
Hintab hintab[] =
{
    683, "b++d+e+",
    687, "b+e++e-",
    691, "b++d+e+",
    731, "b++d+e+",
    811, "b++d+i+",
    821, "b++e+e+",
    843, "b+d++e+",
    851, "b+f-+e-",
    853, "b++e+e+",
    877, "c++++g-",
    933, "b+c++g-",
    981, "c-+e-d+",
    1375, "b+c+b+h-",
    1675, "d+b++h+",
    2425, "c++f-e+",
    2675, "c+d++f-",
    2750, "b+d-b+h-",
    2775, "c-+g-e-",
    3125, "b++e+g+",
    3275, "b+c+g+e+",
    3350, "c++++i+",
    3475, "c-+e-f-",
    3525, "c-+d+g-",
    3625, "c-+e-j+",
    3675, "b+d+d+e+",
    3725, "b+d-+h+",
    3925, "b+d+f-d-",
    4275, "b+g++e+",
    4325, "b+h-+d+",
    4425, "b+b+g-j-",
    4525, "b+d-d+f+",
    4675, "c++d-g+",
    4775, "b+d+b+g-",
    4825, "c+c-+i-",
    4850, "c++++i-",
    4925, "b++e-g-",
    4975, "c+f++e-",
    5500, "b+g-c+d+",
    6700, "d+b++i+",
    9700, "d++++j-",
    11000, "b+f-c-h-",
    11750, "b+d+g+j-",
    12500, "b+c+e-k+",
    13250, "b+d+e-f+",
    13750, "b+h-c-d+",
    14250, "b+g-c+e-",
    14500, "c+f+j-d-",
    14750, "d-g--f+",
    16750, "b+e-d-n+",
    17750, "c+h-b+e+",

```

18250, "d+b+h-d",
 18750, "b+g-++f+",
 19250, "b+e+b+h+",
 19750, "b++h--f-",
 20250, "b+e-l-c+",
 20750, "c++bi+e-",
 21250, "b+i+l+c+",
 22000, "b+e+d-g-",
 22250, "b+d-h+k-",
 22750, "b+d-e-g+",
 23250, "b+c+h+e-",
 23500, "b+g-c-g-",
 23750, "b+g-b+h-",
 24250, "c++g+m-",
 24750, "b+e+e+j-",
 25000, "b++dh+g+",
 25250, "b+e+d-g-",
 25750, "b+e+b+j+",
 26250, "b+h+c+e+",
 26500, "b+h+c+g+",
 26750, "b+d+e+g-",
 27250, "b+e+e+f+",
 27500, "c-i-c-d+",
 27750, "b+bd++j+",
 28250, "d-d-++i-",
 28500, "c+c-h-e-",
 29000, "b+g-d-f+",
 29500, "c+h+++e-",
 29750, "b+g+f-c+",
 30250, "b+f-g-c+",
 33500, "c-f-d-n+",
 33750, "b+d-b+j-",
 34250, "c+e+++i+",
 35250, "e+b+d+k+",
 35500, "c+e+d-g-",
 35750, "c+i-++e+",
 36250, "b+bh-d+e+",
 36500, "c+c-h-e-",
 36750, "d+e--i+",
 37250, "b+g+g+b+",
 37500, "b+h-b+f+",
 37750, "c+be++j-",
 38500, "b+e+b+i+",
 38750, "d+i-b+d+",
 39250, "b+g-l-+d+",
 39500, "b+g-c+g-",
 39750, "b+bh-c+f-",
 40250, "b+bf+d+g-",
 40500, "b+g-c+g+",
 40750, "c+b+i-e+",
 41250, "d++bf+h+",
 41500, "b+j+c+d-",
 41750, "c+f+b+h-",
 42500, "c+h++g+",
 42750, "b+g+d-f-",
 43250, "b+l-e+d-",
 43750, "c+bd+h+f-",
 44000, "b+f+g-d-",
 44250, "b+d-g--f+",
 44500, "c+e+c+h+",

```

44750, "b+e+d-h-",
45250, "b++g+j-g+",
45500, "c+d+e-g+",
45750, "b+d-h-e-",
46250, "c+bd++j+",
46500, "b+d-c-j-",
46750, "e-e-b+g-",
47000, "b+c+d-j-",
47250, "b+e+e-g-",
47500, "b+g-c-h-",
47750, "b+f-c+h-",
48250, "d--h+n-",
48500, "b+c-g+m-",
48750, "b+e+e-g+",
49500, "c-f+e+j-",
49750, "c+c+g++f-",
50000, "b+e+e+k+",
50250, "b++i++g+",
50500, "c+g+f-i+",
50750, "b+e+d+k-",
51500, "b+i+c-f+",
51750, "b+bd+g-e-",
52250, "b+d+g-j+",
52500, "c+c+f+g+",
52750, "b+c+e+i+",
53000, "b+i+c+g+",
53500, "c+g+g-n+",
53750, "b+j+d-c+",
54250, "b+d-g-j-",
54500, "c-f+e+f+",
54750, "b+f--+c+g+",
55000, "b+g-d-g-",
55250, "b+e+e+g+",
55500, "b+cd++j+",
55750, "b+bh-d-f-",
56250, "c+d-b+j-",
56500, "c+d+c+i+",
56750, "b+e+d++h-",
57000, "b+d+g-f+",
57250, "b+f-m+d-",
57750, "b+i+c+e-",
58000, "b+e+d+h+",
58250, "c+b+g+g+",
58750, "d-e-j--e+",
59000, "d-i-+e+",
59250, "e--h-m+",
59500, "c+c-h+f-",
59750, "b+bh-e+i-",
60250, "b+bh-e-e-",
60500, "c+c-g-g-",
60750, "b+e-l-e-",
61250, "b+g-g-c+",
61750, "b+g-c+g+",
62250, "f--+c-i-",
62750, "e+f--+g+",
64750, "b+f+d+p-",
};

```

```

⟨global hintabsz (5c/mul.c)(arm) 326⟩≡
int hintabsz = nelem(hintab);

```

(540c)

Uses hintab 323.

```
<global hintabsize(arm) 327>≡ (535b)
int hintabsize;
```

12.6 Associative-commutative arithmetic optimisations

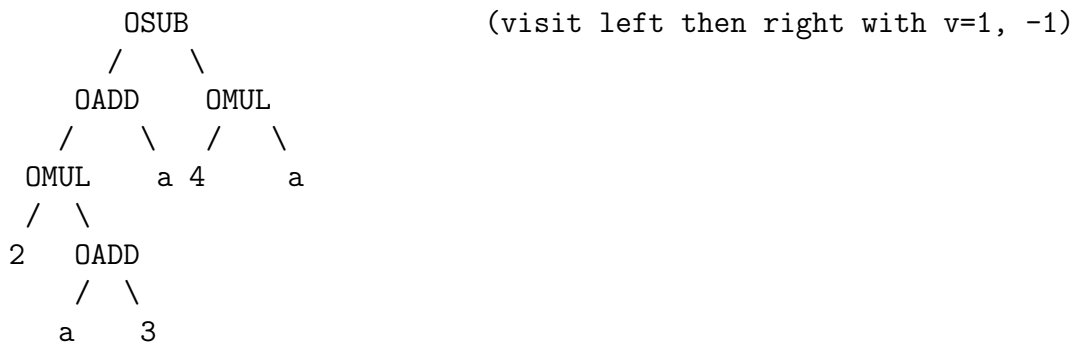
`acom` implements the classic associative-commutative rearrangement from Aho and Ullman: it flattens a tree of additions or multiplications into a flat list of `Terms`, then recombines them to expose common factors and simplify the arithmetic. For example, `c1*i + c1*c2*j` is transformed into `c1*(i + c2*j)`, saving a multiplication. This is particularly useful for array index calculations, where the compiler can factor out the element size.

12.6.1 Term

A `Term` pairs a node with an integer multiplier. `acom` flattens an expression tree like `a + 2*b - a` into an array of terms `(1,a)`, `(2,b)`, `(-1,a)`, then combines like terms (`a` cancels out) and rebuilds the tree. The `NTERM` limit of 10 means expressions with more than 10 leaves are not fully optimized.

A worked example. Suppose the parser builds the tree for `2 * (a + 3) - a + 4 * a` (a contrived but realistic result of macro expansion). Element 0 of `term` is reserved for the running constant, and `acom1` walks the tree, distributing each multiplier `v` downward:

Step 1, flatten with `acom1(1, root)`:



Effect on the `term[]` array (shown by index : mult x node):

```
term[0] : +0 + 1*(2*3) = 6      (constant term, accumulated)
term[1] : 1*(2*a)      = 2 * a
term[2] : -1 * a       = -1 * a
term[3] : 1*(4*a)      = 4 * a
```

Step 2, sort by node identity, merge like terms:

```
term[0] : 6
term[1] : (2 + -1 + 4) * a = 5 * a
```

Step 3, rebuild:

```
  OADD
 /   \
OMUL  OCONST(6)
 /   \
5    a
```


The original tree had 6 leaves and 5 internal nodes; the rebuilt tree has 2 leaves and 2 internal nodes. The win compounds inside loops where the same expression appears many times—one `acom` pass is worth dozens of peephole applications. The hard limit `NTERM = 10` keeps the search bounded; if an expression has more than 10 distinct atoms, `acom` gives up on it (the recursive call tree would otherwise explode for nested polynomials).

```
<constant NTERM 328a>≡ (504b)
#define NTERM 10
```

```
<struct Term 328b>≡ (504b)
struct Term
{
    vlong mult;
    Node *node;
};
```

```
<global term 328c>≡ (511d)
Term term[NTERM];
```

Uses NTERM 328a.

```
<global nterm 328d>≡ (511d)
int nterm;
```

12.6.2 `acom()`

`acom` first calls `apts` to flatten the expression tree into the `term` array, collecting multiplied terms. Then it sorts the terms so identical nodes are adjacent, merges duplicate terms by adding their multipliers, and rebuilds the expression tree from the simplified term list. Terms with a zero multiplier are eliminated entirely. This optimization is especially effective for address arithmetic where the compiler generates redundant add/subtract chains.

```
<function acom 328e>≡ (528a)
void
acom(Node *n)
{
    Type *t;
    Node *l, *r;
    int i;

    switch(n->op)
    {

    case ONAME:
    case OCONST:
    case OSTRING:
    case OINDREG:
    case OREGISTER:
        return;

    case ONEG:
        l = n->left;
        if(addo(n) && addo(l))
            break;
        acom(l);
        return;

    case OADD:
    case OSUB:
    case OMUL:
```

```

    l = n->left;
    r = n->right;
    if(addo(n)) {
        if(addo(r))
            break;
        if(addo(l))
            break;
    }
    acom(l);
    acom(r);
    return;

default:
    l = n->left;
    r = n->right;
    if(l != Z)
        acom(l);
    if(r != Z)
        acom(r);
    return;
}

/* bust terms out */
t = n->type;
term[0].mult = 0;
term[0].node = Z;
nterm = 1;
acom1(1, n);

if(debug['m'])
    for(i=0; i<nterm; i++) {
        print("%d %3lld ", i, term[i].mult);
        prttree1(term[i].node, 1, 0);
    }

if(nterm < NTERM)
    acom2(n, t);
n->type = t;
}

```

Uses NTERM 328a, OADD 114c, OCONST 111a, OINDREG 237c, OMUL 114c, ONAME 27, ONEG 114e, OREGISTER 208f, OSTRING 112c, OSUB 114c, Z 29a, acom() 328e, acom1() 330, acom2() 332b, addo() 329, debug 474a, nterm 328d, prttree1() 487c, and term 328c.

⟨function addo 329⟩≡ (528a)

```

int
addo(Node *n)
{
    if(n != Z)
        if(!typefd[n->type->etype])
            if(!typev[n->type->etype] || ewidth[TVLONG] == ewidth[TIND])
                switch(n->op) {

                    case OCAST:
                        if(nilcast(n->left->type, n->type))
                            return 1;
                        break;

                    case ONEG:
                    case OADD:

```

```

case OSUB:
    return 1;

case OMUL:
    if(n->left->op == OCONST)
        return 1;
    if(n->right->op == OCONST)
        return 1;
}
return 0;
}

```

Uses OADD 114c, OCAST 118a, OCONST 111a, OMUL 114c, ONEG 114e, OSUB 114c, TIND 31c, TVLONG 30b, Z 29a, nilcast() 299a, typefd 149c, and typev 280a.

```

⟨function acom1 330⟩≡ (528a)
void
acom1(vlong v, Node *n)
{
    Node *l, *r;

    if(v == 0 || nterm >= NTERM)
        return;
    if(!addo(n)) {
        if(n->op == OCONST)
            if(!typefd[n->type->etype]) {
                term[0].mult += v*n->vconst;
                return;
            }
        term[nterm].mult = v;
        term[nterm].node = n;
        nterm++;
        return;
    }
    switch(n->op) {

    case OCAST:
        acom1(v, n->left);
        break;

    case ONEG:
        acom1(-v, n->left);
        break;

    case OADD:
        acom1(v, n->left);
        acom1(v, n->right);
        break;

    case OSUB:
        acom1(v, n->left);
        acom1(-v, n->right);
        break;

    case OMUL:
        l = n->left;
        r = n->right;
        if(l->op == OCONST)
            if(!typefd[n->type->etype]) {
                acom1(v*l->vconst, r);
                break;
            }
    }
}

```

```

    }
    if(r->op == OCONST)
    if(!typefd[n->type->etype]) {
        acom1(v*r->vconst, 1);
        break;
    }
    break;

default:
    diag(n, "not addo");
}
}

```

Uses NTERM 328a, OADD 114c, OCAST 118a, OCONST 111a, OMUL 114c, ONEG 114e, OSUB 114c, acom1() 330, addo() 329, diag() 496b, nterm 328d, term 328c, and typefd 149c.

```

⟨function acast 331a⟩≡ (528a)
static Node*
acast(Type *t, Node *n)
{
    if(n->type->etype != t->etype || n->op == OBIT) {
        n = new1(OCAST, n, Z);
        if(nocast(n->left->type, t))
            *n = *n->left;
        n->type = t;
    }
    return n;
}

```

Uses OBIT 418d, OCAST 118a, Z 29a, new1() 29h, and nocast() 249b.

```

⟨function acomcmp1 331b⟩≡ (528a)
int
acomcmp1(const void *a1, const void *a2)
{
    vlong c1, c2;
    Term *t1, *t2;

    t1 = (Term*)a1;
    t2 = (Term*)a2;
    c1 = t1->mult;
    if(c1 < 0)
        c1 = -c1;
    c2 = t2->mult;
    if(c2 < 0)
        c2 = -c2;
    if(c1 > c2)
        return 1;
    if(c1 < c2)
        return -1;
    c1 = 1;
    if(t1->mult < 0)
        c1 = 0;
    c2 = 1;
    if(t2->mult < 0)
        c2 = 0;
    if(c2 == c1)
        return c2;
    if(t2 > t1)
        return 1;
    return -1;
}

```

<function acomcmp2 332a>≡ (528a)

```
int
acomcmp2(const void *a1, const void *a2)
{
    vlong c1, c2;
    Term *t1, *t2;

    t1 = (Term*)a1;
    t2 = (Term*)a2;
    c1 = t1->mult;
    c2 = t2->mult;
    if(c1 > c2)
        return 1;
    if(c1 < c2)
        return -1;
    if(t2 > t1)
        return 1;
    return -1;
}
```

<function acom2 332b>≡ (528a)

```
void
acom2(Node *n, Type *t)
{
    Node *l, *r;
    Term trm[NTERM];
    int et, nt, i, j;
    vlong c1, c2;

    /*
     * copy into automatic
     */
    c2 = 0;
    nt = nterm;
    for(i=0; i<nt; i++)
        trm[i] = term[i];
    /*
     * recur on subtrees
     */
    j = 0;
    for(i=1; i<nt; i++) {
        c1 = trm[i].mult;
        if(c1 == 0)
            continue;
        l = trm[i].node;
        if(l != Z) {
            j = 1;
            acom(l);
        }
    }
    c1 = trm[0].mult;
    if(j == 0) {
        n->oldop = n->op;
        n->op = OCONST;
        n->vconst = c1;
        return;
    }
    et = t->etype;

    /*
```

```

* prepare constant term,
* combine it with an addressing term
*/
if(c1 != 0) {
    l = new1(OCONST, Z, Z);
    l->type = t;
    l->vconst = c1;
    trm[0].mult = 1;
    for(i=1; i<nt; i++) {
        if(trm[i].mult != 1)
            continue;
        r = trm[i].node;
        if(r->op != OADDR)
            continue;
        r->type = t;
        l = new1(OADD, r, l);
        l->type = t;
        trm[i].mult = 0;
        break;
    }
    trm[0].node = l;
}
/*
* look for factorable terms
* c1*i + c1*c2*j -> c1*(i + c2*j)
*/
qsort(trm+1, nt-1, sizeof(trm[0]), acomcmp1);
for(i=nt-1; i>=0; i--) {
    c1 = trm[i].mult;
    if(c1 < 0)
        c1 = -c1;
    if(c1 <= 1)
        continue;
    for(j=i+1; j<nt; j++) {
        c2 = trm[j].mult;
        if(c2 < 0)
            c2 = -c2;
        if(c2 <= 1)
            continue;
        if(c2 % c1)
            continue;
        r = trm[j].node;
        if(r->type->etype != et)
            r = acast(t, r);
        c2 = trm[j].mult/trm[i].mult;
        if(c2 != 1 && c2 != -1) {
            r = new1(OMUL, r, new(OCONST, Z, Z));
            r->type = t;
            r->right->type = t;
            r->right->vconst = c2;
        }
        l = trm[i].node;
        if(l->type->etype != et)
            l = acast(t, l);
        r = new1(OADD, l, r);
        r->type = t;
        if(c2 == -1)
            r->op = OSUB;
        trm[i].node = r;
        trm[j].mult = 0;
    }
}

```

```

    }
}
if(debug['m']) {
    print("\n");
    for(i=0; i<nt; i++) {
        print("%d %3lld ", i, trm[i].mult);
        prttree1(trm[i].node, 1, 0);
    }
}

/*
 * put it all back together
 */
qsort(trm+1, nt-1, sizeof(trm[0]), acomcmp2);
l = Z;
for(i=nt-1; i>=0; i--) {
    c1 = trm[i].mult;
    if(c1 == 0)
        continue;
    r = trm[i].node;
    if(r->type->etype != et || r->op == OBIT)
        r = acast(t, r);
    if(c1 != 1 && c1 != -1) {
        r = new1(OMUL, r, new(OCONST, Z, Z));
        r->type = t;
        r->right->type = t;
        if(c1 < 0) {
            r->right->vconst = -c1;
            c1 = -1;
        } else {
            r->right->vconst = c1;
            c1 = 1;
        }
    }
}
if(l == Z) {
    l = r;
    c2 = c1;
    continue;
}
if(c1 < 0)
    if(c2 < 0)
        l = new1(OADD, l, r);
    else
        l = new1(OSUB, l, r);
else
    if(c2 < 0) {
        l = new1(OSUB, r, l);
        c2 = 1;
    } else
        l = new1(OADD, l, r);
l->type = t;
}
if(c2 < 0) {
    r = new1(OCONST, 0, 0);
    r->vconst = 0;
    r->type = t;
    l = new1(OSUB, r, l);
    l->type = t;
}
*n = *l;

```

}

Uses NTERM 328a, OADD 114c, OADDR 116e, OBIT 418d, OCONST 111a, OMUL 114c, OSUB 114c, Z 29a, acast() 331a, acom() 328e, acomcmp1() 331b, acomcmp2() 332a, debug 474a, new() 29d, new1() 29h, nterm 328d, prttree1() 487c, and term 328c.

Chapter 13

Assembly-level Optimisations

Assembly-level optimizations work on the generated `Prog` instruction list. These include nop detection (removing redundant `MOV R0, R0`), ARM-specific instruction selection (using `RSB` when the operand order allows a reverse subtract), the peephole optimizer (`peep`) that combines instruction sequences, and the register allocator (`regopt`) that tries to keep frequently-used variables in registers. Use `5c -N` to disable these optimizations and see the raw output.

13.1 Nop detection

The simplest optimization: when `gmove` would emit a move from a register to itself (`MOVW R0, R0`), it returns without emitting anything. This happens frequently, for example when a function's first parameter is already in `R0` (the return register) and the body is `return x`.

```
<gmove() if samaddr return 336a>≡ (214)
    if(a == AMOVW || a == AMOVF || a == AMOVD)
        if(samaddr(f, t))
            return;
```

```
<function samaddr 336b>≡ (537b)
    bool
    samaddr(Node *f, Node *t)
    {

        if(f->op != t->op)
            return false;
        switch(f->op) {
        case OREGISTER:
            return (f->reg == t->reg);
        }
        return false;
    }
```

Uses `OREGISTER 208f`.

13.2 ARM special instructions

ARM has a reverse-subtract instruction (`RSB`) that computes `b - a` instead of `a - b`, which avoids needing to swap operands into registers. When a subtraction has a constant on the left (`5 - x`), `gopcode` uses `RSB` instead of `SUB` to avoid loading the constant first.

```
<gopcode() when OSUB, ARSB opportunity 336c>≡ (230c)
    if(f2 && f2->op == OCONST) {
```

```

Node *t = f1;
f1 = f2;
f2 = t;
a = ARSB;
}

```

Uses OCONST 111a.

13.3 Register passing argument: REGARG

On ARM, the first word-sized argument is passed in R0 (= REGARG). At function entry, `codgen` immediately saves R0 to the parameter's stack slot so the rest of the function can treat it like any other variable. This spill is removed by the register allocator if the parameter can stay in a register throughout.

```

⟨codgen() locals 337a⟩+≡ (194c) ◁194d
Node nod, nod1;

```

```

⟨codgen() if use REGARG 337b⟩≡ (194c)
/*
 * isolate first argument
 */
if(REGARG >= 0) {
  ⟨codegen() if use REGARG, if complex return type 273a⟩
  else
  if(firstarg && typeword[firstargtype->etype]) {
    nod1 = znodel;
    nod1.op = ONAME;
    nod1.sym = firstarg;
    nod1.type = firstargtype;
    nod1.etype = firstargtype->etype;
    nod1.class = CPARAM;
    nod1.xoffset = align(0, firstargtype, Aarg1);
    xcom(&nodel);
    nodreg(&nodel, &nodel, REGARG);
    gmove(&nodel, &nodel);
  }
}

```

Uses Aarg1 276b, CPARAM 33b, ONAME 27, firstarg 274j, firstargtype 275a, and typeword 337c.

```

⟨global typeword 337c⟩≡ (511d)
// set<type_kind>
char* typeword;

```

```

⟨tinit() initialise 32 bits defaults type sets 337d⟩+≡ (50a) ◁271b
/* 32-bit defaults */
typeword = typechlp;

```

Uses typechlp 215g and typeword 337c.

```

⟨cgenrel() OFUNC case, if use REGARG, part one 337e⟩≡ (257)
if(REGARG >= 0)
  o = reg[REGARG];

```

```

⟨cgenrel() OFUNC case, if use REGARG, part two 337f⟩≡ (257)
if(REGARG >= 0)
  if(o != reg[REGARG])
    reg[REGARG]--;

```

```

⟨garg1() if use REGARG and curarg is zero and simple type 338a⟩≡ (259a)
    if(REGARG >= 0 && curarg == 0 && typechlp[n->type->etype]) {
        regaalloc1(tn1, n);
        if(n->complex >= FNX) {
            cgen(*fnxp, tn1);
            (*fnxp)++;
        } else
            cgen(n, tn1);
    }

```

Uses typechlp 215g.

```

⟨function regaalloc1(arm) 338b⟩≡ (537b)
    void
    regaalloc1(Node *n, Node *nn)
    {
        nodreg(n, nn, REGARG);
        reg[REGARG]++;
        curarg = align(curarg, nn->type, Aarg1);
        curarg = align(curarg, nn->type, Aarg2);
        maxargsafe = maxround(maxargsafe, cursafe+curarg);
    }

```

Uses Aarg1 276b and Aarg2 276b.

13.4 Register allocation optimisations

The register allocator is a post-pass that promotes frequently-used local variables from the stack into registers. It runs in 9 passes: passes 1–3 build liveness information using a dataflow analysis over the instruction list, pass 4 estimates the “cost” (benefit) of registerizing each variable, pass 5 assigns registers to the most profitable variables, and passes 6–9 rewrite the instructions to use the assigned registers. The allocator works on **Var** and **Reg** structures that map variables to registers and track which registers are available.

Why a separate post-pass? Recall from the **cgen** chapter that the inline code generator only does local register allocation: a temporary lives from its **regalloc** to its **regfree**, and at any function call all caller-save registers are spilled. The result of **cgen** is correct but conservative—a local **int x** declared at the top of a function is loaded from **xoffset(SP)** every time it is read and stored back every time it is written, even when it sits live across the whole function with no intervening call. The job of **regopt** is to hoist that local into a real register, replacing every **MOV xoffset(SP), R0** with a register reference and deleting the now-dead loads and stores. On a register-rich machine like ARM (10 callee-save registers in the kernel calling convention), this is typically a 30–50% speedup on integer-heavy code.

Here is the conceptual flow of the 9 passes:

```

        instruction list from cgen
            |
            v
    pass 1 ---> build Reg nodes; for each instr, record
                variables-used and variables-set
            |
            v
    pass 2 ---> resolve branch targets to Reg* pointers,
                link successors and predecessors
            |
            v
    pass 2.5 --> identify natural loops via dominator analysis

```

```

      |
      v
pass 3 ---> backwards liveness dataflow, fixed-point
            iteration; result: at each Reg, which vars
            are live
      |
      v
pass 4 ---> forward synchrony: which vars hold the
            same value as which registers
      |
      v
pass 5 ---> for each variable, walk its live region
            and compute a benefit score (loops weight
            5x via the log5 sampling); sort by score
      |
      v
pass 6 ---> greedy assignment: for each region in
            decreasing-score order, pick a free
            register and rewrite all instructions in
            the region to use it (paint2/paint3)
      |
      v
pass 7 ---> peephole on the rewritten code
      |
      v
pass 8 ---> recompute pc values, fix branch targets
      |
      v
pass 9 ---> final cleanup and emit

```

The interesting design choice is the greedy pass 6, not graph coloring. Classic register allocators (e.g., Chaitin’s algorithm in gcc) build an interference graph and color it; if coloring fails, they iteratively spill and retry. 5c takes a much simpler route: “compute a benefit score for each variable, sort, walk in decreasing order, pick the first free register that covers the variable’s live range.” This misses some opportunities a coloring allocator would catch, but it runs in $O(n \log n)$ instead of cubic time, and—in practice on the small functions typical of Plan 9 code—produces nearly identical assembly. Speed of compilation was always a higher priority than peak optimization in 5c.

13.4.1 regopt()

```

⟨codgen() register optimisation 339a⟩≡ (194c)
    if(!debug['N'] || debug['R'] || debug['P'])
        regopt(sp);

```

Uses debug 474a.

```

⟨function regopt(arm) 339b⟩≡ (541)
    void
    regopt(Prog *p)
    {
        ⟨regopt() locals 340l⟩

        ⟨regopt() initialisation 341a⟩
    }

```

```

    <regopt() pass 1 341f>
    <regopt() pass 2 344a>
    <regopt() pass 2.5 344b>
    <regopt() pass 3 345a>
    <regopt() pass 4 345b>
    <regopt() pass 5 346>
    <regopt() pass 6 347>
    <regopt() pass 7 348a>
    <regopt() pass 8 348b>
    <regopt() pass 9 349a>
}

<global externs 340a>≡ (535b)
    Bits externs;

<global params 340b>≡ (535b)
    Bits params;

<global consts 340c>≡ (535b)
    Bits consts;

<global addr 340d>≡ (535b)
    Bits addr;

<global regbits 340e>≡ (535b)
    long regbits;

<global exregbits 340f>≡ (535b)
    long exregbits;

<global change 340g>≡ (535b)
    int change;

<global first 340h>≡ (535b)
    Reg* first;

<global last 340i>≡ (535b)
    Reg* last;

<global zreg 340j>≡ (535b)
    Reg zreg;

<global freer 340k>≡ (535b)
    Reg* freer;

<regopt() locals 340l>≡ (339b)
    Reg *r, *r1, *r2;
    Prog *p1;
    int i, z;
    long initpc, val, npc;
    ulong vreg;
    Bits bit;
    struct
    {
        long m;
        long c;
        Reg* p;
    } log5[6], *lp;

```

```

⟨regopt() initialisation 341a⟩≡ (339b)
    firstr = R;
    lastr = R;
    nvar = 0;
    regbits = 0;
    for(z=0; z<BITS; z++) {
        externs.b[z] = 0;
        params.b[z] = 0;
        consts.b[z] = 0;
        addrs.b[z] = 0;
    }

```

Uses BITS 349b.

13.4.2 noretval()

`noretval` emits fake NOP instructions that reference `REGRET` (integer return register) and `FREGRET` (float return register). These are not real instructions—they serve as sentinels for the register allocator, marking the point where the return value must be in the return register. Without them, the optimizer might move a value out of `REGRET` before the `RET` instruction reads it.

```

⟨codgen() before RET 341b⟩≡ (194c)
    noretval(1 | 2);

```

```

⟨gen() case ORETURN with no argument, before RET 341c⟩≡ (206b)
    noretval(1 | 2);

```

```

⟨gen() case ORETURN with argument, before RET 341d⟩≡ (206b)
    if(typefd[n->type->etype])
        noretval(1);
    else
        noretval(2);

```

Uses `typefd` 149c.

```

⟨function noretval(arm) 341e⟩≡ (540a)
    void
    noretval(int n)
    {

        if(n & 1) {
            gins(ANOP, Z, Z);
            p->to.type = D_REG;
            p->to.reg = REGRET;
        }
        if(n & 2) {
            gins(ANOP, Z, Z);
            p->to.type = D_FREG;
            p->to.reg = FREGRET;
        }
    }

```

Uses Z 29a.

13.4.3 Pass 1

Pass 1 builds the control-flow graph: for each instruction, it creates a `Reg` node recording which variables are used and which are set (defined). It also identifies the “log5” sampling points for estimating loop nesting depth.

```

⟨regopt() pass 1 341f⟩≡ (339b)

```

```

/*
 * pass 1
 * build aux data structure
 * allocate pcs
 * find use and set of variables
 */
val = 5L * 5L * 5L * 5L * 5L;
lp = log5;
for(i=0; i<5; i++) {
    lp->m = val;
    lp->c = 0;
    lp->p = R;
    val /= 5L;
    lp++;
}
val = 0;
for(; p != P; p = p->link) {
    switch(p->as) {
        case ADATA:
        case AGLOBL:
        case ANAME:
        case ASIGNAME:
            continue;
    }
    r = rega();
    if(firststr == R) {
        firststr = r;
        lastr = r;
    } else {
        lastr->link = r;
        r->p1 = lastr;
        lastr->s1 = r;
        lastr = r;
    }
    r->prog = p;
    r->pc = val;
    val++;

    lp = log5;
    for(i=0; i<5; i++) {
        lp->c--;
        if(lp->c <= 0) {
            lp->c = lp->m;
            if(lp->p != R)
                lp->p->log5 = r;
            lp->p = r;
            (lp+1)->c = 0;
            break;
        }
        lp++;
    }

    r1 = r->p1;
    if(r1 != R)
        switch(r1->prog->as) {
            case ARET:
            case AB:
            case ARFE:
                r->p1 = R;
                r1->s1 = R;

```

```

}

/*
 * left side always read
 */
bit = mkvar(&p->from, p->as==AMOVW);
for(z=0; z<BITS; z++)
    r->use1.b[z] |= bit.b[z];

/*
 * right side depends on opcode
 */
bit = mkvar(&p->to, 0);
if(bany(&bit))
switch(p->as) {
default:
    diag(Z, "reg: unknown asop: %A", p->as);
    break;

/*
 * right side write
 */
case ANOP:
case AMOVB:
case AMOVBU:
case AMOVH:
case AMOVHU:
case AMOVW:
case AMOVF:
case AMOVD:
    for(z=0; z<BITS; z++)
        r->set.b[z] |= bit.b[z];
    break;

/*
 * funny
 */
case ABL:
    for(z=0; z<BITS; z++)
        addrs.b[z] |= bit.b[z];
    break;
}

if(p->as == AMOVM) {
    if(p->from.type == D_CONST)
        z = p->from.offset;
    else
        z = p->to.offset;
    for(i=0; z; i++) {
        if(z&1)
            regbits |= RtoB(i);
        z >>= 1;
    }
}

}

if(firststr == R)
    return;
initpc = pc - val;
npc = val;

```


Uses BITS 349b, Z 29a, bany() 350c, and diag() 496b.

13.4.4 Pass 2

Pass 2 resolves branch targets: each branch instruction's pc offset is converted into a pointer to the corresponding Reg node, and successor/predecessor links are built.

```
<regopt() pass 2 344a>≡ (339b)
/*
 * pass 2
 * turn branch references to pointers
 * build back pointers
 */
for(r = firststr; r != R; r = r->link) {
    p = r->prog;
    if(p->to.type == D_BRANCH) {
        val = p->to.offset - initpc;
        r1 = firststr;
        while(r1 != R) {
            r2 = r1->log5;
            if(r2 != R && val >= r2->pc) {
                r1 = r2;
                continue;
            }
            if(r1->pc == val)
                break;
            r1 = r1->link;
        }
        if(r1 == R) {
            nearln = p->lineno;
            diag(Z, "ref not found\nnP", p);
            continue;
        }
        if(r1 == r) {
            nearln = p->lineno;
            diag(Z, "ref to self\nnP", p);
            continue;
        }
        r->s2 = r1;
        r->p2link = r1->p2;
        r1->p2 = r;
    }
}
if(debug['R']) {
    p = firststr->prog;
    print("\n%L %D\n", p->lineno, &p->from);
}
```

Uses Z 29a, debug 474a, diag() 496b, and nearln 29g.

```
<regopt() pass 2.5 344b>≡ (339b)
/*
 * pass 2.5
 * find looping structure
 */
for(r = firststr; r != R; r = r->link)
    r->active = 0;
change = 0;
loopit(firststr, npc);
```

13.4.5 Pass 3

Pass 3 is the liveness dataflow: starting from RET instructions (where no variables are live-out), it propagates use/set information backwards through the control-flow graph until a fixed point is reached. The result is that each Reg node knows which variables are live at that point.

```
<regopt() pass 3 345a>≡ (339b)
/*
 * pass 3
 * iterate propagating usage
 * back until flow graph is complete
 */
loop1:
  change = 0;
  for(r = firststr; r != R; r = r->link)
    r->active = 0;
  for(r = firststr; r != R; r = r->link)
    if(r->prog->as == ARET)
      prop(r, zbits, zbits);
loop11:
  /* pick up unreachable code */
  i = 0;
  for(r = firststr; r != R; r = r1) {
    r1 = r->link;
    if(r1 && r1->active && !r->active) {
      prop(r, zbits, zbits);
      i = 1;
    }
  }
  if(i)
    goto loop11;
  if(change)
    goto loop1;
Uses zbits 350a.
```

13.4.6 Pass 4

Pass 4 propagates register/variable synchronization forward: it determines for each point in the program which variables currently hold the same value as which registers. This is needed so that when a variable is promoted to a register, the allocator knows where the register's value is still valid.

```
<regopt() pass 4 345b>≡ (339b)
/*
 * pass 4
 * iterate propagating register/variable synchrony
 * forward until graph is complete
 */
loop2:
  change = 0;
  for(r = firststr; r != R; r = r->link)
    r->active = 0;
  synch(firststr, zbits);
  if(change)
    goto loop2;

  addsplits();

  if(debug['R'] && debug['v']) {
    print("\nprop structure:\n");
```

```

for(r = first; r != R; r = r->link) {
    print("%ld:%P", r->loop, r->prog);
    for(z=0; z<BITS; z++)
        bit.b[z] = r->set.b[z] |
            r->refahead.b[z] | r->calahead.b[z] |
            r->refbehind.b[z] | r->calbehind.b[z] |
            r->use1.b[z] | r->use2.b[z];
    if(bany(&bit)) {
        print("\t");
        if(bany(&r->use1))
            print(" u1=%B", r->use1);
        if(bany(&r->use2))
            print(" u2=%B", r->use2);
        if(bany(&r->set))
            print(" st=%B", r->set);
        if(bany(&r->refahead))
            print(" ra=%B", r->refahead);
        if(bany(&r->calahead))
            print(" ca=%B", r->calahead);
        if(bany(&r->refbehind))
            print(" rb=%B", r->refbehind);
        if(bany(&r->calbehind))
            print(" cb=%B", r->calbehind);
    }
    print("\n");
}
}

```

Uses BITS 349b, addsplits() 354a, bany() 350c, debug 474a, and zbits 350a.

13.4.7 pass 5

Pass 5 identifies “regions” — connected portions of the program where a variable is live — and computes the cost (benefit) of registerizing each variable, weighted by loop nesting depth. Variables with the highest benefit-to-cost ratio are chosen for register promotion.

```

⟨regopt() pass 5 346⟩≡ (339b)
/*
 * pass 5
 * isolate regions
 * calculate costs (paint1)
 */
r = first;
if(r) {
    for(z=0; z<BITS; z++)
        bit.b[z] = (r->refahead.b[z] | r->calahead.b[z]) &
            ~(externs.b[z] | params.b[z] | addrs.b[z] | consts.b[z]);
    if(bany(&bit)) {
        nearln = r->prog->lineno;
        warn(Z, "used and not set: %B", bit);
        if(debug['R'] && !debug['w'])
            print("used and not set: %B\n", bit);
    }
}

for(r = first; r != R; r = r->link)
    r->act = zbits;
rgp = region;
nregion = 0;
for(r = first; r != R; r = r->link) {

```

```

for(z=0; z<BITS; z++)
    bit.b[z] = r->set.b[z] &
        ~(r->refahead.b[z] | r->calahead.b[z] | addrs.b[z]);
if(bany(&bit)) {
    nearln = r->prog->lineno;
    warn(Z, "set and not used: %B", bit);
    if(debug['R'])
        print("set and not used: %B\n", bit);
    excise(r);
}
for(z=0; z<BITS; z++)
    bit.b[z] = LOAD(r) & ~(r->act.b[z] | addrs.b[z]);
while(bany(&bit)) {
    i = bnum(bit);
    rgp->enter = r;
    rgp->varno = i;
    change = 0;
    if(debug['R'] && debug['v'])
        print("\n");
    paint1(r, i);
    bit.b[i/32] &= ~(1L<<(i%32));
    if(change <= 0) {
        if(debug['R'])
            print("%L $%d: %B\n",
                r->prog->lineno, change, blsh(i));
        continue;
    }
    rgp->cost = change;
    nregion++;
    if(nregion >= NRGN) {
        warn(Z, "too many regions");
        goto brk;
    }
    rgp++;
}
}
brk:
    qsort(region, nregion, sizeof(region[0]), rcmp);
Uses BITS 349b, Z 29a, bany() 350c, blsh() 351a, bnum() 350e, debug 474a, nearln 29g, warn() 142, and zbits 350a.

```

13.4.8 pass 6

Pass 6 assigns physical registers to the most profitable regions (sorted by cost from pass 5) and rewrites the instructions: memory references to the promoted variable are replaced with register references.

```

⟨regopt() pass 6 347⟩≡ (339b)
/*
 * pass 6
 * determine used registers (paint2)
 * replace code (paint3)
 */
rgp = region;
for(i=0; i<nregion; i++) {
    bit = blsh(rgp->varno);
    vreg = paint2(rgp->enter, rgp->varno);
    vreg = allreg(vreg, rgp);
    if(debug['R']) {
        if(rgp->regno >= NREG)
            print("%L $%d F%d: %B\n",

```

```

        rgp->enter->prog->lineno,
        rgp->cost,
        rgp->regno-NREG,
        bit);
    else
        print("%L $%d R%d: %B\n",
            rgp->enter->prog->lineno,
            rgp->cost,
            rgp->regno,
            bit);
}
if(rgp->regno != 0)
    paint3(rgp->enter, rgp->varno, vreg, rgp->regno);
rgp++;
}

```

Uses blsh() 351a and debug 474a.

13.4.9 pass 7, peep hole optimizations

Pass 7 runs the peephole optimizer (`peep`) on the modified instruction list. Peephole optimization is disabled when `-R` is used alone (to isolate register allocation effects).

`<regopt() pass 7 348a>≡` (339b)

```

/*
 * pass 7
 * peep-hole on basic block
 */
if(!debug['R'] || debug['P'])
    peep();

```

Uses debug 474a.

13.4.10 pass 8

Pass 8 recalculates PC values for each instruction (since register promotion and peephole may have added or removed instructions), then fixes up branch targets so they point to the correct PCs.

`<regopt() pass 8 348b>≡` (339b)

```

/*
 * pass 8
 * recalculate pc
 */
val = initpc;
for(r = firststr; r != R; r = r1) {
    r->pc = val;
    p = r->prog;
    p1 = P;
    r1 = r->link;
    if(r1 != R)
        p1 = r1->prog;
    for(; p != p1; p = p->link) {
        switch(p->as) {
            default:
                val++;
                break;

            case ANOP:
            case ADATA:
            case AGLOBL:

```

```

        case ANAME:
        case ASIGNAME:
            break;
    }
}
}
pc = val;

/*
 * fix up branches
 */
if(debug['R'])
    if(bany(&addrs))
        print("addrs: %B\n", addrs);

r1 = 0; /* set */
for(r = firststr; r != R; r = r->link) {
    p = r->prog;
    if(p->to.type == D_BRANCH)
        p->to.offset = r->s2->pc;
    r1 = r;
}

```

Uses `bany()` 350c and `debug` 474a.

13.4.11 pass 9

The final pass strips NOP instructions from the linked list (they were used as placeholders) and frees the `Reg` auxiliary structures.

```

⟨regopt() pass 9 349a⟩≡ (339b)
/*
 * last pass
 * eliminate nops
 * free aux structures
 */
for(p = firststr->prog; p != P; p = p->link){
    while(p->link && p->link->as == ANOP)
        p->link = p->link->link;
}
if(r1 != R) {
    r1->link = freer;
    freer = firststr;
}

```

13.4.12 Bits

`Bits` is a fixed-size bitset used by the register allocator to track variable liveness. Each bit corresponds to a `Var` (a candidate for registerization). The dataflow analysis uses set operations (`bor`, `band`, `bnum`) on these bitsets to propagate “live-in” and “live-out” information through the control-flow graph.

```

⟨constant BITS 349b⟩≡ (504b)
#define BITS 5

```

```

⟨struct Bits 349c⟩≡ (504b)
struct Bits
{
    ulong b[BITS];
};

```

<global zbits 350a>≡ (511d)
 Bits zbits;

<function bor 350b>≡ (521a)
 Bits
 bor(Bits a, Bits b)
 {
 Bits c;
 int i;

 for(i=0; i<BITS; i++)
 c.b[i] = a.b[i] | b.b[i];
 return c;
 }

Uses BITS 349b.

<function bany 350c>≡ (521a)
 /*
 Bits
 bnot(Bits a)
 {
 Bits c;
 int i;

 for(i=0; i<BITS; i++)
 c.b[i] = ~a.b[i];
 return c;
 }
 */

 int
 bany(Bits *a)
 {
 int i;

 for(i=0; i<BITS; i++)
 if(a->b[i])
 return 1;
 return 0;
 }

Uses BITS 349b.

<function beq 350d>≡ (521a)
 int
 beq(Bits a, Bits b)
 {
 int i;

 for(i=0; i<BITS; i++)
 if(a.b[i] != b.b[i])
 return 0;
 return 1;
 }

Uses BITS 349b.

<function bnum 350e>≡ (521a)
 int
 bnum(Bits a)
 {

```

    int i;
    long b;

    for(i=0; i<BITS; i++)
        if(b = a.b[i])
            return 32*i + bitno(b);
    diag(Z, "bad in bnum");
    return 0;
}

```

Uses BITS 349b, Z 29a, bitno() 501d, and diag() 496b.

```

⟨function blsh 351a⟩≡ (521a)
    Bits
    blsh(uint n)
    {
        Bits c;

        c = zbits;
        c.b[n/32] = 1L << (n%32);
        return c;
    }

```

Uses zbits 350a.

```

⟨function bset 351b⟩≡ (521a)
    // @Scheck: also defined as a macro in gc.h, TODO fix that!
    int bset(Bits a, uint n)
    {
        if(a.b[n/32] & (1L << (n%32)))
            return 1;
        return 0;
    }

```

```

⟨function bset(arm) 351c⟩≡ (531b)
    #define bset(a,n) ((a).b[(n)/32]&(1L<<(n)%32))

```

13.4.13 Var

A **Var** identifies a candidate for register promotion: it records the symbol, its type, and its stack offset. The global **var** array holds up to **NVAR** (160) variables, each represented by a single bit in the **Bits** bitset that the dataflow analysis manipulates.

```

⟨constant NVAR 351d⟩≡ (504b)
    #define NVAR (BITS*sizeof(ulong)*8)

```

```

⟨global nvar 351e⟩≡ (535b)
    int nvar;

```

```

⟨global var 351f⟩≡ (535b)
    Var var[NVAR];

```

Uses NVAR 351d.

```

⟨struct Var 351g⟩≡ (531b)
    struct Var
    {
        Sym* sym;
        //enum<sym_kind>
        char symkind;
        //enum<type_kind>
        char etype;
        long offset;
    };

```


13.4.14 Reg

A **Reg** node represents one instruction in the control-flow graph. It carries the dataflow sets (**use1**, **use2**, **set**), liveness information (**refahead**, **refbehind**, **calahead**, **calbehind**), loop nesting depth, and CFG edges (**p1/s1** for fallthrough, **p2/s2** for branches). The **log5** field is a skip-list pointer for faster branch target resolution.

<struct Reg 352a>≡ (531b)

```
struct Reg
{
    long pc;
    long rpo; /* reverse post ordering */

    Bits set;
    Bits use1;
    Bits use2;

    Bits refbehind;
    Bits refahead;
    Bits calbehind;
    Bits calahead;
    Bits regdiff;
    Bits act;

    long regu;
    long loop; /* could be shorter */

    Reg* log5;
    long active;

    Reg* p1;
    Reg* p2;
    Reg* p2link;
    Reg* s1;
    Reg* s2;
    Reg* link;

    Prog* prog;
};
```

<constant R 352b>≡ (531b)

```
#define R ((Reg*)nil)
```

13.4.15 XXX

This subsection collects the helper functions for the register allocator: **rega/freea** manage a free list of **Reg** nodes, **mkvar** maps an **Adr** operand to a **Var** slot (creating one if needed), **prop/synch** implement the backward and forward dataflow propagation, **paint1/paint2/paint3** assign registers to variables by painting regions, **loopit** computes loop nesting depth, and **addsplits** inserts MOV instructions at points where a register's value must be synchronized with its stack slot.

<function rega 352c>≡ (541)

```
Reg*
rega(void)
{
    Reg *r;

    r = freer;
    if(r == R) {
```

```

        r = alloc(sizeof(*r));
    } else
        freer = r->link;

    *r = zreg;
    return r;
}

```

Uses alloc() 500b.

<function rcmp 353a>≡ (541)

```

int
rcmp(const void *a1, const void *a2)
{
    Rgn *p1, *p2;
    int c1, c2;

    p1 = (Rgn*)a1;
    p2 = (Rgn*)a2;
    c1 = p2->cost;
    c2 = p1->cost;
    if(c1 != c2)
        return c1;
    return p2->varno - p1->varno;
}

```

<function LOAD(arm) 353b>≡ (531b)

```

#define LOAD(r) (~r->refbehind.b[z] & r->refahead.b[z])

```

<function STORE(arm) 353c>≡ (531b)

```

#define STORE(r) (~r->calbehind.b[z] & r->calahead.b[z])

```

<constant NRGN(arm) 353d>≡ (531b)

```

#define NRGN 1000 /* was 600; raised for paranoia.c */

```

<global region 353e>≡ (535b)

```

Rgn region[NRGN];

```

<global rgp 353f>≡ (535b)

```

Rgn* rgp;

```

<struct Rgn 353g>≡ (531b)

```

struct Rgn
{
    Reg* enter;
    short cost;
    short varno;
    short regno;
};

```

<global nregion 353h>≡ (535b)

```

int nregion;

```

*<function addsplits(*arm*) 354a>*≡ (541)

```
void
addsplits(void)
{
    Reg *r, *r1;
    int z, i;
    Bits bit;

    for(r = firststr; r != R; r = r->link) {
        if(r->loop > 1)
            continue;
        if(r->prog->as == ABL)
            continue;
        for(r1 = r->p2; r1 != R; r1 = r1->p2link) {
            if(r1->loop <= 1)
                continue;
            for(z=0; z<BITS; z++)
                bit.b[z] = r1->calbehind.b[z] &
                    (r->refahead.b[z] | r->use1.b[z] | r->use2.b[z]) &
                    ~(r->calahead.b[z] & addrs.b[z]);
            while(bany(&bit)) {
                i = bnum(bit);
                bit.b[i/32] &= ~(1L << (i%32));
            }
        }
    }
}
```

Uses BITS 349b, bany() 350c, and bnum() 350e.

*<function addmove(*arm*) 354b>*≡ (541)

```
/*
 * add mov b,rn
 * just after r
 */
void
addmove(Reg *r, int bn, int rn, int f)
{
    Prog *p, *p1;
    Adr *a;
    Var *v;

    p1 = alloc(sizeof(Prog));
    *p1 = zprog;
    p = r->prog;

    p1->link = p->link;
    p->link = p1;
    p1->lineno = p->lineno;

    v = var + bn;

    a = &p1->to;
    a->sym = v->sym;
    a->symkind = v->symkind;
    a->offset = v->offset;
    a->etype = v->etype;
    a->type = D_OREG;
    if(v->etype == TARRAY || a->sym == S)
        a->type = D_CONST;
```

```

p1->as = AMOVW;
if(v->etype == TCHAR || v->etype == TUCCHAR)
    p1->as = AMOVB;
if(v->etype == TSHORT || v->etype == TUSHORT)
    p1->as = AMOVH;
if(v->etype == TFLOAT)
    p1->as = AMOVF;
if(v->etype == TDOUBLE)
    p1->as = AMOVD;

p1->from.type = D_REG;
p1->from.reg = rn;
if(rn >= NREG) {
    p1->from.type = D_FREG;
    p1->from.reg = rn-NREG;
}
if(!f) {
    p1->from = *a;
    *a = zprog.from;
    a->type = D_REG;
    a->reg = rn;
    if(rn >= NREG) {
        a->type = D_FREG;
        a->reg = rn-NREG;
    }
    if(v->etype == TUCCHAR)
        p1->as = AMOVBU;
    if(v->etype == TUSHORT)
        p1->as = AMOVHU;
}
if(debug['R'])
    print("%P\t.a%P\n", p, p1);
}

```

Uses S 35d, TARRAY 31c, TCHAR 30b, TDOUBLE 31a, TFLOAT 31a, TSHORT 30b, TUCCHAR 30b, TUSHORT 30b, alloc() 500b, and debug 474a.

⟨function mkvar(*arm*) 355⟩≡ (541)

```

Bits
mkvar(Adr *a, int docon)
{
    Var *v;
    int i, n, z;
    long o;
    Bits bit;
    Sym *s;
    //enum<operand_kind>
    int t;
    //enum<type_kind>
    int et;

    t = a->type;
    if(t == D_REG && a->reg != R_NONE)
        regbits |= RtoB(a->reg);
    if(t == D_FREG && a->reg != R_NONE)
        regbits |= FtoB(a->reg);
    s = a->sym;
    o = a->offset;
    et = a->etype;
    if(s == S) {
        if(t != D_CONST || !docon || a->reg != R_NONE)

```

```

        goto none;
    et = TLONG;
}
if(t == D_CONST) {
    if(s == S && sval(o))
        goto none;
}

n = a->symkind;
v = var;
for(i=0; i<nvar; i++) {
    if(s == v->sym)
        if(n == v->symkind)
            if(o == v->offset)
                goto out;
    v++;
}
if(s)
    if(s->name[0] == '.')
        goto none;
if(nvar >= NVAR) {
    if(debug['w'] > 1 && s)
        warn(Z, "variable not optimized: %s", s->name);
    goto none;
}
i = nvar;
nvar++;
v = &var[i];
v->sym = s;
v->offset = o;
v->etype = et;
v->symkind = n;
if(debug['R'])
    print("bit=%2d et=%2d %D\n", i, et, a);
out:
bit = blsh(i);
if(n == N_EXTERN || n == N_INTERN)
    for(z=0; z<BITS; z++)
        externs.b[z] |= bit.b[z];
if(n == N_PARAM)
    for(z=0; z<BITS; z++)
        params.b[z] |= bit.b[z];
if(v->etype != et || !typechlpfd[et]) /* funny punning */
    for(z=0; z<BITS; z++)
        addrs.b[z] |= bit.b[z];
if(t == D_CONST) {
    if(s == S) {
        for(z=0; z<BITS; z++)
            consts.b[z] |= bit.b[z];
        return bit;
    }
    if(et != TARRAY)
        for(z=0; z<BITS; z++)
            addrs.b[z] |= bit.b[z];
    for(z=0; z<BITS; z++)
        params.b[z] |= bit.b[z];
    return bit;
}
if(t == D_OREG)
    return bit;

```

```

none:
    return zbits;
}

Uses BITS 349b, FtoB() 366a, NVAR 351d, S 35d, TARRAY 31c, TLONG 30b, Z 29a, blsh() 351a, debug 474a, sval() 357c,
typechlpfd 357a, warn() 142, and zbits 350a.

⟨global typechlpfd 357a⟩≡ (518c)
    char typechlpfd[NTYPE];

Uses NTYPE 30a.

⟨global typechlpfdinit 357b⟩≡ (518c)
    int typechlpfdinit[] =
    {
        TCHAR, TUCCHAR, TSHORT, TUSHORT, TINT, TUINT, TLONG, TULONG, TFLOAT, TDOUBLE, TIND, -1,
    };

Uses TCHAR 30b, TDOUBLE 31a, TFLOAT 31a, TIND 31c, TINT 30b, TLONG 30b, TSHORT 30b, TUCCHAR 30b, TUINT 30b, TULONG 30b,
and TUSHORT 30b.

⟨function sval(arm) 357c⟩≡ (537b)
    int
    sval(long v)
    {
        int i;

        for(i=0; i<16; i++) {
            if((v & ~0xff) == 0)
                return 1;
            if((~v & ~0xff) == 0)
                return 1;
            v = (v<<2) | ((ulong)v>>30);
        }
        return 0;
    }

⟨function prop(arm) 357d⟩≡ (541)
    void
    prop(Reg *r, Bits ref, Bits cal)
    {
        Reg *r1, *r2;
        int z;

        for(r1 = r; r1 != R; r1 = r1->p1) {
            for(z=0; z<BITS; z++) {
                ref.b[z] |= r1->refahead.b[z];
                if(ref.b[z] != r1->refahead.b[z]) {
                    r1->refahead.b[z] = ref.b[z];
                    change++;
                }
                cal.b[z] |= r1->calahead.b[z];
                if(cal.b[z] != r1->calahead.b[z]) {
                    r1->calahead.b[z] = cal.b[z];
                    change++;
                }
            }
            switch(r1->prog->as) {
            case ABL:
                for(z=0; z<BITS; z++) {
                    cal.b[z] |= ref.b[z] | externs.b[z];

```

```

        ref.b[z] = 0;
    }
    break;

case ATEXT:
    for(z=0; z<BITS; z++) {
        cal.b[z] = 0;
        ref.b[z] = 0;
    }
    break;

case ARET:
    for(z=0; z<BITS; z++) {
        cal.b[z] = externs.b[z];
        ref.b[z] = 0;
    }
}
for(z=0; z<BITS; z++) {
    ref.b[z] = (ref.b[z] & ~r1->set.b[z]) |
        r1->use1.b[z] | r1->use2.b[z];
    cal.b[z] &= ~(r1->set.b[z] | r1->use1.b[z] | r1->use2.b[z]);
    r1->refbehind.b[z] = ref.b[z];
    r1->calbehind.b[z] = cal.b[z];
}
if(r1->active)
    break;
r1->active = 1;
}
for(; r != r1; r = r->p1)
    for(r2 = r->p2; r2 != R; r2 = r2->p2link)
        prop(r2, r->refbehind, r->calbehind);
}

```

Uses BITS 349b.

```

⟨function rpolca 358a⟩≡ (541)
    long
    rpolca(long *idom, long rpo1, long rpo2)
    {
        long t;

        if(rpo1 == -1)
            return rpo2;
        while(rpo1 != rpo2){
            if(rpo1 > rpo2){
                t = rpo2;
                rpo2 = rpo1;
                rpo1 = t;
            }
            while(rpo1 < rpo2){
                t = idom[rpo2];
                if(t >= rpo2)
                    fatal(Z, "bad idom");
                rpo2 = t;
            }
        }
        return rpo1;
    }
}

```

Uses Z 29a and fatal() 496c.

```

⟨function doms 358b⟩≡ (541)

```

```

int
doms(long *idom, long r, long s)
{
    while(s > r)
        s = idom[s];
    return s == r;
}

⟨function loophead 359a⟩≡ (541)
int
loophead(long *idom, Reg *r)
{
    long src;

    src = r->rpo;
    if(r->p1 != R && doms(idom, src, r->p1->rpo))
        return 1;
    for(r = r->p2; r != R; r = r->p2link)
        if(doms(idom, src, r->rpo))
            return 1;
    return 0;
}

⟨constant LOOP(arm) 359b⟩≡ (531b)
#define LOOP 3

⟨function loopmark 359c⟩≡ (541)
void
loopmark(Reg **rpo2r, long head, Reg *r)
{
    if(r->rpo < head || r->active == head)
        return;
    r->active = head;
    r->loop += LOOP;
    if(r->p1 != R)
        loopmark(rpo2r, head, r->p1);
    for(r = r->p2; r != R; r = r->p2link)
        loopmark(rpo2r, head, r);
}

⟨global idom 359d⟩≡ (535b)
long* idom;

⟨global rpo2r 359e⟩≡ (535b)
Reg** rpo2r;

⟨global maxnr 359f⟩≡ (535b)
long maxnr;

⟨function loopit 359g⟩≡ (541)
void
loopit(Reg *r, long nr)
{
    Reg *r1;
    long i, d, me;

    if(nr > maxnr) {
        rpo2r = alloc(nr * sizeof(Reg*));
        idom = alloc(nr * sizeof(long));
        maxnr = nr;
    }
}

```



```

}

d = postorder(r, rpo2r, 0);
if(d > nr)
    fatal(Z, "too many reg nodes");
nr = d;
for(i = 0; i < nr / 2; i++){
    r1 = rpo2r[i];
    rpo2r[i] = rpo2r[nr - 1 - i];
    rpo2r[nr - 1 - i] = r1;
}
for(i = 0; i < nr; i++)
    rpo2r[i]->rpo = i;

idom[0] = 0;
for(i = 0; i < nr; i++){
    r1 = rpo2r[i];
    me = r1->rpo;
    d = -1;
    if(r1->p1 != R && r1->p1->rpo < me)
        d = r1->p1->rpo;
    for(r1 = r1->p2; r1 != nil; r1 = r1->p2link)
        if(r1->rpo < me)
            d = rpolca(idom, d, r1->rpo);
    idom[i] = d;
}

for(i = 0; i < nr; i++){
    r1 = rpo2r[i];
    r1->loop++;
    if(r1->p2 != R && loophead(idom, r1))
        loopmark(rpo2r, i, r1);
}
}

Uses Z 29a, alloc() 500b, and fatal() 496c.

⟨function synch 360⟩≡ (541)
void
synch(Reg *r, Bits dif)
{
    Reg *r1;
    int z;

    for(r1 = r; r1 != R; r1 = r1->s1) {
        for(z=0; z<BITS; z++) {
            dif.b[z] = (dif.b[z] &
                ~(~r1->refbehind.b[z] & r1->refahead.b[z])) |
                r1->set.b[z] | r1->regdiff.b[z];
            if(dif.b[z] != r1->regdiff.b[z]) {
                r1->regdiff.b[z] = dif.b[z];
                change++;
            }
        }
        if(r1->active)
            break;
        r1->active = 1;
        for(z=0; z<BITS; z++)
            dif.b[z] &= ~(~r1->calbehind.b[z] & r1->calahead.b[z]);
        if(r1->s2 != R)
            synch(r1->s2, dif);
    }
}

```

```
}

```

```
}

```

Uses BITS 349b.

$\langle$ function allreg(*arm*) 361a $\rangle \equiv$ (541)

```
ulong
allreg(ulong b, Rgn *r)
{
    Var *v;
    int i;

    v = var + r->varno;
    r->regno = 0;
    switch(v->etype) {

    case TCHAR:
    case TUCCHAR:
    case TSHORT:
    case TUSHORT:
    case TINT:
    case TUINT:
    case TLONG:
    case TULONG:
    case TIND:
    case TARRAY:
        i = BtoR(~b);
        if(i && r->cost >= 0) {
            r->regno = i;
            return RtoB(i);
        }
        break;

    case TVLONG:
    case TDOUBLE:
    case TFLOAT:
        i = BtoF(~b);
        if(i && r->cost >= 0) {
            r->regno = i+NREG;
            return FtoB(i);
        }
        break;

    default:
        diag(Z, "unknown etype %d/%d", bitno(b), v->etype);
        break;

    }
    return 0;
}
```

Uses BtoF() 366b, FtoB() 366a, TARRAY 31c, TCHAR 30b, TDOUBLE 31a, TFLOAT 31a, TIND 31c, TINT 30b, TLONG 30b, TSHORT 30b, TUCCHAR 30b, TUINT 30b, TULONG 30b, TUSHORT 30b, TVLONG 30b, Z 29a, bitno() 501d, and diag() 496b.

$\langle$ constant CLOAD(*arm*) 361b $\rangle \equiv$ (531b)

```
#define CLOAD 4
```

$\langle$ constant CREF(*arm*) 361c $\rangle \equiv$ (531b)

```
#define CREF 5
```

⟨function paint1(*arm*) 362)≡

(541)

```

void
paint1(Reg *r, int bn)
{
    Reg *r1;
    Prog *p;
    int z;
    ulong bb;

    z = bn/32;
    bb = 1L<<(bn%32);
    if(r->act.b[z] & bb)
        return;
    for(;;) {
        if(!(r->refbehind.b[z] & bb))
            break;
        r1 = r->p1;
        if(r1 == R)
            break;
        if(!(r1->refahead.b[z] & bb))
            break;
        if(r1->act.b[z] & bb)
            break;
        r = r1;
    }

    if(LOAD(r) & ~(r->set.b[z] & ~(r->use1.b[z]|r->use2.b[z])) & bb) {
        change -= CLOAD * r->loop;
        if(debug['R'] && debug['v'])
            print("%ld%P\tld %B %d\n", r->loop,
                r->prog, blsh(bn), change);
    }
    for(;;) {
        r->act.b[z] |= bb;
        p = r->prog;

        if(r->use1.b[z] & bb) {
            change += CREF * r->loop;
            if(debug['R'] && debug['v'])
                print("%ld%P\tu1 %B %d\n", r->loop,
                    p, blsh(bn), change);
        }

        if((r->use2.b[z]|r->set.b[z]) & bb) {
            change += CREF * r->loop;
            if(debug['R'] && debug['v'])
                print("%ld%P\tu2 %B %d\n", r->loop,
                    p, blsh(bn), change);
        }

        if(STORE(r) & r->regdiff.b[z] & bb) {
            change -= CLOAD * r->loop;
            if(debug['R'] && debug['v'])
                print("%ld%P\tst %B %d\n", r->loop,
                    p, blsh(bn), change);
        }

        if(r->refbehind.b[z] & bb)
            for(r1 = r->p2; r1 != R; r1 = r1->p2link)
                if(r1->refahead.b[z] & bb)

```

```

        paint1(r1, bn);

    if(!(r->refahead.b[z] & bb))
        break;
    r1 = r->s2;
    if(r1 != R)
        if(r1->refbehind.b[z] & bb)
            paint1(r1, bn);
    r = r->s1;
    if(r == R)
        break;
    if(r->act.b[z] & bb)
        break;
    if(!(r->refbehind.b[z] & bb))
        break;
}
}

```

Uses blsh() 351a and debug 474a.

```

⟨function paint2(arm) 363⟩≡ (541)
    ulong
    paint2(Reg *r, int bn)
    {
        Reg *r1;
        int z;
        ulong bb, vreg;

        z = bn/32;
        bb = 1L << (bn%32);
        vreg = regbits;
        if(!(r->act.b[z] & bb))
            return vreg;
        for(;;) {
            if(!(r->refbehind.b[z] & bb))
                break;
            r1 = r->p1;
            if(r1 == R)
                break;
            if(!(r1->refahead.b[z] & bb))
                break;
            if(!(r1->act.b[z] & bb))
                break;
            r = r1;
        }
        for(;;) {
            r->act.b[z] &= ~bb;

            vreg |= r->regu;

            if(r->refbehind.b[z] & bb)
                for(r1 = r->p2; r1 != R; r1 = r1->p2link)
                    if(r1->refahead.b[z] & bb)
                        vreg |= paint2(r1, bn);

            if(!(r->refahead.b[z] & bb))
                break;
            r1 = r->s2;
            if(r1 != R)
                if(r1->refbehind.b[z] & bb)
                    vreg |= paint2(r1, bn);
        }
    }

```

```

    r = r->s1;
    if(r == R)
        break;
    if(!(r->act.b[z] & bb))
        break;
    if(!(r->refbehind.b[z] & bb))
        break;
}
return vreg;
}

```

⟨function paint3 364⟩≡ (541)

```

void
paint3(Reg *r, int bn, long rb, int rn)
{
    Reg *r1;
    Prog *p;
    int z;
    ulong bb;

    z = bn/32;
    bb = 1L << (bn%32);
    if(r->act.b[z] & bb)
        return;
    for(;;) {
        if(!(r->refbehind.b[z] & bb))
            break;
        r1 = r->p1;
        if(r1 == R)
            break;
        if(!(r1->refahead.b[z] & bb))
            break;
        if(r1->act.b[z] & bb)
            break;
        r = r1;
    }

    if(LOAD(r) & ~(r->set.b[z] & ~(r->use1.b[z]|r->use2.b[z])) & bb)
        addmove(r, bn, rn, 0);
    for(;;) {
        r->act.b[z] |= bb;
        p = r->prog;

        if(r->use1.b[z] & bb) {
            if(debug['R'])
                print("%P", p);
            addreg(&p->from, rn);
            if(debug['R'])
                print("\t.c%P\n", p);
        }
        if((r->use2.b[z]|r->set.b[z]) & bb) {
            if(debug['R'])
                print("%P", p);
            addreg(&p->to, rn);
            if(debug['R'])
                print("\t.c%P\n", p);
        }

        if(STORE(r) & r->regdiff.b[z] & bb)
            addmove(r, bn, rn, 1);
    }
}

```

```

r->regu |= rb;

if(r->refbehind.b[z] & bb)
    for(r1 = r->p2; r1 != R; r1 = r1->p2link)
        if(r1->refahead.b[z] & bb)
            paint3(r1, bn, rb, rn);

if(!(r->refahead.b[z] & bb))
    break;
r1 = r->s2;
if(r1 != R)
    if(r1->refbehind.b[z] & bb)
        paint3(r1, bn, rb, rn);
r = r->s1;
if(r == R)
    break;
if(r->act.b[z] & bb)
    break;
if(!(r->refbehind.b[z] & bb))
    break;
}
}

```

Uses debug 474a.

```

⟨function addreg(arm) 365a⟩≡ (541)
void
addreg(Adr *a, int rn)
{

```

```

    a->sym = 0;
    a->symkind = N_NONE;
    a->type = D_REG;
    a->reg = rn;
    if(rn >= NREG) {
        a->type = D_FREG;
        a->reg = rn-NREG;
    }
}

```

```

⟨function RtoB(arm) 365b⟩≡ (541)
/*

```

```

 * bit reg
 * 0 R0
 * 1 R1
 * ... ...
 * 10 R10
 */

```

```

long
RtoB(int r)
{

```

```

    if(r >= REGMIN && r <= REGMAX)
        return 1L << r;
    return 0;
}

```

```

⟨function BtoR(arm) 365c⟩≡ (541)
int

```

```

BtoR(long b)
{

```

```

    b &= 0x01fcL; // excluded R9 and R10 for m and g
    if(b == 0)
        return 0;
    return bitno(b);
}

```

Uses bitno() 501d.

<function FtoB(arm) 366a> ≡ (541)

```

/*
 * bit reg
 * 18 F2
 * 19 F3
 * ... ...
 * 23 F7
 */
long
FtoB(int f)
{
    if(f < 2 || f > NFREG-1)
        return 0;
    return 1L << (f + 16);
}

```

<function BtoF(arm) 366b> ≡ (541)

```

int
BtoF(long b)
{
    b &= 0xfc0000L;
    if(b == 0)
        return 0;
    return bitno(b) - 16;
}

```

Uses bitno() 501d.

13.5 Peephole optimizer: peep()

The peephole optimizer scans the instruction list looking for small patterns that can be improved: redundant moves (MOV R1,R2; MOV R2,R1 → MOV R1,R2), constant propagation (replacing a register with its known constant value), copy propagation (bypassing intermediate copies), and ARM-specific tricks like folding shifts into operands. It iterates until no more improvements are found.

<function peep(arm) 366c> ≡ (538)

```

/// codgen -> regopt -> <>
void
peep(void)
{
    Reg *r, *r1, *r2;
    Prog *p, *p1;
    int t;
/*
 * complete R structure
 */
    t = 0;
    for(r=firststr; r!=R; r=r1) {
        r1 = r->link;
    }
}

```

```

    if(r1 == R)
        break;
    p = r->prog->link;
    while(p != r1->prog)
        switch(p->as) {
        default:
            r2 = rega();
            r->link = r2;
            r2->link = r1;

            r2->prog = p;
            r2->p1 = r;
            r->s1 = r2;
            r2->s1 = r1;
            r1->p1 = r2;

            r = r2;
            t++;

        case ADATA:
        case AGLOBL:
        case ANAME:
        case ASIGNAME:
            p = p->link;
        }
    }

loop1:
    t = 0;
    for(r=firststr; r!=R; r=r->link) {
        p = r->prog;
        if(p->as == ASLL || p->as == ASRL || p->as == ASRA) {
            /*
             * elide shift into D_SHIFT operand of subsequent instruction
             */
            if(shiftprop(r)) {
                excise(r);
                t++;
            }
        }
        if(p->as == AMOVW || p->as == AMOVF || p->as == AMOVD)
            if(regtyp(&p->to)) {
                if(p->from.type == D_CONST)
                    constprop(&p->from, &p->to, r->s1);
                else if(regtyp(&p->from))
                    if(p->from.type == p->to.type) {
                        if(copyprop(r)) {
                            excise(r);
                            t++;
                        } else
                            if(subprop(r) && copyprop(r)) {
                                excise(r);
                                t++;
                            }
                    }
            }
    }
    if(t)
        goto loop1;
    /*

```



```

    * look for MOVB x,R; MOVB R,R
    */
for(r=first; r!=R; r=r->link) {
    p = r->prog;
    switch(p->as) {
    default:
        continue;
    case AEOR:
        /*
         * EOR -1,x,y => MVN x,y
         */
        if(p->from.type == D_CONST && p->from.offset == -1) {
            p->as = AMVN;
            p->from.type = D_REG;
            if(p->reg != R_NONE)
                p->from.reg = p->reg;
            else
                p->from.reg = p->to.reg;
            p->reg = R_NONE;
        }
        continue;
    case AMOVH:
    case AMOVHU:
    case AMOVB:
    case AMOVBU:
        if(p->to.type != D_REG)
            continue;
        break;
    }
    r1 = r->link;
    if(r1 == R)
        continue;
    p1 = r1->prog;
    if(p1->as != p->as)
        continue;
    if(p1->from.type != D_REG || p1->from.reg != p->to.reg)
        continue;
    if(p1->to.type != D_REG || p1->to.reg != p->to.reg)
        continue;
    excise(r1);
}

for(r=first; r!=R; r=r->link) {
    p = r->prog;
    switch(p->as) {
    case AMOVW:
    case AMOVB:
    case AMOVBU:
        if(p->from.type == D_OREG && p->from.offset == 0)
            xtramodes(r, &p->from);
        else if(p->to.type == D_OREG && p->to.offset == 0)
            xtramodes(r, &p->to);
        else
            continue;
        break;
    case ACMP:
        /*
         * elide CMP $0,x if calculation of x can set condition codes
         */
        if(p->from.type != D_CONST || p->from.offset != 0)

```

```

        continue;
r2 = r->s1;
if(r2 == R)
    continue;
t = r2->prog->as;
switch(t) {
default:
    continue;
case ABEQ:
case ABNE:
case ABMI:
case ABPL:
    break;
case ABGE:
    t = ABPL;
    break;
case ABLT:
    t = ABMI;
    break;
case ABHI:
    t = ABNE;
    break;
case ABLS:
    t = ABEQ;
    break;
}
r1 = r;
do
    r1 = uniqp(r1);
while (r1 != R && r1->prog->as == ANOP);
if(r1 == R)
    continue;
p1 = r1->prog;
if(p1->to.type != D_REG)
    continue;
if(p1->to.reg != p->reg)
if(!(p1->as == AMOVW && p1->from.type == D_REG && p1->from.reg == p->reg))
    continue;
switch(p1->as) {
default:
    continue;
case AMOVW:
    if(p1->from.type != D_REG)
        continue;
case AAND:
case AEOR:
case AORR:
case ABIC:
case AMVN:
case ASUB:
case ARSB:
case AADD:
case AADC:
case ASBC:
case ARSC:
    break;
}
p1->scond |= C_SBIT;
r2->prog->as = t;
excise(r);

```

```

        continue;
    }
}

predicate();
}

```

Uses `constprop()` 374, `predicate()` 391a, `shiftprop()` 375b, and `xtramodes()` 380b.

```

⟨function excise(arm) 370a⟩≡ (538)
void
excise(Reg *r)
{
    Prog *p;

    p = r->prog;
    p->as = ANOP;
    p->scond = zprog.scond;
    p->from = zprog.from;
    p->to = zprog.to;
    p->reg = zprog.reg; /**/
}

```

```

⟨function uniqp 370b⟩≡ (538)
Reg*
uniqp(Reg *r)
{
    Reg *r1;

    r1 = r->p1;
    if(r1 == R) {
        r1 = r->p2;
        if(r1 == R || r1->p2link != R)
            return R;
    } else
        if(r->p2 != R)
            return R;
    return r1;
}

```

```

⟨function uniqs 370c⟩≡ (538)
Reg*
uniqs(Reg *r)
{
    Reg *r1;

    r1 = r->s1;
    if(r1 == R) {
        r1 = r->s2;
        if(r1 == R)
            return R;
    } else
        if(r->s2 != R)
            return R;
    return r1;
}

```

```

⟨function regtyp(arm) 370d⟩≡ (538)
int
regtyp(Adr *a)
{

```

```

    if(a->type == D_REG)
        return 1;
    if(a->type == D_FREG)
        return 1;
    return 0;
}

```

*<function subprop(*arm*) 371>*≡ (538)

```

/*
 * the idea is to substitute
 * one register for another
 * from one MOV to another
 * MOV a, R0
 * ADD b, R0 / no use of R1
 * MOV R0, R1
 * would be converted to
 * MOV a, R1
 * ADD b, R1
 * MOV R1, R0
 * hopefully, then the former or latter MOV
 * will be eliminated by copy propagation.
 */
int
subprop(Reg *r0)
{
    Prog *p;
    Adr *v1, *v2;
    Reg *r;
    int t;

    p = r0->prog;
    v1 = &p->from;
    if(!regtyp(v1))
        return 0;
    v2 = &p->to;
    if(!regtyp(v2))
        return 0;
    for(r=uniqp(r0); r!=R; r=uniqp(r)) {
        if(uniqs(r) == R)
            break;
        p = r->prog;
        switch(p->as) {
            case ABL:
                return 0;

            case ACMP:
            case ACMN:
            case AADD:
            case ASUB:
            case ARSB:
            case ASLL:
            case ASRL:
            case ASRA:
            case AORR:
            case AAND:
            case AEOR:
            case AMUL:
            case ADIV:
            case ADIVU:

```

```

case ACMPF:
case ACMPD:
case AADDD:
case AADDF:
case ASUBD:
case ASUBF:
case AMULD:
case AMULF:
case ADIVD:
case ADIVF:
    if(p->to.type == v1->type)
    if(p->to.reg == v1->reg) {
        if(p->reg == R_NONE)
            p->reg = p->to.reg;
        goto gotit;
    }
    break;

case AMOVF:
case AMOVD:
case AMOVW:
    if(p->to.type == v1->type)
    if(p->to.reg == v1->reg)
        goto gotit;
    break;

case AMOVM:
    t = 1<<v2->reg;
    if((p->from.type == D_CONST && (p->from.offset&t)) ||
        (p->to.type == D_CONST && (p->to.offset&t)))
        return 0;
    break;
}
if(copyau(&p->from, v2) ||
    copyau1(p, v2) ||
    copyau(&p->to, v2))
    break;
if(copysub(&p->from, v1, v2, 0) ||
    copysub1(p, v1, v2, 0) ||
    copysub(&p->to, v1, v2, 0))
    break;
}
return 0;

gotit:
copysub(&p->to, v1, v2, 1);
if(debug['P']) {
    print("gotit: %D->%D\n%P", v1, v2, r->prog);
    if(p->from.type == v2->type)
        print(" excise");
    print("\n");
}
for(r=uniqs(r); r!=r0; r=uniqs(r)) {
    p = r->prog;
    copysub(&p->from, v1, v2, 1);
    copysub1(p, v1, v2, 1);
    copysub(&p->to, v1, v2, 1);
    if(debug['P'])
        print("%P\n", r->prog);
}

```

```

}
t = v1->reg;
v1->reg = v2->reg;
v2->reg = t;
if(debug['P'])
    print("%P last\n", r->prog);
return 1;
}

```

Uses copyau1() 386c, copysub1() 387b, and debug 474a.

<function copyprop 373a>≡ (538)

```

/*
 * The idea is to remove redundant copies.
 * v1->v2 F=0
 * (use v2 s/v2/v1/)*
 * set v1 F=1
 * use v2 return fail
 * -----
 * v1->v2 F=0
 * (use v2 s/v2/v1/)*
 * set v1 F=1
 * set v2 return success
 */
int
copyprop(Reg *r0)
{
    Prog *p;
    Adr *v1, *v2;
    Reg *r;

    p = r0->prog;
    v1 = &p->from;
    v2 = &p->to;
    if(copyas(v1, v2))
        return 1;
    for(r=firststr; r!=R; r=r->link)
        r->active = 0;
    return copy1(v1, v2, r0->s1, 0);
}

```

<function copy1 373b>≡ (538)

```

int
copy1(Adr *v1, Adr *v2, Reg *r, int f)
{
    int t;
    Prog *p;

    if(r->active) {
        if(debug['P'])
            print("act set; return 1\n");
        return 1;
    }
    r->active = 1;
    if(debug['P'])
        print("copy %D->%D f=%d\n", v1, v2, f);
    for(; r != R; r = r->s1) {
        p = r->prog;
        if(debug['P'])
            print("%P", p);
        if(!f && uniqp(r) == R) {

```

```

    f = 1;
    if(debug['P'])
        print("; merge; f=%d", f);
}
t = copyu(p, v2, A);
switch(t) {
case 2: /* rar, cant split */
    if(debug['P'])
        print("; %Drar; return 0\n", v2);
    return 0;

case 3: /* set */
    if(debug['P'])
        print("; %Dset; return 1\n", v2);
    return 1;

case 1: /* used, substitute */
case 4: /* use and set */
    if(f) {
        if(!debug['P'])
            return 0;
        if(t == 4)
            print("; %Dused+set and f=%d; return 0\n", v2, f);
        else
            print("; %Dused and f=%d; return 0\n", v2, f);
        return 0;
    }
    if(copyu(p, v2, v1)) {
        if(debug['P'])
            print("; sub fail; return 0\n");
        return 0;
    }
    if(debug['P'])
        print("; sub%D/%D", v2, v1);
    if(t == 4) {
        if(debug['P'])
            print("; %Dused+set; return 1\n", v2);
        return 1;
    }
    break;
}
if(!f) {
    t = copyu(p, v1, A);
    if(!f && (t == 2 || t == 3 || t == 4)) {
        f = 1;
        if(debug['P'])
            print("; %Dset and !f; f=%d", v1, f);
    }
}
if(debug['P'])
    print("\n");
if(r->s2)
    if(!copy1(v1, v2, r->s2, f))
        return 0;
}
return 1;
}

```

Uses debug 474a.

$\langle \text{function constprop}(\text{arm}) \text{ 374} \rangle \equiv$

(538)

```

/*
 * The idea is to remove redundant constants.
 * $c1->v1
 * ($c1->v2 s/$c1/v1)*
 * set v1 return
 * The v1->v2 should be eliminated by copy propagation.
 */
void
constprop(Adr *c1, Adr *v1, Reg *r)
{
    Prog *p;

    if(debug['C'])
        print("constprop %D->%D\n", c1, v1);
    for(; r != R; r = r->s1) {
        p = r->prog;
        if(debug['C'])
            print("%P", p);
        if(uniqp(r) == R) {
            if(debug['C'])
                print("; merge; return\n");
            return;
        }
        if(p->as == AMOVW && copyas(&p->from, c1)) {
            if(debug['C'])
                print("; sub%D/%D", &p->from, v1);
            p->from = *v1;
        } else if(copyu(p, v1, A) > 1) {
            if(debug['C'])
                print("; %Dset; return\n", v1);
            return;
        }
        if(debug['C'])
            print("\n");
        if(r->s2)
            constprop(c1, v1, r->s2);
    }
}

```

Uses constprop() 374 and debug 474a.

⟨function FAIL(*arm*) 375a)≡ (538)

```

/*
 * ASLL x,y,w
 * .. (not use w, not set x y w)
 * AXXX w,a,b (a != w)
 * .. (not use w)
 * (set w)
 * ----- changed to
 * ..
 * AXXX (x<<y),a,b
 * ..
 */
#define FAIL(msg) { if(debug['H']) print("\t%s; FAILURE\n", msg); return 0; }

```

⟨function shiftprop(*arm*) 375b)≡ (538)

```

int
shiftprop(Reg *r)
{
    Reg *r1;
    Prog *p, *p1, *p2;

```



```

int n, o;
Adr a;

p = r->prog;
if(p->to.type != D_REG)
    FAIL("BOTCH: result not reg");
n = p->to.reg;
a = zprog.from;
if(p->reg != R_NONE && p->reg != p->to.reg) {
    a.type = D_REG;
    a.reg = p->reg;
}
if(debug['H'])
    print("shiftprop\nnP", p);
r1 = r;
for(;;) {
    /* find first use of shift result; abort if shift operands or result are changed */
    r1 = uniqs(r1);
    if(r1 == R)
        FAIL("branch");
    if(uniqp(r1) == R)
        FAIL("merge");
    p1 = r1->prog;
    if(debug['H'])
        print("\nP", p1);
    switch(copyu(p1, &p->to, A)) {
    case 0: /* not used or set */
        if((p->from.type == D_REG && copyu(p1, &p->from, A) > 1) ||
            (a.type == D_REG && copyu(p1, &a, A) > 1))
            FAIL("args modified");
        continue;
    case 3: /* set, not used */
        FAIL("BOTCH: noref");
    }
    break;
}
/* check whether substitution can be done */
switch(p1->as) {
default:
    FAIL("non-dpi");
case AAND:
case AEOR:
case AADD:
case AADC:
case AORR:
case ASUB:
case ARSB:
case ASBC:
case ARSC:
    if(p1->reg == n || (p1->reg == R_NONE && p1->to.type == D_REG && p1->to.reg == n)) {
        if(p1->from.type != D_REG)
            FAIL("can't swap");
        p1->reg = p1->from.reg;
        p1->from.reg = n;
        switch(p1->as) {
        case ASUB:
            p1->as = ARSB;
            break;
        case ARSB:
            p1->as = ASUB;

```

```

        break;
    case ASBC:
        p1->as = ARSC;
        break;
    case ARSC:
        p1->as = ASBC;
        break;
    }
    if(debug['H'])
        print("\t=>%P", p1);
}
case ABIC:
case ACMP:
case ACMN:
    if(p1->reg == n)
        FAIL("can't swap");
    if(p1->reg == R_NONE && p1->to.reg == n)
        FAIL("shift result used twice");
case AMVN:
    if(p1->from.type == D_SHIFT)
        FAIL("shift result used in shift");
    if(p1->from.type != D_REG || p1->from.reg != n)
        FAIL("BOTCH: where is it used?");
    break;
}
/* check whether shift result is used subsequently */
p2 = p1;
if(p1->to.reg != n)
for (;;) {
    r1 = uniqs(r1);
    if(r1 == R)
        FAIL("inconclusive");
    p1 = r1->prog;
    if(debug['H'])
        print("\n%P", p1);
    switch(copyu(p1, &p->to, A)) {
    case 0: /* not used or set */
        continue;
    case 3: /* set, not used */
        break;
    default: /* used */
        FAIL("reused");
    }
    break;
}
/* make the substitution */
p2->from.type = D_SHIFT;
p2->from.reg = R_NONE;
o = p->reg;
if(o == R_NONE)
    o = p->to.reg;
switch(p->from.type){
case D_CONST:
    o |= (p->from.offset&0x1f)<<7;
    break;
case D_REG:
    o |= (1<<4) | (p->from.reg<<8);
    break;
}
switch(p->as){

```

```

case ASLL:
    o |= 0<<5;
    break;
case ASRL:
    o |= 1<<5;
    break;
case ASRA:
    o |= 2<<5;
    break;
}
p2->from.offset = o;
if(debug['H'])
    print("\t=>%P\tSUCCEED\n", p2);
return 1;
}

```

Uses FAIL-194 375a and debug 474a.

*<function findpre(*arm*) 378a>*≡ (538)

```

Reg*
findpre(Reg *r, Adr *v)
{
    Reg *r1;

    for(r1=uniqp(r); r1!=R; r=r1,r1=uniqp(r)) {
        if(uniqs(r1) != r)
            return R;
        switch(copyu(r1->prog, v, A)) {
            case 1: /* used */
            case 2: /* read-alter-rewrite */
                return R;
            case 3: /* set */
            case 4: /* set and used */
                return r1;
        }
    }
    return R;
}

```

*<function findinc(*arm*) 378b>*≡ (538)

```

Reg*
findinc(Reg *r, Reg *r2, Adr *v)
{
    Reg *r1;
    Prog *p;

    for(r1=uniqs(r); r1!=R && r1!=r2; r=r1,r1=uniqs(r)) {
        if(uniqp(r1) != r)
            return R;
        switch(copyu(r1->prog, v, A)) {
            case 0: /* not touched */
                continue;
            case 4: /* set and used */
                p = r1->prog;
                if(p->as == AADD)
                    if(p->from.type == D_CONST)
                        if(p->from.offset > -4096 && p->from.offset < 4096)
                            return r1;
            default:
                return R;
        }
    }
}

```

```

    }
}
return R;
}

```

<function nochange(arm) 379a>≡ (538)

```

int
nochange(Reg *r, Reg *r2, Prog *p)
{
    Adr a[3];
    int i, n;

    if(r == r2)
        return 1;
    n = 0;
    if(p->reg != R_NONE && p->reg != p->to.reg) {
        a[n].type = D_REG;
        a[n++].reg = p->reg;
    }
    switch(p->from.type) {
    case D_SHIFT:
        a[n].type = D_REG;
        a[n++].reg = p->from.offset&0xf;
    case D_REG:
        a[n].type = D_REG;
        a[n++].reg = p->from.reg;
    }
    if(n == 0)
        return 1;
    for(; r!=R && r!=r2; r=uniqu(r)) {
        p = r->prog;
        for(i=0; i<n; i++)
            if(copyu(p, &a[i], A) > 1)
                return 0;
    }
    return 1;
}

```

<function findu1(arm) 379b>≡ (538)

```

int
findu1(Reg *r, Adr *v)
{
    for(; r != R; r = r->s1) {
        if(r->active)
            return 0;
        r->active = 1;
        switch(copyu(r->prog, v, A)) {
        case 1: /* used */
        case 2: /* read-alter-rewrite */
        case 4: /* set and used */
            return 1;
        case 3: /* set */
            return 0;
        }
        if(r->s2)
            if (findu1(r->s2, v))
                return 1;
    }
    return 0;
}

```

Uses findu1() 379b.

```
(function finduse(arm) 380a)≡ (538)
  int
  finduse(Reg *r, Adr *v)
  {
    Reg *r1;

    for(r1=firststr; r1!=R; r1=r1->link)
      r1->active = 0;
    return findu1(r, v);
  }
```

Uses findu1() 379b.

```
(function xtramodes(arm) 380b)≡ (538)
  int
  xtramodes(Reg *r, Adr *a)
  {
    Reg *r1, *r2, *r3;
    Prog *p, *p1;
    Adr v;

    p = r->prog;
    if(debug['h'] && p->as == AMOVb && p->from.type == D_OREG) /* byte load */
      return 0;
    v = *a;
    v.type = D_REG;
    r1 = findpre(r, &v);
    if(r1 != R) {
      p1 = r1->prog;
      if(p1->to.type == D_REG && p1->to.reg == v.reg)
        switch(p1->as) {
          case AADD:
            if(p1->from.type == D_REG ||
               (p1->from.type == D_SHIFT && (p1->from.offset&(1<<4)) == 0 &&
                (p->as != AMOVb || (a == &p->from && (p1->from.offset&~0xf) == 0))) ||
               (p1->from.type == D_CONST &&
                p1->from.offset > -4096 && p1->from.offset < 4096))
              if(nochange(uniqu(r1), r, p1)) {
                if(a != &p->from || v.reg != p->to.reg)
                  if (finduse(r->s1, &v)) {
                    if(p1->reg == R_NONE || p1->reg == v.reg)
                      /* pre-indexing */
                      p->scond |= C_WBIT;
                    else return 0;
                  }
              }
            switch (p1->from.type) {
              case D_REG:
                /* register offset */
                a->type = D_SHIFT;
                a->offset = p1->from.reg;
                break;
              case D_SHIFT:
                /* scaled register offset */
                a->type = D_SHIFT;
              case D_CONST:
                /* immediate offset */
                a->offset = p1->from.offset;
                break;
            }
          }
    }
```

```

        if(p1->reg != R_NONE)
            a->reg = p1->reg;
        excise(r1);
        return 1;
    }
    break;
case AMOVW:
    if(p1->from.type == D_REG)
        if((r2 = findinc(r1, r, &p1->from)) != R) {
            for(r3=uniqs(r2); r3->prog->as==ANOP; r3=uniqs(r3))
                ;
            if(r3 == r) {
                /* post-indexing */
                p1 = r2->prog;
                a->reg = p1->to.reg;
                a->offset = p1->from.offset;
                p->scond |= C_PBIT;
                if(!finduse(r, &r1->prog->to))
                    excise(r1);
                excise(r2);
                return 1;
            }
        }
    break;
}
}
if(a != &p->from || a->reg != p->to.reg)
if((r1 = findinc(r, R, &v)) != R) {
    /* post-indexing */
    p1 = r1->prog;
    a->offset = p1->from.offset;
    p->scond |= C_PBIT;
    excise(r1);
    return 1;
}
return 0;
}

```

Uses debug 474a, findinc() 378b, findpre() 378a, finduse() 380a, and nochange() 379a.

⟨function copyu(arm) 381⟩≡ (538)

```

/*
 * return
 * 1 if v only used (and substitute),
 * 2 if read-alter-rewrite
 * 3 if set
 * 4 if set and used
 * 0 otherwise (not touched)
 */
int
copyu(Prog *p, Adr *v, Adr *s)
{
    switch(p->as) {

    default:
        if(debug['P'])
            print(" (???)");
        return 2;

    case AMOVM:

```

```

if(v->type != D_REG)
    return 0;
if(p->from.type == D_CONST) { /* read reglist, read/rar */
    if(s != A) {
        if(p->from.offset & (1 << v->reg))
            return 1;
        if(copysub(&p->to, v, s, 1))
            return 1;
        return 0;
    }
    if(copyau(&p->to, v)) {
        if(p->scond & C_WBIT)
            return 2;
        return 1;
    }
    if(p->from.offset & (1 << v->reg))
        return 1;
} else { /* read/rar, write reglist */
    if(s != A) {
        if(p->to.offset & (1 << v->reg))
            return 1;
        if(copysub(&p->from, v, s, 1))
            return 1;
        return 0;
    }
    if(copyau(&p->from, v)) {
        if(p->scond & C_WBIT)
            return 2;
        if(p->to.offset & (1 << v->reg))
            return 4;
        return 1;
    }
    if(p->to.offset & (1 << v->reg))
        return 3;
}
return 0;

```

```

case ANOP: /* read, write */
case AMOVW:
case AMOVF:
case AMOVD:
case AMOVH:
case AMOVHU:
case AMOVB:
case AMOVBU:
case AMOVDW:
case AMOVWD:
case AMOVFD:
case AMOVDF:
    if(p->scond & (C_WBIT | C_PBIT))
        if(v->type == D_REG) {
            if(p->from.type == D_OREG || p->from.type == D_SHIFT) {
                if(p->from.reg == v->reg)
                    return 2;
            } else {
                if(p->to.reg == v->reg)
                    return 2;
            }
        }
    if(s != A) {

```

```

        if(copysub(&p->from, v, s, 1))
            return 1;
        if(!copyas(&p->to, v))
            if(copysub(&p->to, v, s, 1))
                return 1;
        return 0;
    }
    if(copyas(&p->to, v)) {
        if(copyau(&p->from, v))
            return 4;
        return 3;
    }
    if(copyau(&p->from, v))
        return 1;
    if(copyau(&p->to, v))
        return 1;
    return 0;

```

```

case AADD: /* read, read, write */

```

```

case ASUB:

```

```

case ARSB:

```

```

case ASLL:

```

```

case ASRL:

```

```

case ASRA:

```

```

case AORR:

```

```

case AAND:

```

```

case AEOR:

```

```

case AMUL:

```

```

case ADIV:

```

```

case ADIVU:

```

```

case AADDF:

```

```

case AADDD:

```

```

case ASUBF:

```

```

case ASUBD:

```

```

case AMULF:

```

```

case AMULD:

```

```

case ADIVF:

```

```

case ADIVD:

```

```

case ACMPF:

```

```

case ACMPD:

```

```

case ACMP:

```

```

case ACMN:

```

```

case ACASE:

```

```

    if(s != A) {
        if(copysub(&p->from, v, s, 1))
            return 1;
        if(copysub1(p, v, s, 1))
            return 1;
        if(!copyas(&p->to, v))
            if(copysub(&p->to, v, s, 1))
                return 1;
        return 0;
    }
    if(copyas(&p->to, v)) {
        if(p->reg == R_NONE)
            p->reg = p->to.reg;
        if(copyau(&p->from, v))
            return 4;
    }

```



```

        if(copyau1(p, v))
            return 4;
        return 3;
    }
    if(copyau(&p->from, v))
        return 1;
    if(copyau1(p, v))
        return 1;
    if(copyau(&p->to, v))
        return 1;
    return 0;

case ABEQ: /* read, read */
case ABNE:
case ABHS:
case ABLO:
case ABMI:
case ABPL:
case ABVS:
case ABVC:
case ABHI:
case ABLT:
case ABGE:
case ABGT:
case ABLE:
    if(s != A) {
        if(copysub(&p->from, v, s, 1))
            return 1;
        return copysub1(p, v, s, 1);
    }
    if(copyau(&p->from, v))
        return 1;
    if(copyau1(p, v))
        return 1;
    return 0;

case AB: /* funny */
    if(s != A) {
        if(copysub(&p->to, v, s, 1))
            return 1;
        return 0;
    }
    if(copyau(&p->to, v))
        return 1;
    return 0;

case ARET: /* funny */
    if(v->type == D_REG)
    if(v->reg == REGRET)
        return 2;
    if(v->type == D_FREG)
    if(v->reg == FREGRET)
        return 2;

case ABL: /* funny */
    if(v->type == D_REG) {
        if(v->reg <= REGEXT && v->reg > exregoffset)
            return 2;
        if(v->reg == (uchar)REGARG)

```

```

        return 2;
    }
    if(v->type == D_FREG)
        if(v->reg <= FREGEXT && v->reg > exfregoffset)
            return 2;

    if(s != A) {
        if(copysub(&p->to, v, s, 1))
            return 1;
        return 0;
    }
    if(copyau(&p->to, v))
        return 4;
    return 3;

case ATEXT: /* funny */
    if(v->type == D_REG)
        if(v->reg == (uchar)REGARG)
            return 3;
    return 0;
}
}

```

Uses copyau1() 386c, copysub1() 387b, and debug 474a.

```

⟨function a2type(arm) 385⟩≡ (538)
    int
    a2type(Prog *p)
    {

        switch(p->as) {

            case ACMP:
            case ACMN:

            case AADD:
            case ASUB:
            case ARSB:
            case ASLL:
            case ASRL:
            case ASRA:
            case AORR:
            case AAND:
            case AEOR:
            case AMUL:
            case ADIV:
            case ADIVU:
                return D_REG;

            case ACMPF:
            case ACMPD:

            case AADDF:
            case AADDD:
            case ASUBF:
            case ASUBD:
            case AMULF:
            case AMULD:
            case ADIVF:
            case ADIVD:
                return D_FREG;
        }
    }

```

```

    }
    return D_NONE;
}

⟨function copyas(arm) 386a⟩≡ (538)
/*
 * direct reference,
 * could be set/use depending on
 * semantics
 */
int
copyas(Adr *a, Adr *v)
{
    if(regtyp(v)) {
        if(a->type == v->type)
            if(a->reg == v->reg)
                return 1;
    } else if(v->type == D_CONST) { /* for constprop */
        if(a->type == v->type)
            if(a->symkind == v->symkind)
                if(a->sym == v->sym)
                    if(a->reg == v->reg)
                        if(a->offset == v->offset)
                            return 1;
    }
    return 0;
}

⟨function copyau(arm) 386b⟩≡ (538)
/*
 * either direct or indirect
 */
int
copyau(Adr *a, Adr *v)
{
    if(copyas(a, v))
        return 1;
    if(v->type == D_REG) {
        if(a->type == D_OREG) {
            if(v->reg == a->reg)
                return 1;
        } else if(a->type == D_SHIFT) {
            if((a->offset&0xf) == v->reg)
                return 1;
            if((a->offset&(1<<4)) &&
                (a->offset>>8) == v->reg)
                return 1;
        }
    }
    return 0;
}

⟨function copyau1(arm) 386c⟩≡ (538)
int
copyau1(Prog *p, Adr *v)
{
    if(regtyp(v)) {

```

```

        if(a2type(p) == v->type)
        if(p->reg == v->reg) {
            if(a2type(p) != v->type)
                print("botch a2type %P\n", p);
            return 1;
        }
    }
    return 0;
}

```

Uses a2type() 385.

⟨function copysub(*arm*) 387a)≡ (538)

```

/*
 * substitute s for v in a
 * return failure to substitute
 */
int
copysub(Adr *a, Adr *v, Adr *s, int f)
{
    if(f)
    if(copyau(a, v)) {
        if(a->type == D_SHIFT) {
            if((a->offset&0xf) == v->reg)
                a->offset = (a->offset&~0xf)|s->reg;
            if((a->offset&(1<<4)) &&
                (a->offset>>8) == v->reg)
                a->offset = (a->offset&~(0xf<<8))|(s->reg<<8);
        } else
            a->reg = s->reg;
    }
    return 0;
}

```

⟨function copysub1(*arm*) 387b)≡ (538)

```

int
copysub1(Prog *p1, Adr *v, Adr *s, int f)
{
    if(f)
    if(copyau1(p1, v))
        p1->reg = s->reg;
    return 0;
}

```

Uses copyau1() 386c.

⟨global predinfo(*arm*) 387c)≡ (538)

```

struct {
    int opcode;
    int notopcode;
    int scnd;
    int notscnd;
} predinfo[] = {
    { ABEQ, ABNE, 0x0, 0x1, },
    { ABNE, ABEQ, 0x1, 0x0, },
    { ABHS, ABLO, 0x2, 0x3, },
    { ABLO, ABHS, 0x3, 0x2, },
    { ABMI, ABPL, 0x4, 0x5, },
    { ABPL, ABMI, 0x5, 0x4, },
    { ABVS, ABVC, 0x6, 0x7, },

```

```

{ ABVC, ABVS, 0x7, 0x6, },
{ ABHI, ABLs, 0x8, 0x9, },
{ ABLs, ABHI, 0x9, 0x8, },
{ ABGE, ABLT, 0xA, 0xB, },
{ ABLT, ABGE, 0xB, 0xA, },
{ ABGT, ABLE, 0xC, 0xD, },
{ ABLE, ABGT, 0xD, 0xC, },

```

```
};
```

Uses `__anon_struct_11` 387c.

(enum __anon_ (arm) 388a) ≡ (538)

```

enum {
    Join,
    Split,
    End,
    Branch,
    Setcond,
    Toolong
};

```

(enum __anon_ (5c/peep.c) (arm) 388b) ≡ (538)

```

enum {
    Falsecond,
    Truecond,
    Delbranch,
    Keepbranch
};

```

(function isbranch (arm) 388c) ≡ (538)

```

int
isbranch(Prog *p)
{
    return (ABEQ <= p->as) && (p->as <= ABLE);
}

```

(function predicable (arm) 388d) ≡ (538)

```

int
predicable(Prog *p)
{
    if (isbranch(p)
        || p->as == ANOP
        || p->as == AXXX
        || p->as == ADATA
        || p->as == AGLOBL
        || p->as == AGOK
        || p->as == AHISTORY
        || p->as == ANAME
        || p->as == ASIGNAME
        || p->as == ATEXT
        || p->as == AWORD
//      || p->as == ADYNT
//      || p->as == AINIT
        || p->as == ABCASE
        || p->as == ACASE)
        return 0;
    return 1;
}

```

Uses `isbranch()` 388c.

*<function modifiescpsr(*arm*) 389a>*≡ (538)

```
/*
 * Depends on an analysis of the encodings performed by 51.
 * These seem to be all of the opcodes that lead to the "S" bit
 * being set in the instruction encodings.
 *
 * C_SBIT may also have been set explicitly in p->scond.
 */
int
modifiescpsr(Prog *p)
{
    return (p->scond&C_SBIT)
        || p->as == ATST
        || p->as == ATEQ
        || p->as == ACMN
        || p->as == ACMP
        || p->as == AMULU
        || p->as == ADIVU
        || p->as == AMUL
        || p->as == ADIV
        || p->as == AMOD
        || p->as == AMODU
        || p->as == ABL;
}
```

<struct Joininfo 389b>≡ (538)

```
typedef struct Joininfo {
    Reg *start;
    Reg *last;
    Reg *end;
    int len;
} Joininfo;
```

Uses Joininfo 389b.

*<function joinsplit(*arm*) 389c>*≡ (538)

```
/*
 * Find the maximal chain of instructions starting with r which could
 * be executed conditionally
 */
int
joinsplit(Reg *r, Joininfo *j)
{
    j->start = r;
    j->last = r;
    j->len = 0;
    do {
        if (r->p2 && (r->p1 || r->p2->p2link)) {
            j->end = r;
            return Join;
        }
        if (r->s1 && r->s2) {
            j->end = r;
            return Split;
        }
        j->last = r;
        if (r->prog->as != ANOP)
            j->len++;
        if (!r->s1 && !r->s2) {
            j->end = r->link;
            return End;
        }
    } while (r = r->link);
}
```

```

    }
    if (r->s2) {
        j->end = r->s2;
        return Branch;
    }
    if (modifiescpsr(r->prog)) {
        j->end = r->s1;
        return Setcond;
    }
    r = r->s1;
} while (j->len < 4);
j->end = r;
return Toolong;
}

```

Uses Branch-198 388a, End-197 388a, Join-195 388a, Setcond-199 388a, Split-196 388a, Toolong-200 388a, and modifiescpsr() 389a.

<function successor(arm) 390a>≡ (538)

```

Reg *
successor(Reg *r)
{
    if (r->s1)
        return r->s1;
    else
        return r->s2;
}

```

<function applypred(arm) 390b>≡ (538)

```

void
applypred(Reg *rstart, Joininfo *j, int cond, int branch)
{
    int pred;
    Reg *r;

    if(j->len == 0)
        return;
    if (cond == Truecond)
        pred = predinfo[rstart->prog->as - ABEQ].scond;
    else
        pred = predinfo[rstart->prog->as - ABEQ].notscond;

    for (r = j->start; ; r = successor(r)) {
        if (r->prog->as == AB) {
            if (r != j->last || branch == Delbranch)
                excise(r);
            else {
                if (cond == Truecond)
                    r->prog->as = predinfo[rstart->prog->as - ABEQ].opcode;
                else
                    r->prog->as = predinfo[rstart->prog->as - ABEQ].notopcode;
            }
        }
        else if (predicable(r->prog))
            r->prog->scond = (r->prog->scond & ~C_SCOND) | pred;
        if (r->s1 != r->link) {
            r->s1 = r->link;
            r->link->p1 = r;
        }
        if (r == j->last)
            break;
    }
}

```

```

    }
}

```

Uses Delbranch-203 388b, Truecond-202 388b, predicable() 388d, predinfo 387c, and successor() 390a.

```

⟨function predicate(arm) 391a⟩≡ (538)
void
predicate(void)
{
    Reg *r;
    int t1, t2;
    Joininfo j1, j2;

    for(r=first; r!=R; r=r->link) {
        if (isbranch(r->prog)) {
            t1 = joinsplit(r->s1, &j1);
            t2 = joinsplit(r->s2, &j2);
            if(j1.last->link != j2.start)
                continue;
            if(j1.end == j2.end)
                if((t1 == Branch && (t2 == Join || t2 == Setcond)) ||
                   (t2 == Join && (t1 == Join || t1 == Setcond))) {
                    applypred(r, &j1, Falsecond, Delbranch);
                    applypred(r, &j2, Truecond, Delbranch);
                    excise(r);
                    continue;
                }
            if(t1 == End || t1 == Branch) {
                applypred(r, &j1, Falsecond, Keepbranch);
                excise(r);
                continue;
            }
        }
    }
}

```

Uses Branch-198 388a, Delbranch-203 388b, End-197 388a, Falsecond-201 388b, Join-195 388a, Keepbranch-204 388b, Setcond-199 388a, Truecond-202 388b, applypred() 390b, isbranch() 388c, and joinsplit() 389c.

13.6 Dominators

The register allocator needs to identify loops so it can weight variables used inside loops more heavily (promoting them to registers saves more when the code executes many times). The `postorder` function computes a reverse postorder numbering of the control-flow graph, then the dominator algorithm (Hecht-Ullman) finds loop headers — any node with a predecessor it dominates marks a loop back-edge.

```

⟨function postorder 391b⟩≡ (541)
/*
 * find looping structure
 *
 * 1) find reverse postordering
 * 2) find approximate dominators,
 * the actual dominators if the flow graph is reducible
 * otherwise, dominators plus some other non-dominators.
 * See Matthew S. Hecht and Jeffrey D. Ullman,
 * "Analysis of a Simple Algorithm for Global Data Flow Problems",
 * Conf. Record of ACM Symp. on Principles of Prog. Langs, Boston, Massachusetts,
 * Oct. 1-3, 1973, pp. 207-217.
 * 3) find all nodes with a predecessor dominated by the current node.

```



```

* such a node is a loop head.
* recursively, all preds with a greater rpo number are in the loop
*/
long
postorder(Reg *r, Reg **rpo2r, long n)
{
    Reg *r1;

    r->rpo = 1;
    r1 = r->s1;
    if(r1 && !r1->rpo)
        n = postorder(r1, rpo2r, n);
    r1 = r->s2;
    if(r1 && !r1->rpo)
        n = postorder(r1, rpo2r, n);
    rpo2r[n] = r;
    n++;
    return n;
}

```

Chapter 14

Linking Support

Plan 9's compiler supports two linking-related features: `#pragma lib` embeds a library path in the object file so the linker automatically links the right libraries (no need for `-l` flags), and `5c -T` generates type signatures for functions so the linker can detect mismatched declarations across compilation units.

14.1 `#pragma lib` and automagic linking

When a header file says `#pragma lib "libfoo.a"`, the compiler stores the library path in the object file via a special `Hist` entry with offset `-1`. The linker reads these entries and automatically links the specified libraries, eliminating the need for Makefile library lists.

```
<macprag() other locals 393a>≡ (96a)
    int c0, c;
    char *hp;
    Hist *h;

<macprag() if pragma xxx 393b>≡ (96a) 398a▷
    if(s && strcmp(s->name, "lib") == 0) {
        c0 = getnsc();
        if(c0 != '') {
            c = c0;
            if(c0 != '<')
                goto bad;
            c0 = '>';
        }
        for(hp = symb;;) {
            c = getc();
            if(c == c0)
                break;
            if(c == '\n')
                goto bad;
            *hp++ = c;
        }
        *hp = '\0';
        c = getcom();
        if(c != '\n')
            goto bad;

        /*
         * put pragma-line in as a funny history
         */
        c = strlen(symb) + 1;
        while(c & 3)
            c++;
```

```

while(nhunk < c)
    gethunk();
hp = hunk;
memcpy(hunk, symb, c);
nhunk -= c;
hunk += c;

h = alloc(sizeof(Hist));
h->name = hp;
h->line = lineno;
h->offset = -1; // pragma

// add_list(h, hist)
h->link = H;
if(ehist == H) {
    hist = h;
    ehist = h;
    return;
}
ehist->link = h;
ehist = h;

return;

bad:
    unget(c);
    yyerror("syntax in #pragma lib");
    macend();

}

```

Uses H 80c, alloc() 500b, ehist 81a, getc() 78d, getcom() 83c, gethunk() 499f, getnsc() 77c, hist 80e, hunk 499b, lineno 29f, macend() 78c, nhunk 499c, symb 35g, unget() 78b, and yyerror() 495c.

14.2 Safe linking with type signatures: 5c -T

When invoked with -T, the compiler computes a hash of each external symbol's type and embeds it in the object file. The linker can then verify that all compilation units agree on the types of shared symbols, catching mismatches between a header's declaration and the actual definition. This is similar to OCaml's interface checksums.

```

<enum signature 394a>≡ (504b)
enum signature {
    SIGNONE = 0,
    SIGDONE = 1,
    SIGINTERN = 2,

    // ???
    SIGINTERN = 1729*325*1729,
};

```

```

<Sym identifier value, code generation fields 394b>+≡ (36b) ◁287b
// enum<signature>
char sig;

```

`<zname() if generate signature for symbol s 395a>≡` (290a)

```

if(debug['T'] &&
    t == N_EXTERN &&
    s->sig != SIGDONE &&
    s->type != types[TENUM] &&
    s != symrathole){

    sig = sign(s);
    bf[0] = ASIGNAME;
    bf[1] = sig;
    bf[2] = sig>>8;
    bf[3] = sig>>16;
    bf[4] = sig>>24;
    bf[5] = t;
    bf[6] = s->symidx;
    Bwrite(b, bf, 7);
    s->sig = SIGDONE;
}

```

Uses SIGDONE 394a, TENUM 31c, debug 474a, sign() 395b, and types 32b.

14.2.1 sign()

`<function sign 395b>≡` (525)

```

/// main -> compile -> gclean -> outcode -> zname -> <>
ulong
sign(Sym *s)
{
    ulong v;
    Type *t;

    if(s->sig == SIGINTERN)
        return SIGNINTERN;
    if((t = s->type) == T)
        return 0;
    v = signature(t);
    if(v == 0)
        v = SIGNINTERN;
    return v;
}

```

Uses SIGINTERN 394a, SIGNINTERN 394a, T 31i, and signature() 394a 395c.

`<function signature 395c>≡` (525)

```

/// sign -> <>
ulong
signature(Type *t)
{
    ulong s;
    Typetab tt;

    tt.n = 0;
    tt.a = nil;
    s = signat(t, &tt);
    free(tt.a);
    return s;
}

```

Uses free() 501a and signat() 396a.

```

⟨function signat 396a⟩≡ (525)
    /// sign -> signature -> <>
    static ulong
    signat(Type *t, Typetab *tt)
    {
        int i;
        Type *t1;
        long s;

        s = 0;
        for(; t; t=t->link) {
            s = s*thash1 + thash[t->etype];
            if(t->garb&GINCOMPLETE)
                return s;
            switch(t->etype) {
            case TARRAY:
                s = s*thash2 + 0; /* was t->width */
                break;
            case TFUNC:
                for(t1=t->down; t1; t1=t1->down)
                    s = s*thash3 + signat(t1, tt);
                break;
            case TSTRUCT:
            case TUNION:
                if((i = sigind(t, tt)) >= 0){
                    s = s*thash2 + i;
                    return s;
                }
                for(t1=t->link; t1; t1=t1->down)
                    s = s*thash3 + signat(t1, tt);
                return s;
            case TIND:
                break;
            default:
                return s;
            }
        }
        return s;
    }
}

```

Uses GINCOMPLETE 399, TARRAY 31c, TFUNC 31c, TIND 31c, TSTRUCT 31c, TUNION 31c, sigind() 397c, signat() 396a, thash 396e, thash1 396b, thash2 396c, and thash3 396d.

```

⟨global thash1 396b⟩≡ (518c)
    ulong thash1 = 0x2edab8c9;

```

Uses thash1 396b.

```

⟨global thash2 396c⟩≡ (518c)
    ulong thash2 = 0x1dc74fb8;

```

Uses thash2 396c.

```

⟨global thash3 396d⟩≡ (518c)
    ulong thash3 = 0x1f241331;

```

Uses thash3 396d.

```

⟨global thash 396e⟩≡ (518c)
    ulong thash[NALLTYPES];

```

Uses NALLTYPES 120e.

<global thashinit 397a>≡ (518c)

```
Init thashinit[] =
{
    TXXX, 0x17527bbd, 0,

    TCHAR, 0x5cedd32b, 0,
    TUCCHAR, 0x552c4454, 0,
    TSHORT, 0x63040b4b, 0,
    TUSHORT, 0x32a45878, 0,
    TINT, 0x4151d5bd, 0,
    TUINT, 0x5ae707d6, 0,
    TLONG, 0x5ef20f47, 0,
    TULONG, 0x36d8eb8f, 0,
    TVLONG, 0x6e5e9590, 0,
    TUVLONG, 0x75910105, 0,
    TFLOAT, 0x25fd7af1, 0,
    TDOUBLE, 0x7c40a1b2, 0,

    TIND, 0x1b832357, 0,
    TFUNC, 0x6bab9cb, 0,
    TARRAY, 0x7c50986d, 0,
    TVOID, 0x44112eff, 0,
    TSTRUCT, 0x7c2da3bf, 0,
    TUNION, 0x3eb25e98, 0,
    TENUM, 0x44b54f61, 0,

    TOLD, 0x22b15988, 0,
    TDOT, 0x0204f6b3, 0,

    -1, 0, 0,
};
```

Uses TARRAY 31c, TCHAR 30b, TDOT 137f, TDOUBLE 31a, TENUM 31c, TFLOAT 31a, TFUNC 31c, TIND 31c, TINT 30b, TLONG 30b, TOLD 424d, TSHORT 30b, TSTRUCT 31c, TUCCHAR 30b, TUINT 30b, TULONG 30b, TUNION 31c, TUSHORT 30b, TUVLONG 30b, TVLONG 30b, TVOID 31b, and TXXX 30a.

<struct Typetab 397b>≡ (525)

```
struct Typetab {
    int n;
    Type **a;
};
```

<function sigind 397c>≡ (525)

```
/// signat -> <>
static int
sigind(Type *t, Typetab *tt)
{
    int n;
    Type **a, **na, **p, **e;

    n = tt->n;
    a = tt->a;
    e = a+n;
    /* linear search seems ok */
    for(p = a ; p < e; p++)
        if(sametype(*p, t))
            return p-a;
    if((n&15) == 0){
        na = malloc((n+16)*sizeof(Type*));
        memmove(na, a, n*sizeof(Type*));
        free(a);
    }
```

```

    a = tt->a = na;
}
a[tt->n++] = t;
return -1;
}

```

Uses free() 501a, malloc() 500d, and sametype() 145c.

14.2.2 #pragma incomplete

```

⟨macprag() if pragma xxx 398a⟩+≡ (96a) ◁393b 412d▷
    if(s && strcmp(s->name, "incomplete") == 0) {
        pragincomplete();
        return;
    }

```

Uses pragincomplete() 398b.

```

⟨function pragincomplete 398b⟩≡ (528b)
    void
    pragincomplete(void)
    {
        Sym *s;
        Type *t;
        bool istag;
        int w, et;

        istag = false;
        s = getsym();
        if(s == nil)
            goto out;
        et = 0;
        w = s->lexical;
        if(w == LSTRUCT)
            et = TSTRUCT;
        else if(w == LUNION)
            et = TUNION;
        if(et != 0){
            s = getsym();
            if(s == nil){
                yyerror("missing struct/union tag in pragma incomplete");
                goto out;
            }
            if(s->lexical != LNAME && s->lexical != LTYPE){
                yyerror("invalid struct/union tag: %s", s->name);
                goto out;
            }
            dotag(s, et, 0);
            istag = true;
        }else if(strcmp(s->name, "_off_") == 0){
            debug['T'] = false;
            goto out;
        }else if(strcmp(s->name, "_on_") == 0){
            debug['T'] = true;
            goto out;
        }

        t = s->type;
        if(istag)
            t = s->suetag;
    }

```

```

if(t == T)
    yyerror("unknown type %s in pragma incomplete", s->name);
else if(!typesu[t->etype])
    yyerror("not struct/union type in pragma incomplete: %s", s->name);
else
    t->garb |= GINCOMPLETE;
out:
    while(getnsc() != '\n')
        ;
    if(debug['f'])
        print("%s incomplete\n", s->name);
}

```

Uses GINCOMPLETE 399, LNAME, LSTRUCT, LTYPE, LUNION, T 31i, TSTRUCT 31c, TUNION 31c, debug 474a, dotag() 129a, getnsc() 77c, getsym() 77b, typesu 149a, and yyerror() 495c.

⟨Qualifier *other cases* 399⟩≡ (33a)
 GINCOMPLETE = 1<<2,

Chapter 15

Debugging Support

The compiler embeds rich debugging metadata in the object file: line number information for every instruction (via `Prog.lineno`), source file history (for `#include` tracking), and—with `5c -a`—acid declarations that describe the types and offsets of all structures and globals, enabling the `acid` debugger to display stack traces with parameter values and local variables by name.

15.1 General debugging metadata

Every `Prog` instruction carries a `lineno` field that records which C source line generated it. This is set by `nextpc` from the global `nearln`, which `codgen` sets to the current function's starting line. The linker preserves this mapping in the executable, so the debugger `acid` can show source lines alongside disassembly.

15.2 Acid debugger metadata: `5c -a`

With `-a`, the compiler outputs `acid` function definitions to standard output instead of an object file. These definitions describe the layout of each structure (field names, offsets, sizes) and each function's parameters and locals, allowing `acid` to interpret raw memory as typed C values. For structures, the generated `acid` code can print each field by name; for functions, it emits `complex` declarations that match the stack frame layout.

```
<compile() if writing acid to standard output 400a>≡ (47h)
/*
 * if we're writing acid to standard output, don't keep scratching
 * outbuf.
 */
if((debug['a'] || debug['Z']) && !debug['n']) {
    if (first) {
        outfile = nil;
        Binit(&outbuf, dup(1, -1), OWRITE);
        dup(2, 1);
    }
}
```

Uses `debug` 474a and `outfile` 45c.

```
<compile() adjust p for outfile if acid option 400b>≡ (48e)
    if(debug['a'] && debug['n'])
        strcat(p, ".acid");
```

Uses `debug` 474a.

15.2.1 Entities

For each declared entity (global, local, parameter), the compiler emits an `acid` function definition that prints the entity's value with the correct type. For example, `int x` generates `acid` code equivalent to “`x() { return *fmt(x,`”. This allows the debugger to inspect any variable by calling its name as an `acid` function.

```
<dodecl() case ONAME, debugging support 401a>≡ (104d)
    acidvar(s);
```

Uses `acidvar()` 401c.

```
<doenum() debugging support 401b>≡ (134c)
    acidvar(s);
```

Uses `acidvar()` 401c.

```
<function acidvar 401c>≡ (523)
/// doenum? | dodecl? -> <>
void
acidvar(Sym *s)
{
    int n;
    Io *i;
    Type *t;
    Sym *s1, *s2;

    <acidvar() return if no -a or -s 402a>
    <acidvar() if -aa 402b>

    t = s->type;
    while(t && t->etype == TIND)
        t = t->link;
    if(t == T)
        return;
    if(t->etype == TENUM) {
        Bprint(&outbuf, "%s = ", amap(s->name));
        if(!typefd[t->etype])
            Bprint(&outbuf, "%lld;\n", s->vconst);
        else
            Bprint(&outbuf, "%f\n;", s->fconst);
        return;
    }
    if(!typesu[t->etype])
        return;
    s1 = acidsue(t->link);
    if(s1 == S)
        return;
    switch(s->class) {
    case CAUTO:
    case CPARAM:
        s2 = acidfun(thisfntype);
        if(s2)
            Bprint(&outbuf, "complex %s %s:%s;\n",
                amap(s1->name), amap(s2->name), amap(s->name));
        break;

    case CEXTERN:
    case CGLOBL:
    case CSTATIC: case CLOCAL:
        Bprint(&outbuf, "complex %s %s;\n",
            amap(s1->name), amap(s->name));
        break;
```

```

    }
}

```

Uses CAUTO 33b, CEXTERN 33b, CGLOBL 33b, CLOCAL 415b, CPARAM 33b, CSTATIC 33b, S 35d, T 31i, TENUM 31c, TIND 31c, acidfun() 402d, acidsue() 402c, amap() 403a, thisfntype 105b, typefd 149c, and typesu 149a.

```

⟨acidvar() return if no -a or -s 402a⟩≡ (401c)
    if(!debug['a'] || debug['s'])
        return;

```

Uses debug 474a.

```

⟨acidvar() if -aa 402b⟩≡ (401c)
    if(debug['a'] > 1) {
        n = 0;
        for(i=iostack; i; i=i->link)
            n++;
        if(n > 1)
            return;
    }

```

Uses debug 474a and iostack 52c.

```

⟨function acidsue 402c⟩≡ (523)
    /// acidvar -> <>
    Sym*
    acidsue(Type *t)
    {
        int h;
        Sym *s;

        if(t != T)
            for(h=0; h<nelem(hash); h++)
                for(s = hash[h]; s != S; s = s->link)
                    if(s->suetag && s->suetag->link == t)
                        return s;
        return 0;
    }

```

Uses S 35d, T 31i, and hash 35a.

```

⟨function acidfun 402d⟩≡ (523)
    /// acidvar -> <>
    Sym*
    acidfun(Type *t)
    {
        int h;
        Sym *s;

        for(h=0; h<nelem(hash); h++)
            for(s = hash[h]; s != S; s = s->link)
                if(s->type == t)
                    return s;
        return 0;
    }

```

Uses S 35d and hash 35a.

```

⟨global kwd 402e⟩≡ (523)
    static char *kwd[] =
    {
        "$adt", "$aggr", "$append", "$builtin", "$complex", "$defn",
        "$delete", "$do", "$else", "$eval", "$head", "$if",
        "$local", "$loop", "$return", "$tail", "$then",
        "$union", "$whatis", "$while",
    };

```

<function amap 403a>≡ (523)

```
char*
amap(char *s)
{
    int i, bot, top, new;

    bot = 0;
    top = bot + nelem(kwd) - 1;
    while(bot <= top){
        new = bot + (top - bot)/2;
        i = strcmp(kwd[new]+1, s);
        if(i == 0)
            return kwd[new];

        if(i < 0)
            bot = new + 1;
        else
            top = new - 1;
    }
    return s;
}
```

Uses kwd-80 402e.

15.2.2 Structure/union definitions

acidtype emits an acid “complex” definition for each struct or union, listing its fields with their types and offsets. This lets the acid debugger display struct members by name rather than as raw memory, and supports nested structs and arrays through recursive type description.

<function acidtype 403b>≡ (523)

```
void
acidtype(Type *t)
{
    Sym *s;
    Type *l;
    Io *i;
    int n;
    char *an;

    <acidtype() return if no -a 404a>
    <acidtype() if -aa 404b>

    s = acidsue(t->link);
    if(s == S)
        return;
    switch(t->etype) {
    case TUNION:
    case TSTRUCT:
        if(debug['s'])
            goto asmstr;
        an = amap(s->name);
        Bprint(&outbuf, "sizeof%s = %ld;\n", an, t->width);
        Bprint(&outbuf, "aggr %s\n{\n", an);
        for(l = t->link; l != T; l = l->down)
            acidmember(l, 0, 1);
        Bprint(&outbuf, "};\n\n");

        Bprint(&outbuf, "defn\n%s(addr) {\n\tcomplex %s addr;\n", an, an);
        for(l = t->link; l != T; l = l->down)
```

```

        acidmember(l, 0, 0);
Bprint(&outbuf, "};\n\n");
break;

```

```

asmstr:
    if(s == S)
        break;
    for(l = t->link; l != T; l = l->down)
        if(l->sym != S)
            Bprint(&outbuf, "#define\t%s.%s\t%ld\n",
                s->name,
                l->sym->name,
                l->offset);
        break;

```

```

default:
    Bprint(&outbuf, "T%d\n", t->etype);
    return;

```

```

}

```

```

}

```

Uses S 35d, T 31i, TSTRUCT 31c, TUNION 31c, acidmember() 405b, acidsue() 402c, amap() 403a, and debug 474a.

```

<acidtype() return if no -a 404a>≡ (403b)
    if(!debug['a'])
        return;

```

Uses debug 474a.

```

<acidtype() if -aa 404b>≡ (403b)
    if(debug['a'] > 1) {
        n = 0;
        for(i=iostack; i; i=i->link)
            n++;
        if(n > 1)
            return;
    }

```

Uses debug 474a and iostack 52c.

```

<global acidchar 404c>≡ (523)
    char acidchar[NTYPE];

```

Uses NTYPE 30a.

```

<global acidcinit 404d>≡ (523)
    Init acidcinit[] =
    {
        TCHAR, 'C', 0,
        TUCCHAR, 'b', 0,
        TSHORT, 'd', 0,
        TUSHORT, 'u', 0,
        TLONG, 'D', 0,
        TULONG, 'U', 0,
        TVLONG, 'V', 0,
        TUVLONG, 'W', 0,
        TFLOAT, 'f', 0,
        TDOUBLE, 'F', 0,
        TARRAY, 'a', 0,
        TIND, 'X', 0,
        -1, 0, 0,
    };

```

Uses TARRAY 31c, TCHAR 30b, TDOUBLE 31a, TFLOAT 31a, TIND 31c, TLONG 30b, TSHORT 30b, TUCCHAR 30b, TULONG 30b, TUSHORT 30b, TUVLONG 30b, and TVLONG 30b.

⟨function acidinit 405a⟩≡ (523)

```
static void
acidinit(void)
{
    Init *p;

    for(p=acidcinit; p->code >= 0; p++)
        acidchar[p->code] = p->value;

    acidchar[TINT] = acidchar[TLONG];
    acidchar[TUINT] = acidchar[TULONG];
    if(types[TINT]->width != types[TLONG]->width) {
        acidchar[TINT] = acidchar[TSHORT];
        acidchar[TUINT] = acidchar[TUSHORT];
        if(types[TINT]->width != types[TSHORT]->width)
            warn(Z, "acidmember int not long or short");
    }
    if(types[TIND]->width == types[TUVLONG]->width)
        acidchar[TIND] = 'Y';
}
```

Uses TIND 31c, TINT 30b, TLONG 30b, TSHORT 30b, TUINT 30b, TULONG 30b, TUSHORT 30b, TUVLONG 30b, Z 29a, acidchar 404c, acidcinit 404d, types 32b, and warn() 142.

⟨function acidmember 405b⟩≡ (523)

```
void
acidmember(Type *t, long off, int flag)
{
    Sym *s, *s1;
    Type *l;
    static bool acidcharinit = false;

    if(acidcharinit == false) {
        acidinit();
        acidcharinit = true;
    }
    s = t->sym;
    switch(t->etype) {
    default:
        Bprint(&outbuf, " T%d\n", t->etype);
        break;

    case TIND:
        if(s == S)
            break;
        if(flag) {
            for(l=t; l->etype==TIND; l=l->link)
                ;
            if(typesu[l->etype]) {
                s1 = acidsue(l->link);
                if(s1 != S) {
                    Bprint(&outbuf, " 'A' %s %ld %s;\n",
                        amap(s1->name),
                        t->offset+off, amap(s->name));
                    break;
                }
            }
        }
        } else {
            Bprint(&outbuf,
                "\tprint(\"%t%s\t\", addr.%s\\X, \"%\\n\\n\");\n",
```

```

        amap(s->name), amap(s->name));
    break;
}

case TINT:
case TUINT:
case TCHAR:
case TUCCHAR:
case TSHORT:
case TUSHORT:
case TLONG:
case TULONG:
case TVLONG:
case TUVLONG:
case TFLOAT:
case TDOUBLE:
case TARRAY:
    if(s == S)
        break;
    if(flag) {
        Bprint(&outbuf, " '%c' %ld %s;\n",
            acidchar[t->etype], t->offset+off, amap(s->name));
    } else {
        Bprint(&outbuf, "\tprint(\"%t%s\t\", addr.%s, \"%\\n\");\n",
            amap(s->name), amap(s->name));
    }
    break;

case TSTRUCT:
case TUNION:
    s1 = acidsue(t->link);
    if(s1 == S)
        break;
    if(flag) {
        if(s == S) {
            Bprint(&outbuf, " {\n");
            for(l = t->link; l != T; l = l->down)
                acidmember(l, t->offset+off, flag);
            Bprint(&outbuf, " };\n");
        } else {
            Bprint(&outbuf, " %s %ld %s;\n",
                amap(s1->name),
                t->offset+off, amap(s->name));
        }
    } else {
        if(s != S) {
            Bprint(&outbuf, "\tprint(\"%s %s {\\n\");\n",
                amap(s1->name), amap(s->name));
            Bprint(&outbuf, "\t%s(addr.%s);\n",
                amap(s1->name), amap(s->name));
            Bprint(&outbuf, "\tprint(\"}%\\n\");\n");
        } else {
            Bprint(&outbuf, "\tprint(\"%s {\\n\");\n",
                amap(s1->name));
            Bprint(&outbuf, "\t\t%s(addr+%ld);\n",
                amap(s1->name), t->offset+off);
            Bprint(&outbuf, "\tprint(\"}%\\n\");\n");
        }
    }
    break;

```

```

    }
}

```

Uses S 35d, T 31i, TARRAY 31c, TCHAR 30b, TDOUBLE 31a, TFLOAT 31a, TIND 31c, TINT 30b, TLONG 30b, TSHORT 30b, TSTRUCT 31c, T UCHAR 30b, TUINT 30b, TULONG 30b, TUNION 31c, TUSHORT 30b, TUVLONG 30b, TVLONG 30b, acidchar 404c, acidinit() 405a, acidmember() 405b, acidsue() 402c, amap() 403a, and typesu 149a.

15.3 Pickle: 5c -Z

With -Z, the compiler generates C code for serializing (“pickling”) and deserializing structures, automating the creation of marshalling functions from type definitions. This is a Plan 9 extension that avoids hand-written serialization code for IPC and file formats.

```

⟨compile() adjust p for outfile if pickle option 407a⟩≡ (48e)
    else if(debug['Z'] && debug['n'])
        strcat(p, "_pickle.c");

```

Uses debug 474a.

```

⟨global kwd (cc/pickle.c) 407b⟩≡ (524a)
    static char *kwd[] =
    {
        "$adt", "$aggr", "$append", "$complex", "$defn",
        "$delete", "$do", "$else", "$eval", "$head", "$if",
        "$local", "$loop", "$return", "$tail", "$then",
        "$union", "$whatis", "$while",
    };

```

```

⟨global picklestr 407c⟩≡ (524a)
    static char picklestr[] = "\tpickle(s, un, ";

```

Uses picklestr-79 407c.

```

⟨function pmap 407d⟩≡ (524a)
    static char*
    pmap(char *s)
    {
        int i, bot, top, new;

        bot = 0;
        top = bot + nelem(kwd) - 1;
        while(bot <= top){
            new = bot + (top - bot)/2;
            i = strcmp(kwd[new]+1, s);
            if(i == 0)
                return kwd[new];

            if(i < 0)
                bot = new + 1;
            else
                top = new - 1;
        }
        return s;
    }

```

Uses kwd-78 407b.

<function picklesue 408a>≡ (524a)

```
Sym*
picklesue(Type *t)
{
    int h;
    Sym *s;

    if(t != T)
    for(h=0; h<nelem(hash); h++)
        for(s = hash[h]; s != S; s = s->link)
            if(s->suetag && s->suetag->link == t)
                return s;
    return 0;
}
```

Uses S 35d, T 31i, and hash 35a.

<global picklechar 408b>≡ (524a)

```
char picklechar[NTYPE];
```

Uses NTYPE 30a.

<global picklecinit 408c>≡ (524a)

```
Init picklecinit[] =
{
    TCHAR,  'C', 0,
    TUCCHAR, 'b', 0,
    TSHORT, 'd', 0,
    TUSHORT, 'u', 0,
    TLONG,  'D', 0,
    TULONG, 'U', 0,
    TVLONG, 'V', 0,
    TUVLONG, 'W', 0,
    TFLOAT, 'f', 0,
    TDOUBLE, 'F', 0,
    TARRAY, 'a', 0,
    TIND, 'X', 0,
    -1, 0, 0,
};
```

Uses TARRAY 31c, TCHAR 30b, TDOUBLE 31a, TFLOAT 31a, TIND 31c, TLONG 30b, TSHORT 30b, TUCCHAR 30b, TULONG 30b, TUSHORT 30b, TUVLONG 30b, and TVLONG 30b.

<function pickleinit 408d>≡ (524a)

```
static void
pickleinit(void)
{
    Init *p;

    for(p=picklecinit; p->code >= 0; p++)
        picklechar[p->code] = p->value;

    picklechar[TINT] = picklechar[TLONG];
    picklechar[TUINT] = picklechar[TULONG];
    if(types[TINT]->width != types[TLONG]->width) {
        picklechar[TINT] = picklechar[TSHORT];
        picklechar[TUINT] = picklechar[TUSHORT];
        if(types[TINT]->width != types[TSHORT]->width)
            warn(Z, "picklemember int not long or short");
    }
}
```

Uses TINT 30b, TLONG 30b, TSHORT 30b, TUINT 30b, TULONG 30b, TUSHORT 30b, Z 29a, picklechar 408b, picklecinit 408c, types 32b, and warn() 142.

```

<function picklemember 409>≡ (524a)
void
picklemember(Type *t, long off)
{
    Sym *s, *s1;
    static int picklecharinit = 0;

    if(picklecharinit == 0) {
        pickleinit();
        picklecharinit = 1;
    }
    s = t->sym;
    switch(t->etype) {
    default:
        Bprint(&outbuf, " T%d\n", t->etype);
        break;

    case TIND:
        if(s == S)
            Bprint(&outbuf,
                "%s\"p\\", (char*)addr+%ld+_i*%ld);\n",
                picklestr, t->offset+off, t->width);
        else
            Bprint(&outbuf,
                "%s\"p\\", &addr->%s);\n",
                picklestr, pmap(s->name));
        break;

    case TINT:
    case TUINT:
    case TCHAR:
    case TUCCHAR:
    case TSHORT:
    case TUSHORT:
    case TLONG:
    case TULONG:
    case TVLONG:
    case TUVLONG:
    case TFLOAT:
    case TDOUBLE:
        if(s == S)
            Bprint(&outbuf, "%s\"%c\\", (char*)addr+%ld+_i*%ld);\n",
                picklestr, picklechar[t->etype], t->offset+off, t->width);
        else
            Bprint(&outbuf, "%s\"%c\\", &addr->%s);\n",
                picklestr, picklechar[t->etype], pmap(s->name));
        break;

    case TARRAY:
        Bprint(&outbuf, "\tfor(_i = 0; _i < %ld; _i++) {\n\t",
            t->width/t->link->width);
        picklemember(t->link, t->offset+off);
        Bprint(&outbuf, "\t}\n\t_i = 0;\n\tUSED(_i);\n");
        break;

    case TSTRUCT:
    case TUNION:
        s1 = picklesue(t->link);

```

```

    if(s1 == S)
        break;
    if(s == S) {
        Bprint(&outbuf, "\tpickle_%s(s, un, (%s*)((char*)addr+%ld+_i*%ld));\n",
            pmap(s1->name), pmap(s1->name), t->offset+off, t->width);
    } else {
        Bprint(&outbuf, "\tpickle_%s(s, un, &addr->%s);\n",
            pmap(s1->name), pmap(s->name));
    }
    break;
}
}

```

Uses S 35d, TARRAY 31c, TCHAR 30b, TDOUBLE 31a, TFLOAT 31a, TIND 31c, TINT 30b, TLONG 30b, TSHORT 30b, TSTRUCT 31c, TUCCHAR 30b, TUINT 30b, TULONG 30b, TUNION 31c, TUSHORT 30b, TUVLONG 30b, TVLONG 30b, picklechar 408b, pickleinit() 408d, picklemember() 409, picklestr-79 407c, picklesue() 408a, and pmap() 407d.

(function pickletype 410)≡ (524a)

```

void
pickletype(Type *t)
{
    Sym *s;
    Type *l;
    Io *i;
    int n;
    char *an;

    if(!debug['P'])
        return;
    if(debug['P'] > 1) {
        n = 0;
        for(i=iostack; i; i=i->link)
            n++;
        if(n > 1)
            return;
    }
    s = picklesue(t->link);
    if(s == S)
        return;
    switch(t->etype) {
    default:
        Bprint(&outbuf, "T%d\n", t->etype);
        return;

    case TUNION:
    case TSTRUCT:
        if(debug['s'])
            goto asmstr;
        an = pmap(s->name);

        Bprint(&outbuf, "void\npickle_%s(void *s, int un, %s *addr)\n{\n\tint _i = 0;\n\n\tUSED(_i);\n", an, an);
        for(l = t->link; l != T; l = l->down)
            picklemember(l, 0);
        Bprint(&outbuf, "}\n\n");
        break;

    asmstr:
        if(s == S)
            break;
        for(l = t->link; l != T; l = l->down)
            if(l->sym != S)
                Bprint(&outbuf, "#define\t%s.%s\t%ld\n",

```

```

        s->name,
        l->sym->name,
        l->offset);
    break;
}
}

```

Uses S [35d](#), T [31i](#), TSTRUCT [31c](#), TUNION [31c](#), debug [474a](#), iostack [52c](#), picklemember() [409](#), picklesue() [408a](#), and pmap() [407d](#).

The compiler supports profiling with `5c -p`, which inserts ATEXT flags that tell the linker to add `_profin/_profout` calls at function entry and exit (see LINKER book [\[Pad15c\]](#) and PROFILER book [\[Pad26\]](#)). The `-s` flag outputs structure offset definitions as assembly `#defines`, useful for writing assembly code that accesses C structures.

Chapter 16

Profiling Support

Profiling is enabled by default: each TEXT pseudo-instruction is emitted without the NOPROF flag, so the linker inserts a call to `_profin` at function entry (see PROFILER book [Pad26]). The `#pragma profile off` directive sets NOPROF on subsequent functions, disabling profiling for hot inner loops or interrupt handlers where the overhead is unacceptable.

```
<global profileflg 412a>≡ (511d)
    bool profileflg;
```

```
<main() remaining initialisation 412b>+≡ (45d) <47b 419e>
    profileflg = true; /* #pragma can turn it off */
```

```
<gpseudo() if TEXT, set possible TEXT attributes 412c>≡ (195d)
    if(a == ATEXT)
        p->reg = (profileflg ? 0 : NOPROF);
```

Uses `profileflg 412a`.

16.1 Text attributes

16.2 #pragma profile

```
<macprag() if pragma xxx 412d>+≡ (96a) <398a 448a>
    if(s && strcmp(s->name, "profile") == 0) {
        pragprofile();
        return;
    }
```

Uses `pragprofile() 412e`.

```
<function pragprofile 412e>≡ (528b)
    void
    pragprofile(void)
    {
        Sym *s;

        profileflg = false;
        s = getsym();
        if(s) {
            profileflg = atoi(s->name+1);
            if(strcmp(s->name, "on") == 0 ||
               strcmp(s->name, "yes") == 0)
                profileflg = true;
        }
    }
```

```

while(getnsc() != '\n')
;
⟨pragprofile() debug if -f 413⟩
}

```

Uses `getnsc()` 77c, `getsym()` 77b, and `profileflg` 412a.

```

⟨pragprofile() debug if -f 413⟩≡
if(debug['f'])
if(profileflg)
print("%4ld: profileflg %d\n", lineno, profileflg);
else
print("%4ld: profileflg off\n", lineno);

```

(412e)

Uses `debug` 474a, `lineno` 29f, and `profileflg` 412a.

Chapter 17

Advanced Topics

This chapter collects C language features and compiler extensions that are orthogonal to the main compilation pipeline: automatic type conversions (functions to pointers, arrays to pointers), bitfields, old-style prototypes, and kence-specific extensions like unnamed struct elements, struct constructors, `extern register` for per-processor storage, `typestr` for type reflection, and `signof` for signature reflection. It also covers floats, 64-bit integer operations, endianness, and miscellaneous optimizations like `#pragma pack`.

17.1 Advanced C features

This section covers C features that require non-trivial compiler support: automatic conversions between functions and function pointers, between arrays and pointers, local `static` and `volatile` variables, bitfields, old-style function prototypes, and `_Noreturn`.

17.1.1 Function and function pointer automatic conversions

C implicitly converts between function types and function pointers: calling through a function pointer (`fp(args)`) automatically inserts a dereference (`(*fp)(args)`), and passing a function name where a function pointer is expected automatically takes its address. The compiler also converts function-typed parameters to function pointer parameters, since functions cannot be passed by value.

```
<tcom0() when OFUNC case, add OIND if pointer function call 414a>≡ (172b)
    if(l->type->etype == TIND && l->type->link->etype == TFUNC) {
        l = new1(OIND, l, Z);
        l->type = l->left->type->link;
        n->left = l;
    }
```

Uses OIND 116e, TFUNC 31c, TIND 31c, Z 29a, and new1() 29h.

```
<paramconv() switch etype cases 414b>≡ (182c) 415a▷
    case TFUNC:
        t = typ(TIND, t);
        t->width = types[TIND]->width;
        break;
```

Uses TFUNC 31c, TIND 31c, typ() 32a, and types 32b.

17.1.2 Array and pointer automatic conversions

When an array is passed as a function parameter, C converts it to a pointer to its first element—the array size information is lost. Similarly, `int foo(int x[45])` is treated as `int foo(int *x)`: the 45 is silently discarded

by the compiler, which can be a source of bugs.

```

⟨paramconv() switch etype cases 415a⟩+≡ (182c) ◁414b
    case TARRAY:
        t = typ(TIND, t->link);
        t->width = types[TIND]->width;
        break;

```

Uses TARRAY 31c, TIND 31c, typ() 32a, and types 32b.

17.1.3 Local static

A static local variable has the lifetime of a global but the visibility of a local. The compiler handles it with a special storage class CLOCAL: the variable is allocated in the global data segment (like CGLOBL) and emitted as a static symbol (with a <> suffix for uniqueness), but its declaration is scoped to the block it appears in. Initialization is done via ADATA at file scope, not at function entry.

```

⟨Storage_class cases 415b⟩+≡ (33b) ◁137i 432a▷
    CLOCAL, // local static

```

```

⟨adec1() adjust storage to CLOCAL if static local variable 415c⟩≡ (140c)
    if(c == CSTATIC)
        c = CLOCAL;

```

Uses CLOCAL 415b and CSTATIC 33b.

```

⟨dodecl() case ONAME, if local static variable 415d⟩≡ (104d)
    if(s->class == CLOCAL)
        s = mkstatic(s);

```

Uses CLOCAL 415b and mkstatic() 415f.

```

⟨name rule, if local static variable 415e⟩≡ (114b)
    if($1->class == CLOCAL)
        $1 = mkstatic($1);

```

```

⟨function mkstatic 415f⟩≡ (525)
    Sym*
    mkstatic(Sym *s)
    {
        Sym *s1;

        ⟨mkstatic() sanity check s 415g⟩
        snprintf(symb, NSYMB, "%s$%d", s->name, s->block);
        s1 = lookup();

        if(s1->class != CSTATIC) {
            s1->type = s->type;
            s1->offset = s->offset;
            s1->block = s->block;
            s1->class = CSTATIC;
        }
        return s1;
    }

```

Uses CSTATIC 33b, NSYMB 35h, and symb 35g.

```

⟨mkstatic() sanity check s 415g⟩≡ (415f)
    if(s->class != CLOCAL) // should never happen, use errexit()?
        return s;

```

Uses CLOCAL 415b.

17.1.4 Local volatile

A **volatile** local must not be promoted to a register by the optimizer, since its value can change outside the compiler's knowledge (e.g., signal handlers, memory-mapped I/O). The compiler ensures this by inserting a hidden `USED(&var)` call at scope exit via `revertdecl`, which forces the variable to have an address (and thus live on the stack). The register allocator sees this reference and avoids promoting the variable.

`<revertdecl() other locals 416a>≡` (40d)
Node *n1;

`<revertdecl() if volatile symbol 416b>≡` (41d)
if(s->type && (s->type->garb & GVOLATILE)) {
 // add USED(&local_volatile);
 n1 = new(ONAME, Z, Z);

 n1->sym = s;
 n1->type = s->type;
 n1->etype = (s->type != T) ? s->type->etype : TVOID;
 n1->xoffset = s->xoffset;
 n1->class = s->class;

 n1 = new(OADDR, n1, Z);
 n1 = new(OUSED, n1, Z);

 // add_list(n1, n)
 if(n == Z)
 n = n1;
 else
 n = new(OLIST, n1, n);
}

Uses `GVOLATILE 33a`, `OADDR 116e`, `OLIST 29b`, `ONAME 27`, `OUSED 184c`, `T 31i`, `TVOID 31b`, `Z 29a`, and `new() 29d`.

`<xdecl rule, adjust block body with possible hidden generated nodes 416c>≡` (105c)
if(n)
 \$6 = new(OLIST, n, \$6);

`<ulstmt rule, adjust block body with possible hidden generated nodes 416d>≡` (106f)
if(n)
 \$\$ = new(OLIST, n, \$2);

`<ulstmt rule, adjust forexpr with possible hidden generated nodes 416e>≡` (107e)
if(n){
 if(\$4)
 \$4 = new(OLIST, n, \$4);
 else
 \$4 = n;
}

`<usedset() switch node op cases 416f>≡` (184f)
case OADDR: /* volatile */
 gins(ANOP, n, Z);
 break;

Uses `OADDR 116e` and `Z 29a`.

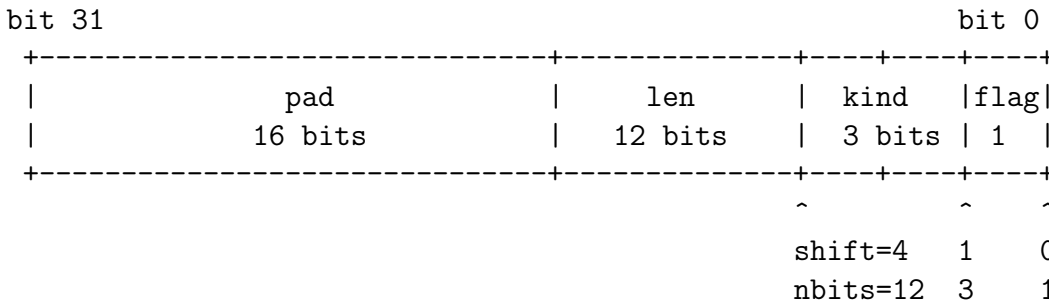
17.1.5 Bitfields

Bitfield support is spread across parsing (tracking the current bit position within a storage word via `firstbit/lastbit`), type-checking (using `Type.shift` and `Type.nbits` to locate the field within its word), and code generation (emitting shift/mask sequences to extract or store the field). The complexity comes from packing adjacent bitfields into the same machine word while respecting alignment.

A worked example makes this concrete. Take the struct

```
struct Pkt {
    uint  flag  :1;
    uint  kind  :3;
    uint  len   :12;
    uint  pad   :16;
};
```

The compiler packs all four fields into a single 32-bit word. As `dodecl` processes each `OBIT`, it advances `lastbit` by the requested width and stores the resulting `shift` (bit offset within the word) and `nbits` (width in bits) on the field's `Type`. The final layout, on a little-endian ARM:



To read `p->kind`, the code generator emits a load of the whole word followed by a shift+mask sequence that `bitload` generates:

```
MOVW    p+0(R0), R1      ; load the word
ASHL    $28, R1, R1      ; (32 - shift - nbits) = 28, push to top
ASHR    $29, R1, R1      ; (32 - nbits) = 29, sign-extend back down
```

The two-step shift is the trick that makes bitfield extraction sign-correct in one shot. Pushing the field to the top of the register and then arithmetic-shifting back either sign-extends (for `int` fields) or zero-extends (for `unsigned`, using `LSHR` instead of `ASHR`) the result. To write `p->kind = 5`, `bitstore` load-mask-or-store: load the word, AND with `~(mask << shift)` to clear the field, OR in the new value shifted into place, and store the word back. The constraint that triggers a fresh storage word is in the parser: when `lastbit + new_field_width > tfield_bits`, `lastbit` is reset to the new field's width and the next field starts a new 32-bit cell. This is why `int :0` is a useful idiom: a zero-width unnamed field forces alignment to the next storage unit.

<global firstbit 417a>≡ (511d)
`bool firstbit;`

<global lastbit 417b>≡ (511d)
`long lastbit;`

<dodecl() case ONAME, other initializations 417c>≡ (104d)
`firstbit = false;`

Uses `firstbit 417a`.

<sbody rule, initializations before parsing the fields 417d>≡ (131b)
`lastbit = 0;`
`firstbit = true;`

<edecor rule, set fields after parsed a field declarator 418a>≡ (132b)

```
lastbit = 0;
firstbit = true;
```

<global lastfield 418b>≡ (511d)

```
// option<long> None = -1 | 0
long lastfield;
```

<dodecl() other initializations 418c>≡ (103a)

```
lastfield = 0;
```

Uses lastfield 418b.

<declaration nodes 418d>+≡ (27) <137c

```
OBIT,
```

<edecor rule 418e>+≡ (132a) <132b

```
| tag ':' lexpr { $$ = new(OBIT, $1, $3); }
| ':' lexpr { $$ = new(OBIT, Z, $2); }
```

<dodecl() switch node kind cases 418f>+≡ (103a) <135i

```
case OBIT:
    n1 = n->right;
    complex(n1);

    lastfield = -1; // None
    if(n1->op == OCONST)
        lastfield = n1->vconst;
    if(lastfield < 0) {
        diag(n, "field width must be non-negative constant");
        lastfield = 1;
    }

    if(lastfield == 0) {
        lastbit = 0;
        firstbit = true;
        if(n->left != Z) {
            diag(n, "zero width named field");
            lastfield = 1;
        }
    }
    if(!typei[t->etype]) {
        diag(n, "field type must be int-like");
        t = types[TINT];
        lastfield = 1;
    }
    if(lastfield > tfield->width*8) {
        diag(n, "field width larger than field unit");
        lastfield = 1;
    }

    lastbit += lastfield;
    if(lastbit > tfield->width*8) {
        lastbit = lastfield;
        firstbit = true;
    }
    n = n->left;
    goto loop;
```

Uses OBIT 418d, OCONST 111a, TINT 30b, Z 29a, complex() 210a, diag() 496b, firstbit 417a, lastbit 417b, lastfield 418b, tfield 419b, typei 148c, and types 32b.

```

⟨edec1() if lastfield 419a⟩≡ (132c)
    if(lastfield) {
        t->shift = lastbit - lastfield;
        t->nbits = lastfield;
        if(firstbit)
            t->shift = -t->shift;

        if(typeu[t->etype])
            t->etype = tufield->etype;
        else
            t->etype = tfield->etype;
    }

```

Uses firstbit 417a, lastbit 417b, lastfield 418b, tfield 419b, tufield 419c, and typeu 231h.

```

⟨global tfield 419b⟩≡ (511d)
    Type* tfield;

```

```

⟨global tufield 419c⟩≡ (511d)
    Type* tufield;

```

```

⟨ginit() misc initialisations 419d⟩+≡ (51c) <281e 433a>
    tfield = types[TLONG];

```

Uses TLONG 30b, tfield 419b, and types 32b.

```

⟨main() remaining initialisation 419e⟩+≡ (45d) <412b
    tufield = simple1((1L<<tfield->etype) | BUNSIGNED);

```

```

⟨tcomo() when OADDR, check if bitfield 419f⟩≡ (170a)
    if(l->type->nbits) {
        diag(n, "address of a bit field");
        goto bad;
    }

```

Uses diag() 496b.

```

⟨tcomo() when OSIZE, check if sizeof bitfield 419g⟩≡ (178a)
    if(l->op == OBIT) {
        diag(n, "sizeof bitfield");
        goto bad;
    }

```

Uses OBIT 418d and diag() 496b.

```

⟨init1() sanity check t has no bitfields 419h⟩≡ (262d)
    if(t->nbits)
        diag(Z, "cannot initialize bitfields");

```

Uses Z 29a and diag() 496b.

```

⟨makedot() if bitfields 419i⟩≡ (241b)
    if(t->nbits) {
        n1 = new(OXXX, Z, Z);
        *n1 = *n;
        n->op = OBIT;
        n->left = n1;
        n->right = Z;
        n->type = t;
        n->addable = n1->left->addable;
        n = n1;
    }

```

Uses OBIT 418d, OXXX 27, Z 29a, and new() 29d.

`<cgenrel() switch node kind cases 420a>+≡ (223c) <257 420e>`

```
case OBIT:
    <cgenrel() nullwarn check if nn is null 189a>
    bitload(n, &nod, Z, Z, nn);
    gopcode(OAS, &nod, Z, nn);
    regfree(&nod);
    break;
```

Uses OAS 116a, OBIT 418d, and Z 29a.

`<cgenrel() when OAS, if bitfield assignment 420b>≡ (234d)`

```
if(l->op == OBIT)
    goto bitas;
```

Uses OBIT 418d.

`<cgenrel() when OASMUL/OASDIV/etc, if bitfield assignment 420c>≡ (239c)`

```
if(l->op == OBIT)
    goto asbitop;
```

Uses OBIT 418d.

`<cgenrel() when OASADD/OASSUB/etc, if bitfield assignment 420d>≡ (239c)`

```
if(l->op == OBIT)
    goto asbitop;
```

Uses OBIT 418d.

`<cgenrel() switch node kind cases 420e>+≡ (223c) <420a 421a>`

```
asbitop:
    regalloc(&nod4, n, nn);
    if(l->complex >= r->complex) {
        bitload(l, &nod, &nod1, &nod2, &nod4);
        regalloc(&nod3, r, Z);
        cgen(r, &nod3);
    } else {
        regalloc(&nod3, r, Z);
        cgen(r, &nod3);
        bitload(l, &nod, &nod1, &nod2, &nod4);
    }
    gmove(&nod, &nod4);
    gopcode(o, &nod3, Z, &nod4);
    regfree(&nod3);
    gmove(&nod4, &nod);
    regfree(&nod4);
    bitstore(l, &nod, &nod1, &nod2, nn);
    break;
```

Uses Z 29a.

`<cgenrel() when OPOSTINC/OPOSTDEC, if bitfield 420f>≡ (250b)`

```
if(l->op == OBIT)
    goto bitinc;
```

Uses OBIT 418d.

`<cgenrel() when OPREINC/OPREDEC, if bitfield 420g>≡ (251)`

```
if(l->op == OBIT)
    goto bitinc;
```

Uses OBIT 418d.

`<cgenrel() switch node kind cases 421a>+≡ (223c) <420e 421b>`

```

bitinc:
    if(nn != Z && (o == OPOSTINC || o == OPOSTDEC)) {
        bitload(1, &nod, &nod1, &nod2, Z);
        gopcode(OAS, &nod, Z, nn);
        gopcode(OADD, nodconst(v), Z, &nod);
        bitstore(1, &nod, &nod1, &nod2, Z);
        break;
    }
    bitload(1, &nod, &nod1, &nod2, nn);
    gopcode(OADD, nodconst(v), Z, &nod);
    bitstore(1, &nod, &nod1, &nod2, nn);
    break;

```

Uses OADD 114c, OAS 116a, OPOSTDEC 118j, OPOSTINC 118j, and Z 29a.

`<cgenrel() switch node kind cases 421b>+≡ (223c) <421a`

```

bitas:
    n = l->left;
    regalloc(&nod, r, nn);
    if(l->complex >= r->complex) {
        reglcgen(&nod1, n, Z);
        cgen(r, &nod);
    } else {
        cgen(r, &nod);
        reglcgen(&nod1, n, Z);
    }
    regalloc(&nod2, n, Z);
    gopcode(OAS, &nod1, Z, &nod2);
    bitstore(1, &nod, &nod1, &nod2, nn);
    break;

```

Uses OAS 116a and Z 29a.

`<function bitload(arm) 421c>≡ (540b)`

```

void
bitload(Node *b, Node *n1, Node *n2, Node *n3, Node *nn)
{
    int sh;
    long v;
    Node *l;

    /*
     * n1 gets adjusted/masked value
     * n2 gets address of cell
     * n3 gets contents of cell
     */
    l = b->left;
    if(n2 != Z) {
        regalloc(n1, l, nn);
        reglcgen(n2, l, Z);
        regalloc(n3, l, Z);
        gopcode(OAS, n2, Z, n3);
        gopcode(OAS, n3, Z, n1);
    } else {
        regalloc(n1, l, nn);
        cgen(l, n1);
    }
    if(b->type->shift == 0 && typeu[b->type->etype]) {
        v = ~0 + (1L << b->type->nbits);
        gopcode(OAND, nodconst(v), Z, n1);
    } else {

```

```

sh = 32 - b->type->shift - b->type->nbits;
if(sh > 0)
    gopcode(OASHL, nodconst(sh), Z, n1);
sh += b->type->shift;
if(sh > 0)
    if(typeu[b->type->etype])
        gopcode(OLSHR, nodconst(sh), Z, n1);
    else
        gopcode(OASHR, nodconst(sh), Z, n1);
}

```

Uses OAND 115a, OAS 116a, OASHL 115a, OASHR 115a, OLSHR 232b, Z 29a, and typeu 231h.

```

⟨function bitstore(arm) 422a⟩≡ (540b)
void
bitstore(Node *b, Node *n1, Node *n2, Node *n3, Node *nn)
{
    long v;
    Node nod, *l;
    int sh;

    /*
     * n1 has adjusted/masked value
     * n2 has address of cell
     * n3 has contents of cell
     */
    l = b->left;
    regalloc(&nod, l, Z);
    v = ~0 + (1L << b->type->nbits);
    gopcode(OAND, nodconst(v), Z, n1);
    gopcode(OAS, n1, Z, &nod);
    if(nn != Z)
        gopcode(OAS, n1, Z, nn);
    sh = b->type->shift;
    if(sh > 0)
        gopcode(OASHL, nodconst(sh), Z, &nod);
    v <=< sh;
    gopcode(OAND, nodconst(~v), Z, n3);
    gopcode(OOR, n3, Z, &nod);
    gopcode(OAS, &nod, Z, n2);

    regfree(&nod);
    regfree(n1);
    regfree(n2);
    regfree(n3);
}

```

Uses OAND 115a, OAS 116a, OASHL 115a, OOR 115a, and Z 29a.

17.1.6 Old style prototypes

K&R-style (“old style”) function declarations list parameter names without types: `int f(x, y) int x; int y; \...\. The parser handles this with a separate pdecl rule that associates types with previously-unnamed parameters. This requires a second pass (tmerge) to reconcile the old-style parameter list with any prior forward declaration that used a prototype. 5c warns when it encounters old-style functions but still supports them for backwards compatibility.`

```

⟨parameter declarator rules 422b⟩+≡ (101d) <136c
/*

```

```

    * parameter declarator
    */
pdecl:
    /* empty */
    | pdecl ctlist pdlist ';'

pdlist:
    xdecor                { dodecl(pdecl, lastclass, lasttype, $1); }
    | pdlist ',' pdlist

⟨constant OLDPROTO 423a⟩≡ (525)
#define OLDPROTO 2

⟨argmark() if old proto style 423b⟩≡ (138a)
if(pass != 0 && anyproto(n->right) == OLDPROTO) {
    t = typ(TFUNC, n->left->sym->type->link);
    t->down = typ(TOLD, T);
    t->down->down = ofnproto(n->right);
    tmerge(t, n->left->sym);
    n->left->sym->type = t;
}

```

Uses OLDPROTO-91 423a, T 31i, TFUNC 31c, TOLD 424d, anyproto() 424b, ofnproto() 423d, tmerge() 148b, and typ() 32a.

```

⟨walkparam() when OPROTO case, if no name found and old style decl 423c⟩≡ (139b)
dodecl(NODECL, CPARAM, n->type, n->left);
pdecl(CPARAM, lastdcltype, S);

```

Uses CPARAM 33b, NODECL 118d, S 35d, dodecl() 103a, lastdcltype 103b, and pdecl() 139c.

```

⟨function ofnproto 423d⟩≡ (525)
Type*
ofnproto(Node *n)
{
    Type *tl, *tr, *t;

    if(n == Z)
        return T;
    switch(n->op) {
    case OLIST:
        tl = ofnproto(n->left);
        tr = ofnproto(n->right);
        if(tl == T)
            return tr;
        tl->down = tr;
        return tl;

    case ONAME:
        t = copytyp(n->sym->type);
        t->down = T;
        return t;
    }
    return T;
}

```

Uses OLIST 29b, ONAME 27, T 31i, Z 29a, copytyp() 127d, and ofnproto() 423d.

```

⟨constant ANSIPROTO 423e⟩≡ (525)
#define ANSIPROTO 1

```


<fnproto() old prototype style handling 424a>≡ (136a)

```
int r;

r = anyproto(n->right);
if(r == 0 || (r & OLDPROTO)) {
    if(r & ANSIPROTO)
        diag(n, "mixed ansi/old function declaration: %F", n->left);
    return T;
}
```

Uses ANSIPROTO-90 423e, OLDPROTO-91 423a, T 31i, anyproto() 424b, and diag() 496b.

<function anyproto 424b>≡ (525)

```
bool
anyproto(Node *n)
{
    int r;

    r = false;

loop:
    if(n == Z)
        return r;
    switch(n->op) {
    case OLIST:
        r |= anyproto(n->left);
        n = n->right;
        goto loop;

    case ODOTDOT:
    case OPROTO:
        return r | ANSIPROTO;
    }
    return r | OLDPROTO;
}
```

Uses ANSIPROTO-90 423e, ODOTDOT 137c, OLDPROTO-91 423a, OLIST 29b, OPROTO 136f, Z 29a, and anyproto() 424b.

<walkparam() switch op cases 424c>+≡ (138b) <139a

```
case ONAME:
    s = n->sym;
    if(pass == 0) {
        push1(s);
        s->offset = -1;
        break;
    }
    if(s->offset != -1) {
        if(autooffset == 0) {
            firstarg = s;
            firstargtype = s->type;
        }
        autooffset = align(autooffset, s->type, Aarg1);
        s->offset = autooffset;
        autooffset = align(autooffset, s->type, Aarg2);
    } else
        dodecl(pdecl, CXXX, types[TINT], n);
    break;
```

Uses Aarg1 276b, Aarg2 276b, CXXX 33b, ONAME 27, TINT 30b, autooffset 196d, dodecl() 103a, firstarg 274j, firstargtype 275a, pdecl() 139c, push1() 39f, and types 32b.

<Type_kind_bis misc cases 424d>≡ (120e)

```
TOLD,
```

`<rsametype()` *continue if old style type 425a*≡ (146b)

```

if(t1->etype == TOLD) {
    t1 = t1->down;
    continue;
}
if(t2->etype == TOLD) {
    t2 = t2->down;
    continue;
}

```

Uses TOLD 424d.

`<tmerge()` *switch etype t1 cases 425b*≡ (148b)

```

case TFUNC:
    ta = t1->down;
    tb = t2->down;
    if(ta == T) {
        t1->down = tb;
        break;
    }
    if(tb == T)
        break;
    while(ta != T && tb != T) {
        if(ta == tb)
            break;
        /* ignore old-style flag */
        if(ta->etype == TOLD) {
            ta = ta->down;
            continue;
        }
        if(tb->etype == TOLD) {
            tb = tb->down;
            continue;
        }
        /* checking terminated by ... */
        if(ta->etype == TDOT && tb->etype == TDOT) {
            ta = T;
            tb = T;
            break;
        }
        if(!sametype(ta, tb))
            break;
        ta = ta->down;
        tb = tb->down;
    }
    if(ta != tb)
        diag(Z, "function inconsistently declared: %s", s->name);

    /* take new-style over old-style */
    ta = t1->down;
    tb = t2->down;
    if(ta != T && ta->etype == TOLD)
        if(tb != T && tb->etype != TOLD)
            t1->down = tb;
    break;

```

Uses T 31i, TDOT 137f, TFUNC 31c, TOLD 424d, Z 29a, diag() 496b, and sametype() 145c.

`<tcomo()` *when OFUNC case, warn function args not checked if old proto 425c*≡ (172b)

```

if(!debug['B'])
    if(l->type->down == T || l->type->down->etype == TOLD) {
        nerrors--;
    }

```

```

    diag(n, "function args not checked: %F", 1);
}

```

Uses T 31i, TOLD 424d, debug 474a, diag() 496b, and nerrors 495a.

17.1.7 _Noreturn()

The `_Noreturn` attribute marks functions that never return (like `exits` or `abort`). This suppresses false “missing return” warnings at call sites and enables better dead-code analysis after calls to such functions.

17.2 C extensions

Plan 9’s compiler extends ANSI C with several useful features: unnamed struct/union elements (fields are promoted to the enclosing struct), struct constructors (literals like `(Point)\1, 2\`), `extern register` for per-processor storage (used for the `up` pointer to the current process), `typestr` for runtime type reflection, and `signof` for type signature strings. These extensions are used pervasively in the Plan 9 kernel and libraries.

17.2.1 Typing extensions: typeext()

`typeext` is a hook called at every type-compatibility checkpoint (assignments, comparisons, function arguments, returns). It adjusts the right-hand node’s type to match the left-hand type in three cases: (1) nil pointer constants are promoted to the target pointer type, (2) integer-to-float and float-to-integer implicit conversions are inserted, and (3) unnamed substructure members are resolved by walking the struct hierarchy to find a matching type.

<tcomo() when OEQ/ONE, typing extensions 426a>≡ (164a)

```

    typeext(l->type, r);
    typeext(r->type, l);

```

Uses `typeext()` 426f.

<tcomo() when OAS, typing extension 426b>≡ (167b 166a)

```

    typeext(l->type, r);

```

Uses `typeext()` 426f.

<tcomo() when OASADD, typing extension 426c>≡ (167c)

```

    typeext(l->type, r);

```

Uses `typeext()` 426f.

<tcomo() when ORETURN, typing extensions 426d>≡ (179a)

```

    typeext(n->type, l);

```

Uses `typeext()` 426f.

<tcoma() typing extension on argument 426e>≡ (173a)

```

    typeext(t, n);

```

Uses `typeext()` 426f.

<function typeext 426f>≡ (518c)

```

void
typeext(Type *st, Node *l)
{

```

```

    Type *lt = l->type;

```

```

    <typeext() other locals 427f>

```

```

    <typeext() sanity check lt 427a>

```

```

    <typeext() nil pointer 427b>

```

```

    <typeext() float handling 450a>

```

```

    <typeext() unnamed substructure handling 428a>

```

```

}

```

`<typeext() sanity check lt 427a>≡` (426f)

```
    if(lt == T)
        return;
```

Uses T 31i.

`<typeext() nil pointer 427b>≡` (426f)

```
    if(st->etype == TIND && vconst(1) == 0) {
        l->type = st;
        l->vconst = 0;
        return;
    }
```

Uses TIND 31c and vconst() 176c.

17.2.2 Unnamed structure elements

Plan 9 extends C with unnamed struct/union fields: writing `Lock`; inside a struct definition embeds all of `Lock`'s fields directly, like poor man's inheritance. The `dotsearch` function recursively searches embedded types when resolving a field access, so `p->lock` works even though `lock` is a field of the embedded `Lock`, not of the enclosing struct.

`<zedList rule 427c>≡` (131c)

```
zedlist:                /* extension */
/* empty */
{
    lastfield = 0;
    edecl(CXXX, lasttype, S);
}
| edlist
```

`<edecl() if unnamed structure element 427d>≡` (132c)

```
if(s == S) {
    if(!typesu[t->etype])
        diag(Z, "unnamed structure element must be struct/union");
    if(c != CXXX)
        diag(Z, "unnamed structure element cannot have class");
}
```

Uses CXXX 33b, S 35d, Z 29a, diag() 496b, and typesu 149a.

`<dotsearch() if unnamed substructures 427e>≡` (171c)

```
/*
 * look it up in unnamed substructures
 */
for(t1 = t; t1 != T; t1 = t1->down)
    if(t1->sym == S && typesu[t1->etype]){
        // recurse!
        rt = dotsearch(s, t1->link, n, off);
        if(rt != T) {
            if(xt != T)
                goto ambig;
            xt = rt;
            *off += t1->offset;
        }
    }
```

Uses S 35d, T 31i, dotsearch() 171c, and typesu 149a.

`<typeext() other locals 427f>≡` (426f)

```
Node *n1, *n2;
long o;
```

```

⟨typeext() unnamed substructure handling 428a⟩≡ (426f)
/*
 * extension of C
 * if assign of struct containing unnamed sub-struct
 * to type of sub-struct, insert the DOT.
 * if assign of *struct containing unnamed substruct
 * to type of *sub-struct, insert the add-offset
 */
if(typesu[st->etype] && typesu[lt->etype]) {
    o = dotoffset(st, lt, 1);
    if(o >= 0) {
        n1 = new1(OXXX, Z, Z);
        *n1 = *l;
        l->op = ODOT;
        l->left = n1;
        l->right = Z;
        makedot(l, st, o);
    }
    return;
}
// else

if(st->etype == TIND && typesu[st->link->etype])
if(lt->etype == TIND && typesu[lt->link->etype]) {
    o = dotoffset(st->link, lt->link, 1);
    if(o >= 0) {
        l->type = st;
        if(o == 0)
            return;
        // else
        n1 = new1(OXXX, Z, Z);
        *n1 = *l;
        n2 = new1(OCONST, Z, Z);
        n2->vconst = o;
        n2->type = st;
        l->op = OADD;
        l->left = n1;
        l->right = n2;
    }
    return;
}

```

Uses OADD 114c, OCONST 111a, ODOT 117a, OXXX 27, TIND 31c, Z 29a, dotoffset() 428b, makedot() 241b, new1() 29h, and typesu 149a.

```

⟨function dotoffset 428b⟩≡ (518c)
/// typeext -> <>
long
dotoffset(Type *st, Type *lt, Node *n)
{
    Type *t;
    Sym *g;
    long o, o1;

    o = -1;
    /*
     * first try matching at the top level
     * for matching tag names
     */
    g = st->tag;
    if(g != S)

```

```

    for(t=lt->link; t!=T; t=t->down)
        if(t->sym == S)
            if(g == t->tag) {
                if(o >= 0)
                    goto ambig;
                o = t->offset;
            }
    if(o >= 0)
        return o;

/*
 * second try matching at the top level
 * for similar types
 */
for(t=lt->link; t!=T; t=t->down)
    if(t->sym == S)
        if(sametype(st, t)) {
            if(o >= 0)
                goto ambig;
            o = t->offset;
        }
    if(o >= 0)
        return o;

/*
 * last try matching sub-levels
 */
for(t=lt->link; t!=T; t=t->down)
    if(t->sym == S)
        if(typesu[t->etype]) {
            // recurse
            o1 = dotoffset(st, t, n);
            if(o1 >= 0) {
                if(o >= 0)
                    goto ambig;
                o = o1 + t->offset;
            }
        }
    return o;

ambig:
    diag(n, "ambiguous unnamed structure element");
    return o;
}

```

Uses S [35d](#), T [31i](#), diag() [496b](#), dotoffset() [428b](#), sametype() [145c](#), and typesu [149a](#).

17.2.3 Struct constructors

Struct constructors (e.g., (Point)\1, 2\) are a Plan 9 extension that creates a struct value inline as an expression. The compiler allocates a hidden static variable for the constructor's data, initializes it via ADATA, and replaces the constructor expression with a reference to that static. This is essentially syntactic sugar for what would otherwise require a named temporary variable.

$\langle \text{expression nodes } 429 \rangle + \equiv$ (27) $\langle 262b \ 442a \rangle$
 OSTRUCT,
 OUNION,

<xuexpr rule 430a>+≡ (110a) *<118b*

```
| '(' tlist abdecor ')' '{' ilist '}' /* extension */
{
    $$ = new(OSTRUCT, $6, Z);
    dodecl(NODECL, CXXX, $2, $3);
    $$->type = lastdcltype;
}
```

<tcom() switch node kind cases 430b>+≡ (153b) *<227e 442f>*

```
case OSTRUCT:
    if(tcomx(n))
        goto bad;
    break;
```

Uses OSTRUCT 429 and tcomx() 430c.

<function tcomx 430c>≡ (529)

```
bool
tcomx(Node *n)
{
    Type *t;
    Node *l, *r, **ar, **al;
    bool e;

    e = false;
    if(n->type->etype != TSTRUCT) {
        diag(n, "constructor must be a structure");
        return true;
    }
    l = invert(n->left);
    n->left = l;
    al = &n->left;
    for(t = n->type->link; t != T; t = t->down) {
        if(l == Z) {
            diag(n, "constructor list too short");
            return true;
        }
        if(l->op == OLIST) {
            r = l->left;
            ar = &l->left;
            al = &l->right;
            l = l->right;
        } else {
            r = l;
            ar = al;
            l = Z;
        }
        if(tcom(r))
            e++;
        typeext(t, r);
        if(tcompat(n, t, r->type, tasign))
            e++;
        constas(n, t, r->type);
        if(!e && !sametype(t, r->type)) {
            r = new1(OCAST, r, Z);
            r->type = t;
            *ar = r;
        }
    }
    if(l != Z) {
        diag(n, "constructor list too long");
    }
}
```

```

        return true;
    }
    return e;
}

```

Uses OCAST 118a, OLIST 29b, T 31i, TSTRUCT 31c, Z 29a, constas() 190d, diag() 496b, invert() 106a, new1() 29h, sametype() 145c, tasign 166c, tcom() 152b, tcompat() 150a, and typeext() 426f.

$\langle \text{sugen}() \text{ switch op cases } 431 \rangle \equiv (243b)$

```

case OSTRUCT:
/*
 * rewrite so lhs has no fn call
 */
if(nn != Z && nn->complex >= FNX) {
    nod1 = *n;
    nod1.type = typ(TIND, n->type);
    regret(&nod2, &nod1);
    lcgen(nn, &nod2);
    regsalloc(&nod0, &nod1);
    gopcode(OAS, &nod2, Z, &nod0);
    regfree(&nod2);

    nod1 = *n;
    nod1.op = OIND;
    nod1.left = &nod0;
    nod1.right = Z;
    nod1.complex = 1;

    sugen(n, &nod1, w);
    return;
}

r = n->left;
for(t = n->type->link; t != T; t = t->down) {
    l = r;
    if(r->op == OLIST) {
        l = r->left;
        r = r->right;
    }
    if(nn == Z) {
        cgen(l, nn);
        continue;
    }
/*
 * hand craft *(&nn + o) = 1
 */
    nod0 = znode;
    nod0.op = OAS;
    nod0.type = t;
    nod0.left = &nod1;
    nod0.right = l;

    nod1 = znode;
    nod1.op = OIND;
    nod1.type = t;
    nod1.left = &nod2;

    nod2 = znode;
    nod2.op = OADD;
    nod2.type = typ(TIND, t);
    nod2.left = &nod3;

```



```

    nod2.right = &nod4;

    nod3 = znode;
    nod3.op = OADDR;
    nod3.type = nod2.type;
    nod3.left = nn;

    nod4 = znode;
    nod4.op = OCONST;
    nod4.type = nod2.type;
    nod4.vconst = t->offset;

    ccom(&nod0);
    acom(&nod0);
    xcom(&nod0);
    nod0.addable = 0;

    cgen(&nod0, Z);
}
break;

```

Uses OADD 114c, OADDR 116e, OAS 116a, OCONST 111a, OIND 116e, OLIST 29b, OSTRUCT 429, T 31i, TIND 31c, Z 29a, acom() 328e, ccom() 292, and typ() 32a.

17.2.4 Per-processor storage: extern register

The `extern register` storage class reserves a hardware register for a global variable that is private to each processor. The Plan 9 kernel uses this for the `up` pointer (current process) and `m` pointer (current machine): by keeping them in dedicated registers, context-switch code simply changes which physical processor is running, and each processor sees its own `up/m` without any memory access.

```

⟨Storage_class cases 432a⟩+≡ (33b) <415b 434f>
    CEXREG, // extern register, kencext (used in kernel for mips)

```

```

⟨ginit() reg and resvreg initialisation 432b⟩+≡ (51c) <208c
    /* keep two external registers */
    reg[REGEXT] = 1; // R10
    reg[REGEXT-1] = 1; // R9

```

```

⟨simplec() cases 432c⟩+≡ (126c) <137h 434g>
    case BEXTERN|BREGISTER:
        return CEXREG;

```

Uses BEXTERN 121, BREGISTER 121, and CEXREG 432a.

```

⟨xdecl() switch class cases 432d⟩+≡ (104e) <143e 434h>
    case CEXREG:
        o = exreg(t);
        if(o == 0)
            class = CEXTERN;
        if(s->class == CGLOBL)
            class = CGLOBL;
        break;

```

Uses CEXREG 432a, CEXTERN 33b, and CGLOBL 33b.

```

⟨global exregoffset 432e⟩≡ (535b)
    long exregoffset;

```

```

⟨global exfregoffset 432f⟩≡ (535b)
    long exfregoffset;

```

```

⟨ginit() misc initialisations 433a⟩+≡ (51c) <419d 479b>
    exregoffset = REGEXT;
    exfregoffset = FREEXT;

```

```

⟨function exreg(arm) 433b⟩≡ (537b)
    long
    exreg(Type *t)
    {
        long o;

        if(typechlp[t->etype]) {
            if(exregoffset <= REGEXT-2)
                return 0;

            o = exregoffset;
            if(reg[o] && !resvreg[o])
                return 0;
            resvreg[o] = reg[o] = 1;
            exregoffset--;
            return o;
        }
        if(typefd[t->etype]) {
            if(exfregoffset <= NFREG-1)
                return 0;

            o = exfregoffset + NREG;
            if(reg[o] && !resvreg[o])
                return 0;
            resvreg[o] = reg[o] = 1;
            exfregoffset--;
            return o;
        }
        return 0;
    }

```

Uses resvreg-193 208b, typechlp 215g, and typefd 149c.

```

⟨tcomo() in ONAME case, if extern register 433c⟩≡ (155d)
    if(n->class == CEXREG) {
        n->op = OREGISTER;
        ⟨tcomo() in ONAME case, if extern register, if x86 433d⟩
        n->reg = n->sym->offset;
        n->xoffset = 0;
        break;
    }

```

Uses CEXREG 432a and OREGISTER 208f.

```

⟨tcomo() in ONAME case, if extern register, if x86 433d⟩≡ (433c) ??▷
    if(thechar == '8')
        n->op = OEXREG;

```

Uses OEXREG and thechar 45a.

```

⟨tcomo() when OADDR, check if register 433e⟩≡ (170a)
    if(l->op == OREGISTER) {
        diag(n, "address of a register");
        goto bad;
    }

```

Uses OREGISTER 208f and diag() 496b.

17.2.5 Type reflection: `typestr()`

`typestr` is a Plan 9 extension that embeds a string representation of a variable's type in the object file. When storage class `CTYPESTR` is used, the compiler calls `typestrn` to produce a string encoding of the type graph (e.g., “`*int`” for a pointer to `int`) which is stored as a static string. This is used by `acid` for type-aware formatting: the debugger can retrieve the type string at runtime to decide how to print a value.

```
<itab entries, kenc extensions 434a>+≡ (59) <184b 442c>
    "typestr", LTYPESTR, 0,
```

Uses `LTYPESTR`.

```
<token declarations 434b>+≡ (100a) <184a 442b>
    %token LTYPESTR
```

```
<cname rule 434c>+≡ (122c) <137g
    | LTYPESTR { $$ = BTYPESTR; }
```

```
<Bxxx cases 434d>≡ (121)
    BTYPESTR = 1L<<TTYPESTR,
```

Uses `TTYPESTR` 434e.

```
<Type_kind_bis storage cases 434e>+≡ (120e) <126a
    TTYPESTR,
```

```
<Storage_class cases 434f>+≡ (33b) <432a
    CTYPESTR,
```

```
<simplec() cases 434g>+≡ (126c) <432c
    case BTYPESTR:
        return CTYPESTR;
```

Uses `BTYPESTR` 434d and `CTYPESTR` 434f.

```
<xdecl() switch class cases 434h>+≡ (104e) <432d
    case CTYPESTR:
        if(!typesuv[t->etype]) {
            diag(Z, "typestr must be struct/union: %s", s->name);
            break;
        }
        dclfunct(t, s);
        break;
```

Uses `CTYPESTR` 434f, `Z` 29a, `dclfunct()` 434i, `diag()` 496b, and `typesuv` 271c.

```
<function dclfunct 434i>≡ (527)
    /// dodecl(xdecl, ...) -> <>
    void
    dclfunct(Type *t, Sym *s)
    {
        Funct *f;
        Node *n;
        Type *f1, *f2, *f3, *f4;
        int o, i, c;
        char str[100];

        if(t->funct)
            return;

        // recognize generated tag of dorm _%d_
        if(t->tag == S)
            goto bad;
```

```

for(i=0; c = t->tag->name[i]; i++) {
    if(c == '_' ) {
        if(i == 0 || t->tag->name[i+1] == 0)
            continue;
        break;
    }
    if(c < '0' || c > '9')
        break;
}
if(c == 0)
    goto bad;

f = alloc(sizeof(*f));
for(o=0; o<sizeof(f->sym); o++) // bugfix?nelem(f->sym) in latest plan9
    f->sym[o] = S;

t->funct = f;

f1 = typ(TFUNC, t);
f1->down = copytyp(t);
f1->down->down = t;

f2 = typ(TFUNC, types[TINT]);
f2->down = copytyp(t);
f2->down->down = t;

f3 = typ(TFUNC, t);
f3->down = typ(TIND, t);
f3->down->down = t;

f4 = typ(TFUNC, t);
f4->down = t;

for(i=0;; i++) {
    o = ftabinit[i].op;
    if(o == 0XXX)
        break;
    sprintf(str, "%s_%s_", t->tag->name, ftabinit[i].name);
    n = new(ONAME, Z, Z);
    n->sym = slookup(str);
    f->sym[o] = n->sym;

    switch(ftabinit[i].typ) {

case 1: // T f(T,T) +
        dodecl(xdecl, CEXTERN, f1, n);
        break;

case 2: // int f(T,T) ==
        dodecl(xdecl, CEXTERN, f2, n);
        break;

case 3: // void f(T*,T) +=
        dodecl(xdecl, CEXTERN, f3, n);
        break;

case 4: // T f(T) ~
        dodecl(xdecl, CEXTERN, f4, n);
        break;

```

```

    default:
        diag(Z, "dclfunct op missing %d\n", ftabinit[i].typ);
        break;
    }
}
for(i=0;; i++) {
    o = gtabinit[i].etype;
    if(o == TXXX)
        break;

    /*
     * OCAST types T1 _T2_T1_(T2)
     */
    sprintf(str, "_%s%s_", gtabinit[i].name, t->tag->name);
    n = new(ONAME, Z, Z);
    n->sym = slookup(str);
    f->castto[o] = n->sym;

    f1 = typ(TFUNC, t);
    f1->down = types[o];
    dodecl(xdecl, CEXTERN, f1, n);

    sprintf(str, "%s%s_", t->tag->name, gtabinit[i].name);
    n = new(ONAME, Z, Z);
    n->sym = slookup(str);
    f->castfr[o] = n->sym;

    f1 = typ(TFUNC, types[o]);
    f1->down = t;
    dodecl(xdecl, CEXTERN, f1, n);
}
return;
bad:
    diag(Z, "dclfunct bad %T %s\n", t, s->name);
}

```

Uses CEXTERN 33b, ONAME 27, OXXX 27, S 35d, TFUNC 31c, TIND 31c, TINT 30b, TXXX 30a, Z 29a, alloc() 500b, copytyp() 127d, diag() 496b, dodecl() 103a, ftabinit 437, gtabinit 436c, new() 29d, slookup() 35f, typ() 32a, types 32b, and xdecl() 104e.

⟨struct Ftab 436a⟩≡ (527)

```

struct Ftab
{
    char op;
    char* name;
    char typ;
};

```

⟨struct Gtab 436b⟩≡ (527)

```

struct Gtab
{
    char etype;
    char* name;
};

```

⟨global gtabinit 436c⟩≡ (527)

```

Gtab gtabinit[NTYPE] =
{
    TCHAR,  "c",
    TUCCHAR, "uc",
    TSHORT, "h",
    TUSHORT, "uh",

```

```

TINT,  "i",
TUINT, "ui",
TLONG, "l",
TULONG, "ul",
TVLONG, "v",
TUVLONG, "uv",
TFLOAT, "f",
TDOUBLE, "d",

```

```

TXXX

```

```

};

```

Uses NTYPE 30a, TCHAR 30b, TDOUBLE 31a, TFLOAT 31a, TINT 30b, TLONG 30b, TSHORT 30b, TCHAR 30b, TUINT 30b, TULONG 30b, TUSHORT 30b, TUVLONG 30b, TVLONG 30b, and TXXX 30a.

(global ftabinit 437)≡ (527)

```

Ftab ftabinit[OEND] =

```

```

{

```

```

    OADD,  "add",  1,
    OAND,  "and",  1,
    OASHL, "ashl", 1,
    OASHR, "ashr", 1,
    ODIV,  "div",  1,
    OLDIV, "ldiv", 1,
    OLMOD, "lmod", 1,
    OLMUL, "lmul", 1,
    OLSHR, "lshr", 1,
    OMOD,  "mod",  1,
    OMUL,  "mul",  1,
    OOR,   "or",   1,
    OSUB,  "sub",  1,
    OXOR,  "xor",  1,

```

```

    OEQ,   "eq",   2,
    OGE,   "ge",   2,
    OGT,   "gt",   2,
    OHI,   "hi",   2,
    OHS,   "hs",   2,
    OLE,   "le",   2,
    OLO,   "lo",   2,
    OLS,   "ls",   2,
    OLT,   "lt",   2,
    ONE,   "ne",   2,

```

```

    OASADD, "asadd", 3,
    OASAND, "asand", 3,
    OASASHL, "asashl", 3,
    OASASHR, "asashr", 3,
    OASDIV, "asdiv", 3,
    OASLDIV, "asldiv", 3,
    OASLMOD, "aslmod", 3,
    OASLMUL, "aslmul", 3,
    OASLSHR, "aslshr", 3,
    OASMOD, "asmod", 3,
    OASMUL, "asmul", 3,
    OASOR,  "asor",  3,
    OASSUB, "assub", 3,
    OASXOR, "asxor", 3,

```

```

    OPOS,  "pos",  4,
    ONEG,  "neg",  4,

```

```

    OCOM, "com", 4,
    ONOT, "not", 4,

// OPOSTDEC,
// OPOSTINC,
// OPREDEC,
// OPREINC,

    OXXX,
};

Uses OADD 114c, OAND 115a, OASADD 116c, OASAND 116c, OASASHL 116c, OASASHR 116c, OASDIV 116c, OASHL 115a, OASHR 115a,
OASLDIV 233b, OASLMOD 233b, OASLMUL 233b, OASLSHR 233b, OASMOD 116c, OASMUL 116c, OASOR 116c, OASSUB 116c, OASXOR 116c,
OCOM 115e, ODIV 114c, OEND 27, OEQ 115g, OGE 115g, OGT 115g, OHI 300d, OHS 300d, OLDIV 232d, OLE 115g, OLMOD 232f,
OLMUL 232d, OLO 300d, OLS 300d, OLSHR 232b, OLT 115g, OMOD 114c, OMUL 114c, ONE 115g, ONEG 114e, ONOT 115e, OOR 115a,
OPOS 114e, OSUB 114c, OXOR 115a, and OXXX 27.

⟨tcomo() break if isfunct 438a⟩≡ (177 175d 169 168f 167c 166a 165 164 159 158 157 156)
    if(isfunct(n))
        break;

Uses isfunct() 438d.

⟨Type other fields 438b⟩+≡ (31d) <31f
    Funct* funct;

⟨struct Funct 438c⟩≡ (504b)
    struct Funct
    {
        Sym* sym[OEND];
        Sym* castto[NTYPE];
        Sym* castfr[NTYPE];
    };

⟨function isfunct 438d⟩≡ (527)
    bool
    isfunct(Node *n)
    {
        Type *t, *t1;
        Funct *f;
        Node *l;
        Sym *s;
        int o;

        o = n->op;
        if(n->left == Z)
            goto no;
        t = n->left->type;
        if(t == T)
            goto no;
        f = t->funct;

        switch(o) {
        case OAS: // put cast on rhs
        case OASI:
        case OASADD:
        case OASSUB:
        case OASMUL:
        case OASDIV:
        case OASMOD:
        case OASASHR:
        case OASLMUL:

```

```

case OASLDIV:
case OASLMOD:
case OASLSHR:
case OASASHL:
case OASAND:
case OASOR:
case OASXOR:
    if(n->right == Z)
        goto no;
    t1 = n->right->type;
    if(t1 == T)
        goto no;
    if(t1->funct == f)
        break;

    l = new(OXXX, Z, Z);
    *l = *n->right;

    n->right->left = l;
    n->right->right = Z;
    n->right->type = t;
    n->right->op = OCAST;

    if(!isfunct(n->right))
        prttree(n, "isfunc !");
    break;

case OCAST: // t f(T) or T f(t)
    t1 = n->type;
    if(t1 == T)
        goto no;
    if(f != nil) {
        s = f->castfr[t1->etype];
        if(s == S)
            goto no;
        n->right = n->left;
        goto build;
    }
    f = t1->funct;
    if(f != nil) {
        s = f->castto[t->etype];
        if(s == S)
            goto no;
        n->right = n->left;
        goto build;
    }
    goto no;
}

if(f == nil)
    goto no;
s = f->sym[o];
if(s == S)
    goto no;

/*
 * the answer is yes,
 * now we rewrite the node

```



```

    * and give diagnostics
    */
switch(o) {
case OADD: // T f(T, T)
case OAND:
case OASHL:
case OASHR:
case ODIV:
case OLDIV:
case OLMOD:
case OLMUL:
case OLSHR:
case OMOD:
case OMUL:
case OOR:
case OSUB:
case OXOR:

case OEQ: // int f(T, T)
case OGE:
case OGT:
case OHI:
case OHS:
case OLE:
case OLO:
case OLS:
case OLT:
case ONE:
    if(n->right == Z)
        goto bad;
    t1 = n->right->type;
    if(t1 == T)
        goto bad;
    if(t1->funct != f)
        goto bad;
    n->right = new(OLIST, n->left, n->right);
    break;

case OAS: // structure copies done by the compiler
case OASI:
    goto no;

case OASADD: // T f(T*, T)
case OASAND:
case OASASHL:
case OASASHR:
case OASDIV:
case OASLDIV:
case OASLMOD:
case OASLMUL:
case OASLSHR:
case OASMOD:
case OASMUL:
case OASOR:
case OASSUB:
case OASXOR:
    if(n->right == Z)
        goto bad;
    t1 = n->right->type;
    if(t1 == T)

```

```

        goto bad;
    if(t1->funct != f)
        goto bad;
    n->right = new(OLIST, new(OADDR, n->left, Z), n->right);
    break;

case OPOS: // T f(T)
case ONEG:
case ONOT:
case OCOM:
    n->right = n->left;
    break;

default:
    diag(n, "isfunct op missing %0\n", o);
    goto bad;

}

build:
    l = new(ONAME, Z, Z);
    l->sym = s;
    l->type = s->type;
    l->etype = s->type->etype;
    l->xoffset = s->offset;
    l->class = s->class;
    tcomo(l, 0);

    n->op = OFUNC;
    n->left = l;
    n->type = l->type->link;
    if(tcompat(n, T, l->type, tfunct))
        goto bad;
    if(tcoma(n->left, n->right, l->type->down, true))
        goto bad;
    return true;

no:
    return false;

bad:
    diag(n, "cant rewrite typestr for op %0\n", o);
    prttree(n, "isfunct");
    n->type = T;
    return true;
}

```

Uses OADD 114c, OADDR 116e, OAND 115a, OAS 116a, OASADD 116c, OASAND 116c, OASASHL 116c, OASASHR 116c, OASDIV 116c, OASHL 115a, OASHR 115a, OASI 262b, OASLDIV 233b, OASLMOD 233b, OASLMUL 233b, OASLSHR 233b, OASMOD 116c, OASMUL 116c, OASOR 116c, OASSUB 116c, OASXOR 116c, OCAST 118a, OCOM 115e, ODIV 114c, OEQ 115g, OFUNC 117c, OGE 115g, OGT 115g, OHI 300d, OHS 300d, OLDIV 232d, OLE 115g, OLIST 29b, OLMOD 232f, OLMUL 232d, OLO 300d, OLS 300d, OLSHR 232b, OLT 115g, OMOD 114c, OMUL 114c, ONAME 27, ONE 115g, ONEG 114e, ONOT 115e, OOR 115a, OPOS 114e, OSUB 114c, OXOR 115a, OXXX 27, S 35d, T 31i, Z 29a, diag() 496b, isfunct() 438d, new() 29d, prttree() 487b, tcoma() 173a, tcomo() 153b, tcompat() 150a, and tfunct 172c.

17.2.6 Signature reflection: `signof()`

`signof` is a compile-time operator (like `sizeof`) that returns the type signature hash of its argument. This is the same hash used by 5c -T for safe linking. The kernel uses it to verify at runtime that the types expected

by a device driver match the types defined in the kernel headers.

<expression nodes 442a>+≡ (27) <429
 OSIGN,

<token declarations 442b>+≡ (100a) <434b
 %token LSIGNOF

<itab entries, kenc extensions 442c>+≡ (59) <434a
 "signof", LSIGNOF, 0,
 Uses LSIGNOF.

<uepr rule 442d>+≡ (110a) <119c
 | LSIGNOF uepr { \$\$ = new(OSIGN, \$2, Z); }

<pepr rule 442e>+≡ (110a) <119b
 | LSIGNOF '(' tlist abdecor ')' {
 {
 \$\$ = new(OSIGN, Z, Z);
 dodecl(NODECL, CXXX, \$3, \$4);
 \$\$->type = lastdcltype;
 }

<tcomo() switch node kind cases 442f>+≡ (153b) <430b
 case OSIGN: /* extension signof(type) returns a hash */
 if(l != Z) {
 if(l->op != OSTRING && l->op != OLSTRING)
 if(tcomo(l, 0))
 goto bad;
<tcomo() when OSIGN, sanity check no bitfield 442g>
 n->type = l->type;
 }
<tcomo() when OSIGN, typecheck and sanity check 442h>
 n->op = OCONST;
 n->left = Z;
 n->right = Z;
 n->vconst = convvtox(signature(n->type), TULONG);
 n->type = types[TULONG];
 break;

Uses OCONST 111a, OLSTRING 113a, OSIGN 442a, OSTRING 112c, TULONG 30b, Z 29a, convvtox() 65d, signature() 394a 395c, tcomo() 153b, and types 32b.

<tcomo() when OSIGN, sanity check no bitfield 442g>≡ (442f)
 if(l->op == OBIT) {
 diag(n, "signof bitfield");
 goto bad;
 }

Uses OBIT 418d and diag() 496b.

<tcomo() when OSIGN, typecheck and sanity check 442h>≡ (442f)
 if(n->type == T)
 goto bad;
 if(n->type->width < 0) {
 diag(n, "signof undefined type");
 goto bad;
 }

Uses T 31i and diag() 496b.

17.2.7 Format's arguments checking

With `#pragma argcheck`, the compiler can verify that `printf`-style format strings match their arguments at compile time — similar to gcc's `__attribute__((format))` or OCaml's type-safe `Printf`. The compiler parses the format string, extracts the expected types for each `%d`, `%s`, etc., and checks them against the actual argument types.

<enum fxxx 443a>≡ (528b)

```
enum fxxx
{
    Fnone = 0,

    Fl,
    Fvl,
    Fignor,
    Fstar,
    Fadj,

    Fverb = 10,
};
```

<struct Tprot 443b>≡ (528b)

```
struct Tprot
{
    Type* type;
    Bits flag;
    Tprot* link;
};
```

<struct Tname 443c>≡ (528b)

```
struct Tname
{
    char* name;
    int param;
    Tname* link;
};
```

<global indchar 443d>≡ (528b)

```
static Type* indchar;
```

<global flagbits 443e>≡ (528b)

```
static uchar flagbits[512];
```

<global fmtbuf 443f>≡ (528b)

```
static char fmtbuf[100];
```

<global lastadj 443g>≡ (528b)

```
static int lastadj;
```

<global lastverb 443h>≡ (528b)

```
static int lastverb;
```

<global nstar 443i>≡ (528b)

```
static int nstar;
```

<global tprot 443j>≡ (528b)

```
static Tprot* tprot;
```

<global tname 443k>≡ (528b)

```
static Tname* tname;
```

<function argflag 444a>≡ (528b)

```
void
argflag(int c, int v)
{

    switch(v) {
    case Fignor:
    case Fstar:
    case Fl:
    case Fvl:
        flagbits[c] = v;
        break;
    case Fverb:
        flagbits[c] = lastverb;
/*print("flag-v %c %d\n", c, lastadj);*/
        lastverb++;
        break;
    case Fadj:
        flagbits[c] = lastadj;
/*print("flag-l %c %d\n", c, lastadj);*/
        lastadj++;
        break;
    }
}
```

Uses Fadj-183 443a, Fignor-181 443a, Fl-179 443a, Fstar-182 443a, Fverb-184 443a, Fvl-180 443a, flagbits-186 443e, lastadj-188 443g, and lastverb-189 443h.

<function getflag 444b>≡ (528b)

```
Bits
getflag(char *s)
{
    Bits flag;
    int f;
    char *fmt;
    Rune c;

    fmt = fmtbuf;
    flag = zbits;
    nstar = 0;
    for(;;) {
        s += chartorune(&c, s);
        fmt += runetochar(fmt, &c);
        if(c == 0 || c >= nelem(flagbits))
            break;
        f = flagbits[c];
        switch(f) {
        case Fnone:
            argflag(c, Fverb);
            f = flagbits[c];
            break;
        case Fstar:
            nstar++;
        case Fignor:
            continue;
        case Fl:
            if(bset(flag, Fl))
                flag = bor(flag, blsh(Fvl));
        }
        flag = bor(flag, blsh(f));
        if(f >= Fverb)
```

```

        break;
    }
    *fmt = 0;
    return flag;
}

```

Uses Fignor-181 443a, Fl-179 443a, Fnone-178 443a, Fstar-182 443a, Fverb-184 443a, Fvl-180 443a, argflag() 444a, blsh() 351a, bor() 350b, flagbits-186 443e, fmtbuf-187 443f, nstar-190 443i, and zbits 350a.

```

⟨function arginit 445a⟩≡ (528b)
void
arginit(void)
{
    int i;

    /* debug['F'] = 1;*/
    /* debug['w'] = 1;*/

    lastadj = Fadj;
    lastverb = Fverb;
    indchar = typ(TIND, types[TCHAR]);

    memset(flagbits, Fnone, sizeof(flagbits));

    for(i='0'; i<='9'; i++)
        argflag(i, Fignor);
    argflag('.', Fignor);
    argflag('#', Fignor);
    argflag('u', Fignor);
    argflag('h', Fignor);
    argflag('+', Fignor);
    argflag('-', Fignor);

    argflag('*', Fstar);
    argflag('l', Fl);

    argflag('o', Fverb);
    flagbits['x'] = flagbits['o'];
    flagbits['X'] = flagbits['o'];
}

```

Uses Fadj-183 443a, Fignor-181 443a, Fl-179 443a, Fnone-178 443a, Fstar-182 443a, Fverb-184 443a, TCHAR 30b, TIND 31c, argflag() 444a, flagbits-186 443e, indchar-185 443d, lastadj-188 443g, lastverb-189 443h, typ() 32a, and types 32b.

```

⟨function nextarg 445b⟩≡ (528b)
Node*
nextarg(Node *n, Node **a)
{
    if(n == Z) {
        *a = Z;
        return Z;
    }
    if(n->op == OLIST) {
        *a = n->left;
        return n->right;
    }
    *a = n;
    return Z;
}

```

Uses OLIST 29b and Z 29a.

<function checkargs 446>≡

(528b)

```
void
checkargs(Node *nn, char *s, int pos)
{
    Node *a, *n;
    Bits flag;
    Tprot *l;

    if(!debug['F'])
        return;
    n = nn;
    for(;;) {
        s = strchr(s, '%');
        if(s == 0) {
            nextarg(n, &a);
            if(a != Z)
                warn(nn, "more arguments than format %T",
                    a->type);
            return;
        }
        s++;
        flag = getflag(s);
        while(nstar > 0) {
            n = nextarg(n, &a);
            pos++;
            nstar--;
            if(a == Z) {
                warn(nn, "more format than arguments %s",
                    fmtbuf);
                return;
            }
            if(a->type == T)
                continue;
            if(!sametype(types[TINT], a->type) &&
                !sametype(types[TUINT], a->type))
                warn(nn, "format mismatch '*' in %s %T, arg %d",
                    fmtbuf, a->type, pos);
        }
        for(l=tprot; l; l=l->link)
            if(sametype(types[TVOID], l->type)) {
                if(beq(flag, l->flag)) {
                    s++;
                    goto loop;
                }
            }

        n = nextarg(n, &a);
        pos++;
        if(a == Z) {
            warn(nn, "more format than arguments %s",
                fmtbuf);
            return;
        }
        if(a->type == 0)
            continue;
        for(l=tprot; l; l=l->link)
            if(sametype(a->type, l->type)) {
                /*print("checking %T/%ulx %T/%ulx\n", a->type, flag.b[0], l->type, l->flag.b[0]);*/
                if(beq(flag, l->flag))
                    goto loop;
            }
    }
}
```

```

    }
    warn(nn, "format mismatch %s %T, arg %d", fmtbuf, a->type, pos);
loop:;
}
}

```

Uses T 31i, TINT 30b, TUINT 30b, TVOID 31b, Z 29a, beq() 350d, debug 474a, fmtbuf-187 443f, getflag() 444b, nextarg() 445b, nstar-190 443i, sametype() 145c, tprot-191 443j, types 32b, and warn() 142.

<tcomo() when OFUNC case, format check 447a>≡ (172b)
dpcheck(n);

Uses dpcheck() 447b.

<function dpcheck 447b>≡ (528b)

```

void
dpcheck(Node *n)
{
    char *s;
    Node *a, *b;
    Tname *l;
    int i;

    if(n == Z)
        return;
    b = n->left;
    if(b == Z || b->op != ONAME)
        return;
    s = b->sym->name;
    for(l=tname; l; l=l->link)
        if(strcmp(s, l->name) == 0)
            break;
    if(l == 0)
        return;

    i = l->param;
    b = n->right;
    while(i > 0) {
        b = nextarg(b, &a);
        i--;
    }
    if(a == Z) {
        warn(n, "cant find format arg");
        return;
    }
    if(!sametype(indchar, a->type)) {
        warn(n, "format arg type %T", a->type);
        return;
    }
    if(a->op != OADDR || a->left->op != ONAME || a->left->sym != symstring) {
/* warn(n, "format arg not constant string");*/
        return;
    }
    s = a->left->cstring;
    checkargs(b, s, l->param);
}

```

Uses OADDR 116e, ONAME 27, Z 29a, checkargs() 446, indchar-185 443d, nextarg() 445b, sametype() 145c, symstring 226c, tname-192 443k, and warn() 142.

#pragma varargck xxx

```
<macprag() if pragma xxx 448a>+≡ (96a) <412d 469a>
    if(s && strcmp(s->name, "varargck") == 0) {
        pragvararg();
        return;
    }
```

Uses pragvararg() 448b.

```
<function pragvararg 448b>≡ (528b)
void
pragvararg(void)
{
    Sym *s;
    int n, c;
    char *t;
    Rune r;
    Type *ty;

    if(!debug['F'])
        goto out;

    s = getsym();
    if(s && strcmp(s->name, "argpos") == 0)
        goto ckpos;
    if(s && strcmp(s->name, "type") == 0)
        goto cktype;
    if(s && strcmp(s->name, "flag") == 0)
        goto ckflag;
    yyerror("syntax in #pragma varargck");
    goto out;

ckpos:
    /*#pragma varargck argpos warn 2*/
    s = getsym();
    if(s == S)
        goto bad;
    n = getnsn();
    if(n < 0)
        goto bad;
    newname(s->name, n);
    goto out;

ckflag:
    /*#pragma varargck flag 'c'*/
    c = getnsc();
    if(c != '\')
        goto bad;
    c = getr();
    if(c == '\\')
        c = getr();
    else if(c == '\')
        goto bad;
    if(c == '\n')
        goto bad;
    if(getc() != '\')
        goto bad;
    argflag(c, Fignor);
    goto out;
```

```

cktype:
/**pragma varargck type 0 int*/
    c = getnsc();
    if(c != '')
        goto bad;
    t = fmtbuf;
    for(;;) {
        r = getr();
        if(r == ' ' || r == '\n')
            goto bad;
        if(r == '')
            break;
        t += runetochar(t, &r);
    }
    *t = 0;
    t = strdup(fmtbuf);
    s = getsym();
    if(s == S)
        goto bad;
    ty = s->type;
    while((c = getnsc()) == '*')
        ty = typ(TIND, ty);
    unget(c);
    newprot(s, ty, t);
    goto out;

bad:
    yyerror("syntax in #pragma varargck");

out:
    while(getnsc() != '\n')
        ;
}

```

Uses Fignor-181 443a, S 35d, TIND 31c, argflag() 444a, debug 474a, fmtbuf-187 443f, getc() 78d, getnsc() 77c, getnsn() 97, getr() 72a, getsym() 77b, newname() 449a, newprot() 449b, typ() 32a, unget() 78b, and yyerror() 495c.

```

⟨function newname 449a⟩≡ (528b)
void
newname(char *s, int p)
{
    Tname *l;

    for(l=tname; l; l=l->link)
        if(strcmp(l->name, s) == 0) {
            if(l->param != p)
                yyerror("vargck %s already defined\n", s);
            return;
        }
    l = alloc(sizeof(*l));
    l->name = s;
    l->param = p;
    l->link = tname;
    tname = l;
}

```

Uses alloc() 500b, tname-192 443k, and yyerror() 495c.

```

⟨function newprot 449b⟩≡ (528b)
void
newprot(Sym *m, Type *t, char *s)
{

```

```

Bits flag;
Tprot *l;

if(t == T) {
    warn(Z, "%s: newprot: type not defined", m->name);
    return;
}
flag = getflag(s);
for(l=tprot; l; l=l->link)
    if(beq(flag, l->flag) && sametype(t, l->type))
        return;
l = alloc(sizeof(*l));
l->type = t;
l->flag = flag;
l->link = tprot;
tprot = l;
}

```

Uses T 31i, Z 29a, alloc() 500b, beq() 350d, getflag() 444b, sametype() 145c, tprot-191 443j, and warn() 142.

17.3 Floats

Floating-point support is spread across all compiler phases: lexing (parsing float literals), typing (implicit int→float conversions), and code generation (ARM VFP instructions for arithmetic, loads/stores, and conversions). The type system distinguishes TFLOAT (32-bit) and TDOUBLE (64-bit), with implicit widening from float to double in mixed expressions.

⟨typeext() float handling 450a⟩≡ (426f)
typeext1(st, l);

Uses typeext1() 450d.

⟨tcomo() when OASMUL/OASDIV, typing extensions 450b⟩≡ (168f)
typeext1(l->type, r);

Uses typeext1() 450d.

⟨tcomo() when OLT/OGE/OGT/OLE, typing extensions 450c⟩≡ (164d)
typeext1(l->type, r);
typeext1(r->type, l);

Uses typeext1() 450d.

⟨function typeext1 450d⟩≡ (518c)
void
typeext1(Type *st, Node *l)
{
 if(st->etype == TFLOAT && allfloat(l, false))
 allfloat(l, true);
}

Uses TFLOAT 31a and allfloat() 450e.

⟨function allfloat 450e⟩≡ (518c)
/*
* look into tree for floating point constant expressions
*/
bool
allfloat(Node *n, bool flag)
{
 if(n != Z) {

```

    if(n->type->etype != TDOUBLE)
        return true;
    switch(n->op) {
    case OCONST:
        if(flag)
            n->type = types[TFLOAT];
        return true;
    case OADD: /* no need to get more exotic than this */
    case OSUB:
    case OMUL:
    case ODIV:
        if(!allfloat(n->right, flag))
            break;
    case OCAST:
        if(!allfloat(n->left, flag))
            break;
        if(flag)
            n->type = types[TFLOAT];
        return true;
    }
}
return false;
}

```

Uses OADD 114c, OCAST 118a, OCONST 111a, ODIV 114c, OMUL 114c, OSUB 114c, TDOUBLE 31a, TFLOAT 31a, Z 29a, allfloat() 450e, and types 32b.

$\langle \text{gopcode() when OADD, adjust a if float type 451a} \rangle \equiv$ (230b)

```

    if(et == TFLOAT)
        a = AADDF;
    else
        if(et == TDOUBLE || et == TVLONG)
            a = AADDD;

```

Uses TDOUBLE 31a, TFLOAT 31a, and TVLONG 30b.

$\langle \text{gopcode() when OSUB, adjust a if float type 451b} \rangle \equiv$ (230c)

```

    if(et == TFLOAT)
        a = ASUBF;
    else
        if(et == TDOUBLE || et == TVLONG)
            a = ASUBD;

```

Uses TDOUBLE 31a, TFLOAT 31a, and TVLONG 30b.

$\langle \text{gopcode() when OMUL, adjust a if float type 451c} \rangle \equiv$ (231e)

```

    if(et == TFLOAT)
        a = AMULF;
    else
        if(et == TDOUBLE || et == TVLONG)
            a = AMULD;

```

Uses TDOUBLE 31a, TFLOAT 31a, and TVLONG 30b.

$\langle \text{gopcode() when ODIV, adjust a if float type 451d} \rangle \equiv$ (231f)

```

    if(et == TFLOAT)
        a = ADIVF;
    else
        if(et == TDOUBLE || et == TVLONG)
            a = ADIVD;

```

Uses TDOUBLE 31a, TFLOAT 31a, and TVLONG 30b.

<naddr() if float type 452a>≡ (220a)

```

    if(typeefd[n->type->etype]) {
        a->type = D_FCONST;
        a->dval = n->fconst;
    }

```

Uses typeefd 149c.

<naddr() if float register, adjust a 452b>≡ (220b)

```

    if(a->reg >= NREG) {
        a->type = D_FREG;
        a->reg -= NREG;
    }

```

<regret() if float expression, adjust return register 452c>≡ (207a)

```

    if(typeefd[nn->type->etype])
        r = FREGRET+NREG;

```

Uses typeefd 149c.

<global fconstnode 452d>≡ (535b)

```

Node fconstnode;

```

<ginit() special nodes initialisation 452e>+≡ (51c) <272c

```

fconstnode.op = OCONST;
fconstnode.class = CXXX;
fconstnode.complex = 0;
fconstnode.addable = 20;
fconstnode.type = types[TDOUBLE];

```

Uses CXXX 33b, OCONST 111a, TDOUBLE 31a, and types 32b.

<function nodfconst 452f>≡ (537b)

```

Node*
nodfconst(double d)
{
    fconstnode.fconst = d;
    return &fconstnode;
}

```

<function ieeeedtod 452g>≡ (530a)

```

void
ieeedtod(Ieee *ieee, double native)
{
    double fr, ho, f;
    int exp;

    if(native < 0) {
        ieeedtod(ieee, -native);
        ieee->h |= 0x80000000L;
        return;
    }
    if(native == 0) {
        ieee->l = 0;
        ieee->h = 0;
        return;
    }
    fr = frexp(native, &exp);
    f = 2097152L; /* shouldnt use fp constants here */
    fr = modf(fr*f, &ho);
    ieee->h = ho;
    ieee->h &= 0xfffffL;

```

```

    ieee->h |= (exp+1022L) << 20;
    f = 65536L;
    fr = modf(fr*f, &ho);
    ieee->l = ho;
    ieee->l <= 16;
    ieee->l |= (long)(fr*f);
}

```

Uses `ieeeedtod()` 452g.

```

⟨gclean() sanity check reg 453a⟩+≡ (49c) <208e
    for(i=NREG; i<NREG+NFREG; i++)
        if(reg[i] && !resvreg[i])
            diag(Z, "freg %d left allocated", i-NREG);

```

Uses `Z` 29a, `diag()` 496b, and `resvreg-193` 208b.

```

⟨function raddr(arm) 453b⟩≡ (537b)
    /// gmove -> <>
    void
    raddr(Node *n, Prog *p)
    {
        Adr a;

        naddr(n, &a);
        if(a.type != D_REG && a.type != D_FREG) {
            if(n)
                diag(n, "bad in raddr: %0", n->op);
            else
                diag(n, "bad in raddr: <null>");
            p->reg = R_NONE;
        } else
            p->reg = a.reg;
    }

```

Uses `diag()` 496b.

```

⟨gmove() switch from type cases 453c⟩+≡ (214) <218c
    case TDOUBLE:
    case TVLONG:
    case TFLOAT:
        switch(tt) {
            case TDOUBLE:
            case TVLONG:
                a = AMOVD;
                if(ft == TFLOAT)
                    a = AMOVFD;
                break;
            case TFLOAT:
                a = AMOVDF;
                if(ft == TFLOAT)
                    a = AMOVF;
                break;
            case TINT:
            case TUINT:
            case TLONG:
            case TULONG:
            case TIND:
                a = AMOVDW;
                if(ft == TFLOAT)
                    a = AMOVFW;
                break;
        }

```

```

case TSHORT:
case TUSHORT:
case TCHAR:
case TUCCHAR:
    a = AMOVDW;
    if(ft == TFLOAT)
        a = AMOVFW;
    break;
}
break;

```

Uses TCHAR 30b, TDOUBLE 31a, TFLOAT 31a, TIND 31c, TINT 30b, TLONG 30b, TSHORT 30b, TUCCHAR 30b, TUINT 30b, TULONG 30b, TUSHORT 30b, and TVLONG 30b.

<gmove() switch from type cases, TUINT/TULONG case, if float target 454a>≡ (216d)

```

if(tt == TFLOAT || tt == TDOUBLE) {
    // ugly and probably longer than necessary,
    // but vfp has a single instruction for this,
    // so hopefully it won't last long.
    //
    // tmp = f
    // tmp1 = tmp & 0x80000000
    // tmp ^= tmp1
    // t = float(int32(tmp))
    // if(tmp1)
    //   t += 2147483648.
    //
    regalloc(&nod, f, Z);
    regalloc(&nod1, f, Z);
    gins(AMOVW, f, &nod);
    gins(AMOVW, &nod, &nod1);
    gins(AAND, nodconst(0x80000000), &nod1);
    gins(AEOR, &nod1, &nod);
    if(tt == TFLOAT)
        gins(AMOVWF, &nod, t);
    else
        gins(AMOVWD, &nod, t);
    gins(ACMP, nodconst(0), Z);
    raddr(&nod1, p);
    gins(ABEQ, Z, Z);
    regfree(&nod);
    regfree(&nod1);
    p1 = p;
    regalloc(&nod, t, Z);
    if(tt == TFLOAT) {
        gins(AMOVF, nodfconst(2147483648.), &nod);
        gins(AADDF, &nod, t);
    } else {
        gins(AMOVD, nodfconst(2147483648.), &nod);
        gins(AADDD, &nod, t);
    }
    regfree(&nod);
    patch(p1, pc);
    return;
}

```

Uses TDOUBLE 31a, TFLOAT 31a, Z 29a, and raddr() 453b.

<gmove() switch from type cases, TINT/TLONG case, if float target 454b>≡ (216d)

```

case TDOUBLE:
    gins(AMOVWD, f, t);
    return;

```

```

case TFLOAT:
    gins(AMOVWF, f, t);
    return;

```

Uses TDOUBLE 31a and TFLOAT 31a.

$\langle \text{gmove() switch from type cases, TSHORT case, if float target 455a} \rangle \equiv$ (217c)

```

case TDOUBLE:
    regalloc(&nod, f, Z);
    gins(AMOVH, f, &nod);
    gins(AMOVWD, &nod, t);
    regfree(&nod);
    return;
case TFLOAT:
    regalloc(&nod, f, Z);
    gins(AMOVH, f, &nod);
    gins(AMOVWF, &nod, t);
    regfree(&nod);
    return;

```

Uses TDOUBLE 31a, TFLOAT 31a, and Z 29a.

$\langle \text{gmove() switch from type cases, TUSHORT case, if float target 455b} \rangle \equiv$ (218a)

```

case TDOUBLE:
    regalloc(&nod, f, Z);
    gins(AMOVHU, f, &nod);
    gins(AMOVWD, &nod, t);
    regfree(&nod);
    return;
case TFLOAT:
    regalloc(&nod, f, Z);
    gins(AMOVHU, f, &nod);
    gins(AMOVWF, &nod, t);
    regfree(&nod);
    return;

```

Uses TDOUBLE 31a, TFLOAT 31a, and Z 29a.

$\langle \text{gmove() switch from type cases, TCHAR case, if float target 455c} \rangle \equiv$ (218b)

```

case TDOUBLE:
    regalloc(&nod, f, Z);
    gins(AMOVb, f, &nod);
    gins(AMOVWD, &nod, t);
    regfree(&nod);
    return;
case TFLOAT:
    regalloc(&nod, f, Z);
    gins(AMOVb, f, &nod);
    gins(AMOVWF, &nod, t);
    regfree(&nod);
    return;

```

Uses TDOUBLE 31a, TFLOAT 31a, and Z 29a.

$\langle \text{gmove() switch from type cases, TUCCHAR case, if float target 455d} \rangle \equiv$ (218c)

```

case TDOUBLE:
    regalloc(&nod, f, Z);
    gins(AMOVBU, f, &nod);
    gins(AMOVWD, &nod, t);
    regfree(&nod);
    return;
case TFLOAT:
    regalloc(&nod, f, Z);

```



```

gins(AMOVBU, f, &nod);
gins(AMOVWF, &nod, t);
regfree(&nod);
return;

```

Uses TDOUBLE 31a, TFLOAT 31a, and Z 29a.

```

⟨regalloc() switch tn type, float or vlong case 456⟩≡ (221f)
case TFLOAT:
case TDOUBLE:

case TVLONG:
    if(o != Z && o->op == OREGISTER) {
        i = o->reg;
        if(i >= NREG && i < NREG+NFREG)
            goto out;
    }
    j = 0*2 + NREG;
    for(i=NREG; i<NREG+NFREG; i++) {
        if(j >= NREG+NFREG)
            j = NREG;
        if(reg[j] == 0) {
            i = j;
            goto out;
        }
        j++;
    }
    diag(tn, "out of float registers");
    goto err;

```

Uses OREGISTER 208f, TDOUBLE 31a, TFLOAT 31a, TVLONG 30b, Z 29a, and diag() 496b.

17.4 64 bits operations

The ARM is a 32-bit processor, so 64-bit `vlong` operations require synthesis from pairs of 32-bit instructions. `com64` rewrites `vlong` expressions in the AST into sequences of 32-bit operations on the high and low halves, and the code generator (`bool64`, `cgen64`) emits the corresponding instruction pairs. This is similar to how a modern compiler's lowering pass handles types wider than the native word size.

5c's strategy is interesting because it does not inline the 32-bit pairs the way most compilers do. Instead it rewrites every `vlong` arithmetic operation into a call to a runtime helper in `libc`: `a + b` (where both are `vlong`) becomes `_addv(a, b)`. The helpers (`_addv`, `_subv`, `_mulv`, `_divv`, etc.) live in `lib_core/libc/arm/vlop.s` and are hand-written ARM assembly. For example, the C source

```
vlong add_vlong(vlong a, vlong b) { return a + b; }
```

compiles roughly to

```

TEXT    add_vlong+0(SB), $24
MOVW    a_lo+0(FP), R0      ; first 4 bytes of a
MOVW    a_hi+4(FP), R1      ; next 4 bytes of a
MOVW    b_lo+8(FP), R2
MOVW    b_hi+12(FP), R3
BL      , _addv(SB)
MOVW    R0, ret_lo+16(FP)   ; _addv leaves the result in R0/R1
MOVW    R1, ret_hi+20(FP)
RET

```

The trade-off versus inlining is clear. Inlining a 64-bit add as “`ADDS r0,r2; ADC r1,r3`” (using the carry flag) gives 2 instructions versus a function call’s prologue, BL, and epilogue. But 5c is a one-pass compiler with no global instruction selector and no carry-tracking in its register allocator, so wiring up the `ADDS/ADC` pair through the rest of the pipeline would have required plumbing condition codes through every node. The function-call approach keeps the compiler small (`com64.c` is 700 lines, vs. the thousands gcc spends on this) at the cost of slower `vlong` arithmetic. In practice `vlong` is rare in Plan 9 code—it is mostly used for file offsets and timestamps—so the slowdown rarely matters.

The mapping in `com64` is mechanical: each AST opcode gets translated to its corresponding runtime helper node (`OADD` → `nodaddv`, `OMUL` → `nodmulv`, etc.) and the original binary node is rewritten into an `OFUNC` call with the two operands as arguments. The compound assignments (`OASADD`, `OASSUB`, ...) share a single helper `_vasop` that takes a code byte identifying which operation; this avoids inflating `libc` with N versions of each operator.

17.4.1 `com64init()`

`com64init` creates `Node` references to the runtime helper functions for 64-bit arithmetic: `_addv`, `_subv`, `_mulv`, `_divv`, `_lshv`, etc. These functions are implemented in `libc` (in assembly, for performance) and called by the compiler whenever a `vlong` operation cannot be expressed with a single pair of ARM instructions. The `fvn` helper creates a function-call node for each named builtin.

```
<ginit() com64init initialisation 457a>≡ (51c)
    com64init();
```

Uses `com64init()` 457b.

```
<function com64init 457b>≡ (524b)
    void
    com64init(void)
    {
        Init *p;

        nodaddv = fvn("_addv", TVLONG);
        nodsubv = fvn("_subv", TVLONG);
        nodmulv = fvn("_mulv", TVLONG);
        noddivv = fvn("_divv", TVLONG);
        noddivvu = fvn("_divvu", TVLONG);
        nodmodv = fvn("_modv", TVLONG);
        nodmodvu = fvn("_modvu", TVLONG);
        nodlshv = fvn("_lshv", TVLONG);
        nodrshav = fvn("_rshav", TVLONG);
        nodrshlv = fvn("_rshlv", TVLONG);
        nodandv = fvn("_andv", TVLONG);
        nodorv = fvn("_orv", TVLONG);
        nodxurv = fvn("_xurv", TVLONG);
        nodnegv = fvn("_negv", TVLONG);
        nodcomv = fvn("_comv", TVLONG);

        nodtestv = fvn("_testv", TLONG);
        nodeqv = fvn("_eqv", TLONG);
        nodnev = fvn("_nev", TLONG);
        nodlev = fvn("_lev", TLONG);
        nodltv = fvn("_ltv", TLONG);
        nodgev = fvn("_gev", TLONG);
        nodgtv = fvn("_gtv", TLONG);
        nodhiv = fvn("_hiv", TLONG);
        nodhsv = fvn("_hsv", TLONG);
        nodlov = fvn("_lov", TLONG);
        nodlsv = fvn("_lsv", TLONG);
```

```

nodf2v = fvn("_f2v", TVLONG);
nodd2v = fvn("_d2v", TVLONG);
nodp2v = fvn("_p2v", TVLONG);
nodsi2v = fvn("_si2v", TVLONG);
nodui2v = fvn("_ui2v", TVLONG);
nodsl2v = fvn("_sl2v", TVLONG);
nodul2v = fvn("_ul2v", TVLONG);
nodsh2v = fvn("_sh2v", TVLONG);
noduh2v = fvn("_uh2v", TVLONG);
nodsc2v = fvn("_sc2v", TVLONG);
noduc2v = fvn("_uc2v", TVLONG);

nodv2f = fvn("_v2f", TFLOAT);
nodv2d = fvn("_v2d", TDOUBLE);
nodv2sl = fvn("_v2sl", TLONG);
nodv2ul = fvn("_v2ul", TULONG);
nodv2si = fvn("_v2si", TINT);
nodv2ui = fvn("_v2ui", TUINT);
nodv2sh = fvn("_v2sh", TSHORT);
nodv2uh = fvn("_v2ul", TUSHORT);
nodv2sc = fvn("_v2sc", TCHAR);
nodv2uc = fvn("_v2uc", TUCCHAR);

nodvpp = fvn("_vpp", TVLONG);
nodppv = fvn("_ppv", TVLONG);
nodvmm = fvn("_vmm", TVLONG);
nodmmv = fvn("_mmv", TVLONG);

nodvasop = fvn("_vasop", TVLONG);

for(p = initetconv; p->code >= 0; p++)
    etconv[p->code] = p->value;
}

```

Uses TCHAR 30b, TDOUBLE 31a, TFLOAT 31a, TINT 30b, TLONG 30b, TSHORT 30b, TUCCHAR 30b, TUINT 30b, TULONG 30b, TUSHORT 30b, TVLONG 30b, etconv 458a, fvn() 459b, initetconv 458b, nodaddv, nodandv, nodcomv, nodd2v, nodddivv, nodddivvu, nodeqv, nodf2v, nodgev, nodgtv, nodhiv, nodhsv, nodlev, nodlov, nodlshv, nodlsv, nodltv, nodmmv, nodmodv, nodmodvu, nodmulv, nodnegv, nodnev, nodorv, nodp2v, nodppv, nodrshv, nodrshlv, nodsc2v, nodsh2v, nodsi2v, nodsl2v, nodsubv, nodtestv, noduc2v, noduh2v, nodui2v, nodul2v, nodv2d, nodv2f, nodv2sc, nodv2sh, nodv2si, nodv2sl, nodv2uc, nodv2uh, nodv2ui, nodv2ul, nodvasop, nodvmm, nodvpp, and nodxorv.

<global etconv 458a>≡ (524b)
char etconv[NTYPE]; /* for _vasop */

Uses NTYPE 30a.

<global initetconv 458b>≡ (524b)
Init initetconv[] =
{
TCHAR, 1, 0,
TUCCHAR, 2, 0,
TSHORT, 3, 0,
TUSHORT, 4, 0,
TLONG, 5, 0,
TULONG, 6, 0,
TVLONG, 7, 0,
TUVLONG, 8, 0,
TINT, 9, 0,
TUINT, 10, 0,
-1, 0, 0,
};

Uses TCHAR 30b, TINT 30b, TLONG 30b, TSHORT 30b, Tuchar 30b, TUINT 30b, TULONG 30b, TUSHORT 30b, TUVLONG 30b, and TVLONG 30b.

<global fntypes 459a>≡ (511d)
 Type* fntypes[Ntype];

Uses Ntype 30a.

<function fvn 459b>≡ (524b)
 Node*
 fvn(char *name, int type)
 {
 Node *n;

 n = new(ONAME, Z, Z);
 n->sym = slookup(name);
 n->sym->sig = SIGINTERN;
 if(fntypes[type] == nil)
 fntypes[type] = typ(TFUNC, types[type]);
 n->type = fntypes[type];
 n->etype = type; // type of return?
 n->class = CGLOBL;
 n->addable = 10;
 n->complex = 0;
 return n;
 }

Uses CGLOBL 33b, ONAME 27, SIGINTERN 394a, TFUNC 31c, Z 29a, fntypes 459a, new() 29d, slookup() 35f, typ() 32a, and types 32b.

17.4.2 Rewriting: com64()

<xcom() if 64 bits operation, transform and return 459c>≡ (213a)
 if(com64(n))
 return;

Uses com64() 459d.

<function com64 459d>≡ (524b)
 bool
 com64(Node *n)
 {
 Node *l, *r;
 Node *a, *t;
 bool lv, rv;

 if(n->type == nil)
 return false;

 l = n->left;
 r = n->right;

 lv = false;
 if(l && l->type && typev[l->type->etype])
 lv = true;
 rv = false;
 if(r && r->type && typev[r->type->etype])
 rv = true;

 if(lv) {
 switch(n->op) {

```

case OEQ:
    a = nodeqv;
    goto setbool;
case ONE:
    a = nodnev;
    goto setbool;
case OLE:
    a = nodlev;
    goto setbool;
case OLT:
    a = nodltv;
    goto setbool;
case OGE:
    a = nodgev;
    goto setbool;
case OGT:
    a = nodgtv;
    goto setbool;
case OHI:
    a = nodhiv;
    goto setbool;
case OHS:
    a = nodhsv;
    goto setbool;
case OLO:
    a = nodlov;
    goto setbool;
case OLS:
    a = nodlsv;
    goto setbool;

case OANDAND:
case OOROR:
    if(machcap(n))
        return true;

    if(rv) {
        r = new(OFUNC, nodtestv, r);
        n->right = r;
        r->complex = FNX;
        r->op = OFUNC;
        r->type = types[TLONG];
    }

case OCOND:
case ONOT:
    if(machcap(n))
        return true;

    l = new(OFUNC, nodtestv, l);
    n->left = l;
    l->complex = FNX;
    l->op = OFUNC;
    l->type = types[TLONG];
    n->complex = FNX;
    return 1;
}

if(rv) {

```

```

    if(machcap(n))
        return true;
    switch(n->op) {
    case OANDAND:
    case OOROR:
        r = new(OFUNC, nodtestv, r);
        n->right = r;
        r->complex = FNX;
        r->op = OFUNC;
        r->type = types[TLONG];
        return 1;
    case OCOND:
        return 1;
    }
}

if(typev[n->type->etype]) {
    if(machcap(n))
        return true;
    switch(n->op) {
    default:
        diag(n, "unknown vlong %0", n->op);
    case OFUNC:
        n->complex = FNX;
    case ORETURN:
    case OAS:
    case OIND:
    case OLIST:
    case OCOMMA:
        return 1;
    case OADD:
        a = nodaddv;
        goto setbop;
    case OSUB:
        a = nodsubv;
        goto setbop;
    case OMUL:
    case OLMUL:
        a = nodmulv;
        goto setbop;
    case ODIV:
        a = nodddivv;
        goto setbop;
    case OLDIV:
        a = nodddivvu;
        goto setbop;
    case OMOD:
        a = nodmodv;
        goto setbop;
    case OLMOD:
        a = nodmodvu;
        goto setbop;
    case OASHL:
        a = nodlshv;
        goto setbop;
    case OASHR:
        a = nodrshav;
        goto setbop;
    case OLSHR:
        a = nodrshlv;

```

```

        goto setbop;
case OAND:
    a = nodandv;
    goto setbop;
case OOR:
    a = nodorv;
    goto setbop;
case OXOR:
    a = nodxorv;
    goto setbop;
case OPOSTINC:
    a = nodvpp;
    goto setvinc;
case OPOSTDEC:
    a = nodvmm;
    goto setvinc;
case OPREINC:
    a = nodppv;
    goto setvinc;
case OPREDEC:
    a = nodmmv;
    goto setvinc;
case ONEG:
    a = nodnegv;
    goto setfnx;
case OCOM:
    a = nodcomv;
    goto setfnx;
case OCAST:
    switch(l->type->etype) {
        case TCHAR:
            a = nodsc2v;
            goto setfnxl;
        case TUCCHAR:
            a = noduc2v;
            goto setfnxl;
        case TSHORT:
            a = nodsh2v;
            goto setfnxl;
        case TUSHORT:
            a = noduh2v;
            goto setfnxl;
        case TINT:
            a = nodsi2v;
            goto setfnx;
        case TUINT:
            a = nodui2v;
            goto setfnx;
        case TLONG:
            a = nodsl2v;
            goto setfnx;
        case TULONG:
            a = nodul2v;
            goto setfnx;
        case TFLOAT:
            a = nodf2v;
            goto setfnx;
        case TDOUBLE:
            a = nodd2v;
            goto setfnx;
    }

```

```

        case TIND:
            a = nodp2v;
            goto setfnx;
    }
    diag(n, "unknown %T->vlong cast", l->type);
    return true;
case OASADD:
    a = nodaddv;
    goto setasop;
case OASSUB:
    a = nodsubv;
    goto setasop;
case OASMUL:
case OASLMUL:
    a = nodmulv;
    goto setasop;
case OASDIV:
    a = nodddivv;
    goto setasop;
case OASLDIV:
    a = nodddivvu;
    goto setasop;
case OASMOD:
    a = nodmodv;
    goto setasop;
case OASLMOD:
    a = nodmodvu;
    goto setasop;
case OASASHL:
    a = nodlshv;
    goto setasop;
case OASASHR:
    a = nodrshav;
    goto setasop;
case OASLSHR:
    a = nodrshlv;
    goto setasop;
case OASAND:
    a = nodandv;
    goto setasop;
case OASOR:
    a = nodorv;
    goto setasop;
case OASXOR:
    a = nodxrv;
    goto setasop;
}
}

```

```

if(typefd[n->type->etype] && l && l->op == OFUNC) {
    switch(n->op) {
        case OASADD:
        case OASSUB:
        case OASMUL:
        case OASLMUL:
        case OASDIV:
        case OASLDIV:
        case OASMOD:
        case OASLMOD:
        case OASASHL:

```



```

    case OASASHR:
    case OASLSHR:
    case OASAND:
    case OASOR:
    case OASXOR:
        if(l->right && typev[l->right->etype]) {
            diag(n, "sorry float <asop> vlong not implemented\n");
        }
    }
}

if(n->op == OCAST) {
    if(l->type && typev[l->type->etype]) {
        if(machcap(n))
            return true;
        switch(n->type->etype) {
            case TDOUBLE:
                a = nodv2d;
                goto setfnx;
            case TFLOAT:
                a = nodv2f;
                goto setfnx;
            case TLONG:
                a = nodv2sl;
                goto setfnx;
            case TULONG:
                a = nodv2ul;
                goto setfnx;
            case TINT:
                a = nodv2si;
                goto setfnx;
            case TUINT:
                a = nodv2ui;
                goto setfnx;
            case TSHORT:
                a = nodv2sh;
                goto setfnx;
            case TUSHORT:
                a = nodv2uh;
                goto setfnx;
            case TCHAR:
                a = nodv2sc;
                goto setfnx;
            case TUCHAR:
                a = nodv2uc;
                goto setfnx;
            case TIND: // small pun here
                a = nodv2ul;
                goto setfnx;
        }
        diag(n, "unknown vlong->%T cast", n->type);
        return true;
    }
}

return false;

setbop:
    n->left = a;
    n->right = new(OLIST, l, r);

```

```

    n->complex = FNX;
    n->op = OFUNC;
    return true;

setfnxl:
    l = new(OCAST, l, 0);
    l->type = types[TLONG];
    l->complex = l->left->complex;

setfnx:
    n->left = a;
    n->right = l;
    n->complex = FNX;
    n->op = OFUNC;
    return true;

setvinc:
    n->left = a;
    l = new(OADDR, l, Z);
    l->type = typ(TIND, l->left->type);
    l->complex = l->left->complex;
    n->right = new(OLIST, l, r);
    n->complex = FNX;
    n->op = OFUNC;
    return true;

setbool:
    if(machcap(n))
        return true;
    n->left = a;
    n->right = new(OLIST, l, r);
    n->complex = FNX;
    n->op = OFUNC;
    n->type = types[TLONG];
    return true;

setasop:
    if(l->op == OFUNC) {
        l = l->right;
        goto setasop;
    }

    t = new(OCONST, 0, 0);
    t->vconst = etconv[l->type->etype];
    t->type = types[TLONG];
    t->addable = 20;
    r = new(OLIST, t, r);

    t = new(OADDR, a, 0);
    t->type = typ(TIND, a->type);
    r = new(OLIST, t, r);

    t = new(OADDR, l, 0);
    t->type = typ(TIND, l->type);
    t->complex = l->complex;
    r = new(OLIST, t, r);

    n->left = nodvasop;
    n->right = r;
    n->complex = FNX;

```

```

n->op = OFUNC;

return true;
}

```

Uses FNX-177 213d, OADD 114c, OADDR 116e, OAND 115a, OANDAND 115c, OAS 116a, OASADD 116c, OASAND 116c, OASASHL 116c, OASASHR 116c, OASDIV 116c, OASHL 115a, OASHR 115a, OASLDIV 233b, OASLMD 233b, OASLMUL 233b, OASLSHR 233b, OASMOD 116c, OASMUL 116c, OASOR 116c, OASSUB 116c, OASXOR 116c, OCAST 118a, OCOM 115e, OCOMMA 110g, OCOND 118f, OCONST 111a, ODIV 114c, OEQ 115g, OFUNC 117c, OGE 115g, OGT 115g, OHI 300d, OHS 300d, OIND 116e, OLDIV 232d, OLE 115g, OLIST 29b, OLMOD 232f, OLMUL 232d, OLO 300d, OLS 300d, OLSHR 232b, OLT 115g, OMOD 114c, OMUL 114c, ONE 115g, ONEG 114e, ONOT 115e, OOR 115a, OOROR 115c, OPOSTDEC 118j, OPOSTINC 118j, OPREDEC 118h, OPREINC 118h, ORETURN 108b, OSUB 114c, OXOR 115a, TCHAR 30b, TDOUBLE 31a, TFLOAT 31a, TIND 31c, TINT 30b, TLONG 30b, TSHORT 30b, Tuchar 30b, Tuint 30b, Tulong 30b, Tushort 30b, Z 29a, diag() 496b, etconv 458a, new() 29d, nodaddv, nodandv, nodcomv, nodd2v, nodddivv, nodddivvu, nodeqv, nodf2v, nodgev, nodgtv, nodhiv, nodhsv, nodlev, nodlov, nodlshv, nodlsv, nodltv, nodmmv, nodmodv, nodmodvu, nodmulv, nodnegv, nodnev, nodorv, nodp2v, nodppv, nodrshv, nodrshlv, nodsc2v, nodsh2v, nodsi2v, nodsl2v, nodsubv, nodtestv, noduc2v, noduh2v, nodui2v, nodul2v, nodv2d, nodv2f, nodv2sc, nodv2sh, nodv2si, nodv2sl, nodv2uc, nodv2uh, nodv2ui, nodv2ul, nodvasop, nodvmm, nodvpp, nodxorv, typ() 32a, typefd 149c, types 32b, and typev 280a.

17.4.3 bool64()

<bcomplex() possibly convert node to funcall if unsupported 64 bit op 466a>≡ (199e)
 bool64(n);

Uses bool64() 466b.

<function bool64 466b>≡ (524b)

```

void
bool64(Node *n)
{
    Node *n1;

    if(machcap(Z))
        return;

    if(typev[n->type->etype]) {
        n1 = new(OXXX, 0, 0);
        *n1 = *n;

        n->right = n1;
        n->left = nodtestv;
        n->complex = FNX;
        n->addable = 0;
        n->op = OFUNC;
        n->type = types[TLONG];
    }
}

```

Uses FNX-177 213d, OFUNC 117c, OXXX 27, TLONG 30b, Z 29a, new() 29d, nodtestv, types 32b, and typev 280a.

17.5 Endianness

The -l flag is MIPS-only: it switches the compiler from big-endian vc to little-endian 0c (SPIM). ARM is always little-endian in this compiler.

17.6 Using an external preprocessor: /bin/cpp

When -p is given, 5c forks /bin/cpp as a child process connected by a pipe, rather than using its built-in preprocessor. This is needed for APE (the ANSI/POSIX environment) compatibility, where the external cpp

handles features that 5c's embedded preprocessor does not support (like `#if` constant expressions).

```
<constant CPP 467a>≡ (521b)
#define CPP "/bin/cpp"

<compile() locals 467b>+≡ (47h) <49a
char **av, opt[256];
int i, c, fd[2];

<compile() if use ANSI preprocessor 467c>≡ (47h)
/* Use an ANSI preprocessor */
if(debug['p']) {
    if(myaccess(infile) < 0) {
        diag(Z, "%s does not exist", infile);
        errexit();
    }
    if(pipe(fd) < 0) {
        diag(Z, "pipe failed");
        errexit();
    }
    switch(fork()) {
    case -1:
        diag(Z, "fork failed");
        errexit();

    case 0:
        close(fd[0]);
        dup(fd[1], 1);
        close(fd[1]);
        av = alloc((3 + ndef + ninclude + 2) * sizeof *av);
        av[0] = CPP;
        i = 1;
        if(debug['.'])
            av[i++] = strdup("-.");
        /* 1999 ANSI C requires recognising // comments */
        av[i++] = strdup("--");
        for(c = 0; c < ndef; c++) {
            sprintf(opt, "-D%s", defs[c]);
            av[i++] = strdup(opt);
        }
        for(c = 0; c < ninclude; c++) {
            sprintf(opt, "-I%s", include[c]);
            av[i++] = strdup(opt);
        }
        if(strcmp(infile, "stdin") != 0)
            av[i++] = infile;
        av[i] = 0;
        if(debug['p'] > 1) {
            for(c = 0; c < i; c++)
                fprintf(2, "%s ", av[c]);
            fprintf(2, "\n");
        }

        exec(av[0], av);
        fprintf(2, "can't exec C preprocessor %s: %r\n", CPP);
        errexit();

    default:
        close(fd[1]);
        newfile(infile, fd[0]);
        break;
    }
}
```

```

    }
}
Uses Z 29a, alloc() 500b, debug 474a, diag() 496b, errexit() 495b, include 79b, myaccess() 513b, newfile() 53g,
and ninclude 79c.

```

17.7 Processing multiple files

5c can compile multiple files in a single invocation, but this feature is rarely used since `mk` handles parallelism. Each file is compiled sequentially by calling `compile` in a loop.

```

<main() locals 468a>+≡ (45d) <47c
    int nproc, nout, status, i;

<main() multiple files handling 468b>≡ (45d)
    if(argc > 1) {
        nproc = 1;
        /*
         * if we're writing acid to standard output, don't compile
         * concurrently, to avoid interleaving output.
         */
        if(((!debug['a'] && !debug['Z']) || debug['n']) &&
            (p = getenv("NPROC")) != nil)
            nproc = atol(p); /* */
        c = 0;
        nout = 0;
        for(;;) {
            while(nout < nproc && argc > 0) {
                i = fork();
                if(i < 0) {
                    i = mywait(&status);
                    if(i < 0) {
                        print("cannot create a process\n");
                        errexit();
                    }
                    if(status)
                        c++;
                    nout--;
                    continue;
                }
                if(i == 0) {
                    fprintf(2, "%s:\n", *argv);
                    if (compile(*argv, defs, ndef))
                        errexit();
                    exits(0);
                }
                nout++;
                argc--;
                argv++;
            }
            i = mywait(&status);
            if(i < 0) {
                if(c)
                    errexit();
                exits(0);
            }
            if(status)
                c++;
            nout--;

```

```

    }
}

```

17.8 Other optimizations

17.8.1 Packing: #pragma pack

#pragma pack overrides the default struct field alignment, telling the compiler to pack fields tightly without padding. This is useful for data structures that must match a hardware register layout or an on-disk/on-wire format. The packflg global is checked during struct layout to suppress alignment padding when non-zero.

```

<macprag() if pragma xxx 469a>+≡ (96a) <448a 511b>
    if(s && strcmp(s->name, "pack") == 0) {
        pragpack();
        return;
    }

```

Uses pragpack() 469c.

```

<global packflg 469b>≡ (511d)
    int packflg;

```

```

<function pragpack 469c>≡ (528b)
    void
    pragpack(void)
    {
        Sym *s;

        packflg = 0;
        s = getsym();
        if(s) {
            packflg = atoi(s->name+1);
            if(strcmp(s->name, "on") == 0 ||
               strcmp(s->name, "yes") == 0)
                packflg = 1;
        }
        while(getnsc() != '\n')
            ;
        <pragpack() debug if -f 469d>
    }

```

Uses getnsc() 77c, getsym() 77b, and packflg 469b.

```

<pragpack() debug if -f 469d>≡ (469c)
    if(debug['f'])
        if(packflg)
            print("%4ld: pack %d\n", lineno, packflg);
        else
            print("%4ld: pack off\n", lineno);

```

Uses debug 474a, lineno 29f, and packflg 469b.

```

<align() if packflg override w 469e>≡ (275)
    if(packflg)
        w = packflg;

```

Uses packflg 469b.

17.8.2 Minimizing #line history

The file/line history embedded in the object file can grow large. This optimization deduplicates consecutive `#line` directives from the same file by overwriting the previous entry when no AST nodes have been allocated since (tracked by `newflag`, set in `new`).

$\langle \text{linehist}() \text{ possibly overwrite last line directive 470a} \rangle \equiv$ (81d)

```
/*
 * overwrite the last #line directive if
 * no alloc has happened since the last one
 */
if(!newflag && ehist != H && offset != 0 && ehist->offset != 0)
    if(f && ehist->name && strcmp(f, ehist->name) == 0) {
        ehist->line = lineno;
        ehist->offset = offset;
        return;
    }
newflag = false;
```

Uses H 80c, ehist 81a, lineno 29f, and newflag 470b.

$\langle \text{global newflag 470b} \rangle \equiv$ (511d)

```
bool newflag;
```

$\langle \text{new() set lineno 470c} \rangle + \equiv$ (29d) $\triangleleft$ 29e

```
newflag = true;
```

Uses newflag 470b.

Chapter 18

Conclusion

You now know how the Plan 9 C compiler `5c` works, to the smallest details, and more generally how many compilers work.

`5c` is a complete C compiler in a single program—lexer, preprocessor, parser, type checker, code generator, optimizer, and object file emitter—all in roughly 15 000 lines of C. This compactness comes from a direct, stream-oriented architecture: each function is parsed, checked, compiled to assembly, optimized, and emitted before moving to the next, with no intermediate representation beyond the AST and the assembly instruction list. There is no control-flow graph, no SSA form, no separate assembler pass—just a series of tree transformations guided by type and addressability attributes on each node. Along the way, you have seen the full compilation pipeline: a hand-written lexer with an integrated preprocessor, a Yacc-generated parser building ASTs, type checking through declaration scoping, recursive code generation (`cgen` for expressions, `gen` for statements), register allocation and peephole optimization at the assembly level, and finally object file generation in the same AST-based format that the linker `5l` reads.

18.1 Patterns and techniques

These techniques apply far beyond compilers:

- *Streaming processing*: processing one function at a time and discarding the AST before moving on. The same bounded-memory strategy is used by SAX XML parsers and ETL pipelines: trade global optimization for the ability to handle inputs of arbitrary size.
- *Recursive tree walking*: `cgen()` evaluates children, then combines results. This bottom-up traversal is how expression evaluators, pretty-printers, and React’s virtual DOM diffing all work—any computation over a tree naturally takes this form.
- *Peephole optimization*: scanning for local patterns (redundant moves, strength reductions) and replacing them in place. This achieves surprisingly good results because most inefficiencies are local—the same technique is used in database query optimizers and regular expression engines.
- *Graph coloring for resource allocation*: allocating k physical registers by coloring an interference graph. The same algorithm schedules exam rooms (no two exams sharing students at the same time) and assigns radio frequencies (no two adjacent towers on the same channel).

18.2 Connections to other books

- ASSEMBLER book [Pad15a]: `5c` produces object files in the same format as `5a`, sharing data structures (`Prog`, `Adr`). The compiler effectively contains its own assembler—it goes directly from C to object code without emitting assembly text.

- **LINKER** book [Pad15c]: `5l` reads the object files that `5c` produces and performs the actual machine code generation. The linker also handles profiling instrumentation (`-p`) and debugging metadata that `5c` embeds.
- **KERNEL** book [Pad14]: the kernel is compiled by `5c`, and several compiler features exist specifically for the kernel: `extern register` for fast access to the `up` pointer, and the `#pragma profile 0` directive to disable profiling in interrupt handlers.
- **SHELL** book [Pad18]: `rc`'s compilation pipeline (lexer, parser, bytecode compiler) follows the same structure as `5c` but is much simpler—having seen `rc` first makes `5c`'s architecture easier to follow.
- **DEBUGGER** book [Pad16c]: `5c` generates the symbol table and line number information that the debugger reads, and can emit `acid` type descriptions for source-level debugging.
- **PROFILER** book [Pad26]: `5c` cooperates with the linker to support profiling: it emits function boundaries and the linker inserts the `_profin/_profout` hooks.
- **GENERATORS** book [?]: the Yacc-generated parser in `5c` uses the same shift-reduce algorithm described in the **GENERATORS** book [?].

18.3 Beyond the Plan 9 compiler

`5c` is a fast, simple compiler that generates correct code. Modern compilers go much further:

- *Intermediate representations*: GCC uses GIMPLE and RTL; LLVM uses LLVM IR. These IRs enable optimization passes that are independent of both the source language and the target architecture. `5c` has no IR—it goes from AST directly to target-specific assembly, which keeps the compiler simple but limits the optimizations it can perform.
- *SSA form and advanced optimization*: modern compilers convert their IR to Static Single Assignment form, enabling powerful optimizations: global value numbering, loop-invariant code motion, strength reduction, auto-vectorization, and interprocedural analysis. `5c`'s optimizations are limited to AST-level transformations (constant folding, expression rewriting) and assembly-level peephole optimization and register allocation.
- *Link-time optimization (LTO)*: LLVM and GCC can defer optimization to link time, when the entire program is visible, enabling cross-module inlining and dead code elimination. `5c`'s one-function-at-a-time model precludes this.
- *Multiple frontends and backends*: LLVM supports C, C++, Rust, Swift, and many other languages through separate frontends, all sharing the same optimizer and code generators for x86, ARM, RISC-V, and other targets. `5c` handles only C and only ARM (with sibling compilers `8c`, `6c` for other architectures, each a separate codebase).
- *Sanitizers and static analysis*: modern compilers can instrument code to detect memory errors (AddressSanitizer), undefined behavior (UBSan), data races (ThreadSanitizer), and can perform static analysis to catch bugs at compile time. `5c` does basic type checking and `#pragma` format-string checking, but nothing comparable.
- *C++ and beyond*: `5c` compiles a subset of C (roughly C89 with Plan 9 extensions like anonymous structs). Modern compilers handle C++ (with templates, exceptions, RTTI, move semantics), and languages like Rust that add ownership types, pattern matching, and traits—each requiring substantially more complex parsing, type checking, and code generation.

The gap between `5c` and a modern compiler like LLVM/Clang is enormous—LLVM alone is millions of lines of code. But the basic structure is the same: lex, parse, check types, generate code, optimize, emit. `5c` shows how far you can get with a direct, no-frills implementation of this pipeline, producing a compiler that is fast enough to build an entire operating system and simple enough to read in its entirety.

Appendix A

Debugging

The compiler has extensive self-debugging support via single-letter flags stored in the `debug` array. Each flag enables output for a particular phase: `-L` prints symbol lookups, `-d` prints declarations, `-t` prints type trees, `-g/-G` trace code generation, `-S` prints assembly output, `-r` prints register allocation, and `-m` prints arithmetic trees and macro expansion. The `prtree` function dumps any AST subtree in a readable format.

```
<global debug 474a>≡ (511d)
char debug[256];
```

A.1 Dumpers

The compiler registers custom format verbs (`%O` for node kinds, `%T` for types, `%N` for symbol kinds, `%B` for bitsets, `%A` for opcodes, etc.) so that `diag` and `print` can display compiler data structures in a readable form. The portable verbs are in `cc/` and the ARM-specific ones in `5c/`.

A.1.1 cc/ Fmt data structures

```
<cinit() fmtinstall 474b>≡ (50d) 497c▷
fmtinstall('O', Oconv);
fmtinstall('T', Tconv);
fmtinstall('F', FNconv);
fmtinstall('Q', Qconv);
fmtinstall('|', VBconv);
```

Uses `FNconv()` 478a, `Oconv()` 474c, `Qconv()` 478b, `Tconv()` 476, and `VBconv()` 479a.

```
<function Oconv 474c>≡ (521b)
// enum<Node_kind> -> unit
int Oconv(Fmt *fp)
{
    int a;

    a = va_arg(fp->args, int);
    if(a < OXXX || a > OEND)
        return fmtprint(fp, "***bad0 %d***", a);

    return fmtstrcpy(fp, onames[a]);
}
```

Uses `OEND` 27, `OXXX` 27, and `onames` 474d.

```
<global onames 474d>≡ (518c)
char* onames[OEND+1];
```

Uses `OEND` 27.

<global onamesinit 475>≡

(518c)

```
Init onamesinit[] =
{
    OXXX, 0, "OXXX",
    OADD, 0, "ADD",
    OADDR, 0, "ADDR",
    OAND, 0, "AND",
    OANDAND, 0, "ANDAND",
    OARRAY, 0, "ARRAY",
    OAS, 0, "AS",
    OASI, 0, "ASI",
    OASADD, 0, "ASADD",
    OASAND, 0, "ASAND",
    OASASHL, 0, "ASASHL",
    OASASHR, 0, "ASASHR",
    OASDIV, 0, "ASDIV",
    OASHL, 0, "ASHL",
    OASHR, 0, "ASHR",
    OASLDIV, 0, "ASLDIV",
    OASLMOD, 0, "ASLMOD",
    OASLMUL, 0, "ASLMUL",
    OASLSHR, 0, "ASLSHR",
    OASMOD, 0, "ASMOD",
    OASMUL, 0, "ASMUL",
    OASOR, 0, "ASOR",
    OASSUB, 0, "ASSUB",
    OASXOR, 0, "ASXOR",
    OBIT, 0, "BIT",
    OBREAK, 0, "BREAK",
    OCASE, 0, "CASE",
    OCAST, 0, "CAST",
    OCOMMA, 0, "COMMA",
    OCOND, 0, "COND",
    OCONST, 0, "CONST",
    OCONTINUE, 0, "CONTINUE",
    ODIV, 0, "DIV",
    ODOT, 0, "DOT",
    ODOTDOT, 0, "DOTDOT",
    ODWHILE, 0, "DWHILE",
    OEQ, 0, "EQ",
    OFOR, 0, "FOR",
    OFUNC, 0, "FUNC",
    OGE, 0, "GE",
    OGOTO, 0, "GOTO",
    OGT, 0, "GT",
    OHI, 0, "HI",
    OHS, 0, "HS",
    OIF, 0, "IF",
    OIND, 0, "IND",
    OINDREG, 0, "INDREG",
    OINIT, 0, "INIT",
    OLABEL, 0, "LABEL",
    OLDIV, 0, "LDIV",
    OLE, 0, "LE",
    OLIST, 0, "LIST",
    OLMOD, 0, "LMOD",
    OLMUL, 0, "LMUL",
    OLO, 0, "LO",
    OLS, 0, "LS",
    OLSHR, 0, "LSHR",
```

```

OLT, 0, "LT",
OMOD, 0, "MOD",
OMUL, 0, "MUL",
ONAME, 0, "NAME",
ONE, 0, "NE",
ONOT, 0, "NOT",
OOR, 0, "OR",
OOROR, 0, "OROR",
OPOSTDEC, 0, "POSTDEC",
OPOSTINC, 0, "POSTINC",
OPREDEC, 0, "PREDEC",
OPREINC, 0, "PREINC",
OPROTO, 0, "PROTO",
OREGISTER, 0, "REGISTER",
ORETURN, 0, "RETURN",
OSET, 0, "SET",
OSIGN, 0, "SIGN",
OSIZE, 0, "SIZE",
OSTRING, 0, "STRING",
OLSTRING, 0, "LSTRING",
OSTRUCT, 0, "STRUCT",
OSUB, 0, "SUB",
OSWITCH, 0, "SWITCH",
OUNION, 0, "UNION",
OUSED, 0, "USED",
OWHILE, 0, "WHILE",
OXOR, 0, "XOR",
OPOS, 0, "POS",
ONEG, 0, "NEG",
OCOM, 0, "COM",
OELEM, 0, "ELEM",
OINDEX, 0, "INDEX",
OREGPAIR, 0, "REGPAIR",
OEXREG, 0, "EXREG",
OEND, 0, "END",
-1, 0, 0,
};

```

Uses OADD 114c, OADDR 116e, OAND 115a, OANDAND 115c, OARRAY 119i, OAS 116a, OASADD 116c, OASAND 116c, OASASHL 116c, OASASHR 116c, OASDIV 116c, OASHL 115a, OASHR 115a, OASI 262b, OASLDIV 233b, OASLMD 233b, OASLMUL 233b, OASLSHR 233b, OASMOD 116c, OASMUL 116c, OASOR 116c, OASSUB 116c, OASXOR 116c, OBIT 418d, OBREAK 108b, OCASE 109b, OCAST 118a, OCOM 115e, OCOMMA 110g, OCOND 118f, OCONST 111a, OCONTINUE 108b, ODIV 114c, ODOT 117a, ODOTDOT 137c, ODWHILE 107c, OELEM 119h, OEND 27, OEQ 115g, OEXREG, OFOR 107c, OFUNC 117c, OGE 115g, OGOTO 108e, OGT 115g, OHI 300d, OHS 300d, OIF 107a, OIND 116e, OINDEX, OINDREG 237c, OINIT 119e, OLABEL 108e, OLDIV 232d, OLE 115g, OLIST 29b, OLMOD 232f, OLMUL 232d, OLO 300d, OLS 300d, OLSHR 232b, OLSTRING 113a, OLT 115g, OMOD 114c, OMUL 114c, ONAME 27, ONE 115g, ONEG 114e, ONOT 115e, OOR 115a, OOROR 115c, OPOS 114e, OPOSTDEC 118j, OPOSTINC 118j, OPREDEC 118h, OPREINC 118h, OPROTO 136f, OREGISTER 208f, OREGPAIR, ORETURN 108b, OSET 184c, OSIGN 442a, OSIZE 119a, OSTRING 112c, OSTRUCT 429, OSUB 114c, OSWITCH 109b, OUNION 429, OUSED 184c, OWHILE 107c, OXOR 115a, and OXXX 27.

```

⟨function Tconv 476⟩≡ (521b)
// option<Type> -> string
int
Tconv(Fmt *fp)
{
    char str[STRINGSZ+20], s[STRINGSZ+20];
    Type *t, *t1;
    int et;
    long n;

    str[0] = '\0';
    for(t = va_arg(fp->args, Type*); t != T; t = t->link) {

```

```

et = t->etype;
if(str[0])
    strcat(str, " ");
if(t->garb&~GINCOMPLETE) {
    sprintf(s, "%s ", gnames[t->garb&~GINCOMPLETE]);
    if(strlen(str) + strlen(s) < STRINGSZ)
        strcat(str, s);
}
sprintf(s, "%s", tnames[et]);
if(strlen(str) + strlen(s) < STRINGSZ)
    strcat(str, s);
if(et == TFUNC && (t1 = t->down)) {
    sprintf(s, "(%T", t1);
    if(strlen(str) + strlen(s) < STRINGSZ)
        strcat(str, s);
    while(t1 = t1->down) {
        sprintf(s, ", %T", t1);
        if(strlen(str) + strlen(s) < STRINGSZ)
            strcat(str, s);
    }
    if(strlen(str) + strlen(s) < STRINGSZ)
        strcat(str, ")");
}
if(et == TARRAY) {
    n = t->width;
    if(t->link && t->link->width)
        n /= t->link->width;
    sprintf(s, "[%ld]", n);
    if(strlen(str) + strlen(s) < STRINGSZ)
        strcat(str, s);
}
if(t->nbits) {
    sprintf(s, " %d:%d", t->shift, t->nbits);
    if(strlen(str) + strlen(s) < STRINGSZ)
        strcat(str, s);
}
if(typesu[et]) {
    if(t->tag) {
        strcat(str, " ");
        if(strlen(str) + strlen(t->tag->name) < STRINGSZ)
            strcat(str, t->tag->name);
    } else
        strcat(str, " {}");
    break;
}
}
return fmtstrcpy(fp, str);
}

```

Uses GINCOMPLETE 399, STRINGSZ 81e, T 31i, TARRAY 31c, TFUNC 31c, gnames 486a, tnames 477a, and typesu 149a.

<global tnames 477a> ≡ (518c)
char* tnames[NALLTYPES];

Uses NALLTYPES 120e.

<global tnamesinit 477b> ≡ (518c)
Init tnamesinit[] =
{
TXXX, 0, "TXXX",
TCHAR, 0, "CHAR",

```

TCHAR, 0, "UCHAR",
TSHORT, 0, "SHORT",
TUSHORT, 0, "USHORT",
TINT, 0, "INT",
TUINT, 0, "UINT",
TLONG, 0, "LONG",
TULONG, 0, "ULONG",
TVLONG, 0, "VLONG",
TUVLONG, 0, "UVLONG",
TFLOAT, 0, "FLOAT",
TDOUBLE, 0, "DOUBLE",
TIND, 0, "IND",
TFUNC, 0, "FUNC",
TARRAY, 0, "ARRAY",
TVOID, 0, "VOID",
TSTRUCT, 0, "STRUCT",
TUNION, 0, "UNION",
TENUM, 0, "ENUM",
TOLD, 0, "OLD",
TDOT, 0, "DOT",

```

```

-1, 0, 0,

```

```

};

```

Uses TARRAY 31c, TCHAR 30b, TDOT 137f, TDOUBLE 31a, TENUM 31c, TFLOAT 31a, TFUNC 31c, TIND 31c, TINT 30b, TLONG 30b, TOLD 424d, TSHORT 30b, TSTRUCT 31c, TCHAR 30b, TUINT 30b, TULONG 30b, TUNION 31c, TUSHORT 30b, TUVLONG 30b, TVLONG 30b, TVOID 31b, and TXXX 30a.

```

⟨function FNconv 478a⟩≡ (521b)
// option<Node identifier cases> -> string
int
FNconv(Fmt *fp)
{
    char *str;
    Node *n;

    n = va_arg(fp->args, Node*);
    str = "<indirect>";
    if(n != Z && (n->op == ONAME || n->op == ODOT || n->op == OELEM))
        str = n->sym->name;
    return fmtstrcpy(fp, str);
}

```

Uses ODOT 117a, OELEM 119h, ONAME 27, and Z 29a.

```

⟨function Qconv 478b⟩≡ (521b)
// ??
int
Qconv(Fmt *fp)
{
    char str[STRINGSZ+20], *s;
    long b;
    int i;

    str[0] = 0;
    for(b = va_arg(fp->args, long); b;) {
        i = bitno(b);
        if(str[0])
            strcat(str, " ");
        s = qnames[i];
        if(strlen(str) + strlen(s) >= STRINGSZ)
            break;
    }
}

```

```

        strcat(str, s);
        b &= ~(1L << i);
    }
    return fmtstrcpy(fp, str);
}

```

Uses STRINGSZ 81e, bitno() 501d, and qnames 485c.

```

⟨function VBconv 479a⟩≡ (521b)
// ??
int
VBconv(Fmt *fp)
{
    char str[STRINGSZ];
    int i, n, t, pc;

    n = va_arg(fp->args, int);
    pc = 0; /* BUG: was printcol */
    i = 0;
    while(pc < n) {
        t = (pc+4) & ~3;
        if(t <= n) {
            str[i++] = '\t';
            pc = t;
            continue;
        }
        str[i++] = ' ';
        pc++;
    }
    str[i] = '\0';

    return fmtstrcpy(fp, str);
}

```

Uses STRINGSZ 81e.

A.1.2 5c/ Fmt data structures

```

⟨ginit() misc initialisations 479b⟩+≡ (51c) <433a
listinit();

```

```

⟨function listinit(arm) 479c⟩≡ (537a)
void
listinit(void)
{

    fmtinstall('A', Aconv);
    fmtinstall('P', Pconv);
    fmtinstall('S', Sconv);
    fmtinstall('N', Nconv);
    fmtinstall('B', Bconv);
    fmtinstall('D', Dconv);
    fmtinstall('R', Rconv);
}

```

Uses Nconv() 484.

```

⟨function Bconv 479d⟩≡ (537a)
// Bits -> string
int
Bconv(Fmt *fp)

```



```

{
    char str[STRINGSZ], ss[STRINGSZ], *s;
    Bits bits;
    int i;

    str[0] = 0;
    bits = va_arg(fp->args, Bits);
    while(bany(&bits)) {
        i = bnum(bits);
        if(str[0])
            strcat(str, " ");
        if(var[i].sym == S) {
            snprintf(ss, sizeof(ss), "$%ld", var[i].offset);
            s = ss;
        } else
            s = var[i].sym->name;
        if(strlen(str) + strlen(s) + 1 >= STRINGSZ)
            break;
        strcat(str, s);
        bits.b[i/32] &= ~(1L << (i%32));
    }
    return fmtstrcpy(fp, str);
}

```

Uses S 35d, STRINGSZ 81e, bany() 350c, and bnum() 350e.

```

⟨function Pconv(arm) 480⟩≡ (537a)
// Prog -> string
int
Pconv(Fmt *fp)
{
    char str[STRINGSZ], sc[20];
    Prog *p;
    int a, s;

    p = va_arg(fp->args, Prog*);
    a = p->as;
    s = p->scond;
    strcpy(sc, extra[s & C_SCOND]);
    if(s & C_SBIT)
        strcat(sc, ".S");
    if(s & C_PBIT)
        strcat(sc, ".P");
    if(s & C_WBIT)
        strcat(sc, ".W");
    if(s & C_UBIT) /* ambiguous with FBIT */
        strcat(sc, ".U");
    if(a == AMOVM) {
        if(p->from.type == D_CONST)
            snprintf(str, sizeof(str), "%A%s %R,%D", a, sc, &p->from, &p->to);
        else
            if(p->to.type == D_CONST)
                snprintf(str, sizeof(str), "%A%s %D,%R", a, sc, &p->from, &p->to);
            else
                snprintf(str, sizeof(str), "%A%s %D,%D", a, sc, &p->from, &p->to);
    } else
        if(a == ADATA)
            snprintf(str, sizeof(str), "%A %D/%d,%D", a, &p->from, p->reg, &p->to);
        else
            if(p->as == ATEXT)
                snprintf(str, sizeof(str), "%A %D,%d,%D", a, &p->from, p->reg, &p->to);

```

```

else
if(p->reg == R_NONE)
    snprintf(str, sizeof(str), " %A%s %D,%D", a, sc, &p->from, &p->to);
else
if(p->from.type != D_FREG)
    snprintf(str, sizeof(str), " %A%s %D,R%d,%D", a, sc, &p->from, p->reg, &p->to);
else
    snprintf(str, sizeof(str), " %A%s %D,F%d,%D", a, sc, &p->from, p->reg, &p->to);
return fmtstrcpy(fp, str);
}

```

Uses STRINGSZ 81e and extra 481a.

```

⟨global extra(arm) 481a⟩≡ (537a)
char *extra [] = {
    ".EQ", ".NE", ".CS", ".CC",
    ".MI", ".PL", ".VS", ".VC",
    ".HI", ".LS", ".GE", ".LT",
    ".GT", ".LE", "", ".NV",
};

```

```

⟨function Aconv 481b⟩≡ (537a)
// enum<opcode_kind> -> string
int
Aconv(Fmt *fp)
{
    char *s;
    int a;

    a = va_arg(fp->args, int);
    s = "???";
    if(a >= AXXX && a < ALAST)
        s = anames[a];
    return fmtstrcpy(fp, s);
}

```

```

⟨function Dconv(arm) 481c⟩≡ (537a)
int
Dconv(Fmt *fp)
{
    char str[STRINGSZ];
    Adr *a;
    char *op;
    int v;

    a = va_arg(fp->args, Adr*);
    switch(a->type) {

default:
    snprintf(str, sizeof(str), "GOK-type(%d)", a->type);
    break;

case D_NONE:
    str[0] = 0;
    if(a->symkind != N_NONE || a->reg != R_NONE || a->sym != S)
        snprintf(str, sizeof(str), "%N(R%d)(NONE)", a, a->reg);
    break;

case D_CONST:
    if(a->reg != R_NONE)
        snprintf(str, sizeof(str), "$%N(R%d)", a, a->reg);

```

```

    else
        snprintf(str, sizeof(str), "$%N", a);
    break;

case D_SHIFT:
    v = a->offset;
    op = "<<>>->@>" + (((v>>5) & 3) << 1);
    if(v & (1<<4))
        snprintf(str, sizeof(str), "R%d%c%cR%d", v&15, op[0], op[1], (v>>8)&15);
    else
        snprintf(str, sizeof(str), "R%d%c%c%d", v&15, op[0], op[1], (v>>7)&31);
    if(a->reg != R_NONE)
        sprintf(str+strlen(str), "(R%d)", a->reg);
    break;

case D_OREG:
    if(a->reg != R_NONE)
        snprintf(str, sizeof(str), "%N(R%d)", a, a->reg);
    else
        snprintf(str, sizeof(str), "%N", a);
    break;

case D_REG:
    snprintf(str, sizeof(str), "R%d", a->reg);
    if(a->symkind != N_NONE || a->sym != S)
        snprintf(str, sizeof(str), "%N(R%d)(REG)", a, a->reg);
    break;

case D_FREG:
    snprintf(str, sizeof(str), "F%d", a->reg);
    if(a->symkind != N_NONE || a->sym != S)
        snprintf(str, sizeof(str), "%N(R%d)(REG)", a, a->reg);
    break;

case D_PSR:
    snprintf(str, sizeof(str), "PSR");
    if(a->symkind != N_NONE || a->sym != S)
        snprintf(str, sizeof(str), "%N(PSR)(REG)", a);
    break;

case D_BRANCH:
    snprintf(str, sizeof(str), "%ld(PC)", a->offset-pc);
    break;

case D_FCONST:
    snprintf(str, sizeof(str), "$%.17e", a->dval);
    break;

case D_SCONST:
    snprintf(str, sizeof(str), "$\"%S\"", a->sval);
    break;
}
return fmtstrcpy(fp, str);
}

```

Uses S 35d and STRINGSZ 81e.

```

⟨function Rconv(arm) 482⟩≡ (537a)
// enum<operand_kind(register-only??)> -> string
int
Rconv(Fmt *fp)

```

```

{
    char str[STRINGSZ], *p, *e;
    Adr *a;
    int i, v;

    a = va_arg(fp->args, Adr*);
    snprintf(str, sizeof(str), "GOK-reglist");
    switch(a->type) {
    case D_CONST:
        if(a->reg != R_NONE)
            break;
        if(a->sym != S)
            break;
        v = a->offset;
        p = str;
        e = str+sizeof(str);
        for(i=0; i<NREG; i++) {
            if(v & (1<<i)) {
                if(p == str)
                    p = seprint(p, e, "[R%d", i);
                else
                    p = seprint(p, e, ",R%d", i);
            }
        }
        seprint(p, e, "]");
    }
    return fmtstrcpy(fp, str);
}

```

Uses S 35d and STRINGSZ 81e.

```

⟨function Sconv(arm) 483⟩≡ (537a)
int
Sconv(Fmt *fp)
{
    int i, c;
    char str[STRINGSZ], *p, *a;

    a = va_arg(fp->args, char*);
    p = str;
    for(i=0; i<NSNAME; i++) {
        c = a[i] & 0xff;
        if(c >= 'a' && c <= 'z' ||
           c >= 'A' && c <= 'Z' ||
           c >= '0' && c <= '9' ||
           c == ' ' || c == '%') {
            *p++ = c;
            continue;
        }
        *p++ = '\\';
        switch(c) {
        case 0:
            *p++ = 'z';
            continue;
        case '\\':
        case '"':
            *p++ = c;
            continue;
        case '\\n':
            *p++ = 'n';
            continue;

```

```

        case '\t':
            *p++ = 't';
            continue;
        case '\r':
            *p++ = 'r';
            continue;
        case '\f':
            *p++ = 'f';
            continue;
    }
    *p++ = (c>>6) + '0';
    *p++ = ((c>>3) & 7) + '0';
    *p++ = (c & 7) + '0';
}
*p = 0;
return fmtstrcpy(fp, str);
}

```

Uses STRINGSZ 81e.

```

⟨function Nconv(arm) 484⟩≡ (537a)
int
Nconv(Fmt *fp)
{
    char str[STRINGSZ];
    Adr *a;
    Sym *s;

    a = va_arg(fp->args, Adr*);
    s = a->sym;
    if(s == S) {
        snprintf(str, sizeof(str), "%ld", a->offset);
        goto out;
    }
    switch(a->symkind) {
    case N_NONE:
        snprintf(str, sizeof(str), "%ld", a->offset);
        break;

    case N_EXTERN:
        snprintf(str, sizeof(str), "%s+%ld(SB)", s->name, a->offset);
        break;

    case N_INTERN:
        snprintf(str, sizeof(str), "%s<+>%ld(SB)", s->name, a->offset);
        break;

    case N_LOCAL:
        snprintf(str, sizeof(str), "%s-%ld(SP)", s->name, -a->offset);
        break;

    case N_PARAM:
        snprintf(str, sizeof(str), "%s+%ld(FP)", s->name, a->offset);
        break;

    default:
        snprintf(str, sizeof(str), "GOK-name(%d)", a->symkind);
        break;
    }
out:

```

```
    return fmtstrcpy(fp, str);
}
```

Uses S 35d and STRINGSZ 81e.

A.1.3 xxxnames[]

Name arrays provide string representations for debugging: **onames** for node kinds, **tnames** for type kinds, **cnames** for storage classes, **gnames** for qualifiers, **bnames** for alignment phases, and **qnames** for the extended type kinds. Each is filled at startup from an **Init** array via **tinit**.

```
<global cnames 485a>≡ (518c)
```

```
char* cnames[NCTYPES];
```

Uses NCTYPES 33b.

```
<global cnamesinit 485b>≡ (518c)
```

```
Init cnamesinit[] =
```

```
{
    CXXX      , 0, "CXXX",
    CAUTO     , 0, "AUTO",
    CEXTERN   , 0, "EXTERN",
    CGLOBL    , 0, "GLOBL",
    CSTATIC   , 0, "STATIC",
    CLOCAL    , 0, "LOCAL",
    CTYPEDEF  , 0, "TYPEDEF",
    CTYPESTR  , 0, "TYPESTR",
    CPARAM    , 0, "PARAM",
    CEXREG    , 0, "EXREG",
    -1,      0, 0,
};
```

Uses CAUTO 33b, CEXREG 432a, CEXTERN 33b, CGLOBL 33b, CLOCAL 415b, CPARAM 33b, CSTATIC 33b, CTYPEDEF 137i, CTYPESTR 434f, and CXXX 33b.

```
<global qnames 485c>≡ (518c)
```

```
char* qnames[NALLTYPES];
```

Uses NALLTYPES 120e.

```
<global qnamesinit 485d>≡ (518c)
```

```
Init qnamesinit[] =
```

```
{
    TXXX      , 0, "TXXX",

    TCHAR     , 0, "CHAR",
    Tuchar    , 0, "UCHAR",
    TSHORT    , 0, "SHORT",
    TUSHORT   , 0, "USHORT",
    TINT      , 0, "INT",
    TUINT     , 0, "UINT",
    TLONG     , 0, "LONG",
    TULONG    , 0, "ULONG",
    TVLONG    , 0, "VLONG",
    TUVLONG   , 0, "UVLONG",
    TFLOAT    , 0, "FLOAT",
    TDOUBLE   , 0, "DOUBLE",
    TIND      , 0, "IND",
    TFUNC     , 0, "FUNC",
    TARRAY    , 0, "ARRAY",
    TVOID     , 0, "VOID",
    TSTRUCT   , 0, "STRUCT",
};
```

```

TUNION      , 0, "UNION",
TENUM       , 0, "ENUM",

TAUTO       , 0, "AUTO",
TEXTERN     , 0, "EXTERN",
TSTATIC     , 0, "STATIC",
TTYPEDEF    , 0, "TYPEDEF",
TTYPESTR    , 0, "TYPESTR",
TREGISTER   , 0, "REGISTER",
TCONSTNT    , 0, "CONSTNT",
TVOLATILE   , 0, "VOLATILE",
TUNSIGNED   , 0, "UNSIGNED",
TSIGNED     , 0, "SIGNED",
TDOT        , 0, "DOT",
TOLD        , 0, "OLD",

```

```

-1, 0, 0,
};

```

Uses TARRAY 31c, TAUTO 126a, TCHAR 30b, TCONSTNT 127a, TDOT 137f, TDOUBLE 31a, TENUM 31c, TEXTERN 126a, TFLOAT 31a, TFUNC 31c, TIND 31c, TINT 30b, TLONG 30b, TOLD 424d, TREGISTER 126a, TSHORT 30b, TSIGNED 123g, TSTATIC 126a, TSTRUCT 31c, TTYPEDEF 126a, TTYPESTR 434e, Tuchar 30b, TUINT 30b, TULONG 30b, TUNION 31c, TUNSIGNED 123g, TUSHORT 30b, TUVLONG 30b, TVLONG 30b, TVOID 31b, TVOLATILE 127a, and TXXX 30a.

```

⟨global gnames 486a⟩≡ (518c)
char* gnames[NGTYPES];

```

Uses NGTYPES 33a.

```

⟨global gnamesinit 486b⟩≡ (518c)
Init gnamesinit[] =
{
    GXXX, 0, "GXXX",
    GCONSTNT, 0, "CONST",
    GVOLATILE, 0, "VOLATILE",
    GVOLATILE|GCONSTNT, 0, "CONST-VOLATILE",
    -1, 0, 0,
};

```

Uses GCONSTNT 33a, GVOLATILE 33a, and GXXX 33a.

```

⟨tinit() initialise xxxnames debugging arrays 486c⟩≡ (50a)
for(p=bnamesinit; p->code >= 0; p++) {
    urk("bnames", nelem(bnames), p->code);
    bnames[p->code] = p->s;
}
for(p=tnamesinit; p->code >= 0; p++) {
    urk("tnames", nelem(tnames), p->code);
    tnames[p->code] = p->s;
}
for(p=gnamesinit; p->code >= 0; p++) {
    urk("gnames", nelem(gnames), p->code);
    gnames[p->code] = p->s;
}
for(p=qnamesinit; p->code >= 0; p++) {
    urk("qnames", nelem(qnames), p->code);
    qnames[p->code] = p->s;
}
for(p=cnamesinit; p->code >= 0; p++) {
    urk("cnames", nelem(cnames), p->code);
    cnames[p->code] = p->s;
}
}

```

```

for(p=onamesinit; p->code >= 0; p++) {
    urk("onames", nelem(onames), p->code);
    onames[p->code] = p->s;
}

```

Uses `bnames` 276c, `bnamesinit` 276d, `cnames` 485a, `cnamesinit` 485b, `gnames` 486a, `gnamesinit` 486b, `onames` 474d, `onamesinit` 475, `qnames` 485c, `qnamesinit` 485d, `tnames` 477a, `tnamesinit` 477b, and `urk()` 50c.

A.1.4 AST dumper: `prtree()` and `5c -x`

`prtree` recursively prints an AST subtree in a readable format, showing node kinds, types, values, and tree structure via indentation. The `-x` flag dumps the AST for each function after parsing, which is invaluable for understanding what the parser produces and what the type checker transforms.

```

⟨xdecl rule, debug function body 487a⟩≡ (105c)
    if(debug['x']) {
        prtree($2, "func");
        prtree($6, "body");
    }

```

```

⟨function prtree 487b⟩≡ (518c)
    void
    prtree(Node *n, char *s)
    {

        print(" == %s ==\n", s);
        prtreen1(n, 0, false);
        print("\n");
    }

```

Uses `prtreen1()` 487c.

```

⟨function prtreen1 487c⟩≡ (518c)
    void
    prtreen1(Node *n, int d, bool f)
    {
        //bitset<Left|Right> to decide whether to recurse left and right child
        int i;

        if(f)
            for(i=0; i<d; i++)
                print(" ");
        if(n == Z) {
            print("Z\n");
            return;
        }

        if(n->op == OLIST) {
            prtreen1(n->left, d, false);
            prtreen1(n->right, d, true);
            return;
        }
        d++;
        print("%0", n->op);
        i = 3; // 1 | 2

        switch(n->op) {
        case OCONST:
            if(typeefd[n->type->etype])
                print(" \".8e\"", n->fconst);

```



```

        else
            print(" \"%lld\"", n->vconst);
        i = 0;
        break;

case OSTRING:
    print(" \"%s\"", n->cstring);
    i = 0;
    break;
case OLSTRING:
    if(sizeof(TRune) == sizeof(Rune))
        print(" \"%S\"", (Rune*)n->rstring);
    else
        print(" \"...\"");
    i = 0;
    break;

case ONAME:
    print(" \"%F\"", n);
    print(" %ld", n->xoffset);
    i = 0;
    break;
case ODOT:
case OELEM:
    print(" \"%F\"", n);
    break;

case OREGISTER:
    if(n->xoffset)
        print(" %ld+R%d", n->xoffset, n->reg);
    else
        print(" R%d", n->reg);
    i = 0;
    break;
case OINDREG:
    print(" %ld(R%d)", n->xoffset, n->reg);
    i = 0;
    break;

}

if(n->type != T)
    print(" %T", n->type);

if(n->addable != 0)
    print(" <%d>", n->addable);
if(n->complex != 0)
    print(" (%d)", n->complex);

print(" %L\n", n->lineno);

if(i & 2)
    prttree1(n->left, d, true);
if(i & 1)
    prttree1(n->right, d, true);
}

```

Uses OCONST 111a, ODOT 117a, OELEM 119h, OINDREG 237c, OLIST 29b, OLSTRING 113a, ONAME 27, OREGISTER 208f, OSTRING 112c, T 31i, Z 29a, prttree1() 487c, and typefd 149c.

A.2 5c -v, verbose mode

A.3 Preprocessing debugging

A.3.1 Macro debugging: 5c -m

`<dodefine() debug 489a>≡` (84c)

```
if(debug['m'])
    print("#define (-D) %s %s\n", s->name, s->macro+1);
```

Uses debug 474a.

`<macdef() debug 489b>≡` (85b)

```
if(debug['m'])
    print("#define %s %s\n", s->name, s->macro+1);
```

Uses debug 474a.

`<macexpand() debug macro expansion 489c>≡` (90)

```
if(debug['m'])
    print("#expand %s %s\n", s->name, ob);
```

Uses debug 474a.

A.3.2 File inclusion debugging: 5c -e

`<newfile() debug 489d>≡` (53g)

```
if(debug['e'])
    print("%L: %s\n", lineno, s);
```

Uses debug 474a and lineno 29f.

A.3.3 Line information debugging: 5c -f:

`<linehist() debug 489e>≡` (81d)

```
if(debug['f'])
    if(f) {
        if(offset)
            print("%4ld: %s (#line %d)\n", lineno, f, offset);
        else
            print("%4ld: %s\n", lineno, f);
    } else
        print("%4ld: <pop>\n", lineno);
```

Uses debug 474a and lineno 29f.

A.4 Parsing debugging

A.4.1 Printing names: 5c -L

`<yylex() in talph case, debug symbol 489f>≡` (60g)

```
if(debug['L'])
    print("%L: %s\n", lineno, symb);
```

Uses debug 474a, lineno 29f, and symb 35g.

A.4.2 Printing declarations: 5c -d

`<dodecl() debug declaration 490a>≡` (104d)
 if(debug['d'])
 dbgdecl(s);

Uses dbgdecl() 490e and debug 474a.

`<dlllabel() debug declaration 490b>≡` (109a)
 if(debug['d'])
 dbgdecl(s);

Uses dbgdecl() 490e and debug 474a.

`<doinit() debug declaration 490c>≡` (144b)
 if(debug['d'])
 dbgdecl(s);

Uses dbgdecl() 490e and debug 474a.

`<doenum() debug declaration 490d>≡` (134c)
 if(debug['d'])
 dbgdecl(s);

Uses dbgdecl() 490e and debug 474a.

`<function dbgdecl 490e>≡` (525)
 void
 dbgdecl(Sym *s)
 {
 print("decl \"%s\": C=%s [B=%d:O=%ld] T=%T\n",
 s->name, cnames[s->class], s->block, s->offset, s->type);
 }

Uses cnames 485a.

`<revertdecl() debug revert DSUE 490f>≡` (41b)
 if(debug['d'])
 print("revert2 \"%s\"\n", s->name);

Uses debug 474a.

`<revertdecl() debug revert DLABEL 490g>≡` (41c)
 if(debug['d'])
 print("revert3 \"%s\"\n", s->name);

Uses debug 474a.

`<revertdecl() debug revert DAUTO 490h>≡` (41d)
 if(debug['d'])
 print("revert1 \"%s\"\n", s->name);

Uses debug 474a.

A.5 Typing debugging

A.5.1 Printing expression type trees: 5c -t

`<complex() debug tree pre complex 490i>≡` (210a)
 if(debug['t'])
 if(n->op != OCONST)
 prttree(n, "pre complex");

Uses OCONST 111a, debug 474a, and prttree() 487b.

$\langle \text{complex}() \text{ debug tree after tcom and comma 491a} \rangle \equiv$ (210a)

```
if(debug['t'])
    if(n->op != OCONST)
        prttree(n, "t complex");
```

Uses OCONST 111a, debug 474a, and prttree() 487b.

$\langle \text{complex}() \text{ debug tree after ccom 491b} \rangle \equiv$ (210a)

```
if(debug['t'])
    if(n->op != OCONST)
        prttree(n, "c complex");
```

Uses OCONST 111a, debug 474a, and prttree() 487b.

$\langle \text{complex}() \text{ debug tree after acom 491c} \rangle \equiv$ (210a)

```
if(debug['t'])
    if(n->op != OCONST)
        prttree(n, "a complex");
```

Uses OCONST 111a, debug 474a, and prttree() 487b.

$\langle \text{complex}() \text{ debug tree after xcom 491d} \rangle \equiv$ (210a)

```
if(debug['t'])
    if(n->op != OCONST)
        prttree(n, "x complex");
```

Uses OCONST 111a, debug 474a, and prttree() 487b.

A.6 Code generation debugging

A.6.1 Printing processed opcodes: 5c -G

$\langle \text{gen}() \text{ debug opcode 491e} \rangle \equiv$ (197e)

```
if(debug['G'] && (o != OLIST))
    print("%L %0\n", nearln, o);
```

Uses OLIST 29b, debug 474a, and nearln 29g.

A.6.2 Printing code generation information: 5c -g

$\langle \text{cgenrel}() \text{ debug 491f} \rangle \equiv$ (223c)

```
if(debug['g']) {
    prttree(nn, "cgen lhs");
    prttree(n, "cgen");
}
```

Uses debug 474a and prttree() 487b.

$\langle \text{boolgen}() \text{ debug 491g} \rangle \equiv$ (253)

```
if(debug['g']) {
    prttree(nn, "boolgen lhs");
    prttree(n, "boolgen");
}
```

Uses debug 474a and prttree() 487b.

$\langle \text{gins}() \text{ debug 491h} \rangle \equiv$ (219a)

```
if(debug['g'])
    print("%P\n", p);
```

Uses debug 474a.

`<gopcode() debug 492a>≡` (229c)

```
    if(debug['g'])
        print("%P\n", p);
```

Uses debug 474a.

`<lcgen() debug 492b>≡` (235c)

```
    if(debug['g']) {
        prtree(nn, "lcgen lhs");
        prtree(n, "lcgen");
    }
```

Uses debug 474a and prtree() 487b.

`<sugen() debug 492c>≡` (243b)

```
    if(debug['g']) {
        prtree(nn, "sugen lhs");
        prtree(n, "sugen");
    }
```

Uses debug 474a and prtree() 487b.

A.6.3 Printing generated assembly: 5c -S

`<outcode() debug 492d>≡` (287c)

```
    if(debug['S']) {
        for(p = firstp; p != P; p = p->link)
            if(p->as != ADATA && p->as != AGLOBL)
                pc--;
        for(p = firstp; p != P; p = p->link) {
            print("%P\n", p);
            if(p->as != ADATA && p->as != AGLOBL)
                pc++;
        }
    }
```

Uses debug 474a.

`<global anames(arm) 492e>≡` (531a)

```
    char* anames[] =
    {
        "XXX",
        "NOP",
        "AND",
        "ORR",
        "EOR",
        "BIC",
        "ADD",
        "SUB",
        "RSB",
        "ADC",
        "SBC",
        "RSC",
        "MVN",
        "MUL",
        "DIV",
        "MOD",
        "MULA",
        "MULL",
        "MULAL",
        "MULLU",
        "MULALU",
    }
```

"MULU",
"DIVU",
"MODU",
"SLL",
"SRL",
"SRA",
"MOVW",
"MOVB",
"MOVB",
"MOVB",
"MOVH",
"MOVHU",
"MOVM",
"SWPW",
"SWPBU",
"CMP",
"TST",
"TEQ",
"CMN",
"B",
"BEQ",
"BNE",
"BHS",
"BLO",
"BMI",
"BPL",
"BVS",
"BVC",
"BHI",
"BLS",
"BGE",
"BLT",
"BGT",
"BLE",
"BL",
"RET",
"SWI",
"RFE",
"MOVWD",
"MOVWF",
"MOVDW",
"MOVFW",
"MOVFD",
"MOVDF",
"MOVF",
"MOVD",
"CMPF",
"CMPD",
"ADDF",
"ADDD",
"SUBF",
"SUBD",
"MULF",
"MULD",
"DIVF",
"DIVD",
"SQRTF",
"SQRTD",
"TEXT",
"GLOBL",
"DATA",

```

"WORD",
"END",
"NAME",
"HISTORY",
"CASE",
"BCASE",
"SIGNAME",
"GOK",
"DYNT",
"INIT",
"LAST",
};

```

A.7 Optimization debugging

A.7.1 Printing arithmetic trees: 5c -m

A.7.2 Printing registerization: 5c -r

A.8 Printing initializations: 5c -i

```

<doinit() debug initialization 494a>≡ (144b)
    if(debug['i']) {
        print("t = %T; o = %ld; n = %s\n", t, o, s->name);
        prttree(a, "doinit value");
    }

```

Uses debug 474a and prttree() 487b.

```

<init1() debug initialization 494b>≡ (262d)
    if(debug['i']) {
        print("t = %T; o = %ld; n = %s\n", t, o, s->name);
        prttree(a, "init1 value");
    }

```

Uses debug 474a and prttree() 487b.

```

<contig() debug initialization 494c>≡ (268b)
    if(debug['i']) {
        print("contig v = %ld; s = %s\n", v, s->name);
        prttree(n, "doinit value");
    }

```

Uses debug 474a and prttree() 487b.

Appendix B

Error Management

Error management uses a graduated severity system: `warn` prints warnings (only with `-w`), `yyerror` reports syntax/semantic errors with accurate file:line locations, `diag` reports general diagnostics, and `fatal` aborts immediately on internal errors. The `errorexit` function cleans up the output file before exiting, so a failed compilation does not leave a corrupt `.5` file behind.

```
<global nerrors 495a>≡ (511d)
    int nerrors;
```

B.0.1 `errorexit()`

```
<function errorexit 495b>≡ (514a)
    /// main | compile | newio | newfile | yylex | ... -> <>
    void
    errorexit(void)
    {
        if(outfile)
            remove(outfile);
        exits("error");
    }
```

Uses `outfile` 45c.

B.0.2 `yyerror()`

```
<function yyerror 495c>≡ (514a)
    void
    yyerror(char *fmt, ...)
    {
        char buf[STRINGSZ];
        va_list arg;

        <yyerror() when called from yyparse 496a>

        va_start(arg, fmt);
        vfprintf(buf, buf+sizeof(buf), fmt, arg);
        va_end(arg);

        Bprint(&diagbuf, "%L %s\n", lineno, buf);
        nerrors++;

        if(nerrors > 10) {
            Bprint(&diagbuf, "too many errors\n");
            errorexit();
        }
```



```

    }
}

```

Uses STRINGSZ 81e, diagbuf 47f, errexit() 495b, lineno 29f, and nerrors 495a.

<yyerror() when called from yyparse 496a>≡ (495c)

```

/*
 * hack to intercept message from yaccpar
 */
if(strcmp(fmt, "syntax error") == 0) {
    yyerror("syntax error, last name: %s", symb);
    return;
}

```

Uses symb 35g and yyerror() 495c.

B.0.3 diag()

<function diag 496b>≡ (518c)

```

void
diag(Node *n, char *fmt, ...)
{
    char buf[STRINGSZ];
    va_list arg;

    va_start(arg, fmt);
    vfprintf(buf, buf+sizeof(buf), fmt, arg);
    va_end(arg);

    Bprintf(&diagbuf, "%L %s\n", (n==Z)? nearln: n->lineno, buf);

    <fatal() or diag() abort if -X 497a>
    <fatal() or diag() print diagnostic if -v 497b>
    nerrors++;
    if(nerrors > 10) {
        Bprintf(&diagbuf, "too many errors\n");
        errexit();
    }
}

```

Uses STRINGSZ 81e, Z 29a, diagbuf 47f, errexit() 495b, nearln 29g, and nerrors 495a.

B.0.4 fatal()

<function fatal 496c>≡ (518c)

```

/// code generation stuff -> <>
void
fatal(Node *n, char *fmt, ...)
{
    char buf[STRINGSZ];
    va_list arg;

    va_start(arg, fmt);
    vfprintf(buf, buf+sizeof(buf), fmt, arg);
    va_end(arg);
    Bprintf(&diagbuf, "%L %s\n", (n==Z)? nearln: n->lineno, buf);

    <fatal() or diag() abort if -X 497a>
    <fatal() or diag() print diagnostic if -v 497b>
    nerrors++;
}

```

```

    errexit();
}

```

Uses STRINGSZ 81e, Z 29a, diagbuf 47f, errexit() 495b, nearln 29g, and nerrors 495a.

```

⟨fatal() or diag() abort if -X 497a⟩≡ (496)
    if(debug['X']){
        Bflush(&diagbuf);
        abort();
    }

```

Uses debug 474a and diagbuf 47f.

```

⟨fatal() or diag() print diagnostic if -v 497b⟩≡ (496)
    if(n != Z)
        if(debug['v'])
            prtreen(n, "diagnostic");

```

Uses Z 29a, debug 474a, and prtreen() 487b.

B.0.5 Error location

The %L format verb converts a global line number into a human-readable “file:line” string by walking the Hist linked list. It maintains a stack of active include files (Atab) and applies the line offset deltas to reconstruct the local line number within each file. This is used by diag, yyerror, and warn to produce accurate error messages even through deep #include nesting and #line directives.

```

⟨cinit() fmtinstall 497c⟩+≡ (50d) <474b
    fmtinstall('L', Lconv);

```

Uses Lconv() 497f.

```

⟨constant HISTSZ 497d⟩≡ (504b)
    #define HISTSZ 20

```

```

⟨struct Atab 497e⟩≡ (521b)
    struct Atab
    {
        Hist* incl; /* start of this include file */
        long idel; /* delta line number to apply to include */
        Hist* line; /* start of this #line directive */
        long ldel; /* delta line number to apply to #line */
    };

```

```

⟨function Lconv 497f⟩≡ (521b)
    // int -> string?
    int
    Lconv(Fmt *fp)
    {
        char str[STRINGSZ], s[STRINGSZ];
        Hist *h;
        long l, d;
        int i, n;
        struct Atab a[HISTSZ];

        l = va_arg(fp->args, long);
        n = 0;
        for(h = hist; h != H; h = h->link) {
            if(l < h->line)
                break;
            if(h->name) {
                if(h->offset != 0) { /* #line directive, not #pragma */

```

```

        if(n > 0 && n < HISTSZ && h->offset >= 0) {
            a[n-1].line = h;
            a[n-1].ldel = h->line - h->offset + 1;
        }
    } else {
        if(n < HISTSZ) { /* beginning of file */
            a[n].incl = h;
            a[n].idel = h->line;
            a[n].line = 0;
        }
        n++;
    }
    continue;
}
n--;
if(n > 0 && n < HISTSZ) {
    d = h->line - a[n].incl->line;
    a[n-1].ldel += d;
    a[n-1].idel += d;
}
}
if(n > HISTSZ)
    n = HISTSZ;
str[0] = 0;
for(i=n-1; i>=0; i--) {
    if(i != n-1) {
        if(fp->flags & ~(FmtWidth|FmtPrec)) /* BUG ROB - was f3 */
            break;
        strcat(str, " ");
    }
    if(a[i].line)
        snprintf(s, STRINGSZ, "%s:%ld[%s:%ld]",
            a[i].line->name, l-a[i].ldel+1,
            a[i].incl->name, l-a[i].idel+1);
    else
        snprintf(s, STRINGSZ, "%s:%ld",
            a[i].incl->name, l-a[i].idel+1);
    if(strlen(s)+strlen(str) >= STRINGSZ-10)
        break;
    strcat(str, s);
    l = a[i].incl->line - 1; /* now print out start of this file */
}
if(n == 0)
    strcat(str, "<eof>");

return fmtstrcpy(fp, str);
}

```

Uses Atab 497e, H 80c, HISTSZ 497d, STRINGSZ 81e, and hist 80e.

Appendix C

Libc

The compiler uses a few custom library routines rather than standard libc functions. The most notable is the arena allocator (`alloc/allocn`) built on a “hunk” of 50 KB blocks: since the compiler never frees AST nodes or type structures (they live until the function is compiled and emitted), this is faster than `malloc` and simpler than a garbage collector.

C.1 Memory management

The allocator maintains a global “hunk” pointer into a 50 KB block. `alloc` bumps `hunk` by the requested size, 4-byte aligned; when the block is exhausted, `gethunk` allocates a new one from the OS. `allocn` extends an existing allocation (like a simple `realloc`). Since the compiler processes one function at a time and never frees anything, this bump allocator is both fast and leak-free. The same pattern appears in the assembler and linker.

```
<constant NHUNK 499a>≡ (504b)
#define NHUNK 50000L
```

```
<global hunk 499b>≡ (511d)
char* hunk;
```

```
<global nhunk 499c>≡ (511d)
long nhunk;
```

```
<global thunk 499d>≡ (511d)
long thunk;
```

```
<cinit() memory globals initialization 499e>≡ (50d)
nhunk = 0;
```

Uses `nhunk 499c`.

```
<function gethunk 499f>≡ (514a)
void
gethunk(void)
{
    char *h;
    long nh;

    nh = NHUNK;
    if(thunk >= 10L*NHUNK)
        nh = 10L*NHUNK;

    h = (char*)sbrk(nh);
    if(h == (char*)-1) {
        yyerror("out of memory");
```

```

        errexit();
    }
    hunk = h;
    nhunk = nh;
    thunk += nh;
}

```

Uses NHUNK 499a, errexit() 495b, hunk 499b, nhunk 499c, thunk 499d, and yerror() 495c.

```

⟨constant MAXALIGN 500a⟩≡ (504b)
#define MAXALIGN 7

```

```

⟨function alloc 500b⟩≡ (514a)
/*
 * real allocs
 */
void*
alloc(long n)
{
    void *p;

    while((uintptr)hunk & MAXALIGN) {
        hunk++;
        nhunk--;
    }
    while(nhunk < n)
        gethunk();
    p = hunk;
    nhunk -= n;
    hunk += n;
    return p;
}

```

Uses MAXALIGN 500a, gethunk() 499f, hunk 499b, and nhunk 499c.

```

⟨function allocn 500c⟩≡ (514a)
void*
allocn(void *p, long on, long n)
{
    void *q;

    q = (uchar*)p + on;
    if(q != hunk || nhunk < n) {
        while(nhunk < on+n)
            gethunk();
        memmove(hunk, p, on);
        p = hunk;
        hunk += on;
        nhunk -= on;
    }
    hunk += n;
    nhunk -= n;
    return p;
}

```

Uses gethunk() 499f, hunk 499b, and nhunk 499c.

```

⟨function malloc 500d⟩≡ (513b)
/*
 * fake mallocs
 */
void*

```

```

malloc(ulong n)
{
    return alloc(n);
}

```

Uses alloc() 500b.

```

⟨function free 501a⟩≡ (513b)
void
free(void*)
{
}

```

```

⟨function setmalloctag 501b⟩≡ (513b)
/*@Scheck: looks dead, but because we redefine malloc/free we must also redefine that
void setmalloctag(void*, ulong)
{
}

```

C.2 String management

C.3 Bit operations

These utility functions manipulate individual bits: `topbit` returns the index of the highest set bit, `bitno` returns the index of the single set bit (or -1), and `log2` returns n if the value is 2^n (used by the power-of-two optimization in `vlog`).

```

⟨function topbit 501c⟩≡ (518c)
int
topbit(ulong v)
{
    int i;

    for(i = -1; v; i++)
        v >>= 1;
    return i;
}

```

```

⟨function bitno 501d⟩≡ (518c)
int
bitno(long b)
{
    int i;

    for(i=0; i<32; i++)
        if(b & (1L<<i))
            return i;
    diag(Z, "bad in bitno");
    return 0;
}

```

Uses Z 29a and diag() 496b.

```

⟨function SIGN 501e⟩≡ (504b)
#define SIGN(n) (1ULL<<(n-1))

```

```

⟨function MASK 501f⟩≡ (504b)
#define MASK(n) (SIGN(n) | (SIGN(n)-1))

```

C.4 Mathematic functions

<function round 502a>≡ (525)

```
long
round(long v, int w)
{
    int r;

    if(w <= 0 || w > 8) {
        diag(Z, "rounding by %d", w);
        w = 1;
    }
    r = v/w;
    if(r)
        v += w-r;
    return v;
}
```

Uses Z 29a and diag() 496b.

<function log2 502b>≡ (518c)

```
/*
 * return log(n) if n is a power of 2 constant
 */
int
log2(uvlong v)
{
    int s, i;
    uvlong m;

    s = 0;
    m = MASK(8*sizeof(uvlong));
    for(i=32; i; i>>=1) {
        m >>= i;
        if(!(v & m)) {
            v >>= i;
            s += i;
        }
    }
    if(v == 1)
        return s;
    return -1;
}
```

Uses MASK 501f.

C.5 Key/value data structure

<struct Init 502c>≡ (504b)

```
struct Init /* general purpose initialization */
{
    int code;
    ulong value;
    char* s;
};
```

C.6 Portability

The compiler supports cross-compilation from multiple host OSes (Plan 9, Unix/Linux, formerly Windows). The `systemtype` function checks the current OS, and OS-specific differences (file path separators, file creation modes, byte order) are abstracted through these portability wrappers.

```
<enum os 503>≡ (504b)
enum os /* also in ../{8a,5a,0a}.h */
{
    Plan9 = 1<<0,
    Unix = 1<<1,
    //Windows = 1<<2,
};
```


Appendix D

Extra Code

D.1 cc/

D.1.1 cc/cc.h

```
<constant YYMAXDEPTH 504a>≡ (504b)
#define YYMAXDEPTH 1500

<cc/cc.h 504b>≡
#include <u.h>
#include <libc.h>
#include <bio.h>
#include <ctype.h>

#pragma lib "../cc/cc.a$0" //$

typedef struct Node Node;
typedef struct Type Type;
typedef struct Sym Sym;
typedef struct Decl Decl;
typedef struct Funct Funct;
typedef struct Io Io;
typedef struct Hist Hist;
typedef struct Term Term;
typedef struct Init Init;
typedef struct Bits Bits;

typedef Rune TRune; /* target system type */

<constant NHUNK 499a>
<constant BUFSIZ 54a>
<constant NSYMB 35h>
<constant NHASH 35b>
<constant STRINGSZ 81e>
<constant HISTSZ 497d>
<constant YYMAXDEPTH 504a>
<constant NTERM 328a>
<constant MAXALIGN 500a>

<function SIGN 501e>
<function MASK 501f>

<constant BITS 349b>
<constant NVAR 351d>
<struct Bits 349c>
```

```

<struct Node 28>
<constant Z 29a>

<struct Sym 34c>
<constant S 35d>

<enum signature 394a>

<struct Decl 38b>
<constant D 39c>

<struct Type 31d>
<constant T 31i>
<constant NODECL 118d>

<struct Init 502c>

<struct Fi 54c>
extern struct Fi fi;

<struct Io 52b>
<constant I 52e>

<struct Hist 80b>
<constant H 80c>
extern Hist* hist;

<struct Term 328b>

<enum os 503>
<enum node_kind 27>
<enum type_kind 30a>
<enum type_kind_bis 120e>
<enum align 276b>
<enum dxxx 38a>
<enum storage_class 33b>
<enum qualifier 33a>
<enum bxxx 121>

<struct Funct 438c>

<struct En 133a>
extern struct En en;

extern int autobn;
extern long autooffset;
extern int blockno;
extern Decl* dclstack;
extern char debug[256];
extern Hist* ehist;
extern bool firstbit;
extern Sym* firstarg;
extern Type* firstargtype;
extern Decl* firstdcl;
extern Sym* hash[NHASH];
extern char* hunk;
extern char** include;
extern Io* iofree;

```

```

extern Io* ionext;
extern Io* iostack;
extern long lastbit;
extern char lastclass;
extern Type* lastdcltype;
extern long lastfield;
extern Type* lasttype;
extern long lineno;
extern long nearln;
extern int maxinclude;
extern int nerrors;
extern bool newflag;
extern long nhunk;
extern int ninclude;
extern Node* nodproto;
extern Node* nodcast;
extern Biobuf outbuf;
extern Biobuf diagbuf;
extern char* outfile;
extern char* pathname;
extern int peekc;
extern long stkoff;
extern Type* strf;
extern Type* strl;
extern char symb[NSYMB];
extern Sym* symstring;
extern int taggen;
extern Type* tfield;
extern Type* tufield;
extern int thechar;
extern char* thestring;
extern Type* thisfntype;
extern long thunk;
extern Type* types[NTYPE];

extern Type* fntypes[NTYPE];
extern Node* initlist;
extern Term term[NTERM];
extern int nterm;
extern int packflg;
extern int fpoundflg;
extern bool profileflg;
extern int ncontin;
extern int canreach;
extern int warnreach;
extern Bits zbits;

extern char *onames[], *tnames[], *gnames[];
extern char *cnames[], *qnames[], *bnames[];

extern char comrel[], invrel[], logrel[];
extern long ncast[], tadd[], tand[];
extern long targ[], tasadd[], tasign[], tcast[];
extern long tdot[], tfunct[], tindir[], tmul[];
extern long tnot[], trel[], tsub[];

extern char typeaf[];
extern char typefd[];
extern char typei[];
extern char typesu[];

```

```

extern char typesuv[];
extern char typeu[];
extern char typev[];
extern char typeilp[];
extern char typechl[];

// not used on ARM
extern char typechlv[];
extern char typeil[];

extern char typechlp[];
extern char typechlpfd[];

extern char* typeswitch;
extern char* typeword;
extern char* typecmlpx;

extern ulong thash1;
extern ulong thash2;
extern ulong thash3;
extern ulong thash[];

/*
 * compat.c/unix.c/windows.c
 */
int mywait(int*);
int mycreat(char*, int);
int myaccess(char*);
int pathchar(void);
bool systemtype(int);

// utils.c
void gethunk(void);
void* allocn(void*, long, long);
void* alloc(long);
void errexit(void);
void yyerror(char*, ...);

/*
 * parser
 */
/*@Scheck: def in y.tab.c from cc.y
int yyparse(void);

/*
 * lex.c
 */
long yylex(void);

Sym* lookup(void);
Sym* slookup(char*);

int filbuf(void);
int getc(void);
int getnsc(void);
void newfile(char*, int);
void newio(void);
void pushio(void);
void unget(int);

```

```

// used by dpchk.c
long gettr(void);

//!!!! (hmmm in lex.c, as well as cinit(), compile())
void main(int, char*[]);

/*
 * mac.c
 */
void dodefne(char*);
void domacro(void);
void linehist(char*, int);
void macexpand(Sym*, char*);

// for dpchk.c
Sym* getsym(void);
long getnsn(void);

/*
 * dcl.c
 */
/*@Scheck: useful, used by cc.y
Type* tcopy(Type*);
/*@Scheck: useful, used by cc.y
Node* doinit(Sym*, Type*, long, Node*);
/*@Scheck: useful, used by cc.y
void adecl(int, Type*, Sym*);
void argmark(Node*, int);
Node* dcllabel(Sym*, int);
Node* dodecl(void(*) (int, Type*, Sym*), int, Type*, Node*);
/*@Scheck: useful, used by cc.y
Sym* mkstatic(Sym*);
void doenum(Sym*, Node*);
void snap(Type*);
Type* dotag(Sym*, int, int);
void edecl(int, Type*, Sym*);
void markdcl(void);
/*@Scheck: useful, used by cc.y
void pdecl(int, Type*, Sym*);
Node* revertdcl(void);
long round(long, int);
int sametype(Type*, Type*);
ulong sign(Sym*);
ulong signature(Type*);
void sualign(Type*);
void xdecl(int, Type*, Sym*);
Node* contig(Sym*, Node*, long);

/*
 * com.c
 */
void ccom(Node*);
void complex(Node*);
int tcom(Node*);
int tcoma(Node*, Node*, Type*, int);
int tcomo(Node*, int);
void constas(Node*, Type*, Type*);
Node* uncomma(Node*);

```

```

/*
 * con.c
 */
void acom(Node*);
void evconst(Node*);

/*
 * funct.c
 */
int isfunct(Node*);
void dclfunct(Type*, Sym*);

/*
 * sub.c
 */
void arith(Node*, int);
int deadheads(Node*);
Type* dotsearch(Sym*, Type*, Node*, long*);
Node* invert(Node*);
int bitno(long);
void makedot(Node*, Type*, long);
int mixedasop(Type*, Type*);
Node* new(int, Node*, Node*);
Node* new1(int, Node*, Node*);
int nilcast(Type*, Type*);
int nocast(Type*, Type*);
void prttree(Node*, char*);
void prttree1(Node*, int, int);
void relcon(Node*, Node*);
int relindex(int);
/*@Scheck: useful, used by cc.y
int simpleg(long);
Type* garbt(Type*, long);
int simplec(long);
Type* simplet(long);
int stcompat(Node*, Type*, Type*, long[]);
int tcompat(Node*, Type*, Type*, long[]);
void tinit(void);
Type* typ(int, Type*);
Type* copytyp(Type*);
void typeext(Type*, Node*);
void typeext1(Type*, Node*);
int side(Node*);
int vconst(Node*);
int log2(uvlong);
int vlog(Node*);
int topbit(ulong);
void simplifyshift(Node*);
long typebitor(long, long);
void diag(Node*, char*, ...);
void warn(Node*, char*, ...);
void fatal(Node*, char*, ...);

/*
 * acid.c
 */
void acidtype(Type*);
void acidvar(Sym*);

/*

```

```

    * pickle.c
    */
void pickletype(Type*);

/*
    * bits.c
    */
Bits bor(Bits, Bits);
int bany(Bits*);
int bnum(Bits);
Bits blsh(uint);
int beq(Bits, Bits);
int bset(Bits, uint);
//Bits band(Bits, Bits);
//Bits bnot(Bits);

/*
    * dpchk.c
    */
void dpcheck(Node*);
void arginit(void);

void pragvararg(void);
void pragpack(void);
void pragfpround(void);
void pragprofile(void);
void pragincomplete(void);

/*
    * calls to machine depend part
    */
//todo: could define an interface instantiated by each xc
void codgen(Node*, Node*);
void gclean(void);
void gextern(Sym*, Node*, long, long);
void ginit(void);
long outstring(char*, long);
long outlstring(TRune*, long);
void xcom(Node*);
long exreg(Type*);
long align(long, Type*, int);
long maxround(long, long);

extern schar ewidth[];

/*
    * com64
    */
int com64(Node*);
void com64init(void);
void bool64(Node*);
vlong convvtov(vlong, int);
//double convvtov(vlong);
//vlong convftov(double);
//double convftov(double, int);

/*
    * machcap
    */
int machcap(Node*);

```

```
#pragma varargck argpos warn 2
#pragma varargck argpos diag 2
#pragma varargck argpos yyerror 1
```

```
#pragma varargck type "F" Node*
#pragma varargck type "L" long
#pragma varargck type "Q" long
#pragma varargck type "O" int
#pragma varargck type "T" Type*
#pragma varargck type "|" int
```

Uses Bits 504b, Decl 504b, Funct 504b, Hist 504b, Init 504b, Io 504b, Node 504b, Sym 504b, Term 504b, and Type 504b.

D.1.2 cc/globals.c

```
<global fproundflg 511a>≡ (511d)
int fproundflg;
```

```
<macprag() if pragma xxx 511b>+≡ (96a) <469a
if(s && strcmp(s->name, "fpround") == 0) {
    pragfpround();
    return;
}
```

Uses pragfpround() 511c.

```
<function pragfpround 511c>≡ (528b)
void
pragfpround(void)
{
    Sym *s;

    fproundflg = 0;
    s = getsym();
    if(s) {
        fproundflg = atoi(s->name+1);
        if(strcmp(s->name, "on") == 0 ||
            strcmp(s->name, "yes") == 0)
            fproundflg = 1;
    }
    while(getnsc() != '\n')
        ;
    if(debug['f'])
        if(fproundflg)
            print("%4ld: fproundflg %d\n", lineno, fproundflg);
        else
            print("%4ld: fproundflg off\n", lineno);
}
```

Uses debug 474a, fproundflg 511a, getnsc() 77c, getsym() 77b, and lineno 29f.

```
<cc/globals.c 511d>≡
#include "cc.h"
```

```
<global fi 54d>
<global hist 80e>
<global en 133b>
```

```
<global autobn 39h>
<global autooffset 196d>
```


<global blockno 39g>
 <global dclstack 39a>
 <global debug 474a>
 <global ehist 81a>
 <global firstbit 417a>
 <global firstarg 274j>
 <global firstargtype 275a>
 <global firstdcl 105a>
 <global hash 35a>
 <global hunk 499b>
 <global include 79b>
 <global iofree 53c>
 <global ionext 52f>
 <global iostack 52c>
 <global lastbit 417b>
 <global lastclass 102b>
 <global lastdcl 103b>
 <global lastfield 418b>
 <global lasttype 102c>
 <global lineno 29f>
 <global nearln 29g>
 <global maxinclude 79d>
 <global nerrors 495a>
 <global newflag 470b>
 <global nhunk 499c>
 <global ninclude 79c>
 <global nodproto 175b>
 <global nodcast 262a>
 <global outbuf 47e>
 <global diagbuf 47f>
 <global outfile 45c>
 <global pathname 51a>
 <global peekc 57c>
 <global stkoff 196c>
 <global strf 130f>
 <global strl 130g>
 <global symb 35g>
 <global symstring 226c>
 <global taggen 130c>
 <global tfield 419b>
 <global tufield 419c>
 <global thechar 45a>
 <global thestring 45b>
 <global thisfn 105b>
 <global thunk 499d>
 <global types 32b>
 <global fntypes 459a>
 <global initlist 120b>
 <global term 328c>
 <global nterm 328d>
 <global packflg 469b>
 <global fproundflg 511a>
 <global profileflg 412a>
 <global ncontin 205h>
 <global canreach 185d>
 <global warnreach 185e>
 <global zbits 350a>

 <global typeswitch 179d>
 <global typeword 337c>

<global typecmplx 271a>

D.1.3 cc/omachcap.c

<cc/omachcap.c 513a>≡
#include "cc.h"

```
/* default, like old cc */  
//int  
//machcap(Node*)  
//{  
// return 0;  
//}
```

D.1.4 cc/compat.c

<cc/compat.c 513b>≡
#include "cc.h"

```
//#include "compat"  
//TODO copy paste with aa/, maybe could factorize with another lib?  
//
```

```
int  
myaccess(char *f)  
{  
    return access(f, AEXIST);  
}
```

```
int  
mycreat(char *n, int p)  
{  
  
    return create(n, 1, p);  
}
```

```
int  
mywait(int *s)  
{  
    int p;  
    Waitmsg *w;  
  
    if((w = wait()) == nil)  
        return -1;  
    else{  
        p = w->pid;  
        *s = 0;  
        if(w->msg[0])  
            *s = 1;  
        free(w);  
        return p;  
    }  
}
```

```
bool  
systemtype(int sys)  
{
```

```
    return sys & Plan9;
}
```

```
int
pathchar(void)
{
    return '/';
}
```

<function malloc 500d>

<function free 501a>

<function setmalloctag 501b>

Uses Plan9 503 and free() 501a.

D.1.5 cc/utils.c

<cc/utils.c 514a>≡
#include "cc.h"

<function errexit 495b>

<function gethunk 499f>

<function alloc 500b>

<function allocn 500c>

<function yyerror 495c>

D.1.6 cc/mac.c

<cc/mac.c 514b>≡
#include "cc.h"

```
//old: #include "macbody"
//TODO copy paste with aa/, maybe could factorize with another lib?
```

```
void macdef(void);
void macinc(void);
void macprag(void);
void maclin(void);
void macif(int);
void macend(void);
void macund(void);
```

```
long getnsn(void);
Sym* getsym(void);
```

<constant VARMAC 86a>

<function getnsn 97>

<function getsym 77b>

<function getsymdots 87a>

<function getcom 83c>
 <function dodefine 84c>
 <global mactab 76b>
 <function domacro 76c>
 <function macund 94a>
 <constant NARG 85a>
 <function macdef 85b>
 <function macexpand 90c>
 <function macinc 81f>
 <function maclin 96b>
 <function macif 94b>
 <function macprag 96a>
 <function macend 78c>
 <function linehist 81d>

D.1.7 cc/sub.c

```

<function simplifyshift 515>≡ (518c)
/*
 * try to rewrite shift & mask
 */
void
simplifyshift(Node *n)
{
    ulong c3;
    int o, s1, s2, c1, c2;

    if(!typechlp[n->type->etype])
        return;
    switch(n->op) {
    default:
        return;
    case OASHL:
        s1 = 0;
        break;
    case OLSHR:
        s1 = 1;
        break;
    case OASHR:
        s1 = 2;
        break;
    }
    if(n->right->op != OCONST)
        return;
    if(n->left->op != OAND)

```

```

        return;
if(n->left->right->op != OCONST)
    return;
switch(n->left->left->op) {
default:
    return;
case OASHL:
    s2 = 0;
    break;
case OLSHR:
    s2 = 1;
    break;
case OASHR:
    s2 = 2;
    break;
}
if(n->left->left->right->op != OCONST)
    return;

c1 = n->right->vconst;
c2 = n->left->left->right->vconst;
c3 = n->left->right->vconst;

/*
if(debug['h'])
    print("%.3o %ld %ld %d #%.lux\n",
        (s1<<3)|s2, c1, c2, topbit(c3), c3);
*/

o = n->op;
switch((s1<<3)|s2) {
case 000: /* (((e <<u c2) & c3) <<u c1) */
    c3 >>= c2;
    c1 += c2;
    if(c1 >= 32)
        break;
    goto rewrite1;

case 002: /* (((e >>s c2) & c3) <<u c1) */
    if(topbit(c3) >= (32-c2))
        break;

case 001: /* (((e >>u c2) & c3) <<u c1) */
    if(c1 > c2) {
        c3 <= c2;
        c1 -= c2;
        o = OASHL;
        goto rewrite1;
    }
    c3 <= c1;
    if(c1 == c2)
        goto rewrite0;
    c1 = c2-c1;
    o = OLSHR;
    goto rewrite2;

case 022: /* (((e >>s c2) & c3) >>s c1) */
    if(c2 <= 0)
        break;

case 012: /* (((e >>s c2) & c3) >>u c1) */
    if(topbit(c3) >= (32-c2))

```

```

        break;
    goto s11;
case 021: /* (((e >>u c2) & c3) >>s c1) */
    if(topbit(c3) >= 31 && c2 <= 0)
        break;
    goto s11;
case 011: /* (((e >>u c2) & c3) >>u c1) */
s11:
    c3 <<= c2;
    c1 += c2;
    if(c1 >= 32)
        break;
    o = OLSHR;
    goto rewrite1;

case 020: /* (((e <<u c2) & c3) >>s c1) */
    if(topbit(c3) >= 31)
        break;
case 010: /* (((e <<u c2) & c3) >>u c1) */
    c3 >>= c1;
    if(c1 == c2)
        goto rewrite0;
    if(c1 > c2) {
        c1 -= c2;
        goto rewrite2;
    }
    c1 = c2 - c1;
    o = OASHL;
    goto rewrite2;
}
return;

```

```

rewrite0: /* get rid of both shifts */
if(debug['<'])prtree(n, "rewrite0");
*n = *n->left;
n->left = n->left->left;
n->right->vconst = c3;
return;
rewrite1: /* get rid of lower shift */
if(debug['<'])prtree(n, "rewrite1");
n->left->left = n->left->left->left;
n->left->right->vconst = c3;
n->right->vconst = c1;
n->op = o;
return;
rewrite2: /* get rid of upper shift */
if(debug['<'])prtree(n, "rewrite2");
*n = *n->left;
n->right->vconst = c3;
n->left->right->vconst = c1;
n->left->op = o;
}

```

Uses OAND 115a, OASHL 115a, OASHR 115a, OCONST 111a, OLSHR 232b, debug 474a, prtree() 487b, topbit() 501c, and typechlp 215g.

$\langle \text{global typechlv } 517a \rangle \equiv$ (518c)
char typechlv[NTYPE];

Uses NTYPE 30a.

$\langle \text{global typeil } 517b \rangle \equiv$ (518c)

```
char typeil[NTYPE];
```

Uses NTYPE 30a.

```
<global typeilinit 518a>≡ (518c)
int typeilinit[] =
{
    TINT, TUINT, TLONG, TULONG, -1,
};
```

Uses TINT 30b, TLONG 30b, TUINT 30b, and TULONG 30b.

```
<tinit() initialise type:xxx type sets 518b>+≡ (50a) <149e
for(ip=typeilinit; *ip>=0; ip++) {
    urk("typeil", nelem(typeil), *ip);
    typeil[*ip] = 1;
}
```

Uses typeil 517b, typeilinit 518a, and urk() 50c.

```
<cc/sub.c 518c>≡
#include "cc.h"

int simpleg(long);
long dotoffset(Type*, Type*, Node*);

extern char tab[NTYPE][NTYPE];
```

<function new 29d>

<function new1 29h>

<function prttree 487b>

<function prttree1 487c>

<function typ 32a>

<function copytyp 127d>

<function garbt 127c>

<function simpleg 127e>

<function simplec 126c>

<function simplet 124e>

<function stcompat 150c>

<function tcompat 150a>

<function makedot 241b>

<function dotsearch 171c>

<function dotoffset 428b>

<function allfloat 450e>

<function constas 190d>

<function typeext1 450d>
 <function typeext 426f>
 <function nocast 249b>
 <function nilcast 299a>
 <function arith 160a>
 <function simplifyshift 515>
 <function side 295b>
 <function vconst 176c>
 <function log2 502b>
 <function vlog 310b>
 <function topbit 501c>
 <function relcon 298b>
 <function relindex 233a>
 <function invert 106a>
 <function bitno 501d>
 <function typebitor 123c>
 <function diag 496b>
 <function warn 142>
 <function fatal 496c>
 <global thash1 396b>
 <global thash2 396c>
 <global thash3 396d>
 <global thash 396e>
 <global thashinit 397a>
 <global bnames 276c>
 <global bnamesinit 276d>
 <global tnames 477a>
 <global tnamesinit 477b>
 <global gnames 486a>
 <global gnamesinit 486b>
 <global qnames 485c>
 <global qnamesinit 485d>
 <global cnames 485a>
 <global cnamesinit 485b>
 <global onames 474d>
 <global onamesinit 475>

$\langle \text{global comrel } 256a \rangle$
 $\langle \text{global invrel } 256b \rangle$
 $\langle \text{global logrel } 232i \rangle$

$\langle \text{global typei } 148c \rangle$
 $\langle \text{global typeiinit } 148d \rangle$
 $\langle \text{global typeu } 231h \rangle$
 $\langle \text{global typeuinit } 232a \rangle$

$\langle \text{global typesuv } 271c \rangle$
 $\langle \text{global typesuvinit } 271d \rangle$

// not used on ARM
 $\langle \text{global typechlv } 517a \rangle$
 $\langle \text{global typeil } 517b \rangle$
 $\langle \text{global typeilinit } 518a \rangle$

$\langle \text{global typeilp } 215e \rangle$
 $\langle \text{global typeilpinit } 215f \rangle$

$\langle \text{global typechl } 180b \rangle$
 $\langle \text{global typechlinit } 180c \rangle$

$\langle \text{global typechlp } 215g \rangle$
 $\langle \text{global typechlpinit } 215h \rangle$

$\langle \text{global typechlpfd } 357a \rangle$
 $\langle \text{global typechlpfdinit } 357b \rangle$

$\langle \text{global typev } 280a \rangle$
 $\langle \text{global typevinit } 280b \rangle$

$\langle \text{global typefd } 149c \rangle$
 $\langle \text{global typefdinit } 149d \rangle$

$\langle \text{global typeaf } 154b \rangle$
 $\langle \text{global typeafinit } 154c \rangle$

$\langle \text{global typesu } 149a \rangle$
 $\langle \text{global typesuinit } 149b \rangle$

$\langle \text{global tasign } 166c \rangle$
 $\langle \text{global tasigninit } 166d \rangle$

$\langle \text{global tasadd } 168d \rangle$
 $\langle \text{global tasaddinit } 168e \rangle$

$\langle \text{global tcast } 175e \rangle$
 $\langle \text{global tcastinit } 175f \rangle$

$\langle \text{global tadd } 151c \rangle$
 $\langle \text{global taddinit } 151d \rangle$

$\langle \text{global tsub } 156d \rangle$
 $\langle \text{global tsubinit } 156e \rangle$

$\langle \text{global tmul } 157b \rangle$
 $\langle \text{global tmulinit } 157c \rangle$

<global tand 159b>
 <global tandinit 159c>

 <global trel 164b>
 <global trelnit 164c>

 <global tfunct 172c>

 <global tindir 170c>

 <global tdot 171a>

 <global tnot 165b>

 <global targ 173b>

 <global tab 161a>

 <function urk 50c>

 <function tinit 50a>

 <function deadhead 186b>

 <function deadheads 187a>

 <function mixedasop 168c>

D.1.8 cc/bits.c

<cc/bits.c 521a>≡
 #include "cc.h"

 <function bor 350b>

 <function bany 350c>

 <function beq 350d>

 <function bnum 350e>

 <function blsh 351a>

 <function bset 351b>

D.1.9 cc/lex.c

<cc/lex.c 521b>≡
 #include "cc.h"
 #include "y.tab.h"

 int compile(char*, char**, int);
 void syminit(Sym*);
 int mpatov(char*, vlong*);
 long getr(void);
 long escchar(long, int, int);
 void cinit(void);

```

int Oconv(Fmt*);
int Lconv(Fmt*);
int Tconv(Fmt*);
int FNconv(Fmt*);
int Qconv(Fmt*);
int VBconv(Fmt*);

void setinclude(char*);

#ifdef CPP
<constant CPP 467a>
#endif

/*
 * known debug flags
 * -a  acid declaration output
 * -A  !B
 * -B  non ANSI
 * -d  print declarations
 * -D  name  define
 * -F  format specification check
 * -i  print initialization
 * -I  path  include
 * -l  generate little-endian code
 * -L  print every NAME symbol
 * -M  constant multiplication
 * -m  print add/sub/mul trees
 * -n  print acid to file (%.c=%.acid) (with -a or -aa)
 * -o  file  output file
 * -p  use standard cpp ANSI preprocessor (not on windows)
 * -r  print registerization
 * -s  print structure offsets (with -a or -aa)
 * -S  print assembly
 * -t  print type trees
 * -V  enable void* conversion warnings
 * -v  verbose printing
 * -w  print warnings
 * -X  abort on error
 * -.  Inhibit search for includes in source directory
 */

<function main 45d>

<function compile 47h>

<function pushio 83b>

<function newio 53b>

<function newfile 53g>

<function slookup 35f>

<function lookup 36a>

<function syminit 37h>

```

<constant EOF 54f>
 <constant IGN 57d>
 <constant ESC 74a>
 <function GETC 54e>

 <enum numxxx 64d>

 <function yylex 57a>

 <function mpatov 66>
 <functiongetc 78d>
 <functiongetr 72a>
 <functiongetnsc 77c>
 <functionunget 78b>
 <functionescchar 71a>
 <global itab 59>
 <functioncinit 50d>
 <functionfilbuf 55a>

 <functionOconv 474c>
 <structAtab 497e>
 <functionLconv 497f>
 <functionTconv 476>
 <functionFNconv 478a>
 <functionQconv 478b>
 <functionVBconv 479a>
 <functionsetinclude 79e>

D.1.10 cc/acid.c

<cc/acid.c 523>≡
 #include "cc.h"

 <global kwd 402e>

 <function amap 403a>

 <function acidsue 402c>

 <function acidfun 402d>

<global acidchar 404c>
 <global acidcinit 404d>

 <function acidinit 405a>

 <function acidmember 405b>

 <function acidtype 403b>

 <function acidvar 401c>

D.1.11 cc/pickle.c

<cc/pickle.c 524a>≡
 #include "cc.h"

 <global kwd (cc/pickle.c) 407b>
 <global picklestr 407c>

 <function pmap 407d>

 <function picklesue 408a>

 <global picklechar 408b>
 <global picklecinit 408c>

 <function pickleinit 408d>

 <function picklemember 409>

 <function pickletype 410>

D.1.12 cc/com64.c

<cc/com64.c 524b>≡
 #include "cc.h"

 <constant FNX 213d>

 <global nodaddv ??>
 <global nodsubv ??>
 <global nodmulv ??>
 <global noddivv ??>
 <global noddivvu ??>
 <global nodmodv ??>
 <global nodmodvu ??>
 <global nodlshv ??>
 <global nodrshav ??>
 <global nodrshlv ??>
 <global nodandv ??>
 <global nodorv ??>
 <global nodxorv ??>
 <global nodnegv ??>
 <global nodcomv ??>

 <global nodtestv ??>
 <global nodeqv ??>

```

<global nodnev ??>
<global nodlev ??>
<global nodltv ??>
<global nodgev ??>
<global nodgtv ??>
<global nodhiv ??>
<global nodhsv ??>
<global nodlov ??>
<global nodlsv ??>

<global nodf2v ??>
<global nodd2v ??>
<global nodp2v ??>
<global nodsi2v ??>
<global nodui2v ??>
<global nodsl2v ??>
<global nodul2v ??>
<global nodsh2v ??>
<global noduh2v ??>
<global nodsc2v ??>
<global noduc2v ??>

<global nodv2f ??>
<global nodv2d ??>
<global nodv2ui ??>
<global nodv2si ??>
<global nodv2ul ??>
<global nodv2sl ??>
<global nodv2uh ??>
<global nodv2sh ??>
<global nodv2uc ??>
<global nodv2sc ??>

<global nodvpp ??>
<global nodppv ??>
<global nodvmm ??>
<global nodmmv ??>

<global nodvasop ??>

<global etconv 458a>
<global initetconv 458b>

<function fvn 459b>

<function com64init 457b>

<function com64 459d>

<function bool64 466b>

<function convvttox 65d>

```

D.1.13 cc/dcl.c

```

<cc/dcl.c 525>≡
#include "cc.h"

Sym* mkstatic(Sym*);

```

```

Type* tcopy(Type*);
Node* doinit(Sym*, Type*, long, Node*);
Node* init1(Sym*, Type*, long, int);
Node* newlist(Node*, Node*);
void walkparam(Node*, int);
Type* fnproto(Node*);
Type* fnproto1(Node*);
int anyproto(Node*);
void dbgdecl(Sym*);
Decl* push(void);
Decl* push1(Sym*);
int rsametype(Type*, Type*, int, int);
Type* paramconv(Type*, int);
void adecl(int, Type*, Sym*);
void pdecl(int, Type*, Sym*);
void tmerge(Type*, Sym*);

```

<function dodecl 103a>

<function mkstatic 415f>

<function tcopy 182f>

<function doinit 120c>

<function peekinit 267a>

<function nextinit 267b>

<function isstruct 266b>

<function init1 262d>

<function newlist 266a>

<function sualign 277d>

<function round 502a>

<function ofnproto 423d>

<constant ANSIPROTO 423e>

<constant OLDPROTO 423a>

<function argmark 138a>

<function walkparam 138b>

<function markdcl 40a>

<function reverttdcl 40d>

<function fnproto 136a>

<function anyproto 424b>

<function fnproto1 136b>

<function dbgdecl 490e>

⟨function push 39e⟩

⟨function push1 39f⟩

⟨function sametype 145c⟩

⟨function rsametype 145d⟩

typedef struct Typetab Typetab;

⟨struct Typetab 397b⟩

⟨function sigind 397c⟩

⟨function signat 396a⟩

⟨function signature 395c⟩

⟨function sign 395b⟩

⟨function snap 147b⟩

⟨function dotag 129a⟩

⟨function dlllabel 109a⟩

⟨function paramconv 182c⟩

⟨function adecl 140c⟩

⟨function pdecl 139c⟩

⟨function xdecl 104e⟩

⟨function tmerge 148b⟩

⟨function edecl 132c⟩

⟨function maxtype 181b⟩

⟨function doenum 134c⟩

⟨function symadjust 269⟩

⟨function contig 268b⟩

Uses Typetab 397b.

D.1.14 cc/funct.c

⟨cc/funct.c 527⟩≡
#include "cc.h"

typedef struct Ftab Ftab;
typedef struct Gtab Gtab;

⟨struct Ftab 436a⟩

⟨struct Gtab 436b⟩

⟨global gtabinit 436c⟩

<global ftabinit 437>

<function isfunct 438d>

<function dclfunct 434i>

Uses Ftab 436a and Gtab 436b.

D.1.15 cc/scon.c

<cc/scon.c 528a>≡

```
#include "cc.h"
```

```
void acom1(vlong, Node*);
void acom2(Node*, Type*);
int acomcmp1(const void*, const void*);
int acomcmp2(const void*, const void*);
int addo(Node*);
```

<function acast 331a>

<function evconst 304a>

<function acom 328e>

<function acomcmp1 331b>

<function acomcmp2 332a>

<function acom2 332b>

<function acom1 330>

<function addo 329>

D.1.16 cc/dpchk.c

<cc/dpchk.c 528b>≡

```
#include "cc.h"
```

```
#include "y.tab.h"
```

<enum fxxx 443a>

```
typedef struct Tprot Tprot;
```

<struct Tprot 443b>

```
typedef struct Tname Tname;
```

<struct Tname 443c>

<global indchar 443d>

<global flagbits 443e>

<global fmtbuf 443f>

<global lastadj 443g>

<global lastverb 443h>

<global nstar 443i>

<global tprot 443j>

<global tname 443k>

<function argflag 444a>

<function getflag 444b>
 <function arginit 445a>
 <function nextarg 445b>
 <function checkargs 446>
 <function dpcheck 447b>

 <function pragpack 469c>
 <function newname 449a>
 <function newprot 449b>
 <function pragvararg 448b>
 <function pragfpround 511c>
 <function pragprofile 412e>
 <function pragincomplete 398b>

Uses Tname 443c and Tprot 443b.

D.1.17 cc/com.c

<cc/com.c 529>≡
 #include "cc.h"

 int tcomd(Node*);
 int tcomx(Node*);
 int tlvalue(Node*);

 typedef struct Com Com;
 typedef struct Big Big;

 <struct Com 308a>

 int compar(Node*, int);
 static void comma(Node*);
 static Node* commas(Com*, Node*);

 <function complex 210a>

 <enum _anon_ (cc/com.c) 153a>

 <function tcom 152b>
 <function tcomo 153b>
 <function tcoma 173a>
 <function tcomd 171b>
 <function tcomx 430c>

⟨function tlvalue 166b⟩

⟨function comargs 310a⟩

⟨function commas 309⟩

⟨function comma 308b⟩

⟨function ccom 292⟩

⟨global cmps 300e⟩

⟨struct Big 299b⟩

⟨function cmp 300b⟩

⟨function add 300c⟩

⟨function big 300a⟩

⟨function compar 300f⟩

Uses Big 299b and Com 308a.

D.2 cc2/

D.2.1 cc2/pswt.c

⟨cc2/pswt.c 530a⟩≡
#include "gc.h"

⟨function swcmp 281g⟩

⟨function doswit 281h⟩

⟨function casf 283a⟩

⟨function outlstring 228a⟩

⟨function nullwarn 189b⟩

⟨function ieedtod 452g⟩

D.2.2 cc2/pgen.c

⟨cc2/pgen.c 530b⟩≡
#include "gc.h"

```
void gen(Node*);  
void usedset(Node*, int);  
int bcomplex(Node*, Node*);
```

⟨function codgen 194c⟩

⟨function supgen 188b⟩

⟨function uncomma 206d⟩

⟨function gen 197e⟩

<function usedset 184f>

<function bcomplex 199e>

D.3 5c/

D.3.1 5c/enam.c

<5c/enam.c 531a>≡
<global anames(arm) 492e>

D.3.2 5c/gc.h

<5c/gc.h 531b>≡
#include "../cc/cc.h"

#include <common.out.h>
#include <5.out.h>

/*
 * 5c/arm
 * Arm
 */

<constant SZ_CHAR(arm) 274a>
<constant SZ_SHORT(arm) 274b>
<constant SZ_INT(arm) 274c>
<constant SZ_LONG(arm) 274d>
<constant SZ_IND(arm) 274e>
<constant SZ_FLOAT(arm) 274f>
<constant SZ_VLONG(arm) 274g>
<constant SZ_DOUBLE(arm) 274h>
<constant FN(arm) 213c>
<constant BTRUE(arm) 252d>

typedef struct Adr Adr;
typedef struct Prog Prog;
typedef struct Case Case;
typedef struct C1 C1;
typedef struct Multab Multab;
typedef struct Hintab Hintab;
typedef struct Var Var;
typedef struct Reg Reg;
typedef struct Rgn Rgn;

<struct Adr(arm) 43d>
<constant A 44a>

<constant INDEXED(arm) 211a>
<struct Prog(arm) 43a>
<constant P 193g>

<struct Case 281a>
<constant C 281b>

<struct C1 281c>

<struct Multab(arm) 316a>

<struct Hintab(arm) 317a>

<struct Var 351g>

<struct Reg 352a>

<constant R 352b>

<constant NRGN(arm) 353d>

<struct Rgn 353g>

```
extern long breakpc;
extern long nbreak;
extern Case* cases;
extern Node constnode;
extern Node fconstnode;
extern long continpc;
extern long curarg;
extern long cursafe;
extern Prog* firstp;
extern Prog* lastp;
extern long maxargsafe;
extern int mnstring;
extern Multab multab[20];
extern int hintabsize;
extern Node* nodrat;
extern Node* nodret;
extern Node* nodsafe;
extern long nrathole;
extern long nstring;
extern Prog* p;
extern long pc;
extern Node regnode;
extern char string[NSNAME];
extern Sym* symrathole;
extern Node znode;
extern Prog zprog;
extern char reg[NREG+NFREG];
extern long exregoffset;
extern long exfregoffset;
extern int suppress;
```

<function LOAD(arm) 353b>

<function STORE(arm) 353c>

<function bset(arm) 351c>

<constant CLOAD(arm) 361b>

<constant CREF(arm) 361c>

<constant LOOP(arm) 359b>

```
extern Rgn region[NRGN];
extern Rgn* rgp;
extern int nregion;
extern int nvar;
```

```
extern Bits externs;
```

```

extern Bits params;
extern Bits consts;
extern Bits addr;

extern long regbits;
extern long exregbits;

extern int change;

extern Reg* first;
extern Reg* last;
extern Reg zreg;
extern Reg* freer;
extern Var var[NVAR];
extern long* idom;
extern Reg** rpo2r;
extern long maxnr;

extern char* anames[];
extern Hintab hintab[];

/*
 * sgen.c
 */
void codgen(Node*, Node*);
void gen(Node*);
void noretval(int);
void usedset(Node*, int);
void xcom(Node*);
int bcomplex(Node*, Node*);

/*
 * cgen.c
 */
void cgen(Node*, Node*);
void cgenrel(Node*, Node*, int);
void reglcodegen(Node*, Node*, Node*);
void lcgen(Node*, Node*);
void bcgen(Node*, int);
void boolgen(Node*, int, Node*);
void sugen(Node*, Node*, long);
void layout(Node*, Node*, int, int, Node*);

/*
 * txt.c
 */
void ginit(void);
void gclean(void);
void nextpc(void);
void gargs(Node*, Node*, Node*);
void garg1(Node*, Node*, Node*, int, Node**);
Node* nodconst(long);
Node* nod32const(vlong);
Node* nodfconst(double);
void nodreg(Node*, Node*, int);
void regret(Node*, Node*);
int tmpreg(void);
void regalloc(Node*, Node*, Node*);
void regfree(Node*);
void regialloc(Node*, Node*, Node*);

```

```

void regsalloc(Node*, Node*);
void regaalloc1(Node*, Node*);
void regaalloc(Node*, Node*);
void regind(Node*, Node*);
void gprep(Node*, Node*);
void raddr(Node*, Prog*);
void naddr(Node*, Adr*);
void gmovm(Node*, Node*, int);
void gmove(Node*, Node*);
void gmover(Node*, Node*);
void gins(int a, Node*, Node*);
void gopcode(int, Node*, Node*, Node*);
int samaddr(Node*, Node*);
void gbranch(int);
void patch(Prog*, long);
int sconst(Node*);
int sval(long);
void gpseudo(int, Sym*, Node*);

/*
 * swt.c
 */
int swcmp(const void*, const void*);
void doswit(Node*);
void swit1(C1*, int, long, Node*);
void swit2(C1*, int, long, Node*, Node*);
void casf(void);
void bitload(Node*, Node*, Node*, Node*, Node*);
void bitstore(Node*, Node*, Node*, Node*, Node*);
long outstring(char*, long);
int mulcon(Node*, Node*);
Multab* mulcon0(long);
void nullwarn(Node*, Node*);
void gextern(Sym*, Node*, long, long);
void outcode(void);
void ieeeedtod(Ieee*, double);

/*
 * list
 */
void listinit(void);
int Pconv(Fmt*);
int Aconv(Fmt*);
int Dconv(Fmt*);
int Sconv(Fmt*);
int Nconv(Fmt*);
int Bconv(Fmt*);
int Rconv(Fmt*);

/*
 * reg.c
 */
Reg* rega(void);
int rcmp(const void*, const void*);
void regopt(Prog*);
void addmove(Reg*, int, int, int);
Bits mkvar(Adr*, int);
void prop(Reg*, Bits, Bits);
void loopit(Reg*, long);
void synch(Reg*, Bits);

```

```

ulong allreg(ulong, Rgn*);
void paint1(Reg*, int);
ulong paint2(Reg*, int);
void paint3(Reg*, int, long, int);
void addreg(Adr*, int);

/*
 * peep.c
 */
void peep(void);
void excise(Reg*);
Reg* uniqp(Reg*);
Reg* uniqs(Reg*);
int regtyp(Adr*);
int regzer(Adr*);
int anyvar(Adr*);
int subprop(Reg*);
int copyprop(Reg*);
int shiftprop(Reg*);
void constprop(Adr*, Adr*, Reg*);
int copy1(Adr*, Adr*, Reg*, int);
int copyu(Prog*, Adr*, Adr*);

int copyas(Adr*, Adr*);
int copyau(Adr*, Adr*);
int copyau1(Prog*, Adr*);
int copysub(Adr*, Adr*, Adr*, int);
int copysub1(Prog*, Adr*, Adr*, int);

long RtoB(int);
long FtoB(int);
int BtoR(long);
int BtoF(long);

void predicate(void);
int isbranch(Prog *);
int predicable(Prog *p);
int modifiescpsr(Prog *p);

#pragma varargck type "A" int
#pragma varargck type "B" Bits
#pragma varargck type "D" Adr*
#pragma varargck type "N" Adr*
#pragma varargck type "R" Adr*
#pragma varargck type "P" Prog*
#pragma varargck type "S" char*

```

Uses Adr 531b, C1 531b, Case 531b, Hintab 531b, Multab 531b, Prog 531b, Reg 531b, Rgn 531b, and Var 531b.

D.3.3 5c/machcap.c

```

⟨5c/machcap.c 535a⟩≡
#include "../cc/cc.h"

⟨function machcap(arm) 158c⟩

```

D.3.4 5c/globals2.c

```

⟨5c/globals2.c 535b⟩≡

```



```

#include "gc.h"

⟨global breakpc 201b⟩
⟨global nbreak 205i⟩
⟨global cases 281d⟩
⟨global constnode 195a⟩
⟨global fconstnode 452d⟩
⟨global continpc 201c⟩

⟨global curarg 258b⟩
⟨global cursafe 258e⟩

⟨global firstp 193e⟩
⟨global lastp 193h⟩

⟨global maxargsafe 258c⟩

⟨global mnstring 226g⟩
⟨global multab(arm) 316b⟩
⟨global hintabsz(arm) 327⟩
⟨global nodrat 242b⟩
⟨global nodret 272b⟩
⟨global nodsafe 260a⟩
⟨global nrathole 242g⟩
⟨global nstring 226f⟩
⟨global p 193d⟩
⟨global pc 193a⟩
⟨global regnode 209a⟩
⟨global string 226j⟩
⟨global symrathole 242c⟩
⟨global znodel 29i⟩
⟨global zprog 44d⟩
⟨global reg(arm) 207b⟩
⟨global exregoffset 432e⟩
⟨global exfregoffset 432f⟩
⟨global suppress 188a⟩

⟨global region 353e⟩
⟨global rgp 353f⟩
⟨global nregion 353h⟩

⟨global externs 340a⟩
⟨global params 340b⟩
⟨global consts 340c⟩
⟨global addrs 340d⟩

⟨global regbits 340e⟩
⟨global exregbits 340f⟩

⟨global change 340g⟩

⟨global firstr 340h⟩
⟨global lastr 340i⟩
⟨global zreg 340j⟩
⟨global freer 340k⟩
⟨global var 351f⟩
⟨global nvar 351e⟩

⟨global idom 359d⟩
⟨global rpo2r 359e⟩

```

⟨*global* maxnr 359f⟩

D.3.5 5c/list.c

⟨5c/list.c 537a⟩≡
#include "gc.h"

⟨*function* listinit(*arm*) 479c⟩

⟨*function* Bconv 479d⟩

⟨*global* extra(*arm*) 481a⟩

⟨*function* Pconv(*arm*) 480⟩

⟨*function* Aconv 481b⟩

⟨*function* Dconv(*arm*) 481c⟩

⟨*function* Rconv(*arm*) 482⟩

⟨*function* Sconv(*arm*) 483⟩

⟨*function* Nconv(*arm*) 484⟩

D.3.6 5c/txt.c

⟨5c/txt.c 537b⟩≡
#include "gc.h"

⟨*global* resvreg(*arm*) 208b⟩

⟨*function* ginit(*arm*) 51c⟩

⟨*function* gclean(*arm*) 49c⟩

⟨*function* nextpc 193c⟩

⟨*function* gargs 258g⟩

⟨*function* garg1(*arm*) 259a⟩

⟨*function* nodconst 195c⟩

⟨*function* nod32const(*arm*) 273c⟩

⟨*function* nodfconst 452f⟩

⟨*function* nodreg(*arm*) 208h⟩

⟨*function* regret(*arm*) 207a⟩

⟨*function* tmpreg(*arm*) 247b⟩

⟨*function* regalloc(*arm*) 221f⟩

⟨*function* regialloc 237a⟩

<function regfree(*arm*) 209c>
 <function regsalloc 260c>
 <function regaalloc1(*arm*) 338b>
 <function regaalloc(*arm*) 259b>
 <function regind 237b>
 <function raddr(*arm*) 453b>
 <function naddr(*arm*) 219b>
 <function gmovm(*arm*) 247a>
 <function gmove(*arm*) 214>
 <function gmover(*arm*) 248b>
 <function gins(*arm*) 219a>
 <function gopcode(*arm*) 229c>
 <function samaddr 336b>
 <function gbranch(*arm*) 196a>
 <function patch 196b>
 <function gpseudo(*arm*) 195d>
 <function sconst(*arm*) 239a>
 <function sval(*arm*) 357c>
 <function exreg(*arm*) 433b>
 <global ewidth 273d>
 <global ncast 249c>

D.3.7 5c/peep.c

<5c/peep.c 538>≡
 #include "gc.h"

 int xtramodes(Reg*, Adr*);

 <function peep(*arm*) 366c>
 <function excise(*arm*) 370a>
 <function uniqp 370b>
 <function uniqs 370c>
 <function regtyp(*arm*) 370d>

<function subprop(*arm*) 371>
 <function copyprop 373a>
 <function copy1 373b>
 <function constprop(*arm*) 374>
 <function FAIL(*arm*) 375a>
 <function shiftprop(*arm*) 375b>
 <function findpre(*arm*) 378a>
 <function findinc(*arm*) 378b>
 <function nochange(*arm*) 379a>
 <function findu1(*arm*) 379b>
 <function finduse(*arm*) 380a>
 <function xtramodes(*arm*) 380b>
 <function copyu(*arm*) 381>
 <function a2type(*arm*) 385>
 <function copyas(*arm*) 386a>
 <function copyau(*arm*) 386b>
 <function copyau1(*arm*) 386c>
 <function copysub(*arm*) 387a>
 <function copysub1(*arm*) 387b>
 <global predinfo(*arm*) 387c>
 <struct Joininfo 389b>
 <enum _anon_(*arm*) 388a>
 <enum _anon_ (5c/peep.c)(*arm*) 388b>
 <function isbranch(*arm*) 388c>
 <function predicable(*arm*) 388d>
 <function modifiescpsr(*arm*) 389a>
 <function joinsplit(*arm*) 389c>
 <function successor(*arm*) 390a>
 <function applypred(*arm*) 390b>
 <function predicate(*arm*) 391a>

D.3.8 5c/sgen.c

```
<5c/sgen.c 540a>≡  
#include "gc.h"  
  
<function noretval(arm) 341e>  
  
<function xcom(arm) 210b>
```

D.3.9 5c/swt.c

```
<5c/swt.c 540b>≡  
#include "gc.h"  
  
<function swit1(arm) 283b>  
  
<function swit2(arm) 284>  
  
<function bitload(arm) 421c>  
  
<function bitstore(arm) 422a>  
  
<function outstring(arm) 227a>  
  
<function mulcon(arm) 314b>  
  
<function gextern(arm) 270d>  
  
void zname(Biobuf*, Sym*, int);  
char* zaddr(char*, Adr*, int);  
void zwrite(Biobuf*, Prog*, int, int);  
void outhist(Biobuf*);  
  
<function zwrite(arm) 288a>  
  
<struct Htab 287a>  
  
<function outcode(arm) 287c>  
  
<function outhist(arm) 290b>  
  
<function zname(arm) 290a>  
  
<function zaddr(arm) 288b>  
  
<function align(arm) 275b>  
  
<function maxround 276a>
```

D.3.10 5c/mul.c

```
<5c/mul.c 540c>≡  
#include "gc.h"  
  
/*  
 * code sequences for multiply by constant.  
 * [a-l][0-3]  
 * lsl $(A-'a'),r0,r1
```

```

* [+] [0-7]
* add r0,r1,r2
* [-] [0-7]
* sub r0,r1,r2
*/

```

```

⟨global maxmulops(arm) 316c⟩
⟨global multabp(arm) 316d⟩
⟨global mulval(arm) 316e⟩
⟨global mulcp(arm) 316f⟩
⟨global valmax(arm) 316g⟩
⟨global shmax(arm) 316h⟩

```

```

static int docode(char *hp, char *cp, int r0, int r1);
static int gen1(int len);
static int gen2(int len, long r1);
static int gen3(int len, long r0, long r1, int flag);
⟨enum _anon_ (5c/mul.c)(arm) 316i⟩

```

```

⟨function mulcon0(arm) 317b⟩

```

```

⟨function docode(arm) 318⟩

```

```

⟨function gen1(arm) 320a⟩

```

```

⟨function gen2(arm) 320b⟩

```

```

⟨function gen3(arm) 321⟩

```

```

⟨global hintab(arm) 323⟩
⟨global hintabsize (5c/mul.c)(arm) 326⟩

```

D.3.11 5c/reg.c

```

⟨5c/reg.c 541⟩≡
#include "gc.h"

```

```

void addsplits(void);

```

```

⟨function rega 352c⟩

```

```

⟨function rcmp 353a⟩

```

```

⟨function regopt(arm) 339b⟩

```

```

⟨function addsplits(arm) 354a⟩

```

```

⟨function addmove(arm) 354b⟩

```

```

⟨function mkvar(arm) 355⟩

```

```

⟨function prop(arm) 357d⟩

```

```

⟨function postorder 391b⟩

```

```

⟨function rpolca 358a⟩

```

```

⟨function doms 358b⟩

```

⟨function loophead 359a⟩
 ⟨function loopmark 359c⟩
 ⟨function loopit 359g⟩
 ⟨function synch 360⟩
 ⟨function allreg(*arm*) 361a⟩
 ⟨function paint1(*arm*) 362⟩
 ⟨function paint2(*arm*) 363⟩
 ⟨function paint3 364⟩
 ⟨function addreg(*arm*) 365a⟩
 ⟨function RtoB(*arm*) 365b⟩
 ⟨function BtoR(*arm*) 365c⟩
 ⟨function FtoB(*arm*) 366a⟩
 ⟨function BtoF(*arm*) 366b⟩

D.3.12 5c/cgen.c

⟨5c/cgen.c 542⟩≡
 #include "gc.h"
 ⟨function cgen(*arm*) 223b⟩
 ⟨function cgenrel(*arm*) 223c⟩
 ⟨function reglcgen(*arm*) 238c⟩
 ⟨function reglpcgen(*arm*) 247c⟩
 ⟨function lcgen(*arm*) 235c⟩
 ⟨function bcgen 255⟩
 ⟨function boolgen(*arm*) 253⟩
 ⟨function sugen(*arm*) 243b⟩

Glossary

SUE = Structure/Union/Enum
SU = Structure/Union

Indexes

Here is a list of the identifiers used, and where they appear. Underlined entries indicate the place of definition. This index is generated automatically.

A: [44a](#)
a2type(): [385](#), [386c](#)
Aarg0: [197a](#), [275b](#), [276b](#), [276d](#)
Aarg1: [225a](#), [228a](#), [243b](#), [259b](#), [273b](#), [275b](#), [276b](#), [276d](#), [337b](#), [338b](#), [424c](#)
Aarg2: [225a](#), [259b](#), [275b](#), [276b](#), [276d](#), [338b](#), [424c](#)
Aaut3: [225c](#), [260c](#), [268b](#), [275b](#), [276b](#), [276d](#)
acast(): [331a](#), [332b](#)
acidchar: [404c](#), [405a](#), [405b](#)
acidcinit: [404d](#), [405a](#)
acidfun(): [401c](#), [402d](#)
acidinit(): [405a](#), [405b](#)
acidmember(): [403b](#), [405b](#), [405b](#)
acidsue(): [401c](#), [402c](#), [403b](#), [405b](#)
acidtype(): [277d](#), [403b](#)
acidvar(): [401a](#), [401b](#), [401c](#)
acom(): [210a](#), [328e](#), [328e](#), [332b](#), [431](#)
acom1(): [328e](#), [330](#), [330](#)
acom2(): [328e](#), [332b](#)
acomcmp1(): [331b](#), [332b](#)
acomcmp2(): [332a](#), [332b](#)
Aconv(): [481b](#)
add(): [300c](#), [300f](#)
addmove(): [354b](#)
addo(): [328e](#), [329](#), [330](#)
addreg(): [365a](#)
ADDROF-92: [152b](#), [153a](#), [154d](#), [175d](#)
ADDRROP-94: [153a](#), [154d](#), [170a](#)
addrs: [340d](#)
addsplits(): [345b](#), [354a](#)
adecl(): [139c](#), [140c](#)
Adr: [531b](#), [531b](#)
Adr (typedef): [531b](#)
Ael1: [275b](#), [276b](#), [276d](#), [277d](#)
Ael2: [275b](#), [276b](#), [276d](#), [277d](#)
align: [275b](#), [276b](#)
align(): [275b](#), [276b](#)
allfloat(): [450d](#), [450e](#), [450e](#)
alloc(): [29d](#), [32a](#), [36a](#), [39e](#), [53e](#), [74c](#), [74d](#), [80a](#), [81d](#), [193c](#), [281h](#), [283a](#), [352c](#), [354b](#), [359g](#), [393b](#), [434i](#), [449a](#),

[449b](#), [467c](#), [500b](#), [500d](#)
 allocn(): [51b](#), [74c](#), [74d](#), [87c](#), [88b](#), [88c](#), [88d](#), [89a](#), [89b](#), [500c](#)
 allreg(): [361a](#)
 amap(): [401c](#), [403a](#), [403b](#), [405b](#)
 anames: [492e](#)
 ANSIPROTO-90: [423e](#), [424a](#), [424b](#)
 anyproto(): [423b](#), [424a](#), [424b](#), [424b](#)
 applypred(): [390b](#), [391a](#)
 argflag(): [444a](#), [444b](#), [445a](#), [448b](#)
 arginit(): [445a](#)
 argmark(): [138a](#)
 arith(): [156b](#), [156c](#), [157a](#), [157d](#), [158a](#), [158b](#), [159a](#), [159d](#), [160a](#), [164a](#), [164d](#), [165c](#), [167c](#), [168f](#), [169b](#), [176a](#)
 Asu2: [275b](#), [276b](#), [276d](#), [277d](#)
 Atab: [497e](#), [497f](#)
 Atab.idel: [497e](#)
 Atab.incl: [497e](#)
 Atab.ldel: [497e](#)
 Atab.line: [497e](#)
 autobn: [39h](#), [39i](#), [40a](#), [41a](#), [129a](#), [140c](#), [145a](#)
 autoffset: [40a](#), [41a](#), [196d](#), [196e](#), [197a](#), [197b](#), [225a](#), [225c](#), [268b](#), [274i](#), [424c](#)
 Axxx: [276b](#), [276d](#)
 bany(): [341f](#), [345b](#), [346](#), [348b](#), [350c](#), [354a](#), [479d](#)
 BARRAY: [121](#)
 BAUTO: [121](#), [124c](#), [126c](#), [126d](#)
 bcgen(): [255](#)
 BCHAR: [121](#), [124e](#), [151b](#), [249c](#)
 BCLASS: [124c](#), [124e](#), [126c](#)
 bcomplex(): [198c](#), [199e](#), [201f](#), [202](#)
 BCONSTNT: [121](#), [124d](#), [127e](#)
 Bconv(): [479d](#)
 BDOT: [121](#)
 BDOUBLE: [121](#), [124e](#), [151b](#), [249c](#)
 BENUM: [121](#)
 beq(): [350d](#), [446](#), [449b](#)
 BEXTERN: [121](#), [124c](#), [126c](#), [432c](#)
 BFLOAT: [121](#), [124e](#), [151b](#), [249c](#)
 BFUNC: [121](#), [172c](#)
 BGARB: [124d](#), [124e](#), [127c](#), [127e](#)
 Big: [299b](#), [529](#)
 big(): [300a](#), [300f](#)
 Big.a: [299b](#)
 Big.b: [299b](#)
 Big (typedef): [529](#)
 BIND: [121](#), [151a](#), [151d](#), [156e](#), [164c](#), [165b](#), [166d](#), [170c](#), [173b](#), [175f](#), [249c](#)
 BINT: [121](#), [124e](#), [151b](#), [249c](#)
 BINTEGER: [151b](#), [151b](#), [151d](#), [156e](#), [159c](#), [168e](#), [175f](#)
 bitload(): [421c](#)
 bitno(): [350e](#), [361a](#), [365c](#), [366b](#), [478b](#), [501d](#)
 BITS: [341a](#), [341f](#), [345b](#), [346](#), [349b](#), [350b](#), [350c](#), [350d](#), [350e](#), [354a](#), [355](#), [357d](#), [360](#)

Bits: [504b](#), [504b](#)
 Bits (typedef): [504b](#)
 bitstore(): [422a](#)
 blockno: [39g](#), [39i](#), [40a](#)
 BLONG: [121](#), [123c](#), [124e](#), [151b](#), [249c](#)
 blsh(): [346](#), [347](#), [351a](#), [355](#), [362](#), [444b](#)
 bnames: [275b](#), [276c](#), [486c](#)
 bnamesinit: [276d](#), [486c](#)
 bnum(): [346](#), [350e](#), [354a](#), [479d](#)
 BNUMBER: [151b](#), [151d](#), [156e](#), [157c](#), [159c](#), [164c](#), [165b](#), [166d](#), [168e](#), [173b](#), [175f](#)
 bool64(): [466a](#), [466b](#)
 boolgen(): [253](#)
 bor(): [350b](#), [444b](#)
 Branch-198: [388a](#), [389c](#), [391a](#)
 breakpc: [201b](#)
 BREGISTER: [121](#), [124c](#), [126d](#), [432c](#)
 bset: [351b](#), [351c](#)
 bset(): [351b](#), [351c](#)
 BSHORT: [121](#), [124e](#), [151b](#), [249c](#)
 BSIGNED: [121](#), [124e](#)
 BSTATIC: [121](#), [124c](#), [126c](#)
 BSTRUCT: [121](#), [166d](#), [167a](#), [171a](#), [173b](#), [175f](#), [249c](#)
 BtoF(): [361a](#), [366b](#)
 BtoR(): [365c](#)
 BTRUE: [252d](#), [252f](#), [253](#)
 BTYPEDEF: [121](#), [124c](#), [137h](#)
 BTYPESTR: [124c](#), [434d](#), [434g](#)
 BUCHAR: [121](#), [151b](#), [249c](#)
 BUFSIZ: [54a](#), [55a](#)
 BUINT: [121](#), [151b](#), [249c](#)
 BULONG: [121](#), [151b](#), [249c](#)
 BUNION: [121](#), [166d](#), [167a](#), [171a](#), [173b](#), [175f](#), [249c](#)
 BUNSIGNED: [121](#), [124e](#)
 BUSHORT: [121](#), [151b](#), [249c](#)
 BUVLONG: [121](#), [151b](#), [249c](#)
 BVLONG: [121](#), [123c](#), [124e](#), [151b](#), [249c](#)
 BVOID: [121](#), [124e](#), [175f](#)
 BVOLATILE: [121](#), [124d](#), [127e](#)
 Bxxx: [121](#)
 C: [281b](#)
 C1: [531b](#), [531b](#)
 C1 (typedef): [531b](#)
 canreach: [185d](#), [185f](#), [185g](#), [187c](#), [188c](#), [199b](#), [199c](#), [199d](#), [200b](#), [200d](#), [202](#), [203a](#), [203b](#), [203c](#), [203e](#), [205f](#), [205j](#),
 [279b](#), [280c](#)
 Case: [531b](#), [531b](#)
 Case (typedef): [531b](#)
 cases: [281d](#)
 casf(): [279b](#), [280c](#), [283a](#)
 CASTOF-93: [153a](#), [175d](#)

CAUTO: [33b](#), [126c](#), [126d](#), [143e](#), [144d](#), [144e](#), [145a](#), [183e](#), [225c](#), [225d](#), [260b](#), [262d](#), [401c](#), [485b](#)
ccom(): [190a](#), [210a](#), [292](#), [292](#), [293c](#), [293d](#), [294a](#), [294b](#), [295a](#), [296a](#), [296b](#), [297a](#), [297b](#), [298a](#), [303a](#), [303b](#), [303c](#),
[431](#)
CEXREG: [144d](#), [209b](#), [432a](#), [432c](#), [432d](#), [433c](#), [485b](#)
CEXTERN: [33b](#), [126c](#), [143c](#), [143d](#), [143e](#), [144a](#), [144b](#), [144d](#), [145a](#), [225e](#), [401c](#), [432d](#), [434i](#), [485b](#)
cgen(): [223b](#)
cgenrel(): [223b](#), [223c](#), [253](#)
CGLOBL: [33b](#), [143c](#), [143d](#), [144a](#), [144b](#), [145a](#), [225e](#), [242e](#), [270c](#), [401c](#), [432d](#), [459b](#), [485b](#)
change: [340g](#)
checkargs(): [446](#), [447b](#)
cinit(): [50d](#)
CLOAD: [361b](#)
CLOCAL: [144d](#), [145a](#), [401c](#), [415b](#), [415c](#), [415d](#), [415g](#), [485b](#)
cmp(): [300b](#), [300f](#)
cmps-95: [300e](#), [300f](#)
cnames: [143e](#), [144a](#), [144d](#), [145b](#), [485a](#), [486c](#), [490e](#)
cnamesinit: [485b](#), [486c](#)
codgen(): [194c](#)
Com: [308a](#), [529](#)
Com.n: [308a](#)
Com.t: [308a](#)
com64(): [459c](#), [459d](#)
com64init(): [457a](#), [457b](#)
Com (typedef): [529](#)
comargs(): [309](#), [310a](#), [310a](#)
comma(): [210a](#), [308b](#), [309](#)
commas(): [308b](#), [309](#), [309](#), [310a](#)
compar(): [298a](#), [300f](#)
compile(): [47h](#)
complex(): [180f](#), [181d](#), [184f](#), [199e](#), [206b](#), [210a](#), [223a](#), [242e](#), [260b](#), [262d](#), [272c](#), [272d](#), [279b](#), [280c](#), [418f](#)
comrel: [253](#), [256a](#), [300f](#)
constas(): [190b](#), [190c](#), [190d](#), [191](#), [430c](#)
constnode: [195a](#)
constprop(): [366c](#), [374](#), [374](#)
consts: [340c](#)
contig(): [268b](#)
continpc: [201c](#)
convvtox(): [65c](#), [65d](#), [68c](#), [68f](#), [69a](#), [70](#), [73](#), [178a](#), [181a](#), [267b](#), [298b](#), [304a](#), [304d](#), [314b](#), [442f](#)
copy1(): [373b](#)
copyas(): [386a](#)
copyau(): [386b](#)
copyau1(): [371](#), [381](#), [386c](#), [387b](#)
copyprop(): [373a](#)
copysub(): [387a](#)
copysub1(): [371](#), [381](#), [387b](#)
copytyp(): [127c](#), [127d](#), [132c](#), [182f](#), [423d](#), [434i](#)
copyu(): [381](#)
CPARAM: [33b](#), [138b](#), [140c](#), [144c](#), [145a](#), [182b](#), [183e](#), [225a](#), [225b](#), [272c](#), [337b](#), [401c](#), [423c](#), [485b](#)
CREF: [361c](#)

CSTATIC: [33b](#), [126c](#), [144a](#), [144d](#), [145a](#), [195d](#), [225f](#), [226e](#), [270c](#), [401c](#), [415c](#), [415f](#), [485b](#)
CTYPEDEF: [62a](#), [137h](#), [137i](#), [172a](#), [485b](#)
CTYPESTR: [62a](#), [172a](#), [434f](#), [434g](#), [434h](#), [485b](#)
curarg: [258b](#)
cursafe: [258e](#)
CXXX: [33b](#), [37h](#), [126c](#), [126d](#), [132c](#), [134c](#), [136h](#), [143c](#), [144c](#), [144d](#), [144e](#), [195b](#), [424c](#), [427d](#), [452e](#), [485b](#)
D: [39c](#), [39d](#), [40e](#)
DAUTO: [38a](#), [39f](#), [41d](#)
dbgdecl(): [490a](#), [490b](#), [490c](#), [490d](#), [490e](#)
dclfunct(): [434h](#), [434i](#)
dcllabel(): [109a](#)
dclstack: [39a](#), [39d](#), [39e](#), [40d](#), [109a](#), [134c](#)
Dconv(): [481c](#)
deadhead(): [186b](#), [186b](#), [187a](#)
deadheads(): [186a](#), [187a](#), [203b](#)
debug: [47h](#), [48d](#), [142](#), [143a](#), [143b](#), [147c](#), [187c](#), [188c](#), [275b](#), [281h](#), [284](#), [300f](#), [308b](#), [314b](#), [328e](#), [332b](#), [339a](#), [344a](#),
[345b](#), [346](#), [347](#), [348a](#), [348b](#), [354b](#), [355](#), [362](#), [364](#), [371](#), [373b](#), [374](#), [375b](#), [380b](#), [381](#), [395a](#), [398b](#), [400a](#), [400b](#),
[402a](#), [402b](#), [403b](#), [404a](#), [404b](#), [407a](#), [410](#), [413](#), [425c](#), [446](#), [448b](#), [467c](#), [469d](#), [474a](#), [489a](#), [489b](#), [489c](#), [489d](#), [489e](#),
[489f](#), [490a](#), [490b](#), [490c](#), [490d](#), [490f](#), [490g](#), [490h](#), [490i](#), [491a](#), [491b](#), [491c](#), [491d](#), [491e](#), [491f](#), [491g](#), [491h](#), [492a](#),
[492b](#), [492c](#), [492d](#), [494a](#), [494b](#), [494c](#), [497a](#), [497b](#), [511c](#), [515](#)
Decl: [504b](#), [504b](#)
Decl (typedef): [504b](#)
Delbranch-203: [388b](#), [390b](#), [391a](#)
diag(): [40e](#), [48a](#), [72a](#), [103a](#), [124e](#), [126c](#), [129b](#), [132c](#), [134c](#), [136b](#), [136e](#), [138b](#), [139d](#), [143a](#), [143e](#), [144c](#), [144d](#),
[145a](#), [145b](#), [150a](#), [150b](#), [153b](#), [153c](#), [154a](#), [155e](#), [160a](#), [166b](#), [171b](#), [171c](#), [173a](#), [174b](#), [174c](#), [177](#), [178b](#), [179c](#),
[181d](#), [182d](#), [185b](#), [188c](#), [194e](#), [196a](#), [201a](#), [204b](#), [204d](#), [208e](#), [209c](#), [215a](#), [219b](#), [222a](#), [222b](#), [223c](#), [230a](#), [233a](#),
[236a](#), [237b](#), [242a](#), [243b](#), [247b](#), [260e](#), [262c](#), [262d](#), [268b](#), [275b](#), [277d](#), [280c](#), [281h](#), [288b](#), [297a](#), [314b](#), [330](#), [341f](#),
[344a](#), [350e](#), [361a](#), [418f](#), [419f](#), [419g](#), [419h](#), [424a](#), [425b](#), [425c](#), [427d](#), [428b](#), [430c](#), [433e](#), [434h](#), [434i](#), [438d](#), [442g](#),
[442h](#), [453a](#), [453b](#), [456](#), [459d](#), [467c](#), [496b](#), [501d](#), [502a](#)
diagbuf: [47f](#), [47h](#), [142](#), [145b](#), [495c](#), [496b](#), [496c](#), [497a](#)
DLABEL: [38a](#), [41c](#), [109a](#)
DMARK: [38a](#), [40a](#), [41a](#)
docode(): [317b](#), [318](#), [318](#)
dodecl(): [103a](#), [136h](#), [138b](#), [423c](#), [424c](#), [434i](#)
doenum(): [134c](#)
doinit(): [120c](#), [262d](#)
domacro(): [76a](#), [76c](#)
doms(): [358b](#)
doswit(): [279b](#), [281h](#)
dotag(): [129a](#), [398b](#)
dotoffset(): [428a](#), [428b](#), [428b](#)
dotsearch(): [171b](#), [171c](#), [427e](#)
dpcheck(): [447a](#), [447b](#)
DSUE: [38a](#), [41b](#), [130b](#)
edecl(): [132c](#)
ehist: [81a](#), [81d](#), [393b](#), [470a](#)
En: [133a](#), [133b](#)
en: [133b](#), [134c](#), [180g](#), [181a](#)
En.cenum: [133a](#)

En.floatenum: [180e](#)
 En.lastenum: [180e](#)
 En.tenum: [133a](#)
 End-197: [388a](#), [389c](#), [391a](#)
 EOF-81: [54f](#), [55a](#), [55b](#), [58d](#), [58e](#), [70](#), [71a](#), [73](#), [74c](#), [74d](#), [78d](#)
 errexit(): [48a](#), [53f](#), [53g](#), [58e](#), [78d](#), [83b](#), [153c](#), [467c](#), [495b](#), [495c](#), [496b](#), [496c](#), [499f](#)
 ESC-83: [71b](#), [72b](#), [74a](#), [74c](#)
 escchar(): [70](#), [71a](#), [73](#), [74c](#), [74d](#)
 etconv: [457b](#), [458a](#), [459d](#)
 evconst(): [292](#), [293c](#), [304a](#)
 ewidth: [273d](#)
 excise(): [370a](#)
 exfregoffset: [432f](#)
 exreg(): [433b](#)
 exregbits: [340f](#)
 exregoffset: [432e](#)
 externs: [340a](#)
 extra: [480](#), [481a](#)
 Fadj-183: [443a](#), [444a](#), [445a](#)
 FAIL-194: [375a](#), [375b](#)
 Falsecond-201: [388b](#), [391a](#)
 fatal(): [281h](#), [308b](#), [309](#), [358a](#), [359g](#), [496c](#)
 fconstnode: [452d](#)
 Fi: [54c](#), [54d](#)
 fi: [53g](#), [54d](#), [55a](#), [55b](#), [83b](#), [87c](#), [90a](#), [90b](#)
 Fi.c: [54c](#)
 Fi.p: [54c](#)
 Fignor-181: [443a](#), [444a](#), [444b](#), [445a](#), [448b](#)
 filbuf(): [55a](#), [87c](#)
 findinc(): [378b](#), [380b](#)
 findpre(): [378a](#), [380b](#)
 findu1(): [379b](#), [379b](#), [380a](#)
 finduse(): [380a](#), [380b](#)
 firstarg: [274i](#), [274j](#), [337b](#), [424c](#)
 firstargtype: [274i](#), [275a](#), [337b](#), [424c](#)
 firstbit: [417a](#), [417c](#), [418f](#), [419a](#)
 firstdcl: [105a](#), [109a](#)
 firstp: [193e](#)
 firstr: [340h](#)
 Fl-179: [443a](#), [444a](#), [444b](#), [445a](#)
 flagbits-186: [443e](#), [444a](#), [444b](#), [445a](#)
 fmtbuf-187: [443f](#), [444b](#), [446](#), [448b](#)
 FNconv(): [474b](#), [478a](#)
 Fnone-178: [443a](#), [444b](#), [445a](#)
 fnproto(): [135i](#), [136a](#)
 fnproto1(): [136a](#), [136b](#), [137b](#)
 fntypes: [459a](#), [459b](#)
 FNX: [213c](#)
 FNX-177: [213d](#), [459d](#), [466b](#)

fproundflg: [511a](#), [511c](#)
 free(): [395c](#), [397c](#), [501a](#), [513b](#)
 freer: [340k](#)
 Fstar-182: [443a](#), [444a](#), [444b](#), [445a](#)
 Ftab: [436a](#), [527](#)
 Ftab.name: [436a](#)
 Ftab.op: [436a](#)
 Ftab.typ: [436a](#)
 Ftab (typedef): [527](#)
 ftabinit: [434i](#), [437](#)
 FtoB(): [355](#), [361a](#), [366a](#)
 Funct: [504b](#), [504b](#)
 Funct (typedef): [504b](#)
 Fverb-184: [443a](#), [444a](#), [444b](#), [445a](#)
 Fvl-180: [443a](#), [444a](#), [444b](#)
 fvn(): [457b](#), [459b](#)
 fxxx: [443a](#)
 garbt(): [127c](#)
 garg1(): [259a](#)
 gargs(): [258g](#)
 gbranch(): [196a](#)
 gclean(): [49c](#)
 GCONSTNT: [33a](#), [127e](#), [190d](#), [486b](#)
 gen(): [187c](#), [188b](#), [194c](#), [197e](#), [198a](#), [198c](#), [201f](#), [202](#), [279b](#)
 gen1(): [317b](#), [320a](#)
 gen2(): [320a](#), [320b](#)
 gen3(): [320b](#), [321](#), [321](#)
 getc(): [71b](#), [72a](#), [72b](#), [77b](#), [78d](#), [82b](#), [83c](#), [84a](#), [84b](#), [86d](#), [87a](#), [88b](#), [88c](#), [88d](#), [89a](#), [91b](#), [92a](#), [92b](#), [93a](#), [95c](#), [96b](#),
[97](#), [393b](#), [448b](#)
 GETC-84: [54e](#), [57a](#), [58e](#), [60g](#), [61b](#), [62b](#), [62c](#), [62d](#), [62e](#), [63a](#), [63b](#), [63c](#), [63d](#), [63e](#), [64a](#), [64b](#), [64e](#), [67](#), [68b](#), [68e](#), [69c](#),
[69d](#), [73](#), [77c](#), [78d](#)
 getcom(): [82b](#), [83c](#), [95a](#), [96b](#), [393b](#)
 getflag(): [444b](#), [446](#), [449b](#)
 gethunk(): [81f](#), [84c](#), [96b](#), [393b](#), [499f](#), [500b](#), [500c](#)
 getnsc(): [77b](#), [77c](#), [78c](#), [82b](#), [83c](#), [86d](#), [87a](#), [90c](#), [96a](#), [97](#), [393b](#), [398b](#), [412e](#), [448b](#), [469c](#), [511c](#)
 getnsn(): [96b](#), [97](#), [448b](#)
 getr(): [58e](#), [71a](#), [72a](#), [448b](#)
 getsym(): [76c](#), [77b](#), [86b](#), [87a](#), [94a](#), [95a](#), [95c](#), [96a](#), [398b](#), [412e](#), [448b](#), [469c](#), [511c](#)
 getsymdots(): [86d](#), [87a](#)
 gextern(): [270d](#)
 GINCOMPLETE: [396a](#), [398b](#), [399](#), [476](#)
 ginit(): [51c](#)
 gins(): [219a](#)
 gmove(): [214](#)
 gmover(): [248a](#), [248b](#)
 gmovm(): [243b](#), [247a](#)
 gnames: [476](#), [486a](#), [486c](#)
 gnamesinit: [486b](#), [486c](#)
 gopcode(): [229c](#)

gpseudo(): [195d](#)
 Gtab: [436b](#), [527](#)
 Gtab.etype: [436b](#)
 Gtab.name: [436b](#)
 Gtab (typedef): [527](#)
 gtabinit: [434i](#), [436c](#)
 GVOLATILE: [33a](#), [127e](#), [416b](#), [486b](#)
 GXXX: [32a](#), [33a](#), [127e](#), [486b](#)
 H: [80c](#), [81d](#), [290b](#), [393b](#), [470a](#), [497f](#)
 hash: [35a](#), [35e](#), [36a](#), [270b](#), [402c](#), [402d](#), [408a](#)
 Hintab: [531b](#), [531b](#)
 hintab: [317b](#), [323](#), [326](#)
 Hintab (typedef): [531b](#)
 hintabsize: [327](#)
 Hist: [504b](#), [504b](#)
 hist: [80e](#), [81d](#), [290b](#), [393b](#), [497f](#)
 Hist (typedef): [504b](#)
 HISTSZ: [497d](#), [497f](#)
 Htab: [287a](#), [287c](#)
 Htab.sym: [287a](#)
 Htab.type: [287a](#)
 hunk: [81f](#), [84c](#), [87c](#), [96b](#), [393b](#), [499b](#), [499f](#), [500b](#), [500c](#)
 I: [52e](#), [53d](#), [53e](#), [55a](#), [55b](#), [83b](#)
 idom: [359d](#)
 ieeeedtod(): [288b](#), [452g](#), [452g](#)
 IGN-82: [57d](#), [57e](#), [58a](#), [78a](#), [79a](#), [90b](#)
 include: [48d](#), [79b](#), [79e](#), [80a](#), [83a](#), [467c](#)
 indchar-185: [443d](#), [445a](#), [447b](#)
 INDEXED: [211a](#)
 Init: [504b](#), [504b](#)
 init1(): [120c](#), [262d](#), [262d](#)
 Init (typedef): [504b](#)
 initetconv: [457b](#), [458b](#)
 initlist: [120b](#), [120c](#), [262c](#), [267a](#), [267b](#)
 invert(): [106a](#), [430c](#)
 invrel: [253](#), [256b](#)
 Io: [504b](#), [504b](#)
 Io (typedef): [504b](#)
 iofree: [53c](#), [53d](#), [53e](#), [55b](#)
 ionext: [52f](#), [53b](#), [53g](#), [90a](#)
 iostack: [52c](#), [53d](#), [53g](#), [55a](#), [55b](#), [83b](#), [90a](#), [402b](#), [404b](#), [410](#)
 isbranch(): [388c](#), [388d](#), [391a](#)
 isfunct(): [438a](#), [438d](#), [438d](#)
 isstruct(): [262d](#), [266b](#)
 itab: [59](#), [60b](#)
 Join-195: [388a](#), [389c](#), [391a](#)
 Joininfo: [389b](#), [389b](#)
 Joininfo.end: [389b](#)
 Joininfo.last: [389b](#)

Joininfo.len: [389b](#)
 Joininfo.start: [389b](#)
 Joininfo (typedef): [389b](#)
 joinsplit(): [389c](#), [391a](#)
 Keepbranch-204: [388b](#), [391a](#)
 kwd-78: [407b](#), [407d](#)
 kwd-80: [402e](#), [403a](#)
 LANDAND: [63e](#)
 LANDE: [63e](#)
 lastadj-188: [443g](#), [444a](#), [445a](#)
 lastbit: [417b](#), [418f](#), [419a](#)
 lastclass: [102b](#)
 lastdcltype: [103a](#), [103b](#), [136h](#), [182a](#), [423c](#)
 lastfield: [418b](#), [418c](#), [418f](#), [419a](#)
 lastp: [193h](#)
 lastr: [340i](#)
 lasttype: [102c](#)
 lastverb-189: [443h](#), [444a](#), [445a](#)
 LAUTO: [59](#)
 LBREAK: [59](#)
 LCASE: [59](#)
 lcgen(): [235c](#)
 LCHAR: [59](#)
 LCONST: [65b](#), [70](#)
 LCONSTNT: [59](#)
 LCONTINUE: [59](#)
 Lconv(): [497c](#), [497f](#)
 LDCONST: [69d](#)
 LDEFAULT: [59](#)
 LDO: [59](#)
 LDOUBLE: [59](#)
 LDVE: [58e](#)
 LELSE: [59](#)
 LENUM: [59](#)
 LEXTERN: [59](#)
 LFCONST: [69d](#)
 LFLOAT: [59](#)
 LFOR: [59](#)
 LGE: [63a](#)
 LGOTO: [59](#)
 LIF: [59](#)
 linehist(): [81b](#), [81c](#), [81d](#), [96b](#)
 lineno: [29e](#), [29f](#), [52a](#), [57b](#), [65c](#), [70](#), [72a](#), [77c](#), [78b](#), [78d](#), [81d](#), [87c](#), [103a](#), [104d](#), [393b](#), [413](#), [469d](#), [470a](#), [489d](#),
[489e](#), [489f](#), [495c](#), [511c](#)
 LINLINE: [59](#)
 LINT: [59](#)
 listinit(): [479c](#)
 LLCONST: [69a](#)
 LLE: [63b](#)

LLONG: 59
LLSH: 63b
LLSHE: 63b
LLSTRING: 74d
LMDE: 62c
LME: 62e
LMG: 62e
LMLE: 62b
LMM: 62e
LNAME: 37h, 398b
LNE: 63d
LOAD: 353b
log2(): 310b, 502b
logrel: 232h, 232i
LOOP: 359b
loophead(): 359a
loopit(): 359g
loopmark(): 359c
LORE: 64a
LOROR: 64a
LPE: 62d
LPP: 62d
LREGISTER: 59
LRESTRICT: 59
LRETURN: 59
LRSH: 63a
LRSHE: 63a
LSET: 184b
LSHORT: 59
LSIGNED: 59
LSIGNOF: 442c
LSIZEOF: 59
LSTATIC: 59
LSTRING: 74c
LSTRUCT: 59, 398b
LSWITCH: 59
LTYPE: 62a, 398b
LTYPEDEF: 59
LTYPESTR: 434a
LUCONST: 68c, 73
LULCONST: 69a
LUNION: 59, 398b
LUNSIGNED: 59
LUSED: 184b
LUVLCONST: 68f
LVLCONST: 68f
LVOID: 59
LVOLATILE: 59
LWHILE: 59

LXORE: [64b](#)
 macdef(): [76b](#), [85b](#)
 macend(): [76b](#), [76c](#), [78c](#), [81f](#), [85b](#), [94a](#), [94b](#), [95c](#), [96b](#), [393b](#)
 macexpand(): [90a](#), [90c](#)
 machcap(): [158c](#)
 macif(): [76c](#), [94b](#)
 macinc(): [76b](#), [81f](#)
 maclin(): [76b](#), [96b](#)
 macprag(): [76b](#), [96a](#)
 mactab: [76b](#), [76c](#)
 macund(): [76b](#), [94a](#)
 makedot(): [241a](#), [241b](#), [428a](#)
 malloc(): [397c](#), [500d](#)
 markdcl(): [40a](#)
 MASK: [65d](#), [273c](#), [501f](#), [502b](#)
 MAXALIGN: [74c](#), [74d](#), [500a](#), [500b](#)
 maxargsafe: [258c](#)
 maxinclude: [79d](#), [80a](#)
 maxmulops-205: [316c](#), [316c](#), [317b](#)
 maxnr: [359f](#)
 maxround(): [276a](#)
 maxtype(): [134c](#), [181b](#)
 mixedasop(): [168a](#), [168c](#)
 mkstatic(): [415d](#), [415f](#)
 mkvar(): [355](#)
 mnstring: [226g](#)
 modifiescpsr(): [389a](#), [389c](#)
 mpatov(): [65b](#), [66](#)
 mulcon(): [314a](#), [314b](#)
 mulcon0(): [314b](#), [317b](#), [317b](#)
 mulcp-208: [316f](#), [317b](#), [320a](#), [320b](#), [321](#)
 Multab: [531b](#), [531b](#)
 multab: [316b](#), [317b](#)
 Multab (typedef): [531b](#)
 multabp-206: [316d](#), [317b](#)
 mulval-207: [316e](#), [317b](#), [318](#), [320a](#), [320b](#), [321](#)
 myaccess(): [467c](#), [513b](#)
 mycreat(): [47h](#), [513b](#)
 mywait(): [513b](#)
 naddr(): [219b](#)
 NALIGN: [276b](#), [276c](#)
 NALLTYPES: [120e](#), [396e](#), [477a](#), [485c](#)
 Namespace: [38a](#)
 NARG-77: [85a](#), [85b](#), [86d](#), [91a](#)
 nbreak: [205i](#)
 ncast: [249c](#)
 ncontin: [203d](#), [204a](#), [205a](#), [205d](#), [205g](#), [205h](#)
 Nconv(): [479c](#), [484](#)
 NCTYPES: [33b](#), [485a](#)

nearln: [29g](#), [29h](#), [65c](#), [70](#), [72a](#), [103a](#), [142](#), [145b](#), [183e](#), [187c](#), [193c](#), [194c](#), [197e](#), [210a](#), [344a](#), [346](#), [491e](#), [496b](#), [496c](#)
nerrors: [47h](#), [48b](#), [425c](#), [495a](#), [495c](#), [496b](#), [496c](#)
new(): [29d](#), [29h](#), [109a](#), [158a](#), [158b](#), [165c](#), [175c](#), [242e](#), [243b](#), [260b](#), [262d](#), [266a](#), [266b](#), [267b](#), [268b](#), [272c](#), [272d](#),
[332b](#), [416b](#), [419i](#), [434i](#), [438d](#), [459b](#), [459d](#), [466b](#)
new1(): [29h](#), [154e](#), [159d](#), [162c](#), [162d](#), [162e](#), [163b](#), [163c](#), [166a](#), [167b](#), [168a](#), [169a](#), [173a](#), [179a](#), [241b](#), [243b](#), [308b](#),
[331a](#), [332b](#), [414a](#), [428a](#), [430c](#)
newfile(): [47h](#), [53g](#), [81f](#), [467c](#)
newflag: [470a](#), [470b](#), [470c](#)
newio(): [53a](#), [53b](#), [81f](#), [90a](#)
newlist(): [262d](#), [266a](#)
newname(): [448b](#), [449a](#)
newprot(): [448b](#), [449b](#)
nextarg(): [445b](#), [446](#), [447b](#)
nextinit(): [262d](#), [267b](#)
nextpc(): [193c](#)
NGTYPES: [33a](#), [486a](#)
NHASH: [35a](#), [35b](#), [35e](#), [36a](#), [270b](#)
NHUNK: [499a](#), [499f](#)
nhunk: [81f](#), [84c](#), [96b](#), [393b](#), [499c](#), [499e](#), [499f](#), [500b](#), [500c](#)
nilcast(): [298b](#), [299a](#), [329](#)
ninclude: [79c](#), [79e](#), [83a](#), [467c](#)
nocast(): [249a](#), [249b](#), [293c](#), [331a](#)
nochange(): [379a](#), [380b](#)
nod32const(): [243b](#), [273b](#), [273c](#)
nodaddv: [457b](#), [459d](#)
nodandv: [457b](#), [459d](#)
nodcast: [262a](#), [262d](#)
nodcomv: [457b](#), [459d](#)
nodconst(): [195c](#)
nodd2v: [457b](#), [459d](#)
noddivv: [457b](#), [459d](#)
noddivvu: [457b](#), [459d](#)
Node: [504b](#), [504b](#)
Node (typedef): [504b](#)
NODECL: [118d](#), [118e](#), [136h](#), [423c](#)
nodeqv: [457b](#), [459d](#)
Node_kind: [27](#)
nodf2v: [457b](#), [459d](#)
nodfconst(): [452f](#)
nodgev: [457b](#), [459d](#)
nodgtv: [457b](#), [459d](#)
nodhiv: [457b](#), [459d](#)
nodhsv: [457b](#), [459d](#)
nodlev: [457b](#), [459d](#)
nodlov: [457b](#), [459d](#)
nodlshv: [457b](#), [459d](#)
nodlsv: [457b](#), [459d](#)
nodltv: [457b](#), [459d](#)
nodmmv: [457b](#), [459d](#)

nodmodv: 457b, 459d
 nodmodvu: 457b, 459d
 nodmulv: 457b, 459d
 nodnegv: 457b, 459d
 nodnev: 457b, 459d
 nodorv: 457b, 459d
 nodp2v: 457b, 459d
 nodppv: 457b, 459d
 nodproto: 173a, 175b, 175c
 nodrat: 242b
 nodreg(): 208h
 nodret: 272b
 nodrshav: 457b, 459d
 nodrshlv: 457b, 459d
 nodsafv: 260a
 nodsc2v: 457b, 459d
 nodsh2v: 457b, 459d
 nodsi2v: 457b, 459d
 nodsl2v: 457b, 459d
 nodsubv: 457b, 459d
 nodtestv: 457b, 459d, 466b
 noduc2v: 457b, 459d
 noduh2v: 457b, 459d
 nodui2v: 457b, 459d
 nodul2v: 457b, 459d
 nodv2d: 457b, 459d
 nodv2f: 457b, 459d
 nodv2sc: 457b, 459d
 nodv2sh: 457b, 459d
 nodv2si: 457b, 459d
 nodv2sl: 457b, 459d
 nodv2uc: 457b, 459d
 nodv2uh: 457b, 459d
 nodv2ui: 457b, 459d
 nodv2ul: 457b, 459d
 nodvasop: 457b, 459d
 nodvmm: 457b, 459d
 nodvpp: 457b, 459d
 nodxorv: 457b, 459d
 noretval(): 341e
 nrathole: 242g
 nregion: 353h
 NRGN: 353d
 nstar-190: 443i, 444b, 446
 nstring: 226f
 NSYMB: 35g, 35h, 77b, 415f
 NTERM: 328a, 328c, 328e, 330, 332b
 nterm: 328d, 328e, 330, 332b
 NTYPE: 30a, 32b, 126a, 148c, 149a, 149c, 151c, 154b, 156d, 157b, 159b, 161a, 164b, 166c, 168d, 175e, 180b,

215e, 215g, 231h, 249c, 271c, 273d, 280a, 357a, 404c, 408b, 436c, 458a, 459a, 517a, 517b
 nullwarn(): 189a, 189b, 189c, 243b
 Numdec-85: 64d, 64e
 Numflt-89: 64d, 69d
 Numlong-87: 68d, 68e, 69a, 69d
 Numuns-86: 68a, 68b, 68c, 68f, 69a
 Numvlong-88: 68d, 68e, 68f
 Numxxx: 64d
 NVAR: 351d, 351f, 355
 nvar: 351e
 OADD: 114c, 156b, 158a, 212f, 221c, 228b, 230b, 237e, 238d, 241b, 250b, 251, 294b, 295b, 297b, 305a, 312a, 313c, 314b, 328e, 329, 330, 332b, 421a, 428a, 431, 437, 438d, 450e, 459d, 475
 OADDR: 116e, 154e, 170a, 212d, 221a, 235b, 235c, 241b, 243b, 262d, 268b, 293d, 294a, 295b, 332b, 416b, 416f, 431, 438d, 447b, 459d, 475
 OAND: 115a, 159a, 228b, 231a, 295a, 295b, 306b, 312a, 312b, 313b, 313c, 421c, 422a, 437, 438d, 459d, 475, 515
 OANDAND: 115c, 165d, 252a, 253, 261a, 295b, 303a, 307c, 309, 459d, 475
 OARRAY: 119i, 135f, 262d, 475
 OAS: 116a, 166a, 167b, 190a, 228b, 234d, 235c, 236h, 239c, 240a, 240c, 243b, 248a, 250b, 251, 253, 257, 259a, 260e, 261a, 261b, 268b, 272e, 314b, 420a, 421a, 421b, 421c, 422a, 431, 438d, 459d, 475
 OASADD: 116c, 167c, 190a, 230b, 239c, 437, 438d, 459d, 475
 OASAND: 116c, 169b, 190a, 231a, 239c, 312c, 437, 438d, 459d, 475
 OASASHL: 116c, 169a, 190a, 231c, 239c, 311b, 437, 438d, 459d, 475
 OASASHR: 116c, 169a, 190a, 231d, 233c, 239c, 437, 438d, 459d, 475
 OASDIV: 116c, 168f, 190a, 231f, 233d, 239c, 437, 438d, 459d, 475
 OASHL: 115a, 159d, 228b, 231c, 295b, 296a, 306c, 311a, 314b, 421c, 422a, 437, 438d, 459d, 475, 515
 OASHR: 115a, 159d, 228b, 231d, 232c, 295b, 296a, 306c, 310c, 312a, 421c, 437, 438d, 459d, 475, 515
 OASI: 167b, 262b, 262d, 268b, 438d, 475
 OASLDIV: 168f, 190a, 233b, 233d, 234c, 239c, 311d, 437, 438d, 459d, 475
 OASLMOD: 169b, 190a, 233b, 233e, 234b, 239c, 312c, 437, 438d, 459d, 475
 OASLMUL: 168f, 190a, 233b, 233d, 234a, 239c, 311b, 437, 438d, 459d, 475
 OASLSHR: 169a, 190a, 233b, 233c, 233f, 239c, 311d, 437, 438d, 459d, 475
 OASMOD: 116c, 169b, 190a, 231g, 233e, 239c, 437, 438d, 459d, 475
 OASMUL: 116c, 168f, 190a, 231e, 233d, 239c, 311b, 437, 438d, 459d, 475
 OASOR: 116c, 169b, 190a, 230d, 239c, 437, 438d, 459d, 475
 OASSUB: 116c, 167c, 190a, 230c, 239c, 437, 438d, 459d, 475
 OASXOR: 116c, 169b, 190a, 231b, 239c, 437, 438d, 459d, 475
 OBIT: 331a, 332b, 418d, 418f, 419g, 419i, 420a, 420b, 420c, 420d, 420f, 420g, 442g, 475
 OBREAK: 108b, 185g, 186b, 204c, 475
 OCASE: 109b, 185g, 186b, 256c, 280c, 284, 475
 OCAST: 118a, 151a, 159d, 162c, 162d, 163c, 166a, 167b, 168a, 168b, 169a, 173a, 175d, 179a, 248a, 262d, 293c, 295b, 298b, 300f, 304d, 329, 330, 331a, 430c, 438d, 450e, 459d, 475
 OCOM: 115e, 165c, 306a, 437, 438d, 459d, 475
 OCOMMA: 110g, 155a, 185g, 198a, 206d, 224a, 236d, 243b, 253, 261a, 295b, 308b, 309, 459d, 475
 OCOND: 118f, 176a, 236f, 243b, 250a, 253, 261a, 295b, 303c, 309, 459d, 475
 OCONST: 111a, 134c, 155b, 156a, 158a, 158b, 162e, 163b, 163c, 165c, 176c, 178a, 181d, 186a, 189d, 190a, 195b, 205b, 205c, 211b, 220a, 221c, 239a, 240c, 241b, 242a, 243b, 253, 256c, 262d, 266b, 267b, 268b, 269, 273b, 280c, 292, 293a, 293c, 294c, 295b, 297b, 298b, 300f, 303b, 303c, 304a, 304c, 309, 310b, 313a, 313b, 313c, 314b, 328e, 329, 330, 332b, 336c, 418f, 428a, 431, 442f, 450e, 452e, 459d, 475, 487c, 490i, 491a, 491b, 491c, 491d, 515

OCONTINUE: [108b](#), [186b](#), [204a](#), [475](#)
 Oconv(): [474b](#), [474c](#)
 ODIV: [114c](#), [157a](#), [163c](#), [229a](#), [231f](#), [232e](#), [295b](#), [297a](#), [305a](#), [312a](#), [437](#), [438d](#), [450e](#), [459d](#), [475](#)
 ODOT: [117a](#), [170d](#), [242a](#), [243b](#), [261a](#), [295b](#), [428a](#), [475](#), [478a](#), [487c](#)
 ODOTDOT: [137c](#), [137e](#), [139a](#), [175a](#), [266b](#), [424b](#), [475](#)
 ODWHILE: [107c](#), [185g](#), [186b](#), [201f](#), [268b](#), [475](#)
 OELEM: [119h](#), [262d](#), [266b](#), [475](#), [478a](#), [487c](#)
 OEND: [27](#), [437](#), [474c](#), [474d](#), [475](#)
 OEQ: [115g](#), [164a](#), [232i](#), [233a](#), [243b](#), [252c](#), [253](#), [256a](#), [256b](#), [256c](#), [284](#), [295b](#), [298a](#), [300f](#), [307b](#), [313c](#), [437](#), [438d](#), [459d](#), [475](#)
 OEXREG: [433d](#), [475](#)
 ofnproto(): [423b](#), [423d](#), [423d](#)
 OFOR: [107c](#), [185g](#), [186b](#), [202](#), [475](#)
 OFUNC: [117c](#), [135i](#), [138a](#), [172b](#), [213b](#), [243b](#), [257](#), [258a](#), [260d](#), [261a](#), [309](#), [438d](#), [459d](#), [466b](#), [475](#)
 OGE: [115g](#), [164d](#), [233a](#), [252c](#), [253](#), [256a](#), [256b](#), [256c](#), [295b](#), [298a](#), [300f](#), [306d](#), [312a](#), [437](#), [438d](#), [459d](#), [475](#)
 OGOTO: [29e](#), [108e](#), [186b](#), [196a](#), [198c](#), [199f](#), [200a](#), [200c](#), [201f](#), [202](#), [204a](#), [204c](#), [236f](#), [243b](#), [250a](#), [253](#), [255](#), [279b](#), [281h](#), [284](#), [312a](#), [475](#)
 OGT: [115g](#), [164d](#), [233a](#), [252c](#), [253](#), [256a](#), [256b](#), [256c](#), [284](#), [295b](#), [298a](#), [300f](#), [306d](#), [437](#), [438d](#), [459d](#), [475](#)
 OHI: [232i](#), [233a](#), [252c](#), [253](#), [256a](#), [256b](#), [256c](#), [298a](#), [300d](#), [300f](#), [307a](#), [437](#), [438d](#), [459d](#), [475](#)
 OHS: [232i](#), [233a](#), [252c](#), [253](#), [256a](#), [256b](#), [256c](#), [298a](#), [300d](#), [300f](#), [307a](#), [437](#), [438d](#), [459d](#), [475](#)
 OIF: [107a](#), [186b](#), [198c](#), [475](#)
 OIND: [116e](#), [135c](#), [170b](#), [212e](#), [215c](#), [216a](#), [220d](#), [236c](#), [236h](#), [238d](#), [241b](#), [243b](#), [260e](#), [268b](#), [272c](#), [272d](#), [293d](#), [294a](#), [295b](#), [414a](#), [431](#), [459d](#), [475](#)
 OINDEX: [189c](#), [475](#)
 OINDREG: [209d](#), [212c](#), [215c](#), [216a](#), [237b](#), [237c](#), [237d](#), [239b](#), [259b](#), [269](#), [293b](#), [309](#), [328e](#), [475](#), [487c](#)
 OINIT: [119e](#), [120c](#), [262d](#), [266b](#), [475](#)
 OLABEL: [108e](#), [185g](#), [186b](#), [200a](#), [475](#)
 OLDIV: [157a](#), [228b](#), [232d](#), [232e](#), [234c](#), [295b](#), [297a](#), [305b](#), [311c](#), [437](#), [438d](#), [459d](#), [475](#)
 OLDPROTO-91: [423a](#), [423b](#), [424a](#), [424b](#)
 OLE: [115g](#), [164d](#), [233a](#), [252c](#), [253](#), [256a](#), [256b](#), [256c](#), [295b](#), [298a](#), [300f](#), [306d](#), [437](#), [438d](#), [459d](#), [475](#)
 OLIST: [29b](#), [106a](#), [137b](#), [138b](#), [173a](#), [184f](#), [185g](#), [186b](#), [198a](#), [243b](#), [259a](#), [266a](#), [267a](#), [267b](#), [268b](#), [310a](#), [416b](#), [423d](#), [424b](#), [430c](#), [431](#), [438d](#), [445b](#), [459d](#), [475](#), [487c](#), [491e](#)
 OLMOD: [157d](#), [228b](#), [232f](#), [232g](#), [234b](#), [295b](#), [305b](#), [312b](#), [437](#), [438d](#), [459d](#), [475](#)
 OLMUL: [157a](#), [228b](#), [232d](#), [232e](#), [234a](#), [295b](#), [296b](#), [305b](#), [311a](#), [314a](#), [437](#), [438d](#), [459d](#), [475](#)
 OLO: [232i](#), [233a](#), [252c](#), [253](#), [256a](#), [256b](#), [256c](#), [298a](#), [300d](#), [300f](#), [307a](#), [437](#), [438d](#), [459d](#), [475](#)
 OLS: [232i](#), [233a](#), [252c](#), [253](#), [256a](#), [256b](#), [256c](#), [298a](#), [300d](#), [300f](#), [307a](#), [437](#), [438d](#), [459d](#), [475](#)
 OLSHR: [159d](#), [228b](#), [232b](#), [232c](#), [233f](#), [295b](#), [296a](#), [306c](#), [311c](#), [421c](#), [437](#), [438d](#), [459d](#), [475](#), [515](#)
 OLSTRING: [113a](#), [178a](#), [227e](#), [262d](#), [266b](#), [267b](#), [269](#), [295b](#), [442f](#), [475](#), [487c](#)
 OLT: [115g](#), [164d](#), [233a](#), [252c](#), [253](#), [256a](#), [256b](#), [256c](#), [295b](#), [298a](#), [300f](#), [306d](#), [437](#), [438d](#), [459d](#), [475](#)
 OMOD: [114c](#), [157d](#), [229a](#), [231g](#), [295b](#), [305a](#), [437](#), [438d](#), [459d](#), [475](#)
 OMUL: [114c](#), [157a](#), [162e](#), [163b](#), [228b](#), [231e](#), [232e](#), [295b](#), [296b](#), [305a](#), [311a](#), [314a](#), [328e](#), [329](#), [330](#), [332b](#), [437](#), [438d](#), [450e](#), [459d](#), [475](#)
 ONAME: [27](#), [104d](#), [136e](#), [138a](#), [138b](#), [155d](#), [184f](#), [194e](#), [212a](#), [215c](#), [216a](#), [220c](#), [226a](#), [227e](#), [242e](#), [251](#), [260b](#), [262d](#), [268b](#), [269](#), [272c](#), [293a](#), [295b](#), [300f](#), [309](#), [328e](#), [337b](#), [416b](#), [423d](#), [424c](#), [434i](#), [438d](#), [447b](#), [459b](#), [475](#), [478a](#), [487c](#)
 onames: [474c](#), [474d](#), [486c](#)
 onamesinit: [475](#), [486c](#)
 ONE: [115g](#), [164a](#), [232i](#), [233a](#), [252c](#), [253](#), [256a](#), [256b](#), [256c](#), [295b](#), [298a](#), [300f](#), [307b](#), [313c](#), [437](#), [438d](#), [459d](#), [475](#)
 ONEG: [114e](#), [158b](#), [306a](#), [328e](#), [329](#), [330](#), [437](#), [438d](#), [459d](#), [475](#)
 ONOT: [115e](#), [165a](#), [252b](#), [253](#), [295b](#), [307c](#), [437](#), [438d](#), [459d](#), [475](#)

OOR: [115a](#), [159a](#), [228b](#), [230d](#), [294b](#), [295b](#), [306b](#), [313c](#), [422a](#), [437](#), [438d](#), [459d](#), [475](#)
 OOROR: [115c](#), [165d](#), [252a](#), [253](#), [261a](#), [295b](#), [303b](#), [307c](#), [309](#), [459d](#), [475](#)
 OPOS: [114e](#), [158a](#), [437](#), [438d](#), [475](#)
 OPOSTDEC: [118j](#), [177](#), [250b](#), [268b](#), [421a](#), [459d](#), [475](#)
 OPOSTINC: [118j](#), [177](#), [250b](#), [421a](#), [459d](#), [475](#)
 OPREDEC: [118h](#), [177](#), [251](#), [459d](#), [475](#)
 OPREINC: [118h](#), [177](#), [251](#), [459d](#), [475](#)
 OPROTO: [136f](#), [136h](#), [138b](#), [175c](#), [424b](#), [475](#)
 OREGISTER: [208f](#), [209b](#), [209d](#), [212b](#), [220b](#), [222d](#), [237b](#), [239b](#), [269](#), [281h](#), [293b](#), [309](#), [328e](#), [336b](#), [433c](#), [433e](#), [456](#),
[475](#), [487c](#)
 OREGPAIR: [281h](#), [475](#)
 ORETURN: [108b](#), [179a](#), [186b](#), [194c](#), [196a](#), [206b](#), [272e](#), [309](#), [459d](#), [475](#)
 os: [503](#)
 OSET: [184c](#), [184e](#), [184f](#), [186b](#), [475](#)
 OSIGN: [295b](#), [442a](#), [442f](#), [475](#)
 OSIZE: [119a](#), [178a](#), [295b](#), [475](#)
 OSTRING: [112c](#), [155c](#), [178a](#), [262d](#), [266b](#), [267b](#), [269](#), [295b](#), [309](#), [328e](#), [442f](#), [475](#), [487c](#)
 OSTRUCT: [429](#), [430b](#), [431](#), [475](#)
 OSUB: [114c](#), [156c](#), [158b](#), [163c](#), [228b](#), [230c](#), [243b](#), [250b](#), [251](#), [284](#), [295b](#), [297b](#), [305a](#), [312a](#), [314b](#), [328e](#), [329](#), [330](#),
[332b](#), [437](#), [438d](#), [450e](#), [459d](#), [475](#)
 OSWITCH: [109b](#), [186b](#), [279b](#), [475](#)
 OUNION: [429](#), [475](#)
 OUSED: [184c](#), [184e](#), [186b](#), [262d](#), [267b](#), [416b](#), [475](#)
 outcode(): [287c](#)
 outfile: [45c](#), [45c](#), [47h](#), [48a](#), [48e](#), [400a](#), [495b](#)
 outhist(): [290b](#)
 outlstring(): [227e](#), [228a](#)
 outstring(): [227a](#)
 OWHILE: [107c](#), [185g](#), [186b](#), [201f](#), [475](#)
 OXOR: [115a](#), [159a](#), [165c](#), [228b](#), [231b](#), [294b](#), [295b](#), [306b](#), [313c](#), [437](#), [438d](#), [459d](#), [475](#)
 OXXX: [27](#), [109a](#), [154e](#), [163c](#), [173a](#), [308b](#), [419i](#), [428a](#), [434i](#), [437](#), [438d](#), [466b](#), [474c](#), [475](#)
 P: [193g](#)
 p: [193d](#)
 packflg: [469b](#), [469c](#), [469d](#), [469e](#)
 paint1(): [362](#)
 paint2(): [363](#)
 paint3(): [364](#)
 paramconv(): [182a](#), [182b](#), [182c](#)
 params: [340b](#)
 patch(): [196b](#)
 pathchar(): [48d](#), [290b](#), [513b](#)
 pathname: [51a](#), [51b](#), [290b](#)
 pc: [193a](#)
 Pconv(): [480](#)
 pdecl(): [138b](#), [139c](#), [423c](#), [424c](#)
 peekc: [57c](#), [57e](#), [58a](#), [58c](#), [58d](#), [60g](#), [65b](#), [69d](#), [70](#), [73](#), [78a](#), [78b](#), [79a](#), [90b](#)
 peekinit(): [262d](#), [267a](#)
 peep(): [366c](#)
 picklechar: [408b](#), [408d](#), [409](#)

picklecinit: [408c](#), [408d](#)
 pickleinit(): [408d](#), [409](#)
 picklemember(): [409](#), [409](#), [410](#)
 picklestr-79: [407c](#), [407c](#), [409](#)
 picklesue(): [408a](#), [409](#), [410](#)
 pickletype(): [277d](#), [410](#)
 Plan9: [49b](#), [503](#), [513b](#)
 pmap(): [407d](#), [409](#), [410](#)
 postorder(): [391b](#)
 pragfpround(): [511b](#), [511c](#)
 pragincomplete(): [398a](#), [398b](#)
 pragpack(): [469a](#), [469c](#)
 pragprofile(): [412d](#), [412e](#)
 pragvararg(): [448a](#), [448b](#)
 predicable(): [388d](#), [390b](#)
 predicate(): [366c](#), [391a](#)
 predinfo: [387c](#), [390b](#)
 profileflg: [412a](#), [412c](#), [412e](#), [413](#)
 Prog: [531b](#), [531b](#)
 Prog (typedef): [531b](#)
 prop(): [357d](#)
 prtrees(): [143b](#), [300f](#), [308b](#), [438d](#), [487b](#), [490i](#), [491a](#), [491b](#), [491c](#), [491d](#), [491f](#), [491g](#), [492b](#), [492c](#), [494a](#), [494b](#),
[494c](#), [497b](#), [515](#)
 prtrees1(): [328e](#), [332b](#), [487b](#), [487c](#), [487c](#)
 push(): [39e](#), [39f](#), [40a](#), [109a](#), [130b](#)
 push1(): [39f](#), [134c](#), [138b](#), [140c](#), [424c](#)
 pushio(): [81f](#), [83b](#), [90a](#)
 Qconv(): [474b](#), [478b](#)
 qnames: [478b](#), [485c](#), [486c](#)
 qnamesinit: [485d](#), [486c](#)
 Qualifier: [33a](#)
 R: [352b](#)
 raddr(): [256c](#), [453b](#), [454a](#)
 rcmp(): [353a](#)
 Rconv(): [482](#)
 Reg: [531b](#), [531b](#)
 reg: [207b](#)
 Reg (typedef): [531b](#)
 rega(): [352c](#)
 regaalloc(): [259b](#)
 regaalloc1(): [338b](#)
 regalloc(): [221f](#)
 regbits: [340e](#)
 regfree(): [209c](#)
 regialloc(): [237a](#)
 regind(): [237b](#)
 region: [353e](#)
 reglcgen(): [238c](#)
 reglpcgen(): [243b](#), [247c](#)

regnode: [209a](#)
 regopt(): [339b](#)
 regret(): [207a](#)
 regsalloc(): [260c](#)
 regtyp(): [370d](#)
 relcon(): [298a](#), [298b](#)
 relindex(): [232h](#), [233a](#), [253](#), [300f](#)
 resvreg-193: [208b](#), [208c](#), [208e](#), [221f](#), [433b](#), [453a](#)
 reverttdcl(): [40d](#)
 Rgn: [531b](#), [531b](#)
 Rgn (typedef): [531b](#)
 rgp: [353f](#)
 round(): [275b](#), [276a](#), [277d](#), [502a](#)
 rpo2r: [359e](#)
 rpolca(): [358a](#)
 rsametype(): [145c](#), [145d](#), [146b](#), [147a](#)
 RtoB(): [365b](#)
 S: [32a](#), [35d](#), [35e](#), [36a](#), [77a](#), [77b](#), [85b](#), [86b](#), [86d](#), [87a](#), [94a](#), [95a](#), [95c](#), [129a](#), [172a](#), [220a](#), [220b](#), [237d](#), [270b](#), [287c](#),
[354b](#), [355](#), [401c](#), [402c](#), [402d](#), [403b](#), [405b](#), [408a](#), [409](#), [410](#), [423c](#), [427d](#), [427e](#), [428b](#), [434i](#), [438d](#), [448b](#), [479d](#), [481c](#),
[482](#), [484](#)
 samaddr(): [336b](#)
 sametype(): [145b](#), [145c](#), [151a](#), [162c](#), [162d](#), [166a](#), [167a](#), [167b](#), [168a](#), [172a](#), [173a](#), [174b](#), [174c](#), [176a](#), [179a](#), [262d](#),
[266b](#), [397c](#), [425b](#), [428b](#), [430c](#), [446](#), [447b](#), [449b](#)
 sconst(): [239a](#)
 Sconv(): [483](#)
 Setcond-199: [388a](#), [389c](#), [391a](#)
 setinclude(): [49b](#), [79e](#)
 setmalloctag(): [501b](#)
 shiftprop(): [366c](#), [375b](#)
 shmax-210: [316h](#), [320a](#), [321](#)
 side(): [295a](#), [295b](#), [295b](#), [296a](#), [296b](#), [297a](#)
 SIGDONE: [394a](#), [395a](#)
 sigind(): [396a](#), [397c](#)
 SIGINTERN: [394a](#), [395b](#), [459b](#)
 SIGN: [65d](#), [501e](#)
 sign(): [395a](#), [395b](#)
 signat(): [395c](#), [396a](#), [396a](#)
 signature: [394a](#), [395b](#), [395c](#), [442f](#)
 signature(): [394a](#), [395b](#), [395c](#), [442f](#)
 SIGNINTERN: [394a](#), [395b](#)
 SIGNONE: [37h](#), [394a](#)
 simplec(): [126c](#)
 simpleg(): [127c](#), [127e](#)
 simplet(): [124e](#)
 simplifyshift(): [515](#)
 slookup(): [35f](#), [60b](#), [77a](#), [87a](#), [226e](#), [242e](#), [260b](#), [272c](#), [434i](#), [459b](#)
 snap(): [147a](#), [147b](#), [154a](#), [163a](#), [177](#)
 Split-196: [388a](#), [389c](#)
 SR0-212: [316i](#), [321](#)

SR1-211: [316i](#), [321](#)
 stcompat(): [150a](#), [150c](#), [173a](#)
 stkoff: [194c](#), [196c](#), [197a](#), [197b](#), [225c](#), [260c](#), [268b](#)
 Storage_class: [33b](#)
 STORE: [353c](#)
 strf: [130f](#), [132c](#)
 string: [226j](#)
 STRINGSZ: [81e](#), [81f](#), [142](#), [476](#), [478b](#), [479a](#), [479d](#), [480](#), [481c](#), [482](#), [483](#), [484](#), [495c](#), [496b](#), [496c](#), [497f](#)
 strl: [130g](#), [132c](#)
 sualign(): [277d](#)
 subprop(): [371](#)
 successor(): [390a](#), [390b](#)
 sugen(): [243b](#)
 supgen(): [187c](#), [188b](#)
 suppress: [188a](#)
 sval(): [355](#), [357c](#)
 swcmp(): [281g](#), [281h](#)
 swit1(): [283b](#)
 swit2(): [283b](#), [284](#), [284](#)
 Sym: [504b](#), [504b](#)
 Sym (typedef): [504b](#)
 symadjust(): [268b](#), [269](#), [269](#)
 symb: [35f](#), [35g](#), [36a](#), [60d](#), [60e](#), [61a](#), [64e](#), [65b](#), [65c](#), [67](#), [69c](#), [69d](#), [74c](#), [74d](#), [77b](#), [81f](#), [83a](#), [84c](#), [88b](#), [96b](#), [393b](#),
[415f](#), [489f](#), [496a](#)
 syminit(): [36a](#), [37h](#)
 symrathole: [242c](#)
 symstring: [226c](#), [226e](#), [226i](#), [227a](#), [447b](#)
 synch(): [360](#)
 systemtype(): [49b](#), [513b](#)
 SZ_CHAR: [274a](#)
 SZ_DOUBLE: [274h](#)
 SZ_FLOAT: [274f](#)
 SZ_IND: [274e](#)
 SZ_INT: [274c](#)
 SZ_LONG: [274d](#)
 SZ_SHORT: [274b](#)
 SZ_VLONG: [274g](#)
 T: [31i](#), [32a](#), [32c](#), [37h](#), [104d](#), [120d](#), [127d](#), [129a](#), [130b](#), [132c](#), [134c](#), [136b](#), [136e](#), [136h](#), [137b](#), [137e](#), [145b](#), [145d](#), [146b](#),
[147a](#), [147b](#), [147c](#), [148b](#), [150b](#), [150c](#), [153b](#), [153d](#), [155e](#), [160c](#), [163c](#), [165a](#), [165d](#), [170b](#), [170d](#), [171b](#), [171c](#), [172a](#),
[172b](#), [173a](#), [174a](#), [174b](#), [175a](#), [175d](#), [176c](#), [178b](#), [180d](#), [181b](#), [182a](#), [182f](#), [190d](#), [199f](#), [206c](#), [223c](#), [229c](#), [236g](#),
[243b](#), [249b](#), [255](#), [262d](#), [270c](#), [272d](#), [277d](#), [279b](#), [280c](#), [299a](#), [300f](#), [304a](#), [395b](#), [398b](#), [401c](#), [402c](#), [403b](#), [405b](#),
[408a](#), [410](#), [416b](#), [423b](#), [423d](#), [424a](#), [425b](#), [425c](#), [427a](#), [427e](#), [428b](#), [430c](#), [431](#), [438d](#), [442h](#), [446](#), [449b](#), [476](#), [487c](#)
 tab: [160a](#), [161a](#)
 tadd: [151c](#), [152a](#), [156b](#), [177](#)
 taddinit: [151d](#), [152a](#)
 taggen: [130c](#)
 tand: [152a](#), [157d](#), [159a](#), [159b](#), [159d](#), [165c](#), [169a](#), [169b](#)
 tandinit: [152a](#), [159c](#)
 targ: [173a](#), [173b](#)

TARRAY: [31c](#), [121](#), [135f](#), [146c](#), [148b](#), [154c](#), [154e](#), [182f](#), [226e](#), [242e](#), [262d](#), [275b](#), [300f](#), [354b](#), [355](#), [361a](#), [396a](#), [397a](#),
[404d](#), [405b](#), [408c](#), [409](#), [415a](#), [476](#), [477b](#), [485d](#)
tasadd: [152a](#), [167c](#), [168d](#)
tasaddinit: [152a](#), [168e](#)
tasign: [152a](#), [166a](#), [166c](#), [167a](#), [167b](#), [173a](#), [179a](#), [430c](#)
tasigninit: [152a](#), [166d](#)
TAUTO: [121](#), [126a](#), [485d](#)
tcast: [152a](#), [175d](#), [175e](#)
tcastinit: [152a](#), [175f](#)
TCHAR: [30b](#), [32c](#), [59](#), [70](#), [121](#), [124e](#), [148d](#), [151d](#), [156e](#), [157c](#), [159c](#), [161a](#), [164c](#), [166d](#), [168e](#), [174d](#), [174e](#), [175f](#), [176c](#),
[180c](#), [182e](#), [215h](#), [216d](#), [217a](#), [217b](#), [217c](#), [218a](#), [218b](#), [218c](#), [221f](#), [226e](#), [227c](#), [228a](#), [242e](#), [243b](#), [248b](#), [262d](#),
[267b](#), [273b](#), [298b](#), [354b](#), [357b](#), [361a](#), [397a](#), [404d](#), [405b](#), [408c](#), [409](#), [436c](#), [445a](#), [453c](#), [457b](#), [458b](#), [459d](#), [477b](#),
[485d](#)
tcom(): [152b](#), [155a](#), [156b](#), [156c](#), [157a](#), [157d](#), [158a](#), [158b](#), [159a](#), [159d](#), [164a](#), [164d](#), [165a](#), [165c](#), [165d](#), [166a](#), [167b](#),
[167c](#), [168f](#), [169a](#), [169b](#), [170b](#), [170d](#), [173a](#), [175d](#), [176a](#), [177](#), [179a](#), [210a](#), [266b](#), [430c](#)
tcoma(): [172b](#), [173a](#), [173a](#), [438d](#)
tcomd(): [170d](#), [171b](#)
tcomo(): [152b](#), [153b](#), [170a](#), [172b](#), [175d](#), [178a](#), [438d](#), [442f](#)
tcompat(): [150a](#), [156b](#), [156c](#), [157a](#), [157d](#), [158a](#), [158b](#), [159a](#), [159d](#), [164a](#), [164d](#), [165a](#), [165c](#), [165d](#), [166a](#), [167b](#),
[167c](#), [168f](#), [169a](#), [169b](#), [170b](#), [170d](#), [172b](#), [173a](#), [175d](#), [176a](#), [177](#), [179a](#), [180d](#), [430c](#), [438d](#)
tcomx(): [430b](#), [430c](#)
TCONSTNT: [121](#), [127a](#), [485d](#)
Tconv(): [474b](#), [476](#)
tcopy(): [182f](#), [182f](#)
TDOT: [121](#), [137e](#), [137f](#), [174a](#), [397a](#), [425b](#), [477b](#), [485d](#)
tdot: [170d](#), [171a](#)
TDOUBLE: [31a](#), [32c](#), [59](#), [121](#), [124e](#), [149d](#), [151d](#), [156e](#), [157c](#), [161a](#), [164c](#), [166d](#), [168e](#), [174e](#), [175f](#), [176c](#), [182e](#), [217a](#),
[217b](#), [256c](#), [262d](#), [354b](#), [357b](#), [361a](#), [397a](#), [404d](#), [405b](#), [408c](#), [409](#), [436c](#), [450e](#), [451a](#), [451b](#), [451c](#), [451d](#), [452e](#),
[453c](#), [454a](#), [454b](#), [455a](#), [455b](#), [455c](#), [455d](#), [456](#), [457b](#), [459d](#), [477b](#), [485d](#)
TENUM: [31c](#), [32c](#), [121](#), [134c](#), [145b](#), [156a](#), [270c](#), [395a](#), [397a](#), [401c](#), [477b](#), [485d](#)
Term: [504b](#), [504b](#)
term: [328c](#), [328e](#), [330](#), [332b](#)
Term (typedef): [504b](#)
TEXTERN: [121](#), [126a](#), [485d](#)
tfield: [277d](#), [418f](#), [419a](#), [419b](#), [419d](#)
TFLOAT: [31a](#), [32c](#), [59](#), [121](#), [124e](#), [149d](#), [151d](#), [156e](#), [157c](#), [161a](#), [164c](#), [166d](#), [168e](#), [174e](#), [175f](#), [176c](#), [182e](#), [217a](#),
[217b](#), [256c](#), [262d](#), [354b](#), [357b](#), [361a](#), [397a](#), [404d](#), [405b](#), [408c](#), [409](#), [436c](#), [450d](#), [450e](#), [451a](#), [451b](#), [451c](#), [451d](#),
[453c](#), [454a](#), [454b](#), [455a](#), [455b](#), [455c](#), [455d](#), [456](#), [457b](#), [459d](#), [477b](#), [485d](#)
TFUNC: [31c](#), [32c](#), [121](#), [135i](#), [144d](#), [146b](#), [154c](#), [178b](#), [396a](#), [397a](#), [414a](#), [414b](#), [423b](#), [425b](#), [434i](#), [459b](#), [476](#), [477b](#),
[485d](#)
tfunct: [172b](#), [172c](#), [438d](#)
thash: [50b](#), [396a](#), [396e](#)
thash1: [396a](#), [396b](#), [396b](#)
thash2: [396a](#), [396c](#), [396c](#)
thash3: [396a](#), [396d](#), [396d](#)
thashinit: [50b](#), [397a](#)
thechar: [45a](#), [46c](#), [48e](#), [53g](#), [281h](#), [433d](#)
thestring: [45b](#), [46c](#), [49b](#)
thisfntype: [105b](#), [188c](#), [197a](#), [272d](#), [273a](#), [401c](#)

thunk: [499d](#), [499f](#)

TIND: [31c](#), [32c](#), [121](#), [135c](#), [147c](#), [151a](#), [151d](#), [154e](#), [156e](#), [161a](#), [161c](#), [162e](#), [163b](#), [163c](#), [164c](#), [166d](#), [168e](#), [170a](#), [175f](#), [176b](#), [176c](#), [177](#), [190d](#), [215f](#), [215h](#), [216d](#), [217c](#), [218a](#), [218b](#), [218c](#), [221f](#), [232a](#), [235c](#), [237a](#), [241b](#), [242e](#), [243b](#), [250b](#), [251](#), [262d](#), [268b](#), [272c](#), [272d](#), [275b](#), [281h](#), [329](#), [357b](#), [361a](#), [396a](#), [397a](#), [401c](#), [404d](#), [405a](#), [405b](#), [408c](#), [409](#), [414a](#), [414b](#), [415a](#), [427b](#), [428a](#), [431](#), [434i](#), [445a](#), [448b](#), [453c](#), [459d](#), [477b](#), [485d](#)

tindir: [170b](#), [170c](#)

tinit(): [50a](#)

TINT: [30b](#), [32c](#), [59](#), [65b](#), [68c](#), [121](#), [124e](#), [134c](#), [136e](#), [148d](#), [151d](#), [156e](#), [157c](#), [158a](#), [158b](#), [159c](#), [159d](#), [161a](#), [162a](#), [164a](#), [164c](#), [164d](#), [165a](#), [165c](#), [165d](#), [166d](#), [168e](#), [169a](#), [174d](#), [174e](#), [175f](#), [176c](#), [177](#), [178a](#), [180c](#), [182e](#), [215f](#), [215h](#), [216d](#), [217c](#), [218a](#), [218b](#), [218c](#), [221f](#), [260b](#), [262d](#), [311a](#), [311b](#), [311c](#), [311d](#), [357b](#), [361a](#), [397a](#), [405a](#), [405b](#), [408d](#), [409](#), [418f](#), [424c](#), [434i](#), [436c](#), [446](#), [453c](#), [457b](#), [458b](#), [459d](#), [477b](#), [485d](#), [518a](#)

TLONG: [30b](#), [32c](#), [59](#), [69a](#), [121](#), [124e](#), [148d](#), [151d](#), [156e](#), [157c](#), [159c](#), [161a](#), [163c](#), [164c](#), [166d](#), [168e](#), [175d](#), [175f](#), [176c](#), [180c](#), [195b](#), [209b](#), [215f](#), [215h](#), [216d](#), [217c](#), [218a](#), [218b](#), [218c](#), [221f](#), [229c](#), [241b](#), [243b](#), [247c](#), [262d](#), [268b](#), [279b](#), [281h](#), [355](#), [357b](#), [361a](#), [397a](#), [404d](#), [405a](#), [405b](#), [408c](#), [408d](#), [409](#), [419d](#), [436c](#), [453c](#), [457b](#), [458b](#), [459d](#), [466b](#), [477b](#), [485d](#), [518a](#)

tlvalue(): [154e](#), [166a](#), [166b](#), [167b](#), [167c](#), [168f](#), [169a](#), [169b](#), [170a](#), [177](#)

tmerge(): [148a](#), [148b](#), [423b](#)

tmpreg(): [243b](#), [247b](#)

tmul: [152a](#), [157a](#), [157b](#), [168f](#)

tmulinit: [152a](#), [157c](#)

Tname: [443c](#), [528b](#)

tname-192: [443k](#), [447b](#), [449a](#)

Tname.link: [443c](#)

Tname.name: [443c](#)

Tname.param: [443c](#)

Tname (typedef): [528b](#)

tnames: [476](#), [477a](#), [486c](#)

tnamesinit: [477b](#), [486c](#)

tnot: [165a](#), [165b](#), [165d](#), [180d](#)

TOLD: [174a](#), [397a](#), [423b](#), [424d](#), [425a](#), [425b](#), [425c](#), [477b](#), [485d](#)

Toolong-200: [388a](#), [389c](#)

topbit(): [501c](#), [515](#)

Tprot: [443b](#), [528b](#)

tprot-191: [443j](#), [446](#), [449b](#)

Tprot.flag: [443b](#)

Tprot.link: [443b](#)

Tprot.type: [443b](#)

Tprot (typedef): [528b](#)

TREGISTER: [121](#), [126a](#), [485d](#)

trel: [152a](#), [164a](#), [164b](#), [164d](#), [176a](#)

treinit: [152a](#), [164c](#)

Truecond-202: [388b](#), [390b](#), [391a](#)

TRUNE: [73](#), [113c](#), [227e](#), [267b](#)

TRune (typedef): [504b](#)

TSHORT: [30b](#), [32c](#), [59](#), [121](#), [124e](#), [148d](#), [151d](#), [156e](#), [157c](#), [159c](#), [161a](#), [164c](#), [166d](#), [168e](#), [174d](#), [174e](#), [175f](#), [176c](#), [180c](#), [182e](#), [215h](#), [216d](#), [217a](#), [217b](#), [217c](#), [218a](#), [218b](#), [218c](#), [221f](#), [248b](#), [262d](#), [298b](#), [354b](#), [357b](#), [361a](#), [397a](#), [404d](#), [405a](#), [405b](#), [408c](#), [408d](#), [409](#), [436c](#), [453c](#), [457b](#), [458b](#), [459d](#), [477b](#), [485d](#)

TSIGNED: [121](#), [123g](#), [485d](#)

TSTATIC: [121](#), [126a](#), [485d](#)

TSTRUCT: [31c](#), [121](#), [148b](#), [149b](#), [166d](#), [175f](#), [262d](#), [271d](#), [277d](#), [396a](#), [397a](#), [398b](#), [403b](#), [405b](#), [409](#), [410](#), [430c](#), [477b](#), [485d](#)
tsub: [152a](#), [156c](#), [156d](#), [158a](#), [158b](#)
tsubinit: [152a](#), [156e](#)
TTYPEDEF: [121](#), [126a](#), [485d](#)
TTYPESTR: [434d](#), [434e](#), [485d](#)
TUCHAR: [30b](#), [32c](#), [70](#), [121](#), [124e](#), [148d](#), [151d](#), [156e](#), [157c](#), [159c](#), [161a](#), [164c](#), [166d](#), [168e](#), [174d](#), [174e](#), [175f](#), [176c](#), [180c](#), [182e](#), [215h](#), [216d](#), [217a](#), [217b](#), [217c](#), [218a](#), [218b](#), [218c](#), [221f](#), [232a](#), [248b](#), [262d](#), [298b](#), [354b](#), [357b](#), [361a](#), [397a](#), [404d](#), [405b](#), [408c](#), [409](#), [436c](#), [453c](#), [457b](#), [458b](#), [459d](#), [477b](#), [485d](#)
tufield: [419a](#), [419c](#)
TUINT: [30b](#), [32c](#), [68c](#), [113c](#), [121](#), [124e](#), [148d](#), [151d](#), [156e](#), [157c](#), [159c](#), [161a](#), [164c](#), [166d](#), [168e](#), [174d](#), [174e](#), [175f](#), [176c](#), [180c](#), [182e](#), [215f](#), [215h](#), [216d](#), [217c](#), [218a](#), [218b](#), [218c](#), [221f](#), [232a](#), [262d](#), [357b](#), [361a](#), [397a](#), [405a](#), [405b](#), [408d](#), [409](#), [436c](#), [446](#), [453c](#), [457b](#), [458b](#), [459d](#), [477b](#), [485d](#), [518a](#)
TULONG: [30b](#), [32c](#), [69a](#), [121](#), [124e](#), [148d](#), [151d](#), [156e](#), [157c](#), [159c](#), [161a](#), [164c](#), [166d](#), [168e](#), [175f](#), [176c](#), [180c](#), [215f](#), [215h](#), [216d](#), [217c](#), [218a](#), [218b](#), [218c](#), [221f](#), [232a](#), [262d](#), [357b](#), [361a](#), [397a](#), [404d](#), [405a](#), [405b](#), [408c](#), [408d](#), [409](#), [436c](#), [442f](#), [453c](#), [457b](#), [458b](#), [459d](#), [477b](#), [485d](#), [518a](#)
TUNION: [31c](#), [121](#), [148b](#), [149b](#), [166d](#), [175f](#), [262d](#), [271d](#), [277d](#), [396a](#), [397a](#), [398b](#), [403b](#), [405b](#), [409](#), [410](#), [477b](#), [485d](#)
TUNSIGNED: [121](#), [123g](#), [485d](#)
TUSHORT: [30b](#), [32c](#), [113c](#), [121](#), [124e](#), [148d](#), [151d](#), [156e](#), [157c](#), [159c](#), [161a](#), [164c](#), [166d](#), [168e](#), [174d](#), [174e](#), [175f](#), [176c](#), [180c](#), [182e](#), [215h](#), [216d](#), [217a](#), [217b](#), [217c](#), [218a](#), [218b](#), [218c](#), [221f](#), [232a](#), [248b](#), [262d](#), [298b](#), [354b](#), [357b](#), [361a](#), [397a](#), [404d](#), [405a](#), [405b](#), [408c](#), [408d](#), [409](#), [436c](#), [453c](#), [457b](#), [458b](#), [459d](#), [477b](#), [485d](#)
TUVLONG: [30b](#), [32c](#), [68f](#), [121](#), [124e](#), [148d](#), [151d](#), [156e](#), [157c](#), [159c](#), [161a](#), [164c](#), [166d](#), [168e](#), [175f](#), [176c](#), [232a](#), [262d](#), [271d](#), [280b](#), [300f](#), [397a](#), [404d](#), [405a](#), [405b](#), [408c](#), [409](#), [436c](#), [458b](#), [477b](#), [485d](#)
TVLONG: [30b](#), [32c](#), [68f](#), [121](#), [124e](#), [148d](#), [151d](#), [156e](#), [157c](#), [159c](#), [161a](#), [163c](#), [164c](#), [166d](#), [168e](#), [175f](#), [176c](#), [217b](#), [256c](#), [262d](#), [268b](#), [271d](#), [279b](#), [280b](#), [329](#), [361a](#), [397a](#), [404d](#), [405b](#), [408c](#), [409](#), [436c](#), [451a](#), [451b](#), [451c](#), [451d](#), [453c](#), [456](#), [457b](#), [458b](#), [477b](#), [485d](#)
TVOID: [31b](#), [32c](#), [59](#), [104d](#), [121](#), [124e](#), [138b](#), [147c](#), [173a](#), [174b](#), [174c](#), [175f](#), [179a](#), [188c](#), [242e](#), [243b](#), [262d](#), [293d](#), [304d](#), [397a](#), [416b](#), [446](#), [477b](#), [485d](#)
TVOLATILE: [121](#), [127a](#), [485d](#)
TXXX: [30a](#), [32c](#), [127d](#), [136h](#), [160c](#), [300f](#), [397a](#), [434i](#), [436c](#), [477b](#), [485d](#)
typ(): [32a](#), [32c](#), [127d](#), [129a](#), [135c](#), [135f](#), [135i](#), [136e](#), [136h](#), [137e](#), [154e](#), [170a](#), [226e](#), [241b](#), [242e](#), [243b](#), [268b](#), [272d](#), [414b](#), [415a](#), [423b](#), [431](#), [434i](#), [445a](#), [448b](#), [459b](#), [459d](#)
Type: [504b](#), [504b](#)
Type (typedef): [504b](#)
typeaf: [149e](#), [154b](#), [154d](#)
typeafinit: [149e](#), [154c](#)
typebitor(): [123c](#)
typechl: [149e](#), [180a](#), [180b](#)
typechlinit: [149e](#), [180c](#)
typechlp: [149e](#), [215d](#), [215g](#), [248b](#), [299a](#), [337d](#), [338a](#), [433b](#), [515](#)
typechlpfd: [149e](#), [355](#), [357a](#)
typechlpfdinit: [149e](#), [357b](#)
typechlpinit: [149e](#), [215h](#)
typechlv: [149e](#), [517a](#)
typecplx: [271a](#), [271b](#), [272d](#), [272e](#), [273a](#)
typeext(): [426a](#), [426b](#), [426c](#), [426d](#), [426e](#), [426f](#), [430c](#)
typeext1(): [450a](#), [450b](#), [450c](#), [450d](#)
typefd: [149c](#), [149e](#), [156a](#), [168c](#), [180g](#), [181a](#), [187c](#), [239a](#), [240c](#), [250b](#), [251](#), [253](#), [256c](#), [293c](#), [297b](#), [299a](#), [304a](#), [304d](#), [306d](#), [307b](#), [307c](#), [310b](#), [313a](#), [313b](#), [314b](#), [329](#), [330](#), [341d](#), [401c](#), [433b](#), [452a](#), [452c](#), [459d](#), [487c](#)

typefdinit: [149d](#), [149e](#)
 typei: [148c](#), [149e](#), [262d](#), [418f](#)
 typeiinit: [148d](#), [149e](#)
 typeil: [517b](#), [518b](#)
 typeilinit: [518a](#), [518b](#)
 typeilp: [149e](#), [215d](#), [215e](#)
 typeilpinit: [149e](#), [215f](#)
 types: [32b](#), [32c](#), [60b](#), [65c](#), [124e](#), [134c](#), [138b](#), [154e](#), [158a](#), [158b](#), [159d](#), [161b](#), [163c](#), [164a](#), [164d](#), [165a](#), [165c](#), [165d](#),
[169a](#), [170a](#), [173a](#), [174b](#), [174c](#), [174d](#), [174e](#), [175d](#), [177](#), [178a](#), [182e](#), [195b](#), [209b](#), [226e](#), [227c](#), [227e](#), [228a](#), [235c](#),
[237a](#), [241b](#), [242e](#), [243b](#), [247c](#), [260b](#), [268b](#), [270c](#), [272c](#), [273b](#), [275b](#), [279b](#), [281h](#), [311a](#), [311b](#), [311c](#), [311d](#), [395a](#),
[405a](#), [408d](#), [414b](#), [415a](#), [418f](#), [419d](#), [424c](#), [434i](#), [442f](#), [445a](#), [446](#), [450e](#), [452e](#), [459b](#), [459d](#), [466b](#)
 typesu: [147a](#), [147b](#), [149a](#), [149e](#), [154a](#), [172a](#), [182d](#), [182f](#), [398b](#), [401c](#), [405b](#), [427d](#), [427e](#), [428a](#), [428b](#), [476](#)
 typesuinit: [149b](#), [149e](#)
 typesuv: [149e](#), [271b](#), [271c](#), [271e](#), [272a](#), [275b](#), [434h](#)
 typesuvinit: [149e](#), [271d](#)
 typeswitch: [179c](#), [179d](#), [180a](#), [280c](#)
 Typetab: [397b](#), [525](#)
 Typetab.a: [397b](#)
 Typetab.n: [397b](#)
 Typetab (typedef): [525](#)
 typeu: [65d](#), [149e](#), [231h](#), [232c](#), [232e](#), [232g](#), [232h](#), [233c](#), [233d](#), [233e](#), [293c](#), [419a](#), [421c](#)
 typeuinit: [149e](#), [232a](#)
 typev: [149e](#), [243b](#), [273b](#), [279b](#), [280a](#), [280c](#), [281h](#), [329](#), [459d](#), [466b](#)
 typevinit: [149e](#), [280b](#)
 typeword: [337b](#), [337c](#), [337d](#)
 Type.kind: [30a](#)
 type_kind_bis: [120e](#)
 uncomma(): [206b](#), [206d](#)
 unget(): [71b](#), [72b](#), [77b](#), [78b](#), [81f](#), [86d](#), [87a](#), [91b](#), [93a](#), [96b](#), [97](#), [393b](#), [448b](#)
 uniqp(): [370b](#)
 uniqs(): [370c](#)
 Unix: [503](#)
 UR0-214: [316i](#), [320b](#), [321](#)
 UR1-213: [316i](#), [320b](#), [321](#)
 urk(): [50b](#), [50c](#), [149e](#), [152a](#), [486c](#), [518b](#)
 usedset(): [184e](#), [184f](#), [184f](#)
 valmax-209: [316g](#), [320a](#), [321](#)
 Var: [531b](#), [531b](#)
 var: [351f](#)
 Var (typedef): [531b](#)
 VARMAC-76: [85b](#), [86a](#), [90c](#)
 VBconv(): [474b](#), [479a](#)
 vconst(): [176b](#), [176c](#), [205b](#), [205c](#), [253](#), [262d](#), [294b](#), [295a](#), [296a](#), [296b](#), [297a](#), [303a](#), [303c](#), [305a](#), [305b](#), [427b](#)
 vlog(): [310b](#), [310c](#), [311a](#), [311b](#), [311c](#), [311d](#), [312a](#), [312b](#), [312c](#)
 walkparam(): [138a](#), [138b](#), [138b](#)
 warn(): [65c](#), [70](#), [123c](#), [142](#), [144a](#), [145a](#), [154d](#), [179a](#), [183e](#), [185c](#), [185g](#), [188c](#), [189b](#), [189d](#), [190a](#), [190d](#), [203a](#),
[203b](#), [203d](#), [242a](#), [243b](#), [262d](#), [281h](#), [300f](#), [305a](#), [305b](#), [346](#), [355](#), [405a](#), [408d](#), [446](#), [447b](#), [449b](#)
 warnreach: [185e](#), [185f](#), [185g](#), [187c](#), [188b](#), [199d](#), [200d](#), [203a](#), [203b](#), [203d](#), [203e](#), [205f](#), [279b](#)
 xcom(): [210b](#)

xdecl(): [104e](#), [134c](#), [434i](#)
xtramodes(): [366c](#), [380b](#)
yyerror(): [53f](#), [53g](#), [58e](#), [65b](#), [67](#), [69d](#), [70](#), [71a](#), [73](#), [76c](#), [77b](#), [78d](#), [81f](#), [83b](#), [84b](#), [85b](#), [86b](#), [86d](#), [87a](#), [87c](#), [88d](#), [90c](#), [91b](#), [94a](#), [94b](#), [96b](#), [393b](#), [398b](#), [448b](#), [449a](#), [495c](#), [496a](#), [499f](#)
yylex(): [57a](#)
yylval: [60g](#), [65b](#), [65c](#), [69d](#), [70](#), [73](#), [74c](#), [74d](#)
YYMAXDEPTH: [504a](#)
yyparse(): [47h](#)
Z: [29a](#), [40d](#), [40e](#), [41c](#), [48a](#), [65c](#), [70](#), [72a](#), [103a](#), [106a](#), [109a](#), [120d](#), [123c](#), [124e](#), [126c](#), [129b](#), [132c](#), [136b](#), [138a](#), [138b](#), [139d](#), [142](#), [143b](#), [143e](#), [144a](#), [144c](#), [144d](#), [145a](#), [145b](#), [153c](#), [154e](#), [158a](#), [158b](#), [159d](#), [160b](#), [162c](#), [162d](#), [162e](#), [163b](#), [163c](#), [165c](#), [166a](#), [167b](#), [168a](#), [169a](#), [173a](#), [174b](#), [175c](#), [176c](#), [178a](#), [179a](#), [181d](#), [182d](#), [183e](#), [184f](#), [185b](#), [186a](#), [186b](#), [188b](#), [188c](#), [189a](#), [189b](#), [189c](#), [194e](#), [196a](#), [197e](#), [198c](#), [199e](#), [201a](#), [201f](#), [202](#), [206b](#), [206d](#), [208e](#), [210a](#), [210b](#), [213a](#), [215a](#), [216a](#), [219a](#), [219b](#), [222d](#), [223a](#), [223c](#), [224a](#), [228b](#), [229b](#), [229c](#), [230a](#), [233a](#), [234e](#), [235a](#), [235c](#), [236b](#), [236g](#), [236h](#), [238a](#), [238b](#), [239c](#), [240b](#), [240c](#), [241b](#), [242a](#), [242e](#), [243b](#), [247b](#), [247c](#), [248a](#), [250b](#), [251](#), [252a](#), [253](#), [255](#), [256c](#), [257](#), [259a](#), [260b](#), [260d](#), [260e](#), [261a](#), [261b](#), [262c](#), [262d](#), [266a](#), [266b](#), [267a](#), [267b](#), [268b](#), [272c](#), [272d](#), [272e](#), [275b](#), [277d](#), [279b](#), [280c](#), [283b](#), [284](#), [288b](#), [292](#), [295b](#), [304a](#), [308b](#), [309](#), [310a](#), [312a](#), [313a](#), [313b](#), [314b](#), [328e](#), [329](#), [331a](#), [332b](#), [341e](#), [341f](#), [344a](#), [346](#), [350e](#), [355](#), [358a](#), [359g](#), [361a](#), [405a](#), [408d](#), [414a](#), [416b](#), [416f](#), [418f](#), [419h](#), [419i](#), [420a](#), [420e](#), [421a](#), [421b](#), [421c](#), [422a](#), [423d](#), [424b](#), [425b](#), [427d](#), [428a](#), [430c](#), [431](#), [434h](#), [434i](#), [438d](#), [442f](#), [445b](#), [446](#), [447b](#), [449b](#), [450e](#), [453a](#), [454a](#), [455a](#), [455b](#), [455c](#), [455d](#), [456](#), [459b](#), [459d](#), [466b](#), [467c](#), [478a](#), [487c](#), [496b](#), [496c](#), [497b](#), [501d](#), [502a](#)
zaddr(): [288b](#)
zbits: [345a](#), [345b](#), [346](#), [350a](#), [351a](#), [355](#), [444b](#)
zname(): [290a](#)
znode: [29i](#)
zprog: [44d](#)
zreg: [340j](#)
zwrite(): [287c](#), [288a](#), [290b](#)
__anon_enum_12: [388a](#)
__anon_enum_13: [388b](#)
__anon_enum_14: [316i](#)
__anon_enum_4: [153a](#)
__anon_struct_11.notopcode: [387c](#)
__anon_struct_11.notscond: [387c](#)
__anon_struct_11.opcode: [387c](#)
__anon_struct_11.scond: [387c](#)
__anon_struct_11: [387c](#), [387c](#)
__anon_struct_2.macf: [76b](#)
__anon_struct_2.macname: [76b](#)
__anon_struct_2: [76b](#), [76b](#)
__anon_struct_3.lexical: [59](#)
__anon_struct_3.name: [59](#)
__anon_struct_3.type: [59](#)
__anon_struct_3: [59](#), [59](#)

Bibliography

- [ALSU06] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, 2nd edition, 2006. cited page(s) 15, 16
- [App04] Andrew Appel. *Modern Compiler in ML*. Cambridge University Press, 2004. cited page(s) 15
- [CT11] Keith D. Cooper and Linda Torczon. *Engineering a Compiler*. Morgan Kaufmann, 2nd edition, 2011. cited page(s) 15
- [Joh79] Stephen C. Johnson. Yacc: Yet another compiler-compiler. In *Unix Programmer's Manual Vol 2b*, 1979. Also available at [generators/docs/yacc.pdf](https://github.com/aryx/generators/docs/yacc.pdf). cited page(s) 13
- [Knu92] Donald E. Knuth. *Literate Programming*. Center for the Study of Language and Information, 1992. cited page(s) 15
- [Pad09] Yoann Padioleau. Syncweb, literate programming meets unison, 2009. <https://github.com/aryx/syncweb>. cited page(s) 15
- [Pad14] Yoann Padioleau. *Principia Softwarica: The Plan 9 Kernel 9*. 2014. cited page(s) 12, 472
- [Pad15a] Yoann Padioleau. *Principia Softwarica: The ARM Assembler 5a*. 2015. cited page(s) 15, 16, 22, 23, 42, 219, 286, 471
- [Pad15b] Yoann Padioleau. *Principia Softwarica: The ARM Emulator 5i*. 2015. cited page(s) 15, 23
- [Pad15c] Yoann Padioleau. *Principia Softwarica: The ARM Linker 5l*. 2015. cited page(s) 23, 42, 100, 279, 286, 411, 472
- [Pad16a] Yoann Padioleau. *Principia Softwarica: The Build System mk*. 2016. cited page(s) 12
- [Pad16b] Yoann Padioleau. *Principia Softwarica: The Plan 9 Core Libraries*. 2016. cited page(s) 19
- [Pad16c] Yoann Padioleau. *Principia Softwarica: The Plan 9 Debuggers and Tracers*. 2016. cited page(s) 472
- [Pad18] Yoann Padioleau. *Principia Softwarica: The Plan 9 Shell rc*. 2018. cited page(s) 12, 17, 472
- [Pad26] Yoann Padioleau. *Principia Softwarica: The The Plan 9 Profilers*. 2026. cited page(s) 411, 412, 472